

Manual de MikroTik

RouterOS do básico ao avançado, com foco na operação de provedor de internet (fibra e rádio)

Provedor de Fibra Óptica

2026-07-07

Índice

Apresentação	21
Para quem é este manual	21
Como este manual funciona	21
Mapa de pré-requisitos	21
As três fases	23
Convenções usadas no livro	23
I Volume 1 — Primeiros passos	24
1 O que é MikroTik e RouterOS	25
1.1 A empresa e o sistema	25
1.1.1 Um pouco de história	26
1.1.2 RouterOS ou SwOS?	26
1.2 Por que os provedores escolhem MikroTik?	26
1.3 Onde o MikroTik se encaixa na rede de um provedor	27
1.4 O CHR: seu laboratório gratuito	28
1.5 Laboratório	28
1.5.1 Ambiente	28
1.5.2 Comandos passo a passo	29
1.5.3 Verificação	30
1.5.4 Desafios	30
1.6 Resumo	31
Bibliografia do capítulo	32
2 Linhas de hardware MikroTik	33
2.1 O catálogo visto de longe	33
2.2 Decifrando o nome dos produtos	33
2.3 As famílias, papel por papel	34
2.3.1 CCR — Cloud Core Router: o núcleo	35
2.3.2 CRS — Cloud Router Switch: o switch do POP	35
2.3.3 hEX, hAP e L009/RB5009 — roteadores de mesa	35
2.3.4 SXT, LHG, DISC, Wire — a família de rádio	36
2.3.5 CHR — o “hardware” que é software	36
2.4 As especificações que realmente importam	36
2.5 Laboratório	37
2.5.1 Ambiente	37
2.5.2 Comandos passo a passo	37

2.5.3	Verificação	38
2.5.4	Desafios	38
2.6	Resumo	39
	Bibliografia do capítulo	39
3	Licenças RouterOS e RouterBOOT	40
3.1	Licença: o que você compra junto com a caixa	40
3.1.1	A licença do CHR: outro modelo	42
3.2	RouterBOOT: o que acorda o RouterOS	42
3.2.1	As proteções do RouterBOOT	43
3.3	Laboratório	44
3.3.1	Ambiente	44
3.3.2	Comandos passo a passo	44
3.3.3	Verificação	45
3.3.4	Desafios	45
3.4	Resumo	46
	Bibliografia do capítulo	46
4	Formas de acesso ao RouterOS	47
4.1	Muitas portas para a mesma casa	47
4.1.1	WinBox — a ferramenta símbolo	47
4.1.2	WebFig — o WinBox no navegador	48
4.1.3	SSH — a porta da linha de comando	49
4.1.4	Console serial — o cabo da UTI	49
4.1.5	MAC-Telnet — terminal sem IP	49
4.2	Safe Mode: o cinto de segurança	50
4.3	Os serviços em <code>/ip service</code>	50
4.4	Laboratório	51
4.4.1	Ambiente	51
4.4.2	Comandos passo a passo	51
4.4.3	Verificação	52
4.4.4	Desafios	52
4.5	Resumo	53
	Bibliografia do capítulo	53
5	Montando o laboratório com CHR	55
5.1	Um laboratório para o livro inteiro	55
5.1.1	Os três tipos de rede do VirtualBox	56
5.2	GNS3 e EVE-NG: quando crescer	57
5.3	Laboratório	57
5.3.1	Ambiente	57
5.3.2	Comandos passo a passo	57
5.3.3	Verificação	59
5.3.4	Desafios	59
5.4	Resumo	60
	Bibliografia do capítulo	60

6	Primeiro acesso, usuários e segurança inicial	62
6.1	O que vem de fábrica	62
6.2	Usuários e grupos: aposentando o <code>admin</code> compartilhado	63
6.3	Fechando as portas: serviços e acesso	64
6.4	Relógio certo: segurança, não estética	64
6.5	Laboratório	65
6.5.1	Ambiente	65
6.5.2	Comandos passo a passo	65
6.5.3	Verificação	67
6.5.4	Desafios	67
6.6	Resumo	68
	Bibliografia do capítulo	68
7	Backup, export e reset	69
7.1	Duas fotografias diferentes da mesma configuração	69
7.2	Backup binário: gerar, baixar, restaurar	70
7.3	Export: a configuração como texto	71
7.4	Reset: recomeçar sem sustos	71
7.5	Laboratório	72
7.5.1	Ambiente	72
7.5.2	Comandos passo a passo	72
7.5.3	Verificação	74
7.5.4	Desafios	74
7.6	Resumo	75
	Bibliografia do capítulo	76
8	Netinstall	77
8.1	Quando o reset não basta	77
8.2	Como funciona por baixo	78
8.3	Laboratório	79
8.3.1	Ambiente	79
8.3.2	Comandos passo a passo	80
8.3.3	Verificação	80
8.3.4	Desafios	80
8.4	Resumo	81
	Bibliografia do capítulo	82
II	Volume 2 — CLI e configuração essencial	83
9	Anatomia do terminal	84
9.1	A árvore e os verbos	84
9.2	Itens, números e o perigo escondido	85
9.3	TAB e ?: o manual que mora no equipamento	86
9.4	<code>print</code> afiado e <code>export</code> como lupa	87

9.5	Laboratório	88
9.5.1	Ambiente	88
9.5.2	Comandos passo a passo	88
9.5.3	Verificação	89
9.5.4	Desafios	90
9.6	Resumo	90
	Bibliografia do capítulo	91
10	Endereçamento IP e rota padrão	92
10.1	O endereço e a máscara, sem mistério	92
10.2	/ip address: o que você dá e o que o RouterOS deduz	93
10.3	Rotas estáticas: ensinando o caminho	94
10.3.1	distance: o desempate entre rotas	94
10.3.2	A rota padrão e o check-gateway	95
10.4	Laboratório	95
10.4.1	Ambiente	95
10.4.2	Comandos passo a passo	96
10.4.3	Verificação	98
10.4.4	Desafios	98
10.5	Resumo	99
	Bibliografia do capítulo	99
11	DHCP client e server	100
11.1	O empréstimo de endereços	100
11.2	DHCP client: a WAN que se configura sozinha	101
11.3	DHCP server: a LAN que se serve sozinha	102
11.3.1	Leases: o livro de registros	102
11.4	Laboratório	103
11.4.1	Ambiente	103
11.4.2	Comandos passo a passo	103
11.4.3	Verificação	105
11.4.4	Desafios	105
11.5	Resumo	106
	Bibliografia do capítulo	106
12	DNS no RouterOS	107
12.1	O catálogo telefônico da internet	107
12.2	Configurando o resolvidor	108
12.2.1	allow-remote-requests: servidor, e para quem?	108
12.3	Entradas estáticas: o DNS interno do provedor	109
12.4	Laboratório	109
12.4.1	Ambiente	109
12.4.2	Comandos passo a passo	110
12.4.3	Verificação	111
12.4.4	Desafios	111
12.5	Resumo	112

Bibliografia do capítulo	113
13 NAT masquerade	114
13.1 O problema: cartas sem remetente válido	114
13.2 Como a máscara funciona	114
13.3 A regra no RouterOS	115
13.3.1 masquerade ou src-nat?	116
13.3.2 O que o NAT não é	116
13.4 Laboratório	117
13.4.1 Ambiente	117
13.4.2 Comandos passo a passo	117
13.4.3 Verificação	118
13.4.4 Desafios	118
13.5 Resumo	119
Bibliografia do capítulo	120
14 Projeto: roteador de borda	121
14.1 A ordem de serviço	121
14.2 Laboratório	122
14.2.1 Ambiente	122
14.2.2 Comandos passo a passo	122
14.2.3 Verificação	125
14.2.4 Desafios	125
14.3 Resumo	126
Bibliografia do capítulo	127
III Volume 3 — Firewall	128
15 Connection tracking	129
15.1 A memória que muda tudo	129
15.2 Os quatro estados	130
15.3 Lendo a tabela	131
15.4 O custo da memória	131
15.5 Laboratório	132
15.5.1 Ambiente	132
15.5.2 Comandos passo a passo	132
15.5.3 Verificação	133
15.5.4 Desafios	134
15.6 Resumo	135
Bibliografia do capítulo	135
16 Chains e a lógica do firewall	136
16.1 Três tribunais, uma pergunta	136
16.2 A lógica: de cima para baixo, primeira que casa decide	137
16.2.1 As actions	137

16.2.2	Chains próprias: organização por <code>jump</code>	138
16.3	O esqueleto que você vai repetir para sempre	138
16.4	Laboratório	139
16.4.1	Ambiente	139
16.4.2	Comandos passo a passo	139
16.4.3	Verificação	141
16.4.4	Desafios	141
16.5	Resumo	142
	Bibliografia do capítulo	142
17	Protegendo o roteador	143
17.1	Default deny: a inversão que muda tudo	143
17.2	Interface lists: regras que sobrevivem à topologia	143
17.3	O inventário: o que o input precisa aceitar	144
17.3.1	A política de ICMP	145
17.4	A sequência completa	145
17.5	Laboratório	146
17.5.1	Ambiente	146
17.5.2	Comandos passo a passo	146
17.5.3	Verificação	147
17.5.4	Desafios	148
17.6	Resumo	149
	Bibliografia do capítulo	149
18	Forward e address-lists	150
18.1	O forward de um provedor não é o de uma empresa	150
18.2	Address lists: separar política de dados	151
18.3	Listas dinâmicas: o firewall que aprende	151
18.4	Laboratório	152
18.4.1	Ambiente	152
18.4.2	Comandos passo a passo	152
18.4.3	Verificação	154
18.4.4	Desafios	154
18.5	Resumo	155
	Bibliografia do capítulo	156
19	dstnat e port forwarding	157
19.1	O espelho do masquerade	157
19.2	Firewall e dstnat: aceitando com precisão	158
19.3	Hairpin NAT: o defeito clássico	158
19.4	<code>redirect</code> : dstnat para o próprio roteador	159
19.5	Laboratório	160
19.5.1	Ambiente	160
19.5.2	Comandos passo a passo	160
19.5.3	Verificação	161
19.5.4	Desafios	161

19.6	Resumo	162
	Bibliografia do capítulo	163
20	Mangle e RAW	164
20.1	A linha de montagem completa	164
20.2	Mangle: o almoxarifado de etiquetas	165
20.3	RAW: agir antes de existir	165
20.4	Laboratório	166
20.4.1	Ambiente	166
20.4.2	Comandos passo a passo	167
20.4.3	Verificação	168
20.4.4	Desafios	168
20.5	Resumo	169
	Bibliografia do capítulo	169
21	Hardening completo	171
21.1	Defesa em camadas: o mapa do volume	171
21.2	As peças finais	172
21.3	O checklist consolidado	173
21.4	Laboratório	174
21.4.1	Ambiente	174
21.4.2	Comandos passo a passo	174
21.4.3	Verificação	176
21.4.4	Desafios	176
21.5	Resumo	177
	Bibliografia do capítulo	177
IV	Volume 4 — Bridge, switching e VLANs	179
22	Bridge e hardware offload	180
22.1	Camada 2: o andar de baixo	180
22.2	Bridge no RouterOS	181
22.3	Hardware offload: quem carrega o piano?	181
22.4	Laboratório	183
22.4.1	Ambiente	183
22.4.2	Comandos passo a passo	183
22.4.3	Verificação	184
22.4.4	Desafios	185
22.5	Resumo	186
	Bibliografia do capítulo	186
23	VLANs e 802.1Q	187
23.1	O problema: um cabo, várias redes	187
23.2	VLANs no RouterOS: a interface <code>vlan</code>	188

23.3	Laboratório	189
23.3.1	Ambiente	189
23.3.2	Comandos passo a passo	189
23.3.3	Verificação	191
23.3.4	Desafios	191
23.4	Resumo	192
	Bibliografia do capítulo	192
24	Bridge VLAN filtering	194
24.1	Do consentimento à lei	194
24.1.1	Peça 1: a tabela de VLANs da bridge	195
24.1.2	Peça 2: o PVID — a etiqueta de entrada	195
24.1.3	Peça 3: frame-types — o que a porta admite	195
24.2	A gerência e a pegadinha do tagged=bridge	196
24.3	Laboratório	197
24.3.1	Ambiente	197
24.3.2	Comandos passo a passo	197
24.3.3	Verificação	199
24.3.4	Desafios	199
24.4	Resumo	200
	Bibliografia do capítulo	200
25	Switch chip nos CRS	201
25.1	O porão do switch	201
25.2	Dois mundos, uma decisão	202
25.3	O que mais mora no chip	202
25.4	Laboratório	203
25.4.1	Ambiente	203
25.4.2	Comandos passo a passo	204
25.4.3	Verificação	204
25.4.4	Desafios	205
25.5	Resumo	206
	Bibliografia do capítulo	206
26	Segmentação da rede do provedor	207
26.1	A ordem de serviço	207
26.2	Os três princípios	208
26.3	O firewall enxergando segmentos	208
26.4	Laboratório	210
26.4.1	Ambiente	210
26.4.2	Comandos passo a passo	210
26.4.3	Verificação	211
26.4.4	Desafios	211
26.5	Resumo	212
	Bibliografia do capítulo	213

V	Volume 5 — Wireless e enlaces de rádio	214
27	Fundamentos de rádio e regulamentação	215
27.1	A língua dos decibéis	215
27.2	As bandas do provedor	216
27.3	Visada e a zona de Fresnel	216
27.4	O link budget: a conta que fecha enlaces	217
27.5	Anatel: as regras do jogo	218
27.6	Laboratório	219
27.6.1	Ambiente	219
27.6.2	Comandos passo a passo	219
27.6.3	Verificação	219
27.6.4	Desafios	220
27.7	Resumo	221
	Bibliografia do capítulo	221
28	Wireless no RouterOS	222
28.1	Duas pilhas, um aviso	222
28.2	Os modos: quem fala, quem escuta, quem repassa	223
28.3	O enlace mínimo: banda, canal, SSID, segurança	223
28.4	A registration table: o estetoscópio	224
28.5	Laboratório	225
28.5.1	Ambiente	225
28.5.2	Comandos passo a passo	225
28.5.3	Verificação	226
28.5.4	Desafios	226
28.6	Resumo	227
	Bibliografia do capítulo	227
29	Enlaces PtP: Nstreme e NV2	228
29.1	O problema: 802.11 foi feito para dentro de casa	228
29.2	Nstreme: o polling da primeira geração	229
29.3	NV2: TDMA de verdade	230
29.3.1	Os dois parâmetros que você precisa entender	230
29.4	Qual protocolo usar?	231
29.4.1	Taxa aérea não é throughput	231
29.5	Laboratório	232
29.5.1	Comandos passo a passo	232
29.5.2	Verificação	234
29.5.3	Desafios	234
29.6	Resumo	235
	Bibliografia do capítulo	235
30	PtMP rural: a torre que atende dezenas de clientes	236
30.1	Anatomia de uma célula PtMP	236
30.2	O AP setorial	237

30.3	access-list: o porteiro do setor	238
30.3.1	Limites de banda no rádio	239
30.4	O CPE do cliente	239
30.5	Laboratório	240
30.5.1	Comandos passo a passo	240
30.5.2	Verificação	241
30.5.3	Desafios	241
30.6	Resumo	242
	Bibliografia do capítulo	243
31	Alinhamento e troubleshooting de enlaces	244
31.1	Alinhar é maximizar, não “achar”	244
31.1.1	O método (vale para PtP e CPE)	245
31.2	A caixa de ferramentas de diagnóstico	246
31.3	As quatro assinaturas de enlace degradado	247
31.4	Laboratório	248
31.4.1	Comandos passo a passo	248
31.4.2	Verificação	249
31.4.3	Desafios	249
31.5	Resumo	250
	Bibliografia do capítulo	250
32	CAPsMAN e o WiFi moderno	251
32.1	Os dois stacks de wireless do v7	251
32.2	O modelo: controlador e CAPs	252
32.3	Configurando o controlador	253
32.4	Ligando um CAP	254
32.4.1	Quando <i>não</i> usar CAPsMAN	255
32.5	Laboratório	255
32.5.1	Comandos passo a passo	255
32.5.2	Verificação	256
32.5.3	Desafios	256
32.6	Resumo	257
	Bibliografia do capítulo	258
VI	Volume 6 — PPPoE e gestão de assinantes	259
33	Como funciona o PPPoE	260
33.1	Por que não DHCP e pronto?	260
33.2	Fase 1: descoberta (PPPoED)	261
33.3	Fase 2: sessão (PPP)	261
33.4	O pedágio: MTU 1492 e o site que não abre	262
33.5	Laboratório	263
33.5.1	Comandos passo a passo	263
33.5.2	Verificação	265

33.5.3	Desafios	265
33.6	Resumo	266
	Bibliografia do capítulo	266
34	PPPoE server: profiles, pools e secrets	268
34.1	As três camadas do concentrador	268
34.2	Camada 1 — o pool	269
34.3	Camada 2 — profiles (os planos)	269
34.4	Camada 3 — secrets (os assinantes)	270
34.5	Publicando o serviço	271
34.6	Operações do dia a dia	272
34.7	Laboratório	273
34.7.1	Comandos passo a passo	273
34.7.2	Verificação	274
34.7.3	Desafios	274
34.8	Resumo	275
	Bibliografia do capítulo	275
35	RADIUS, User Manager e gestão	276
35.1	AAA: o modelo por trás de tudo	276
35.2	Os atributos: onde mora a autorização	277
35.3	O lado concentrador: cliente RADIUS	278
35.4	O lado servidor: User Manager	279
35.5	CoA: mudar a sessão sem derrubá-la	279
35.6	Laboratório	280
35.6.1	Comandos passo a passo	280
35.6.2	Verificação	281
35.6.3	Desafios	282
35.7	Resumo	283
	Bibliografia do capítulo	283
36	Boas práticas de concentrador (BNG)	284
36.1	O que limita primeiro	284
36.2	Keepalive: detectar a queda sem criar avalanche	285
36.3	Blindando o papel	286
36.4	Múltiplos concentradores	286
36.5	Laboratório	287
36.5.1	Comandos passo a passo	287
36.5.2	Verificação	289
36.5.3	Desafios	289
36.6	Resumo	290
	Bibliografia do capítulo	290

VII Volume 7 — QoS e controle de banda	291
37 Fundamentos de QoS e simple queues	292
37.1 O que uma fila realmente faz	292
37.2 Simple queue: o canivete suíço	293
37.3 A ordem importa (muito)	294
37.4 Lendo uma fila como um instrumento	294
37.5 Laboratório	295
37.5.1 Comandos passo a passo	295
37.5.2 Verificação	296
37.5.3 Desafios	297
37.6 Resumo	297
Bibliografia do capítulo	298
38 Burst e planos de velocidade	299
38.1 Burst: velocidade emprestada por tempo limitado	299
38.2 limit-at: a garantia (CIR)	300
38.3 Levando para o PPPoE	302
38.4 Laboratório	302
38.4.1 Comandos passo a passo	302
38.4.2 Verificação	303
38.4.3 Desafios	303
38.5 Resumo	304
Bibliografia do capítulo	305
39 Mangle e queue tree: priorizando por tipo de tráfego	306
39.1 Marcar barato: o padrão connection-mark → packet-mark	306
39.2 Queue tree: o HTB exposto	307
39.2.1 Onde pendurar: interface ou global	309
39.3 Laboratório	309
39.3.1 Comandos passo a passo	310
39.3.2 Verificação	311
39.3.3 Desafios	311
39.4 Resumo	312
Bibliografia do capítulo	312
40 PCQ: controle de banda por assinante em escala	314
40.1 Tipos de fila: o algoritmo dentro da fila	314
40.2 O mecanismo	315
40.2.1 Os dois modos do PCQ	316
40.3 PCQ ou PPPoE? O critério	316
40.4 Laboratório	317
40.4.1 Comandos passo a passo	317
40.4.2 Verificação	318
40.4.3 Desafios	318
40.5 Resumo	319

Bibliografia do capítulo	320
VIII Volume 8 — Roteamento	321
41 Rotas estáticas, distância e failover	322
41.1 Como o roteador decide	322
41.2 Anatomia de uma rota no v7	323
41.3 Failover: rotas flutuantes + check-gateway	324
41.4 Blackhole: descartar com propósito	325
41.5 Diagnóstico em dois comandos	325
41.6 Laboratório	326
41.6.1 Comandos passo a passo	326
41.6.2 Verificação	327
41.6.3 Desafios	327
41.7 Resumo	328
Bibliografia do capítulo	329
42 OSPF: conceitos e configuração	330
42.1 Link-state: cada roteador tem o mapa inteiro	330
42.2 Vizinhança, custo e áreas	331
42.3 Configurando OSPFv2 no RouterOS v7	332
42.4 Lendo o OSPF	333
42.5 Laboratório	333
42.5.1 Comandos passo a passo	334
42.5.2 Verificação	335
42.5.3 Desafios	335
42.6 Resumo	336
Bibliografia do capítulo	336
43 OSPF avançado: áreas, autenticação e filtros	337
43.1 Por que múltiplas áreas	337
43.2 Tipos de área: encolhendo a tabela da ponta	338
43.3 Autenticação: OSPF de produção não roda “no escuro”	339
43.4 Contendo o flapping: temporizadores de SPF e LSA	339
43.5 Filtrando o que entra e sai do OSPF	340
43.6 Laboratório	340
43.6.1 Comandos passo a passo	341
43.6.2 Verificação	342
43.6.3 Desafios	342
43.7 Resumo	343
Bibliografia do capítulo	343
44 BGP: conceitos e peering	344
44.1 AS, ASN e a lógica do BGP	344
44.2 eBGP e iBGP: dois sabores, um protocolo	345

44.3	Como o BGP decide	345
44.4	Configurando BGP no RouterOS v7	346
44.5	Lendo e diagnosticando a sessão	347
44.6	Laboratório	347
44.6.1	Comandos passo a passo	347
44.6.2	Verificação	348
44.6.3	Desafios	349
44.7	Resumo	349
	Bibliografia do capítulo	350
45	BGP: filtros, communities e segurança	351
45.1	Routing filters: firewall de rotas	351
45.2	O filtro de saída mínimo seguro	352
45.3	O filtro de entrada: bogons e o próprio bloco	352
45.4	Communities: etiquetas que disparam política	353
45.5	AS-path prepend: influenciando o tráfego de entrada	353
45.6	RPKI e ROA: validação de origem (conceitual)	354
45.7	Laboratório	355
45.7.1	Comandos passo a passo	355
45.7.2	Verificação	356
45.7.3	Desafios	356
45.8	Resumo	357
	Bibliografia do capítulo	357
46	VRF e boas práticas de roteamento no v7	359
46.1	O problema que a VRF resolve	359
46.2	VRF, routing tables e routing rules no v7	360
46.3	Criando e usando uma VRF	360
46.4	Route leaking: vazar com controle	361
46.5	Policy routing: tráfego por caminho, sem VRF	361
46.6	Checklist de boas práticas de roteamento no v7	362
46.7	Laboratório	362
46.7.1	Comandos passo a passo	362
46.7.2	Verificação	363
46.7.3	Desafios	364
46.8	Resumo	365
	Bibliografia do capítulo	365
IX	Volume 9 — CGNAT e IPv6	366
47	CGNAT e netmap	367
47.1	Por que CGNAT e onde ele fica	367
47.2	O espaço 100.64.0.0/10 (RFC 6598)	368
47.3	masquerade, srcnat e netmap	368
47.4	CGNAT determinístico com netmap	368

47.5	Portas por assinante e oversubscription	369
47.6	Laboratório	370
47.6.1	Comandos passo a passo	370
47.6.2	Verificação	371
47.6.3	Desafios	371
47.7	Resumo	372
	Bibliografia do capítulo	372
48	Logging de CGNAT e requisitos legais	373
48.1	O que a lei pede (em português de engenheiro)	373
48.2	Dois caminhos: logar tudo vs. determinístico	374
48.3	O registro do lado do assinante: quem teve qual 100.64	375
48.4	Configurando o registro no RouterOS	375
48.5	Laboratório	376
48.5.1	Comandos passo a passo	376
48.5.2	Verificação	377
48.5.3	Desafios	377
48.6	Resumo	378
	Bibliografia do capítulo	378
49	IPv6 no RouterOS: fundamentos	379
49.1	Lendo um endereço IPv6	379
49.2	Os tipos de endereço que importam	380
49.3	ND, RA e SLAAC: como o host se configura sozinho	380
49.4	Habilitando IPv6 no RouterOS	381
49.5	Planejando o endereçamento do provedor	381
49.6	Laboratório	382
49.6.1	Comandos passo a passo	382
49.6.2	Verificação	383
49.6.3	Desafios	383
49.7	Resumo	384
	Bibliografia do capítulo	384
50	DHCPv6-PD e dual stack no PPPoE	386
50.1	Prefix Delegation: delegar uma rede	386
50.2	O servidor DHCPv6-PD no concentrador	387
50.3	Dual stack sobre PPPoE	387
50.4	O lado do cliente: pedindo o prefixo (client-PD)	388
50.5	Diagnóstico ponta a ponta	388
50.6	Laboratório	389
50.6.1	Comandos passo a passo	389
50.6.2	Verificação	390
50.6.3	Desafios	390
50.7	Resumo	391
	Bibliografia do capítulo	392

51 Firewall IPv6	393
51.1 Por que IPv6 sem firewall é um erro	393
51.2 O que muda: ICMPv6 não é o ICMP do IPv4	394
51.3 Protegendo o roteador (chain input)	394
51.4 Protegendo a LAN do cliente (chain forward)	395
51.5 O CPE dual stack: firewall dos dois lados	395
51.6 Laboratório	396
51.6.1 Comandos passo a passo	396
51.6.2 Verificação	397
51.6.3 Desafios	397
51.7 Resumo	398
Bibliografia do capítulo	399
 X Volume 10 — Túneis e VPN	 400
52 Panorama de túneis no RouterOS	401
52.1 O que é um túnel, em uma frase	401
52.2 O menu de túneis do RouterOS	402
52.3 O overhead e o problema de MTU (o inimigo comum)	403
52.4 Escolhendo por cenário de provedor	403
52.5 Laboratório	404
52.5.1 Comandos passo a passo	404
52.5.2 Verificação	405
52.5.3 Desafios	405
52.6 Resumo	406
Bibliografia do capítulo	406
 53 WireGuard	 407
53.1 O modelo mental do WireGuard	407
53.2 allowed-address: o conceito que tudo decide	408
53.3 Montando: túnel de gerência	408
53.4 Montando: site-to-site entre POPs	409
53.5 persistent-keepalive: mantendo vivo através do NAT	409
53.6 Laboratório	410
53.6.1 Comandos passo a passo	410
53.6.2 Verificação	411
53.6.3 Desafios	411
53.7 Resumo	412
Bibliografia do capítulo	412
 54 IPsec	 413
54.1 As peças do IPsec no RouterOS	413
54.2 Duas fases: IKE negocia, ESP cifra	414
54.3 policy-based vs. tunnel/VTI	414
54.4 Montando um site-to-site IKEv2 (PSK)	415

54.5	Diagnóstico do IPsec	415
54.6	Laboratório	416
54.6.1	Comandos passo a passo	416
54.6.2	Verificação	417
54.6.3	Desafios	417
54.7	Resumo	418
	Bibliografia do capítulo	418
55	GRE, EoIP, L2TP/SSTP e ZeroTier	419
55.1	GRE: a interface para transportar roteamento	419
55.2	EoIP: esticar a camada 2 entre sites	420
55.3	L2TP e SSTP: acesso remoto	420
55.4	ZeroTier: malha que atravessa NAT	421
55.5	Escolhendo entre eles	421
55.6	Laboratório	422
55.6.1	Comandos passo a passo	422
55.6.2	Verificação	423
55.6.3	Desafios	423
55.7	Resumo	424
	Bibliografia do capítulo	424
56	Interligação de POPs por túnel	425
56.1	O desenho: túnel + roteamento + criptografia	425
56.2	Caso 1: os dois POPs têm IP público	426
56.3	Caso 2: um POP atrás de CGNAT	426
56.4	Redundância: dois túneis, failover por OSPF	427
56.5	Integração com o roteamento do provedor	427
56.6	Laboratório	428
56.6.1	Comandos passo a passo	428
56.6.2	Verificação	429
56.6.3	Desafios	429
56.7	Resumo	430
	Bibliografia do capítulo	430
XI	Volume 11 — MPLS	431
57	MPLS: conceitos e LDP	432
57.1	Comutar por rótulo, não por IP	432
57.2	A pilha de rótulos e o LSP	433
57.3	LDP: distribuindo rótulos automaticamente	433
57.4	Habilitando MPLS + LDP no RouterOS v7	434
57.5	Lendo e diagnosticando	434
57.6	Laboratório	435
57.6.1	Comandos passo a passo	435
57.6.2	Verificação	436

57.6.3	Desafios	436
57.7	Resumo	437
	Bibliografia do capítulo	437
58	VPLS: camada 2 sobre o backbone	438
58.1	O que a VPLS entrega	438
58.2	Montando VPLS no RouterOS v7	439
58.3	MTU, MACs e os perigos da L2 distribuída	440
58.4	VPLS ou roteamento? A pergunta certa	440
58.5	Laboratório	441
58.5.1	Comandos passo a passo	441
58.5.2	Verificação	442
58.5.3	Desafios	442
58.6	Resumo	443
	Bibliografia do capítulo	443
59	Traffic Engineering	444
59.1	O problema: a IGP não pensa em capacidade	444
59.2	Túneis TE, caminho explícito e reserva	445
59.3	RSVP-TE e o estado no RouterOS	445
59.4	TE na prática de um provedor MikroTik	446
59.5	Laboratório	447
59.5.1	Comandos passo a passo	447
59.5.2	Verificação	448
59.5.3	Desafios	448
59.6	Resumo	449
	Bibliografia do capítulo	449
XII	Volume 12 — Scripts e automação	450
60	Linguagem de script do RouterOS	451
60.1	Variáveis e tipos	451
60.2	Condicionais e a estrutura do controle	452
60.3	Loops e a operação sobre itens de menu	452
60.4	Capturando saída e parseando	453
60.5	Funções reutilizáveis	453
60.6	Tratamento de erro e idempotência	454
60.7	Laboratório	455
60.7.1	Comandos passo a passo	455
60.7.2	Verificação	456
60.7.3	Desafios	456
60.8	Resumo	457
	Bibliografia do capítulo	457

61 Scheduler, netwatch e backups automáticos	458
61.1 Scheduler: rodar no tempo certo	458
61.2 Netwatch: reagir a eventos de rede	459
61.3 Backups automáticos: binário, export e envio externo	459
61.4 Juntando tudo: uma rotina de provedor	460
61.5 Laboratório	461
61.5.1 Comandos passo a passo	461
61.5.2 Verificação	462
61.5.3 Desafios	462
61.6 Resumo	463
Bibliografia do capítulo	463
62 API e provisionamento em massa	464
62.1 As três formas de falar com o RouterOS de fora	464
62.2 Habilitando a REST API com segurança	465
62.3 Fazendo chamadas REST	465
62.4 Provisionamento em massa: o padrão template	466
62.5 Na caixa ou de fora?	467
62.6 Laboratório	467
62.6.1 Comandos passo a passo	467
62.6.2 Verificação	468
62.6.3 Desafios	469
62.7 Resumo	470
Bibliografia do capítulo	470
Apêndices	471
Referências	471

Apresentação

Bem-vindo ao **Manual de MikroTik**, um livro aberto da série *Manuais de Ciências*. Este manual leva você do zero absoluto — “o que é um roteador MikroTik?” — até a operação do backbone de um provedor de internet real, com fibra óptica e enlaces de rádio em área rural.

Para quem é este manual

Para quem está começando em redes e quer chegar ao nível de **operar a rede de um provedor (ISP)**: técnicos de campo que querem migrar para o NOC, donos de pequenos provedores, estudantes e curiosos. Não pressupomos conhecimento prévio de RouterOS; cada conceito é explicado antes de ser usado, e a ordem dos capítulos respeita os pré-requisitos entre eles.

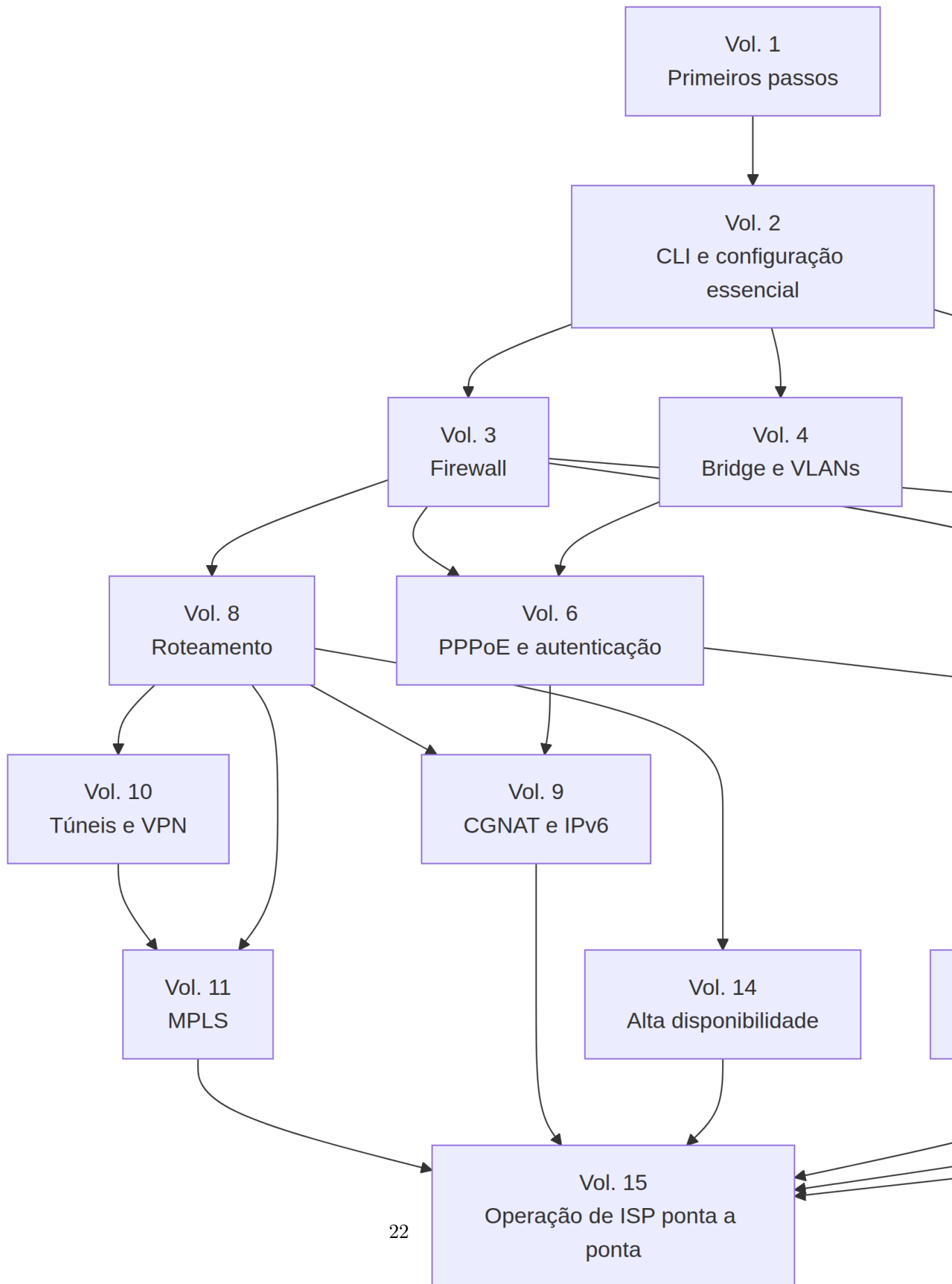
Se você quiser reforçar a teoria de redes por trás dos capítulos (modelo OSI, endereçamento IP, fibra óptica, rádio), o [Manual de Redes de Computadores](#) da mesma série é o companheiro ideal deste livro.

Como este manual funciona

- **Progressão gradual**: os volumes formam três fases — fundamentos, provedor na prática e operação avançada. Cada capítulo declara o que você precisa ter visto antes.
- **Laboratório em todo capítulo**: você pratica *enquanto* estuda, usando o **MikroTik CHR** (RouterOS em máquina virtual) — sem precisar comprar hardware. O Capítulo 5 ensina a montar o ambiente uma única vez; os demais capítulos apenas indicam a topologia necessária.
- **Comandos reais**: todos os blocos de código são RouterOS de verdade, prontos para copiar e colar no terminal, sempre explicados linha a linha.
- **Desafios com solução**: cada laboratório termina com exercícios; as soluções ficam escondidas em caixas colapsáveis para você tentar antes de olhar.

Mapa de pré-requisitos

O grafo abaixo mostra a dependência entre os volumes: siga as setas para saber o que estudar antes de cada assunto.



Grafo de pré-requisitos entre os volumes do manual.

As três fases

Fase 1 — Fundamentos e operação básica (Volumes 1–4): o que é o RouterOS, como acessá-lo e operá-lo com segurança; a linha de comando; endereçamento, DHCP, DNS e NAT; firewall completo; bridges, switching e VLANs. Ao final desta fase você configura, do zero, um roteador de borda seguro e segmentado.

Fase 2 — Provedor na prática (Volumes 5–9): os pilares do dia a dia de um ISP — enlaces de rádio PtP/PtMP, PPPoE com RADIUS, planos de velocidade com QoS, roteamento OSPF/BGP, CGNAT e IPv6.

Fase 3 — Avançado e operação (Volumes 10–15): túneis e VPN entre POPs, MPLS no backbone, automação com scripts, monitoramento, alta disponibilidade e, fechando o livro, o desenho ponta a ponta da rede de um provedor real de fibra e rádio.

Convenções usadas no livro

- Blocos de código `routeros` contêm comandos prontos para colar no terminal. O prompt `[admin@MikroTik] >` só aparece quando ilustramos uma sessão interativa com a saída do equipamento.
- Caixas de aviso () antecedem **todo** comando destrutivo (`reset`, `Netinstall`, remoção de regras).
- Referências bibliográficas seguem o padrão ABNT e estão listadas ao final de cada capítulo.

Bons estudos — e boas configurações!

Parte I

Volume 1 — Primeiros passos

1 O que é MikroTik e RouterOS

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que é a MikroTik (a empresa) e o que é o RouterOS (o sistema);
- Diferenciar RouterOS de SwOS e entender por que essa distinção importa;
- Reconhecer o que torna o MikroTik tão popular entre provedores de internet;
- Localizar, num diagrama de rede de provedor (fibra + rádio), os pontos onde equipamentos MikroTik costumam trabalhar;
- Dar o primeiro passo prático: baixar o CHR (RouterOS em máquina virtual) e fazer o primeiro login.

1.1 A empresa e o sistema

MikroTik é uma empresa da Letônia, fundada em 1996 na capital Riga, que faz duas coisas: um sistema operacional de rede chamado **RouterOS** e o hardware que roda esse sistema, a linha **RouterBOARD**. Essa dupla — software poderoso e hardware barato — transformou a MikroTik numa das marcas mais usadas por provedores de internet do mundo, especialmente em países como o Brasil, onde milhares de ISPs regionais construíram suas redes sobre ela.

Aqui vale uma analogia. Pense num computador comum: ele tem um hardware (a máquina física) e um sistema operacional (Windows, Linux) que faz o hardware ser útil. Com o MikroTik é igual:

- O **RouterOS** é o sistema operacional. É ele que roteia pacotes, filtra tráfego no firewall, autentica assinantes por PPPoE, fala OSPF e BGP com outros roteadores — tudo o que este manual ensina é, no fundo, “como operar o RouterOS”.
- A **RouterBOARD** é a família de hardware: roteadores de mesa, roteadores de rack com dezenas de núcleos, switches, antenas e rádios. Todos saem de fábrica com o RouterOS instalado.

O detalhe importante — e que explica boa parte da fama da marca — é que **o RouterOS é o mesmo em todos os equipamentos**. O sistema que roda num roteadorzinho de R\$ 300 é o mesmo que roda no roteador de rack que segura o BGP de um provedor com dezenas de milhares de clientes. Muda o desempenho do hardware; não muda a linguagem, os menus

nem os conceitos. É por isso que tudo o que você aprender neste manual num laboratório virtual gratuito vale, sem tradução, para qualquer equipamento físico da marca.

Analogia para guardar

RouterOS está para a RouterBOARD assim como o Android está para um celular: o mesmo sistema, em aparelhos de tamanhos e preços muito diferentes. Aprenda o sistema uma vez e você sabe operar todos.

1.1.1 Um pouco de história

A MikroTik nasceu vendendo software: no fim dos anos 1990, o RouterOS era instalado em PCs comuns para transformá-los em roteadores — uma alternativa muito mais barata que os grandes roteadores comerciais da época. Em 2002 a empresa passou a fabricar o próprio hardware (a RouterBOARD), e o conjunto decolou entre provedores pequenos e médios: preço de equipamento doméstico, recursos de equipamento de operadora.

Duas versões do sistema importam para você hoje:

- **RouterOS v6** — dominou as redes por mais de uma década; você ainda vai encontrá-lo em muito equipamento antigo em campo.
- **RouterOS v7** — a geração atual, com motor de roteamento reescrito (OSPF/BGP), WireGuard, suporte a hardware novo e melhorias contínuas. **Este manual ensina RouterOS v7**; quando uma diferença do v6 for relevante na vida real do provedor, ela será apontada explicitamente.

1.1.2 RouterOS ou SwOS?

Alguns equipamentos MikroTik — em especial os switches da linha CRS — podem inicializar com um segundo sistema, o **SwOS**: um firmware minimalista que só faz switching (camada 2) e é configurado por uma página web simples. Ele existe porque, quando um switch só precisa comutar quadros, um sistema enxuto dedicado a isso é mais simples de operar.

A regra prática deste manual: **usamos RouterOS em tudo**. O RouterOS também faz switching (e o faz em hardware, como você verá no Volume 4), e manter um único sistema em todo o parque simplifica a operação, os backups e a automação. Saiba que o SwOS existe para não se confundir ao encontrá-lo, mas nosso caminho é o RouterOS.

1.2 Por que os provedores escolhem MikroTik?

Quatro razões aparecem em praticamente toda rede de ISP construída com a marca:

1. **Custo**: um roteador capaz de concentrar centenas de assinantes PPPoE custa uma fração do equivalente das marcas tradicionais de operadora.

2. **Completeness:** firewall com estado, NAT e CGNAT, PPPoE com RADIUS, QoS, OSPF, BGP, MPLS, VPNs, scripts — tudo incluso no sistema, sem licenças adicionais por recurso.
3. **Versatilidade:** o mesmo sistema opera na borda (BGP com a operadora), no concentrador de assinantes, no switch do POP, no rádio da torre e na casa do cliente.
4. **Comunidade:** fóruns, documentação aberta (MikroTik, 2026a), treinamentos e uma enorme base de conhecimento acumulada por provedores do mundo todo.

Há também um ponto de atenção honesto: todo esse poder vem com **muita responsabilidade de configuração**. O RouterOS entrega as ferramentas; quem decide se a rede será segura e estável é quem a configura. É exatamente essa a lacuna que este manual fecha — inclusive dedicando um volume inteiro a firewall e proteção (Volume 3).

1.3 Onde o MikroTik se encaixa na rede de um provedor

Para entender o que você vai construir ao longo do livro, veja o caminho que os dados percorrem num provedor típico do interior, que atende clientes por fibra óptica na cidade e por rádio na zona rural (Figura 1.1):

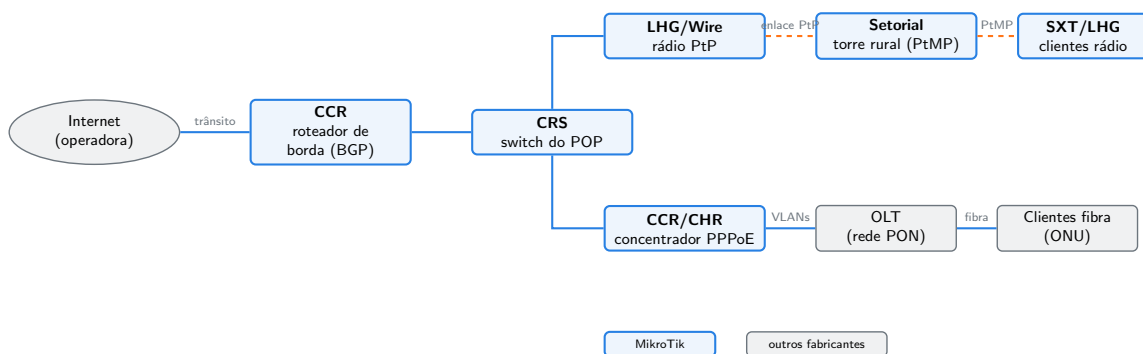


Figura 1.1: Onde os equipamentos MikroTik (em azul) trabalham na rede de um provedor típico de fibra e rádio.

Acompanhe o diagrama da esquerda para a direita:

- **Roteador de borda:** um CCR conversa via **BGP** com a operadora que vende trânsito ao provedor (Volume 8). É a porta de entrada e saída de toda a rede.
- **Switch do POP:** um CRS distribui o tráfego dentro do ponto de presença, separando os serviços em **VLANs** (Volume 4).
- **Concentrador PPPoE (BNG):** um CCR — ou um CHR virtualizado — autentica cada assinante, entrega IP e aplica o plano de velocidade contratado (Volumes 6 e 7).
- **Ramo fibra:** a OLT e as ONUs dos clientes são, em geral, de outros fabricantes — mas quem roteia, autentica e controla a banda desses clientes continua sendo o MikroTik (Volume 15 detalha essa integração).

- **Ramo rádio:** enlaces **PtP** levam a internet até torres na zona rural; lá, antenas setoriais atendem em **PtMP** os rádios na casa de cada cliente — tudo MikroTik, das linhas LHG, SXT e afins (Volume 5).

Guarde esta imagem: os quinze volumes deste manual são, essencialmente, um tour guiado por esse diagrama, do centro para as pontas.

! O conceito mais importante do capítulo

Um único sistema — o RouterOS — opera todos os papéis azuis do diagrama. Aprender RouterOS não é aprender “um roteador”: é aprender a operar **a rede inteira do provedor**.

1.4 O CHR: seu laboratório gratuito

Como praticar tudo isso sem gastar um centavo em equipamento? A resposta é o **CHR** — **Cloud Hosted Router** (MikroTik, 2026b): uma versão do RouterOS empacotada como **máquina virtual**, feita pela própria MikroTik para rodar em VirtualBox, VMware, Proxmox, GNS3, EVE-NG e afins.

O CHR é um RouterOS completo: mesma linha de comando, mesmo firewall, mesmo OSPF e BGP. Sem licença, ele funciona para sempre com uma única limitação — cada interface fica limitada a **1 Mbps** de velocidade, o que não atrapalha em nada o estudo. É nele que rodam todos os laboratórios deste livro.

Neste capítulo você fará o primeiro contato: baixar o CHR, dar o boot e entrar no sistema. A montagem completa do ambiente de estudos — com várias máquinas virtuais interligadas em topologias — é o assunto do Capítulo 5, que você montará uma única vez e reutilizará no livro inteiro.

1.5 Laboratório

1.5.1 Ambiente

Neste primeiro laboratório você precisa apenas de um computador com o **VirtualBox** instalado (gratuito, disponível para Windows, Linux e macOS em [virtualbox.org](https://www.virtualbox.org)). Nada de topologias ainda: uma única máquina virtual basta.

1. Acesse mikrotik.com/download e, na seção **Cloud Hosted Router**, baixe a imagem **VDI** da versão estável mais recente (7.x).
2. No VirtualBox, crie uma máquina virtual nova: tipo **Linux**, versão **Other Linux (64-bit)**, com **256 MB** de RAM.
3. Na etapa de disco, escolha **usar um disco existente** e aponte para o arquivo **.vdi** baixado (não crie disco novo).

4. Inicie a máquina virtual.

Ao final do boot aparece o prompt de login do RouterOS na janela da VM.

1.5.2 Comandos passo a passo

Faça login com o usuário padrão de fábrica — usuário **admin**, senha em branco (só pressionar Enter):

```
MikroTik Login: admin
Password:
```

Nas versões atuais, o RouterOS pede imediatamente uma senha nova — escolha uma e anote. Em seguida ele mostra a licença e o banner com o logotipo, e você cai no prompt do terminal:

```
[admin@MikroTik] >
```

Esse prompt diz duas coisas: quem você é (**admin**) e o nome do equipamento (**MikroTik**, a identidade de fábrica). Vamos fazer o RouterOS se apresentar. O comando abaixo mostra o “RG” do sistema — versão, tempo ligado, memória, CPU e placa:

```
/system resource print
```

A saída esperada é parecida com esta (os números variam conforme sua máquina):

```
[admin@MikroTik] > /system resource print
    uptime: 4m12s
    version: 7.15.3 (stable)
    build-time: 2024-07-24 10:39:01
    free-memory: 190.1MiB
    total-memory: 224.0MiB
    cpu: QEMU
    cpu-count: 1
    cpu-load: 1%
    free-hdd-space: 43.1MiB
    total-hdd-space: 61.4MiB
    board-name: CHR
    platform: MikroTik
```

Repare em **version: 7.x** (estamos no RouterOS v7) e **board-name: CHR** (é a versão virtual). Agora veja as interfaces de rede que a VM enxerga:

```
/interface print
```

```
[admin@MikroTik] > /interface print
Flags: R - RUNNING
Columns: NAME, TYPE, ACTUAL-MTU, MAC-ADDRESS
#  NAME      TYPE      ACTUAL-MTU  MAC-ADDRESS
0 R ether1   ether      1500        08:00:27:6A:2B:1C
```

Uma única `ether1`: é a placa de rede virtual do VirtualBox. Nos próximos capítulos adicionaremos mais. Por fim, dê um nome ao seu roteador — boa prática que você levará para a vida de provedor, onde cada equipamento do parque precisa de identidade única:

```
/system identity set name=lab-r1
```

O prompt muda na hora, confirmando a alteração:

```
[admin@lab-r1] >
```

1.5.3 Verificação

Confirme os três resultados do laboratório:

1. `/system resource print` mostra `version` começando com 7. e `board-name: CHR`;
2. `/interface print` lista ao menos `ether1` com a flag `R` (running);
3. O prompt exibe `[admin@lab-r1]` — a identidade foi gravada.

Se a VM não deu boot, verifique se você **apontou o disco existente** (o `.vdi` baixado) em vez de criar um disco vazio — esse é o erro mais comum.

1.5.4 Desafios

1. Descubra, usando apenas a tecla `?` no terminal, o que faz o comando `/system clock print` — e execute-o. O horário está certo?
2. O RouterOS organiza os comandos em menus, como pastas. Entre no menu `/system` (digite `/system` e Enter) e liste o que há dentro dele com `?`. Depois volte à raiz com `/`.
3. Sem consultar a internet: qual comando você usaria para trocar a identidade do roteador para `chr-teste`? Execute-o e desfaça em seguida, voltando para `lab-r1`.
4. Desligue o roteador **pelo terminal** (não feche a janela da VM no X!). Dica: explore o menu `/system` do desafio 2.

💡 Solução dos desafios

1. Digite `/system clock print ?` (ou apenas execute o comando). Ele mostra data, hora e fuso configurados:

```
/system clock print
```

No CHR recém-instalado o fuso costuma vir errado (`gmt` ou detectado pela VM); você aprenderá a ajustá-lo no capítulo de primeiro acesso.

2. Sessão esperada:

```
[admin@lab-r1] > /system  
[admin@lab-r1] /system> ?
```

O `?` lista submenus como `clock`, `identity`, `resource`, `reboot`, `shutdown`... Para voltar à raiz:

```
[admin@lab-r1] /system> /  
[admin@lab-r1] >
```

3. O mesmo comando do corpo do capítulo, com outro valor:

```
/system identity set name=chr-teste  
/system identity set name=lab-r1
```

4. O comando é `/system shutdown`. Ele pede confirmação — responda `y`:

```
[admin@lab-r1] > /system shutdown  
Shutdown, yes? [y/N]:
```

Desligar pelo sistema (em vez de fechar a VM “no cabo”) evita corromper o armazenamento — nos equipamentos físicos de produção, vale a mesma regra.

1.6 Resumo

- **MikroTik** é a empresa (Letônia, 1996); **RouterOS** é o sistema operacional de rede; **RouterBOARD** é a linha de hardware que o roda.
- O RouterOS é **o mesmo** do equipamento mais barato ao mais caro — o que você aprende no laboratório vale para todo o parque.
- **SwOS** é um firmware alternativo minimalista para switches; neste manual, operamos tudo com RouterOS.
- Provedores escolhem MikroTik por **custo, completude, versatilidade e comunidade** — e assumem a responsabilidade de configurar bem.

- Na rede de um ISP, o MikroTik aparece da **borda BGP** ao **rádio do cliente**, passando por switch, concentrador PPPoE e enlaces PtP/PtMP (Figura 1.1).
- O **CHR** é o RouterOS em máquina virtual, gratuito (limitado a 1 Mbps por interface) — o laboratório perfeito, usado no livro inteiro.
- Você já fez o primeiro login, leu `/system resource print` e batizou seu roteador com `/system identity set`.

No próximo capítulo, conheceremos as linhas de hardware — hEX, hAP, CRS, CCR, LHG/SXT — e como escolher o equipamento certo para cada papel do diagrama.

Bibliografia do capítulo

Para aprofundar o que vimos aqui: a documentação oficial do RouterOS (MikroTik, 2026a) e a página do CHR (MikroTik, 2026b) são as fontes primárias que acompanharão todo o livro; Discher (2016) e Burgess (2011) são bons livros introdutórios em inglês; e Tanenbaum; Feamster; Wetherall (2021) cobre a teoria de redes por trás de tudo. As referências completas, em ABNT, estão no apêndice [Referências](#).

2 Linhas de hardware MikroTik

Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Decifrar o nome de um produto MikroTik (ex.: CCR2004-1G-12S+2XS) e saber, só pelo código, quantas e quais portas ele tem;
- Reconhecer as grandes famílias de hardware — hEX, hAP, RB/L009/RB5009, CRS, CCR, SXT/LHG/DISC — e o papel de cada uma na rede de um provedor;
- Escolher equipamento pelo **papel** que ele exercerá (borda, concentrador, switch de POP, rádio de torre, CPE do cliente), e não pelo preço;
- Avaliar as especificações que realmente importam: CPU (núcleos *versus* desempenho por núcleo), portas SFP/SFP+, PoE e uso externo;
- Consultar o catálogo e as fichas técnicas oficiais da MikroTik com olhar crítico.

2.1 O catálogo visto de longe

No Capítulo 1 você aprendeu que o RouterOS é o mesmo em todos os equipamentos da marca. A consequência prática é libertadora: **escolher hardware MikroTik é escolher desempenho e portas, nunca “recursos”**. O firewall, o PPPoE, o OSPF e o BGP estão em todos; o que muda é quanta CPU e quantas interfaces existem para executá-los.

Por isso, este capítulo não é um catálogo decorado — é um mapa. A pergunta certa nunca é “qual MikroTik é o melhor?”, e sim “**qual papel este equipamento vai exercer na minha rede?**”. Cada papel tem uma família de produtos desenhada para ele.

Analogia para guardar

Escolher equipamento por papel é como montar um time de futebol: não existe “o melhor jogador”, existe o melhor goleiro, o melhor zagueiro, o melhor atacante. Um CCR no papel de rádio de cliente é um goleiro de centroavante — caro e mal aproveitado.

2.2 Decifrando o nome dos produtos

Os nomes MikroTik parecem sopa de letrinhas, mas seguem um código consistente. A parte final do nome descreve as **portas**, e cada letra tem significado (Tabela 2.1):

Tabela 2.1: Sufixos de porta na nomenclatura MikroTik

Código	Significado	Velocidade típica
5, 8, 12...	quantidade de portas do tipo que segue	—
G	Ethernet Gigabit (RJ45)	1 Gbps
P	porta com PoE de saída (alimenta outro equipamento)	—
S	gaiola SFP (fibra)	1 Gbps
S+	gaiola SFP+ (fibra)	10 Gbps
XS	gaiola SFP28 (fibra)	25 Gbps
Q+	gaiola QSFP+ (fibra)	40 Gbps

Com a tabela, o nome assustador CCR2004-1G-12S+2XS vira uma frase: é um **C**loud **C**ore **R**outer da geração 2004, com **1** porta **G**igabit, **12** gaiolas **SFP+** (10 Gbps) e **2** gaiolas **SFP28** (25 Gbps). Um roteador nascido para viver num rack cheio de fibra.

Vale o mesmo para os menores: o hEX S é um hEX com uma gaiola **SFP**; o hAP ax² é um hAP com WiFi padrão **ax** de segunda geração. Quando encontrar um nome novo, decifre as portas primeiro — elas dizem para que rede o produto foi feito.

2.3 As famílias, papel por papel

A Figura 2.1 posiciona as famílias no mesmo diagrama de provedor que você conheceu no capítulo anterior — agora com os nomes das linhas de produto em cada posição:

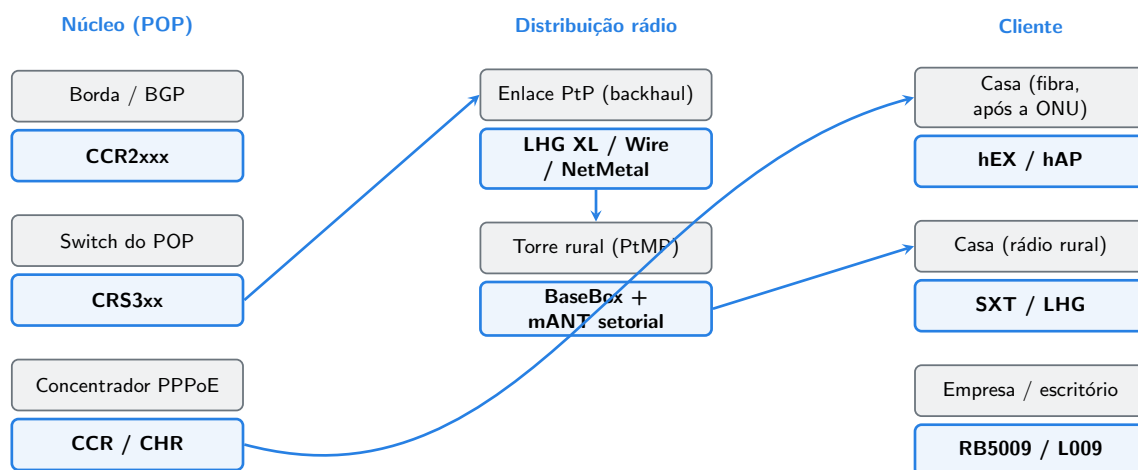


Figura 2.1: As famílias de produto MikroTik posicionadas pelo papel que exercem na rede de um provedor de fibra e rádio.

Vamos percorrer as famílias na mesma ordem — do núcleo para o cliente.

2.3.1 CCR — Cloud Core Router: o núcleo

A linha **CCR** é a série de roteadores de rack da MikroTik: muitos núcleos de CPU, muita porta de fibra, fontes redundantes nos maiores modelos. É o hardware para os papéis mais pesados da rede:

- **Borda BGP** — receber a tabela cheia da internet de uma ou mais operadoras exige memória e CPU; um CCR2004 ou CCR2116 é a escolha natural.
- **Concentrador PPPoE (BNG)** — autenticar milhares de sessões e aplicar filas de velocidade é trabalho de CPU; os CCR de geração 2 (CCR2004, CCR2116, CCR2216) usam núcleos **ARM64** rápidos, muito superiores aos antigos.

Uma nota histórica que ainda importa em campo: os CCR de primeira geração (CCR1009, CCR1016, CCR1036) usam a arquitetura **TILE** — dezenas de núcleos individualmente lentos. Eles brilham quando o tráfego se espalha por muitos núcleos, mas sofrem em tarefas que rodam num núcleo só. Você aprenderá no Volume 6 que uma sessão PPPoE é exatamente esse caso — e por que um CCR2004 com 4 núcleos modernos supera um CCR1036 com 36 núcleos antigos em muitos cenários de provedor.

2.3.2 CRS — Cloud Router Switch: o switch do POP

A linha **CRS** tem cara de switch e alma dupla: roda RouterOS completo, mas carrega um **switch chip** — um circuito dedicado que comuta quadros em hardware, sem gastar CPU. É a família dos POPs: CRS3xx e CRS5xx oferecem combinações de portas Gigabit, SFP+ e até QSFP+ para agregar os enlaces do provedor.

Dois cuidados com essa linha:

- A CPU dos CRS é modesta. Eles são **switches** que sabem rotear em caso de necessidade, não roteadores. Colocar um CRS para concentrar PPPoE é receita de suporte lotado.
- Os modelos CSS (sem o “R”) rodam **apenas SwOS** — o firmware minimalista do Capítulo 1. Confira a letra antes de comprar.

O Volume 4 é inteiro dedicado a extrair o máximo dos CRS: bridge, VLANs e switching em hardware.

2.3.3 hEX, hAP e L009/RB5009 — roteadores de mesa

São os “roteadores de caixinha”, cada um com sua vocação:

- **hEX** — roteador cabeado de 5 portas Gigabit. Barato e confiável, é o clássico atrás da ONU do cliente de fibra que contratou IP fixo ou precisa de um roteador de verdade. O hEX S acrescenta uma gaiola SFP e PoE de saída.
- **hAP** — o hEX com WiFi. A linha atual (hAP ax², hAP ax³) entrega WiFi 6 e é o CPE padrão que muitos provedores instalam na casa do assinante. No Volume 5 você o verá gerenciado em massa via CAPsMAN.

- **L009 e RB5009** — o degrau acima: mais CPU, porta de 2.5/10 Gbps, versões com PoE. Servem pequenas empresas, condomínios e até papéis de POP compacto (um RB5009 aguenta um POP de bairro inteiro).

2.3.4 SXT, LHG, DISC, Wire — a família de rádio

O ramo rural do provedor vive desta família — antenas com o rádio (e o RouterOS) embutidos, prontas para poste e torre:

- **SXT** — antena compacta, abertura de feixe mais larga; o rádio de cliente para distâncias curtas e médias.
- **LHG** — *Light Head Grid*: uma grade parabólica leve com ganho alto; o rádio de cliente para longas distâncias e a opção econômica para enlaces PtP de backhaul (versões LHG XL têm ganho ainda maior).
- **DISC** — parábola sólida fechada, imune a acúmulo de detritos, papel semelhante ao LHG.
- **Wire (Wireless Wire)** — enlaces em **60 GHz**: multi-gigabit em distâncias curtas, perfeitos para atravessar uma rua ou ligar prédios.
- **BaseBox/NetMetal + mANT** — rádios em caixa metálica sem antena integrada, que você combina com uma antena externa — tipicamente uma **setorial mANT** na torre para atender dezenas de clientes em PtMP.

O Volume 5 detalha como projetar esses enlaces; por ora, guarde o padrão: **setorial na torre, SXT/LHG na casa do cliente, LHG XL/Wire no backhaul**.

2.3.5 CHR — o “hardware” que é software

Por fim, lembre que o **CHR** (Capítulo 1) é um membro pleno do catálogo: um RouterOS que roda em máquina virtual sobre um servidor x86. Em produção, provedores usam CHR como concentrador PPPoE, como roteador de borda e como caixa de ferramentas de rede — com a vantagem de escalar CPU e memória comprando servidor, não roteador. Todos os laboratórios deste livro rodam em CHR justamente porque ele é um RouterOS de verdade.

2.4 As especificações que realmente importam

Ficha técnica de roteador tem dezenas de números. Na prática do provedor, cinco decidem a compra:

1. **Desempenho por núcleo da CPU** — tarefas como uma sessão PPPoE ou uma fila de QoS rodam num único núcleo. Quatro núcleos rápidos valem mais que trinta e seis lentos para esses papéis.

2. **Testes de throughput no cenário certo** — a MikroTik publica números em três condições: *fast path* (encaminhamento otimizado, sem firewall), *bridging/routing* simples e **com 25 regras de firewall**. O último é o único próximo da sua realidade. Desconfie de qualquer número de capa.
3. **Portas de fibra (S/S+/XS)** — provedor cresce em fibra. Um equipamento de núcleo sem SFP+ nasce obsoleto.
4. **PoE** — nas pontas, alimentar o rádio pelo próprio cabo de rede (PoE de saída no equipamento de baixo, ou injetor) elimina fonte na torre — menos ponto de falha.
5. **Uso externo e temperatura** — equipamentos de torre (SXT, LHG, BaseBox) são selados para intempérie; equipamentos de mesa, não. Parece óbvio, até você encontrar um hEX dentro de uma marmita de PVC no alto de um poste.

! O conceito mais importante do capítulo

Todo equipamento MikroTik roda o mesmo RouterOS — portanto, **compra-se capacidade, não recurso**. Defina primeiro o papel (borda, concentrador, switch, rádio, CPE), depois dimensione CPU e portas para esse papel, usando os números de teste **com firewall**, não os de capa.

2.5 Laboratório

2.5.1 Ambiente

O mesmo do capítulo anterior: a VM única do CHR no VirtualBox (Capítulo 1). Você também vai precisar de um navegador para consultar mikrotik.com — parte deste laboratório é aprender a ler o catálogo oficial.

2.5.2 Comandos passo a passo

Primeiro, vamos ver o que o RouterOS informa sobre o “hardware” em que está rodando. No CHR:

```
/system resource print
```

Observe três campos na saída: `board-name` (no CHR, CHR; num equipamento físico, o modelo — ex.: hEX S), `cpu-count` e `architecture-name` (x86_64 no CHR; arm64, mipsbe ou tile nos físicos):

```
[admin@lab-r1] > /system resource print
      uptime: 12m4s
      version: 7.15.3 (stable)
architecture-name: x86_64
```

```
board-name: CHR
cpu-count: 1
```

Num equipamento físico existe um menu a mais, que o CHR não tem por ser virtual — ele mostra modelo, número de série e a versão do RouterBOOT (o bootloader, assunto do próximo capítulo):

```
/system routerboard print
```

No CHR esse comando responde `not routerboard` — comportamento esperado, e uma forma rápida de descobrir se você está numa máquina virtual ou física.

Agora a parte de catálogo. Abra mikrotik.com/products no navegador e localize a ficha do **CCR2004-1G-12S+2XS**. Na aba de especificações, encontre a tabela **Test results** e compare as linhas de throughput em *fast path* e com *25 ip filter rules*. Anote os dois números — a diferença entre eles é o “preço” do firewall, e você reencontrará esse conceito no Volume 3.

2.5.3 Verificação

1. `/system resource print` no seu CHR mostra `architecture-name: x86_64` e `board-name: CHR`;
2. `/system routerboard print` responde `not routerboard` (você está numa VM);
3. Você anotou os dois números de throughput do CCR2004 (com e sem firewall) e sabe explicar por que são diferentes.

2.5.4 Desafios

1. Decifre, sem consultar o site: quantas portas, e de que tipo, tem um CRS326-24G-2S+? E um CCR2116-12G-4S+?
2. Um provedor precisa de um switch para um POP novo: 20 clientes de fibra dedicados em portas Gigabit e 2 uplinks de 10 Gbps para o núcleo. Que família e que modelo do desafio 1 atendem?
3. Um cliente rural mora a 12 km da torre, com visada limpa. SXT ou LHG? Por quê?
4. Sua borda BGP com a operadora receberá tabela cheia e crescerá para dois trânsitos. Por que um CCR1036 (36 núcleos TILE) pode ser pior que um CCR2004 (4 núcleos ARM64) nesse papel, mesmo tendo nove vezes mais núcleos?

Solução dos desafios

1. CRS326-24G-2S+: Cloud Router Switch com **24** portas **Gigabit** RJ45 e **2** gaiolas **SFP+** (10 Gbps). CCR2116-12G-4S+: Cloud Core Router com **12** portas **Gigabit** e **4** gaiolas **SFP+**.
2. Papel de switch de POP → família **CRS**. O CRS326-24G-2S+ do desafio 1 serve

como uma luva: 24 portas Gigabit para os clientes (sobram 4) e 2 uplinks SFP+ de 10 Gbps para o núcleo.

3. LHG. A 12 km, o que decide é o ganho da antena: a grade parabólica do LHG concentra o feixe e entrega sinal utilizável onde o SXT (feixe mais aberto, ganho menor) já não alcança com qualidade. O SXT é para distâncias curtas/médias; acima de ~8–10 km, LHG (ou DISC).

4. Processar atualizações de BGP e recalculando a tabela de rotas são tarefas que rodam em **um núcleo** — o que importa é o desempenho por núcleo, e cada núcleo ARM64 do CCR2004 é muito mais rápido que um núcleo TILE de 2012. Os 36 núcleos do CCR1036 só ajudariam se a carga se espalhasse por eles, o que não é o caso do BGP.

2.6 Resumo

- O nome do produto descreve as portas: **G** (Gigabit), **S** (SFP), **S+** (SFP+ 10G), **XS** (25G), **P** (PoE de saída) — decifre o nome antes de ler a ficha (Tabela 2.1).
- Famílias por papel (Figura 2.1): **CCR** no núcleo (borda BGP, concentrador), **CRS** como switch de POP, **hEX/hAP** na casa do cliente, **RB5009/L009** em empresas, **SXT/LHG/DISC/Wire** no rádio, **BaseBox + mANT setorial** na torre.
- **CSS CRS**: modelos CSS rodam apenas SwOS.
- CCR geração 1 (TILE) tem muitos núcleos lentos; geração 2 (ARM64) tem poucos núcleos rápidos — para PPPoE e BGP, vence o desempenho **por núcleo**.
- Avalie throughput sempre no teste **com regras de firewall**, nunca no número de capa (*fast path*).
- O **CHR** é membro pleno do catálogo: RouterOS em VM, usado em produção como concentrador e borda — e como nosso laboratório.

No próximo capítulo, veremos o que dá o “direito de rodar” ao RouterOS — os níveis de licença — e o RouterBOOT, o bootloader que você acabou de procurar com `/system routerboard print`.

Bibliografia do capítulo

O catálogo oficial (MikroTik, 2026c) e as fichas técnicas de cada produto são a fonte primária deste capítulo — em especial as tabelas *Test results*, cuja leitura crítica discutimos. A documentação do RouterOS (MikroTik, 2026a) descreve os menus `/system resource` e `/system routerboard`. Sobre arquiteturas de hardware de rede em geral, Tanenbaum; Feamster; Wetherall (2021) é a referência teórica. As referências completas, em ABNT, estão no apêndice [Referências](#).

3 Licenças RouterOS e RouterBOOT

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar como funciona o licenciamento do RouterOS: níveis 0–6, o que cada um libera e por que a licença acompanha o equipamento para sempre;
- Diferenciar as licenças de equipamento físico das licenças do CHR (**free**, **p1**, **p10**, **p-unlimited**) e da modalidade de teste de 60 dias;
- Consultar a licença de um equipamento com `/system license print`;
- Explicar o que é o RouterBOOT, o que ele faz no boot e por que mantê-lo atualizado junto com o RouterOS;
- Conhecer as proteções do RouterBOOT (senha, **protected-routerboot**) e os riscos de usá-las sem critério.

3.1 Licença: o que você compra junto com a caixa

Uma dúvida comum de quem chega ao MikroTik vindo de outras marcas: “quanto custa a assinatura do sistema?”. A resposta surpreende: **nada**. O modelo da MikroTik não tem mensalidade, contrato de suporte obrigatório nem licença por recurso. Cada equipamento RouterBOARD sai de fábrica com uma **licença perpétua** de um certo **nível**, embutida no preço — e essa licença:

- **nunca expira**;
- inclui **todas as atualizações** do RouterOS dentro da mesma major version (e, na prática, a MikroTik liberou até hoje as trocas de major — v5→v6→v7 — sem custo);
- pertence ao **equipamento**, não a você: se o roteador for vendido, a licença vai junto.

O que os níveis diferenciam não é “recurso” — firewall, BGP, MPLS e scripts existem em todos —, mas **capacidade de alguns serviços**, principalmente quantos túneis e sessões o equipamento pode servir e para que uso o wireless está liberado (1).

Tabela 3.1: O que muda entre os níveis de licença mais comuns (*como servidor; como cliente, ilimitado)

	Nível 3 (CPE)	Nível 4 (WISP)	Nível 5 (WISP AP)	Nível 6 (Controller)
Wireless cliente (station)	sim	sim	sim	sim
Wireless AP (ponto de acesso)	—	sim	sim	sim
Túneis PP-PoE/PPTP/L2TP/OVPN ativos	1*	200	500	ilimitado
Túneis EoIP	1*	200	500	ilimitado
Sessões hotspot ativas	1	200	500	ilimitado
Interfaces de usuário (vlan, queues...)	ilimitado	ilimitado	ilimitado	ilimitado

Os níveis que faltam na tabela são triviais: **nível 0** é o modo de demonstração de 24 horas de uso (o que você obtém instalando o RouterOS x86 sem licença); **nível 1** é a demo registrada, quase inútil hoje; **nível 2** não existe mais (era das versões antigas).

Na prática do provedor, você raramente escolhe nível — ele vem casado com o hardware:

- Equipamentos de **cliente** (SXT, LHG, hEX, hAP) vêm com **nível 3 ou 4**;
- Equipamentos de **torre e POP** (BaseBox, NetMetal, RB4011/5009) vêm com **nível 4 ou 5**;
- Equipamentos de **núcleo** (CCR) vêm com **nível 6**.

Repare como os limites casam com os papéis do 2: o nível 3 não sobe AP wireless (é licença de *station*, de rádio de cliente); o nível 6 não limita túneis (é licença de concentrador PPPoE). A tabela de níveis é o catálogo de papéis visto por outro ângulo.

💡 Analogia para guardar

A licença do RouterOS é como a carta de alforria gravada no chassi de um carro: paga uma vez, vale para sempre e vai junto na venda. Os níveis são como categorias da CNH — todas dirigem, mas cada uma autoriza um porte de veículo (aqui: uma quantidade de sessões e um papel wireless).

3.1.1 A licença do CHR: outro modelo

O CHR usa um licenciamento próprio, por **velocidade de interface** — e é por isso que ele é o laboratório perfeito:

Tabela 3.2: Licenças do CHR

Licença CHR	Limite por interface	Custo
free	1 Mbps	gratuito, para sempre
p1	1 Gbps	pago (perpétuo)
p10	10 Gbps	pago (perpétuo)
p-unlimited	ilimitado	pago (perpétuo)

Além dessas, todo CHR novo pode ativar um **trial de 60 dias** de qualquer nível pago (`/system license renew`, com uma conta gratuita do site da MikroTik) — útil para testar desempenho real antes de comprar. Para os laboratórios deste livro, o **free** com seu 1 Mbps por interface é mais que suficiente: estudamos comportamento, não velocidade.

Consultar a licença é um comando só, e a saída é diferente em cada mundo. No CHR:

```
/system license print
```

```
[admin@lab-r1] > /system license print
system-id: hK3x-Wm9P
level: free
```

Num equipamento físico apareceriam **software-id**, **nlevel** (o nível 3–6) e **features**. O **system-id/software-id** identifica a instalação — é ele que você informa à MikroTik ao comprar upgrade de licença.

3.2 RouterBOOT: o que acorda o RouterOS

Todo computador tem um programa minúsculo que roda antes do sistema operacional — no PC, o firmware UEFI/BIOS; nas RouterBOARD, o **RouterBOOT**. Quando você liga um MikroTik físico, é o RouterBOOT que:

1. inicializa CPU e memória;
2. decide **de onde dar o boot** — do armazenamento interno (normal) ou da rede (modo de recuperação usado pelo Netinstall, Capítulo 8);
3. carrega o RouterOS e lhe entrega o controle.

O RouterBOOT tem firmware próprio, com versão independente do RouterOS — e aqui mora uma tarefa de manutenção que muitos esquecem. Ao atualizar o RouterOS, a versão nova do bootloader vem junta no pacote, mas **só é gravada quando você manda** e reinicia. O fluxo completo:

```
/system routerboard print
```

```
[admin@MikroTik] > /system routerboard print
routerboard: yes
board-name: hEX S
model: RB760iGS
serial-number: HEA08XXXXXX
firmware-type: mt7621L
factory-firmware: 6.46.3
current-firmware: 6.49.10
upgrade-firmware: 7.15.3
```

Quando `current-firmware` está atrás de `upgrade-firmware`, aplique:

```
/system routerboard upgrade
/system reboot
```

Manter RouterOS e RouterBOOT na mesma geração evita uma classe inteira de problemas — de suporte a hardware a correções de segurança do próprio boot. No CHR não existe nada disso (`not routerboard`): quem inicializa a VM é o “BIOS” do hipervisor.

3.2.1 As proteções do RouterBOOT

Por padrão, qualquer pessoa com acesso físico a uma RouterBOARD manda nela: basta segurar o botão de reset ao ligar para limpar a configuração, ou inicializar pela rede e reinstalar o sistema pelo Netinstall. Para equipamento em poste, torre ou armário compartilhado, o RouterBOOT oferece camadas de defesa em `/system routerboard settings`:

- `boot-device=flash-boot-once-etherboot` mantém o boot normal mas ainda permite recuperação; já `boot-protocol` e afins controlam o boot por rede;
- **senha do bootloader** (`/system routerboard settings set boot-protocol=... protected-routerboot=...`) — as opções variam por modelo;
- **`protected-routerboot=enabled`** — o modo forte: desabilita o botão de reset e o boot por rede. O equipamento só reinstala/reseta com uma intervenção muito mais trabalhosa.

Aviso

`protected-routerboot=enabled` corta justamente os mecanismos de **recuperação**: sem botão de reset e sem Netinstall, uma senha esquecida transforma o equipamento num peso de papel até você conseguir desativar a proteção — o que exige acesso admin ao RouterOS. Só ative em equipamento exposto a acesso físico de terceiros, com a senha do admin guardada em gerenciador de senhas do provedor, e **documente** quais equipamentos do parque têm a proteção ligada.

O conceito mais importante do capítulo

No MikroTik, o custo do software é **zero ao longo da vida** do equipamento: licença perpétua, atualizações incluídas. O que os níveis limitam é capacidade de sessões/túneis e o papel wireless — mais um motivo para escolher hardware pelo papel, como no 2. E abaixo do RouterOS vive o RouterBOOT: atualize-o junto com o sistema e proteja-o com critério.

3.3 Laboratório

3.3.1 Ambiente

A VM única do CHR no VirtualBox, como nos capítulos anteriores. (A partir do Capítulo 5 montaremos topologias maiores.)

3.3.2 Comandos passo a passo

Veja a licença do seu CHR:

```
/system license print
```

Saída esperada:

```
[admin@lab-r1] > /system license print
    system-id: hK3x-Wm9P
    level: free
```

`level: free` confirma a licença gratuita — cada interface limitada a 1 Mbps, sem prazo de validade. Agora confirme que o CHR não tem RouterBOOT:

```
/system routerboard print
```

```
[admin@lab-r1] > /system routerboard print
routerboard: no
```

Por fim, explore o menu de histórico do sistema, que registra mudanças de configuração — ele nos ajudará a auditar o que os laboratórios alteram:

```
/system history print
```

A saída lista as últimas ações (como o `set name=lab-r1` do primeiro capítulo), com autor e horário.

3.3.3 Verificação

1. `/system license print` mostra `level: free`;
2. `/system routerboard print` responde `routerboard: no` — logo, nada de RouterBOOT para gerenciar no laboratório;
3. `/system history print` registra as alterações que você fez até aqui.

3.3.4 Desafios

1. Um provedor compra um **hAP ax²** (nível 4) para a casa de um assinante e um **CCR2116** (nível 6) para concentrador PPPoE. Algum dos dois precisará de upgrade de licença para exercer seu papel? Justifique com a
 - 1.
2. No site da MikroTik, cada produto informa o nível de licença na ficha técnica. Verifique o nível do **SXT SA5** e explique por que ele **não** poderia ser usado como AP de uma torre PtMP com essa licença.
3. Seu parque tem 40 rádios em torres de difícil acesso. Monte (em texto) a política de proteção do RouterBOOT: o que ativar, o que documentar e qual o plano B se a senha do admin de um rádio se perder.
4. Sem executar ainda: qual seria o efeito de `/system license renew` num CHR novo? Quando isso seria útil na vida real?

Solução dos desafios

1. Nenhum dos dois. O hAP ax² atenderá **uma** casa: o nível 4 permite AP wireless e até 200 túneis — décadas de folga para um CPE. O CCR2116 como concentrador precisará de **milhares** de sessões PPPoE, e o nível 6 é justamente o ilimitado. A MikroTik casa o nível com o papel de fábrica.
2. O SXT SA5 vem com **nível 3** — licença de *station*: pode conectar-se a um AP, mas não pode **ser** AP (a linha “wireless AP” da 1 é “—” no nível 3). Para a torre PtMP usa-se um equipamento nível 4+ (ex.: BaseBox com mANT setorial), e os SXT ficam

nos clientes. (Existe a variante “SXT SA5 ap”, vendida com nível 4, exatamente para isso.)

3. Política razoável: (a) ativar `protected-routerboot=enabled` apenas nos rádios fisicamente expostos; (b) senha de admin única por equipamento, guardada no gerenciador de senhas do provedor; (c) registrar em inventário quais equipamentos têm a proteção; (d) plano B: com a senha do admin perdida e proteção ativa, resta o procedimento de desbloqueio de fábrica — na prática, RMA. Por isso a proteção só entra onde o risco de acesso físico supera o risco de perda de senha.

4. `/system license renew` ativa o **trial de 60 dias** de um nível pago (`p1/p10/p-unlimited`), pedindo login de uma conta MikroTik. Útil para testar um CHR de produção — por exemplo, medir se um servidor aguenta o papel de concentrador PPPoE a 10 Gbps — antes de pagar a licença perpétua.

3.4 Resumo

- Licença RouterOS: **perpétua**, sem mensalidade, com atualizações incluídas; pertence ao equipamento.
- Níveis 3–6 diferenciam **papel wireless** (station/AP) e **quantidade de túneis/sessões** (200/500/ilimitado) — não recursos (1).
- CHR licencia por **velocidade de interface**: **free** (1 Mbps) para laboratório, `p1/p10/p-unlimited` para produção, trial de 60 dias (Tabela 3.2).
- **RouterBOOT** é o bootloader das RouterBOARD: atualize-o junto com o RouterOS (`/system routerboard upgrade + reboot`).
- Proteções de boot (`protected-routerboot`) defendem equipamento exposto — ao custo de eliminar os caminhos de recuperação. Use com critério e documentação.

No próximo capítulo, deixamos a caixa e vamos às portas de entrada: WinBox, WebFig, SSH, serial e MAC-Telnet — todas as formas de acessar um RouterOS, e quando usar cada uma.

Bibliografia do capítulo

A página oficial de licenciamento do RouterOS e a documentação do CHR (MikroTik, 2026b) detalham níveis e preços atuais; a documentação do RouterOS (MikroTik, 2026a) cobre os menus `/system license` e `/system routerboard` (incluindo o `protected-routerboot`). O catálogo (MikroTik, 2026c) informa o nível de licença de cada produto na ficha técnica. As referências completas, em ABNT, estão no apêndice [Referências](#).

4 Formas de acesso ao RouterOS

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Listar as formas de acesso ao RouterOS — WinBox, WebFig, SSH, Telnet, console serial e MAC-Telnet — e escolher a adequada a cada situação;
- Explicar a diferença entre acesso **por IP** (camada 3) e acesso **por MAC** (camada 2), e por que o segundo salva o dia quando o IP está errado;
- Usar o **Safe Mode**, o cinto de segurança que desfaz alterações quando você perde o próprio acesso;
- Acessar seu CHR de laboratório pelo WinBox, pelo WebFig e por SSH;
- Reconhecer os serviços de acesso em `/ip service` e entender por que eles serão restringidos no capítulo de segurança.

4.1 Muitas portas para a mesma casa

Tudo o que fizemos até aqui usou o **console da máquina virtual** — o equivalente a sentar na frente do equipamento com teclado e monitor. Na vida real de provedor, o roteador está num rack, num poste ou numa torre a 40 km, e você precisa entrar nele pela rede. O RouterOS oferece várias portas de entrada, e a escolha certa depende de duas perguntas:

1. **O equipamento tem IP acessível?** Se sim, qualquer forma serve; se não (equipamento novo, configuração quebrada), você precisa das formas que funcionam em **camada 2**, direto pelo cabo.
2. **Você quer interface gráfica ou linha de comando?** A gráfica é confortável para explorar; a linha de comando é precisa, documentável e automatizável — e é o padrão deste manual.

A Figura 4.1 organiza as opções nesses dois eixos:

Vamos conhecer uma a uma, começando pelas que você mais usará.

4.1.1 WinBox — a ferramenta símbolo

O **WinBox** é o aplicativo gráfico de gerência da MikroTik — tão associado à marca que muita gente “configura pelo WinBox” sem saber o nome do sistema que está configurando.

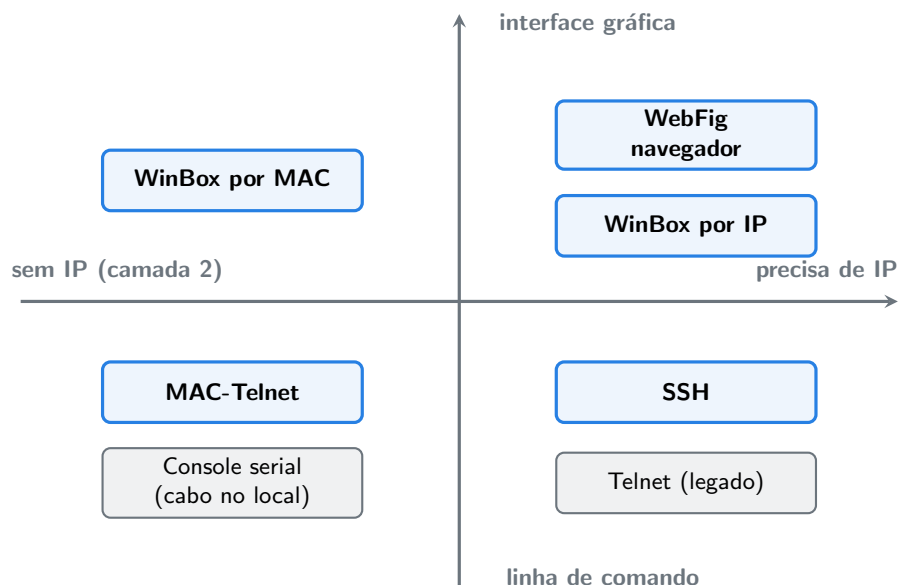


Figura 4.1: As portas de entrada do RouterOS, organizadas por tipo de interface e por dependência de IP.

Ele conversa com o roteador pela porta **8291/TCP** e hoje tem versão nativa para Windows, macOS e Linux.

Três características o tornam a ferramenta de campo por excelência:

- **Descoberta de vizinhos:** a tela inicial lista os equipamentos MikroTik alcançáveis no cabo (via protocolo de descoberta), mesmo os sem IP;
- **Conexão por MAC:** dê um clique no endereço MAC da lista e você entra num equipamento que não tem IP nenhum — mágica de camada 2 que veremos adiante;
- **Espelho da CLI:** cada janela do WinBox corresponde a um menu do terminal (IP → Addresses é `/ip address`). Aprender um é aprender o outro — e é por isso que este manual pode ensinar por linha de comando sem abandonar quem prefere clicar.

4.1.2 WebFig — o WinBox no navegador

O **WebFig** é a interface web embutida: aponte o navegador para o IP do roteador (porta **80/HTTP** ou **443/HTTPS**) e você tem praticamente as mesmas telas do WinBox, sem instalar nada. É a saída natural quando você está numa máquina emprestada ou num sistema onde não pode instalar aplicativos. Nos equipamentos com configuração de fábrica, é o WebFig que recebe o usuário no modo simplificado “Quick Set”.

4.1.3 SSH — a porta da linha de comando

O **SSH** (porta **22/TCP**) entrega o mesmo terminal que você usou no console da VM, com criptografia, a partir de qualquer sistema operacional:

```
ssh admin@192.168.88.1
```

É a forma preferida deste manual para o dia a dia, por três motivos: é universal (todo sistema tem cliente SSH), é **automatizável** (scripts, Ansible e afins entram por aqui — Volume 12) e registra bem o que foi feito (seu terminal guarda o histórico da sessão). O RouterOS também aceita **SFTP/SCP** na mesma porta, para copiar arquivos — algo que usaremos nos backups (3).

O primo velho do SSH, o **Telnet** (porta 23), também existe — sem criptografia alguma. Trate-o como peça de museu: útil apenas em redes de laboratório isoladas, e um dos primeiros serviços que desligaremos no capítulo de segurança.

4.1.4 Console serial — o cabo da UTI

Roteadores de rack (CCR, alguns CRS) trazem uma porta **serial** (RJ45 ou USB). Ligando um cabo console, você acessa o terminal **antes mesmo do RouterOS subir** — vê o boot, o RouterBOOT (Capítulo 3) e o sistema, mesmo com a rede completamente quebrada. É o acesso de UTI: usado raramente, mas insubstituível quando o equipamento não responde por nenhuma outra via e você está fisicamente diante dele.

4.1.5 MAC-Telnet — terminal sem IP

E se o equipamento não tem IP, você não tem o WinBox à mão, mas há **outro MikroTik** no mesmo cabo? O **MAC-Telnet** resolve: de um RouterOS para outro, abre-se um terminal usando só o endereço MAC, em camada 2:

```
/tool mac-telnet 48:A9:8A:12:34:56
```

O RouterOS descobre os vizinhos MikroTik (e seus MACs) sozinho:

```
/ip neighbor print
```

Esse par — descoberta de vizinhos + MAC-Telnet — é o kit de resgate do provedor: o técnico conecta o notebook (ou usa o roteador de cima da torre) e entra no rádio “perdido” sem depender de IP certo. A contrapartida de segurança é evidente: qualquer um no cabo pode tentar o mesmo. Como restringir isso é assunto do 5.

4.2 Safe Mode: o cinto de segurança

Antes do laboratório, a ferramenta mais importante do capítulo. Imagine: você está acessando um rádio de torre **pela própria rede que ele roteia** e desativa, sem querer, a interface errada. A conexão cai — e com ela, seu acesso e o dos clientes. Viagem de 40 km.

O **Safe Mode** existe para impedir exatamente isso. Ao ativá-lo (tecla **Ctrl+X** no terminal; botão *Safe Mode* no WinBox/WebFig), o RouterOS passa a tratar suas alterações como **provisórias**:

- Se você **sair do Safe Mode normalmente** (**Ctrl+X** de novo), as mudanças são confirmadas;
- Se a **conexão cair** com o Safe Mode ativo — inclusive porque sua mudança derrubou o próprio acesso —, o RouterOS **desfaz tudo** o que foi feito desde a ativação, e alguns segundos depois você entra de novo como se nada tivesse acontecido.

No terminal, o modo ativo aparece no prompt:

```
[admin@lab-r1] <SAFE>
```

! O conceito mais importante do capítulo

Toda alteração remota que possa afetar o seu próprio caminho até o equipamento — interfaces, IPs, rotas, firewall — deve ser feita **dentro do Safe Mode**. **Ctrl+X** antes, mexa, confirme com **Ctrl+X** depois. Esse hábito, sozinho, paga o livro: ele elimina a viagem até a torre.

4.3 Os serviços em /ip service

Cada forma de acesso por IP corresponde a um **serviço** que o RouterOS mantém ouvindo, listado em `/ip service`:

Tabela 4.1: Serviços de acesso do RouterOS

Serviço	Porta padrão	Forma de acesso
winbox	8291/TCP	WinBox
www	80/TCP	WebFig (HTTP)
www-ssl	443/TCP	WebFig (HTTPS)
ssh	22/TCP	SSH / SFTP
telnet	23/TCP	Telnet (legado)
ftp	21/TCP	FTP (legado)
api, api-ssl	8728, 8729/TCP	API de automação (Volume 12)

Guarde a existência dessa tabela: no 5 ela vira uma lista de decisões — o que desabilitar, o que restringir por endereço de origem (`allowed-address`) e o que mover de porta. Um roteador de provedor exposto à internet com todos esses serviços abertos é convite para invasão.

4.4 Laboratório

4.4.1 Ambiente

A VM do CHR no VirtualBox — agora com um ajuste de rede que permitirá ao **seu computador** conversar com ela. Com a VM **desligada** (`/system shutdown`):

1. No VirtualBox, abra as **configurações da VM** → **Rede**;
2. No **Adaptador 1**, troque o modo para **Placa de rede exclusiva de hospedeiro** (*host-only adapter*), selecionando a rede padrão (VirtualBox Host-Only Ethernet Adapter, tipicamente 192.168.56.0/24);
3. Inicie a VM.

Assim, o CHR (`ether1`) e o seu computador ficam ligados num “cabo virtual” — exatamente o cenário de um técnico com notebook conectado ao equipamento.

4.4.2 Comandos passo a passo

No **console da VM**, dê um endereço IP à `ether1` (a sintaxe de endereços é assunto do Volume 2 — por ora, digite e observe):

```
/ip address add address=192.168.56.10/24 interface=ether1
```

Confirme:

```
/ip address print
```

```
[admin@lab-r1] > /ip address print
Columns: ADDRESS, NETWORK, INTERFACE
# ADDRESS          NETWORK          INTERFACE
0 192.168.56.10/24  192.168.56.0    ether1
```

Acesso 1 — WebFig. No navegador do seu computador, abra `http://192.168.56.10`. Entre com `admin` e a senha que você definiu no primeiro boot. Explore: localize em **IP** → **Addresses** o endereço que você acabou de criar pelo terminal.

Acesso 2 — SSH. No terminal do seu computador (PowerShell, Terminal do macOS ou shell Linux — todos têm o cliente):

```
ssh admin@192.168.56.10
```

Aceite a chave do servidor (**yes**), informe a senha e observe: é o mesmo prompt `[admin@lab-r1] >` do console. Execute `/system resource print` para provar que tudo que você aprendeu vale aqui.

Acesso 3 — WinBox. Baixe o WinBox em mikrotik.com/download (seção *WinBox*). Ao abrir, a aba **Neighbors** deve listar seu CHR. Conecte duas vezes: primeiro clicando no **IP**, depois clicando no **endereço MAC** — esta segunda é uma conexão de camada 2, que funcionaria mesmo sem o endereço IP configurado.

Acesso 4 — Safe Mode (o exercício mais importante). Na sessão SSH, pressione **Ctrl+X**. O prompt ganha o marcador `<SAFE>`. Agora derrube o próprio acesso de propósito, removendo o endereço IP:

```
/ip address remove numbers=0
```

Sua sessão SSH congela e cai — você “se trancou para fora”. Espere cerca de 15 segundos e conecte de novo por SSH: funciona, porque o Safe Mode detectou a queda da sessão e **desfez a remoção**. Confirme com `/ip address print` que o endereço voltou.

4.4.3 Verificação

1. WebFig abre em `http://192.168.56.10` e mostra o endereço em IP → Addresses;
2. SSH conecta e entrega o prompt `[admin@lab-r1] >`;
3. O WinBox lista o CHR em Neighbors e conecta por IP e por MAC;
4. Após o teste do Safe Mode, `/ip address print` mostra o item 0 restaurado — a prova de que a reversão automática funcionou.

4.4.4 Desafios

1. Liste os serviços ativos do seu CHR com `/ip service print`. Quais das quatro conexões deste laboratório passaram por qual serviço?
2. Ainda em `/ip service`: descubra (com TAB e ?) como **desabilitar** o serviço `telnet` — e desabilite-o. Por que essa é uma boa ideia mesmo em laboratório?
3. Repita a remoção do endereço IP **sem** Safe Mode. O que muda? Como você recupera o acesso desta vez? (Dica: você tem o console da VM — e o WinBox por MAC.)
4. Em produção, um técnico precisa configurar um rádio novo, sem IP, no topo da torre, a partir do POP lá embaixo (há um MikroTik no POP, no mesmo cabo). Descreva o caminho de acesso completo, citando as ferramentas deste capítulo.

💡 Solução dos desafios

1. `/ip service print` lista a Tabela 4.1. WebFig usou `www` (porta 80), SSH usou `ssh` (22), o WinBox por IP usou `winbox` (8291) — e o WinBox **por MAC** não passou por nenhum serviço IP: é camada 2.

2.

```
/ip service disable telnet
/ip service print
```

O Telnet transmite tudo — inclusive a senha — em texto puro. Desabilitar o que não se usa reduz a superfície de ataque; o hábito vale até em laboratório, para virar reflexo.

3. Sem Safe Mode, a remoção é **definitiva**: o SSH cai e não volta, pois o endereço se foi de verdade. A recuperação: entrar pelo **console da VM** (ou WinBox pelo **MAC**) e recriar o endereço com `/ip address add address=192.168.56.10/24 interface=ether1`. É a experiência que ensina o valor do `Ctrl+X`.

4. No MikroTik do POP: `/ip neighbor print` para descobrir o MAC do rádio novo → `/tool mac-telnet <MAC>` para abrir o terminal em camada 2 → dentro da sessão, configurar IP de gerência no rádio → a partir daí, seguir por SSH/WinBox normalmente (e, sendo mudança arriscada em rádio de torre, tudo dentro do Safe Mode).

4.5 Resumo

- Formas de acesso por **IP**: WinBox (8291), WebFig (80/443), SSH (22) — e Telnet/FTP como legados a desativar (Tabela 4.1).
- Formas de acesso por **camada 2** (sem IP): WinBox por MAC e MAC-Telnet (entre RouterOS), com descoberta de vizinhos por `/ip neighbor print`; console serial como acesso físico de emergência (Figura 4.1).
- **WinBox espelha a CLI**: cada janela é um menu do terminal — aprender um é aprender os dois.
- **Safe Mode (Ctrl+X)**: alterações viram provisórias e são desfeitas se a sua sessão cair — obrigatório em qualquer mudança remota que possa derrubar o próprio acesso.
- Cada acesso por IP é um **serviço** em `/ip service`, que será restringido no capítulo de segurança inicial.

No próximo capítulo, montamos de uma vez o laboratório definitivo do livro: múltiplos CHRs em topologia, com acesso confortável do seu computador — o ambiente que reutilizaremos até a última página.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) descreve cada ferramenta nas seções *WinBox*, *WebFig* e *MAC server* (MAC-Telnet), além do menu `/ip service`. O download do WinBox

e do cliente para cada plataforma está em mikrotik.com/download. Discher (2016) dedica bons capítulos iniciais às interfaces de gerência. As referências completas, em ABNT, estão no apêndice [Referências](#).

5 Montando o laboratório com CHR

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Montar, no VirtualBox, o **laboratório padrão do livro**: três CHRs (**lab-r1**, **lab-r2**, **lab-r3**) com rede de gerência fixa e redes internas para as topologias;
- Explicar o papel de cada tipo de rede do VirtualBox no laboratório — *host-only* (gerência), *rede interna* (enlaces entre roteadores) e *NAT* (saída para a internet);
- Clonar VMs e usar **snapshots** para começar cada laboratório limpo;
- Acessar qualquer roteador do laboratório por SSH e WinBox a partir do seu computador, pela rede de gerência;
- Reconhecer quando vale migrar para GNS3 ou EVE-NG — e por que o VirtualBox basta para este livro.

5.1 Um laboratório para o livro inteiro

Este é o capítulo mais rentável do manual: o que você montar aqui será reutilizado em **todos** os laboratórios até a última página. A seção “Ambiente” dos próximos capítulos dirá apenas coisas como “2 CHRs em rede interna (Capítulo 5)” — e você já saberá exatamente o que ligar.

O desenho atende três exigências que acompanham qualquer laboratório de redes:

1. **Gerência estável**: você precisa alcançar os roteadores por SSH/WinBox do seu computador **sempre**, mesmo quando o exercício quebra a rede entre eles. A solução é uma rede de gerência separada, que os exercícios nunca tocam.
2. **Enlaces isolados**: os “cabos” entre roteadores devem ser redes que só existem entre eles, sem interferência do seu computador ou da sua rede doméstica.
3. **Reset rápido**: errar faz parte; recomeçar do zero não pode custar uma reinstalação. Snapshots resolvem.

A topologia-base fica assim (Figura 5.1):

O mapa fixo de interfaces e endereços — decore ou marque esta página:

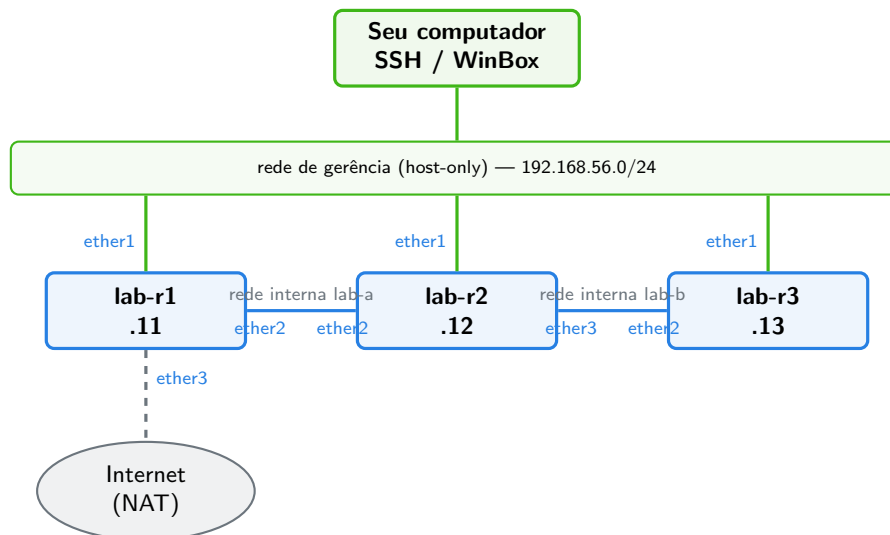


Figura 5.1: A topologia-base do laboratório: gerência host-only fixa (verde), enlaces por redes internas (azul) e saída opcional para a internet via NAT em **lab-r1** (tracejado).

Tabela 5.1: Mapa de interfaces do laboratório padrão

VM	ether1 (gerência)	ether2	ether3
lab-r1	host-only — 192.168.56.11	rede interna lab-a	NAT (internet)
lab-r2	host-only — 192.168.56.12	rede interna lab-a	rede interna lab-b
lab-r3	host-only — 192.168.56.13	rede interna lab-b	—

💡 Analogia para guardar

A rede de gerência é o **corredor de serviço** de um hotel: os hóspedes (o tráfego dos exercícios) circulam pelos corredores normais (**lab-a**, **lab-b**), enquanto a manutenção (você, por SSH) tem um corredor próprio que nunca fecha — mesmo com um quarto em obras.

5.1.1 Os três tipos de rede do VirtualBox

O VirtualBox oferece vários modos de rede; o laboratório usa três, cada um num papel:

- **Host-only** — cria uma rede virtual entre o seu computador e as VMs. Perfeita para gerência: o hospedeiro alcança todas as VMs, e nada disso vaza para sua rede doméstica.

- **Rede interna** (*internal network*) — um “switch virtual” nomeado que só conecta VMs que apontem para o mesmo nome (**lab-a**, **lab-b**). É o cabo entre roteadores: seu computador não participa.
- **NAT** — a VM sai para a internet através do seu computador, como um celular no modo roteador. Usamos apenas na **ether3** do **lab-r1**, nos capítulos que precisam de internet de verdade (atualizações, DNS, NTP).

5.2 GNS3 e EVE-NG: quando crescer

GNS3 e **EVE-NG** são plataformas de emulação de redes completas: desenho de topologia por arrastar-e-soltar, dezenas de nós, imagens de vários fabricantes. Ambas rodam o CHR muito bem, e você encontrará laboratórios MikroTik prontos para elas na internet.

Este livro, porém, padroniza no **VirtualBox** por três razões: é mais leve (roda em qualquer máquina de estudo), não exige imagens adicionais nem servidor dedicado, e três roteadores cobrem quase todos os exercícios — nos poucos capítulos que pedem um quarto nó (o estudo de caso do Volume 15, por exemplo), o padrão de clonagem que você aprenderá aqui resolve em minutos. Se você já domina GNS3/EVE-NG, use-os sem medo: os laboratórios descrevem **topologias**, não cliques — basta reproduzir as redes da Tabela 5.1.

5.3 Laboratório

Este capítulo é um grande laboratório: vamos montar o ambiente definitivo, do zero ao snapshot.

5.3.1 Ambiente

- VirtualBox instalado (o mesmo do Capítulo 1);
- A imagem **.vdi** do CHR baixada de mikrotik.com/download;
- A VM avulsa dos capítulos anteriores pode ser **removida** ao final — o laboratório novo a substitui.

5.3.2 Comandos passo a passo

Parte 1 — preparar a rede host-only. No VirtualBox, menu **Arquivo** → **Ferramentas** → **Gerenciador de Rede** (*Network Manager*): confirme que existe um adaptador host-only com IPv4 **192.168.56.1/24** (é o padrão; crie-o se não existir) e **desative o servidor DHCP** dele — os IPs do laboratório são fixos, e um DHCP intrometido só atrapalha.

Parte 2 — criar a VM-molde. Crie uma VM chamada **lab-r1**, como no primeiro capítulo (Linux / Other Linux 64-bit, 256 MB de RAM), usando uma **cópia** do **.vdi** do CHR como disco. Antes de ligar, em **Configurações** → **Rede**:

1. **Adaptador 1:** habilitado, modo *Placa exclusiva de hospedeiro* (host-only) — será a `ether1`;
2. **Adaptador 2:** habilitado, modo *Rede interna*, nome `lab-a` — será a `ether2`;
3. **Adaptador 3:** habilitado, modo *NAT* — será a `ether3`.

Ligue a VM, faça o primeiro login (`admin`, senha em branco, defina a nova senha) e configure identidade e IP de gerência:

```
/system identity set name=lab-r1
/ip address add address=192.168.56.11/24 interface=ether1 comment=gerencia
```

Do **seu computador**, confirme o acesso antes de continuar:

```
ssh admin@192.168.56.11
```

Parte 3 — clonar. Desligue a VM (`/system shutdown`). No VirtualBox, clique com o botão direito em `lab-r1` → **Clonar**: nome `lab-r2`, tipo **clone completo**, e marque a opção de **gerar novos endereços MAC** para todas as placas (fundamental — MACs duplicados criam defeitos enlouquecedores). Repita para criar `lab-r3`.

Ajuste as redes dos clones conforme a Tabela 5.1:

- `lab-r2`: Adaptador 2 → rede interna `lab-a`; Adaptador 3 → rede interna `lab-b`;
- `lab-r3`: Adaptador 2 → rede interna `lab-b`; Adaptador 3 → **desabilitado**.

Ligue `lab-r2`, e pelo **console** corrija identidade e IP (o clone veio com os dados do molde):

```
/system identity set name=lab-r2
/ip address set numbers=0 address=192.168.56.12/24
```

O mesmo em `lab-r3`:

```
/system identity set name=lab-r3
/ip address set numbers=0 address=192.168.56.13/24
```

Parte 4 — testar os enlaces internos. Com as três VMs ligadas, dê endereços provisórios ao enlace `lab-a` para prová-lo. Em `lab-r1`:

```
/ip address add address=10.0.12.1/30 interface=ether2 comment=teste-lab-a
```

Em `lab-r2`:

```
/ip address add address=10.0.12.2/30 interface=ether2 comment=teste-lab-a
```

E, ainda de `lab-r2`, pingue o vizinho:

```
/ping 10.0.12.1 count=4
```

```
[admin@lab-r2] > /ping 10.0.12.1 count=4
SEQ HOST      SIZE TTL TIME      STATUS
  0 10.0.12.1   56  64 1ms234us
  1 10.0.12.1   56  64 987us
  2 10.0.12.1   56  64 1ms102us
  3 10.0.12.1   56  64 995us
sent=4 received=4 packet-loss=0%
```

`packet-loss=0%` prova o enlace. Remova os endereços de teste nos dois roteadores — os capítulos seguintes sempre começam com os enlaces “crus”:

```
/ip address remove [find comment=teste-lab-a]
```

Parte 5 — snapshot. Desligue as três VMs (`/system shutdown`). No VirtualBox, selecione cada uma → **Snapshots** → **Criar** — nomeie `base-limpa`. Este é o estado oficial de partida: identidade + IP de gerência, e nada mais.

5.3.3 Verificação

O checklist de “laboratório pronto”:

1. As três VMs existem, com os adaptadores exatamente como na Tabela 5.1;
2. Do seu computador, `ssh admin@192.168.56.11`, `.12` e `.13` entram nos três roteadores (com as VMs ligadas), e o prompt de cada um mostra a identidade certa;
3. O WinBox lista os três em **Neighbors** (Capítulo 4);
4. O ping da Parte 4 fechou com `packet-loss=0%` e os endereços de teste foram removidos;
5. Cada VM tem o snapshot `base-limpa`.

A partir de agora, “restaurar o laboratório” = desligar as VMs e restaurar o snapshot `base-limpa` de cada uma. Faça isso sempre que um capítulo pedir ambiente limpo — ou quando um experimento sair do controle.

5.3.4 Desafios

1. Sem olhar a tabela: qual rede interna conecta `lab-r2` a `lab-r3`, e por quais interfaces de cada um?
2. De `lab-r3`, encontre os vizinhos MikroTik com `/ip neighbor print`. Quem aparece, e por quê `lab-r1` (não) aparece?
3. O enlace `lab-b` ainda não foi testado. Repita a Parte 4 nele — desta vez escolhendo você mesmo a sub-rede de teste — e desfaça ao final.

4. Habilite temporariamente o adaptador NAT (**ether3**) do **lab-r1** e, no roteador, rode `/ip dhcp-client add interface=ether3` seguido de `/ip dhcp-client print`. O que aconteceu? (É um gostinho do Volume 2 — desfaça com `/ip dhcp-client remove numbers=0` ao terminar.)

Solução dos desafios

1. Rede interna **lab-b**, entre a **ether3** de **lab-r2** e a **ether2** de **lab-r3** (Tabela 5.1).
2. Aparecem **lab-r2** (vizinho direto pela **lab-b**) e — dependendo do modo do seu adaptador **host-only** — os demais pela rede de gerência, já que ela é um segmento comum a todos. Pela rede **interna**, **lab-r1** não é vizinho de **lab-r3**: eles não compartilham segmento (**lab-a** e **lab-b** são switches separados, e **lab-r2** está no meio).
3. Exemplo com 10.0.23.0/30: em **lab-r2**, `/ip address add address=10.0.23.1/30 interface=ether3`; em **lab-r3**, `/ip address add address=10.0.23.2/30 interface=ether2`; ping de um para o outro; remoção com `/ip address remove [find comment=...]` (se você usou comentário — bom hábito) ou pelo número do item.
4. O DHCP client pediu endereço ao NAT do VirtualBox e recebeu algo como 10.0.2.15/24 com gateway 10.0.2.2 — o roteador ganhou uma “WAN” e já alcança a internet (`/ping 8.8.8.8` funciona). É exatamente assim que o capítulo de DHCP montará a WAN do roteador de borda.

5.4 Resumo

- O laboratório padrão: **3 CHR**s no VirtualBox, gerência **host-only** fixa (192.168.56.11/.12/.13 na **ether1**), enlaces por **redes internas lab-a** e **lab-b**, NAT opcional na **ether3** de **lab-r1** (Figura 5.1, Tabela 5.1).
- Gerência separada dos exercícios = acesso que **nunca cai**, aconteça o que acontecer com a topologia.
- Clones completos com **MACs novos**; identidade e IP ajustados no console logo após clonar.
- Snapshot **base-limpa** em cada VM: restaurá-lo é o “reset de fábrica” do laboratório.
- GNS3/EVE-NG são upgrades válidos; os laboratórios do livro descrevem topologias e funcionam em qualquer plataforma.

No próximo capítulo, usamos o laboratório novo para o assunto que abre toda operação real: o primeiro acesso a um equipamento, a configuração de fábrica e a segurança inicial — usuários, senhas e serviços.

Bibliografia do capítulo

A documentação do CHR (MikroTik, 2026b) cobre instalação em VirtualBox, VMware, Proxmox e nuvens públicas; o manual do VirtualBox (em [virtualbox.org/manual](https://www.virtualbox.org/manual)) detalha

os modos de rede que usamos. Para laboratórios em GNS3/EVE-NG, a documentação de cada plataforma traz guias de importação do CHR. As referências completas, em ABNT, estão no apêndice [Referências](#).

6 Primeiro acesso, usuários e segurança inicial

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Descrever o que um equipamento MikroTik traz de fábrica — a *default configuration* — e decidir entre aproveitá-la ou partir do zero;
- Criar usuários com o **grupo** certo (**read**, **write**, **full**) e abolir o uso compartilhado do **admin**;
- Aplicar o checklist mínimo de segurança inicial: senha forte, serviços desnecessários desligados, acesso restrito por **allowed-address**, descoberta de vizinhos contida e MAC-server sob controle;
- Ajustar relógio e fuso (**/system clock**, NTP) — e explicar por que hora certa é item de **segurança**;
- Deixar qualquer equipamento novo em estado “apresentável” em cinco minutos, seguindo sempre a mesma sequência.

6.1 O que vem de fábrica

Ligue um hAP novo e ele já funciona: distribui IPs na LAN, faz NAT para a porta WAN, tem firewall básico e WiFi com senha na etiqueta. Essa é a **configuração padrão** (*default configuration*), pensada para o roteador doméstico — e ela é boa nesse papel.

No mundo do provedor, porém, a configuração de fábrica é quase sempre **ponto de partida errado**: um rádio de torre não precisa de servidor DHCP; um concentrador não deve ter NAT de fábrica; um switch de POP não quer firewall nas portas. Por isso o costume profissional é começar **limpo** e construir só o que o papel do equipamento pede. Há duas formas de chegar ao estado limpo:

- No primeiro boot de um equipamento físico, o RouterOS mostra a configuração padrão e oferece a tecla **r** para removê-la já no primeiro login;
- A qualquer momento: **/system reset-configuration no-defaults=yes** — que veremos, com os devidos avisos, no 3.

O **CHR nasce limpo** — sem default configuration —, e é por isso que nossos laboratórios nunca precisaram removê-la. O que o CHR *tem*, como todo RouterOS, é o usuário **admin** — e é nele que a segurança inicial começa.

6.2 Usuários e grupos: aposentando o admin compartilhado

Todo RouterOS nasce com o usuário `admin`, do grupo `full`. Numa operação com três técnicos, é tentador todo mundo usar “o admin” — até o dia em que uma configuração some às 3h da manhã e o `/system history print` (Capítulo 3) diz apenas: foi o `admin`. Quem?

A regra profissional: **uma pessoa, um usuário** — e cada usuário com o menor privilégio que cumpre sua função. Os privilégios vêm dos **grupos**, em `/user group`:

Tabela 6.1: Grupos padrão de usuários

Grupo	Pode	Uso típico
<code>read</code>	ver tudo, mudar nada	monitoramento, auditoria, NOC nível 1
<code>write</code>	configurar (exceto gerenciar usuários)	técnicos de campo
<code>full</code>	tudo, inclusive usuários e reboot	administradores

Os comandos essenciais:

```
/user add name=vitor group=full password=SenhaForte!2026 comment="Vitor - admin"
/user add name=noc group=read password=OutraSenha!2026 comment="Monitoramento"
/user print
```

Criado o seu usuário nominal e testado o login com ele, desabilite o `admin` de fábrica — todo scanner de internet conhece esse nome:

```
/user disable admin
```

Sobre as senhas, o essencial cabe em três linhas: longas (frases servem), únicas por equipamento ou por parque (gerenciador de senhas é ferramenta de provedor, não luxo) e **nunca** a mesma do Wi-Fi da sua casa. O RouterOS não impõe complexidade — a disciplina é sua.

Analogia para guardar

Usuário compartilhado é caixa de supermercado sem operador identificado: qualquer diferença no caixa, ninguém foi. Grupos são os cargos — o repositor (`read`) não abre a gaveta; o gerente (`full`) assina a sangria. O `/system history` só vira auditoria quando cada login tem dono.

6.3 Fechando as portas: serviços e acesso

No Capítulo 4 você viu a Tabela 4.1: cada forma de acesso por IP é um serviço em `/ip service`. Segurança inicial é decidir o destino de cada um. A política deste manual, que aplicaremos no laboratório:

1. **Desabilitar o que não se usa:** `telnet` e `ftp` sempre; `www` (HTTP puro) e `api/api-ssl` até que algum uso real os exija;
2. **Restringir o que fica:** os serviços sobreviventes (`ssh`, `winbox`, `www-ssl`) ganham `allowed-address` — uma lista de redes de onde aceitam conexão. Tipicamente, a rede de **gerência** do provedor;
3. **Registrar a decisão:** o `comment` nos itens críticos explica o porquê para o próximo técnico (que pode ser você, daqui a dois anos).

O mesmo raciocínio vale para os protocolos de **camada 2**, que não passam por `/ip service`:

- **Descoberta de vizinhos** (`/ip neighbor discovery-settings`): anuncia o equipamento no cabo. Ótimo na gerência, péssimo nas portas voltadas a clientes — um assinante curioso não precisa ver modelo e versão do seu concentrador;
- **MAC-server** (`/tool mac-server`): controla quem pode abrir MAC-Telnet e WinBox-por-MAC. O kit de resgate do técnico é também o kit de invasão de quem alcança o cabo — restrinja-o a uma lista de interfaces de confiança.

Ambos aceitam o mesmo remédio: uma **interface list** (uma lista nomeada de interfaces, ex.: **gerencia**) usada como filtro. Interface lists reaparecerão como protagonistas no firewall (Volume 3).

6.4 Relógio certo: segurança, não estética

`/system clock print` num equipamento recém-ligado mostra 1970 ou um fuso errado. Parece cosmético — até o dia em que você precisa cruzar o log do concentrador com a reclamação de um cliente (“caiu ontem 21h40”) ou atender um ofício judicial pedindo quem usava tal IP em tal **horário** (assunto sério no Brasil — Volume 9 detalha os requisitos legais de guarda de registros). Log com hora errada é log inútil.

A receita completa, que aplicaremos no laboratório: fuso correto (`America/Sao_Paulo...` ou o seu) e **NTP** — o protocolo que mantém o relógio sincronizado pela rede — apontando para servidores confiáveis; no Brasil, os do projeto `ntp.br`.

! O conceito mais importante do capítulo

Segurança inicial não é um produto que se instala; é uma **sequência que se repete** em todo equipamento que entra no parque: senha forte no usuário nominal → `admin` desabilitado → serviços não usados desligados → acesso restrito à gerência → descoberta

e MAC-server contidos → relógio certo com NTP. Cinco minutos por equipamento; anos de dores de cabeça evitadas.

6.5 Laboratório

6.5.1 Ambiente

O laboratório padrão do Capítulo 5, restaurado no snapshot **base-limpa**. Usaremos apenas **lab-r1** — com o adaptador NAT (**ether3**) habilitado, pois o NTP precisa de internet. Acesse por SSH: `ssh admin@192.168.56.11`.

6.5.2 Comandos passo a passo

Passo 1 — usuário nominal. Crie o seu usuário (troque nome e senha pelos seus), teste-o e desabilite o `admin`:

```
/user add name=vitor group=full password=Lab!Segura2026 comment="dono do lab"
```

Abra **outro** terminal e confirme o login novo antes de fechar o atual — regra de ouro ao mexer em usuários:

```
ssh vitor@192.168.56.11
```

Funcionando, desabilite o usuário de fábrica (na sessão nova):

```
/user disable admin  
/user print
```

```
[vitor@lab-r1] > /user print  
Columns: NAME, GROUP, LAST-LOGGED-IN  
# NAME    GROUP  LAST-LOGGED-IN  
0 ;;; system default user  
  X admin  full   2026-07-06 14:02:11  
1 ;;; dono do lab  
  vitor   full   2026-07-06 14:05:37
```

O **X** marca o `admin` desabilitado.

Passo 2 — serviços. Desligue o que não usamos e restrinja o resto à rede de gerência:

```
/ip service disable telnet,ftp,www,api,api-ssl
/ip service set ssh address=192.168.56.0/24
/ip service set winbox address=192.168.56.0/24
/ip service set www-ssl address=192.168.56.0/24
/ip service print
```

```
[vitor@lab-r1] > /ip service print
Columns: NAME, PORT, ADDRESS
#  NAME      PORT  ADDRESS
0  X telnet   23
1  X ftp      21
2  X www      80
3  ssh       22   192.168.56.0/24
4  winbox    8291  192.168.56.0/24
5  X api      8728
6  www-ssl   443   192.168.56.0/24
7  X api-ssl  8729
```

Passo 3 — camada 2 contida. Crie a interface list **gerencia**, ponha a **ether1** nela e aponte a descoberta de vizinhos e o MAC-server para a lista:

```
/interface list add name=gerencia comment="interfaces de gerencia do lab"
/interface list member add list=gerencia interface=ether1
/ip neighbor discovery-settings set discover-interface-list=gerencia
/tool mac-server set allowed-interface-list=gerencia
/tool mac-server mac-winbox set allowed-interface-list=gerencia
```

Passo 4 — relógio e NTP. Ajuste o fuso e ative o cliente NTP com os servidores brasileiros (garanta que o DHCP client do desafio do capítulo anterior está ativo na **ether3**, ou crie-o: **/ip dhcp-client add interface=ether3**):

```
/system clock set time-zone-name=America/Sao_Paulo
/system ntp client set enabled=yes
/system ntp client servers add address=a.st1.ntp.br
/system ntp client servers add address=b.st1.ntp.br
```

Aguarde alguns segundos e verifique a sincronização:

```
/system ntp client print
```

```
[vitor@lab-r1] > /system ntp client print
      enabled: yes
      mode: unicast
synced-stratum: 1
synced-server: a.st1.ntp.br
```

synced-server preenchido = relógio sob controle. Confira o resultado final com `/system clock print`.

6.5.3 Verificação

1. `ssh vitor@192.168.56.11` entra; tentar `ssh admin@192.168.56.11` é recusado (usuário desabilitado);
2. `/ip service print` mostra `telnet`, `ftp`, `www`, `api` e `api-ssl` com `X`, e os demais restritos a `192.168.56.0/24`;
3. `/ip neighbor discovery-settings print` e `/tool mac-server print` apontam para a lista `gerencia`;
4. `/system clock print` mostra fuso e hora corretos, e `/system ntp client print` mostra um `synced-server`.

Feche com um snapshot novo em `lab-r1` — nomeie `seguranca-inicial`: vários capítulos seguintes partirão dele.

6.5.4 Desafios

1. Crie um usuário `estagiario` do grupo `read` e, logado com ele, tente `/system identity set name=hacker`. O que acontece? E `/user print`, funciona?
2. Nos grupos padrão não existe “técnico que configura mas não reinicia”. Explore `/user group print` e crie um grupo `campo` com as políticas do `write` **sem** a política `reboot`.
3. Sua sessão SSH vem do `192.168.56.1` (o hospedeiro). Aplique, **dentro do Safe Mode**, um `allowed-address` de propósito errado no serviço `ssh` (ex.: `10.99.99.0/24`), deixe a sessão cair e explique o que o Safe Mode fez por você (Capítulo 4).
4. Monte, em texto, o **checklist de entrada** do seu provedor fictício: os passos deste capítulo na ordem em que você os aplicaria num rádio novo antes de ele subir na torre — e um item extra que este capítulo ainda não cobriu, mas o próximo cobrirá (dica: o que você faz *antes* de mexer numa configuração que funciona?).

Solução dos desafios

1. O `set` falha com erro de permissão (`not enough permissions`): o grupo `read` não tem a política `write`. Já `/user print` **também falha** — ver e gerenciar usuários exige a política `sensitive`/política de usuários, que o `read` não tem. Menor privilégio funcionando.

2.

```
/user group add name=campo policy=local,ssh,read,write,winbox,web,!reboot,!policy,!sensi
```

O essencial: partir das políticas do `write` e negar `reboot` (a sintaxe `!politica` nega). Confira com `/user group print` e teste com um usuário de mentira.

3. Com `Ctrl+X` ativo, `/ip service set ssh address=10.99.99.0/24` derruba sua sessão (você não está nessa rede). O Safe Mode detecta a queda e **reverte** o `allowed-`

`address` — reconecte e confirme com `/ip service print`. Sem Safe Mode, seria console da VM para consertar.

4. Sequência razoável: (1) usuário nominal + senha do gerenciador; (2) `admin` desabilitado; (3) serviços: só `ssh+winbox`, com `allowed-address` da gerência; (4) `discovery` e `MAC-server` na lista `gerencia`; (5) fuso + NTP; (6) identidade com o padrão de nomes do parque (ex.: `torre-sul-ntp1`). O item que falta: **backup/export da configuração** antes e depois — assunto do próximo capítulo.

6.6 Resumo

- Equipamentos físicos trazem **default configuration** boa para uso doméstico; no provedor, parte-se limpo (tecla `r` no primeiro login ou reset `no-defaults=yes`). O CHR já nasce limpo.
- **Um usuário por pessoa**, no grupo de menor privilégio (Tabela 6.1); `admin` desabilitado após criar o usuário nominal — e teste o login novo **antes** de fechar a sessão antiga.
- Serviços: desabilite `telnet`, `ftp`, `www`, `api`; restrinja `ssh`, `winbox` e `www-ssl` com `allowed-address` da gerência.
- Camada 2 também se fecha: `discovery-settings` e `mac-server` apontando para uma **interface list** de gerência.
- Fuso + NTP (`ntp.br`) porque log sem hora certa não serve nem ao troubleshooting nem à Justiça.
- Tudo isso é uma sequência repetível de ~5 minutos — o “ritual de entrada” de equipamento novo no parque.

No próximo capítulo, a rede de segurança que faltou ao seu checklist: backup binário, export de configuração, restauração — e o reset, com os avisos que ele merece.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre os menus `/user`, `/ip service`, `/ip neighbor` e `/system ntp`, além da página *Securing your router*, que inspira o checklist deste capítulo. O projeto NTP.br (`ntp.br`) documenta os servidores públicos brasileiros de hora legal. Discher (2016) traz um capítulo dedicado a usuários e segurança de acesso. As referências completas, em ABNT, estão no apêndice [Referências](#).

7 Backup, export e reset

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar a diferença entre o **backup binário** (`/system backup`) e o **export de configuração** (`/export`) — e por que um provedor precisa dos dois;
- Gerar, baixar para fora do equipamento e restaurar cada tipo;
- Ler um export com olhos críticos: o que ele contém, o que omite e como usá-lo para documentar e comparar configurações;
- Resetar um equipamento com `/system reset-configuration`, escolhendo as opções certas (`no-defaults`, `keep-users`, `run-after-reset`);
- Definir a rotina de backup de um parque: quando gerar, onde guardar, como testar.

7.1 Duas fotografias diferentes da mesma configuração

Tudo o que você configura num RouterOS vive numa base de dados interna. O sistema oferece duas maneiras de fotografá-la, e a diferença entre elas é o coração deste capítulo:

- O **backup binário** é um *clone* da base: um arquivo `.backup` que captura tudo — endereços, usuários, senhas, certificados, tudo — num formato que só o RouterOS lê. Restaurá-lo devolve o equipamento exatamente àquele estado, **inclusive identidade e MACs registrados**.
- O **export** é a *receita*: um arquivo de texto `.rsc` com os comandos que reconstroem a configuração — os mesmos comandos que você digitaria. Você pode lê-lo, editá-lo, versioná-lo no Git, aplicar um pedaço em outro equipamento.

Tabela 7.1: Backup binário × export

	Backup binário	Export
Formato	binário (<code>.backup</code>)	texto com comandos (<code>.rsc</code>)
Contém senhas de usuários	sim	não
Contém certificados/chaves	sim	não
Legível/editável	não	sim
Restauração	total, no mesmo equipamento	total ou parcial, em qualquer equipamento

	Backup binário	Export
Uso típico	desastre: voltar o equipamento como era	documentação, migração, auditoria, versionamento

A conclusão prática vem antes da técnica: **um não substitui o outro**. O backup binário é o seguro contra desastre do equipamento X; o export é a documentação viva da configuração — que sobrevive até à morte do hardware, porque pode ser aplicada num substituto de modelo diferente.

💡 Analogia para guardar

O backup binário é a **imagem do disco**; o export é o **código-fonte**. Com a imagem você restaura a máquina idêntica; com o código-fonte você entende, adapta e reconstrói em qualquer máquina. Todo projeto sério guarda os dois.

7.2 Backup binário: gerar, baixar, restaurar

Gerar é um comando (o nome é opcional — sem ele, o RouterOS usa a identidade e a data):

```
/system backup save name=lab-r1-2026-07-06
```

O arquivo aparece no armazenamento interno, visível em `/file print`. E aqui entra a regra número um dos backups: **backup que mora só dentro do equipamento não é backup** — se o hardware morrer, ele morre junto. Baixe para fora. Como o SSH do RouterOS aceita SFTP/SCP (Capítulo 4), do seu computador:

```
scp vitor@192.168.56.11:lab-r1-2026-07-06.backup .
```

Restaurar exige o caminho inverso (subir o arquivo por SCP, se não estiver mais lá) e:

```
/system backup load name=lab-r1-2026-07-06
```

O equipamento pede confirmação e **reinicia** aplicando o estado salvo.

Dois cuidados de quem opera parque:

- O `.backup` carrega **senhas e chaves**: trate o arquivo como material sensível (armazenamento com acesso controlado — não no grupo de WhatsApp da equipe);
- Por padrão o backup é criptografado com a senha do usuário; aceite o hábito de conferir se o arquivo restaura **antes** de precisar dele (o laboratório fará isso).

7.3 Export: a configuração como texto

O `/export` imprime os comandos que reconstroem a configuração — do sistema inteiro ou do menu onde você está:

```
/export file=lab-r1-full
/ip service export file=lab-r1-servicos
```

O primeiro gera `lab-r1-full.rsc` com tudo; o segundo, só a seção `/ip service`. Abra um `.rsc` e você encontra exatamente o que digitaria:

```
# 2026-07-06 14:32:10 by RouterOS 7.15.3
# software id = ...
/ip service
set telnet disabled=yes
set ftp disabled=yes
set www disabled=yes
set ssh address=192.168.56.0/24
```

Três detalhes para ler exports como gente grande:

- Por padrão, o export omite o que está com valor de fábrica. A variante `/export verbose` mostra **tudo** — útil para caçar diferenças sutis;
- Desde o RouterOS v7, `/export show-sensitive` é quem exibe segredos como chaves de PPP (sem essa opção eles saem mascarados); ainda assim, **senhas de usuários nunca saem** num export;
- O export é a base do “diff de configuração”: exporte antes e depois de uma mudança e compare os arquivos — você enxerga exatamente o que mudou. É o complemento perfeito do `/system history`.

Para **aplicar** um `.rsc` (inteiro ou um trecho), o comando é `/import`:

```
/import file-name=lab-r1-servicos.rsc
```

O import executa linha a linha, como se você digitasse — inclusive parando no meio se uma linha falhar. Por isso, exports de seções pequenas e bem escolhidas são mais úteis que um `.rsc` gigante na hora de migrar pedaços de configuração entre equipamentos.

7.4 Reset: recomeçar sem sustos

O comando que zera a configuração é:

```
/system reset-configuration
```

E as opções que o transformam de “botão de pânico” em ferramenta de precisão:

- **no-defaults=yes** — não aplica a configuração de fábrica após o reset: o equipamento volta **vazio**, o ponto de partida profissional (5);
- **keep-users=yes** — preserva os usuários e senhas;
- **skip-backup=yes** — por padrão o RouterOS salva um backup automático antes de resetar; esta opção pula isso (raramente uma boa ideia);
- **run-after-reset=arquivo.rsc** — a joia da coroa: aplica um **.rsc** logo após o reset. Reset + run-after-reset = **provisionamento**: o equipamento renasce já com a configuração-base do seu provedor.

 Aviso

`/system reset-configuration` **apaga toda a configuração** e reinicia o equipamento. Remotamente, isso significa perder o acesso — o equipamento volta ao padrão de fábrica (ou vazio, com **no-defaults=yes**, sem nenhum IP). Antes de resetar qualquer coisa: backup binário e export baixados para fora, e certeza de que você tem um caminho de acesso pós-reset (console físico, WinBox por MAC no cabo, ou o **run-after-reset** bem testado).

 O conceito mais importante do capítulo

A pergunta de operação não é “tenho backup?”, é “**já testei restaurar?**”. Backup não testado é esperança, não backup. A rotina completa: gerar os dois formatos → baixar para fora do equipamento → testar a restauração de tempos em tempos (o laboratório de hoje é o ensaio geral).

7.5 Laboratório

7.5.1 Ambiente

O laboratório padrão (Capítulo 5), usando **lab-r1** a partir do snapshot **seguranca-inicial** do capítulo anterior — assim temos uma configuração de verdade para salvar, destruir e ressuscitar. Acesso: `ssh vitor@192.168.56.11`.

7.5.2 Comandos passo a passo

Passo 1 — fotografar dos dois jeitos. Gere backup binário e export completo:


```
/system backup save name=lab-r1-cap07
/export file=lab-r1-cap07
/file print
```

```
[vitor@lab-r1] > /file print
Columns: NAME, TYPE, SIZE
# NAME                                TYPE      SIZE
0 lab-r1-cap07.backup                 backup    24.3KiB
1 lab-r1-cap07.rsc                    script    1.8KiB
```

Passo 2 — baixar para fora. No terminal do seu computador:

```
scp vitor@192.168.56.11:lab-r1-cap07.backup .
scp vitor@192.168.56.11:lab-r1-cap07.rsc .
```

Abra o `.rsc` num editor de texto e reconheça o capítulo anterior nele: os serviços desabilitados, o `allowed-address`, o NTP. Esta é sua configuração, agora legível e arquivável.

Passo 3 — o desastre. Hora de quebrar tudo de propósito. O reset vai derrubar seu SSH (esperado — é um reset!); a volta será pelo **console da VM**:

```
/system reset-configuration no-defaults=yes skip-backup=yes
```

Confirme com `y`. Após o reboot, entre pelo console da VM: o login voltou a ser **admin sem senha**, não há usuários, não há IP — o equipamento está virgem. Sinta o frio na barriga: em produção, este seria um rádio mudo numa torre.

Passo 4 — ressuscitar pelo backup binário. O arquivo `.backup` ainda está no armazenamento interno (só a *configuração* foi zerada — arquivos sobrevivem ao reset). Pelo console:

```
/system backup load name=lab-r1-cap07
```

Confirme; o roteador reinicia e... está tudo de volta: identidade **lab-r1**, seu usuário, o IP de gerência, os serviços restritos. Prove pelo seu computador:

```
ssh vitor@192.168.56.11
```

Passo 5 — ressuscitar pela receita. Agora o caminho do export. Repita o reset do Passo 3. Pelo console, o equipamento vazio não tem IP — então crie o mínimo para receber o arquivo:

```
/ip address add address=192.168.56.11/24 interface=ether1
```

Do seu computador, suba o `.rsc` (o usuário agora é `admin` sem senha — estado pós-reset):

```
scp lab-r1-cap07.rsc admin@192.168.56.11:
```

E pelo console, importe:

```
/import file-name=lab-r1-cap07.rsc
```

```
Script file loaded and executed successfully
```

Compare com o Passo 4: o import reconstruiu serviços, listas, NTP, identidade — mas **não** os usuários com suas senhas (não estavam no export) nem removeu o que já existia (o IP que você criou à mão). A receita reconstrói a casa; os segredos, você repõe pelo checklist do 5.

7.5.3 Verificação

1. Os dois arquivos (`.backup` e `.rsc`) existem **no seu computador**, não só no roteador;
2. Após o Passo 4, `ssh vitor@...` funciona e `/ip service print` mostra a configuração do capítulo 6 intacta — restauração binária validada;
3. Após o Passo 5, `/system identity print` mostra `lab-r1` e os serviços estão restritos, mas `/user print` **não** tem o usuário `vitor` — você explicou a diferença da Tabela 7.1 na prática;
4. Termine restaurando o snapshot `seguranca-inicial` da VM (ou refazendo o usuário nominal), para os próximos capítulos partirem do estado certo.

7.5.4 Desafios

1. Gere um `/export verbose file=teste-verbose` e compare o tamanho com o export normal (`/file print`). De onde vem a diferença, e quando o verbose é mais útil?
2. Exporte **apenas** a seção `/system ntp client`, delete a configuração de NTP (`/system ntp client servers remove numbers=0,1` e `set enabled=no`) e restaure-a usando só o arquivo exportado.
3. Escreva (em texto) a rotina de backup de um provedor com 200 equipamentos: o que gerar, com que frequência, para onde os arquivos vão, e como você *testaria* as restaurações sem derrubar produção. Que ferramenta do RouterOS poderia agendar a geração? (Palpite livre — a resposta técnica completa é o Volume 12.)
4. Explique por que `run-after-reset` + um `.rsc` bem-feito é a forma mais segura de resetar um equipamento **remoto** — e o que precisa obrigatoriamente estar dentro desse `.rsc` para você não perder o acesso.

💡 Solução dos desafios

1. O verbose sai várias vezes maior: ele imprime **todos** os parâmetros, inclusive os que estão no padrão de fábrica (que o export normal omite). É mais útil para auditoria e comparação fina entre dois equipamentos “teoricamente iguais” — o diff do verbose denuncia qualquer padrão alterado.

2.

```
/system ntp client export file=ntp-backup
/system ntp client servers remove numbers=0,1
/system ntp client set enabled=no
/import file-name=ntp-backup.rsc
/system ntp client print
```

O import reexecuta os comandos da seção e o NTP volta a sincronizar.

3. Rotina defensável: export **diário** + backup binário **semanal** (e ambos antes/depois de qualquer mudança); envio automático para fora (FTP/SFTP de um servidor de backups, e-mail para os pequenos); retenção de algumas gerações; teste de restauração **mensal em laboratório** — um CHR que importa os exports do parque e denuncia erros de sintaxe/seções faltantes. O agendador do RouterOS é o `/system scheduler`, e o Volume 12 monta exatamente essa automação.

4. Porque elimina a janela em que o equipamento fica “vazio e surdo”: o reset já renasce aplicando a configuração-base. O `.rsc` precisa conter, no mínimo: IP de gerência na interface certa (ou DHCP client), usuário nominal com senha (lembre: o export não carrega senhas — num `.rsc` de provisionamento escrito à mão você **pode e deve** incluir o `/user add ... password=...`), e serviços de acesso habilitados/restritos. Sem isso, o “reset seguro” vira viagem à torre.

7.6 Resumo

- **Backup binário** = clone total (com senhas e certificados), restaura o mesmo equipamento; **export** = receita em texto, legível e portátil, sem segredos (Tabela 7.1). Provedor guarda **os dois**.
- Backup que só existe dentro do equipamento não é backup: **baixe por SCP/SFTP** e guarde com controle de acesso.
- `/export` por seção + `/import` é a ferramenta de migração e de “diff” de configuração; `verbose` e `show-sensitive` são os modificadores a conhecer.
- Reset profissional: `no-defaults=yes` para nascer limpo, `keep-users` e `run-after-reset` para não nascer surdo — e **sempre** com backups baixados antes.
- Backup não testado é esperança: ensaie a restauração (você acabou de fazer isso duas vezes).

No próximo capítulo, o último recurso de quem opera hardware de verdade: o **Netinstall** — reinstalar o RouterOS do zero quando nem o reset resolve.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/system backup`, `/export`, `/import` e `/system reset-configuration` (páginas *Backup* e *Configuration Management*). Discher (2016) discute estratégias de backup em parques de provedor. As referências completas, em ABNT, estão no apêndice [Referências](#).

8 Netinstall

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que o Netinstall faz e em que ele difere do reset do capítulo anterior — reinstalação do **sistema**, não só da configuração;
- Reconhecer as situações em que ele é o único caminho: equipamento que não dá boot, senha perdida, suspeita de comprometimento, downgrade;
- Descrever o mecanismo: RouterBOOT em modo *etherboot* + o aplicativo Netinstall respondendo na rede;
- Executar o procedimento completo num equipamento físico, incluindo os preparos que evitam as armadilhas clássicas (cabo direto, IP fixo, firewall do notebook);
- Usar os recursos extras: *keep old configuration*, pacotes por arquitetura, provisionamento com configuração inicial.

8.1 Quando o reset não basta

O 3 encerrou com o reset: apaga-se a configuração, o **sistema** permanece. Mas há defeitos que moram no sistema — e para eles existe o **Netinstall**, a reinstalação completa do RouterOS pela rede. Casos clássicos do provedor:

- o equipamento **não completa o boot** (atualização interrompida por queda de energia na torre, armazenamento corrompido);
- a **senha se perdeu** e não há backup — sem ela não há reset remoto, e o Netinstall zera tudo, sistema e senha juntos;
- suspeita de **comprometimento**: roteador invadido não se “limpa” com reset — a única higiene confiável é reinstalar o sistema de fora;
- **downgrade** controlado ou troca de *release channel* que o upgrade normal não resolve.

A analogia direta: o reset é a *restauração de fábrica* do celular; o Netinstall é *reinstalar o sistema operacional com um cabo no computador*. Um mexe nos dados; o outro regrava a própria plataforma.

O CHR fica de fora deste capítulo por natureza: sendo uma máquina virtual, “reinstalar o sistema” é simplesmente trocar o disco `.vdi` por um novo (Capítulo 5). O Netinstall é ferramenta do mundo **físico**, das RouterBOARD — e por isso este é o único capítulo do

livro cujo laboratório pede um equipamento de verdade (qualquer RouterBOARD barata serve, e um hEX usado é um ótimo investimento de estudo).

8.2 Como funciona por baixo

O truque está no RouterBOOT (Capítulo 3). Além de dar boot pelo armazenamento interno, ele sabe dar boot **pela rede** — o modo *etherboot*: o equipamento liga, envia pedidos de boot pela porta Ethernet e espera que alguém na rede lhe entregue um sistema.

Esse “alguém” é o aplicativo **Netinstall**, da MikroTik, rodando no seu computador, com dois papéis: servir o boot de rede e, em seguida, regravar o RouterOS escolhido no armazenamento do equipamento (Figura 8.1).

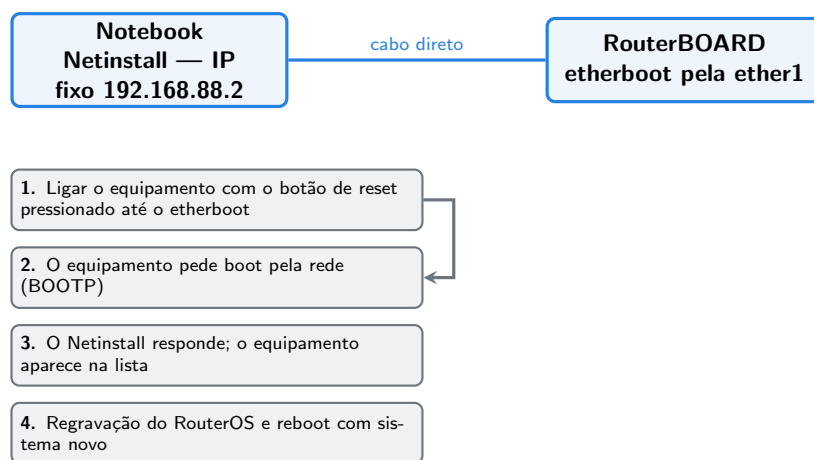


Figura 8.1: O fluxo do Netinstall: notebook e equipamento sozinhos num cabo, boot de rede e gravação do sistema.

Desse mecanismo derivam as regras práticas que evitam 90% das frustrações:

1. **Cabo direto, rede exclusiva.** O boot de rede fala com quem responder primeiro. Notebook e equipamento sozinhos (cabo ponta a ponta) — nunca através da rede do provedor, onde um DHCP alheio atropela o processo. Em muitos modelos, o etherboot só funciona pela **ether1/boot port**.
2. **IP fixo no notebook.** O Netinstall precisa de um endereço estável — o consagrado é 192.168.88.2/24. No aplicativo, configura-se o *Client IP* (ex.: 192.168.88.3) que será oferecido ao equipamento.
3. **Firewall do notebook fora do caminho.** O Windows adora bloquear BOOTP/TFTP silenciosamente; desative o firewall na interface do cabo durante o procedimento.
4. **Pacote da arquitetura certa.** O RouterOS é distribuído por arquitetura (*arm, arm64, mipsbe, tile...*). Baixe o *main package* correspondente ao seu modelo (a ficha do produto informa;

2) — o Netinstall só lista pacotes compatíveis com o equipamento detectado.

O procedimento em si, com os preparos feitos: abrir o Netinstall (como administrador), *Net booting* ativado com o Client IP, ligar o equipamento com o **botão de reset pressionado** até ele entrar em etherboot (os beeps e LEDs indicam; ~15–20 s), o equipamento aparece na lista, seleciona-se o pacote e **Install**. Minutos depois, reboot com sistema zerado — usuário **admin**, sem senha, configuração de fábrica.

Recursos que valem conhecer na janela do aplicativo:

- **Keep old configuration:** tenta preservar a configuração através da reinstalação — útil quando o objetivo é só regravar um sistema corrompido (mas lembre: se a hipótese é invasão, o objetivo é justamente **não** preservar nada);
- **Configure script:** aplica um script de configuração inicial após a instalação — o irmão do **run-after-reset** do capítulo anterior, e a base de provisionamento em massa (equipamento novo → Netinstall → nasce com a configuração-base do parque);
- **Downgrade:** instalar versão mais antiga é tão simples quanto escolher outro pacote — o caminho limpo quando um equipamento específico não se dá bem com uma versão nova.



Aviso

O Netinstall **apaga tudo**: sistema, configuração, usuários, arquivos — inclusive backups guardados no armazenamento interno (mais um motivo para a regra do 3: backup mora **fora** do equipamento). Sem *keep old configuration*, o equipamento volta absolutamente virgem. E não interrompa a gravação no meio — energia estável e paciência; um Netinstall interrompido só se resolve com... outro Netinstall.



O conceito mais importante do capítulo

Com o Netinstall no cinto de ferramentas, **não existe RouterBOARD perdida**: enquanto o hardware estiver são, qualquer estado de software — boot quebrado, senha esquecida, sistema comprometido — se resolve com um cabo, o aplicativo e alguns minutos. É a rede de segurança final de tudo o que você fará com MikroTik daqui em diante.

8.3 Laboratório

8.3.1 Ambiente

Excepcionalmente, este laboratório é **físico**: uma RouterBOARD qualquer (um hEX/hAP usado serve perfeitamente), um cabo Ethernet e um computador Windows com o aplicativo Netinstall (baixe de mikrotik.com/download, seção *Netinstall*) e o *main package* da arquitetura do seu equipamento.

Sem equipamento físico? Faça o “laboratório seco”: leia os passos executando cada preparo possível no seu computador (aplicativo aberto, IP fixo configurado, pacote baixado) e responda os desafios — o procedimento estará ensaiado para o dia em que a torre chamar.

8.3.2 Comandos passo a passo

Preparo do notebook. Configure IP fixo na interface Ethernet: 192.168.88.2, máscara 255.255.255.0 (sem gateway). Desative o firewall dessa interface. Conecte o cabo direto na **ether1** do equipamento (desligado). Abra o Netinstall como administrador, ative *Net booting* e defina o Client IP como 192.168.88.3.

Etherboot. Desconecte a energia do equipamento, pressione e **segure** o botão de reset, reconecte a energia mantendo o botão até o equipamento sinalizar o boot de rede (beep/LEDs, ou até ele aparecer na lista do aplicativo).

Instalação. O equipamento surge na lista do Netinstall com MAC e modelo. Selecione-o, marque o pacote RouterOS baixado e clique **Install**. Acompanhe a barra; ao final, *Installation finished* — e o equipamento reinicia.

Primeiro contato pós-instalação. O equipamento está de fábrica. Reconfigure o IP do notebook para DHCP (a default configuration entrega IP na LAN) ou use o WinBox por MAC (Capítulo 4) e aplique o ritual completo: remoção da default config se for o caso, checklist de segurança do 5, backup do 3. O ciclo do Volume 1, fechado num equipamento real.

8.3.3 Verificação

1. O equipamento apareceu na lista do Netinstall (prova do etherboot + cabo
 - IP corretos);
2. A instalação concluiu sem interrupção e o equipamento reiniciou;
3. O login pós-instalação é **admin** sem senha, e `/system resource print` mostra a versão que você instalou;
4. Ao final, o equipamento está com o checklist de segurança aplicado e um backup baixado — pronto para uso (ou para a próxima brincadeira).

8.3.4 Desafios

1. Por que o Netinstall pede cabo **direto** entre notebook e equipamento, em vez de usar a rede normal do escritório? Cite os dois problemas que a rede compartilhada introduz.
2. Um técnico esqueceu a senha de um rádio na torre, e o rádio **funciona** (clientes navegando). Netinstall já? Monte a árvore de decisão, considerando o que você sabe de acesso (Capítulo 4), backup
 - (3) e as proteções do RouterBOOT (Capítulo 3).

3. Você suspeita que um roteador de borda foi invadido. Por que *keep old configuration* seria um erro nesse caso — e qual é o fluxo correto de recuperação, do Netinstall ao retorno à produção?
4. Desenhe (em texto) a “bancada de provisionamento” de um provedor que ativa 30 rádios por mês: como Netinstall + *configure script* + o checklist do 5 viram uma linha de montagem.

Solução dos desafios

1. (a) **Concorrência de servidores:** na rede do escritório há DHCP e outros serviços que respondem ao boot de rede antes/junto do Netinstall, confundindo o processo; (b) **caminho não confiável:** switches, VLANs e firewalls no meio podem filtrar BOOTP/TFTP. O cabo direto elimina as duas variáveis — e é por isso que é o primeiro item de qualquer checklist de Netinstall que falhou.
2. Netinstall é a **última** parada, porque exige subir na torre e derruba os clientes durante o procedimento. Antes: (1) tentar as vias que não dependem de senha esquecida — outra credencial da equipe, acesso por outro usuário **full**; (2) se há backup binário recente e acesso físico fácil: reset + restaurar backup pode ser mais rápido; (3) sem nada disso — e sem **protected-routerboot** ativo —, Netinstall com janela de manutenção programada. Se **protected-routerboot** estiver ativo, nem o Netinstall entra: é o cenário de RMA que o capítulo 3 avisou.
3. Se o invasor alterou a configuração (usuário oculto, scheduler malicioso, regra de NAT espiã), *keep old configuration* **reinstala a porta dos fundos junto**. Fluxo correto: Netinstall limpo → reconstruir a configuração a partir de um **export conhecido e auditado** (de antes do incidente, lido linha a linha antes do import) → senhas novas em tudo → só então voltar à produção — idealmente entendendo por onde a invasão entrou (o hardening do Volume 3 existe para isso).
4. Linha de montagem: bancada com switch dedicado só para provisionamento; notebook com Netinstall e *configure script* apontando para o **.rsc**-base do parque (IP de gerência por DHCP da bancada, usuário nominal da equipe, serviços restritos, NTP — o checklist do capítulo 6 em forma de script); cada rádio novo: etherboot → install → o script aplica a base → etiqueta com identidade → backup/export arquivados → estoque de “prontos para torre”. Trinta rádios viram uma tarde de trabalho repetitivo e à prova de esquecimento.

8.4 Resumo

- **Netinstall reinstala o sistema;** o reset só limpa a configuração. É o caminho para boot quebrado, senha perdida, comprometimento e downgrade.
- Mecanismo: RouterBOOT em **etherboot** (botão de reset ao ligar) + o aplicativo Netinstall servindo boot e regravando o RouterOS (Figura 8.1).
- Receita à prova de frustração: **cabo direto**, notebook com IP fixo 192.168.88.2, firewall local desligado, pacote da **arquitetura certa**, e paciência durante a gravação.

- *Keep old configuration* preserva a configuração (nunca em caso de invasão); *configure script* provisiona — o **run-after-reset** do mundo físico.
- Netinstall apaga **tudo**, inclusive backups internos — que, você já sabe, deveriam estar fora do equipamento.
- Com ele, fecha-se o ciclo do Volume 1: não existe RouterBOARD irrecuperável.

Este capítulo encerra o Volume 1 — você conhece o sistema, o hardware, as licenças, os acessos, montou seu laboratório e domina o ciclo de vida completo de um equipamento. No Volume 2, mergulhamos de vez na linha de comando: a anatomia do terminal e a construção, peça por peça, de um roteador de borda funcional.

Bibliografia do capítulo

A página *Netinstall* da documentação oficial (MikroTik, 2026a) traz o passo a passo com capturas atualizadas e a tabela de solução de problemas que inspirou nossas regras práticas; o download do aplicativo e dos *main packages* por arquitetura está em mikrotik.com/download. As referências completas, em ABNT, estão no apêndice [Referências](#).

Parte II

Volume 2 — CLI e configuração essencial

9 Anatomia do terminal

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Navegar pela árvore de menus do RouterOS como quem navega por pastas — caminho absoluto, caminho relativo e o retorno à raiz;
- Usar os cinco verbos que operam quase tudo: `print`, `add`, `set`, `remove` e `enable/disable`;
- Entender a **numeração de itens** — por que ela muda, quando usá-la e por que `[find ...]` é mais seguro;
- Explorar comandos desconhecidos sozinho, com `TAB` (completar) e `?` (ajuda embutida);
- Usar `export` como lupa (3) para conferir qualquer seção da configuração;
- Aplicar truques de produtividade: histórico, `/undo` e `/redo`, comentários e flags.

9.1 A árvore e os verbos

Você já usou o terminal do RouterOS em todos os capítulos do Volume 1 — agora vamos entendê-lo de verdade. O modelo mental cabe numa frase: **a configuração é uma árvore de menus; em cada menu vivem itens; e um pequeno conjunto de verbos opera esses itens.**

A árvore você já conhece de tanto digitar: `/ip address` é o menu de endereços dentro da seção IP; `/system identity`, a identidade dentro de system. A barra inicial é a **raiz** — como o `C:\` do Windows ou o `/` do Linux. E, como em qualquer árvore de diretórios, você pode:

<code>/ip address print</code>	<code># caminho absoluto: executa de onde estiver</code>
<code>/ip</code>	<code># entrar no menu ip</code>
<code>address</code>	<code># descer para ip > address</code>
<code>print</code>	<code># executar o comando do menu atual</code>
<code>/</code>	<code># voltar à raiz</code>
<code>..</code>	<code># subir um nível</code>

O prompt sempre mostra onde você está:

```
[vitor@lab-r1] > /ip address
[vitor@lab-r1] /ip/address> print
[vitor@lab-r1] /ip/address> ..
[vitor@lab-r1] /ip>
```

Entrar num menu compensa quando você fará várias operações nele; para comandos avulsos, o caminho absoluto documenta melhor — e é por isso que todos os blocos de comandos deste livro usam caminhos absolutos.

Sobre essa árvore agem os **verbos**. Cinco fazem 95% do trabalho:

Tabela 9.1: Os cinco verbos fundamentais

Verbo	Faz	Exemplo
<code>print</code>	lista os itens do menu	<code>/ip address print</code>
<code>add</code>	cria um item	<code>/ip address add address=... interface=...</code>
<code>set</code>	altera um item existente	<code>/ip service set ssh port=2222</code>
<code>remove</code>	apaga um item	<code>/ip address remove numbers=0</code>
<code>enable / disable</code>	liga/desliga sem apagar	<code>/ip service disable telnet</code>

Repare no padrão dos argumentos: **chave=valor**, separados por espaço, sem vírgulas. `add address=192.168.56.11/24 interface=ether1` lê-se como uma frase: “adicione o endereço X na interface Y”. Aspas só quando o valor tem espaços (`comment="link para a torre"`).

Analogia para guardar

O terminal do RouterOS é um sistema de arquivos onde cada “pasta” (menu) contém “planilhas” (itens em linhas numeradas), e os verbos são as operações de linha: listar, inserir, editar, apagar, ocultar. Quem pensa assim nunca se perde — só pergunta “em que menu estou e que linha quero mexer?”.

9.2 Itens, números e o perigo escondido

Faça um `print` em qualquer menu populado e observe a primeira coluna:

```
[vitor@lab-r1] > /ip service print
Columns: NAME, PORT, ADDRESS
#  NAME      PORT  ADDRESS
```

```
0 X telnet      23
1 X ftp         21
2 X www        80
3  ssh         22  192.168.56.0/24
```

O # é o **número do item** — o endereço da linha, usado pelos verbos: `set 3 port=2222`, `remove numbers=0,1`, `disable numbers=2`. Funciona, e você usará o tempo todo em sessões interativas. Mas guarde o aviso:

Aviso

A numeração **não é estável**: ela é recalculada a cada `print` — remover um item renumera os seguintes, e menus diferentes têm numerações independentes. Número de item em **script**, em **procedimento documentado** ou em comando executado “de memória” sem um `print` imediatamente antes é receita para apagar a linha errada — às vezes, a que segurava seu acesso.

A alternativa profissional é selecionar por **conteúdo**, com `[find ...]`:

```
/ip address remove [find comment=teste-lab-a]
/ip service set [find name=ssh] port=2222
```

O `find` devolve os itens que casam com o critério, e o verbo age sobre eles — não importa em que posição estejam. Você já o usou no laboratório do Capítulo 5; nos scripts do Volume 12 ele será onipresente. Regra prática: **números na exploração interativa; find em tudo que será repetido**.

Duas colunas de metadados ajudam o `find` e os humanos:

- **comment** — todo item aceita um comentário. Em rede de provedor, comentário é documentação de campo: `comment="uplink operadora X"` salva madrugadas;
- **flags** — as letras à esquerda do `print`: X (desabilitado), D (dinâmico: criado por um processo, como DHCP — não se edita, se edita a origem), I (inválido: aponta para algo que não existe mais, ex.: endereço numa interface removida).

9.3 TAB e ?: o manual que mora no equipamento

O RouterOS carrega a própria documentação, e dois atalhos a liberam:

- **TAB** completa o que você começou a digitar: `/ip add + TAB` vira `/ip address`; dois TABs listam as opções possíveis. Além de poupar dedos, o TAB **valida** — se não completa, o comando não existe (ou você está no menu errado);
- **?** mostra a ajuda do ponto onde você está: os submenus de um menu, os argumentos de um comando, os valores válidos de um argumento (`address=?`).

A combinação transforma qualquer menu desconhecido em terreno explorável — foi assim que você descobriu o `/system shutdown` no primeiro capítulo, e é assim que se aprende recurso novo sem sair do terminal.

Complete o kit de exploração com:

- **histórico:** setas ↑/↓ navegam os comandos anteriores da sessão;
- **/undo e /redo:** desfazem/refazem a última ação de configuração — o Ctrl+Z do RouterOS (o Safe Mode do Capítulo 4 continua sendo o cinto de segurança para *sequências* de mudanças);
- **/system history print:** quem fez o quê, quando — o undo sabe até o que desfazer porque essa trilha existe.

9.4 print afiado e export como lupa

O `print` cru despeja tudo; os modificadores o transformam em ferramenta de precisão:

```
/ip address print detail          # todos os campos de cada item
/ip address print where interface=ether1  # filtra por condição
/ip route print count-only        # só conta
/interface print stats            # estatísticas (tráfego, erros)
/ip address print interval=2      # re-executa a cada 2s (monitor ao vivo)
```

O `where` merece atenção: é o irmão interativo do `find` — mesma sintaxe de condições, mas para `ver` em vez de agir. `print where` para investigar, `[find]` para operar.

E quando a pergunta é “como este menu está configurado, por inteiro?”, a resposta é o velho conhecido `export` (3), que funciona em **qualquer** menu:

```
/ip service export
```

```
# 2026-07-06 15:12:40 by RouterOS 7.15.3
/ip service
set telnet disabled=yes
set ftp disabled=yes
set www disabled=yes
set ssh address=192.168.56.0/24
```

`export` responde à pergunta que o `print` não responde: *o que foi mudado em relação à fábrica?* — só as alterações aparecem. Por isso ele é a lupa deste livro: sempre que um laboratório disser “confira a seção X”, o `export` da seção é a conferência.

! O conceito mais importante do capítulo

O terminal inteiro cabe numa gramática: **menu** (onde) + **verbo** (o quê) + **chave=valor** (como) + **seleção por número ou find** (em quem). Com TAB e ? para explorar e **export** para conferir, você opera qualquer recurso do RouterOS — inclusive os que este livro não cobre.

9.5 Laboratório

9.5.1 Ambiente

O laboratório padrão (Capítulo 5): apenas **lab-r1**, a partir do snapshot **seguranca-inicial**. Acesso: `ssh vitor@192.168.56.11`.

9.5.2 Comandos passo a passo

Passo 1 — navegar. Percorra a árvore dos dois jeitos e observe o prompt:

```
/ip
address
print
..
service
print
/
```

Passo 2 — explorar com TAB e ?. No menu raiz, digite `/ip a` e pressione TAB duas vezes: o RouterOS lista os menus que começam com **a** (**address**, **arp**, ...). Entre em `/ip dns` (que nunca usamos) e descubra, usando apenas `?`, qual comando mostra a configuração e qual argumento ativa a resposta a consultas remotas — **não altere nada**; a hora do DNS chega no Capítulo 12.

Passo 3 — os verbos num alvo inofensivo. Vamos exercitar `add/set/disable/remove` num menu sem risco: as entradas estáticas de DNS locais.

```
/ip dns static add name=teste1.lab address=10.99.0.1 comment="cobaia 1"
/ip dns static add name=teste2.lab address=10.99.0.2 comment="cobaia 2"
/ip dns static print
```

```
[vitor@lab-r1] > /ip dns static print
Columns: NAME, ADDRESS, TTL
# NAME      ADDRESS    TTL
0 teste1.lab 10.99.0.1  1d
1 teste2.lab 10.99.0.2  1d
```


Altere, desabilite, observe as flags:

```
/ip dns static set 0 address=10.99.0.100
/ip dns static disable 1
/ip dns static print
```

A entrada 1 agora exibe X. Confira a seção com a lupa:

```
/ip dns static export
```

Passo 4 — sentir a renumeração. Remova o item 0 e imprima de novo:

```
/ip dns static remove numbers=0
/ip dns static print
```

O antigo item 1 (teste2.lab) agora é o 0 — a prova concreta do aviso deste capítulo. Remova-o pelo caminho seguro, por conteúdo:

```
/ip dns static remove [find comment="cobaia 2"]
/ip dns static print
```

Passo 5 — undo e history. Crie uma cobaia nova, desfça e confira a trilha:

```
/ip dns static add name=teste3.lab address=10.99.0.3
/undo
/ip dns static print
/system history print
```

O print volta vazio, e o history mostra as duas ações (o add e o desfazer).

9.5.3 Verificação

1. Você navegou com prompt mudando (/ip/address> etc.) e voltou à raiz;
2. O TAB duplo listou menus, e o ? revelou os argumentos de /ip dns sem você alterar nada;
3. Após o Passo 4, /ip dns static print está vazio — e você viu a renumeração acontecer no meio do caminho;
4. /system history print conta a história completa do laboratório.

9.5.4 Desafios

1. Usando apenas TAB e ? (sem internet), descubra o que faz o comando `/interface ethernet print stats` e identifique duas colunas úteis para diagnosticar um cabo ruim.
2. Em uma linha, desabilite **todas** as entradas de `/ip dns static` que contenham a palavra `lab` no nome. (Dica: `find` aceita `name~"lab"` — o `~` casa por expressão regular.)
3. O comando `/ip service set [find name=ssh] port=2222` muda a porta do SSH. Se você o executasse agora, o que aconteceria com a sua sessão atual? E como você se conectaria depois? (Pense antes; se testar, use o Safe Mode e desfaça.)
4. Explique a diferença entre `print where interface=ether1` e `[find interface=ether1]` — quando cada um é a ferramenta certa?

Solução dos desafios

1. `/interface ethernet print stats` mostra contadores por porta física. Para cabo ruim, procure `rx-fcs-error/rx-align-error` (quadros corrompidos chegando — clássico de cabo/conector ruim) e `rx-too-short`/colisões. Contadores subindo continuamente = camada física suspeita.

2.

```
/ip dns static disable [find name~"lab"]
```

3. A sessão **atual não cai** — conexões TCP já estabelecidas sobrevivem à mudança do serviço; mas a **próxima** conexão precisa de `ssh -p 2222 vitor@192.168.56.11`. O risco real é esquecer a porta nova (ou um firewall no caminho que só libera a 22) — por isso: Safe Mode, testar a conexão nova antes de fechar a antiga (a mesma regra de ouro do 5 com usuários).

4. São a mesma sintaxe de seleção com destinos diferentes: `print where` **mostra** o subconjunto (investigação, sem efeito colateral); `[find]` **entrega os itens a um verbo** (`set`, `remove`, `disable`...) para agir sobre eles. Investigue com `where`, opere com `find` — e nunca opere com números decorados.

9.6 Resumo

- Configuração = **árvore de menus** + **itens numerados** + **verbos** (`print`, `add`, `set`, `remove`, `enable/disable`; Tabela 9.1), com argumentos `chave=valor`.
- Números de item **mudam** (renumeração após remoções): use-os só com um `print` fresco na tela; em tudo repetível, selecione com `[find ...]`.
- Flags contam estado: X desabilitado, D dinâmico, I inválido — e `comment` é a documentação de campo do provedor.
- TAB completa e valida; ? é o manual embutido; ↑/↓, `/undo`, `/redo` e `/system history` completam o kit interativo.

- `print` tem modificadores (`detail`, `where`, `stats`, `interval`); `export` em qualquer menu mostra **o que difere da fábrica** — a lupa para conferir laboratórios e auditar equipamentos.

No próximo capítulo, a gramática entra em serviço: endereçamento IP no RouterOS e a rota padrão — os dois primeiros tijolos do roteador de borda que construiremos ao longo deste volume.

Bibliografia do capítulo

A seção *Console* da documentação oficial (MikroTik, 2026a) detalha a navegação, os modificadores de `print` e a sintaxe completa de seleção (`find`, `where`, expressões regulares). Discher (2016) e Burgess (2011) dedicam capítulos iniciais à filosofia da CLI. As referências completas, em ABNT, estão no apêndice [Referências](#).

10 Endereçamento IP e rota padrão

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Ler e escrever endereços em notação **CIDR** (192.168.10.1/24) e escolher o prefixo certo para cada situação (enlaces /30, LANs /24);
- Configurar endereços em `/ip address` e explicar o que o RouterOS deriva sozinho (campo **network**, rotas conectadas);
- Ler a tabela de rotas (`/ip route print`) com suas flags — D, A, C, S — como quem lê um mapa;
- Criar **rotas estáticas** para redes remotas e a **rota padrão** (0.0.0.0/0), entendendo gateway e **distance**;
- Usar **check-gateway** para que uma rota morta saia de cena sozinha.

10.1 O endereço e a máscara, sem mistério

Um endereço IPv4 são 32 bits escritos em quatro números (192.168.10.1), e todo endereço em uso carrega junto uma informação crucial: **até onde vai a minha rede?** Essa é a função do **prefixo** — o /24 em 192.168.10.1/24, a notação **CIDR** que usaremos no livro inteiro.

O prefixo diz quantos bits iniciais identificam a **rede**; o resto identifica os **hospedeiros** dentro dela. /24 significa “os 3 primeiros números são a rede”: tudo que começa com 192.168.10. é vizinho direto, alcançável sem roteador. Os tamanhos que um provedor usa todo dia:

Tabela 10.1: Prefixos do dia a dia do provedor

Prefixo	Máscara	Endereços úteis	Uso típico
/30	255.255.255.252	2	enlace ponto a ponto entre roteadores
/29	255.255.255.248	6	bloco pequeno de IPs fixos de cliente
/24	255.255.255.0	254	LAN, rede de gerência
/22	255.255.252.0	1022	pool de assinantes

A conta dos “úteis” desconta dois endereços reservados de cada rede: o primeiro (**endereço da rede**, ex.: 192.168.10.0) e o último (**broadcast**, ex.: 192.168.10.255). É por isso

que o enlace entre dois roteadores casa perfeitamente num /30: 4 endereços, 2 reservados, 2 roteadores. Nada sobra, nada falta.

💡 Analogia para guardar

O endereço IP é “rua + número”: o prefixo diz onde termina o nome da rua e começa o número da casa. Dois vizinhos da mesma rua conversam pela janela (mesmo segmento, sem roteador); para outra rua, é preciso ir ao **cruzamento com o guarda de trânsito** — o gateway.

Uma convenção que você já viu no laboratório: redes **privadas** (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 (Rekhter et al., 1996)) são livres para uso interno e não existem na internet. O que acontece quando uma LAN privada precisa sair para a internet é o assunto do Capítulo 13.

10.2 /ip address: o que você dá e o que o RouterOS deduz

Configurar um endereço você já fez desde o Capítulo 4; agora vamos olhar o que acontece por dentro:

```
/ip address add address=10.0.12.1/30 interface=ether2 comment="enlace r1-r2"
/ip address print detail
```

```
[vitor@lab-r1] > /ip address print detail
Flags: X - disabled, I - invalid, D - dynamic
 0   ;;; enlace r1-r2
     address=10.0.12.1/30 network=10.0.12.0 interface=ether2
     actual-interface=ether2
```

Você deu **address** e **interface**; o RouterOS **derivou** o **network** (10.0.12.0, o primeiro endereço do /30). E fez mais uma coisa, silenciosa e importante — criou uma **rota conectada**. Espie a tabela de rotas:

```
/ip route print
```

```
[vitor@lab-r1] > /ip route print
Flags: D - DYNAMIC; A - ACTIVE; c - CONNECT
Columns: DST-ADDRESS, GATEWAY, DISTANCE
  DST-ADDRESS  GATEWAY  DISTANCE
DAc 10.0.12.0/30 ether2      0
DAc 192.168.56.0/24 ether1    0
```

Leia as flags como um crachá de cada rota:

- **D** (*dynamic*) — criada automaticamente por um processo (aqui, pelo endereço que você adicionou; adiante, por DHCP, PPPoE, OSPF...). Não se edita uma rota dinâmica; edita-se a origem dela;
- **A** (*active*) — em uso: é a vencedora para aquele destino agora;
- **c** (*connect*) — rota **conectada**: “esta rede está pendurada direto nesta interface”. É o RouterOS registrando quem são as ruas das suas janelas.

A regra de ouro que essa tabela ensina: **um roteador só alcança o que está na tabela de rotas**. Redes conectadas entram sozinhas; todo o resto — as redes que estão *atrás de outros roteadores* — precisa de rota. É o próximo assunto.

10.3 Rotas estáticas: ensinando o caminho

Imagine a topologia do nosso laboratório (Figura 10.1): **lab-r1** e **lab-r2** conversam pelo enlace 10.0.12.0/30, e atrás de **lab-r2** existe uma LAN 192.168.100.0/24. O **lab-r1** conhece o enlace (conectado), mas a LAN é invisível para ele — não há rota.

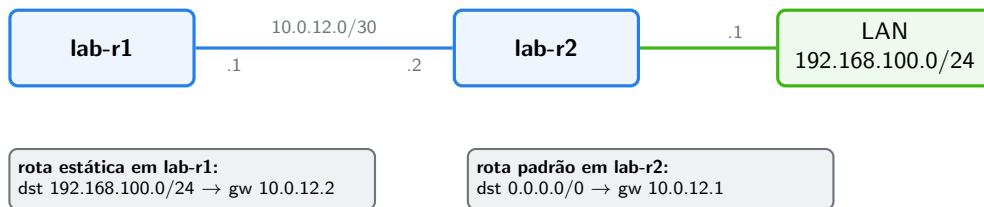


Figura 10.1: A topologia do laboratório deste capítulo, com as duas rotas que a fazem funcionar: a estática (r1 aprende a LAN) e a padrão (r2 manda “todo o resto” para r1).

A solução é uma **rota estática** — você, administrador, ensinando o caminho:

```
/ip route add dst-address=192.168.100.0/24 gateway=10.0.12.2 comment="LAN atrás do r2"
```

Traduzindo: “para chegar em 192.168.100.0/24, entregue o pacote a 10.0.12.2”. O **gateway** precisa ser alcançável por uma rede conectada — é o guarda do cruzamento da sua rua, não de uma rua distante.

No **print**, ela aparece com as flags **AS** (*active, static*). E a coluna que ainda não explicamos entra em cena:

10.3.1 distance: o desempate entre rotas

Duas rotas para o **mesmo destino** podem coexistir — enlace principal e backup, por exemplo. Quem decide a vencedora é a **distância administrativa** (**distance**): menor vence, e só a vencedora fica ativa (**A**). Rotas conectadas têm distância 0 (imbatíveis — a

rede está ali); estáticas nascem com 1; protocolos dinâmicos usam valores maiores (OSPF 110, BGP 20/200 — Volume 8).

O uso clássico no provedor:

```
/ip route add dst-address=0.0.0.0/0 gateway=10.0.12.1 distance=1 comment="saida principal"
/ip route add dst-address=0.0.0.0/0 gateway=10.0.99.1 distance=5 comment="saida backup"
```

A principal vence; se ela sumir da tabela, a backup assume. “Sumir” é o detalhe crucial — e nos leva às duas últimas peças.

10.3.2 A rota padrão e o check-gateway

`dst-address=0.0.0.0/0` casa com **qualquer destino** — é a **rota padrão** (*default route*): “tudo que eu não souber entregar, vai para este gateway”. É ela que leva sua LAN à internet, e é a rota mais importante de qualquer roteador de borda. Na escolha do caminho vale sempre o **prefixo mais específico primeiro**: um pacote para 192.168.100.7 prefere a rota /24 da LAN à rota /0 — a padrão só apanha as sobras. A `distance` só desempata rotas para o **mesmo** prefixo.

Falta o seguro contra gateway morto. Uma rota estática fica ativa enquanto o gateway for *alcançável na tabela* — mas “alcançável” não significa *vivo*: se o roteador do outro lado travar com a interface acesa, a rota continua ativa, despejando pacotes num buraco. O remédio básico:

```
/ip route set [find comment="saida principal"] check-gateway=ping
```

Com `check-gateway=ping`, o RouterOS pinga o gateway periodicamente (a cada 10 s); duas falhas seguidas **desativam a rota** — e a backup de distância 5 assume sozinha. (O failover de gente grande, com detecção de falha *além* do gateway, é assunto do Volume 14.)

! O conceito mais importante do capítulo

Um roteador é a sua **tabela de rotas** — ele não alcança nada fora dela. Redes conectadas entram sozinhas (DAc); o resto é decisão sua: rotas estáticas para o que está atrás de outros roteadores, uma rota padrão (0.0.0.0/0) para “todo o resto”, `distance` para hierarquizar caminhos e `check-gateway` para tirar de cena um caminho morto.

10.4 Laboratório

10.4.1 Ambiente

Laboratório padrão (Capítulo 5): **lab-r1** e **lab-r2**, snapshot base-limpa (em lab-r1, o snapshot `seguranca-inicial` também serve). Acessos: `ssh vitor@192.168.56.11` (r1) e

`ssh admin@192.168.56.12 (r2)`. Topologia da Figura 10.1: enlace pela rede interna lab-a (ether2 de ambos) e a “LAN” na ether3 de lab-r2 (rede interna lab-b, sem mais ninguém — ela existe só para ser anunciada).

10.4.2 Comandos passo a passo

Passo 1 — endereçar o enlace. Em lab-r1:

```
/ip address add address=10.0.12.1/30 interface=ether2 comment="enlace r1-r2"
```

Em lab-r2:

```
/ip address add address=10.0.12.2/30 interface=ether2 comment="enlace r1-r2"  
/ping 10.0.12.1 count=3
```

O ping deve fechar sem perdas — enlace vivo.

Passo 2 — criar a “LAN” atrás do r2. Em lab-r2:

```
/ip address add address=192.168.100.1/24 interface=ether3 comment="LAN simulada"
```

Passo 3 — sentir a falta da rota. De lab-r1, tente alcançar a LAN:

```
/ping 192.168.100.1 count=3
```

```
[vitor@lab-r1] > /ping 192.168.100.1 count=3  
SEQ HOST          SIZE TTL TIME   STATUS  
  0 192.168.100.1                no route to host  
  1 192.168.100.1                no route to host  
  2 192.168.100.1                no route to host  
sent=3 received=0 packet-loss=100%
```

no route to host: o r1 não tem rota — exatamente o diagnóstico que a tabela confirma (/ip route print não tem 192.168.100.0/24).

Passo 4 — a rota estática. Em lab-r1:

```
/ip route add dst-address=192.168.100.0/24 gateway=10.0.12.2 comment="LAN atras do r2"  
/ip route print
```



```
[vitor@lab-r1] > /ip route print
Flags: D - DYNAMIC; A - ACTIVE; c - CONNECT, s - STATIC
Columns: DST-ADDRESS, GATEWAY, DISTANCE
  DST-ADDRESS      GATEWAY    DISTANCE
As 192.168.100.0/24 10.0.12.2      1
DAc 10.0.12.0/30    ether2        0
DAc 192.168.56.0/24 ether1         0
```

E o mesmo ping agora:

```
/ping 192.168.100.1 count=3
```

Sem perdas. Você acabou de rotear entre redes pela primeira vez no livro.

Passo 5 — a rota padrão do r2, com seguro. O r2 conhece o enlace e a própria LAN; ensine-o a mandar “todo o resto” para o r1:

```
/ip route add dst-address=0.0.0.0/0 gateway=10.0.12.1 check-gateway=ping comment="default"
/ip route print where dst-address=0.0.0.0/0
```

Teste com um destino que o r2 **não** conhece — o IP de gerência do r1:

```
/ping 192.168.56.11 count=3
```

Funciona: a rota padrão entregou ao r1, que conhece a rede de gerência (conectada).

Passo 6 — matar o gateway e observar o seguro. Em **lab-r1**, derrube a ponta do enlace (é para isso que a gerência é separada — seu SSH não passa por ali):

```
/interface disable ether2
```

Em **lab-r2**, aguarde ~20 s e imprima:

```
/ip route print where dst-address=0.0.0.0/0
```

A rota padrão perdeu a flag A — o **check-gateway** detectou o gateway morto e a desativou. Reative em **lab-r1** (**/interface enable ether2**), aguarde e confirme que o A voltou.

10.4.3 Verificação

1. `/ip address print` em cada roteador confere com a Figura 10.1 (inclusive o **network** derivado);
2. O ping do Passo 3 falhou com **no route to host** e o do Passo 4 fechou com **packet-loss=0%** — antes e depois da rota;
3. `/ip route print` em **lab-r1** mostra a estática **As** e as conectadas **DAc**;
4. No Passo 6, a rota padrão de **lab-r2** ficou inativa com a **ether2** do **r1** desabilitada e reativou sozinha depois.

10.4.4 Desafios

1. O enlace **r1-r2** usa **/30**. Se você o tivesse endereçado como **/24** (**10.0.12.1/24** e **.2/24**), tudo funcionaria igual — então por que o **/30** é a boa prática em enlaces de provedor? (Duas razões.)
2. Em **lab-r2**, adicione uma **segunda** rota padrão: **gateway=192.168.100.254 distance=5**. Ela aparece ativa? Por quê? E se o **check-gateway** da principal desativá-la (repita o Passo 6), quem assume — e isso seria bom ou ruim nesta topologia?
3. De **lab-r1**, `/ping 192.168.100.1` funciona. Mas se houvesse um computador de verdade na LAN do **r2** (ex.: **192.168.100.50**), o ping de **lab-r1** até ele funcionaria com a configuração atual? Analise o caminho de **volta** do pacote.
4. Qual é a diferença entre remover uma rota (**remove**) e desabilitá-la (**disable**)? Em que situação de operação o **disable** é a escolha certa?



Solução dos desafios

1. (a) **Economia**: um **/24** queimaria 254 endereços úteis onde só existem 2 — em blocos públicos, isso é dinheiro; mesmo em privados, é organização. (b) **Contenção**: no **/30** não existe espaço para um terceiro “morador” — um equipamento intruso ou um erro de configuração não ganha endereço válido no enlace, e a documentação da rede fica inequívoca (cada **/30** é um enlace, com dono conhecido).
2. Ela entra na tabela, mas **sem** a flag **A**: para o mesmo prefixo (**0.0.0.0/0**), vence a menor distância (**1 < 5**). Com a principal desativada pelo **check-gateway**, a de distância 5 assume (**A**). Aqui isso seria **ruim**: **192.168.100.254** não existe (a LAN é simulada, vazia) — o tráfego cairia num buraco. Moral: rota de backup só ajuda se apontar para um caminho que existe; backup mentiroso é pior que nenhum.
3. **Não**. O pacote de ida chega (**r1 → r2 → LAN**), mas o computador responderia para **10.0.12.1** (o endereço de origem do ping) — e a volta depende de o **computador** saber voltar: o gateway dele precisa ser **192.168.100.1** (o **r2**), e o **r2** conhece **10.0.12.0/30** (conectada), então funcionaria **se** o computador tiver o gateway certo. A lição: rota se analisa **nos dois sentidos** — metade dos problemas de “não pinga” mora no caminho de volta.
4. **remove** apaga (e some da configuração, do export, de tudo); **disable** mantém o

item documentado, com comentário e parâmetros, apenas inativo (X). Em operação, **disable** é o certo para mudanças reversíveis e janelas de manutenção — “tirar o backup de cena por uma hora” — e para guardar a configuração de um enlace que volta semana que vem. **remove** é para o que não volta mais.

10.5 Resumo

- **CIDR**: o prefixo diz onde a rede termina; /30 para enlaces, /24 para LANs (Tabela 10.1); cada rede reserva o primeiro (rede) e o último (broadcast) endereços.
- `/ip address add address=X/NN interface=Y` — o RouterOS deriva o **network** e cria a **rota conectada** (DAc) sozinho.
- **A tabela de rotas é o roteador**: conectadas entram sós; o resto exige rota estática (As) com **gateway** alcançável, ou virá de protocolos dinâmicos (Volume 8).
- Rota mais específica vence; entre rotas para o **mesmo** prefixo, vence a menor **distance** (conectada 0, estática 1) — e só a vencedora fica Ativa.
- **0.0.0.0/0** é a rota padrão (“todo o resto”); com **check-gateway=ping**, um gateway morto desativa a rota e o backup de distância maior assume.
- Problema de alcance se analisa **nos dois sentidos** — ida e volta.

No próximo capítulo, tiramos os endereços do modo manual: DHCP client para a WAN receber endereço da operadora, e DHCP server para a LAN distribuir endereços sozinha.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip address` e `/ip route` (incluindo todas as flags e o comportamento do **check-gateway**). O RFC 1918 (Rekhter et al., 1996) define as faixas privadas usadas nos laboratórios. Para a teoria de endereçamento e roteamento IP, Tanenbaum; Feamster; Wetherall (2021) segue sendo a referência completa. As referências, em ABNT, estão no apêndice [Referências](#).

11 DHCP client e server

Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o diálogo DHCP (Discover → Offer → Request → Ack) e o papel do *lease* (empréstimo com prazo);
- Configurar o **DHCP client** numa interface WAN e controlar o que ele aceita do provedor (`add-default-route`, `use-peer-dns`);
- Montar um **DHCP server** completo na LAN — pool, network e server — e explicar o papel de cada peça;
- Gerenciar *leases*: ler a lista, fixar endereço de um equipamento (`make-static`) e escolher o `lease-time` adequado;
- Reconhecer os dois lados na rede de um provedor: onde se é cliente (upstream) e onde se é servidor (clientes internos, gerência de CPEs).

11.1 O empréstimo de endereços

No capítulo anterior, cada endereço foi digitado à mão. Funciona para roteadores — são poucos e não mudam de lugar. Mas a LAN de uma casa tem celulares, TVs e notebooks chegando e saindo; ninguém vai digitar endereço em cada um. O **DHCP** (*Dynamic Host Configuration Protocol*) automatiza: o dispositivo pergunta “alguém me dá um endereço?” e um servidor responde com o pacote completo — endereço, máscara, gateway e DNS.

O diálogo tem quatro passos, fáceis de lembrar pela sigla **DORA** (Figura 11.1):

O detalhe que muda a operação: o endereço é um **empréstimo** (*lease*) com prazo de validade. Na metade do prazo o cliente pede renovação; se o servidor sumir, o endereço expira e volta ao estoque. Esse prazo — o `lease-time` — será uma decisão sua adiante.

Analogia para guardar

DHCP é o balcão de um hotel: o hóspede chega sem quarto (Discover), a recepção oferece o 254 (Offer), o hóspede aceita (Request) e recebe a chave com data de saída (Ack + lease). Hóspede que renova a diária fica; quarto de quem partiu volta ao estoque. E o VIP (seu servidor, sua impressora) tem **quarto reservado** — o lease estático.

No dia a dia do provedor você vive **os dois lados do balcão**:

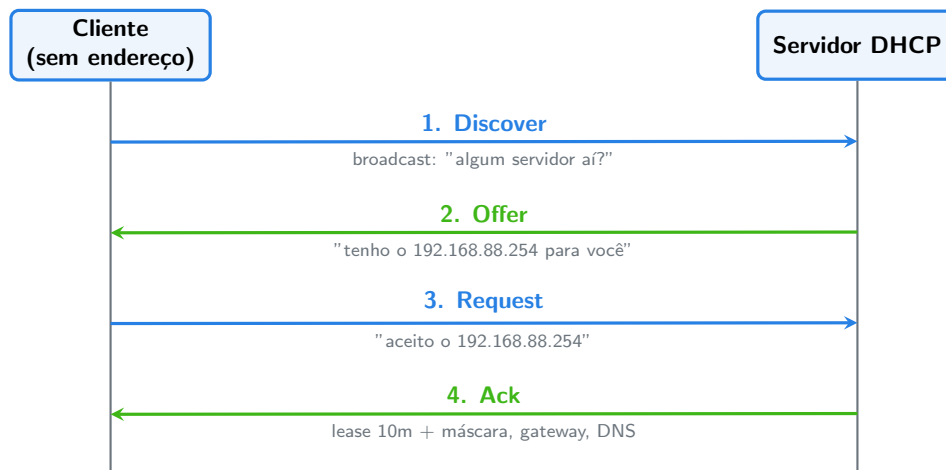


Figura 11.1: O diálogo DORA: o cliente descobre, o servidor oferece, o cliente pede, o servidor confirma — e o endereço vem emprestado, com prazo.

- **Cliente DHCP:** sua WAN recebendo endereço de um upstream — o link da operadora, uma ONU em modo bridge... e o nosso adaptador NAT do VirtualBox, que faz papel de operadora no laboratório;
- **Servidor DHCP:** distribuindo endereços na LAN do cliente final (o hAP na casa do assinante), na rede de gerência dos rádios de uma torre, na rede dos técnicos no POP.

11.2 DHCP client: a WAN que se configura sozinha

Você já rodou o comando no desafio do Capítulo 5; agora com entendimento completo:

```
/ip dhcp-client add interface=ether3
/ip dhcp-client print
```

```
[vitor@lab-r1] > /ip dhcp-client print
Columns: INTERFACE, USE-PEER-DNS, ADD-DEFAULT-ROUTE, STATUS, ADDRESS
# INTERFACE USE-PEER-DNS ADD-DEFAULT-ROUTE STATUS ADDRESS
0 ether3      yes             yes          bound    10.0.2.15/24
```

STATUS: bound = DORA completo, endereço emprestado. E repare no que veio de brinde — confira `/ip address print` e `/ip route print`: um endereço com flag **D** (dinâmico) e uma rota padrão **DAd** apontando para o gateway da operadora. É o Capítulo 10 acontecendo sozinho.

Os dois botões que importam num roteador de provedor:

- **add-default-route (yes/no)** — aceitar a rota padrão oferecida? No roteador de borda simples, sim. Num equipamento **interno** que só usa aquela WAN para um

serviço específico, não: a rota padrão da casa é outra, e uma rota intrusa de distância 1 causa estragos;

- **use-peer-dns** (yes/no) — aceitar os servidores DNS oferecidos? Veremos no Capítulo 12 por que um provedor frequentemente prefere **no** e seus próprios resolvedores.

O contrato de leitura: **configuração dinâmica se gerencia na origem**. O endereço D e a rota D não se editam; ajusta-se o dhcp-client que os criou (**release/renew** estão lá também, para forçar renegociação).

11.3 DHCP server: a LAN que se serve sozinha

O servidor tem três peças, e entender a divisão evita 90% das confusões:

1. **O pool** (/ip pool) — o estoque de endereços para emprestar:

```
/ip pool add name=pool-lan ranges=192.168.88.100-192.168.88.254
```

Repare que o pool **não** cobre a rede inteira: 192.168.88.1 é o roteador, e a faixa .2-.99 fica reservada para endereços fixos (impressoras, câmeras, APs) fora do alcance do sorteio.

2. **O network** (/ip dhcp-server network) — o que dizer aos clientes daquela rede (gateway, DNS, máscara):

```
/ip dhcp-server network add address=192.168.88.0/24 gateway=192.168.88.1 dns-server=1
```

3. **O server** (/ip dhcp-server) — quem atende o balcão, em qual interface, com qual pool e prazo:

```
/ip dhcp-server add name=dhcp-lan interface=ether2 address-pool=pool-lan lease-time=1
```

Pré-requisito que o RouterOS exige: o roteador precisa **ter endereço na rede servida** (192.168.88.1/24 na ether2) — afinal, ele é o gateway que o network anuncia. (Existe um assistente, /ip dhcp-server setup, que faz as três peças em sequência interativa; conhecendo as peças, você sabe o que ele monta — e sabe consertar quando alguém o usou errado.)

11.3.1 Leases: o livro de registros

Cada empréstimo vivo aparece em:

```
/ip dhcp-server lease print
```

```
[vitor@lab-r1] > /ip dhcp-server lease print
Columns: ADDRESS, MAC-ADDRESS, HOST-NAME, SERVER, STATUS
# ADDRESS          MAC-ADDRESS        HOST-NAME  SERVER   STATUS
0 192.168.88.254    08:00:27:3B:71:9E  lab-r2     dhcp-lan bound
```

Dois usos de operação:

- **Fixar o VIP:** `make-static` transforma o lease dinâmico em reserva — aquele MAC receberá **sempre** aquele endereço:

```
/ip dhcp-server lease make-static numbers=0
```

É assim que se garante endereço previsível para a câmera, o AP, o servidor do cliente corporativo — sem configurar nada no dispositivo;

- **lease-time com critério:** prazo curto (10m–1h) em redes de alta rotatividade (hotspot, eventos) devolve endereços rápido ao estoque; prazo longo (1d+) em LANs estáveis reduz tráfego de renovação e mantém endereços previsíveis. O padrão de 10 minutos do RouterOS é conservador — em LAN de cliente, 1h a 1d é o usual.

Aviso

Nunca ative um DHCP server “para testar” numa interface ligada a uma rede que **já tem** servidor DHCP — inclusive a sua rede de gerência do laboratório ou o escritório do provedor. Dois servidores na mesma rede = clientes recebendo configurações conflitantes, aleatoriamente — o clássico *rogue DHCP*, um dos defeitos mais confusos de diagnosticar em campo. (O RouterOS ajuda a caçá-los: `/ip dhcp-server alert`.)

O conceito mais importante do capítulo

DHCP é **estado emprestado com prazo**: o client materializa endereço e rota dinâmicos (flag D) que se gerenciam na origem; o server é pool + network + server, com o pool desenhado para **sobrar espaço fixo** fora do sorteio e leases estáticos para quem precisa de endereço previsível. Provedor opera os dois lados do balcão — e nunca deixa dois balcões na mesma rede.

11.4 Laboratório

11.4.1 Ambiente

Laboratório padrão (Capítulo 5): **lab-r1** e **lab-r2**, snapshot **base-limpa** (+ **seguranca-inicial** no r1, se preferir). Adaptador NAT (**ether3**) do **lab-r1** habilitado. Papéis: **lab-r1** será **cliente** DHCP na WAN (**ether3**, NAT do VirtualBox) e **servidor** DHCP na LAN (**ether2**, rede **lab-a**); **lab-r2** fará o papel do dispositivo do cliente final, pedindo endereço pela **ether2**.

11.4.2 Comandos passo a passo

Passo 1 — r1 como cliente (WAN). Em **lab-r1**:

```
/ip dhcp-client add interface=ether3 comment="WAN do lab (NAT VirtualBox)"
/ip dhcp-client print
```

Aguarde STATUS: bound e inspecione as criações dinâmicas:

```
/ip address print where dynamic
/ip route print where dst-address=0.0.0.0/0
/ping 8.8.8.8 count=3
```

O ping sai para a internet de verdade: WAN pronta, sem digitar um endereço.

Passo 2 — r1 como servidor (LAN). Ainda em lab-r1, as quatro peças na ordem: endereço do gateway, pool, network, server:

```
/ip address add address=192.168.88.1/24 interface=ether2 comment="LAN do lab"
/ip pool add name=pool-lan ranges=192.168.88.100-192.168.88.254
/ip dhcp-server network add address=192.168.88.0/24 gateway=192.168.88.1 dns-server=192.168.88.1
/ip dhcp-server add name=dhcp-lan interface=ether2 address-pool=pool-lan lease-time=1h
```

Passo 3 — r2 como “cliente da casa”. Em lab-r2:

```
/ip dhcp-client add interface=ether2 comment="simula dispositivo na LAN"
/ip dhcp-client print
```

```
[admin@lab-r2] > /ip dhcp-client print
Columns: INTERFACE, USE-PEER-DNS, ADD-DEFAULT-ROUTE, STATUS, ADDRESS
# INTERFACE  USE-PEER-DNS  ADD-DEFAULT-ROUTE  STATUS  ADDRESS
0 ether2     yes           yes                bound   192.168.88.254/24
```

O r2 recebeu endereço **do seu servidor** — DORA completo dentro do seu laboratório. Confira o que veio junto (/ip route print: rota padrão via 192.168.88.1).

Passo 4 — o livro de registros. De volta a lab-r1:

```
/ip dhcp-server lease print
```

O lease do r2 está lá, com MAC e hostname. Fixe-o e renove do lado do cliente para ver a reserva funcionando:

```
/ip dhcp-server lease make-static [find mac-address~":"]
/ip dhcp-server lease print
```

Em lab-r2, force a renegociação e confirme que o endereço **se manteve**:


```
/ip dhcp-client release numbers=0
/ip dhcp-client print
```

11.4.3 Verificação

1. Em lab-r1: dhcp-client bound na ether3, /ping 8.8.8.8 com 0% de perda;
2. Em lab-r1: /ip pool print, /ip dhcp-server network print e /ip dhcp-server print mostram as três peças com os nomes do laboratório;
3. Em lab-r2: endereço dinâmico 192.168.88.x na ether2 e rota padrão D via 192.168.88.1;
4. Em lab-r1: o lease aparece **sem** a flag D após o make-static (virou configuração), e o release/renew do r2 devolveu o mesmo endereço.

11.4.4 Desafios

1. Por que o pool (.100-.254) não começa no .2? Dê dois exemplos de equipamentos de um cliente corporativo que você colocaria na faixa fixa.
2. lab-r2 recebeu rota padrão via 192.168.88.1 — mas o r2 é um roteador do **seu** laboratório, com a própria rota padrão via enlace lab-a em outros capítulos. Que parâmetro do dhcp-client evitaria a rota intrusa, e quando você o usaria em produção?
3. No lab-r1, o que acontece se você apontar o address-pool do server para um pool de **outra** rede (ex.: 10.50.0.100-10.50.0.200)? Raciocine antes; se testar, observe o /log print e desfaça.
4. Um cliente reclama: “de vez em quando meu computador pega IP 169.254.x.x e a internet some”. O que esse endereço significa e quais as duas causas mais prováveis do lado do provedor?

Solução dos desafios

1. A faixa fora do pool é o território dos endereços **previsíveis**: o .1 é o gateway; .2-.99 ficam para leases estáticos e configurações manuais — ex.: a impressora de rede e o DVR de câmeras do cliente corporativo (ambos precisam de endereço fixo para port forwarding, assunto do Volume 3). Se o pool cobrisse tudo, um lease dinâmico poderia colidir com um fixo configurado à mão.
2. add-default-route=no no dhcp-client. Em produção: qualquer equipamento que usa uma WAN DHCP apenas como **serviço secundário** — ex.: um concentrador que recebe uma ONU de backup, um roteador interno que pega endereço de gerência via DHCP — e cuja rota padrão verdadeira vem de outro lugar (estática ou OSPF/BGP).
3. O server entrega um endereço do pool, mas o **network** continua sendo o 192.168.88.0/24 — e um cliente com endereço 10.50.0.x, gateway 192.168.88.1, fica incomunicável (gateway fora da própria rede). O RouterOS registra reclamações no log. Moral: pool, network e o endereço do roteador na interface precisam contar **a mesma história**.

4. 169.254.0.0/16 é *link-local* (APIPA): o sistema se autoendereço quando **nenhum servidor DHCP respondeu** dentro do prazo. Causas prováveis: (a) o DHCP server do CPE/concentrador fora do ar ou sem pool disponível (estoque esgotado — pool pequeno demais para a rotatividade); (b) problema de camada 2 entre o cliente e o servidor (porta, VLAN, rádio) engolindo o broadcast do Discover. Em ambas, o DORA nunca completa.

11.5 Resumo

- DHCP = **DORA** (Discover, Offer, Request, Ack) e endereço **emprestado** com prazo (`lease-time`) — renovado na metade do prazo (Figura 11.1).
- **Client** (`/ip dhcp-client`): WAN automática; controle o que aceita com `add-default-route` e `use-peer-dns`; o que ele cria tem flag D e se gerencia na origem.
- **Server** = três peças: `pool` (estoque, **menor** que a rede), `network` (gateway/DNS anunciados) e `server` (interface + pool + prazo) — mais o endereço do próprio roteador na interface.
- **Leases**: `print` é o livro de registros; `make-static` reserva endereço por MAC; `lease-time` curto para rotatividade, longo para estabilidade.
- Dois servidores na mesma rede = caos (*rogue DHCP*); há `/ip dhcp-server alert` para caçar intrusos.

No próximo capítulo, o terceiro serviço que toda LAN espera do roteador: **DNS** — resolver nomes, cachear respostas e as decisões de segurança que vêm junto.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip dhcp-client`, `/ip dhcp-server` (com o assistente `setup` e os alertas de rogue server) e `/ip pool`. O funcionamento do protocolo (DORA, renovação, opções) está em Tanenbaum; Feamster; Wetherall (2021). As referências completas, em ABNT, estão no apêndice [Referências](#).

12 DNS no RouterOS

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o papel do DNS e o caminho de uma consulta — do dispositivo do cliente ao resolvidor upstream, passando pelo cache do roteador;
- Configurar o resolvidor do RouterOS (`/ip dns`): servidores upstream, cache e o significado exato de `allow-remote-requests=yes`;
- Reconhecer o risco do **open resolver** — por que um roteador respondendo DNS para a internet inteira é um problema seu e dos outros;
- Usar **entradas estáticas** para nomes internos da rede do provedor (e entender seus limites como ferramenta de bloqueio);
- Decidir entre os DNS da operadora (`use-peer-dns`), resolvidores públicos e resolvidor próprio.

12.1 O catálogo telefônico da internet

Nenhum cliente digita 142.250.79.78; ele digita `google.com`. O **DNS** (*Domain Name System*) é o serviço que traduz nomes em endereços — e para o assinante ele **é** a internet: quando o DNS falha, “a internet caiu”, mesmo com o link perfeito. Metade dos chamados de “internet lenta” de um provedor são, na verdade, DNS lento. Por isso este capítulo é curto no protocolo e longo na operação.

O caminho de uma consulta na rede que estamos montando (Figura 12.1):

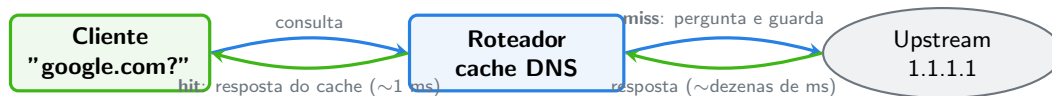


Figura 12.1: O roteador como intermediário com memória: cache hit responde na hora; cache miss consulta o upstream e guarda para o próximo.

O RouterOS não é um resolvidor recursivo completo (ele não sai caçando a resposta pela hierarquia do DNS); ele é um **encaminhador com cache** (*caching forwarder*): repassa a consulta a servidores upstream que você define e **guarda a resposta** pelo tempo indicado (TTL). Esse cache é o que interessa ao provedor: a segunda consulta a `google.com` sai do POP em um milissegundo, sem viajar até o upstream.

12.2 Configurando o resolvedor

Tudo mora num menu só:

```
/ip dns set servers=1.1.1.1,8.8.8.8
/ip dns print
```

```
[vitor@lab-r1] > /ip dns print
      servers: 1.1.1.1,8.8.8.8
dynamic-servers:
allow-remote-requests: no
      cache-size: 2048KiB
      cache-max-ttl: 1w
      cache-used: 32KiB
```

Os campos que decidem a operação:

- **servers** — os upstreams **estáticos** que você escolheu;
- **dynamic-servers** — os que chegaram sozinhos pelo `use-peer-dns=yes` do DHCP client (Capítulo 11). Se ambos existem, o RouterOS usa todos — fonte clássica de comportamento “misterioso”: você configura 1.1.1.1 e as consultas continuam indo para o DNS lento da operadora. Decida **um** dono da verdade: ou `use-peer-dns=no` no client, ou nenhum servidor estático;
- **allow-remote-requests** — o interruptor mais importante do menu, a seguir;
- **cache-size/cache-max-ttl** — o tamanho e a validade máxima do cache; os padrões servem até redes médias.

12.2.1 allow-remote-requests: servidor, e para quem?

Com `allow-remote-requests=no`, o resolvedor só atende o **próprio roteador** (pings e ferramentas locais). Com `yes`, o roteador vira **servidor DNS**: passa a ouvir na porta 53 e responder consultas de terceiros — e é isso que queremos para a LAN: foi por essa razão que o DHCP do Capítulo 11 anunciou `dns-server=192.168.88.1` aos clientes.

O problema: o RouterOS ouve na porta 53 de **todas** as interfaces — a LAN e a WAN. Um roteador com IP público e `allow-remote-requests=yes` sem proteção é um **open resolver**: a internet inteira pode usá-lo como DNS. Isso significa banda e CPU suas servindo estranhos e, pior, participação involuntária em **ataques de amplificação** — o atacante envia consultas pequenas com endereço de origem forjado (o da vítima), e seu roteador despeja respostas grandes na vítima. Seu provedor vira artilharia alheia, e o seu IP entra em listas de bloqueio.

Aviso

`allow-remote-requests=yes` num roteador com interface pública **exige** regras de firewall bloqueando a porta 53 (UDP e TCP) vinda da WAN. Ainda não temos firewall — ele é o assunto do Volume 3, e este é um dos primeiros buracos que fecharemos lá (Capítulo 17). No laboratório estamos seguros: a “WAN” NAT do VirtualBox não é alcançável de fora.

12.3 Entradas estáticas: o DNS interno do provedor

O menu `/ip dns static` — nosso campo de treino no Capítulo 9 — resolve nomes **antes** de consultar o upstream. Usos legítimos de provedor:

```
/ip dns static add name=core-r1.provedor.lan address=10.255.0.1 comment="gerencia core"
/ip dns static add name=noc.provedor.lan address=10.255.0.50
```

Pronto: a equipe digita `ssh core-r1.provedor.lan` em vez de decorar endereços de gerência — um “DNS interno” de custo zero, servido pelo roteador que a equipe já usa. As entradas aceitam expressão regular no nome (`regexp=`), TTL próprio e tipos além de A (AAAA, CNAME, TXT...).

E o uso tentador que exige honestidade: **bloqueio por DNS** (apontar `name=site-indesejado.com` para 127.0.0.1). Funciona como filtro de conveniência — rede de escritório, controle parental básico — e falha como segurança: qualquer dispositivo com DNS alternativo configurado (ou DoH, o DNS sobre HTTPS dos navegadores modernos) passa por cima sem esforço. Bloqueio de verdade se faz no firewall, e mesmo lá tem limites — Volume 3.

O conceito mais importante do capítulo

No provedor, o DNS é **serviço de infraestrutura**: escolha upstreams rápidos e confiáveis (e um só dono da verdade — estático **ou** `use-peer-dns`), sirva a LAN com cache local (`allow-remote-requests=yes`) e **nunca** deixe a porta 53 aberta para a WAN. DNS rápido = “internet rápida” na percepção do cliente; DNS aberto = seu roteador trabalhando para atacantes.

12.4 Laboratório

12.4.1 Ambiente

Continuação direta do laboratório do Capítulo 11 (se você o desfez, repita os Passos 1–3 de lá): `lab-r1` com WAN NAT (`ether3`) e DHCP server na LAN (`ether2`); `lab-r2` como cliente DHCP na `ether2`.

12.4.2 Comandos passo a passo

Passo 1 — um só dono da verdade. O dhcp-client da WAN trouxe DNS dinâmico (use-peer-dns=yes). Vamos assumir o controle: upstreams explícitos e nada de dinâmico:

```
/ip dhcp-client set [find interface=ether3] use-peer-dns=no
/ip dns set servers=1.1.1.1,8.8.8.8
/ip dns print
```

Confira: servers com os dois endereços, dynamic-servers vazio.

Passo 2 — o roteador resolve (para si). Teste o resolvedor local:

```
/resolve mikrotik.com
```

```
[vitor@lab-r1] > /resolve mikrotik.com
159.148.147.196
```

E veja a resposta guardada no cache:

```
/ip dns cache print where name="mikrotik"
```

Passo 3 — o roteador serve a LAN. Ligue o modo servidor:

```
/ip dns set allow-remote-requests=yes
```

Em lab-r2 (que recebeu dns-server=192.168.88.1 via DHCP no capítulo anterior — confira com /ip dns print, campo dynamic-servers), resolva um nome **através do r1**:

```
/ping google.com count=3
```

O ping traduz o nome (via r1) e sai pela rota padrão (via r1) — o mini-provedor do laboratório está completo até a camada de nomes.

Passo 4 — sentir o cache. Em lab-r1, meça a diferença entre miss e hit. Primeiro limpe o cache, depois resolva o mesmo nome duas vezes:

```
/ip dns cache flush
/resolve wikipedia.org
/resolve wikipedia.org
```

A primeira consulta demora dezenas de milissegundos (foi ao upstream); a segunda é instantânea (cache). Veja o estoque acumulado:

```
/ip dns cache print
```

Passo 5 — DNS interno. Crie o nome de gerência do próprio laboratório e use-o de lab-r2:

```
/ip dns static add name=r1.lab address=192.168.88.1 comment="gerencia lab"
```

Em lab-r2:

```
/ping r1.lab count=2
```

O nome interno resolve — servido pela entrada estática do r1, sem consultar upstream nenhum.

12.4.3 Verificação

1. `/ip dns print` em lab-r1: `servers=1.1.1.1,8.8.8.8`, `dynamic-servers` vazio, `allow-remote-requests=yes`;
2. `/ping google.com` funciona em lab-r2 (nome + rota, tudo via r1);
3. `/ip dns cache print` em lab-r1 acumula entradas conforme você resolve nomes — e flush as zera;
4. `/ping r1.lab` responde em lab-r2 (entrada estática servida à LAN).

12.4.4 Desafios

1. Explique a sequência de consulta quando lab-r2 pede r1.lab ao r1: por que a entrada estática responde mesmo com o upstream configurado? E se existisse um r1.lab real na internet?
2. No cache (`/ip dns cache print detail`), localize o TTL de uma entrada. Resolva o mesmo nome de novo e observe o TTL. O que está acontecendo, e por que o `cache-max-ttl=1w` raramente “vale” na prática?
3. Um colega sugere: “aponte `use-peer-dns=yes` de volta e deixe também os servers estáticos — redundância!”. Por que essa “redundância” costuma piorar o serviço em vez de melhorar?
4. Projete (só em texto) o DNS de um provedor de 5 mil assinantes: o que você anunciaria via DHCP/PPPoE aos clientes, onde ficaria o cache, e que upstream escolheria? Considere: latência, privacidade dos dados de consulta dos clientes e o que acontece se o upstream cair.

💡 Solução dos desafios

1. A ordem do resolvidor é: **estáticas primeiro**, depois cache, depois upstream. `r1.lab` casa com a estática e a consulta morre ali — rápido e sem vazamento. Se existisse um `r1.lab` público, os clientes do seu roteador **nunca** o veriam: a estática o “sombreia” (é exatamente assim que o bloqueio por DNS funciona — e por que entradas estáticas erradas causam defeitos misteriosos).
2. O TTL exibido **decrece** a cada segundo: é o prazo restante da resposta, herdado do TTL original definido pelo dono do domínio (minutos a horas, tipicamente). O `cache-max-ttl=1w` é apenas um **teto**: o RouterOS guarda pelo menor entre o TTL da resposta e o teto — e como quase nenhum domínio publica TTL de uma semana, o teto quase nunca decide.
3. O RouterOS distribui consultas entre **todos** os servidores (estáticos + dinâmicos), sem preferência por qualidade: parte das consultas irá ao DNS da operadora mesmo com upstreams melhores configurados. Resultado: desempenho inconsistente (ora 5 ms, ora 80 ms) e diagnóstico difícil. Redundância boa é ter **dois upstreams bons** nos **servers** — não misturar bons e ruins.
4. Desenho razoável: anunciar aos assinantes **o IP do próprio provedor** (o cache no concentrador/POP, ou um par de resolvidores dedicados — dois CHRs com `allow-remote-requests=yes` atrás de firewall); upstreams `1.1.1.1+8.8.8.8` (latência baixa e independência entre si) — ou resolvidor recursivo próprio (ex.: Unbound em servidor Linux) se a privacidade das consultas dos clientes for requisito, pois upstream público vê tudo que seus clientes resolvem. Queda de upstream: com dois independentes, o cache segura o grosso; recursivo próprio elimina a dependência. O que **não** fazer: anunciar upstream público direto aos clientes (perde-se o cache local e o controle).

12.5 Resumo

- O RouterOS é um **encaminhador com cache**: estáticas → cache → upstream (Figura 12.1); cache local faz a “internet do cliente” parecer rápida.
- Um só dono da verdade: **servers** estáticos **ou** `use-peer-dns` — os dois juntos misturam upstreams bons e ruins.
- `allow-remote-requests=yes` liga o servidor DNS para a LAN — e abre a porta 53 em **todas** as interfaces: sem firewall na WAN, você é um **open resolver** servindo ataques de amplificação.
- `/ip dns static`: DNS interno de gerência (ótimo), bloqueio de conteúdo (frágil — DoH e DNS alternativo passam por cima).
- `/resolve`, `/ip dns cache print` e `flush` são o kit de diagnóstico.

No próximo capítulo, a última peça que falta ao nosso roteador de borda: **NAT masquerade** — como uma LAN inteira de endereços privados navega na internet com um único IP público.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip dns` (resolvedor, cache, entradas estáticas e tipos de registro). Sobre o funcionamento do DNS em profundidade (hierarquia, recursão, TTLs), Tanenbaum; Feamster; Wetherall (2021) é a referência. O risco de open resolvers e amplificação é documentado pelo CERT.br em cert.br/docs/ (recomendações para provedores brasileiros). As referências completas, em ABNT, estão no apêndice [Referências](#).

13 NAT masquerade

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar por que uma LAN de endereços privados (Rekhter et al., 1996) não navega na internet sem tradução — analisando o caminho de **volta** do pacote;
- Descrever o que o NAT de origem faz com cada pacote e como o roteador “lembra” cada conexão traduzida;
- Criar a regra clássica de **masquerade** e ler a tabela de conexões que ela alimenta;
- Diferenciar **action=masquerade** de **action=src-nat** — e saber quando o IP fixo vale a troca;
- Reconhecer os limites: NAT não é firewall, quebra a conectividade fim a fim e tem efeitos colaterais que reaparecerão no CGNAT (Volume 9).

13.1 O problema: cartas sem remetente válido

Nosso laboratório terminou o capítulo anterior num estado curioso: **lab-r2** resolve nomes, tem rota padrão via **lab-r1**, o **lab-r1** alcança a internet... e ainda assim um **/ping 8.8.8.8** de **lab-r2** **falha**. Rota existe, DNS existe — o que falta?

Siga o pacote. Ele sai de **lab-r2** com origem 192.168.88.254, o **r1** o encaminha pela WAN, e ele até pode chegar ao destino. O problema é a **resposta**: ela seria endereçada a 192.168.88.254 — um endereço privado (Rekhter et al., 1996), que **não existe na internet**. Nenhum roteador do mundo tem rota para ele (e os provedores descartam esses destinos). A resposta morre no caminho; para quem pinga, é como se a internet não respondesse. É a lição do Capítulo 10 levada ao extremo: o caminho de volta é metade do problema — e na internet pública, para origem privada, **não há volta**.

A solução tem nome pomposo — *Network Address Translation* de origem (**srcnat**) — e lógica simples: na saída, o roteador **troca o remetente**.

13.2 Como a máscara funciona

Com NAT de origem ativo, cada pacote que sai da LAN para a WAN passa por uma transformação (Figura 13.1):

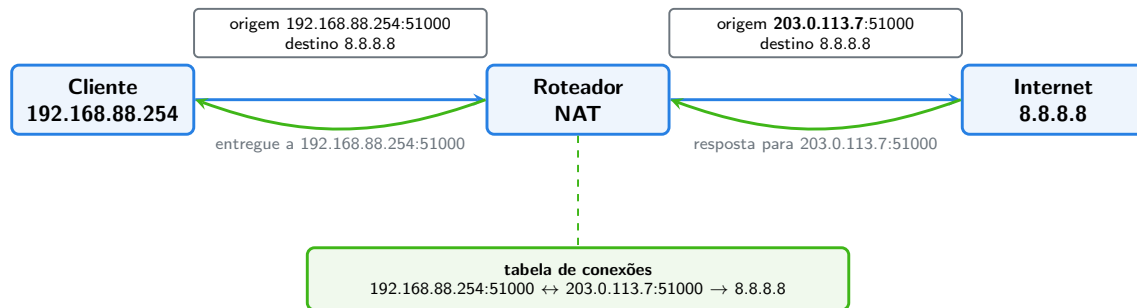


Figura 13.1: O NAT de origem em ação: troca de remetente na ida, anotação na tabela de conexões, tradução desfeita na volta.

Três observações fazem o mecanismo inteiro:

1. **Na ida**, a origem privada é substituída pelo endereço **público** do roteador (e, se preciso, a porta de origem também muda — é assim que centenas de clientes compartilham um IP sem misturar as respostas);
2. **A memória**: cada tradução vira uma linha na **tabela de conexões** do roteador — quem pediu, por qual porta, para onde. Essa tabela (*connection tracking*) é uma peça central do RouterOS e será a estrela do primeiro capítulo do Volume 3;
3. **Na volta**, a resposta chega ao IP público, o roteador consulta a tabela, desfaz a tradução e entrega ao cliente certo.

💡 Analogia para guardar

O NAT é a **portaria do prédio**: as cartas dos moradores saem com o endereço do prédio como remetente, e o porteiro anota num caderno quem mandou o quê. A resposta chega ao prédio, o porteiro consulta o caderno e entrega no apartamento certo. Ninguém lá fora conhece os apartamentos — conhecem o prédio.

13.3 A regra no RouterOS

O NAT vive em `/ip firewall nat` — sim, dentro do firewall: NAT e filtro compartilham o motor de classificação de pacotes (chains, matchers) que o Volume 3 destrinchará. Por ora, o essencial: regras de NAT de origem vivem na chain **srcnat**, e a regra clássica do roteador de borda é uma linha:

```
/ip firewall nat add chain=srcnat out-interface=ether3 action=masquerade comment="NAT da L
```

Lendo por partes:

- **chain=srcnat** — “esta regra trata da origem dos pacotes que saem”;

- `out-interface=ether3` — só pacotes saindo pela WAN. É o **matcher**: a condição que seleciona quem sofre a ação. Sem ele, você mascararia até o tráfego interno;
- `action=masquerade` — “troque a origem pelo endereço **que estiver** na interface de saída”.

13.3.1 masquerade ou src-nat?

O `masquerade` descobre o endereço público sozinho, a cada conexão — feito sob medida para WAN de endereço **dinâmico** (DHCP, PPPoE): mudou o IP, a regra continua certa. A alternativa é dizer o endereço explicitamente:

```
/ip firewall nat add chain=srcnat out-interface=ether3 action=src-nat to-addresses=203.0.1
```

`src-nat` com IP fixo é mais leve (não consulta a interface a cada conexão) e mais previsível — a escolha natural quando o IP público é estático, e obrigatória quando se traduz para um endereço que não é o da interface (situação comum no CGNAT, Volume 9). A regra de bolso: **WAN dinâmica → masquerade; IP fixo → src-nat**.

Um efeito colateral do `masquerade` que vale conhecer desde já: quando o link cai e volta (ou o IP muda), as conexões antigas na tabela apontam para o endereço velho — e ficam zumbis até expirar, causando aqueles minutos de “internet meio quebrada” depois que o link retorna. Guarde o sintoma; o remédio (limpar conexões, ajustar timeouts) aparece com o connection tracking no Volume 3.

13.3.2 O que o NAT não é

- **NAT não é firewall.** A tabela de conexões só deixa *voltar* o que *saiu* — o que parece proteção. Mas qualquer serviço do próprio roteador (o DNS do capítulo anterior, o WinBox...) continua exposto na WAN, e regras de NAT mal escritas abrem caminhos inesperados. Proteção é assunto de **filter rules** — Volume 3, sem atalhos;
- **NAT quebra o fim a fim.** Ninguém de fora consegue *iniciar* conexão com um cliente atrás do NAT — ótimo para esconder, péssimo para servir. Publicar um serviço interno (o DVR do cliente, o servidor da empresa) exigirá a tradução inversa, **dstnat** — também Volume 3;
- **NAT em escala tem custo:** portas são finitas (~64 mil por IP público), e milhares de assinantes por IP esbarram nisso — o mundo do CGNAT, com suas obrigações legais brasileiras, é o Volume 9.

! O conceito mais importante do capítulo

NAT de origem = **troca de remetente + memória por conexão**. A regra `chain=srcnat out-interface=WAN action=masquerade` é uma linha, mas o que ela cria é estado: cada conexão traduzida vive na tabela de conexões do roteador. Entender essa tabela — não a regra — é o que separa quem configura NAT de quem diagnostica

NAT. (E é por ela que o Volume 3 começa.)

13.4 Laboratório

13.4.1 Ambiente

Continuação direta dos capítulos Capítulo 11 e Capítulo 12: lab-r1 com WAN NAT (ether3, dhcp-client) e LAN (ether2, 192.168.88.1/24 + DHCP server); lab-r2 como cliente DHCP da LAN. É o estado em que o capítulo anterior terminou.

13.4.2 Comandos passo a passo

Passo 1 — constatar a falha. Em lab-r2:

```
/ping 8.8.8.8 count=3
```

```
[admin@lab-r2] > /ping 8.8.8.8 count=3
  SEQ HOST      SIZE TTL TIME   STATUS
    0 8.8.8.8                timeout
    1 8.8.8.8                timeout
    2 8.8.8.8                timeout
sent=3 received=0 packet-loss=100%
```

Repare: **timeout**, não no **route to host**. Rota existe (a padrão via r1); o que não existe é resposta — o sintoma clássico de origem privada sem tradução.

Passo 2 — a regra. Em lab-r1:

```
/ip firewall nat add chain=srcnat out-interface=ether3 action=masquerade comment="NAT da LAN"
/ip firewall nat print
```

```
[vitor@lab-r1] > /ip firewall nat print
Flags: X - disabled, I - invalid; D - dynamic
 0      ;;; NAT da LAN para a WAN
      chain=srcnat action=masquerade out-interface=ether3
```

Passo 3 — o milagre. Em lab-r2, repita:

```
/ping 8.8.8.8 count=3
/ping google.com count=3
```

Zero de perda nos dois — endereço, rota, DNS e agora NAT: o pacote completo do acesso à internet, montado peça por peça por você.

Passo 4 — ver a memória. Enquanto o `lab-r2` gera tráfego (deixe um `/ping 8.8.8.8` rodando sem `count`), veja em `lab-r1` a tabela de conexões:

```
/ip firewall connection print where dst-address~"8.8.8.8"
```

```
[vitor@lab-r1] > /ip firewall connection print where dst-address~"8.8.8.8"
Columns: PROTOCOL, SRC-ADDRESS, DST-ADDRESS, REPLY-SRC-ADDRESS, REPLY-DST-ADDRESS
# PROTOCOL SRC-ADDRESS      DST-ADDRESS  REPLY-SRC-ADDRESS  REPLY-DST-ADDRESS
0 icmp      192.168.88.254    8.8.8.8      8.8.8.8            10.0.2.15
```

A linha conta a história completa: origem real (192.168.88.254), e a resposta (REPLY-DST) endereçada ao IP da WAN (10.0.2.15) — a tradução, anotada. Interrompa o ping no r2 (Ctrl+C).

13.4.3 Verificação

1. O ping do Passo 1 falhou com `timeout` (e você sabe explicar por que não foi `no route to host`);
2. `/ip firewall nat print` mostra a regra única de masquerade com o comentário;
3. Após a regra, `lab-r2` pinga IP e nome com 0% de perda;
4. A tabela de conexões exibiu a tradução (origem privada → endereço da WAN).

13.4.4 Desafios

1. No Passo 4, a resposta vai para 10.0.2.15 — que também é um endereço privado! Por que o NAT funciona mesmo assim no laboratório? (Pense em quantas “portarias” existem entre o `lab-r2` e o 8.8.8.8.)
2. Remova o matcher: crie a regra só com `chain=srcnat action=masquerade` (sem `out-interface`), desabilitando a original. O laboratório continua funcionando — então qual é o perigo? Analise o que acontece com o tráfego `lab-r2 → 192.168.56.1` (gerência) e por que “funciona” não é “está certo”. Desfaça ao final.
3. Sua WAN ganhou IP fixo 203.0.113.7. Escreva a regra `src-nat` equivalente e explique **dois** ganhos práticos sobre o masquerade nesse cenário.
4. Um cliente corporativo atrás do seu NAT quer acessar as câmeras dele a partir do celular (de fora). Com o que você aprendeu até aqui, isso é possível? O que falta, conceitualmente — e em que volume mora a resposta?

Solução dos desafios

1. Porque há **NAT em cascata**: o `lab-r1` traduz 192.168.88.254 → 10.0.2.15, e o NAT do VirtualBox traduz 10.0.2.15 → IP do seu computador, que por sua vez

pode estar atrás do NAT do seu roteador doméstico (→ IP público da sua casa). Cada portaria anota no próprio caderno e desfaz na volta. É também um retrato honesto da internet IPv4 atual — e do que o CGNAT (Volume 9) faz em escala de provedor.

2. Sem `out-interface`, a regra mascara **tudo que sai por qualquer interface** — inclusive o tráfego para a rede de gerência: o 192.168.56.1 passa a ver conexões vindas de 192.168.56.11 (o endereço do r1 naquela interface) em vez do endereço real do r2. Você perde a rastreabilidade interna (logs, permissões por origem) e cria defeitos finíssimos de diagnosticar. “Funciona” — correto: NAT se aplica **na fronteira**, não no meio da casa.

3.

```
/ip firewall nat add chain=srcnat out-interface=ether3 action=src-nat to-addresses=203.0
```

Ganhos: (a) menos trabalho por conexão (não resolve o IP da interface a cada vez — relevante em concentrador com milhares de conexões novas por segundo); (b) previsibilidade — o endereço de origem público é sempre o mesmo, o que facilita liberações em firewalls de terceiros, reputação de IP e diagnóstico.

4. Não é possível ainda: o NAT de origem só cria traduções para conexões **iniciadas de dentro**; alguém de fora não tem como alcançar a câmera (o fim a fim está quebrado, por design). Falta a tradução inversa — **dstnat/port forwarding**: “o que chegar ao meu IP público na porta X, entregue ao IP interno Y”. É o Capítulo 19, no Volume 3 — junto com o firewall que impede que esse portão aberto vire problema.

13.5 Resumo

- LAN privada não navega sem tradução: a **resposta** para um endereço RFC 1918 não tem rota na internet — o sintoma é **timeout** com rota existente.
- NAT de origem: **troca o remetente na saída, anota na tabela de conexões, desfaz na volta** (Figura 13.1) — portas de origem distinguem os clientes que compartilham o IP.
- A regra clássica: `chain=srcnat out-interface=WAN action=masquerade` — matcher primeiro, ação depois. **WAN dinâmica** → **masquerade**; **IP fixo** → **src-nat** (mais leve e previsível).
- NAT **não é firewall** (os serviços do roteador seguem expostos), quebra o fim a fim (publicar serviço = `dstnat`, Volume 3) e em escala vira CGNAT (Volume 9).
- `/ip firewall connection print` é a janela para a memória do NAT — e a ponte para o connection tracking do Volume 3.

No próximo capítulo, nada de conceito novo: o **projeto integrador** do volume — um roteador de borda completo, montado do zero, peça por peça, com as mãos.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip firewall nat` (chains, actions e matchers) e a tabela de conexões. O RFC 1918 (Rekhter et al., 1996) define os endereços privados que tornam o NAT necessário. Tanenbaum; Feamster; Wetherall (2021) discute NAT, seus efeitos no fim a fim e a transição para IPv6 — nosso assunto no Volume 9. As referências completas, em ABNT, estão no apêndice [Referências](#).

14 Projeto: roteador de borda

i Objetivos de aprendizagem

Este capítulo não introduz conceitos novos — ele **integra** tudo do volume. Ao final, você será capaz de:

- Transformar uma “ordem de serviço” em uma sequência de configuração planejada, executada e verificada;
- Montar um roteador de borda completo, do equipamento zerado ao cliente navegando: segurança inicial → WAN → LAN → DHCP → DNS → NAT;
- Verificar o serviço em camadas (enlace → endereço → rota → tradução → nome), na ordem que o diagnóstico profissional usa;
- Entregar o serviço **documentado**: export comentado e backup baixado.

14.1 A ordem de serviço

Situação real de qualquer semana num provedor: um cliente corporativo contratou um link dedicado, e você vai configurar o roteador que fica na sede dele. A ordem de serviço diz:

Cliente: Padaria Pão Quente (matriz) **WAN:** via ONU em bridge, endereço por DHCP do concentrador **LAN:** 192.168.10.0/24, roteador no .1 **Serviços:** DHCP na LAN (faixa .100–.199), DNS com cache local, internet para todos os dispositivos **Reservas:** impressora fiscal sempre no .10 **Padrão da empresa:** checklist de segurança, identidade `cli-paoquente-r1`, export e backup arquivados ao final

A Figura 14.1 mostra o que a ordem descreve:

Antes de digitar qualquer coisa, o hábito profissional: **planejar a sequência**. A ordem importa — cada camada depende da anterior, e é a mesma ordem em que se diagnostica depois:

Tabela 14.1: O plano de execução

#	Etapa	Capítulo de referência
0	Base limpa + identidade + segurança	5
1	WAN: DHCP client	Capítulo 11

#	Etapa	Capítulo de referência
2	LAN: endereço do gateway	Capítulo 10
3	DHCP server (pool, network, server) + reserva	Capítulo 11
4	DNS: upstream + cache para a LAN	Capítulo 12
5	NAT masquerade	Capítulo 13
6	Verificação em camadas	todos
7	Documentar: export + backup para fora	3

💡 Analogia para guardar

Configurar rede é construir prédio: fundação (acesso seguro), estrutura (endereços e rotas), instalações (DHCP, DNS, NAT) e a vistoria (verificação em camadas). Ninguém azuleja antes de rebocar — e quem pula a vistoria descobre o vazamento pelo telefone do cliente.

14.2 Laboratório

Este capítulo é o laboratório — o texto acima era o projeto; agora, mãos à obra.

14.2.1 Ambiente

Laboratório padrão (Capítulo 5), com os papéis do projeto:

- **lab-r1** é o roteador do cliente (**cli-paoquente-r1**): restaure o snapshot **base-limpa** — o projeto começa do zero de verdade, sem aproveitar nada dos capítulos anteriores. Adaptador NAT (**ether3**) habilitado: ele fará o papel da ONU/concentrador do provedor;
- **lab-r2** é “um computador da padaria”: também em **base-limpa**, conectado pela **ether2** (rede **lab-a**).

Acesso inicial ao r1 pelo console da VM (equipamento zerado não tem gerência configurada — como na vida real).

14.2.2 Comandos passo a passo

Etapa 0 — base. Pelo console de **lab-r1**, o ritual do 5, condensado (adapte usuário e senha):

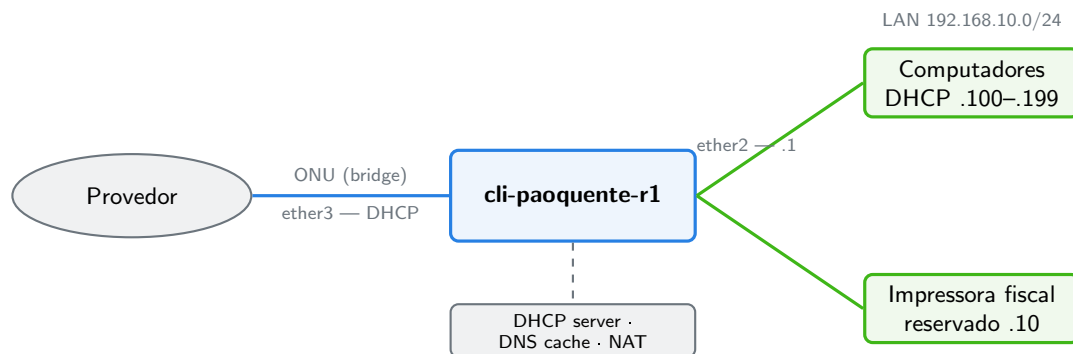


Figura 14.1: O projeto do capítulo: um roteador de borda de cliente corporativo, com WAN por DHCP e LAN completa.

```
/system identity set name=cli-paoquente-r1
/ip address add address=192.168.56.11/24 interface=ether1 comment="gerencia lab"
/user add name=vitor group=full password=Lab!Segura2026 comment="tecnico"
/user disable admin
/ip service disable telnet,ftp,www,api,api-ssl
/ip service set ssh address=192.168.56.0/24
/ip service set winbox address=192.168.56.0/24
/system clock set time-zone-name=America/Sao_Paulo
```

Daqui em diante, trabalhe por SSH (`ssh vitor@192.168.56.11`) — e lembre do Safe Mode (`Ctrl+X`) como cinto.

Etapas 1 — WAN. O link do provedor entrega endereço por DHCP:

```
/ip dhcp-client add interface=ether3 use-peer-dns=no comment="WAN via provedor"
/ip dhcp-client print
```

Espere `bound` e valide a camada: `/ping 8.8.8.8 count=3 do roteador` — a WAN funciona antes de existir LAN. (Repare no `use-peer-dns=no`: a ordem de serviço pede cache DNS com upstream nosso — um dono da verdade, Capítulo 12.)

Etapas 2 — LAN. O gateway da padaria:

```
/ip address add address=192.168.10.1/24 interface=ether2 comment="LAN cliente"
```

Etapas 3 — DHCP server. Pool (repare: `.100-.199`, deixando o resto fixo), `network` e `server` — e o NTP aproveitando a WAN nova:

```
/ip pool add name=pool-lan ranges=192.168.10.100-192.168.10.199
/ip dhcp-server network add address=192.168.10.0/24 gateway=192.168.10.1 dns-server=192.16
/ip dhcp-server add name=dhcp-lan interface=ether2 address-pool=pool-lan lease-time=1h
/system ntp client set enabled=yes
/system ntp client servers add address=a.st1.ntp.br
```

Etapa 4 — DNS. Upstreams e o modo servidor para a LAN:

```
/ip dns set servers=1.1.1.1,8.8.8.8 allow-remote-requests=yes
```

Etapa 5 — NAT. A máscara na fronteira:

```
/ip firewall nat add chain=srcnat out-interface=ether3 action=masquerade comment="NAT LAN-
```

Etapa 6 — o cliente chega. Em lab-r2 (fazendo papel de computador):

```
/system identity set name=pc-padaria
/ip dhcp-client add interface=ether2
/ip dhcp-client print
```

Recebeu 192.168.10.1xx? Então a prova final, na ordem das camadas — gateway, internet por IP, internet por nome:

```
/ping 192.168.10.1 count=2
/ping 8.8.8.8 count=2
/ping google.com count=2
```

Três pings limpos = serviço entregue. Faltava a reserva da impressora: em lab-r1, fixe o lease do “computador” (na vida real seria o MAC da impressora) no .10:

```
/ip dhcp-server lease print
/ip dhcp-server lease make-static numbers=0
/ip dhcp-server lease set 0 address=192.168.10.10
```

Em lab-r2, remova (/ip dhcp-client release numbers=0) e confirme que veio o .10.

Etapa 7 — documentar. Em lab-r1:

```
/system backup save name=cli-paoquente-r1-entrega
/export file=cli-paoquente-r1-entrega
```

E, do seu computador, **para fora** (3):

```
scp vitor@192.168.56.11:cli-paoquente-r1-entrega.backup .
scp vitor@192.168.56.11:cli-paoquente-r1-entrega.rsc .
```

Abra o `.rsc`: ele é o projeto inteiro em uma página — e o gabarito para o próximo cliente igual.

14.2.3 Verificação

A vistoria de entrega, em camadas (decore esta ordem — ela é o seu roteiro de diagnóstico para sempre):

1. **Enlace:** `/interface print` no `r1` — `ether2` e `ether3` com flag `R`;
2. **Endereço:** `/ip address print` — gerência, LAN `.1` e WAN dinâmica (`D`);
3. **Rota:** `/ip route print` — conectadas `DAd` + padrão `DAd` via provedor;
4. **Serviços:** `bound` no `dhcp-client`; `lease` do “`pc`” no `.10` (estático); `/ip dns print` com `upstreams` e `allow-remote-requests=yes`;
5. **Tradução:** regra de `masquerade` presente; `/ip firewall connection print` mostrando conexões do `192.168.10.10` traduzidas;
6. **Experiência do cliente:** os três pings do `lab-r2` (`gateway` → `IP` → `nome`) sem perdas;
7. **Documentação:** `.backup` e `.rsc` no seu computador.

14.2.4 Desafios

1. A padaria abriu uma filial — mesma ordem de serviço, mas LAN `192.168.20.0/24`. Liste **apenas** os comandos que mudam em relação ao projeto (e note quantos são: essa é a força do padrão).
2. O dono quer WiFi para os clientes **separado** da rede do caixa. Com as ferramentas deste volume (sem VLANs ainda — Volume 4), como você atenderia usando a `ether4` do roteador? Esboce os comandos das etapas 2–5 para essa segunda LAN (`192.168.99.0/24`) e diga o que essa solução **não** separa de verdade.
3. Simule o chamado clássico: em `lab-r1`, desabilite o `dhcp-client` da WAN (`/ip dhcp-client disable 0`) e, “sem saber disso”, diagnostique a partir do sintoma do cliente (“caiu a internet”) usando a vistoria em camadas. Em que camada o defeito aparece, e quais camadas continuam OK? Reabilite ao final.
4. O `.rsc` exportado serve de gabarito, mas aplicá-lo cru num equipamento novo tem **duas** lacunas (pense no que o `export` não carrega —
3) e **um** risco (pense no que é dinâmico no projeto). Quais?

Solução dos desafios

1. Mudam apenas os comandos com o endereço da LAN — identidade e cinco linhas:

```

/system identity set name=cli-paoquente-r2
/ip address add address=192.168.20.1/24 interface=ether2 comment="LAN cliente"
/ip pool add name=pool-lan ranges=192.168.20.100-192.168.20.199
/ip dhcp-server network add address=192.168.20.0/24 gateway=192.168.20.1 dns-server=192.
/ip dhcp-server lease set 0 address=192.168.20.10

```

Todo o resto (usuários, serviços, DNS, NAT, NTP) é idêntico — é isso que um `.rsc`-gabarito bem feito captura.

2. Segunda LAN na `ether4`: repetir as etapas com `address=192.168.99.1/24` `interface=ether4`, pool/network próprios, um segundo `/ip dhcp-server` na `ether4` — DNS e NAT existentes já atendem (o masquerade casa por `out-interface`, não por origem). O que **não** separa: tráfego entre as duas LANs — o roteador roteia entre elas livremente; um notebook no WiFi dos clientes alcança a impressora do caixa. Separação de verdade exige **firewall entre as redes** (Volume 3) — e, em escala, VLANs (Volume 4).

3. A vistoria denuncia na camada 2–3 da **WAN**: enlace OK (interfaces R), mas `/ip address print` perdeu o endereço D da `ether3` e `/ip route print` perdeu a rota padrão DAd — enquanto a LAN continua perfeita (cliente pega IP, pinga o gateway). Sintoma no cliente: “pego WiFi mas não navego”; no roteador: `bound` sumiu do `dhcp-client`. Camadas OK: 1 (enlaces), LAN inteira; defeito: endereço/rota da WAN. É o padrão de raciocínio que separa “trocar cabo por desespero” de diagnóstico.

4. Lacunas do export: (a) **senhas de usuários** não vêm — o `/user add` precisa ser refeito (ou mantido num `.rsc`-gabarito escrito à mão); (b) itens **dinâmicos** não são configuração: o lease estático da impressora vem, mas o `bound` da WAN depende do provedor real. O risco: o export carrega o `allowed-address=192.168.56.0/24` dos serviços — aplicado num cliente sem essa rede de gerência, você se tranca para fora no momento do import. Gabarito se **lê e adapta**, não se despeja.

14.3 Resumo

- Projeto profissional = **ordem de serviço** → **plano sequenciado** → **execução** → **vistoria** → **documentação** (Tabela 14.1) — nunca comandos soltos de memória.
- A sequência do roteador de borda: base segura → WAN (`dhcp-client`) → LAN (endereço) → DHCP server (pool menor que a rede + reservas) → DNS (um dono da verdade, cache para a LAN) → NAT (na fronteira) — validando **cada camada antes da seguinte**.
- A vistoria em camadas (enlace → endereço → rota → serviços → tradução → experiência) é o mesmo roteiro do **diagnóstico** — configurar bem é aprender a diagnosticar.
- Entrega termina com `.rsc` e `.backup` **fora** do equipamento — e o `.rsc` de hoje é o gabarito (adaptado, nunca despejado) do cliente de amanhã.

Fecha-se o Volume 2: você domina o terminal e constrói um roteador de borda completo. Mas ele está **nu**: DNS aberto na WAN, WinBox exposto, nada filtrando o forward. O

Volume 3 veste a armadura — firewall, do connection tracking ao hardening completo.

Bibliografia do capítulo

Este capítulo integra os anteriores; as referências são as deles — em especial a documentação oficial (MikroTik, 2026a) (páginas *First Time Configuration* e *Securing your router*, que espelham nossa sequência). Discher (2016) traz projetos guiados semelhantes. As referências completas, em ABNT, estão no apêndice [Referências](#).

Parte III

Volume 3 — Firewall

15 Connection tracking

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que é o **connection tracking** (conntrack) e por que ele é a fundação de todo o firewall do RouterOS;
- Classificar pacotes pelos quatro **estados** de conexão — **new**, **established**, **related** e **invalid** — e prever qual estado um pacote receberá;
- Ler a tabela de conexões (`/ip firewall connection`): protocolos, timeouts, flags e o que cada coluna conta;
- Explicar como o conntrack enxerga protocolos “sem conexão” (UDP, ICMP) e protocolos com canais secundários (FTP);
- Dimensionar e ajustar o conntrack: tamanho máximo da tabela, timeouts e o custo de memória/CPU em equipamentos de provedor.

15.1 A memória que muda tudo

No Capítulo 13 você viu o NAT anotar cada tradução numa tabela e consultá-la na volta. Aquela tabela não pertence ao NAT: ela é o **connection tracking** — o subsistema do RouterOS que observa **todo** pacote roteado e mantém um registro vivo de cada conversa em andamento. O NAT é apenas seu primeiro cliente; o segundo, e mais importante, é o firewall que construiremos neste volume.

Por quê? Porque um firewall sem memória é obrigado a julgar cada pacote isoladamente — e pacotes isolados mentem. Um pacote vindo da internet para o seu cliente pode ser um ataque **ou** a resposta legítima de um site que o cliente acabou de acessar; olhando só o pacote, é impossível distinguir. Com memória, a pergunta muda de “este pacote parece bom?” para “**este pacote pertence a uma conversa que alguém de dentro começou?**” — uma pergunta que a tabela de conexões responde em microssegundos.

Esse é o conceito de **firewall com estado** (*stateful*), e é o modelo de praticamente toda regra que escreveremos: julgamos **conexões** na abertura, e os pacotes seguintes herdam o julgamento.

💡 Analogia para guardar

O conntrack é a **recepcionista com caderno**: a primeira vez que alguém sai para uma reunião, ela anota (conexão **new**). Quando a resposta chega, ela olha o caderno — “ah, é da reunião do fulano” (**established**) — e deixa passar sem interrogatório. Entregador que chega junto com a encomenda esperada (**related**) também entra. Estranho que aparece dizendo “estou voltando de uma reunião” que não está no caderno (**invalid**)? Porta.

15.2 Os quatro estados

Cada pacote que o conntrack examina recebe um **connection state** — e é sobre esses estados que as regras do firewall operam:

Tabela 15.1: Os estados de conexão

Estado	Significado	Tratamento típico
new	primeiro pacote de uma conversa nova	julgar com as regras
established	pertence a uma conversa já registrada	aceitar cedo
related	conversa nova, mas filha de uma registrada	aceitar cedo
invalid	não pertence a nada e não abre nada válido	descartar sempre

Vale detalhar os dois menos óbvios:

- **related**: alguns protocolos abrem canais secundários — o FTP clássico negocia a transferência numa conexão e move os dados por outra; o ICMP de erro (“destino inalcançável”) é resposta a uma conexão que já existe. O conntrack usa *helpers* (em `/ip firewall service-port`) para reconhecer esses filhotes e marcá-los **related** — sem isso, seriam **new** vindos de fora e morreriam no filtro;
- **invalid**: o estado do lixo — pacotes TCP no meio de uma conversa que a tabela não conhece (sobras de conexão expirada, pacotes fora de ordem extremos, varreduras e ataques). **invalid não** é o mesmo que **new**: um SYN legítimo é **new**; um ACK órfão é **invalid**. A regra universal: descartar.

A consequência prática, que será o esqueleto de todas as nossas chains no Capítulo 16:

```
accept established,related    # o grosso do tráfego, barato
drop invalid                  # o lixo
# ... e só o que for NEW é julgado pelas regras seguintes
```

Essa ordem não é estética — é **desempenho**. Numa rede de provedor, mais de 95% dos pacotes pertencem a conexões já estabelecidas: eles casam com a primeira regra e saem do firewall imediatamente, sem atravessar a lista inteira. As regras caras julgam apenas o primeiro pacote de cada conversa.

15.3 Lendo a tabela

A tabela mora em `/ip firewall connection` — você já a espiou no NAT; agora com olhos completos:

```
/ip firewall connection print detail
```

```
[vitor@lab-r1] > /ip firewall connection print detail
Flags: E - expected, S - seen-reply, A - assured, C - confirmed, D - dying,
F - fasttrack, s - srcnat, d - dstnat
0    SAC protocol=tcp src-address=192.168.10.105:51234
      dst-address=142.250.79.78:443 reply-src-address=142.250.79.78:443
      reply-dst-address=10.0.2.15:51234 tcp-state=established
      timeout=23h59m58s orig-packets=52 orig-bytes=8402
      repl-packets=48 repl-bytes=61003
```

O que cada pedaço conta:

- **src/dst-address** — a conversa como o cliente a iniciou; **reply-*** — como ela volta (denunciando NAT no caminho, flag **s**);
- **tcp-state=established** + flags **S** (viu resposta) e **A** (*assured*: tráfego nos dois sentidos consolidado) — uma conexão saudável;
- **timeout** — o prazo de vida sem tráfego novo. Cada protocolo/estado tem o seu: TCP estabelecido dura horas; UDP, segundos a minutos; TCP semiaberto (**syn-sent**), poucos segundos. Tráfego renova o prazo; silêncio deixa a entrada morrer;
- **contadores (orig/repl-packets/bytes)** — quanto já passou em cada sentido: ferramenta de diagnóstico de primeira (“a conexão existe, mas só tem tráfego de ida...”).

E os protocolos “sem conexão”? O conntrack finge que têm: um par pergunta/resposta **UDP** vira uma “conexão” com timeout curto; um **ICMP** echo request/reply idem. É o que permite ao firewall stateful tratar DNS, NTP e ping com a mesma lógica do TCP — e é por isso que o `/ping` do laboratório de NAT apareceu na tabela como **protocol=icmp**.

15.4 O custo da memória

Nada é de graça: cada entrada consome memória, e cada pacote novo consome CPU para busca/criação na tabela. Três números para operar com consciência em `/ip firewall connection tracking`:

```
/ip firewall connection tracking print
```

- **max-entries** — o teto da tabela (calculado da RAM por padrão). Tabela **cheia** = **conexões novas descartadas**: o sintoma é “internet intermitente” com CPU baixa — clássico em concentrador PPPoE subdimensionado ou sob ataque de flood de conexões;
- **timeouts ajustáveis** — `udp-timeout`, `tcp-established-timeout` etc. Encurtar timeouts derruba entradas ociosas mais cedo (alívio em tabela cheia), ao custo de matar conversas legítimas muito quietas;
- **enabled=auto** — o `conntrack` liga sozinho quando existe regra que precisa dele (NAT, filter por estado). Desligá-lo (`no`) transforma o equipamento num roteador “burro e rápido” — sem NAT, sem firewall stateful: faz sentido em roteador de **core** puro que só encaminha (Volume 8), jamais na borda. E há um meio-termo importante, o **RAW/notrack**, que isenta tráfego escolhido a dedo — assunto do

4.

! O conceito mais importante do capítulo

O firewall do RouterOS não julga pacotes — julga **conversas**. O `conntrack` dá a cada pacote um estado (`new/established/related/invalid`), e o firewall eficiente decide caro **uma vez por conexão** (no `new`) e barato em todos os demais pacotes (`established,related` aceitos cedo, `invalid` no lixo). Quem entende a tabela de conexões lê o firewall inteiro; quem não entende, coleciona regras por superstição.

15.5 Laboratório

15.5.1 Ambiente

O roteador de borda pronto do Capítulo 14: `lab-r1` (`cli-paoquente-r1`, WAN NAT + LAN + DHCP + DNS + NAT) e `lab-r2` (`pc-padaria`, cliente DHCP). Se não guardou, remonte pelas etapas 0–6 de lá (10 minutos — o gabarito `.rsc` ajuda). Vamos **observar** o `conntrack`; o filtro começa no próximo capítulo.

15.5.2 Comandos passo a passo

Passo 1 — a tabela em repouso. Em `lab-r1`:

```
/ip firewall connection print
```

Poucas linhas (talvez o seu SSH da gerência, NTP). Anote quantas.

Passo 2 — **nascimento de uma conexão**. Em `lab-r2`, dispare um ping contínuo:

```
/ping 8.8.8.8
```

Em **lab-r1**, encontre-a e examine:

```
/ip firewall connection print detail where protocol=icmp
```

Observe: `src-address=192.168.10.x`, `reply-dst-address` com o IP da WAN (o NAT, flag `s`), timeout de ~10 s **renovando-se** a cada pacote (rode o print duas vezes e compare).

Passo 3 — morte por timeout. Pare o ping no r2 (Ctrl+C) e, no r1, repita o print a cada poucos segundos:

```
/ip firewall connection print where protocol=icmp
```

O `timeout` decresce até a entrada sumir — silêncio mata a conexão, e você assistiu.

Passo 4 — TCP conta mais história. Em **lab-r2**, abra uma conexão TCP de verdade (o fetch baixa uma página):

```
/tool fetch url=https://mikrotik.com keep-result=no
```

Em **lab-r1**, rápido (a conexão fecha ao terminar):

```
/ip firewall connection print detail where protocol=tcp and dst-port=443
```

Compare com o ICMP: `tcp-state=established`, timeout de **horas**, flags `SAC`. Depois que o fetch termina, o RouterOS de lá fecha a conexão educadamente — e a entrada some rápido (estados de fechamento têm timeout curto).

Passo 5 — os limites da casa. Ainda em **lab-r1**:

```
/ip firewall connection tracking print
```

Localize `max-entries` e `total-entries`. Com 256 MB de RAM, o CHR sustenta dezenas de milhares de entradas — guarde a razão `total/max-entries` como um dos sinais vitais de um concentrador.

15.5.3 Verificação

1. Você viu a mesma conexão ICMP com o timeout **renovando** (tráfego) e depois **decrecendo até sumir** (silêncio);
2. A entrada TCP mostrou `tcp-state=established`, flags `SAC` e timeout longo — e você sabe dizer por que difere do ICMP;
3. As flags `s` (`srcnat`) apareceram nas conexões da LAN — a herança do Capítulo 13 explicada;
4. Você sabe onde olhar `max-entries/total-entries` e qual o sintoma de tabela cheia.

15.5.4 Desafios

1. Um pacote TCP ACK chega da internet e a tabela não tem nada sobre aquela conversa. Que estado ele recebe? E um TCP SYN nas mesmas condições? Por que a diferença importa para o firewall?
2. Em `lab-r2`, rode `/resolve mikrotik.com` e ache a conexão correspondente na tabela do `r1`. Protocolo? Timeout? Por que o `conntrack` trata assim um protocolo que “não tem conexão”?
3. Concentrador com 4 GB de RAM, 8 mil assinantes, média de 40 conexões por assinante: estime as entradas em uso. Num flood que abre 100 conexões UDP **por segundo por assinante** infectado (200 infectados), quanto tempo até encher uma tabela de 1M de entradas (UDP timeout 30 s — considere o regime estacionário)? O que o operador vê?
4. `connection tracking enabled=no` deixaria o `lab-r1` mais rápido. O que quebraria **imediatamente** na nossa topologia — e em que tipo de equipamento de provedor essa troca vale a pena?

Solução dos desafios

1. ACK órfão → **invalid** (não está na tabela e não é um começo válido de conversa). SYN → **new** (é exatamente como conversas TCP começam). A diferença é o firewall inteiro: **new** merece julgamento (talvez seja um cliente legítimo abrindo conexão); **invalid** nunca é legítimo em operação normal — descarta-se sem dó. Tratar os dois igual — para qualquer um dos lados — cria ou um buraco ou um firewall que quebra conexões válidas.
2. `protocol=udp dst-port=53` (a consulta ao upstream aparece também, vinda do próprio `r1`). Timeout curto (~10 s pós-resposta). O `conntrack` inventa a “conexão” UDP pareando pergunta e resposta por endereços/portas — é o que permite à resposta do DNS voltar através do NAT e do firewall stateful sem regra especial de entrada.
3. Regime normal: $8\,000 \times 40 = \mathbf{320\,mil}$ entradas — folga numa tabela de 1M. Flood: $200 \times 100 = 20\,mil$ conexões novas/s; com timeout de 30 s, o regime estacionário adiciona $20\,000 \times 30 = \mathbf{600\,mil}$ entradas — somadas às 320 mil, a tabela satura (920 mil — 1M) em ~30 s de ataque. O operador vê `total-entries` cravado no teto, clientes reclamando de conexões novas falhando (páginas que não abrem) com CPU e banda aparentemente normais. Remédios: RAW/notrack e rate-limit de **new** — 4 e Capítulo 21.
4. Quebra na hora: o **NAT** (masquerade depende da tabela) — a LAN inteira para de navegar; e qualquer filtro por estado que existisse. Vale a pena em roteador **interno de core/trânsito**: só encaminha pacotes entre redes públicas, sem NAT e sem stateful (a proteção mora nas bordas) — aí o `conntrack` é custo puro, e desligá-lo libera CPU. Nunca na borda, nunca no concentrador.

15.6 Resumo

- O **connection tracking** é a memória do RouterOS: toda conversa roteada vira uma entrada com estado, timeout e contadores — NAT e firewall stateful são consumidores dela.
- Estados (Tabela 15.1): **new** (julgado), **established/related** (aceitar cedo — é o grosso do tráfego), **invalid** (descartar sempre; e **invalid new**).
- UDP e ICMP ganham “conexões” por pareamento pergunta/resposta com timeouts curtos; helpers marcam canais secundários como **related**.
- A tabela conta histórias: **reply-*** denuncia NAT, **tcp-state** e flags dizem a saúde, contadores dizem o volume, timeout diz o futuro.
- Custo real: memória por entrada e CPU por conexão nova — monitore **total/max-entries**; tabela cheia = internet intermitente misteriosa.

No próximo capítulo, os trilhos por onde as regras correm: as **chains** `input`, `forward` e `output`, e a lógica de avaliação que decide o destino de cada pacote.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip firewall connection` e `connection tracking` (estados, timeouts, helpers em `service-port`). O modelo stateful e o conntrack derivam do Netfilter do Linux, cuja documentação conceitual complementa; Tanenbaum; Feamster; Wetherall (2021) cobre TCP/UDP/ICMP por baixo de tudo isso. As referências completas, em ABNT, estão no apêndice [Referências](#).

16 Chains e a lógica do firewall

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Dizer, para qualquer pacote, em qual **chain** ele será julgado — **input**, **forward** ou **output** — usando uma única pergunta;
- Explicar a avaliação **de cima para baixo, primeira regra que casa decide** — e por que a ordem das regras é metade do firewall;
- Usar as **actions** fundamentais (**accept**, **drop**, **reject**, **log**, **jump**) e escolher entre **drop** e **reject** com critério;
- Montar o **esqueleto stateful** de uma chain (**established/related** → **invalid** → julgamento do **new**) e explicar cada linha;
- Usar contadores e a action **log** para ver regras trabalhando — inclusive no modo “ensaio”, antes de qualquer bloqueio real.

16.1 Três tribunais, uma pergunta

O firewall de filtro vive em `/ip firewall filter`, e cada regra pertence a uma **chain** (corrente/fila de julgamento). As três nativas separam os pacotes por uma pergunta só: **qual a relação do pacote com o roteador?**

- **input** — o pacote é **para** o roteador (destino: um IP dele). O SSH da gerência, o DNS que a LAN consulta, o ping no gateway — tudo **input**;
- **forward** — o pacote **atravessa** o roteador (origem e destino são de terceiros). A navegação dos clientes é 99% disso;
- **output** — o pacote **nasce** no roteador (ele mesmo consultando o upstream de DNS, o NTP, o ping que você dispara do terminal).

Quem decide a chain não é você — é o **destino do pacote**, avaliado logo após a decisão de roteamento (Figura 16.1):

A separação é a sua primeira ferramenta de projeto: **proteger o roteador** é escrever regras no **input** (próximo capítulo); **proteger e policiar os clientes** é escrever no **forward** (Capítulo 18). Misturar os dois é o erro de iniciante mais comum — regras de cliente no **input** não fazem nada, e regras de roteador no **forward** protegem a caixa errada.

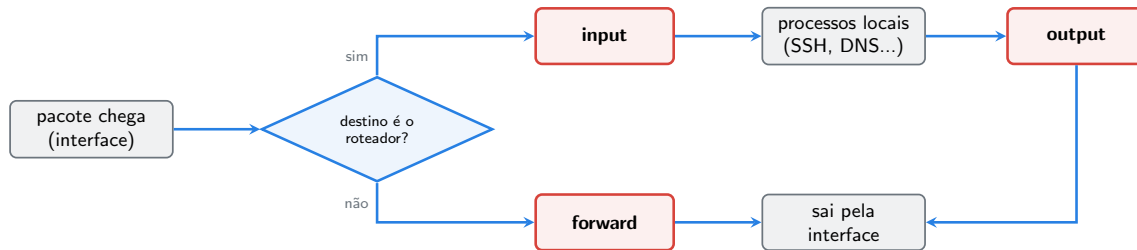


Figura 16.1: O caminho do pacote: a decisão de roteamento distribui cada um à sua chain — para o roteador (**input**), através dele (**forward**) ou nascido nele (**output**).

16.2 A lógica: de cima para baixo, primeira que casa decide

Dentro de cada chain, as regras formam uma **lista ordenada**. Cada pacote da chain percorre a lista do topo para baixo; na primeira regra cujas condições (*matchers*) ele satisfaz, a **action** é executada — e, se ela for terminal (**accept**, **drop**, **reject**), o julgamento **acaba ali**. As regras seguintes nem são olhadas.

Duas consequências que valem um firewall inteiro:

1. **Ordem é semântica.** **drop** geral antes de **accept** específico = o **accept** nunca executa. As regras baratas e frequentes vão no topo (é o esqueleto stateful do 16); as exceções vêm **antes** das regras gerais que elas excetuam;
2. **O fim da lista é uma decisão.** Se nenhuma regra casar, o RouterOS **aceita** o pacote — a política padrão é permissiva. Um firewall de verdade termina, portanto, com uma regra explícita de **drop** do resto (“**drop all**”) — e tudo que deve viver, vive **acima** dela.

💡 Analogia para guardar

Uma chain é uma **fila de carimbos**: o pacote passa de mesa em mesa, e o primeiro fiscal cuja regra o descreve dá o carimbo final — entra ou lixo. Se chegar ao fim da fila sem carimbo, a porta está aberta (política padrão: aceitar). Por isso o último fiscal de um firewall sério carimba “lixo” em tudo que sobrou.

16.2.1 As actions

Tabela 16.1: Actions essenciais do filter

Action	Efeito	Quando usar
accept	deixa passar, fim do julgamento	tráfego aprovado
drop	descarta em silêncio	o padrão para lixo externo

Action	Efeito	Quando usar
reject	descarta e avisa a origem (ICMP/TCP reset)	redes internas, onde a resposta rápida ajuda
log	registra no log e continua para a próxima regra	diagnóstico e ensaio
jump	desvia para uma chain sua (jump-target)	organizar firewalls grandes
return	volta da chain desviada	fecha o ciclo do jump

Dois pares merecem nota:

- **drop × reject**: o **drop** silencia — quem sonda a sua WAN não descobre nem que existe um firewall (e cada pacote de resposta economizado importa sob ataque). O **reject** responde “recusado” — em rede **interna**, poupa usuários e aplicações de esperar timeouts. Regra de bolso: **fora, drop; dentro, reject quando a experiência importar**;
- **log não é terminal**: o pacote é registrado e **segue** para as regras seguintes. Isso permite o padrão mais valioso deste capítulo — o **ensaio**: coloque um **log** onde o futuro **drop** ficará, observe alguns dias o que ele pegaria e só então troque por **drop**. Firewall de produção se estreia sem surpresas. (A action **log** aceita **log-prefix="texto"** — use sempre: log sem prefixo é charada.)

16.2.2 Chains próprias: organização por jump

Além das três nativas, você cria chains suas e desvia tráfego para elas:

```
/ip firewall filter add chain=forward src-address=192.168.10.0/24 action=jump jump-target=
/ip firewall filter add chain=clientes protocol=tcp dst-port=25 action=drop comment="bloqu
/ip firewall filter add chain=clientes action=return
```

O pacote que casa com o jump é julgado pelas regras da chain **clientes**; um **return** (ou o fim da chain) devolve-o para onde parou na **forward**. Em firewalls de provedor com centenas de regras, chains temáticas (**da-wan**, **clientes**, **gerencia**) mantêm a lista legível — e mais rápida, pois grupos inteiros de regras só são percorridos por quem casa com o jump.

16.3 O esqueleto que você vai repetir para sempre

Juntando **conntrack** (16) + chains + ordem, nasce o esqueleto de **toda** chain de filtro bem construída:

```

/ip firewall filter
add chain=input action=accept connection-state=established,related comment="aceita conversas"
add chain=input action=drop connection-state=invalid comment="lixo do conntrack"
# ... daqui para baixo, só pacotes NEW chegam:
# regras que julgam QUEM PODE ABRIR conversa nova
# ...
add chain=input action=drop comment="drop all: o que nao foi aceito acima, morre"

```

Leia-o como narrativa: *conversas aprovadas passam barato (linha 1); lixo morre cedo (linha 2); pedidos novos são julgados um a um (miolo); e tudo que sobrar, morre (última linha)*. O miolo muda conforme a chain e o papel do equipamento — os dois próximos capítulos são exatamente sobre esses miolos.

! O conceito mais importante do capítulo

Firewall no RouterOS é responder três perguntas, nesta ordem: **em qual chain?** (qual a relação do pacote com o roteador), **em que posição?** (primeira que casa decide; baratas no topo, drop all no fim) e **com qual action?** (fora drop, dentro reject, ensaio com log). Quem domina as três perguntas escreve qualquer regra; quem não domina, copia listas da internet sem saber o que protegem.

16.4 Laboratório

16.4.1 Ambiente

O de sempre neste volume: **lab-r1** (borda completa do Capítulo 14) e **lab-r2** (cliente na LAN). Vamos montar o esqueleto do **input em modo ensaio** — nenhum bloqueio real ainda; o aperto de verdade é no próximo capítulo, com rede de segurança.

16.4.2 Comandos passo a passo

Passo 1 — o esqueleto, com ensaio no lugar do drop. Em **lab-r1**:

```

/ip firewall filter add chain=input action=accept connection-state=established,related comment="aceita conversas"
/ip firewall filter add chain=input action=drop connection-state=invalid comment="lixo do conntrack"
/ip firewall filter add chain=input action=log log-prefix="INPUT-NEW: " comment="ENSAIO: o que vem de fora"

```

Repare: onde um dia haverá julgamento + drop all, por ora há um **log** — o firewall observa e **não bloqueia nada** (política padrão aceita o que o log deixa passar).

Passo 2 — gerar tráfego e ler contadores. Em **lab-r2**, movimente a rede: `/ping 192.168.10.1 count=5` (input do r1!), `/ping google.com count=5` (forward + o DNS é input!). Em **lab-r1**:

```
/ip firewall filter print stats
```

```
[vitor@lab-r1] > /ip firewall filter print stats
Flags: X - disabled, I - invalid; D - dynamic
Columns: CHAIN, ACTION, BYTES, PACKETS
#  CHAIN  ACTION  BYTES  PACKETS
0  input  accept  12 480    156
1  input  drop      0      0
2  input  log      1 344    18
```

Os contadores são o eletrocardiograma do firewall: a regra 0 engorda a cada pacote de conversas vivas; a 2 conta só as **aberturas** (new). Zere-os e observe do zero quando quiser: `/ip firewall filter reset-counters-all`.

Passo 3 — ler o ensaio. Veja quem o log pegou:

```
/log print where message~"INPUT-NEW"
```

```
15:41:02 firewall,info INPUT-NEW: input: in:ether2 out:(unknown 0),
proto UDP, 192.168.10.105:44231->192.168.10.1:53, len 62
15:41:07 firewall,info INPUT-NEW: input: in:ether2 out:(unknown 0),
proto ICMP, 192.168.10.105->192.168.10.1, len 84
```

Cada linha é um pedido de conversa nova com o roteador: o DNS da LAN (porta 53), o ping no gateway, o seu SSH da gerência (`in:ether1`, porta 22)... Este é o **inventário real** do que o próximo capítulo precisa permitir antes do drop all.

Passo 4 — sentir a ordem. Prove que ordem é semântica: suba uma regra de log específica **abaixo** da genérica e veja que nunca casa:

```
/ip firewall filter add chain=input protocol=icmp action=log log-prefix="PING: " comment="
```

Pingue o r1 de novo (do r2) e confira: o contador da regra nova avança? Sim — porque **log não é terminal**: o pacote passou pela regra 2 (log genérico) e **continuou** até a 3. Agora troque o log genérico por um **accept** temporário e repita:

```
/ip firewall filter set 2 action=accept
/ip firewall filter reset-counters-all
```

Ping de novo: agora a regra **PING:** fica **zerada** — o **accept** da regra 2 é terminal e ninguém abaixo dela vê pacote algum. Ordem + terminalidade, demonstradas. Desfaça o experimento:

```
/ip firewall filter set 2 action=log
/ip firewall filter remove [find comment="teste de ordem"]
```

16.4.3 Verificação

1. `/ip firewall filter print` mostra o esqueleto: `accept established/related → drop invalid → log` ensaio;
2. Os contadores contam a história certa: regra 0 com milhares de pacotes, regra 1 quase zerada, log só com aberturas;
3. O `/log print` lista as conversas novas reais da sua rede (DNS, ping, SSH) — o inventário do próximo capítulo;
4. Você demonstrou que `log` deixa passar adiante e `accept` termina o julgamento (Passo 4).

16.4.4 Desafios

1. Classifique na chain certa: (a) cliente da LAN navegando; (b) você pingando 8.8.8.8 **do terminal do r1**; (c) um atacante tentando WinBox no IP público do r1; (d) o r2 consultando o DNS do r1; (e) o r1 consultando o upstream 1.1.1.1.
2. No esqueleto, por que o `drop invalid` vem **depois** do `accept established/related`, e não antes? Há diferença prática?
3. Escreva (sem aplicar) a regra que registraria no log, com prefixo **WAN-NEW:**, apenas as tentativas de conexão **novas** chegando pela **ether3** — e diga em que posição da chain ela deveria entrar.
4. Um colega propõe terminar o input com `action=reject` “para o atacante desistir mais rápido”. Argumente contra, com dois motivos.

Solução dos desafios

1. (a) **forward**; (b) **output** (nasce no roteador); (c) **input** (destino é o roteador); (d) **input** (o serviço DNS é do roteador); (e) **output**. O par (d)/(e) é o mesmo “pedido de DNS” visto de dois lados — ótimo teste de compreensão.

2. Funcionalmente, quase nada muda (um pacote não é os dois estados ao mesmo tempo); a razão é **desempenho**: `established/related` é a esmagadora maioria dos pacotes — casando primeiro, eles saem da chain na regra 1. O `invalid` é raro; colocá-lo antes faria todo pacote bom pagar uma verificação inútil. Regra frequente em cima: o princípio de ordenação em uma linha.

3.

```
/ip firewall filter add chain=input in-interface=ether3 connection-state=new action=log
```

Posição: **depois** do `accept established/related` e do `drop invalid` (para só ver aberturas reais), **antes** de qualquer `accept` específico — no nosso esqueleto atual, junto do log de ensaio. É o ensaio focado na fronteira.

4. (a) **reject responde** — cada pacote de sonda gera um pacote seu de volta: sob flood, você paga banda e CPU para ajudar o atacante (e pode virar refletor contra origens forjadas); (b) a resposta **confirma que há um equipamento vivo** e filtrado ali, convidando a sondagem mais cuidadosa. O silêncio do **drop** não custa nada e não informa nada — na WAN, é a escolha certa. (O reject tem lugar: dentro de casa, poupando timeouts.)

16.5 Resumo

- Três chains, uma pergunta: **para** o roteador (**input**), **através** (**forward**), **dele** (**output**) — quem decide é o destino do pacote (Figura 16.1).
- Avaliação **top-down**, **primeira que casa decide**; actions terminais encerram; sem casar nada, a política padrão **aceita** — por isso o firewall sério termina em drop all explícito.
- Actions (Tabela 16.1): fora **drop** (silêncio), dentro **reject** (experiência), **log** para diagnóstico e **ensaio** (não é terminal!), **jump/return** para organizar em chains temáticas.
- O esqueleto universal: accept **established,related** → drop **invalid** → julgamento dos **new** → drop all.
- Contadores (**print stats**) e logs com prefixo são os olhos do firewall — e o modo ensaio (log no lugar do futuro drop) é como se estreia firewall sem derrubar produção.

No próximo capítulo, o esqueleto ganha músculos: fechamos o **input** de verdade — serviços, gerência, ICMP e o drop all — sem perder o próprio acesso no processo.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) descreve o packet flow completo do RouterOS (diagramas de fluxo com todas as chains e etapas) e a referência de **/ip firewall filter** (matchers e actions). Discher (2016) dedica capítulos didáticos à construção de chains. As referências completas, em ABNT, estão no apêndice [Referências](#).

17 Protegendo o roteador

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Aplicar a filosofia **default deny** à **chain input**: tudo fechado, e cada abertura é uma decisão documentada;
- Organizar interfaces em **interface lists** (WAN, LAN, **gerencia**) e escrever regras que sobrevivem a mudanças de topologia;
- Inventariar os serviços que o roteador **precisa** servir (gerência, DNS da LAN, DHCP, ICMP) e liberá-los com o mínimo de exposição;
- Definir uma política de **ICMP** sensata — nem “bloqueia tudo”, nem porta aberta a flood;
- Fechar o **input** com **drop all** sem se trancar para fora — usando Safe Mode, ensaio com log e teste de cada acesso antes do aperto final.

17.1 Default deny: a inversão que muda tudo

Até aqui, o **input** do nosso roteador aceita tudo que as regras não descartam — a política permissiva de fábrica. Este capítulo faz a inversão profissional: **negar por padrão** e abrir apenas o necessário. A diferença filosófica é enorme:

- No modelo permissivo, cada serviço esquecido aberto é uma vulnerabilidade — e você descobre pelo pior caminho (o DNS aberto do Capítulo 12 virando refletor de ataque, o WinBox exposto sendo forçado);
- No modelo default deny, cada porta aberta é uma **decisão consciente**, com dono e comentário. O que você esqueceu fica **fechado** — esquecer vira seguro, não risco.

O preço é o cuidado: fechar o **input** de um roteador remoto é serrar o galho em que se está sentado. Por isso este capítulo é tanto sobre **método** (ensaio → liberações → teste → aperto) quanto sobre regras.

17.2 Interface lists: regras que sobrevivem à topologia

Escrever `in-interface=ether3` funciona — até o dia em que a WAN vira um PPPoE, ganha uma segunda operadora ou muda de porta. A ferramenta profissional são as **interface lists**

(que você conheceu no 5): grupos nomeados de interfaces, usados como matcher:

```
/interface list add name=WAN comment="interfaces de fronteira"  
/interface list add name=LAN comment="interfaces de clientes"  
/interface list member add list=WAN interface=ether3  
/interface list member add list=LAN interface=ether2
```

Daqui em diante, as regras dizem **in-interface-list=WAN** — e quando a topologia mudar, você atualiza a **lista**, não o firewall. Num parque de centenas de equipamentos com o mesmo firewall-base, essa indireção é o que torna o padrão aplicável a todos (o hardening do Capítulo 21 depende disso).

A lista **gerencia** do 5 completa o trio. Nossa convenção de projeto, deste capítulo até o fim do livro:

Tabela 17.1: As três interface lists do projeto

Lista	Contém	Papel no firewall
WAN	interfaces de fronteira (operadora, internet)	de onde nada abre conversa com o roteador
LAN	interfaces de clientes/serviços	abre só os serviços servidos (DNS, DHCP)
gerencia	interfaces (e redes) da equipe	única origem de SSH/WinBox

17.3 O inventário: o que o input precisa aceitar

O log de ensaio do Capítulo 16 respondeu empiricamente; agora o inventário raciocinado do nosso roteador de borda:

1. **Gerência** — SSH e WinBox, **apenas** da lista **gerencia** (no laboratório, a **ether1**; na vida real, a rede de gerência do provedor). Note a defesa em camadas: o **allowed-address** do **/ip service**
(5) continua lá — o firewall é a segunda tranca;
2. **DNS para a LAN** — o roteador é o resolvidor dos clientes (Capítulo 12): UDP e TCP 53, **da LAN apenas**. É exatamente a regra que fecha o buraco do open resolver: da WAN, ninguém alcança a porta 53;
3. **DHCP na LAN** — detalhe que derruba iniciantes: o pedido DHCP (Discover) chega por broadcast **antes de o cliente ter IP**, e é julgado no **input**. Sem liberar UDP 67–68 da LAN, o DHCP server para de servir — e o defeito (“ninguém pega IP depois do firewall!”) é dos que fazem técnico duvidar da própria sanidade;
4. **ICMP** — a seguir, com política própria;

5. **Nada mais.** NTP, upstream de DNS, atualizações — tudo isso o roteador **inicia** (output → volta como established). Não se abre porta para resposta; o conntrack cuida (16).

17.3.1 A política de ICMP

Há duas superstições opostas sobre ping: “bloqueie tudo” (esconde nada e quebra diagnóstico — e pior, quebra *path MTU discovery*, causando defeitos bizarros de páginas que não carregam) e “libere tudo” (porta aberta a flood). O equilíbrio de provedor:

```
add chain=input protocol=icmp limit=20,10:packet action=accept comment="ICMP com teto"
```

Aceita ICMP — ping, os erros de que o TCP depende — mas com **limite de taxa**: 20 pacotes/s com rajada de 10. Diagnóstico funciona; flood de ICMP encontra teto. (Quem quiser esconder o roteador de ping da WAN pode restringir por lista — mas mantenha o ICMP de **erro** vivo, ou pagará em chamados estranhos.)

17.4 A sequência completa

O input final do nosso roteador de borda — o esqueleto do Capítulo 16 com o miolo preenchido:

```
/ip firewall filter
add chain=input action=accept connection-state=established,related comment="conversas em a
add chain=input action=drop connection-state=invalid comment="lixo do conntrack"
add chain=input action=accept in-interface-list=gerencia comment="gerencia: SSH/WinBox/tud
add chain=input action=accept protocol=icmp limit=20,10:packet comment="ICMP com teto"
add chain=input action=accept in-interface-list=LAN protocol=udp dst-port=53 comment="DNS
add chain=input action=accept in-interface-list=LAN protocol=tcp dst-port=53 comment="DNS
add chain=input action=accept in-interface-list=LAN protocol=udp dst-port=67-68 comment="D
add chain=input action=drop comment="drop all"
```

Oito regras. Leia de novo, linha a linha, e note as escolhas: a gerência aceita **tudo** (é rede de confiança — e se ela for comprometida, o firewall é o menor dos problemas); a LAN só alcança os **serviços que o roteador serve a ela**; a WAN não aparece em accept nenhum — para ela, só resta o drop all. O silêncio é a resposta.

! O conceito mais importante do capítulo

No input, a pergunta de projeto é: “**quem precisa abrir conversa com o roteador, e para quê?**” — e a resposta vira uma lista curta de accepts (gerência; serviços servidos à LAN; ICMP com teto) sobre o esqueleto stateful, selada por um drop all. Da WAN, nenhum accept: roteador de borda não conversa com estranhos — ele apenas

encaminha.

17.5 Laboratório

17.5.1 Ambiente

lab-r1 (borda completa) e **lab-r2** (cliente na LAN), continuando do capítulo anterior (o esqueleto com log de ensaio já aplicado). Trabalhe por SSH **da gerência** — e o laboratório inteiro dentro do **Safe Mode** (**Ctrl+X**): é o exercício definitivo do cinto de segurança.

17.5.2 Comandos passo a passo

Passo 0 — Safe Mode. Na sessão SSH do r1, **Ctrl+X**. Confirme o **<SAFE>** no prompt. Se qualquer passo der errado a ponto de derrubar seu acesso, a reversão automática desfaz tudo.

Passo 1 — as listas. Formalize a topologia:

```
/interface list add name=WAN comment="fronteira"  
/interface list add name=LAN comment="clientes"  
/interface list member add list=WAN interface=ether3  
/interface list member add list=LAN interface=ether2
```

(A lista **gerencia** já existe desde o 5, com a **ether1**.)

Passo 2 — o miolo, acima do log. As liberações entram **entre** o **drop invalid** e o log de ensaio (use **place-before** apontando para o número da regra de log — confira com **print**):

```
/ip firewall filter print  
/ip firewall filter add chain=input action=accept in-interface-list=gerencia comment="gere  
/ip firewall filter add chain=input action=accept protocol=icmp limit=20,10:packet comment  
/ip firewall filter add chain=input action=accept in-interface-list=LAN protocol=udp dst-p  
/ip firewall filter add chain=input action=accept in-interface-list=LAN protocol=tcp dst-p  
/ip firewall filter add chain=input action=accept in-interface-list=LAN protocol=udp dst-p
```

Passo 3 — auditar o ensaio. Com as liberações no lugar, o log de ensaio (última regra) só deve pegar o que **morreria** no futuro **drop all**. Zere contadores, gere tráfego normal e leia:

```
/ip firewall filter reset-counters-all
```

Em lab-r2: `/ping 192.168.10.1 count=3`, `/ping google.com count=3`, renove o DHCP (`/ip dhcp-client release numbers=0`). Em lab-r1:

```
/ip firewall filter print stats
/log print where message~"INPUT-NEW"
```

Os accepts engordaram; o log deve estar **silencioso** (ou mostrando só ruído da WAN — que é exatamente o que queremos matar). Se algo legítimo aparecer no log, é serviço que faltou liberar: **investigue antes de apertar**.

Passo 4 — o aperto. Troque o log de ensaio pelo drop all:

```
/ip firewall filter set [find comment~"ENSAIO"] action=drop log=no comment="drop all"
```

Passo 5 — testar tudo, antes de confirmar. A bateria de testes, **ainda dentro do Safe Mode**:

- Sua sessão SSH continua viva (gerência OK — você está nela);
- Em lab-r2: `/ping 192.168.10.1` (ICMP OK), `/ping google.com` (DNS + forward OK), renovar DHCP (DHCP OK);
- Em lab-r2, o teste de **bloqueio**: `ssh vitor@192.168.10.1` deve **falhar** — a LAN não é gerência; o drop all a barrou. (Refleta: antes deste capítulo, esse acesso funcionava — só o `allowed-address` do serviço segurava.)

Tudo verde? `Ctrl+X` para **sair do Safe Mode confirmando**. O firewall está valendo.

Passo 6 — documentar. O ritual do 3:

```
/ip firewall filter export file=input-borda-v1
/system backup save name=pos-input
```

17.5.3 Verificação

1. `/ip firewall filter print` mostra as 8 regras na ordem do capítulo, todas comentadas;
2. A bateria do Passo 5 passou: gerência viva, LAN com DNS/DHCP/ping, e SSH **da LAN** morto no drop all;
3. `print stats` conta a história certa (established/related engordando, drop all pegando só ruído);
4. O export do filter está fora do equipamento.

17.5.4 Desafios

1. Por que a regra da gerência (`accept in-interface-list=gerencia`) está **acima** do ICMP e dos serviços da LAN? O que mudaria (funcional e em desempenho) se ela fosse a última antes do drop?
2. Um técnico esqueceu o notebook em campo e precisa acessar o r1 **da LAN do cliente**, excepcionalmente, por 1 hora. Escreva a liberação mais **estrita** possível (origem única, serviço único) e o plano de remoção — sem editar as regras permanentes.
3. Sua WAN ganhou IPv6? Não — mas o roteador tem um segundo IP público na mesma ether3 (um /29 da operadora). O firewall deste capítulo o protege? Por quê — o que os matchers usam, endereço ou interface?
4. Reproduza o desastre com rede de segurança: dentro do Safe Mode, **desabilite** a regra da gerência e confirme que sua sessão SSH morre em seguida (tente digitar). O que acontece ~15 s depois? Explique a mágica revendo Capítulo 4.

Solução dos desafios

1. Funcionalmente quase nada mudaria (não há drop entre elas que pegasse gerência), mas: (a) **desempenho** — todo pacote da gerência percorreria as regras de ICMP/DNS/DHCP antes de ser aceito; (b) **robustez** — a posição no topo garante que nenhuma regra futura inserida no miolo possa, por engano, descartar gerência antes do `accept`. Regra de sobrevivência primeiro; o resto depois.

2.

```
/ip firewall filter add chain=input src-address=192.168.10.105 protocol=tcp dst-port=22
```

(Como o serviço SSH tem `allowed-address` da gerência, seria preciso também um ajuste temporário lá — as duas trancas existem.) Plano de remoção: comentário com prazo + remover com `remove [find comment~"TEMP"]` — e, elegante, um agendamento no `/system scheduler` que remove sozinho (Volume 12).

3. Protege. Os `accepts` usam **interface lists** e a WAN não tem `accept` algum — o `drop all` barra conversas novas para *qualquer* endereço do roteador chegando por *qualquer* interface. É a vantagem do `default deny` por interface sobre listas de IPs: endereços novos nascem protegidos por padrão.

4. A sessão congela (os pacotes do seu SSH — `established` — continuam aceitos pela regra 0... até você digitar? Não: `established` continua passando! O que morre é **conversa nova**). Surpreso? Teste de verdade: com a regra da gerência desabilitada, a sessão atual sobrevive (`established/related` está acima), mas uma **nova** conexão SSH falha. Para derrubar tudo, seria preciso desabilitar também a regra 0 — e aí sim, ao travar a sessão, o Safe Mode detecta a queda e **reverte tudo** em segundos, devolvendo as duas regras. A mágica: o Safe Mode (Capítulo 4) desfaz mudanças não confirmadas quando a sessão que as fez morre. (E a lição extra: o esqueleto `stateful` protege até você dos seus erros — por camadas.)

17.6 Resumo

- **Default deny** no input: cada abertura é decisão documentada; o esquecido nasce fechado.
- **Interface lists** (WAN/LAN/*gerencia*, Tabela 17.1) desacoplam regras de topologia — o firewall-base do parque agradece.
- O miolo do input de borda: *gerência* (tudo), ICMP **com teto de taxa** (nunca bloquear ICMP inteiro — path MTU!), DNS e DHCP **da LAN** (DHCP = UDP 67–68 no input — a pegadinha clássica), e **nenhum accept da WAN**.
- Método de aplicação: Safe Mode + ensaio com log + auditoria dos logs + bateria de testes + apertado + export. Nessa ordem, sempre.
- As duas trancas convivem: **allowed-address** nos serviços e firewall — defesa em camadas.

No próximo capítulo, o tribunal dos clientes: a chain **forward** — o que pode atravessar o roteador — e as **address lists**, a ferramenta que transforma listas de IPs em política.

Bibliografia do capítulo

A página *Securing your router* e a seção *Building Advanced Firewall* da documentação oficial (MikroTik, 2026a) trazem as recomendações que este capítulo adapta ao roteador de borda de provedor. Discher (2016) cobre proteção de input com exemplos comentados. As referências completas, em ABNT, estão no apêndice [Referências](#).

18 Forward e address-lists

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Projetar a chain **forward** de um provedor — que é **diferente** da de uma empresa: o cliente paga por internet aberta, não por censura;
- Aplicar os filtros de forward que todo provedor sério tem: esqueleto stateful, antispoofing e o bloqueio clássico da porta 25;
- Criar e usar **address lists** estáticas — a ferramenta que separa *política* (regras) de *dados* (quem);
- Criar address lists **dinâmicas** com `add-src-to-address-list + timeout` — listas que se alimentam sozinhas;
- Reconhecer os usos de operação: corte de inadimplentes, armadilhas para scanners e listas de proteção da infraestrutura.

18.1 O forward de um provedor não é o de uma empresa

A chain **forward** julga tudo que **atravessa** o roteador (Capítulo 16) — a navegação dos clientes, portanto. E aqui o projeto exige uma reflexão que firewall de empresa não tem: numa empresa, o forward implementa a política do dono (“RH não acessa rede social”); num **provedor**, o cliente paga por **acesso à internet** — filtrar o tráfego dele é, além de má prática, problema jurídico (a neutralidade de rede do Marco Civil brasileiro, que reencontraremos no Volume 9).

O forward do provedor filtra pouco, e o pouco certo:

1. **O esqueleto stateful** de sempre — eficiência e higiene (16);
2. **Antispoofing** — pacotes *saindo* da sua rede devem ter origem *da sua rede*. Um cliente infectado emitindo pacotes com origem forjada participa de ataques DDoS refletidos mundo afora com o **seu** nome (seu bloco IP, sua reputação). A regra: o que entra pela LAN com origem que não é da LAN, morre;
3. **Porta 25 (SMTP) saindo** — o bloqueio mais consagrado: máquinas infectadas despejando spam pela porta 25 direta são a via expressa para seu bloco IP entrar em listas negras de e-mail (e todos os seus clientes junto). Bloqueia-se 25; e-mail legítimo usa 587 (submissão autenticada) há vinte anos. É a exceção de neutralidade aceita universalmente — o CERT.br a recomenda formalmente;

4. **Proteção da infraestrutura** — clientes não têm por que alcançar a rede de **gerência** do provedor. Uma regra fecha o assunto.

E só. Nada de bloquear “sites”, jogos, VPNs — provedor é estrada, não alfândega.

18.2 Address lists: separar política de dados

Repare no antispoofing: “origem que não é da LAN”. E o corte de inadimplente: “estes 37 IPs não navegam até pagar”. E a gerência: “estas redes são intocáveis”. Todos são a mesma pergunta — **o endereço está neste conjunto?** — e a resposta errada é criar 37 regras.

A ferramenta certa é a **address list** (`/ip firewall address-list`): um conjunto nomeado de endereços/redes, referenciado pelas regras via `src-address-list`/`dst-address-list`:

```
/ip firewall address-list add list=inadimplentes address=192.168.10.105 comment="contrato
/ip firewall address-list add list=inadimplentes address=192.168.10.130 comment="contrato
/ip firewall filter add chain=forward src-address-list=inadimplentes action=reject reject-
```

Uma regra, dados à parte. Cortar/religar cliente vira operação de lista (o sistema de gestão faz via API — Volume 12), sem tocar no firewall. A regra usa **reject** e não **drop** — o cliente cortado recebe resposta imediata em vez de timeouts (lembre a regra de bolso do Capítulo 16: dentro de casa, experiência importa; e um cliente cortado continua sendo seu cliente).

Listas aceitam **redes** (10.0.0.0/8), têm **comment** por entrada e — a parte que muda o jogo — podem ser alimentadas **pelas próprias regras do firewall**.

18.3 Listas dinâmicas: o firewall que aprende

Toda **action** de filter pode, em vez de (ou além de) aceitar/descartar, **anotar o endereço numa lista**:

```
add chain=input in-interface-list=WAN protocol=tcp dst-port=23 action=add-src-to-address-l
add chain=input in-interface-list=WAN src-address-list=scanners action=drop comment="scann
```

Leia o par: ninguém legítimo tenta Telnet no seu IP público (o serviço nem existe — 5); quem tenta é robô de varredura. A primeira regra **anota** a origem na lista **scanners** com validade de 1 dia (**timeout** — a entrada nasce com flag D e expira sozinha); a segunda descarta **tudo** que venha de quem está na lista. O firewall aprendeu: sondou a porta-armadilha, perdeu acesso ao roteador por 24 h.

O mecanismo (Figura 18.1) é um ciclo:

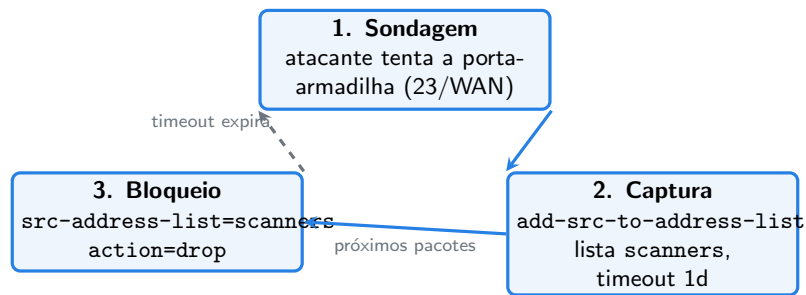


Figura 18.1: O ciclo da lista dinâmica: sondou a armadilha, entrou na lista, perdeu o acesso — até o timeout devolver o benefício da dúvida.

O padrão captura-e-bloqueia serve para muito mais que armadilhas: força bruta em SSH (capturar quem abre conexões demais), rajadas de conexões novas (o flood do desafio do 16), lista de quem baixou de um servidor de atualização... O Capítulo 21 usa esse tijolo várias vezes.

! O conceito mais importante do capítulo

Address lists separam **política** (poucas regras, estáveis) de **dados** (listas que mudam todo dia — por operação, por API ou pelo próprio firewall via `add-src-to-address-list + timeout`). No forward do provedor, a política é curta: stateful, antispoofting, porta 25, infraestrutura protegida — e neutralidade no resto. Quem precisar de “mais firewall” que isso está no lugar errado da rede (o CPE do cliente) ou do livro (Capítulo 21).

18.4 Laboratório

18.4.1 Ambiente

lab-r1 (borda com input fechado do Capítulo 17) e lab-r2 (cliente na LAN). Safe Mode como hábito.

18.4.2 Comandos passo a passo

Passo 1 — o esqueleto do forward. Em lab-r1:

```
/ip firewall filter add chain=forward action=accept connection-state=established,related
/ip firewall filter add chain=forward action=drop connection-state=invalid comment="fwd: 1
```

Passo 2 — antispoofting e porta 25. As duas regras de cidadania:


```
/ip firewall filter add chain=forward in-interface-list=LAN src-address=!192.168.10.0/24 action=reject reject-with=icmp-port-unreachable
/ip firewall filter add chain=forward in-interface-list=LAN protocol=tcp dst-port=25 action=reject reject-with=icmp-port-unreachable
```

(O ! nega o matcher: “origem que **não** é a LAN”. Teste o SMTP de lab-r2: `/tool fetch url=telnet://smtp.google.com:25` falha na hora — **reject** avisando.)

Passo 3 — proteger a gerência. Clientes não alcançam a rede de gerência:

```
/ip firewall filter add chain=forward in-interface-list=LAN dst-address=192.168.56.0/24 action=reject reject-with=icmp-port-unreachable
```

De lab-r2, prove: `/ping 192.168.56.1` — morto. (Antes funcionava — o r1 roteava para a gerência sem cerimônia.)

Passo 4 — corte por lista. Simule a inadimplência do “pc-padaria”:

```
/ip firewall address-list add list=inadimplentes address=192.168.10.105 comment="contrato rescindido"
/ip firewall filter add chain=forward src-address-list=inadimplentes connection-state=new action=reject reject-with=icmp-port-unreachable
```

(Ajuste o .105 para o IP real do r2 — `/ip dhcp-server lease print`.) Em lab-r2: `/ping 8.8.8.8` — cortado. **Religue** o cliente removendo-o da lista (a regra fica):

```
/ip firewall address-list remove [find list=inadimplentes]
```

Ping de novo: navegando. Corte e religação sem tocar em regra — a operação do dia a dia.

Passo 5 — a armadilha dinâmica. No input (a fronteira é da WAN, mas testaremos da LAN para ver o mecanismo — a LAN também não tem por que telnetar o roteador):

```
/ip firewall filter add chain=input protocol=tcp dst-port=23 action=add-src-to-address-list address-list=scanners list-timeout=1h
/ip firewall filter add chain=input src-address-list=scanners action=drop comment="scanner bloqueado"
```

Em lab-r2, caia na armadilha e constate a sentença:

```
/system telnet 192.168.10.1
/ping 192.168.10.1 count=3
```

O telnet falha e o ping morre junto — você está na lista. Em lab-r1, veja o prisioneiro e o prazo:

```
/ip firewall address-list print
```

```
[vitor@lab-r1] > /ip firewall address-list print
Flags: X - disabled, D - dynamic
Columns: LIST, ADDRESS, TIMEOUT
#   LIST      ADDRESS      TIMEOUT
0 D scanners  192.168.10.105 9m41s
```

Flag D, timeout decrescendo: em 10 minutos a pena prescreve (ou solte-o já: `remove numbers=0`).

18.4.3 Verificação

1. O forward tem: esqueleto, antispoofing, SMTP 25 (reject), gerência protegida e a regra de corte — todos comentados (`/ip firewall filter print where chain=forward`);
2. O corte por lista funcionou nos dois sentidos (cortou ao entrar na lista; religou ao sair) **sem editar regras**;
3. A armadilha capturou o r2 (entrada D com timeout) e o drop por lista o bloqueou por completo;
4. lab-r2 navegando normal ao final (fora das listas), e SMTP 25 continua morto.

18.4.4 Desafios

1. O antispoofing usa `src-address=!192.168.10.0/24`. Num concentrador com 30 pools de clientes, essa abordagem escala mal. Reescreva-a com address list — e diga de onde a lista seria alimentada na prática.
2. A regra de corte usa `connection-state=new`. O que muda para o cliente cortado, comparado ao corte sem esse matcher? Qual é mais “justo” na religação?
3. Monte (comandos) a defesa clássica anti força-bruta de SSH **na WAN**: quem abrir mais de 3 conexões novas TCP/22 em 1 minuto entra em `ssh-bruta` por 1 hora. (Dica: uma lista-contador com timeout de 1m como “memória curta” + a captura definitiva; matchers `connection-state=new` e `src-address-list`.)
4. Por que a armadilha do Passo 5 ficou **acima** do drop all mas o drop-por-lista ficou acima **dela**? Desenhe o caminho de um pacote do scanner na segunda visita.

Solução dos desafios

1.

```
/ip firewall address-list add list=redes-clientes address=192.168.10.0/24
/ip firewall filter set [find comment~"antispoofing"] src-address=!0.0.0.0 src-address-l
```

(Na prática: regra nova `in-interface-list=LAN src-address-list=!redes-clientes action=drop`.) A lista é alimentada pelo provisionamento: cada pool/rede de clientes nova entra na lista junto com sua criação — por script ou API (Volume 12). Política parada, dados acompanhando o crescimento.

2. Com **new**, as conexões **existentes** do cliente sobrevivem ao corte (a regra 0 do esqueleto as aceita antes) — só não abre nada novo; sem o matcher, tudo morre na hora. Mais importante é a **religação**: sem **new**, remover da lista já resolve; com qualquer resíduo, conexões travadas expiram sozinhas. O corte com **new** é mais suave (downloads em andamento terminam) e mais usado; o corte total é para abuso. Justiça é critério comercial — o firewall oferece os dois.

3.

```
/ip firewall filter add chain=input in-interface-list=WAN protocol=tcp dst-port=22 conntrack=established
/ip firewall filter add chain=input in-interface-list=WAN protocol=tcp dst-port=22 conntrack=established
/ip firewall filter add chain=input in-interface-list=WAN src-address-list=ssh-bruta action=drop
```

A ordem importa: quem já é **ssh-suspeitos** (memória de 1 m) e tenta de novo sobe para **ssh-bruta** (1 h); senão, apenas é anotado. (Refinável em mais estágios — o padrão em cascata do hardening.)

4. A armadilha precisa vir antes do drop all para sequer executar (depois dele, código morto — nada passa do drop). O drop-por-lista vem antes da armadilha para que o **reincidente** morra imediatamente, sem nem renovar a anotação... quer dizer: na segunda visita, o pacote desce: established? não → invalid? não → drop-por-lista: **casa e morre**. Se o drop viesse depois da armadilha, o scanner na lista ainda “renovaria” o próprio timeout a cada tentativa na porta 23 — detalhe fino: às vezes *queremos* isso (pena renovável!). As duas ordens são defensáveis; o que não se negocia é ambas acima do drop all.

18.5 Resumo

- Forward de **provedor**: esqueleto stateful, **antispoofing** (origem da LAN deve ser da LAN), **porta 25 fechada** (reputação do bloco), **gerência intocável** — e neutralidade no resto; filtro de conteúdo não é papel da estrada.
- **Address lists** = política separada de dados: regras estáveis (**src/dst-address-list**), listas mudando por operação/API — corte de inadimplente é **add/remove** na lista, nunca edição de regra.
- Listas **dinâmicas**: **action=add-src-to-address-list + address-list-timeout** — entradas D que expiram sozinhas; o padrão captura-e-bloqueia (Figura 18.1) constrói armadilhas, defesa de força bruta e rate-limiting por reincidência.
- **reject** para clientes (experiência), **drop** para a WAN (silêncio) — a regra de bolso continua.

No próximo capítulo, o caminho inverso: **dstnat** — publicar serviços de dentro para fora (port forwarding), com o hairpin NAT e os cuidados que todo portão aberto exige.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip firewall address-list` e os `matchers/actions` de lista. A recomendação de gerência da porta 25/TCP para redes de provedores é do CERT.br (cert.br/docs/), alinhada ao MAAWG. Sobre antispoofing na origem, o RFC 2827/BCP 38 é o texto clássico. As referências completas, em ABNT, estão no apêndice [Referências](#).

19 dstnat e port forwarding

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o **dstnat** como o espelho do **srcnat**: troca de **destino** na entrada, para publicar serviços internos;
- Escrever a regra clássica de **port forwarding** e entender o momento em que ela age (antes do roteamento — e antes do filter);
- Fazer firewall e dstnat conversarem: por que o **forward** enxerga o pacote **já traduzido** e como aceitá-lo com precisão (**connection-nat-state**);
- Diagnosticar e resolver o **hairpin NAT** — o clássico “funciona de fora, mas não funciona de dentro”;
- Usar **redirect** (dstnat para o próprio roteador) e publicar serviços com o mínimo de exposição.

19.1 O espelho do masquerade

O Capítulo 13 encerrou com uma porta fechada: ninguém de fora inicia conexão com quem está atrás do NAT — o fim a fim está quebrado, por design. Mas a padaria quer ver as câmeras do celular; o cliente corporativo quer publicar o sistema da empresa; você quer alcançar um equipamento na LAN de um cliente. A resposta é o **dstnat**: se o **srcnat** troca o *remetente* na saída, o **dstnat** troca o **destinatário na entrada**.

A regra clássica — o **port forwarding** — diz: “o que chegar ao meu IP público na porta X, entregue ao IP interno Y, porta Z”:

```
/ip firewall nat add chain=dstnat in-interface-list=WAN protocol=tcp dst-port=8080 action=
```

Lendo por partes, como sempre: **chain=dstnat** (tradução de destino, na entrada), matchers (**in-interface-list=WAN protocol=tcp dst-port=8080** — só o que chega da fronteira àquela porta), action **dst-nat** com o destino real (**to-addresses/to-ports** — que podem diferir da porta pública, como aqui: 8080 fora, 80 dentro).

O detalhe de arquitetura que explica tudo neste capítulo: **o dstnat age antes da decisão de roteamento** (e o **srcnat**, depois). A ordem completa da travessia:

chegada → dstnat → decisão de roteamento → filter forward → srcnat → saída

Duas consequências imediatas:

1. É *por isso* que o dstnat funciona: o destino é trocado **antes** de o roteador decidir para onde encaminhar — a decisão já vê o endereço interno;
2. O **filter forward enxerga o pacote traduzido**: destino 192.168.10.10:80, não IP-público:8080. Suas regras de filtro devem casar com o mundo *pós*-dstnat.

19.2 Firewall e dstnat: aceitando com precisão

E o nosso forward aceita esse pacote? Revise o esqueleto do Capítulo 18: conversas novas vindas da WAN não têm accept — na política permissiva de fábrica passariam, mas o hardening completo do Capítulo 21 fechará o forward com drop all. O jeito **errado** de liberar é abrir `dst-address=192.168.10.10 dst-port=80` — funciona, mas duplica a informação (regra de NAT + regra de filtro dizendo o mesmo, que alguém um dia atualiza pela metade). O jeito certo usa o conntrack, que marca conexões traduzidas:

```
/ip firewall filter add chain=forward connection-state=new connection-nat-state=dstnat in-
```

`connection-nat-state=dstnat` casa com **qualquer** conexão que passou pelo dstnat: uma regra de filtro serve todas as publicações, presentes e futuras — a lista de “o que está publicado” mora num lugar só (o NAT). É o mesmo princípio das address lists: política estável, dados num ponto único.

⚠ Aviso

Todo port forwarding é um **portão na muralha**: o serviço interno passa a receber a internet inteira — inclusive os robôs que varrem portas 24/7. Antes de publicar, pergunte: (1) o serviço precisa mesmo ser público, ou uma VPN de gerência resolve (Volume 10)? (2) o destino aguenta exposição (DVRs e câmeras antigos são o recheio favorito das botnets)? (3) dá para restringir a origem (`src-address-list` de IPs confiáveis)? Porta “escondida” (8080 em vez de 80) **não é segurança** — scanners acham em minutos; é só higiene contra ruído.

19.3 Hairpin NAT: o defeito clássico

Publique o DVR e teste **de dentro da LAN** pelo IP público: não abre. De fora (4G), abre. Bem-vindo ao defeito mais famoso do NAT — e à pergunta de entrevista favorita do mundo ISP. A Figura 19.1 mostra o porquê:

Siga os passos: o cliente da LAN mira o IP público (1); o dstnat troca o destino, mas **a origem continua sendo o vizinho de LAN** (2); o servidor responde **direto** pela rede

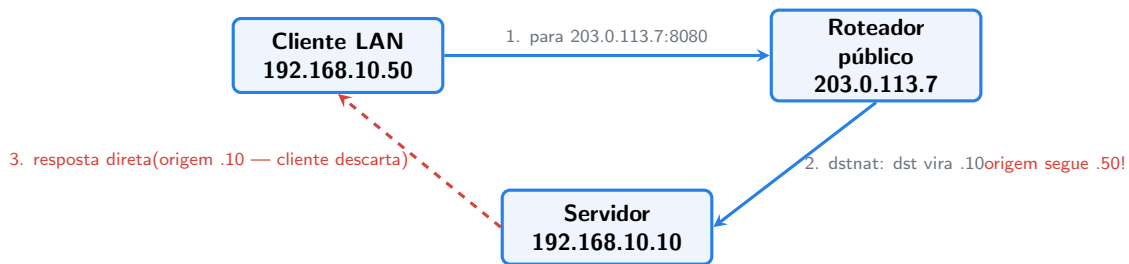


Figura 19.1: O hairpin: a ida faz a volta pelo roteador, mas a resposta corta caminho pela LAN — e chega com o remetente “errado”.

local (3) — e o cliente descarta: ele esperava resposta de 203.0.113.7:8080, recebeu de 192.168.10.10:80. Conversa assimétrica, conexão morta.

A solução: forçar a volta a passar pelo roteador, mascarando **também** a origem quando cliente e servidor são da mesma LAN:

```
/ip firewall nat add chain=srcnat src-address=192.168.10.0/24 dst-address=192.168.10.10 pr
```

Agora o servidor vê a conexão vinda **do roteador** (192.168.10.1) e responde a ele — que desfaz as duas traduções e entrega ao cliente com o remetente esperado. (O preço: o servidor perde o IP real do cliente interno nos logs — todos parecem vir do gateway. A alternativa elegante é *split DNS*: o nome do serviço resolvendo para o IP **interno** quando consultado de dentro — uma entrada em `/ip dns static`, Capítulo 12.)

19.4 redirect: dstnat para o próprio roteador

Uma variação com usos de provedor: `action=redirect` traduz o destino para **o próprio roteador**, na porta indicada:

```
/ip firewall nat add chain=dstnat in-interface-list=LAN protocol=udp dst-port=53 action=re
```

Essa regra intercepta **qualquer** consulta DNS da LAN — mesmo a do dispositivo teimoso configurado com 8.8.8.8 — e a entrega ao resolvidor local (Capítulo 12). Todos usam o cache do provedor, queira o dispositivo ou não. (Com a chegada do DoH essa técnica perdeu alcance — TLS na 443 não se intercepta — mas segue útil para a massa de consultas UDP/53.) O mesmo **redirect** é a base do *transparent proxy* e de portais de captura de hotspot.

! O conceito mais importante do capítulo

Grave a linha do tempo: **dstnat** → **roteamento** → **filter** → **srcnat**. Dela saem as três lições do capítulo: port forwarding funciona porque o destino muda *antes*

do roteamento; o filter deve casar com o pacote *já traduzido* (e `connection-nat-state=dstnat` aceita tudo que o NAT publicou, sem duplicar dados); e o hairpin existe porque a *volta* não repassa pelo roteador — até você mascarar a origem interna também.

19.5 Laboratório

19.5.1 Ambiente

Os três CHRs (Capítulo 5), com um ajuste **só para este capítulo**: no VirtualBox, mova o Adaptador 2 de `lab-r3` da rede interna `lab-b` para a `lab-a` — o `r3` vira um segundo dispositivo na LAN da padaria (cliente do teste de hairpin). Papéis: `lab-r1` = borda (como está), `lab-r2` = “DVR” (servidor a publicar), `lab-r3` = “notebook do dono”.

Em `lab-r3` (console): `/ip dhcp-client add interface=ether2` — ele deve receber um IP `192.168.10.1xx` do `r1`.

19.5.2 Comandos passo a passo

Passo 1 — o “DVR”. O `r2` será o servidor: seu WebFig (porta 80) fará papel da interface do DVR. Em `lab-r2`, garanta o serviço ativo e anote o IP (nos passos abaixo usamos `.105` — substitua pelo real):

```
/ip service enable www
/ip address print where interface=ether2
```

Passo 2 — publicar. Em `lab-r1`, descubra o IP da WAN e crie o port forwarding (8080 fora → 80 dentro):

```
/ip address print where interface=ether3
/ip firewall nat add chain=dstnat in-interface-list=WAN protocol=tcp dst-port=8080 action=
```

E a regra de filtro que aceita o publicado (entra antes de futuros drops do forward):

```
/ip firewall filter add chain=forward connection-state=new connection-nat-state=dstnat in-
```

Passo 3 — testar “de fora” (opcional, mas ótimo). O NAT do VirtualBox esconde o `r1` do mundo, mas o próprio VirtualBox faz port forward: com a VM `lab-r1` selecionada, **Configurações → Rede → Adaptador 3 → Avançado → Redirecionamento de portas**: adicione TCP, porta do hospedeiro 18080, porta do convidado 8080. No navegador do **seu computador**: `http://localhost:18080` → deve abrir o WebFig do `r2`. Você atravessou dois NATs até o “DVR” — de fora, funciona.

Passo 4 — o defeito de dentro. Em lab-r3 (o notebook do dono), acesse pelo IP público do r1 (o da ether3, ex.: 10.0.2.15):

```
/tool fetch url=http://10.0.2.15:8080 keep-result=no
```

Trava e expira — o hairpin da Figura 19.1, ao vivo. (Em lab-r1, a tabela de conexões mostra a conexão presa em SYN: `/ip firewall connection print where dst-port=80.`)

Passo 5 — a cura. Em lab-r1, o masquerade do hairpin:

```
/ip firewall nat add chain=srcnat src-address=192.168.10.0/24 dst-address=192.168.10.105 p
```

Repita o fetch no r3: `status: finished`. De dentro e de fora, o mesmo endereço funciona.

Passo 6 — desfazer o especial. Ao terminar: remova o port forward do VirtualBox (Passo 3), devolva o Adaptador 2 do r3 para lab-b e desative o `www` no r2 (`/ip service disable www`) — o laboratório volta ao padrão.

19.5.3 Verificação

1. `http://localhost:18080` no hospedeiro abriu o WebFig do r2 (se fez o Passo 3) — `dstnat` + filtro por `connection-nat-state` funcionando;
2. O fetch do r3 ao IP público **falhou** antes do Passo 5 e **funcionou** depois — hairpin demonstrado e curado;
3. `/ip firewall nat print` no r1: masquerade original, `dst-nat` da publicação e o masquerade do hairpin, todos comentados;
4. A tabela de conexões mostrou a flag `d` (`dstnat`) nas conexões publicadas.

19.5.4 Desafios

1. Sem executar: um pacote chega da WAN para IP-público:8080. Escreva a sequência completa de processamento no r1 (chains e traduções) até sair para o r2 — e a da resposta.
2. O dono quer as câmeras acessíveis **só do celular dele** (IP móvel muda, mas a operadora publica as faixas). Melhore a publicação do laboratório com uma `address list acesso-dvr` — quais regras mudam?
3. Explique por que o split DNS (entrada em `/ip dns static` apontando o nome do serviço para o IP interno) elimina o hairpin — e cite a desvantagem operacional que ele traz num parque com muitos clientes publicados.
4. Um técnico publicou o WinBox do próprio r1 na WAN (“pra facilitar”): `chain=dstnat dst-port=8291 action=redirect to-ports=8291`. Aponte os dois erros — um de segurança, um conceitual.

💡 Solução dos desafios

1. Ida: chega pela ether3 → **dstnat** casa (dst vira 192.168.10.105:80) → decisão de roteamento (destino é a LAN) → **filter forward** (o pacote já traduzido casa com **connection-nat-state=dstnat** → accept; conexão anotada com flag d) → srcnat (masquerade não casa — **out-interface** é LAN, e a regra de hairpin só casa se a origem for da LAN) → entrega ao r2. Volta: r2 responde ao cliente externo → forward (established) → na saída pela WAN o conntrack **desfaz** o dstnat (origem volta a ser IP-público:8080). Simetria por estado, não por regra.
2. Cria-se a lista e aperta-se o **filtro** (o NAT pode ficar):

```
/ip firewall address-list add list=acesso-dvr address=177.0.0.0/8 comment="faixa da oper  
/ip firewall filter set [find comment="fwd: aceita publicados"] src-address-list=acesso-
```

(Se houver mais publicações com políticas distintas, regras de filtro separadas por serviço — a genérica deixa de servir.)

3. De dentro, o nome resolve para 192.168.10.105: o cliente fala **direto** com o servidor, sem tocar no IP público — não há tradução, não há assimetria, e os logs do servidor veem o IP real do cliente interno (o que o hairpin-masquerade esconde). Desvantagem: é **um cadastro a mais** por serviço/cliente, no DNS de cada roteador envolvido — esquecer de atualizar ao mudar o IP interno gera o defeito inverso (“de fora funciona, de dentro abre a coisa errada”). Automação (Volume 12) ou disciplina.
4. Segurança: expor a **gerência** do roteador à internet é o pecado capital — força bruta e exploits contra WinBox são varredura de rotina; gerência remota se faz por VPN (Volume 10). Conceitual: **redirect** já entrega ao próprio roteador — mas o WinBox **já escuta** na 8291; a regra é inútil (não fosse o **/ip service** com **allowed-address** e o input com drop all, que — felizmente — continuam barrando o acesso; defesa em camadas salvando o dia de novo).

19.6 Resumo

- **dstnat** troca o destino **na entrada, antes do roteamento** — a linha do tempo dstnat → roteamento → filter → srcnat explica o capítulo inteiro.
- Port forwarding: **chain=dstnat in-interface-list=WAN dst-port=X action=dstnat to-addresses=interno to-ports=Z** — e o filtro aceita por **connection-nat-state=dstnat**, sem duplicar dados.
- Publicar é abrir portão: questione a necessidade (VPN?), restrinja origem quando possível, e lembre que porta alta não é segurança.
- **Hairpin**: de dentro, a resposta corta caminho e chega com remetente errado (Figura 19.1) — cura-se mascarando a origem interna (ou, mais elegante, com split DNS).
- **redirect** entrega ao próprio roteador: interceptação de DNS, proxy transparente, portais de hotspot.

No próximo capítulo, as duas tabelas que faltam no arsenal: **mangle** (marcar pacotes e conexões — a fundação do QoS do Volume 7) e **RAW** (agir antes do contrack — a defesa barata contra floods).

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip firewall nat` (actions `dst-nat`, `redirect`, `netmap`) e o diagrama de packet flow que fundamenta a linha do tempo do capítulo; a página *Hairpin NAT* traz o caso clássico com variações. As referências completas, em ABNT, estão no apêndice [Referências](#).

20 Mangle e RAW

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Situar as tabelas **mangle** e **RAW** na linha de montagem do pacote — e por que a posição de cada uma define seu poder;
- Usar o mangle para **marcar** conexões e pacotes (**mark-connection/mark-packet**) com o padrão eficiente “marca a conexão uma vez, herda nos pacotes”;
- Entender para quem as marcas trabalham: filas de QoS (Volume 7) e decisões de rota (**routing-mark**, Volumes 8 e 14);
- Usar o RAW para agir **antes do connection tracking**: descartar floods barato (**drop**) e isentar tráfego do conntrack (**notrack**);
- Prever os efeitos colaterais do **notrack** — o que quebra quando um pacote vira invisível para NAT e firewall stateful.

20.1 A linha de montagem completa

Você já conhece três estações da fábrica: o filter julga, o NAT traduz, o conntrack anota. Faltavam duas — e a posição delas na esteira é a chave do capítulo (Figura 20.1):

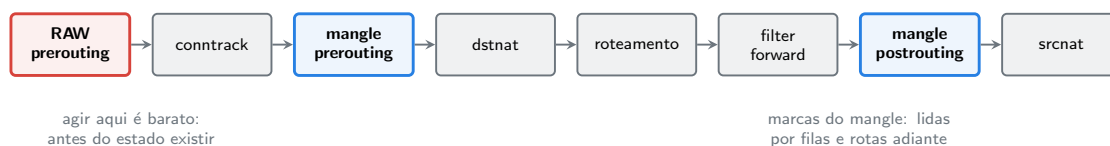


Figura 20.1: A esteira do pacote que atravessa o roteador: o RAW é a primeira estação (antes do conntrack); o mangle marca ao longo do caminho para consumo das estações seguintes.

- O **RAW** é a **primeira** estação: vê o pacote cru, antes mesmo de o conntrack procurar/criar a conexão. Poder: agir **barato** — e decidir se o conntrack sequer verá aquele tráfego;
- O **mangle** tem regras em **todas** as etapas (prerouting, input, forward, output, postrouting) e não aceita nem descarta: ele **anota** — marcas invisíveis que viajam com o pacote *dentro do roteador* e são lidas pelas estações seguintes.

20.2 Mangle: o almoxarifado de etiquetas

O mangle marca três coisas, cada uma com seu consumidor:

Tabela 20.1: As marcas do mangle e seus consumidores

Marca	Action	Quem consome
de conexão	<code>mark-connection</code>	o próprio mangle (herança)
de pacote	<code>mark-packet</code>	filas de QoS (Volume 7)
de rota	<code>mark-routing</code>	decisão de roteamento (Volumes 8 e 14)

As etiquetas não mudam o pacote na rede — são metadados internos. E o padrão de uso profissional aproveita o `conntrack` para marcar **barato**:

```
/ip firewall mangle
add chain=forward connection-state=new protocol=udp dst-port=53 action=mark-connection new-connection-mark=conn-dns
add chain=forward connection-mark=conn-dns action=mark-packet new-packet-mark=pkt-dns pass
```

Leia o par com atenção — ele voltará no QoS inteiro:

1. A primeira regra classifica **só o primeiro pacote** da conexão (`connection-state=new`) e grava a marca **na conexão** (na tabela do `conntrack` — você a verá lá no laboratório);
2. A segunda marca **cada pacote** consultando apenas a marca da conexão (`connection-mark=conn-dns`) — sem reavaliar portas/protocolos a cada pacote. Classificação cara 1× por conexão; etiquetagem barata por pacote. É o `established,related` do mangle;
3. **passthrough: yes** = o pacote continua descendo a lista do mangle (permite acumular marcas: conexão + pacote); **no** = para aqui (a etiquetagem final é terminal). O padrão dos pares classifica-e-etiqueta é exatamente esse: **yes** na primeira, **no** na segunda.

Um terceiro uso do mangle, sem marca alguma, resolve um defeito clássico de provedor: **change-mss** — ajustar o tamanho máximo de segmento TCP quando o caminho tem MTU menor (túneis, PPPoE). Guarde o nome; o Volume 6 o usará no concentrador (`action=change-mss ... tcp-flags=syn`), e ele explica metade dos chamados “abre uns sites e outros não”.

20.3 RAW: agir antes de existir

As regras do RAW (`/ip firewall raw`) vivem em `prerouting` (e `output`), **antes do `conntrack`** — e essa posição paga duas contas:

1. `action=drop` — a defesa barata contra flood. Relembre o desafio do 16: um flood de conexões novas enche a tabela e derruba clientes legítimos. Um **drop** no **filter** descarta o pacote, mas *tarde demais* — o `conntrack` já gastou CPU e criou a entrada. O mesmo drop no **RAW** mata o pacote antes de qualquer estado nascer:

```
/ip firewall raw add chain=prerouting in-interface-list=WAN src-address-list=atacantes act
```

Sob ataque volumétrico, essa diferença é o roteador de pé ou caído. O padrão de operação: detectar as origens (listas dinâmicas do Capítulo 18, ou à mão durante o incidente) e despachá-las no RAW.

2. action=notrack — isentar do conntrack. Tráfego que não precisa de NAT nem de firewall stateful pode ser dispensado da contabilidade:

```
/ip firewall raw add chain=prerouting src-address=10.255.0.0/24 dst-address=10.255.0.0/24
```

Cada pacote isento economiza a busca/criação na tabela — em roteadores de core ou enlaces de infraestrutura com milhões de pacotes por segundo, é CPU de volta ao caixa. Mas o **notrack** tem dentes:

Aviso

Pacote **notrack** é **invisível** para tudo que depende do **conntrack**: NAT não o traduz, matchers de **connection-state** não casam (o estado vira **untracked**) e as regras **established,related** **não o aceitam**. Aplicar **notrack** em tráfego que passa por NAT quebra a navegação na hora — o laboratório demonstra de propósito. Use-o apenas em tráfego que comprovadamente não precisa de estado (infraestrutura, roteamento puro) e **teste os firewalls no caminho**: um esqueleto que só aceita **established,related** + drop all descarta o **untracked** silenciosamente.

O conceito mais importante do capítulo

Posição é poder: o **RAW** age antes do estado existir (defesa barata, isenção de contabilidade — com os riscos de quem fica invisível), e o **mangle** anota ao longo da esteira para as estações seguintes decidirem (filas, rotas). Nenhum dos dois “bloqueia ou libera” — RAW poupa o roteador; mangle informa o roteador. O filter continua sendo o juiz.

20.4 Laboratório

20.4.1 Ambiente

lab-r1 (borda completa dos capítulos anteriores) e **lab-r2** (cliente na LAN). O **r3** já voltou ao padrão (Capítulo 19, Passo 6).

20.4.2 Comandos passo a passo

Passo 1 — o par classifica-e-etiqueta. Em lab-r1, marque o tráfego ICMP dos clientes (nosso “tráfego prioritário” de mentira — no Volume 7 será VoIP):

```
/ip firewall mangle add chain=forward connection-state=new protocol=icmp action=mark-connection
/ip firewall mangle add chain=forward connection-mark=conn-ping action=mark-packet new-packet-mark=conn-ping
```

Passo 2 — ver as etiquetas. Em lab-r2, um ping contínuo a 8.8.8.8. Em lab-r1, veja a **marca na conexão** e os contadores do par:

```
/ip firewall connection print detail where protocol=icmp
/ip firewall mangle print stats
```

Na conexão: **connection-mark=conn-ping**. Nos contadores: a regra 0 (classifica) com **1** pacote; a regra 1 (etiqueta) crescendo a cada ping — o padrão barato, visível nos números. Pare o ping.

Passo 3 — RAW drop: o porteiro de fora. Simule o despacho de um atacante. Em lab-r1:

```
/ip firewall raw add chain=prerouting src-address=192.168.10.0/24 dst-address=1.1.1.1 action=drop
```

Em lab-r2: `/ping 1.1.1.1 count=3` — morto. Agora o detalhe fino: em lab-r1, procure a conexão:

```
/ip firewall connection print where dst-address~"1.1.1.1"
```

Nada. O pacote morreu antes do conntrack — compare com um drop de filter, que deixaria... também nada de conexão (drop no new impede a entrada), mas o contraste real é o custo: no RAW, nem a **busca** na tabela aconteceu. Remova o teste:

```
/ip firewall raw remove [find comment="teste RAW drop"]
```

Passo 4 — notrack quebrando o NAT (de propósito). A demonstração mais instrutiva do capítulo. Em lab-r1:

```
/ip firewall raw add chain=prerouting src-address=192.168.10.0/24 protocol=icmp action=notrack
```

Em lab-r2: `/ping 8.8.8.8 count=3` — **falha**. Por quê? O ping saiu **untracked**: o masquerade (que depende do conntrack) não o traduziu, e o pacote foi para a internet com origem 192.168.10.x — o timeout do Capítulo 13 de novo, agora causado por *excesso de otimização*. Confirme a invisibilidade:

```
/ip firewall connection print where protocol=icmp
```

Vazio, mesmo com o ping rodando. Desfaça e confirme que voltou:

```
/ip firewall raw remove [find comment="teste notrack"]
```

Passo 5 — limpar o mangle de teste. As marcas de ping cumpriram seu papel didático:

```
/ip firewall mangle remove [find comment~"ping"]
```

20.4.3 Verificação

1. A conexão ICMP exibiu `connection-mark=conn-ping` e os contadores do par mostraram 1 pacote na classificação e N na etiquetagem;
2. Com o RAW drop, o destino de teste morreu **sem** gerar entrada no conntrack;
3. Com o notrack, o ping dos clientes quebrou (NAT cego) e a tabela de conexões ficou vazia para ICMP — e você sabe explicar a cadeia causal;
4. Tudo removido ao final (`/ip firewall raw print` e `.. mangle print` limpos).

20.4.4 Desafios

1. No par classifica-e-etiqueta, o que aconteceria se a **primeira** regra usasse `passthrough=no`? E se a **segunda** usasse `passthrough=yes`?
2. Sob um flood UDP de 500 mil pps contra um cliente, compare o custo por pacote de três defesas: drop no filter forward, drop no RAW prerouting, e blackhole na operadora (pedir para o upstream descartar). Quando cada uma é a resposta certa?
3. Escreva a regra de mangle `change-mss` para clientes PPPoE com MTU 1492 (MSS 1452) — chain, matchers de tcp-flags e sentido do tráfego — e explique por que ela casa só com pacotes SYN.
4. Um roteador de core sem NAT e sem filtro stateful roda `connection tracking enabled=auto`. Há regra RAW `notrack` para tudo. O conntrack está de fato desligado? O que o `auto` decide — e qual a diferença prática entre “notrack em tudo” e `enabled=no`?

Solução dos desafios

1. Com `passthrough=no` na primeira, o primeiro pacote da conexão para ali: recebe marca de conexão mas **nunca chega** à regra de pacote — o primeiro pacote de cada conexão viaja sem etiqueta (em filas, o SYN escaparia da fila certa; sutil e real). Com `yes` na segunda, os pacotes etiquetados continuariam descendo o mangle — inofensivo se não há mais regras, mas paga-se o percurso; em listas grandes, é desperdício por pacote.
2. Filter forward: o pacote já passou por RAW+conntrack+mangle+ roteamento —

CPU cheia por pacote; certo quando o volume é pequeno ou a decisão exige estado. RAW: só o custo de chegada + um matcher — aguenta uma ordem de grandeza a mais; a resposta certa on-box para floods. Mas a 500 kpps o **enlace** já está pago: se o flood satura a banda de entrada, descartar dentro de casa não devolve banda — a resposta certa é o **blackhole no upstream** (a operadora descarta lá, e o seu link respira). Escada: filter < RAW < upstream, conforme o gargalo sobe.

3.

```
/ip firewall mangle add chain=forward protocol=tcp tcp-flags=syn action=change-mss new-m
```

O MSS é negociado **no aperto de mão**: só os pacotes SYN carregam a opção — ajustar os demais é inútil. O matcher `tcp-mss=1453-65535` evita *aumentar* MSS já menores. (No Volume 6, a versão nos dois sentidos e a discussão de MTU completa.)

4. Com **auto**, o `conntrack` liga se **existir** regra que o use — e a própria regra RAW conta como uso do subsistema: o módulo fica ativo, processando o notrack por pacote (barato, mas não zero) e mantendo a infraestrutura da tabela. `enabled=no` desliga o subsistema inteiro: nenhuma consulta, nenhum estado, custo mínimo absoluto — e sem exceções possíveis. “Notrack em tudo” = flexível (dá para excetuar a gerência, p.ex.); `no` = mais rápido e radical. Core puro: `no`; core que ainda faz alguma coisa stateful: `notrack` seletivo.

20.5 Resumo

- A esteira completa (Figura 20.1): **RAW** → **conntrack** → **mangle** → **dstnat** → **roteamento** → **filter** → **mangle** → **srcnat** — posição é poder.
- **Mangle** anota (conexão, pacote, rota — Tabela 20.1) para consumidores adiante (filas, roteamento); o padrão **classifica-1×-e-etiqueta-por-herança** com `passthrough` certo é a base de todo QoS eficiente.
- **change-mss** no mangle: o remédio dos “sites que não abrem” em caminhos de MTU reduzida (PPPoE, túneis).
- **RAW**: `drop` antes do `conntrack` é a defesa barata contra floods; `notrack` isenta da contabilidade — e torna o tráfego invisível para NAT e stateful (**quebra NAT**, escapa de `established,related`).
- A escada anti-flood: filter → RAW → blackhole no upstream, conforme o gargalo sobe.

No próximo capítulo, o gran finale do volume: o **hardening completo** — todos os tijolos deste volume montados num checklist consolidado de roteador de provedor.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip firewall mangle` (marcas e `change-mss`), `/ip firewall raw` e o diagrama de packet flow que este capítulo resume. A página

DDoS protection da mesma documentação traz padrões RAW + address lists dinâmicas. As referências completas, em ABNT, estão no apêndice [Referências](#).

21 Hardening completo

i Objetivos de aprendizagem

Este capítulo consolida o volume. Ao final, você será capaz de:

- Enxergar a segurança do roteador como **camadas independentes** — da superfície de serviços ao RAW — e saber o que mora em cada uma;
- Montar e aplicar o **firewall-base** do provedor: o `.rsc` comentado que todo equipamento do parque recebe;
- Acrescentar as peças finais: defesa de força bruta em cascata, descarte de bogons, FastTrack (e seu preço) e logging do que importa;
- Auditar um roteador em produção com um **checklist** repetível;
- Manter o firewall-base **vivo**: versionado, auditado e aplicado ao parque inteiro.

21.1 Defesa em camadas: o mapa do volume

Cada capítulo deste volume construiu uma camada; o hardening é reconhecê-las como um sistema (Figura 21.1):

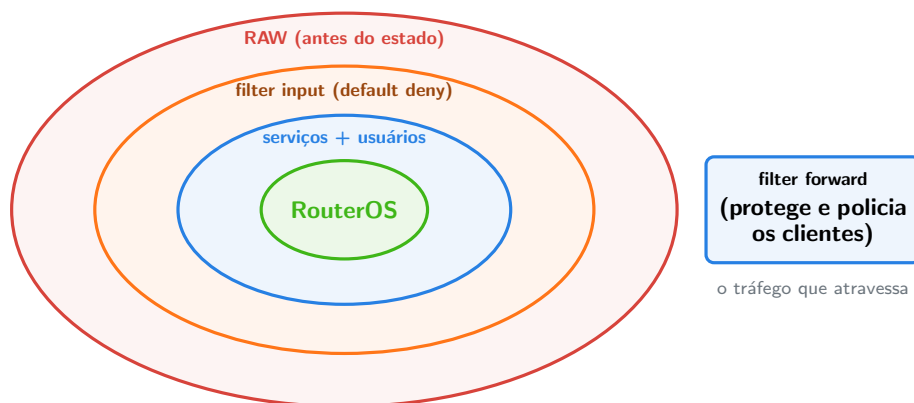


Figura 21.1: As camadas do hardening: cascas concêntricas protegem o roteador; o forward, ao lado, cuida do que atravessa.

A propriedade que importa: as camadas são **independentes**. O `allowed-address` do serviço segura mesmo se o firewall for esvaziado; o input segura mesmo se um serviço for habilitado por engano; o RAW segura a CPU mesmo com o input sob flood. Segurança de

provedor não é uma muralha alta — são várias muralhas medíocres que falham em momentos diferentes.

21.2 As peças finais

Quatro adições transformam o firewall dos capítulos anteriores no firewall-base definitivo.

1. Bogons na fronteira. Pacotes chegando da WAN com origem privada (RFC 1918), loopback ou reservada são, por definição, forjados ou vazados — nenhum deles tem o que fazer batendo na sua borda:

```
/ip firewall address-list
add list=bogons address=10.0.0.0/8 comment="RFC1918"
add list=bogons address=172.16.0.0/12 comment="RFC1918"
add list=bogons address=192.168.0.0/16 comment="RFC1918"
add list=bogons address=127.0.0.0/8 comment="loopback"
add list=bogons address=100.64.0.0/10 comment="CGNAT space"
add list=bogons address=169.254.0.0/16 comment="link-local"

/ip firewall raw add chain=prerouting in-interface-list=WAN src-address-list=bogons action=drop
```

No RAW, pela lição do 4: lixo certo morre barato. (Atenção do mundo real: se a sua “WAN” recebe endereço de CGNAT da operadora — 100.64.0.0/10 — ajuste a lista; o laboratório com NAT do VirtualBox usa 10.0.2.x, e o laboratório tratará disso.)

2. Força bruta em cascata. O desafio do Capítulo 18, agora oficial — a escada de listas dinâmicas que pega quem insiste no SSH da WAN sem punir o técnico que errou a senha uma vez:

```
/ip firewall filter
add chain=input in-interface-list=WAN protocol=tcp dst-port=22 connection-state=new src-address-list=bogons action=drop comment="SSH blocked"
add chain=input in-interface-list=WAN protocol=tcp dst-port=22 connection-state=new src-address-list=ssh-bloqueado action=drop comment="SSH blocked"
add chain=input in-interface-list=WAN protocol=tcp dst-port=22 connection-state=new src-address-list=ssh-bloqueado action=drop comment="SSH blocked"
add chain=input in-interface-list=WAN protocol=tcp dst-port=22 connection-state=new src-address-list=ssh-bloqueado action=drop comment="SSH blocked"
add chain=input in-interface-list=WAN src-address-list=ssh-bloqueado action=drop comment="SSH blocked"
```

(Repare a ordem invertida — a regra da 4ª tentativa vem **primeiro**, senão toda conexão pararia no estágio 1. E lembre: no nosso desenho, SSH da WAN nem deveria existir — a cascata é a rede de segurança para o dia em que alguém abrir uma exceção.)

3. FastTrack — desempenho com letras miúdas. A regra que faz um hEX de R\$ 300 rotear como gente grande:

```
/ip firewall filter add chain=forward action=fasttrack-connection hw-offload=yes connection
```

fasttrack-connection marca a conexão para pular quase toda a esteira da Figura 20.1 nos pacotes seguintes — multiplicando o throughput. O preço: pacotes fastrackeados **não passam** por mangle, filas e pela maior parte do filter. Num roteador de borda simples, ligue; num concentrador com QoS (Volume 7), a decisão muda — as filas precisam ver os pacotes. A regra convive com a gêmea **accept** logo abaixo (nem toda conexão é fastrackeável; a **accept** apanha o resto).

4. Logar o que importa. Log de tudo é ruído; log de nada é cegueira. O equilíbrio do provedor: logar **negações interessantes** (tentativas na gerência, publicados) e **mudanças de listas dinâmicas** — os **add-to-list** já contam a história dos ataques em `/ip firewall address-list print`. Uma regra exemplar:

```
/ip firewall filter add chain=input in-interface-list=WAN protocol=tcp dst-port=8291 actio
```

Se um dia esse prefixo aparecer em rajada no log, você saberá que estão procurando por você — antes de acharem.

21.3 O checklist consolidado

O volume inteiro numa tabela — a auditoria repetível de qualquer roteador do parque:

Tabela 21.1: Checklist de hardening do roteador de provedor

Camada	Item	Referência
Superfície	serviços mínimos + allowed-address ; admin off ; usuários nominais	5
Superfície	discovery e MAC-server só na gerencia	5
Listas	WAN/LAN/gerencia refletem a topologia real	Capítulo 17
Input	esqueleto stateful + gerência + ICMP c/ teto + serviços da LAN + drop all	Capítulo 17
Input	cascata anti força bruta nas portas de gerência	este capítulo

Camada	Item	Referência
Forward	esqueleto + antispoofing + porta 25 + gerência intocável + drop de publicações órfãs	Capítulo 18
Forward	<code>connection-nat-state=dstnat</code> para publicados; FastTrack conforme o papel	Capítulo 19, este
RAW	bogons da WAN; despacho de flood; notrack só onde provado	4, este
Operação	<code>export</code> versionado; contadores/logs revisados; teste de acesso pós-mudança	3

! O conceito mais importante do capítulo

Hardening não é um evento — é um **padrão versionado**. O produto deste volume é um `firewall-base.rsc` comentado, guardado em repositório, que todo equipamento novo recebe (Netinstall + configure script, Capítulo 8) e contra o qual todo equipamento antigo é auditado (Tabela 21.1). Segurança que depende de memória humana regride; segurança que mora num arquivo versionado acumula.

21.4 Laboratório

21.4.1 Ambiente

`lab-r1` com tudo do volume aplicado (é o estado em que o 4 o deixou) e `lab-r2` na LAN. O trabalho final: completar, testar como um auditor e **exportar o padrão**.

21.4.2 Comandos passo a passo

Passo 1 — bogons. Aplique a lista e a regra RAW da seção anterior — com **uma adaptação de laboratório**: a nossa “WAN” do VirtualBox usa 10.0.2.x, que está dentro de 10.0.0.0/8... incluir essa linha bloquearia o gateway do laboratório. Aplique a lista **sem** a primeira entrada (e anote no comentário o porquê — auditoria futura agradece):

```

/ip firewall address-list
add list=bogons address=172.16.0.0/12 comment="RFC1918"
add list=bogons address=192.168.0.0/16 comment="RFC1918"
add list=bogons address=127.0.0.0/8 comment="loopback"
add list=bogons address=169.254.0.0/16 comment="link-local"
add list=bogons address=100.64.0.0/10 comment="CGNAT space"
/ip firewall raw add chain=prerouting in-interface-list=WAN src-address-list=bogons action

```

Passo 2 — cascata anti-bruta. Aplique as cinco regras da seção anterior (copie do texto). Em seguida, o teste: a nossa WAN não recebe conexões externas, então simule **pela LAN** trocando temporariamente o matcher da 1ª tentativa (`in-interface-list=LAN`)... ou melhor: entenda o mecanismo com o desafio 4 do Capítulo 18 já feito e apenas confira a sintaxe com `print`. (Auditor também sabe quando **não** repetir um teste destrutivo.)

Passo 3 — FastTrack. No nosso roteador de borda, sem filas por enquanto — liga:

```

/ip firewall filter add chain=forward action=fasttrack-connection hw-offload=yes connection

```

Em `lab-r2`, gere tráfego (`/tool fetch url=https://mikrotik.com keep-result=no`) e veja as conexões marcadas em `lab-r1`:

```

/ip firewall connection print where fasttrack

```

A flag `F` nas conexões: o atalho está ativo.

Passo 4 — o log-sentinela. A regra do WinBox-da-WAN logado (copie da seção). Confira a posição: acima do `drop all`, abaixo dos `accepts`.

Passo 5 — auditoria completa. Agora, vista o chapéu de auditor e percorra a Tabela 21.1 item a item contra o roteador real:

```

/ip service print
/user print
/interface list member print
/ip firewall filter print where chain=input
/ip firewall filter print where chain=forward
/ip firewall raw print
/ip firewall address-list print

```

Cada linha do checklist deve apontar para algo que você **vê** na saída. Achou divergência? Corrija agora — é exatamente para isso que a auditoria existe.

Passo 6 — a bateria de fumaça. O teste funcional de sempre, agora completo: sessão de gerência viva; `lab-r2` navega (IP e nome); DHCP renova; SSH da LAN ao `r1` morre; SMTP 25 morre com aviso; gerência inalcançável do `r2`; ping ao `r1` funciona (com teto).

Passo 7 — o padrão nasce. Exporte e guarde:

```
/export file=firewall-base-v1
/system backup save name=hardening-completo
```

Baixe os dois (`scp`) e — hábito novo — **versione**: o `firewall-base-v1.rsc` é o primeiro commit do padrão de segurança do seu provedor fictício. Todo capítulo futuro que mexer em firewall parte dele.

E crie o snapshot **hardening-completo** nas VMs `lab-r1` e `lab-r2`: os próximos volumes partem daqui.

21.4.3 Verificação

1. Auditoria do Passo 5 sem divergências (ou com as correções feitas e anotadas);
2. Bateria de fumaça do Passo 6: 7 de 7;
3. Conexões com flag **F** (FastTrack vivo) e bogons na RAW com o comentário de adaptação do laboratório;
4. `firewall-base-v1.rsc` fora do equipamento e snapshot **hardening-completo** criado.

21.4.4 Desafios

1. A cascata anti-bruta tem um falso positivo clássico: o próprio técnico com automação (Ansible, scripts) abrindo várias conexões SSH legítimas em sequência. Proponha duas mitigações que não desliguem a defesa.
2. Com FastTrack ligado, o par classifica-e-etiqueta do 4 voltaria a marcar todos os pacotes de uma conexão? Teste mentalmente (ou no laboratório) e explique — e diga qual regra teria que mudar para um futuro QoS funcionar.
3. Ordene os itens do checklist (Tabela 21.1) do **maior para o menor impacto** se estiverem errados, na sua avaliação — e defenda as duas primeiras posições.
4. Seu parque tem 300 equipamentos com firewalls “cada um de um jeito”. Desenhe (em texto) o plano de convergência para o firewall-base em 90 dias sem derrubar ninguém: fases, ferramentas deste livro e critérios de sucesso.

Solução dos desafios

1. (a) **Isentar a origem certa**: os accepts da gerência (lista `gerencia/address list` dos IPs do NOC) ficam **acima** da cascata — quem é de casa nunca entra na contagem; (b) **contar só falhas de fato**: subir o limiar (estágios com timeout maior, ou contar a partir da 2ª conexão em janela curta) e/ou usar portas distintas para automação (SSH de gestão em porta/endereço dedicados de gerência, fora da WAN). A defesa é para a fronteira; automação não deveria vir de lá.
2. Não: pacotes fastrackeados **pulam o mangle** — só o primeiro punhado de pacotes de cada conexão (antes de a marca FastTrack pegar) seria etiquetado. Filas veriam quase nada. Para QoS funcionar, a regra de FastTrack precisa **excluir** o tráfego que se quer enfileirar — p.ex. `connection-mark=no-mark` (fastrackear só o não classificado)

ou remover o FastTrack no concentrador. É a tensão desempenho × visibilidade do Volume 7.

3. Resposta modelo (defensável, não única): 1º **input sem drop all** — o roteador inteiro exposto é comprometimento de infraestrutura, raio de dano máximo (atacante dentro = todas as outras camadas viram decoração); 2º **superfície de serviços aberta** (senha fraca no admin + WinBox na WAN é a dupla que mais derruba provedor no mundo real — explora sem “furar” firewall nenhum, só entrando pela porta). Os demais causam danos graves porém mais contidos (spoofing/25 = reputação; bogons/RAW = resiliência; logs = tempo de detecção).

4. Fases: (1) **Inventário** (semanas 1–2): coletar `/export` de todos (script/API — Volume 12), diffs contra o base, classificar por distância; (2) **Base mínima sem risco** (semanas 3–6): aplicar em massa o que não muda comportamento (usuários, serviços, listas, logs) via script testado em laboratório-espelho; (3) **Firewall em ensaio** (semanas 7–10): input/forward novos com drop all em modo log por 1 semana por lote de equipamentos, auditar logs, então apertar — lotes pequenos, horário de manutenção, Safe Mode/backup antes de cada um; (4) **Selo** (semanas 11–13): auditoria da Tabela 21.1 por amostragem + alarme de desvio (export diário comparado ao base). Sucesso: 100% dos equipamentos com diff vazio contra o base, zero chamados atribuíveis à migração.

21.5 Resumo

- Segurança é **camadas independentes** (Figura 21.1): superfície → input → forward → RAW — cada uma segura quando as outras falham.
- As peças finais: **bogons** no RAW, **cascata anti-bruta** (ordem invertida!), **FastTrack** (desempenho ao preço da visibilidade — QoS exigirá exceções) e **logs-sentinela** com prefixo.
- O produto do volume é um **firewall-base.rsc** comentado e versionado — aplicado a todo equipamento novo, auditado contra todo equipamento antigo (Tabela 21.1).
- Auditoria é ver cada item do checklist na saída real do equipamento — e a bateria de fumaça funcional fecha toda mudança.

Fecha-se a Fase 1 do trivium do firewall... e abre-se o Volume 4: até aqui, roteamos entre redes; agora vamos **segmentar dentro** delas — bridge, VLANs e o switching em hardware dos CRS.

Bibliografia do capítulo

As páginas *Securing your router*, *Building Advanced Firewall* e *DDoS Protection* da documentação oficial (MikroTik, 2026a) são as fontes dos padrões consolidados aqui — vale lê-las na íntegra agora que você tem o volume todo como contexto. O CERT.br publica

recomendações de segurança para provedores brasileiros (cert.br/docs/). As referências completas, em ABNT, estão no apêndice [Referências](#).

Parte IV

Volume 4 — Bridge, switching e VLANs

22 Bridge e hardware offload

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Diferenciar **rotear** (camada 3) de **comutar** (camada 2) — e dizer qual dos dois cada problema pede;
- Explicar o que uma **bridge** faz: aprendizado de MACs, encaminhamento por tabela e inundação do desconhecido;
- Criar bridges no RouterOS, adicionar portas e ler a *host table*;
- Explicar o **hardware offload** (a flag H): quando o switch chip comuta sozinho, quando o trabalho cai na CPU — e quais recursos matam o offload;
- Reconhecer os papéis da bridge na rede do provedor: LAN de CPE, switch de POP e o motor por trás das VLANs dos próximos capítulos.

22.1 Camada 2: o andar de baixo

Três volumes inteiros olhando endereços IP — e no entanto, quando dois computadores da mesma LAN conversam, o IP quase não trabalha: quem entrega os quadros é a **camada 2**, endereçando por **MAC**. O RouterOS, que até aqui usamos como roteador, também é um **switch** — e a peça que faz isso é a **bridge**.

A distinção que organiza o volume:

- **Rotear** (camada 3): conectar redes **diferentes** — decisão por tabela de rotas, TTL decrementado, firewall na travessia. É o que fizemos até aqui;
- **Comutar** (camada 2): conectar equipamentos da **mesma** rede — decisão por tabela de MACs, transparente para o IP. É o que um switch faz, e o que a bridge do RouterOS faz.

Uma bridge é um **switch de software**: uma interface virtual que agrupa portas físicas e comuta quadros entre elas. O algoritmo cabe em três verbos:

1. **Aprende**: cada quadro que chega ensina “o MAC de origem X mora na porta Y” — anotado na *host table*;
2. **Encaminha**: quadro para MAC conhecido sai **só** pela porta onde ele mora;
3. **Inunda**: quadro para MAC desconhecido (ou broadcast) sai por **todas** as portas, menos a de origem — e a resposta ensinará o caminho.

💡 Analogia para guardar

A bridge é um **carteiro de condomínio** que começa sem conhecer ninguém: a primeira carta para um morador desconhecido é anunciada no pátio (inundação); quando o morador responde, o carteiro anota o bloco (aprendizado) e as próximas cartas vão direto (encaminhamento). O roteador, por contraste, é o **correio entre cidades** — outro andar do sistema.

22.2 Bridge no RouterOS

Criar o switch de software e povoá-lo:

```
/interface bridge add name=bridge-lan comment="LAN do escritorio"  
/interface bridge port add bridge=bridge-lan interface=ether2  
/interface bridge port add bridge=bridge-lan interface=ether3  
/interface bridge port add bridge=bridge-lan interface=ether4
```

Pronto: **ether2–ether4** agora são portas do mesmo “switch”, e dispositivos ligados a elas conversam em camada 2. Três consequências imediatas que mudam como você configura:

- **A bridge é uma interface** — e é *nela* que a configuração de camada 3 mora a partir de agora: o IP da LAN vai em **interface=bridge-lan**, o DHCP server escuta na bridge, as regras de firewall referenciam a bridge. As portas físicas viram “buracos de tomada”: nenhuma configuração IP nelas;
- **A host table conta a verdade** (**/interface bridge host print**): que MAC está em que porta, aprendido ao vivo — a primeira parada de todo diagnóstico “o dispositivo não aparece”;
- **Quem entra na bridge deixa de ser individual**: um endereço IP esquecido numa porta-membro é receita de comportamento bizarro.

Na rede do provedor, a bridge está em toda parte: é a LAN do CPE na casa do cliente (portas + WiFi na mesma bridge), o switch de gerência de uma torre (rádios + roteador no mesmo segmento) e — no CRS — o switch do POP inteiro. E aqui entra a pergunta de desempenho.

22.3 Hardware offload: quem carrega o piano?

Uma bridge de software comuta na **CPU**: cada quadro sobe, é decidido e desce. Num CHR ou num roteador pequeno servindo uma casa, tudo bem. Num CRS com 24 portas a 1 Gbps, seria suicídio — e é por isso que os CRS (2) carregam um **switch chip**: um circuito que comuta sozinho, em velocidade de fio, sem acordar a CPU.

O elo entre os dois mundos é o **hardware offload**: quando as condições permitem, o RouterOS *delega* a comutação da bridge ao switch chip. O sinal está na flag **H** de cada porta:

```
/interface bridge port print
```

```
[admin@crs-pop] > /interface bridge port print
Flags: I - INACTIVE; H - HW-OFFLOAD
Columns: INTERFACE, BRIDGE, HW
#    INTERFACE  BRIDGE      HW
0 H  ether1      bridge-pop  yes
1 H  ether2      bridge-pop  yes
2 H  ether3      bridge-pop  yes
```

H presente = aquele tráfego não gasta CPU. E a regra de ouro do capítulo: **recursos de software matam o offload**. A bridge só delega o que o chip sabe fazer; peça algo que exija a CPU e o RouterOS silenciosamente traz o tráfego de volta para o software — o switch de 200 Gbps vira um roteador de 500 Mbps sem avisar. Os assassinos clássicos do H:

Tabela 22.1: O que mata (e o que preserva) o hardware offload

Recurso na bridge	Efeito no offload
<code>use-ip-firewall=yes</code> (firewall vendo tráfego da bridge)	mata (tudo passa pela CPU)
VLAN filtering em chips antigos (CRS1xx/2xx)	mata; nos CRS3xx+, preservado
STP/RSTP	preservado (protocolos vão à CPU; dados não)
Bonding de portas	depende do chip
Mesma bridge misturando portas de chips diferentes	mata nas portas fora do chip

A tabela tem uma linha que anuncia o futuro: **VLAN filtering com offload nos CRS3xx** é exatamente o assunto dos próximos capítulos — segmentar POPs inteiros em velocidade de fio. E tem uma pegadinha de diagnóstico que vale sublinhar: a perda do offload **não gera erro** — só CPU alta e throughput baixo. `/interface bridge port print` procurando H ausente é o primeiro reflexo do técnico de POP.

! O conceito mais importante do capítulo

A bridge é o switch de software do RouterOS — IP, DHCP e firewall passam a morar **nela**, não nas portas. E desempenho de switching é uma pergunta só: **a flag H está lá?** Com offload, o switch chip comuta em velocidade de fio; sem ele, cada quadro custa CPU — e o RouterOS troca um pelo outro em silêncio quando você pede recursos que

o chip não sabe fazer (Tabela 22.1).

22.4 Laboratório

22.4.1 Ambiente

Os três CHRs (Capítulo 5), a partir dos snapshots **hardening-completo** (r1, r2) e **base-limpa** (r3). Ajuste **para este volume** (vale para os próximos capítulos): no VirtualBox, mova o Adaptador 2 de **lab-r3** para a rede interna **lab-a** — r2 e r3 ficam ambos “espetados” no r1, que fará papel de switch. (No CHR não há switch chip — a flag H não aparecerá; o offload é conceitual aqui e real no CRS do seu POP. Tudo o mais é idêntico.)

22.4.2 Comandos passo a passo

Passo 1 — desmontar a LAN roteada. No lab-r1, a LAN atual mora na **ether2** física. Vamos migrá-la para uma bridge — a operação clássica de quem “ganha portas”. Primeiro, crie a bridge e mova o IP (Safe Mode, claro — mexer onde mora o DHCP da LAN merece cinto):

```
/interface bridge add name=bridge-lan comment="LAN em bridge"  
/ip address set [find address="192.168.10.1"] interface=bridge-lan  
/ip dhcp-server set dhcp-lan interface=bridge-lan
```

Passo 2 — portas na bridge. **ether2** (onde vive o r2/r3 via lab-a) vira porta:

```
/interface bridge port add bridge=bridge-lan interface=ether2
```

E atualize a interface list (o firewall do Volume 3 referencia LAN — eis a recompensa da indireção do Capítulo 17):

```
/interface list member set [find interface=ether2] interface=bridge-lan
```

Passo 3 — dois vizinhos de bridge. Em lab-r2 e lab-r3 (console do r3), garanta DHCP client na **ether2** de cada:

```
/ip dhcp-client add interface=ether2
```

Ambos devem receber 192.168.10.1xx do r1. E o teste de camada 2: de lab-r3, pingue o r2 (IP dele na LAN — veja em `/ip dhcp-server lease print` no r1):

```
/ping 192.168.10.105 count=3
```

Funciona — e repare: esse tráfego r3 r2 **não é roteado** pelo r1; ele é **comutado** pela bridge (mesma rede, camada 2). O firewall **forward** do r1 nem o vê.

Passo 4 — a host table. Em lab-r1, o caderno do carteiro:

```
/interface bridge host print
```

```
[vitor@lab-r1] > /interface bridge host print
Flags: L - LOCAL, D - DYNAMIC
Columns: MAC-ADDRESS, ON-INTERFACE, BRIDGE, AGE
#      MAC-ADDRESS      ON-INTERFACE  BRIDGE      AGE
0 D    08:00:27:3B:71:9E  ether2        bridge-lan  12s
1 D    08:00:27:C4:88:2A  ether2        bridge-lan   4s
2 L    08:00:27:6A:2B:1C  bridge-lan    bridge-lan
```

Os MACs de r2 e r3 aprendidos (flag D, com idade), ambos na **ether2** — no laboratório as duas VMs chegam pela mesma “tomada” (a rede interna lab-a é o switch do VirtualBox); num CRS físico, cada um apareceria na sua porta. A flag L marca o MAC da própria bridge.

Passo 5 — assistir ao aprendizado. Limpe um host e veja-o voltar:

```
/interface bridge host print where !local
```

Anote um MAC, gere silêncio (pare pings), aguarde o **AGE** crescer — e dispare um ping do r3: o AGE zera (quadro novo reensina). É o ciclo aprende-esquece que mantém a tabela enxuta.

Passo 6 — a bateria de sempre. O firewall sobreviveu à migração? Bateria de fumaça do Capítulo 21, versão rápida: r2 navega (/ping google.com), DHCP renova, SSH da LAN ao r1 morre, gerência viva. Se algo quebrou, o suspeito é a interface list do Passo 2.

22.4.3 Verificação

1. O IP da LAN, o DHCP server e a interface list apontam para **bridge-lan** (/ip address print, /ip dhcp-server print, /interface list member print);
2. r2 e r3 têm leases e se pingam **diretamente** (comutados, não roteados);
3. A host table mostra os MACs dinâmicos com AGE vivo;
4. A bateria de fumaça do firewall passou intacta.

22.4.4 Desafios

1. No Passo 3, o ping $r3 \rightarrow r2$ não passa pelo firewall **forward** do $r1$. Isso é um problema de segurança? Em que cenário de provedor a resposta é “sim” — e qual ferramenta (deste volume) resolve?
2. Um técnico deixou um IP configurado direto na **ether2 além** do IP da bridge. Que classe de defeitos isso causa? (Pense no que o RouterOS responde por ARP e por onde ele roteia.)
3. Num CRS326 com 24 portas na bridge, você precisa que o firewall inspecione o tráfego **entre** duas portas específicas. Quais são suas opções, e qual o preço de cada uma? (Tabela 22.1 ajuda.)
4. A host table de um switch de POP mostra o mesmo MAC **alternando** entre duas portas, com **AGE** sempre zerando. O que está acontecendo, e por que é urgente? (Palavra-chave para investigar: loop — e o próximo capítulo do livro que trata disso é o de STP... que não existe; o assunto aparece no 7. Por ora, raciocine.)

Solução dos desafios

1. Para uma LAN de **um** cliente (casa, escritório), não: os dispositivos são do mesmo dono e confiança. Vira problema quando a mesma bridge carrega **clientes diferentes** — vizinhos de condomínio num switch, assinantes numa torre: um infectado ataca os outros em camada 2, invisível para o firewall do roteador. As respostas do volume: separar clientes em **VLANs** (Capítulo 23 em diante) — e, em bridges que precisam mesmo filtrar, o firewall de bridge/**use-ip-firewall** (pagando o offload, Tabela 22.1).
2. O RouterOS passa a ter **dois caminhos** para a mesma rede: a interface física (com o IP esquecido) e a bridge. ARP pode ser respondido pelos dois lados, rotas conectadas duplicam (uma por interface) e o tráfego ora sai por uma, ora por outra — sintomas: conexões que “às vezes funcionam”, ARP incompleto em vizinhos, e a host table confusa. Regra do capítulo: porta-membro não tem configuração própria; camada 3 mora **só** na bridge.
3. Opções: (a) **use-ip-firewall=yes** — o firewall IP passa a ver o tráfego em bridge; preço: **todo** o tráfego da bridge desce para a CPU (mata o offload geral — inaceitável num POP carregado); (b) **bridge filter** (regras de camada 2) — mais barato, mas igualmente processado em CPU para o que casar; (c) a resposta arquitetural: **tirar as duas portas da bridge comum** — VLANs separadas + roteamento entre elas no próprio CRS ou num roteador acima: o firewall vê o tráfego **na travessia de camada 3**, onde ele é natural, e o resto do POP continua em hardware. A opção (c) é o desenho do Capítulo 26.
4. MAC alternando entre portas = o mesmo quadro chegando por dois caminhos: ou há um **loop físico** (dois cabos fechando um anel entre switches — broadcasts circulam e multiplicam até saturar tudo: é uma *broadcast storm* em formação, urgência máxima), ou alguém está **forjando o MAC** (ataque/equipamento defeituoso). Ação imediata: identificar as duas portas e derrubar uma; ação estrutural: STP/RSTP ligado nas bridges de POP (o RouterOS traz RSTP por padrão) e desenho sem anéis não intencionais.

22.5 Resumo

- **Comutar** (camada 2, por MAC, dentro da rede) **rotear** (camada 3, por IP, entre redes) — a bridge é o switch de software do RouterOS: aprende, encaminha, inunda.
- Configuração de camada 3 (IP, DHCP, firewall, listas) mora **na bridge**; portas-membro são tomadas burras. A **host table** é o caderno do carteiro — primeira parada do diagnóstico.
- Tráfego entre portas da mesma bridge **não passa** pelo firewall **forward** — consciência disso é segurança.
- **Hardware offload** (flag H): o switch chip comuta em velocidade de fio; recursos de software o matam **em silêncio** (Tabela 22.1) — CPU alta + throughput baixo num CRS = procurar o H perdido.
- Nos CRS3xx, o VLAN filtering preserva o offload — a porta de entrada dos próximos capítulos.

No próximo capítulo, o divisor de águas da rede de provedor: **VLANs e 802.1Q** — como um cabo só carrega dezenas de redes separadas.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/interface bridge` (portas, host table, STP) e as tabelas de compatibilidade de hardware offload por switch chip — consulte a do seu modelo antes de projetar um POP. Tanenbaum; Feamster; Wetherall (2021) fundamenta a comutação em camada 2 (aprendizado, inundação, spanning tree). As referências completas, em ABNT, estão no apêndice [Referências](#).

23 VLANs e 802.1Q

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o problema que a VLAN resolve: **redes separadas sem cabos separados**;
- Descrever a etiqueta **802.1Q**: onde ela entra no quadro, o que é o **VID** e o que significa um quadro *tagged* ou *untagged*;
- Usar o vocabulário sem tropeçar: **trunk**, **access**, VLAN nativa;
- Criar interfaces `/interface vlan` no RouterOS e montar o *router-on-a-stick* — várias redes roteadas por um cabo só;
- Reconhecer as VLANs na rede do provedor: serviços, clientes e gerência compartilhando a mesma fibra sem se ver.

23.1 O problema: um cabo, várias redes

O capítulo anterior terminou com um alerta: colocar clientes diferentes na mesma bridge é colocá-los na mesma sala. A solução ingênua — um cabo e uma porta para cada rede — morre na primeira conta: o POP atende a rede de clientes, a de gerência, a de servidores e a VLAN da operadora... por **uma** fibra até a torre. Precisamos de redes logicamente separadas sobre o mesmo meio físico.

A resposta tem trinta anos e está em todo equipamento de rede do planeta: a **VLAN** (*Virtual LAN*), padronizada como **IEEE 802.1Q**. A ideia cabe numa frase: **cada quadro Ethernet ganha uma etiqueta dizendo a que rede pertence** — e switches e roteadores tratam cada etiqueta como se fosse um cabo separado.

A etiqueta (*tag*) são 4 bytes inseridos dentro do cabeçalho Ethernet (6), dos quais importam dois campos: o **VID** (*VLAN ID*, 12 bits — redes de 1 a 4094) e a **prioridade** (3 bits — que o QoS do Volume 7 conhece como CoS).

Com a etiqueta, nasce o vocabulário que você usará todos os dias:

- Quadro **tagged**: carrega a etiqueta — viaja pelos enlaces entre equipamentos de rede;
- Quadro **untagged**: quadro comum, sem etiqueta — o que computadores, câmeras e ONUs falam (dispositivos finais em geral ignoram VLANs);
- Porta/enlace **trunk** (tronco): transporta **várias** VLANs, todas tagged — a fibra do POP à torre;

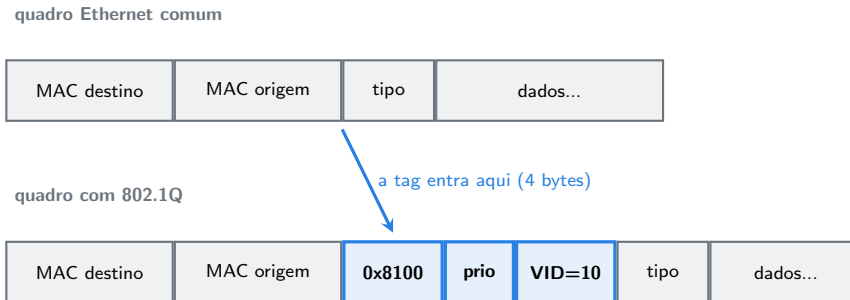


Figura 23.1: A etiqueta 802.1Q: 4 bytes entre o MAC de origem e o tipo — com o VID dizendo “este quadro pertence à VLAN 10”.

- Porta **access** (acesso): pertence a **uma** VLAN; recebe/entrega quadros untagged e o switch etiqueta/desetiqueta por ela — a porta do computador;
- **VLAN nativa**: a VLAN dos quadros untagged que chegam a um trunk (tradicionalmente a VID 1) — fonte clássica de surpresas de segurança; o 7 a domará com o conceito de **PVID**.

💡 Analogia para guardar

O 802.1Q transforma um cabo num **feixe de canos coloridos**: cada quadro ganha uma pulseira de cor (o VID) ao entrar, viaja pelo cano da sua cor e perde a pulseira ao sair para um dispositivo final. Trunk é o feixe inteiro; porta access é a boca de um cano só — e quem decide a cor é o switch, nunca o visitante.

23.2 VLANs no RouterOS: a interface vlan

No RouterOS, cada VLAN que o roteador deve **enxergar em camada 3** vira uma interface virtual sobre a interface física (ou sobre uma bridge):

```
/interface vlan add name=vlan10-clientes vlan-id=10 interface=ether2
/interface vlan add name=vlan20-gerencia vlan-id=20 interface=ether2
```

Cada **vlan** é uma interface completa: recebe IP, DHCP server, entra em interface lists, aparece no firewall — tudo que uma **ether** faz. Os quadros que saem por ela nascem tagged com o VID configurado; os que chegam tagged com aquele VID são entregues a ela.

O desenho que isso habilita é o **router-on-a-stick** (“roteador no palito”): um roteador pendurado por **um** cabo trunk num switch, roteando entre todas as VLANs:

```
/ip address add address=10.10.0.1/24 interface=vlan10-clientes
/ip address add address=10.20.0.1/24 interface=vlan20-gerencia
```

Duas redes, um cabo. O tráfego **dentro** de cada VLAN é comutado pelo switch (camada 2, sem tocar o roteador); o tráfego **entre** VLANs sobe o palito, é **roteado** — e aqui está o prêmio de segurança: a travessia passa pelo firewall **forward**, exatamente o que faltava no desafio do 13. Segmentar em VLANs devolve ao firewall a visão que a bridge única roubava.

Três notas de campo antes do laboratório:

1. **Os dois lados do enlace devem concordar:** VID 10 aqui é VID 10 lá. Uma ponta tagged e outra untagged = silêncio absoluto (o quadro chega, a etiqueta não casa, ninguém reclama). É o defeito número 1 de VLAN — e o motivo de documentar VIDs como se documenta endereçamento;
2. **MTU:** a tag rouba 4 bytes. Equipamentos modernos aceitam quadros de 1522 bytes sem drama; em rádios antigos e caminhos com túneis, VLANs somam-se ao problema de MTU que você já conhece do `change-mss` (4);
3. **/interface vlan não filtra** — ela apenas dá ao roteador presença nas VLANs. Impor “a porta X só fala VLAN 10” é papel do **bridge VLAN filtering**, o próximo capítulo. Aqui as duas pontas cooperam; lá, o switch policia.

! O conceito mais importante do capítulo

VLAN = **um cabo, várias redes**: a tag 802.1Q (VID) faz cada quadro pertencer a uma rede lógica, trunk transporta muitas (tagged), access entrega uma (untagged). No RouterOS, `/interface vlan` dá ao roteador um “pé” em cada VLAN — e rotear **entre** VLANs devolve o tráfego ao firewall. Regra de ouro do campo: as duas pontas do enlace precisam contar a mesma história de VIDs.

23.3 Laboratório

23.3.1 Ambiente

Topologia do volume (13): `lab-r1` com a `bridge-lan`, e `lab-r2/lab-r3` na rede `lab-a`. Neste capítulo, `r1` e `r2` conversarão por VLANs **sobre o mesmo enlace** que já os liga; o `r3` fará o papel de “intruso sem VLAN” na prova de isolamento.

23.3.2 Comandos passo a passo

Passo 1 — as VLANs no r1. Sobre a bridge da LAN (é o caso realista — o “palito” costuma ser uma bridge/trunk), crie duas VLANs com suas redes:

```
/interface vlan add name=vlan10-svc vlan-id=10 interface=bridge-lan comment="servicos"
/interface vlan add name=vlan20-ger vlan-id=20 interface=bridge-lan comment="gerencia de e
/ip address add address=10.10.0.1/24 interface=vlan10-svc
/ip address add address=10.20.0.1/24 interface=vlan20-ger
```

Passo 2 — o outro lado, no r2. O r2 precisa contar a mesma história de VLANs sobre a ether2:

```
/interface vlan add name=vlan10 vlan-id=10 interface=ether2
/interface vlan add name=vlan20 vlan-id=20 interface=ether2
/ip address add address=10.10.0.2/24 interface=vlan10
/ip address add address=10.20.0.2/24 interface=vlan20
```

Passo 3 — provar os canos. De lab-r2, um ping por cano:

```
/ping 10.10.0.1 count=3
/ping 10.20.0.1 count=3
```

Ambos limpos. Repare no que acabou de acontecer: pelo **mesmo cabo** (lab-a) onde já viviam a LAN untagged (192.168.10.0/24) e o DHCP, agora trafegam **mais duas redes**, cada uma no seu cano tagged — três redes, zero cabos novos.

Passo 4 — a prova do isolamento. O r3 está no mesmo cabo, **sem VLANs** configuradas. De lab-r3:

```
/ping 10.10.0.1 count=3
```

no route to host — para o r3, a VLAN 10 **não existe**: os quadros tagged passam pela placa dele e são descartados (etiqueta desconhecida). Mesmo que ele tivesse uma rota, não tem interface naquela rede. Agora dê a ele o “crachá” e veja a porta abrir:

```
/interface vlan add name=vlan10 vlan-id=10 interface=ether2
/ip address add address=10.10.0.3/24 interface=vlan10
/ping 10.10.0.1 count=3
```

Funciona. E é exatamente essa a limitação a entender: **quem controla a própria configuração pode “entrar” na VLAN** — num cabo compartilhado, a tag separa, mas não tranca. A tranca (a porta do switch decidindo quais VLANs aceita) é o bridge VLAN filtering do próximo capítulo. Remova o crachá do r3 ao terminar:

```
/ip address remove [find interface=vlan10]
/interface vlan remove [find name=vlan10]
```

Passo 5 — o roteador vendo os canos. Em lab-r1, o torch mostra os quadros por interface — observe a VLAN 10 enquanto o r2 pinga 10.10.0.1:

```
/tool torch interface=vlan10-svc
```

(Interrompa com q.) E as interfaces contam sua natureza:

```
/interface vlan print
/interface print where type=vlan
```

23.3.3 Verificação

1. r1 e r2 se pingam pelas **duas** VLANs, sobre o mesmo enlace da LAN untagged — três redes num cabo;
2. O r3 sem configuração de VLAN não alcança 10.10.0.1 (e você viu o erro certo: sem interface na rede, **no route to host**);
3. Com o “crachá” (interface vlan + IP), o r3 entrou — e você sabe explicar por que isso é uma limitação, não um recurso;
4. `/interface vlan print` nos dois roteadores conta a **mesma** história de VIDs (10 e 20).

23.3.4 Desafios

1. No r1 as VLANs foram criadas sobre a **bridge-lan**; no r2, sobre a **ether2**. Por que essa assimetria funciona — e quando criar a VLAN sobre a bridge é **obrigatório**?
2. Um provedor entrega ao cliente corporativo “internet na VLAN 100 e telefonia na VLAN 200” por uma fibra. Desenhe (comandos) o lado do cliente num RB5009: as duas VLANs sobre a **sfp-sfpplus1**, com DHCP client na 100 e um IP fixo 10.200.0.2/30 na 200.
3. O ping na VLAN 20 entre r1 e r2 funciona — mas deveria? A VLAN 20 é “gerência de equipamentos” e o r2 faz papel de CPE de cliente. Critique o desenho do laboratório com os olhos do Capítulo 21 e proponha a correção (que o Capítulo 26 consagrará).
4. Sobre o quadro da 6: um quadro já tagged com VID 10 pode receber **outra** tag por fora? Investigue o nome da técnica, o EtherType envolvido e um uso clássico de provedor.

Solução dos desafios

1. A tag viaja no quadro — não importa se quem a lê/escreve é uma interface física ou uma bridge: os VIDs casam e pronto. Criar sobre a **bridge é obrigatório** quando os quadros tagged podem chegar **por mais de uma porta** (o trunk pode migrar de porta, ou há redundância): a VLAN sobre a bridge enxerga todas as portas-membro; sobre uma **ether** específica, só aquele cabo. Regra prática moderna: se existe bridge, crie a VLAN sobre ela (com o filtering do próximo capítulo mandando no resto).

2.

```
/interface vlan add name=vlan100-inet vlan-id=100 interface=sfp-sfpplus1
/interface vlan add name=vlan200-voz vlan-id=200 interface=sfp-sfpplus1
/ip dhcp-client add interface=vlan100-inet
/ip address add address=10.200.0.2/30 interface=vlan200-voz
```

(E, sendo o RB5009 a borda do cliente: NAT com `out-interface=vlan100-inet` e o firewall do Volume 3 — as VLANs entram nas interface lists como qualquer WAN.)

3. O defeito: o CPE do cliente **tem um pé na VLAN de gerência** — qualquer cliente curioso (ou comprometido) com acesso ao próprio CPE alcança a rede que administra o parque. Ferido o princípio “gerência intocável” (Capítulo 18). Correção: gerência em VLAN que **só existe** entre equipamentos do provedor (o switch não a entrega em portas de cliente — filtering, próximo capítulo), CPEs gerenciados por outra via (a própria sessão PPPoE/túnel dedicado), e firewall barrando a VLAN 20 de qualquer origem não-provedor. No laboratório seguimos didáticos; no POP real, a VLAN 20 jamais desceria ao cliente.

4. Pode — é o **QinQ** (802.1ad): uma tag de serviço (S-TAG, EtherType 0x88a8) envelopando a tag do cliente (C-TAG, 0x8100). Uso clássico: o provedor transporta as VLANs **do cliente** (quaisquer VIDs que ele use) através do backbone dentro de uma única VLAN de serviço — “VLAN do cliente 55” viaja intacta por dentro da “VLAN de transporte 1055”. No RouterOS: `/interface vlan` sobre `/interface vlan`, ou `ethertype=0x88a8` no bridge VLAN filtering.

23.4 Resumo

- VLAN = redes lógicas sobre o mesmo físico: a tag **802.1Q** (4 bytes, 6) com o **VID** (1–4094) diz a que rede o quadro pertence.
- Vocabulário: **tagged** (com etiqueta, entre equipamentos), **untagged** (dispositivos finais), **trunk** (muitas VLANs), **access** (uma, untagged), VLAN nativa (untagged no trunk — cuidado).
- `/interface vlan` sobre ether/bridge dá ao roteador um pé em cada VLAN — IP, DHCP, firewall, tudo normal; **router-on-a-stick** roteia entre VLANs por um cabo, devolvendo a travessia ao firewall.
- As duas pontas devem contar a **mesma história de VIDs** — ponta errada não gera erro, gera silêncio.
- A tag separa, mas **não tranca** num cabo compartilhado: quem configura a própria ponta entra na VLAN — a tranca é o bridge VLAN filtering.

No próximo capítulo, exatamente ela: **bridge VLAN filtering** — o switch (de software e de hardware) decidindo quais VIDs entram e saem em cada porta: trunk e access de verdade, com PVID e tudo.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/interface vlan` e os cenários de combinação com bridge (incluindo os casos a evitar). O padrão IEEE 802.1Q define tag, VIDs

e prioridades; Tanenbaum; Feamster; Wetherall (2021) o apresenta didaticamente. As referências completas, em ABNT, estão no apêndice [Referências](#).

24 Bridge VLAN filtering

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Ativar o **VLAN filtering** numa bridge e explicar o que muda: o switch passa a **policar** etiquetas em vez de ignorá-las;
- Configurar portas **trunk** e **access** de verdade, com a tabela `/interface bridge vlan`, o **PVID** e os **frame-types**;
- Explicar o caminho de um quadro pelo switch filtrado: o que acontece na entrada (ingress) e na saída (egress) de cada porta;
- Manter o acesso de **gerência** ao próprio switch numa VLAN — incluindo o detalhe que derruba iniciantes: a bridge como membro tagged;
- Aplicar a ordem de configuração que **não** tranca você para fora.

24.1 Do consentimento à lei

O laboratório do Capítulo 23 terminou com uma demonstração incômoda: o r3 entrou na VLAN 10 sozinho, bastando configurar a própria placa. Num cabo compartilhado, a tag 802.1Q é um **acordo entre pontas** — e acordos não protegem contra quem não os assina.

O **bridge VLAN filtering** transforma o acordo em lei: com ele ativo, a bridge (o switch) passa a **decidir, porta a porta**, quais VIDs entram, quais saem, e com ou sem etiqueta. O cliente pode etiquetar o que quiser; se a porta dele só admite VLAN 10 untagged, é isso que ele terá.

Liga-se com um parâmetro:

```
/interface bridge set bridge-sw vlan-filtering=yes
```

— mas **não rode isso ainda**. A ordem importa (é a pegadinha número 1 do capítulo, tratada adiante). Antes, entenda as três peças que passam a mandar.

24.1.1 Peça 1: a tabela de VLANs da bridge

/interface bridge vlan é o mapa: para cada VLAN, quem participa e como:

```
/interface bridge vlan add bridge=bridge-sw vlan-ids=10 tagged=bridge-sw,ether2 untagged=e
```

Traduzindo: “a VLAN 10 existe nesta bridge; sai **tagged** pelo trunk **ether2** (e para a própria bridge — já explico); sai **untagged** pela porta de acesso **ether3**”. Na **saída** (egress) de cada porta, essa tabela é lei: quadro da VLAN 10 saindo pela ether3 perde a etiqueta; pela ether2, mantém; por uma porta que não está na linha, **não sai**.

24.1.2 Peça 2: o PVID — a etiqueta de entrada

E o quadro **untagged** que *chega* a uma porta — de que VLAN ele é? A resposta é o **PVID** (*Port VLAN ID*) da porta:

```
/interface bridge port set [find interface=ether3] pvid=10
```

Todo quadro sem etiqueta que entrar pela **ether3** é internamente tratado como VLAN 10. PVID é o “carimbo de entrada” — e o par tabela-de-saída + PVID-de-entrada define os dois papéis clássicos:

Tabela 24.1: As receitas de porta access e trunk

Papel	Na tabela vlan	PVID	frame-types
access (VLAN 10)	untagged=porta na VLAN 10	10	admit-only-untagged-and-priority-tagged
trunk	tagged=porta em cada VLAN transportada	(irrelevante se só aceita tagged)	admit-only-vlan-tagged

24.1.3 Peça 3: frame-types — o que a porta admite

O terceiro controle é o filtro de **entrada** (ingress): que tipos de quadro a porta aceita?

- **admit-all** (padrão): tagged e untagged;
- **admit-only-untagged-and-priority-tagged**: só untagged — a porta access ortodoxa: se o cliente mandar quadro etiquetado (como o r3 “esperto” do capítulo passado), **descarte**;
- **admit-only-vlan-tagged**: só tagged — o trunk ortodoxo: quadro sem etiqueta num tronco é acidente ou ataque à VLAN nativa; descarte.

Com as três peças, o caminho do quadro fecha: **ingress** (frame-types admite? untagged ganha o PVID) → comutação **dentro da VLAN** (host table por VLAN) → **egress** (a tabela diz se sai e se sai tagged/untagged).

Analogia para guardar

O switch filtrado é uma **portaria de eventos com pulseiras**: na entrada, o segurança da porta confere o traje (frame-types) e coloca a pulseira da casa em quem chega sem (PVID); lá dentro, cada salão é uma VLAN; na saída, a porta VIP mantém a pulseira (tagged, trunk) e a porta do salão único a corta (untagged, access). Pulseira trazida de casa não vale: quem manda é a portaria.

24.2 A gerência e a pegadinha do tagged=bridge

Releia a linha da tabela: `tagged=bridge-sw,ether2`. Por que a **própria bridge** aparece como membro tagged?

Porque o switch também é um equipamento com IP de gerência — e se a gerência mora numa VLAN (como deve, Capítulo 26), os quadros dela precisam chegar **à CPU do próprio switch**. Na arquitetura do RouterOS, a CPU é alcançada “saindo” pela interface bridge: incluir a bridge como **tagged** na VLAN é abrir o cano entre aquela VLAN e o cérebro do equipamento — e é sobre esse cano que a `/interface vlan` de gerência (com o IP) se pendura:

```
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-sw
/ip address add address=10.99.0.2/24 interface=vlan99-ger
```

Esquecer o **bridge** na lista tagged da VLAN 99 = gerência morta no momento em que o filtering ligar. O que nos leva à pegadinha-mor:

Aviso

vlan-filtering=yes é a última linha, nunca a primeira. No momento em que o filtering liga, a tabela de VLANs vira lei — e tudo que não está nela **para de passar**, incluindo o seu acesso. A ordem segura: (1) montar a tabela `/interface bridge vlan` completa (com a bridge nos tagged das VLANs de gerência); (2) configurar PVIDs e frame-types das portas; (3) conferir tudo com **print**; (4) **só então** ligar o filtering — de preferência com acesso por um caminho fora da bridge (nossa `ether1` de gerência do laboratório, o console serial no CRS real) e Safe Mode.

Uma nota de desempenho para fechar a teoria: nos **CRS3xx**, tudo isso — tabela, PVID, frame-types — é programado **no switch chip**: filtering com flag H viva, em velocidade de fio (13). Nos CRS1xx/2xx antigos, ligar o filtering derruba o tráfego para a CPU — lá, usa-se o menu legado do chip (Capítulo 25). Saber **qual** dos dois mundos é o seu equipamento é o primeiro passo de qualquer projeto de POP.

! O conceito mais importante do capítulo

Com VLAN filtering, quem manda é o **switch**, não as pontas: ingress (frame-types + PVID) decide o que entra e com que VLAN; a tabela `/interface bridge vlan` decide por onde sai e se sai tagged. Porta access e trunk são só combinações dessas peças (Tabela 24.1). E a liturgia de ativação — tabela primeiro, bridge tagged para gerência, filtering por último — é o que separa a segmentação do POP de uma viagem até ele.

24.3 Laboratório

24.3.1 Ambiente

Os três CHRs com papéis novos: **lab-r2 será o switch**. No VirtualBox, devolva o Adaptador 2 de lab-r3 para a rede interna **lab-b** (desfazendo o ajuste do 13). Topologia:

- lab-r1 (roteador): ether2 → lab-a — já tem a `vlan10-svc` (10.10.0.1/24) sobre a `bridge-lan` (Capítulo 23);
- lab-r2 (switch): ether2 → lab-a (**trunk** para o r1), ether3 → lab-b (**access VLAN 10** para o r3). Gerência intacta pela ether1 (fora da bridge — nosso “console serial”);
- lab-r3 (dispositivo final): ether2 → lab-b, **sem nenhuma configuração de VLAN** — o ponto do exercício.

Limpe no r2 as interfaces vlan do capítulo anterior (`/interface vlan remove [find]` após remover os IPs correspondentes: `/ip address remove [find interface~"vlan"]`).

24.3.2 Comandos passo a passo

Passo 1 — a bridge do switch. Em lab-r2:

```
/interface bridge add name=bridge-sw comment="switch do lab"
/interface bridge port add bridge=bridge-sw interface=ether2
/interface bridge port add bridge=bridge-sw interface=ether3
```

Passo 2 — a tabela ANTES do filtering. A VLAN 10 sai tagged pelo trunk e untagged pela porta do r3; e já deixamos a bridge como membro tagged (gerência futura na VLAN — desafio 3 usa):

```
/interface bridge vlan add bridge=bridge-sw vlan-ids=10 tagged=bridge-sw,ether2 untagged=ether3
```

Passo 3 — os papéis das portas. Trunk ortodoxo na ether2, access ortodoxo na ether3:

```
/interface bridge port set [find interface=ether2] frame-types=admit-only-vlan-tagged
/interface bridge port set [find interface=ether3] pvid=10 frame-types=admit-only-untagged
```

Passo 4 — conferir, e só então ligar. A liturgia:

```
/interface bridge vlan print
/interface bridge port print
/interface bridge set bridge-sw vlan-filtering=yes
```

(Seu SSH ao r2 chega pela **ether1**, fora da bridge — o filtering não o toca. Num CRS real, este seria o momento do Safe Mode.)

Passo 5 — o milagre do access. Em lab-r3 — que não sabe o que é VLAN — apenas um endereço na rede da VLAN 10:

```
/ip address add address=10.10.0.30/24 interface=ether2
/ping 10.10.0.1 count=3
```

Funciona. Acompanhe o quadro: sai do r3 untagged → entra na ether3 do switch (frame-types admite; PVID carimba VLAN 10) → comutado na VLAN 10 → sai pelo trunk ether2 **tagged** → chega ao r1, cuja **vlan10-svc** sobre a bridge o reconhece. O r3 nunca soube da etiqueta — o switch cuidou de tudo. É exatamente assim que a ONU do cliente recebe “a VLAN de PPPoE” sem saber.

Passo 6 — a lei nas portas. Duas tentativas de violação:

- (a) Em lab-r3, banque o esperto do capítulo passado — tente entrar tagged pela porta access:

```
/interface vlan add name=vlan10 vlan-id=10 interface=ether2
/ip address add address=10.10.0.31/24 interface=vlan10
/ping 10.10.0.1 count=3
```

Silêncio total — o **frame-types** da access descarta quadros etiquetados na entrada. A tranca que faltava no Capítulo 23 existe. Desfaça (`/ip address remove [find interface=vlan10]`, `/interface vlan remove vlan10`).

- (b) Em lab-r1, tente uma VLAN que **não está na tabela** do switch:

```
/interface vlan add name=vlan30-teste vlan-id=30 interface=bridge-lan
/ip address add address=10.30.0.1/24 interface=vlan30-teste
```

De lab-r3... nada a testar: a VLAN 30 morre no trunk do r2 (egress: não está na tabela, não sai por porta nenhuma). Confirme no r2 que só a 10 (e a 1 padrão) existem: `/interface bridge vlan print`. Limpe o teste no r1.

24.3.3 Verificação

1. `/interface bridge vlan print` no r2 mostra a VLAN 10 com `tagged=bridge-sw, ether2` e `untagged=ether3` (a flag D da VLAN 1 padrão é normal);
2. O r3, **sem configuração de VLAN**, pinga 10.10.0.1 através do switch — access + PVID funcionando;
3. As duas violações do Passo 6 falharam em silêncio (tagged na access; VID fora da tabela no trunk);
4. Sua gerência ao r2 continuou viva o tempo todo (ela está fora da bridge — e você sabe por que isso importou).

24.3.4 Desafios

1. No Passo 6a, o quadro do r3 morreu na **entrada** da porta access. Se o `frame-types` fosse o padrão `admit-all`, o ataque funcionaria? Percorra ingress → comutação → egress e responda com precisão.
2. Migre a gerência do **r2** para dentro da VLAN 99 pelo trunk: acrescente a VLAN 99 à tabela (quem é tagged?), crie a `/interface vlan` sobre a bridge com IP 10.99.0.2/24 e a contraparte no r1 (10.99.0.1/24 sobre a bridge-lan). Liste os comandos nos dois lados e o teste. (Não desative a ether1 — rede de segurança do laboratório.)
3. Um trunk com `pvid=1` e `frame-types=admit-all` transporta as VLANs 10 e 20. Explique o ataque clássico de **VLAN hopping** por dupla etiqueta (802.1Q-in-802.1Q) que essa configuração permite — e como as receitas da Tabela 24.1 o previnem.
4. No CRS326 do POP, você ligou `vlan-filtering=yes` e o tráfego continuou em velocidade de fio; no CRS125 antigo do armário, a CPU foi a 100%. Explique a diferença — e onde a configuração de VLAN do CRS125 deveria viver (dica: próximo capítulo).

Solução dos desafios

1. Com `admit-all` no ingress, o quadro tagged VID 10 **entraria**. Comutação: ele pertence à VLAN 10, que existe na bridge — segue. Egress pelo trunk: a tabela permite a 10 tagged — **sai**. Ou seja: sim, o ataque voltaria a funcionar; a única defesa da porta access contra etiquetas trazidas de casa é o `frame-types` restritivo. (O PVID não ajuda aqui — ele só age sobre quadros *sem* etiqueta.)

2. No r2:

```
/interface bridge vlan add bridge=bridge-sw vlan-ids=99 tagged=bridge-sw, ether2
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-sw
/ip address add address=10.99.0.2/24 interface=vlan99-ger
```

No r1:

```
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-lan
/ip address add address=10.99.0.1/24 interface=vlan99-ger
```

Teste: do r1, `/ping 10.99.0.2`. Os pontos didáticos: a VLAN 99 é tagged **no trunk e na própria bridge** (sem `bridge-sw` na lista, o IP de gerência nunca veria os quadros), e **não** aparece na ether3 — o r3 não tem como alcançá-la.

3. O atacante numa porta com PVID 1 envia quadro com **duas** tags: externa VID 1, interna VID 20. O switch remove a externa (casa com o PVID/nativa do trunk em alguns cenários de egress untagged) e reenvia — o quadro chega ao próximo switch **com a tag interna exposta** (VID 20): o atacante injetou tráfego na VLAN 20 sem pertencer a ela. As receitas previnem por três lados: trunk `admit-only-vlan-tagged` sem VLAN nativa untagged (nada sai untagged no trunk), PVID de access em VLAN sem privilégio, e a VLAN 1 fora do uso (nem na tabela).

4. O CRS326 (série 3xx) programa o filtering **no switch chip** — a CPU só vê o que é para ela (gerência); dados correm em hardware com a flag H. O CRS125 (série 1xx) não sabe fazer 802.1Q da bridge no chip: `vlan-filtering=yes` derruba a comutação para a CPU — 100% e vazão de roteadorzinho. No CRS125, a VLAN se configura no **menu legado do switch chip** (`/interface ethernet switch`), assunto do Capítulo 25 — o mesmo resultado, programado na linguagem daquele hardware.

24.4 Resumo

- VLAN filtering transforma a bridge em **switch que polícia**: ingress (`frame-types + PVID` carimbando untagged), comutação por VLAN, egress pela tabela `/interface bridge vlan` (sai/não sai; tagged/untagged).
- **Access** = untagged na tabela + PVID + só-untagged; **trunk** = tagged na tabela + só-tagged (Tabela 24.1) — e etiqueta trazida pelo cliente morre na porta.
- Gerência em VLAN exige a **bridge como membro tagged** e a `/interface vlan` sobre a bridge — esquecer é gerência morta.
- Liturgia de ativação: **tabela** → **portas** → **conferir** → **filtering por último**, com acesso alternativo garantido (console/ether fora da bridge) — `vlan-filtering=yes` prematuro é a viagem clássica ao POP.
- CRS3xx: filtering em hardware (H vivo); CRS1xx/2xx: filtering na CPU — lá, o caminho é o switch chip legado (próximo capítulo).

No próximo capítulo, descemos ao porão: o **switch chip** dos CRS — o menu legado, as diferenças entre gerações e como extrair velocidade de fio de cada uma.

Bibliografia do capítulo

A página *Bridge VLAN Filtering* da documentação oficial (MikroTik, 2026a) é a referência canônica — inclusive a tabela de quais modelos fazem filtering em hardware. Os guias *Basic VLAN switching* da mesma documentação cobrem cenários por família de produto. As referências completas, em ABNT, estão no apêndice [Referências](#).

25 Switch chip nos CRS

Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que um **switch chip** faz e como o RouterOS o expõe em `/interface ethernet switch`;
- Escolher o caminho de configuração certo por família — **CRS3xx/5xx** (tudo pela bridge, offload automático) *versus* **CRS1xx/2xx** e **RouterBOARDS** (menu legado do chip);
- Usar os recursos que moram no chip além de VLAN: **rule table** (ACLs em velocidade de fio), **port mirroring** e a host table de hardware;
- Reconhecer os limites de cada chip (tamanho de tabelas, recursos) na ficha técnica antes de projetar o POP;
- Diagnosticar o clássico “CRS lento”: descobrir para onde o tráfego está indo — chip ou CPU.

25.1 O porão do switch

Dois capítulos falamos de bridge como se tudo fosse software — e o 13 avisou: nos CRS, quem trabalha de verdade é o **switch chip**, um circuito dedicado (Marvell nos CRS3xx/5xx, Atheros/ Qualcomm nos antigos e nas RouterBOARDS) com suas próprias tabelas de MACs, de VLANs e de regras, comutando em velocidade de fio enquanto a CPU dorme.

O RouterOS expõe o porão em `/interface ethernet switch`:

```
/interface ethernet switch print
```

```
[admin@crs326] > /interface ethernet switch print
Columns: NAME, TYPE, MIRROR-SOURCE, MIRROR-TARGET
# NAME      TYPE          MIRROR-SOURCE  MIRROR-TARGET
0 switch1   Marvell-98DX3236  none           none
```

Ali você vê o chip pelo nome (pesquise-o: a página *Switch Chip Features* da documentação diz exatamente o que **o seu** sabe fazer) e os menus dele: portas, host table de hardware, rule table, espelhamento. Todo equipamento MikroTik com mais de uma porta comutável tem um — até o hEX da casa do cliente.

25.2 Dois mundos, uma decisão

A pergunta prática que define qualquer projeto de switching MikroTik: **por onde configurar as VLANs?** A resposta depende da geração (Figura 25.1):

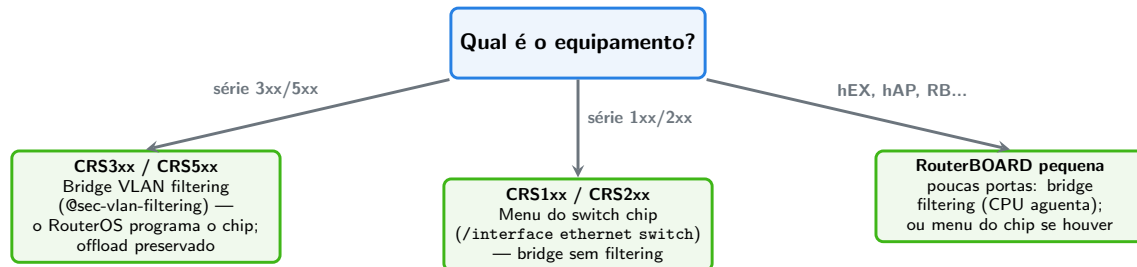


Figura 25.1: A decisão de projeto: em qual “idioma” configurar o switching, conforme a geração do hardware.

- **CRS3xx/5xx — o mundo unificado.** Você configura **só a bridge** (7) e o RouterOS traduz para o chip: tabela de VLANs, PVIDs, tudo vira programação de hardware, com a flag **H** intacta. É o presente e o futuro — e o motivo de este livro ter ensinado a bridge primeiro;
- **CRS1xx/2xx — o mundo legado.** O chip (QCA8511 e afins) é capaz — mas não fala o idioma da bridge. As VLANs se configuram **no menu do chip**: `/interface ethernet switch vlan` (a tabela), ... `port` (modo de cada porta: `vlan-mode`, `vlan-header`, `default-vlan-id` — o PVID daquele mundo). A bridge existe só para agrupar, **sem** `vlan-filtering`. Os conceitos são os mesmos do 7 — muda a sintaxe;
- **RouterBOARDs pequenas** — o hEX e irmãos têm chips básicos: para meia dúzia de portas de escritório, o bridge filtering na CPU resolve; onde o chip suportar (`switch1` com menu de VLAN), o legado serve. Consulte sempre a página do chip.

⚠ Aviso

Não misture os dois idiomas. Bridge com `vlan-filtering=yes` e VLANs no menu do switch chip ao mesmo tempo é receita de comportamento indefinido — cada geração tem seu caminho, e a documentação do modelo diz qual. O sintoma de mistura (ou do idioma errado): o “CRS lento” — CPU alta, throughput de software. Diagnóstico em dois comandos: `/interface bridge port print` procurando **H** ausente, e `/system resource print` com CPU suspeita sob tráfego que “deveria ser de graça”.

25.3 O que mais mora no chip

VLAN é o prato principal, mas o porão tem mais ferramentas — todas em velocidade de fio:

Rule table — ACLs de hardware. Uma tabela de regras estilo firewall (matchers de MAC, IP, porta; ações de drop/redirect/mirror) processada **pelo chip**:

```
/interface ethernet switch rule add switch=switch1 ports=ether5 src-mac-address=!AA:BB:CC:
```

(`new-dst-ports=""` = descartar.) É o jeito de impor “nesta porta, só este equipamento” num POP — sem gastar um ciclo de CPU. As regras variam por chip; as dos CRS3xx são particularmente ricas (é o embrião do *Layer 3 hardware offloading* dos CRS3xx/5xx mais novos — roteamento no chip, fora do nosso escopo, mas bom saber que existe).

Port mirroring — o grampo legítimo. Copiar o tráfego de uma porta para outra, onde um analisador escuta:

```
/interface ethernet switch set switch1 mirror-source=ether5 mirror-target=ether10
```

No **@sec-torch-sniffer** você verá o sniffer do próprio RouterOS; o mirror é a versão “de verdade” para depurar a porta de um cliente sem entrar no caminho.

Host table de hardware. A prima da `/interface bridge host`: `/interface ethernet switch host print` mostra os MACs que o **chip** conhece — e aceita entradas estáticas (prender um MAC a uma porta). Nos CRS3xx integrada à da bridge; nos legados, é a que vale.

E os **limites**: cada chip tem tabelas finitas — tantos mil MACs, tantas VLANs ativas, tantas regras de ACL. A ficha técnica (2) e a página *Switch Chip Features* trazem os números; um POP com mais MACs que a tabela do chip degrada para inundação (o switch vira hub para os excedentes). Projetar é conferir os números **antes**.

! O conceito mais importante do capítulo

O switch chip é um **computador de comutação com tabelas próprias** — VLANs, MACs, ACLs — e o seu trabalho é programá-lo no idioma certo da geração: **bridge filtering nos CRS3xx+** (o RouterOS traduz), **menu do chip nos legados**. Misturar idiomas ou exceder as tabelas derruba o tráfego para a CPU — e o “switch de 10 Gbps lento” quase sempre é isso: hardware esperando alguém falar a língua dele.

25.4 Laboratório

25.4.1 Ambiente

O CHR **não tem switch chip** — este laboratório é em parte “seco” (leitura de especificações e projeto), em parte executável em qualquer RouterBOARD física que você tenha (um hEX serve: chip básico, menus presentes). Sem hardware, faça os passos de projeto e os desafios — o raciocínio é o produto.

25.4.2 Comandos passo a passo

Passo 1 — conhecer o chip (hardware físico). Numa RouterBOARD:

```
/interface ethernet switch print
/interface ethernet switch port print
```

Anote o modelo do chip. Abra a página *Switch Chip Features* da documentação e localize a linha dele: quantas entradas de host table? Rule table existe? VLAN table de que tamanho?

Passo 2 — a host table de hardware. Com dispositivos ligados às portas:

```
/interface ethernet switch host print
```

Compare com `/interface bridge host print` — no hEX, as duas contam a mesma história por caminhos diferentes (chip × software).

Passo 3 — projeto no papel: o POP de 24 portas. Sem equipamento, este passo vale por dois. Dado um **CRS326-24G-2S+** (chip Marvell, série 3xx) servindo um POP: uplink SFP+ 1 = trunk para o roteador de borda; SFP+ 2 = trunk para a torre; ether1–20 = clientes dedicados (cada um numa VLAN própria, 101–120); ether21–24 = equipamentos do provedor (VLAN 99 de gerência). Escreva, em `.rsc`, a configuração completa no idioma **certo** para a série 3xx — bridge, tabela de VLANs, PVIDs, frame-types, interface de gerência. (Solução no desafio 1 — tente antes de olhar.)

Passo 4 — o mesmo POP num CRS125. Reescreva o miolo (as VLANs de duas portas de cliente e a gerência) no idioma legado: `/interface ethernet switch vlan + ... port` com `vlan-mode=secure`, `vlan-header` e `default-vlan-id`. Compare o esforço — e entenda por que a série 3xx venceu.

25.4.3 Verificação

1. (Com hardware) Você identificou o chip do seu equipamento e localizou seus limites na documentação;
2. O `.rsc` do Passo 3 segue a liturgia do 7 (tabela → portas → filtering por último) e a gerência mora na VLAN 99 com a bridge como tagged;
3. Você sabe apontar, nos dois idiomas, onde vive cada conceito: tabela de VLANs, PVID, admissão de quadros;
4. Dado um “CRS lento”, você lista os dois comandos de diagnóstico e os três suspeitos (idioma errado, mistura, tabela estourada).

25.4.4 Desafios

1. Escreva o `.rsc` do Passo 3 (CRS326). Estruture: bridge com todas as portas; VLANs 101–120 (cada uma tagged no uplink SFP+1 e untagged na porta do cliente); VLAN 99 tagged nos dois SFP+ e untagged nas ether21–24; PVIDs e frame-types; `vlan-filtering=yes` ao final. (Abrevie as 20 VLANs de cliente mostrando 2 e indicando o padrão.)
2. No desenho do desafio 1, cada cliente está **sozinho** na própria VLAN — clientes não se enxergam em camada 2. Que problema clássico de condomínio/POP isso resolve? E o que o roteador de borda precisa ter para que os clientes continuem chegando à internet?
3. Um POP com CRS326 tem 6 mil dispositivos atrás de uma porta (uma torre PtMP inteira). A host table do chip suporta 16 mil MACs. Outro integrador propõe ligar **três** torres iguais no mesmo switch. Faça a conta e diga o risco — e duas mitigações de projeto.
4. A rule table permite “nesta porta, só este MAC”. Por que essa regra é ao mesmo tempo uma ótima defesa num POP e uma péssima ideia numa porta de torre PtMP? O que usar em cada caso?

Solução dos desafios

1. Esqueleto (padrão indicado, 2 de 20 clientes mostrados):

```
/interface bridge add name=bridge-pop
/interface bridge port
add bridge=bridge-pop interface=sfp-sfpplus1 frame-types=admit-only-vlan-tagged
add bridge=bridge-pop interface=sfp-sfpplus2 frame-types=admit-only-vlan-tagged
add bridge=bridge-pop interface=ether1 pvid=101 frame-types=admit-only-untagged-and-prio
add bridge=bridge-pop interface=ether2 pvid=102 frame-types=admit-only-untagged-and-prio
# ... ether3-20: pvid=103..120, mesmo frame-types
add bridge=bridge-pop interface=ether21 pvid=99 frame-types=admit-only-untagged-and-prio
# ... ether22-24 idem
/interface bridge vlan
add bridge=bridge-pop vlan-ids=101 tagged=bridge-pop,sfp-sfpplus1 untagged=ether1
add bridge=bridge-pop vlan-ids=102 tagged=bridge-pop,sfp-sfpplus1 untagged=ether2
# ... 103-120 no padrao
add bridge=bridge-pop vlan-ids=99 tagged=bridge-pop,sfp-sfpplus1,sfp-sfpplus2 untagged=e
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-pop
/ip address add address=10.99.1.2/24 interface=vlan99-ger
/interface bridge set bridge-pop vlan-filtering=yes
```

(Refinamento pró: `tagged=bridge-pop` só é necessário nas VLANs que a CPU precisa ver — a 99; nas de cliente, pode sair.)

2. Resolve o **vizinho bisbilhoteiro/atacante**: sem camada 2 comum, não há ARP spoofing entre clientes, não há vírus varrendo o condomínio, não há briga de DHCP

— cada cliente enxerga só o provedor. O roteador de borda precisa **ter presença em cada VLAN** (`/interface vlan 101–120`, tipicamente agregadas por PPPoE ou DHCP por VLAN) e rotear/NAT-ear — o router-on-a-stick do Capítulo 23 em escala. É o desenho que o Capítulo 26 consolida.

3. $3 \times 6\,000 = 18\,000$ MACs $> 16\,000$ da tabela: sob carga plena, ~2 mil MACs não cabem — o chip **inunda** o tráfego deles por todas as portas da VLAN: banda desperdiçada, privacidade vazada, CPU do que escuta castigada. Mitigações: (a) **quebrar o domínio L2**: rotear na torre (cada torre com sua sub-rede — os MACs de clientes morrem lá e o switch vê só o MAC do roteador da torre); (b) distribuir torres em switches distintos / limitar aprendizagem por porta. A lição: host table é recurso de projeto, não detalhe.

4. Na porta de um **cliente dedicado/servidor do POP**, o MAC é um e conhecido: a regra elimina troca de equipamento não autorizada e spoofing — ótima. Numa porta de **torre PtMP**, chegam **milhares** de MACs legítimos (todos os clientes atrás do rádio): travar em um MAC derruba todo mundo. Lá, o controle certo é por VLAN + os mecanismos da camada wireless (access-list do Volume 5) e, para contenção de L2, isolamento entre clientes no próprio rádio.

25.5 Resumo

- O **switch chip** é o comutador de verdade dos CRS e RouterBOARDS — tabelas próprias de MACs, VLANs e regras, expostas em `/interface ethernet switch`.
- A decisão de idioma (Figura 25.1): **CRS3xx/5xx = bridge VLAN filtering** (RouterOS programa o chip, H vivo); **CRS1xx/2xx = menu legado do chip**; nunca os dois juntos.
- Além de VLAN: **rule table** (ACLs em fio), **port mirroring** (grampo de diagnóstico), host table de hardware com entradas estáticas.
- Tabelas são **finitas**: MACs, VLANs e regras têm teto por chip — estourar = inundação silenciosa; projetar é conferir a página *Switch Chip Features* antes.
- “CRS lento” = tráfego na CPU: procure o H perdido, o idioma errado ou a tabela cheia.

No próximo capítulo, o volume fecha com o projeto integrador: a **segmentação completa de um POP** — gerência, clientes e serviços, do switch ao firewall.

Bibliografia do capítulo

As páginas *Switch Chip Features* (tabela por modelo) e *CRS3xx, CRS5xx switch chip features* da documentação oficial (MikroTik, 2026a) são leitura obrigatória de projeto; os manuais de cada CRS indicam o idioma de configuração suportado. As referências completas, em ABNT, estão no apêndice [Referências](#).

26 Segmentação da rede do provedor

i Objetivos de aprendizagem

Este capítulo integra o volume. Ao final, você será capaz de:

- Projetar o **plano de VLANs** de um POP: gerência, clientes e serviços, com numeração e endereçamento documentados;
- Aplicar os três princípios da segmentação de provedor: gerência isolada, cliente não vê cliente, e **toda travessia entre segmentos passa pelo firewall**;
- Montar o desenho completo — roteador de borda + switch filtrado + segmentos — combinando os quatro capítulos do volume;
- Policiar a fronteira entre segmentos com o firewall do Volume 3 (interface lists por VLAN);
- Reconhecer o mesmo desenho nos cenários reais: POP de fibra (VLANs até a OLT), torre de rádio e o condomínio corporativo.

26.1 A ordem de serviço

O provedor cresceu: o POP central, que era um roteador e um switch burro, vai ser reorganizado. A ordem de serviço:

POP Centro — reorganização de rede Segmentos: **gerência** (equipamentos do provedor), **clientes dedicados** (cada um isolado) e **serviços** (servidores do provedor: monitoramento, cache). Requisitos: cliente não enxerga cliente nem gerência; gerência alcançável só pela equipe; toda travessia entre segmentos passa pelo firewall da borda; switch comutando em hardware. Entregáveis: plano de VLANs, configuração de switch e borda, testes.

O primeiro entregável não é comando — é **tabela**. Todo POP bem operado tem um plano de VLANs escrito, e ele é o contrato entre quem configura o switch, quem configura a borda e quem chega de madrugada para consertar:

Tabela 26.1: O plano de VLANs do POP (extrato)

VLAN	Nome	Rede	Quem participa
99	gerência	10.99.0.0/24	equipamentos do provedor + borda
101	cliente-A	100.64.101.0/24	porta do cliente A + borda

VLAN	Nome	Rede	Quem participa
102	cliente-B	100.64.102.0/24	porta do cliente B + borda
200	serviços	10.200.0.0/24	servidores do POP + borda

Repare nas decisões embutidas: numeração com **significado** (99 = infra, 1xx = clientes, 2xx = serviços — o padrão se estende sem reunião); rede casada com a VLAN (o terceiro octeto repete o número — diagnóstico agradece); e **a borda participa de todas** — ela é o único ponto de travessia.

26.2 Os três princípios

Tudo que este volume ensinou converge em três frases:

1. **Gerência isolada.** A VLAN 99 existe apenas entre equipamentos do provedor: o switch não a entrega em porta de cliente (tabela de VLANs, 7), e o firewall da borda só a aceita da equipe (interface list **gerencia**, Capítulo 17). Quem compromete um CPE não ganha um pé na sua infraestrutura;
2. **Cliente não vê cliente.** Cada cliente na própria VLAN (Capítulo 25, desafio 2): sem camada 2 comum, morrem o ARP spoofing entre vizinhos, o vírus que varre o condomínio e a briga de DHCPs. O único caminho entre dois clientes é **subir à borda** — onde há firewall;
3. **Travessia só pelo firewall.** É a soma dos outros dois com o router-on-a-stick (Capítulo 23): o tráfego intra-VLAN é comutado em hardware (rápido, invisível — e tudo bem, é tráfego do mesmo dono); o tráfego **entre** VLANs é roteado na borda, atravessando o **forward** do Volume 3. Segmentar é, no fundo, **decidir o que o firewall vê**.

A Figura 26.1 mostra o desenho completo:

26.3 O firewall enxergando segmentos

O toque final vem do Volume 3, com as interface lists absorvendo as VLANs. Na borda, cada `/interface vlan` entra na lista do seu papel:

```
/interface list member add list=gerencia interface=vlan99-ger
/interface list member add list=LAN interface=vlan101-cliA
/interface list member add list=LAN interface=vlan102-cliB
/interface list add name=SERVICOS
/interface list member add list=SERVICOS interface=vlan200-srv
```

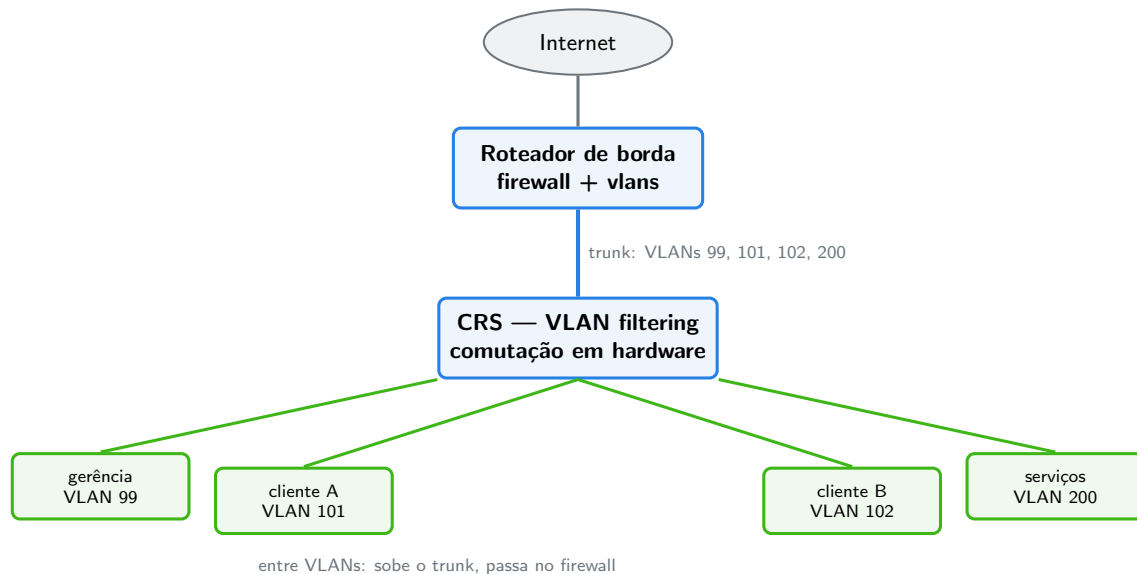



Figura 26.1: O POP segmentado: um trunk entre borda e switch; segmentos em portas access; travessias só pela borda.

E o firewall-base (Capítulo 21) já sabe o que fazer: gerência com acesso total ao input, clientes como LAN (DNS/DHCP servidos, gerência intocável), e as políticas novas entre segmentos viram poucas regras de forward — por exemplo, “clientes alcançam os serviços do POP só no que é público”:

```
/ip firewall filter add chain=forward in-interface-list=LAN out-interface-list=SERVICOS pr
/ip firewall filter add chain=forward in-interface-list=LAN out-interface-list=SERVICOS ac
```

Perceba o que aconteceu no desenho como um todo: o switch garante **quem está em que segmento** (camada 2, em hardware); o firewall garante **o que atravessa entre segmentos** (camada 3, com estado). Cada camada faz o que sabe — é a defesa em camadas do Capítulo 21 ganhando geografia.

! O conceito mais importante do capítulo

Segmentar é **desenhar o que o firewall vê**: VLANs (no switch, em hardware) definem os territórios; a borda, única ponte entre eles, aplica a política. Os três princípios — gerência isolada, cliente não vê cliente, travessia só pelo firewall — cabem em qualquer escala: do condomínio ao POP regional, muda o tamanho da 8, não o desenho.

26.4 Laboratório

26.4.1 Ambiente

O POP em miniatura, com os papéis do capítulo anterior: **lab-r1** = borda, **lab-r2** = switch (bridge **bridge-sw** com filtering do 7), **lab-r3** = cliente A. A VLAN 10 do capítulo anterior será renumerada para o plano oficial (8): 101 = cliente A, 99 = gerência.

26.4.2 Comandos passo a passo

Passo 1 — o plano no switch. Em **lab-r2**, ajuste a tabela ao plano (a VLAN 10 didática vira 101; a 99 entra para a gerência via trunk):

```
/interface bridge vlan set [find vlan-ids=10] vlan-ids=101
/interface bridge port set [find interface=ether3] pvid=101
/interface bridge vlan add bridge=bridge-sw vlan-ids=99 tagged=bridge-sw,ether2
```

Passo 2 — a borda fala o plano. Em **lab-r1**, renomeie o mundo antigo e crie os pés da borda em cada segmento:

```
/interface vlan set [find name=vlan10-svc] name=vlan101-cliA vlan-id=101
/ip address set [find interface=vlan101-cliA] address=100.64.101.1/24
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-lan
/ip address add address=10.99.0.1/24 interface=vlan99-ger
```

E a gerência do switch dentro da VLAN 99 (em **lab-r2**):

```
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-sw
/ip address add address=10.99.0.2/24 interface=vlan99-ger
```

Teste da espinha: de **lab-r1**, `/ping 10.99.0.2 count=3` — a gerência do POP vive na VLAN 99, pelo trunk.

Passo 3 — o cliente no plano. Em **lab-r3** (que continua ignorante de VLANs — porta access cuida):

```
/ip address set [find address="10.10.0"] address=100.64.101.30/24
/ip route add dst-address=0.0.0.0/0 gateway=100.64.101.1
/ping 100.64.101.1 count=3
```

Passo 4 — as listas e a política. Em **lab-r1**, as VLANs entram nas listas do firewall:

```
/interface list member add list=gerencia interface=vlan99-ger
/interface list member add list=LAN interface=vlan101-clia
```

E o teste dos três princípios, direto do lab-r3 (cliente):

```
/ping 10.99.0.2 count=3
/ping 10.99.0.1 count=3
```

Ambos mortos — e repare o *porquê* de cada um: o 10.99.0.2 (switch) morre porque a regra “gerência intocável” do forward (Capítulo 18) barra a travessia LAN→gerência; o 10.99.0.1 (a própria borda) morre no **input...** a menos que o ICMP-com-teto o aceite — verifique! Se o ping na borda responde, é o accept de ICMP do Capítulo 17 fazendo seu papel; o que importa é que **nenhum serviço** da VLAN 99 se alcança (teste `ssh vitor@10.99.0.1` do r3: morto no drop all).

Passo 5 — a prova do hardware (conceitual) e a bateria. No CHR não há chip, mas confira a *intenção*: em lab-r2, `/interface bridge vlan print` e `port print` devem contar exatamente a 8 — num CRS326, essa mesma configuração estaria no chip, flag H viva. Feche com a bateria: r3 navega (`/ping 8.8.8.8` via borda + NAT), gerência do switch acessível **do r1** e inacessível **do r3**, e o export do plano:

```
/interface bridge vlan export file=pop-vlans
```

Crie o snapshot **pop-segmentado** nas três VMs — a Fase 1 do livro termina aqui, e a Fase 2 (wireless, PPPoE) partirá deste estado.

26.4.3 Verificação

1. O plano vive igual nos três lugares: 8 tabela do switch interfaces da borda (mesmos VIDs, mesmas redes);
2. O cliente navega (VLAN 101 → borda → NAT) sem enxergar gerência (VLAN 99 inalcançável de dentro, nos dois testes do Passo 4);
3. A gerência do POP (borda switch) conversa pela VLAN 99 sobre o trunk;
4. Exports arquivados e snapshot **pop-segmentado** criado nas três VMs.

26.4.4 Desafios

1. Chega o cliente B. Liste **todos** os toques necessários (switch, borda, firewall, tabela) — e conte-os. O que esse número diz sobre o desenho?
2. No Passo 4, o ping do r3 à borda (10.99.0.1) pode responder pelo accept de ICMP do input. Isso é um vazamento? Argumente os dois lados e proponha o refinamento se sua resposta for “sim”.

3. A torre de rádio rural entra no POP pelo segundo SFP+ do switch, e atrás dela vêm 300 clientes PtMP. Adapte o plano: a torre é trunk ou access? Que VLANs descem até ela — e qual princípio do capítulo é o mais difícil de manter no rádio (dica: os 300 compartilham o meio)?
4. O desenho do capítulo tem um ponto único de falha evidente. Qual é, o que acontece com cada tipo de tráfego quando ele cai — e quais volumes deste livro atacam esse problema?

Solução dos desafios

1. (a) Switch: 1 linha na tabela de VLANs (102 tagged no trunk, untagged na porta) + PVID/frame-types da porta; (b) borda: `/interface vlan 102 + IP 100.64.102.1/24` + membro da lista LAN; (c) firewall: **zero** (as políticas usam listas); (d) documentação: 1 linha na 8. Meia dúzia de toques, nenhum em regra de firewall — o desenho escala porque política e dados estão separados (Capítulo 18) e as listas absorvem o crescimento.
2. Lados: *não é vazamento* — ICMP com teto é diagnóstico universal e não expõe serviço (o drop all segura SSH/WinBox); *é vazamento* — o cliente consegue **enumerar** a infraestrutura (descobre que 10.99.0.1 existe e responde), primeiro passo de reconhecimento. Refinamento comum de provedor: restringir o accept de ICMP do input por origem (`in-interface-list=!LAN` para os IPs de infra, ou aceitar ICMP da LAN apenas para os IPs que a LAN deve ver — o gateway dela). Não há resposta única; há decisão documentada.
3. A torre é **trunk**: por ela descem a VLAN de gerência (99 — os rádios são equipamentos do provedor) e a(s) VLAN(s) de clientes daquela torre (ex.: 150). O princípio difícil é o “**cliente não vê cliente**”: na VLAN 150, os 300 compartilham o mesmo domínio L2 pelo ar — a segmentação por VLAN individual não escala para PtMP. As respostas moram na camada wireless (isolamento de clientes no AP, *default-forward* off — Volume 5) e no PPPoE (cada cliente numa sessão ponto a ponto — Volume 6), que devolvem o isolamento sem 300 VLANs.
4. O **roteador de borda**: toda travessia e toda saída passam por ele. Caindo: tráfego **intra**-VLAN continua (comutação no switch independe), mas internet, travessias entre segmentos e serviços da borda (DNS, DHCP) morrem — o POP viram ilhas isoladas funcionais. Ataques ao problema: redundância de gateway (VRRP — Volume 14), múltiplos uplinks (ECMP/failover — Volume 14) e desenho de rede com mais de um caminho (OSPF — Volume 8).

26.5 Resumo

- Segmentação começa por **documento**: o plano de VLANs
 - (8) com numeração significativa e redes casadas — o contrato entre switch, borda e madrugada.
- Os três princípios: **gerência isolada** (VLAN que cliente não recebe

- firewall), **cliente não vê cliente** (VLAN por cliente; PtMP resolve na camada wireless/PPPoE), **travessia só pelo firewall** (borda é a única ponte).
- O desenho (Figura 26.1): trunk único borda switch, access nos segmentos, comutação em hardware dentro, roteamento com estado entre.
- As VLANs entram nas **interface lists** e o firewall-base do Volume 3 absorve os segmentos quase sem regras novas — escala é separar política de dados.
- O mesmo desenho serve do condomínio ao POP regional — muda o tamanho da tabela, não a arquitetura.

Fecha-se o Volume 4 — e a **Fase 1** inteira: você opera o RouterOS, constrói a borda, veste o firewall e segmenta o POP. A Fase 2 leva tudo isso para a rua: o Volume 5 sobe na torre — rádio, regulamentação e os enlaces que levam internet aonde a fibra não chega.

Bibliografia do capítulo

Este capítulo integra os anteriores; além das referências deles, os guias *Basic VLAN switching* e os exemplos de segmentação da documentação oficial (MikroTik, 2026a) mostram variações do mesmo desenho por família de hardware. As referências completas, em ABNT, estão no apêndice [Referências](#).

Parte V

**Volume 5 — Wireless e enlaces de
rádio**

27 Fundamentos de rádio e regulamentação

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Falar a língua do rádio: **dBm**, **dBi** e **dB** — e fazer as contas de cabeça que todo projetista de enlace faz;
- Comparar as bandas do provedor — **2,4 GHz**, **5 GHz** e **60 GHz** — em alcance, capacidade e interferência;
- Explicar visada e **zona de Fresnel**: por que “estar vendo a torre” não basta;
- Calcular um **link budget** completo: potência, ganhos, perda de percurso e margem;
- Situar a operação dentro das regras da **Anatel**: radiação restrita, limites de potência EIRP e as responsabilidades do provedor.

27.1 A língua dos decibéis

Rádio se projeta em **decibéis**, e há uma razão prática: as grandezas envolvidas variam por fatores de bilhões (miliwatts transmitidos, femtowatts recebidos), e o decibel transforma multiplicação em **soma**. Três unidades, três papéis:

- **dBm** — potência absoluta, referida a 1 mW: $P_{\text{dBm}} = 10 \log_{10}(P/1 \text{ mW})$. Um rádio que transmite 25 dBm emite ~316 mW; um sinal recebido de -65 dBm são ~0,3 nW;
- **dBi** — ganho de antena, relativo à antena isotrópica (que irradia igual em todas as direções). A grade de um LHG tem ~24,5 dBi: concentra a energia num feixe estreito;
- **dB** — razões e perdas: cabo, conectores, atenuação no caminho.

E as duas contas de cabeça que valem o capítulo:

1. **+3 dB dobra, -3 dB corta pela metade** (potência);
2. **+10 dB é 10×** — logo 20 dBm = 100 mW, 30 dBm = 1 W.

O que torna tudo somável: **potência recebida = potência transmitida + ganhos - perdas**. Essa soma tem nome — *link budget* — e fecharemos o capítulo com ela.

💡 Analogia para guardar

Pense no enlace como uma conversa num campo aberto: dBm é o volume da voz, dBi é fazer concha com as mãos (a mesma voz, concentrada numa direção), e a perda de percurso é a distância engolindo o som. O link budget responde: “gritando a X dBm, com as mãos em concha de Y dBi, a voz chega ao ouvido do outro lado acima do burburinho?”

27.2 As bandas do provedor

Tabela 27.1: As três bandas na prática do provedor

	2,4 GHz	5 GHz	60 GHz
Alcance típico	maior	grande	curto (< 2–3 km)
Largura de canais	3 canais úteis (20 MHz)	dezenas de canais (20–80 MHz)	canais gigantes (2 GHz)
Capacidade	baixa	média/alta	multi-gigabit
Interferência	saturada	moderada (e crescente)	quase nula
Chuva/obstáculos	atravessa vegetação leve	sensível a obstáculos	bloqueada por quase tudo (e chuva forte atenua)
Papel no provedor	legado, IoT, última opção	PtP e PtMP rural — o cavalo de batalha	PtP curto de alta capacidade (Wire, 2)

A física por trás da tabela: quanto **maior a frequência**, mais o sinal se comporta como luz — feixes mais estreitos e mais capacidade (canais largos), porém menos capacidade de contornar obstáculos e mais atenuação. Os 2,4 GHz contornam árvores e chegam longe — e justamente por isso todo mundo os usa, e a banda virou um mercado de peixe de interferência. Os 5 GHz equilibram alcance e espectro limpo o suficiente — são a banda do PtMP rural brasileiro. Os 60 GHz são um laser de dados: multi-gigabit entre dois prédios, inúteis atrás de uma folha de bananeira.

27.3 Visada e a zona de Fresnel

“Da torre eu vejo a casa do cliente” — e o enlace não fecha. O erro está em pensar o rádio como um raio de luz fino: a energia viaja num **volume** elíptico ao redor da linha reta, a **zona de Fresnel** (Figura 27.1). Obstáculos que invadem esse volume — mesmo sem tocar a linha de visada — roubam sinal.

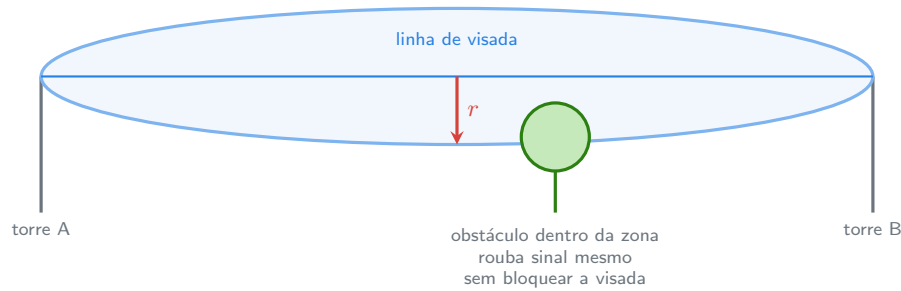


Figura 27.1: A zona de Fresnel: o sinal viaja num volume elíptico — e é esse volume, não a linha fina, que precisa estar livre.

O raio máximo da primeira zona, no meio de um enlace de comprimento d (km) na frequência f (GHz):

$$r \approx 8,66 \sqrt{\frac{d}{f}} \text{ metros}$$

Num enlace de 10 km em 5,8 GHz: $r \approx 8,66 \sqrt{10/5,8} \approx 11,4$ m. Onze metros de folga **no meio do caminho** — é por isso que torres têm a altura que têm, e por que o enlace que fechou em agosto cai em dezembro, quando o eucalipto do vizinho cresceu. Regra de projeto: **pelo menos 60% da primeira zona livre** (no exemplo, ~ 7 m).

27.4 O link budget: a conta que fecha enlaces

A perda de percurso no espaço livre (FSPL) também tem fórmula amigável em dB, com d em km e f em GHz:

$$\text{FSPL} \approx 92,45 + 20 \log_{10}(d) + 20 \log_{10}(f) \text{ dB}$$

E o link budget é a soma completa, de ponta a ponta:

$$P_{rx} = P_{tx} + G_{tx} - \text{FSPL} + G_{rx}$$

Exemplo real — enlace PtP de 10 km em 5,8 GHz com dois LHG 5 (24,5 dBi; transmissor a 20 dBm):

- FSPL: $92,45 + 20 \log_{10}(10) + 20 \log_{10}(5,8) \approx 92,45 + 20 + 15,3 = 127,8$ dB;
- $P_{rx} = 20 + 24,5 - 127,8 + 24,5 = -58,8$ dBm.

Esse número se compara com a **sensibilidade** do rádio na modulação desejada (a ficha técnica diz: p.ex., -80 dBm para MCS alto) e nasce a **margem** — aqui, ~ 21 dB. A regra de campo: **margem mínima de 15–20 dB** para o enlace sobreviver a chuva, desvanecimento e ao eucalipto que cresce. Menos que 10 dB de margem no papel = enlace que dará chamado.

27.5 Anatel: as regras do jogo

As bandas da Tabela 27.1 são de **radiação restrita** (Resolução Anatel nº 680/2017, herdeira da 506): não exigem licença **por estação** e podem ser usadas em caráter secundário — sem direito a proteção contra interferência e sem poder causar interferência em serviços primários. As letras que importam ao provedor:

- **Limites de EIRP** — a potência efetivamente irradiada ($P_{tx} + G_{ant}$) tem tetos por banda e aplicação; em 5,8 GHz ponto-a-ponto há mais folga (o feixe estreito não incomoda os vizinhos), em 2,4 GHz os limites são apertados. O detalhe que muda projeto: antena de mais ganho pode exigir **reduzir** a potência do rádio para respeitar o teto;
- **DFS/radar em parte dos 5 GHz** — faixas compartilhadas com radares meteorológicos exigem detecção e desvio automáticos (o RouterOS implementa); ignorar não é opção;
- **SCM** — o provedor em si opera sob outorga de Serviço de Comunicação Multimídia (dispensada abaixo de 5 mil assinantes, mas o registro tem vantagens) — a camada jurídica acima do rádio;
- **Homologação** — os equipamentos precisam ser homologados pela Anatel (os MikroTik vendidos no Brasil são; o selo importa em fiscalização).

Aviso

Operar acima dos limites de EIRP — o “destravar a potência” folclórico — é infração sancionável pela Anatel, além de má engenharia: potência a mais no seu rádio é interferência nos vizinhos e no seu próprio setor (você grita mais, e ouve pior os clientes distantes). Enlace bom se faz com **antena, altura e alinhamento**, não com watt.

O conceito mais importante do capítulo

Enlace de rádio não se “tenta” — se **calcula**: $P_{rx} = P_{tx} + G_{tx} - \text{FSPL} + G_{rx}$, comparado à sensibilidade, com 15–20 dB de margem e 60% da primeira zona de Fresnel livre. A conta cabe num guardanapo, respeita os tetos da Anatel e prevê no papel o que o campo confirmaria a 40 km de distância — a ordem certa é essa.

27.6 Laboratório

27.6.1 Ambiente

Papel, calculadora — e mapa. Este é o laboratório de **projeto**: o CHR não tem rádio, e as contas deste capítulo são exatamente o que se faz antes de subir em qualquer torre. (Ferramentas de mapa com perfil de elevação — como o modo *caminho* do Google Earth — ajudam nos desafios.)

27.6.2 Comandos passo a passo

Sem comandos hoje — passos de cálculo. Refaça cada um **antes** de olhar o gabarito ao lado:

Passo 1 — traduzir potências. Complete de cabeça: $23 \text{ dBm} = ? \text{ mW}$ (gabarito: $\sim 200 \text{ mW}$ — 20 dBm são 100 mW , $+3 \text{ dB}$ dobra); $316 \text{ mW} = ? \text{ dBm}$ (25 dBm); um cabo com 2 dB de perda deixa passar que fração? ($\sim 63\%$).

Passo 2 — Fresnel. Enlace de 6 km em $5,8 \text{ GHz}$: raio no meio do caminho? ($r = 8,66\sqrt{6/5,8} \approx 8,8 \text{ m}$; 60% $5,3 \text{ m}$ de folga necessária.)

Passo 3 — FSPL. O mesmo enlace: perda de percurso? ($92,45 + 20 \log_{10} 6 + 20 \log_{10} 5,8 \approx 92,45 + 15,6 + 15,3 = 123,3 \text{ dB}$.)

Passo 4 — o budget completo. Dois SXT SA5 *não* — um enlace PtP com duas LHG 5 ($24,5 \text{ dBi}$), rádios a 20 dBm , 6 km , $5,8 \text{ GHz}$: $P_{rx} = 20 + 24,5 - 123,3 + 24,5 = -54,3 \text{ dBm}$. Com sensibilidade de -80 dBm na modulação alvo: **margem $25,7 \text{ dB}$** — enlace confortável.

Passo 5 — o mesmo enlace com equipamento errado. Refaça com duas antenas de 16 dBi (SXT): $P_{rx} = 20 + 16 - 123,3 + 16 = -71,3 \text{ dBm}$ — margem de $\sim 8,7 \text{ dB}$. No papel “funciona”; na primeira chuva com desvanecimento, cai. É a diferença, em números, entre o SXT e o LHG do desafio do 2.

27.6.3 Verificação

1. Suas contas dos Passos 1–4 batem com os gabaritos (erros de $\pm 1 \text{ dB}$ são arredondamento; erros de $\pm 10 \text{ dB}$ são conceito — refaça);
2. Você sabe justificar a escolha LHG $\times$ SXT do Passo 5 **com a margem**, não com achismo;
3. Você sabe dizer, para seu projeto, o teto de EIRP aplicável — e o que fazer quando antena + potência o excedem.

27.6.4 Desafios

1. Um enlace de 15 km em 5,8 GHz precisa fechar com margem 20 dB usando rádios de 25 dBm e sensibilidade -80 dBm. Que ganho mínimo (idêntico nas duas pontas) as antenas precisam ter? Consulte a Tabela 27.1 e o catálogo mental do 2: que produto você especificaria?
2. O mesmo enlace do desafio 1 passa sobre um vale, e no meio do caminho há uma mata cujas copas chegam a 18 m. As torres têm 30 m e o terreno é plano. A primeira zona de Fresnel está suficientemente livre?
3. Um concorrente instalou um setor de 2,4 GHz “no talo” (potência máxima + antena de alto ganho, EIRP acima do permitido) apontado para a sua área. Liste: (a) o efeito técnico na rede **dele**; (b) o efeito na sua; (c) as suas opções técnicas e regulatórias.
4. Por que um enlace de 60 GHz de 1,5 km pode ser melhor e pior que um de 5,8 GHz no mesmo caminho? Dê duas vantagens e dois riscos, e o critério de decisão.

Solução dos desafios

1. FSPL(15 km; 5,8 GHz) = 92,45 + 23,5 + 15,3 = 131,3 dB. Precisamos de $P_{rx} \geq -60$ dBm (-80 + 20 de margem). Então $G_{tx} + G_{rx} \geq -60 - 25 + 131,3 = 46,3$ dB → **23,2 dBi por antena**. Especificação natural: **LHG 5 XL** (~27 dBi) — folga para chuva e envelhecimento; um LHG 5 comum (24,5 dBi) fecharia justo. E atenção ao EIRP: 25 dBm + 27 dBi = 52 dBm — confira o teto da faixa PtP antes de cravar a potência.

2. $r = 8,66\sqrt{15/5,8} \approx 13,9$ m; 60% 8,4 m. A linha de visada corre a 30 m; o limite inferior aceitável da zona fica a 30 - 8,4 = 21,6 m. As copas a 18 m **passam** — mas com 3,6 m de folga apenas: mata cresce (1-2 m/ano em eucalipto!) e o meio do vão é onde a elipse é máxima. Veredicto: fecha hoje, com plano de revisita — ou torres mais altas já.

3. (a) Na rede dele: autointerferência — gritando, ele não ouve os próprios clientes distantes (enlace é via de mão dupla; EIRP alto só na torre não melhora o uplink do cliente); (b) na sua: piso de ruído mais alto em 2,4 GHz, CCQ e MCS caindo nos setores apontados; (c) técnicas: migrar o que puder para 5 GHz (Tabela 27.1), canais mais afastados, antenas mais diretivas, polarização; regulatória: radiação restrita não dá direito a proteção, **mas** operação fora dos limites é infração — medição documentada e denúncia à Anatel é caminho legítimo (e comum) entre provedores.

4. Vantagens do 60 GHz: capacidade multi-gigabit (canais de 2 GHz) e espectro limpo (sem vizinhos, feixe a laser). Riscos: atenuação por chuva forte (enlaces > 1 km podem cair em tempestade) e sensibilidade total a obstáculos (um galho novo = enlace morto). Critério: 60 GHz quando a distância é curta, a visada é estéril (urbana, entre prédios) e a demanda de banda justifica — idealmente com backup em 5 GHz (os Wire AP fazem isso sozinhos); 5,8 GHz quando o caminho é rural, verde e longo.

27.7 Resumo

- Decibéis: **dBm** (potência), **dBi** (ganho), **dB** (perdas); +3 dB dobra, +10 dB é 10× — e tudo vira soma.
- Bandas (Tabela 27.1): 2,4 GHz alcança e está saturada; **5 GHz é o cavalo de batalha** do provedor rural; 60 GHz é um laser multi-gigabit de curta distância.
- Visada não basta: **zona de Fresnel** ($r \approx 8,66\sqrt{d/f}$ m) com **60% livre** — e mata cresce.
- **Link budget**: $P_{rx} = P_{tx} + G_{tx} - \text{FSPL} + G_{rx}$, $\text{FSPL} \approx 92,45 + 20 \log d + 20 \log f$; margem alvo **15–20 dB**.
- Anatel: radiação restrita = sem proteção e com **tetos de EIRP**; DFS em parte dos 5 GHz; equipamentos homologados; SCM como camada de outorga. Enlace se melhora com antena e altura, não com watt ilegal.

No próximo capítulo, o rádio entra no RouterOS: modos de interface, registration table, segurança — o wireless como o sistema o vê.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) traz as fichas com sensibilidade por modulação de cada rádio (essenciais ao link budget). A Resolução Anatel nº 680/2017 e os atos de requisitos técnicos correlatos definem os limites de radiação restrita no Brasil — leia-os na fonte (anatel.gov.br). Tanenbaum; Feamster; Wetherall (2021) cobre a física de transmissão sem fio. As referências completas, em ABNT, estão no apêndice [Referências](#).

28 Wireless no RouterOS

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Distinguir as duas pilhas wireless do RouterOS v7 — o pacote clássico (`/interface wireless`) e o novo (`/interface wifi`) — e saber qual vive em cada equipamento;
- Escolher o **modo** certo de interface (`ap-bridge`, `bridge`, `station`, `station-bridge`) para cada papel: torre, cliente, PtP;
- Configurar um enlace básico: banda, frequência, largura de canal, SSID e **security profile** (WPA2);
- Ler a **registration table** — a tabela que conta a saúde de cada conexão de rádio;
- Entender por que `station` puro quebra bridging — e como `station-bridge` resolve.

28.1 Duas pilhas, um aviso

Antes de qualquer comando, o mapa do território no RouterOS v7:

- **`/interface wireless`** — o pacote clássico, dos rádios de provedor: SXT, LHG, BaseBox, NetMetal e afins. É onde vivem os protocolos proprietários de PtP/PtMP (Nstreme, NV2 — 9) e é a pilha deste e dos próximos três capítulos;
- **`/interface wifi`** — a pilha nova (*wave2/ax*), dos equipamentos WiFi modernos (hAP ax, cAP ax): WPA3, roaming rápido, CAPsMAN novo. É a pilha da casa do cliente — assunto do Capítulo 32.

A separação faz sentido operacional: **enlace de provedor** (torre, backhaul) é um problema de física e disciplina de meio; **WiFi de cliente** é um problema de compatibilidade e gerência em massa. A MikroTik os resolve com pilhas diferentes, e este volume os trata em capítulos diferentes.

28.2 Os modos: quem fala, quem escuta, quem repassa

O parâmetro `mode` define o papel do rádio — e escolhe errado quem não entende **camada 2 sobre o ar** (Figura 28.1):

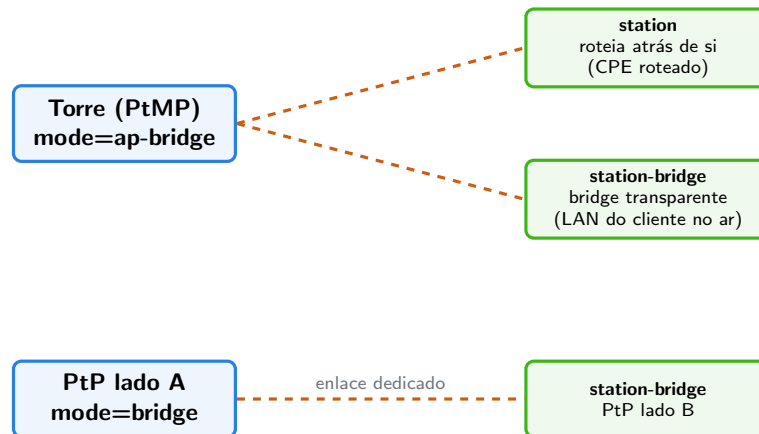


Figura 28.1: Os modos por papel: `ap-bridge` na torre PtMP, `bridge` no PtP, `station/station-bridge` nos clientes.

- **ap-bridge** — o ponto de acesso completo: aceita **vários** clientes. É o modo da setorial na torre (exige licença nível 4+, Capítulo 3);
- **bridge** — um AP que aceita **um** cliente só: o lado “mestre” de um enlace PtP dedicado (e disponível já no nível 3);
- **station** — o cliente ortodoxo do 802.11... com uma pegadinha enorme: o padrão só prevê **um MAC** atrás de cada estação. Um rádio em **station** não consegue fazer bridge da LAN do cliente — os quadros dos dispositivos atrás dele têm MACs que o AP recusaria. **station** serve quando o rádio **roteia** (a LAN do cliente atrás de NAT/rotas do próprio rádio);
- **station-bridge** — a solução proprietária MikroTik: AP e estação combinam um encapsulamento que transporta múltiplos MACs, e a LAN do cliente atravessa o ar como se fosse cabo. Exige MikroTik nas duas pontas — na rede do provedor, é o normal. (Evite o **station-pseudobridge**, uma gambiarra de tradução de MACs para APs de terceiros: funciona “quase sempre”, e “quase” gera chamados.)

28.3 O enlace mínimo: banda, canal, SSID, segurança

Um AP de torre nasce com quatro decisões (os valores são exemplo — o projeto do canal é assunto do 11):

```
/interface wireless security-profiles add name=sp-torre mode=dynamic-keys authentication-t
/interface wireless set wlan1 mode=ap-bridge band=5ghz-a/n/ac channel-width=20/40mhz-Ce fr
```

Linha a linha do **set**:

- **band** — a banda e as gerações aceitas (**5ghz-a/n/ac**: clientes 802.11a, n e ac). Restringir gerações antigas melhora a eficiência do setor — cliente lento ocupa o ar de todos;
- **channel-width** — 20 MHz é o conservador de PtMP (mais setores coexistindo, melhor sensibilidade); 40+ MHz para PtP com espectro limpo. Largura dobrada = capacidade quase dobrada e o dobro de exposição a interferência;
- **frequency** — o canal, em MHz. No **ap-bridge** você **escolhe** (e documenta!); nas estações, usa-se **scan-list** para procurar o AP;
- **ssid** — o nome da rede. Convenção de provedor: nome que identifica torre/setor (**prov-torre-sul-s2**), não “WiFi da firma”;
- **security-profile** — a segurança mora num perfil separado (reutilizável entre interfaces): **WPA2-PSK no mínimo, sempre**. Rádio aberto em torre é convite a vizinho pendurado e a captura de tráfego. Nos rádios de provedor, some-se o **radio-name** e, adiante, a **access-list** (11).

Do lado do cliente, o espelho:

```
/interface wireless set wlan1 mode=station-bridge band=5ghz-a/n/ac ssid=prov-torre-sul sec
```

28.4 A registration table: o estetoscópio

Conectou? A resposta — e a **qualidade** da resposta — mora em:

```
/interface wireless registration-table print stats
```

```
[admin@torre-sul] > /interface wireless registration-table print stats
0 interface=wlan1 mac-address=08:00:27:AA:12:34 ap=no wds=no
  rx-rate="390Mbps-40MHz/2S" tx-rate="390Mbps-40MHz/2S"
  uptime=2d4h11m22s signal-strength=-61dBm@40MHz
  tx-signal-strength=-63dBm tx-ccq=94% p-throughput=182000
```

Os campos que o Capítulo 31 transformará em método; por ora, o vocabulário:

- **signal-strength / tx-signal-strength** — o sinal **nos dois sentidos** (o que eu ouço dele / o que ele ouve de mim). O link budget do Capítulo 27, medido: os -61 dBm daqui deveriam bater com a conta do papel. Assimetria grande entre os dois = problema (potências desiguais, antena desalinhada num lado);
- **tx/rx-rate** — a modulação negociada (MCS): o rádio ajusta a velocidade à qualidade do sinal. Taxa baixa com sinal bom = procurar interferência;
- **tx-ccq** — *Client Connection Quality*: percentual de eficiência real das transmissões (quantas precisaram de retransmissão). **> 90% saudável; < 70% doente**;
- **uptime** — conexão estável ou reconectando toda hora? O contador denuncia.

💡 Analogia para guardar

A registration table é a **ficha médica** de cada cliente do setor: sinal é a pressão arterial (nos dois braços!), CCQ é a saturação de oxigênio, a taxa negociada é o pique na esteira, e o uptime é o histórico de desmaios. Antes de culpar “a internet”, o médico de torre abre a ficha.

! O conceito mais importante do capítulo

Wireless de provedor no RouterOS é: **modo pelo papel** (ap-bridge na torre, bridge no PtP, station-bridge no cliente que faz ponte — nunca station puro para LAN), **quatro decisões** no enlace (banda, largura, canal, WPA2 em security profile) e a **registration table** como fonte da verdade sobre a saúde de cada conexão — sinal nos dois sentidos, CCQ e taxa negociada.

28.5 Laboratório

28.5.1 Ambiente

O CHR não tem rádio — este laboratório usa **hardware físico opcional**: qualquer par de RouterBOARDS com wireless (dois hAP usados, ou um hAP + um SXT) reproduz tudo. Sem hardware, faça o **laboratório de leitura**: os exports e saídas abaixo estão completos — percorra-os linha a linha como se fossem seus, e responda os desafios (todos independentes de equipamento).

28.5.2 Comandos passo a passo

Passo 1 — o AP. No equipamento A (papel de torre/AP):

```
/interface wireless security-profiles add name=sp-lab mode=dynamic-keys authentication-type=wpa-psk  
/interface wireless set wlan1 mode=ap-bridge band=5ghz-a/n/ac channel-width=20mhz frequency=5200
```

Passo 2 — a estação. No equipamento B (papel de cliente):

```
/interface wireless security-profiles add name=sp-lab mode=dynamic-keys authentication-type=wpa-psk  
/interface wireless set wlan1 mode=station-bridge band=5ghz-a/n/ac ssid=lab-ptmp security-profile=sp-lab
```

Passo 3 — ver a conexão nascer. No AP:

```
/interface wireless registration-table print stats
```

Identifique cada campo da seção anterior na sua saída real. Anote sinal, CCQ e taxa — este é o “exame de sangue” de referência do seu enlace de bancada.

Passo 4 — a ponte no ar. Prove o `station-bridge`: no B, coloque a `wlan1` e a `ether2` numa bridge (13) e pendure um notebook na `ether2` — ele deve receber DHCP de quem estiver atrás do AP, atravessando o ar em camada 2. Depois, troque o modo para `station` puro e repita: o notebook fica mudo (os MACs dele não atravessam) — a pegadinha do modo, sentida na prática.

Passo 5 — brincar de qualidade. Com o enlace vivo, degrade-o de propósito: afaste os equipamentos, interponha uma parede, desalinhe. A cada mudança, `registration-table print stats`: veja sinal cair, a taxa negociada despencar junto e o CCQ sofrer com retransmissões. Você está assistindo à física do Capítulo 27 em tempo real.

28.5.3 Verificação

1. A estação aparece na registration table do AP com WPA2 (e não conecta com a chave errada — teste!);
2. Em `station-bridge`, dispositivos atrás da estação atravessam em camada 2; em `station`, não — e você sabe explicar por quê;
3. Você localizou sinal (nos **dois** sentidos), CCQ, taxa e uptime na sua saída (ou na do texto) sem consultar a legenda;
4. (Bancada) O Passo 5 mostrou a tríade $\text{sinal} \downarrow \rightarrow \text{taxa} \downarrow \rightarrow \text{CCQ} \downarrow$ se movendo junta.

28.5.4 Desafios

1. Um provedor liga a setorial da torre em `mode=bridge` “porque bridge é transparente”. Vinte clientes tentam conectar. O que acontece — e qual é o modo certo?
2. Na registration table de um cliente rural: `signal-strength=-58dBm`, `tx-signal-strength=-79dBm`. O download no cliente está ótimo; o upload, péssimo. Explique a coerência entre os três fatos e proponha duas causas prováveis.
3. Por que `station-pseudobridge` “quase funciona”? Descreva o mecanismo de tradução de MACs e invente um cenário concreto (um serviço comum de LAN) em que ele quebra.
4. O `security-profile` do laboratório usa WPA2-PSK — uma chave única para o setor inteiro. Aponte dois problemas operacionais disso num PtMP de 80 clientes e antecipe (sem detalhar) as alternativas que os próximos capítulos e o Volume 6 trarão.

Solução dos desafios

1. `mode=bridge` aceita **uma** estação: o primeiro cliente conecta, os outros dezenove ficam de fora (tentando, enchendo o log). O modo da torre PtMP é **ap-bridge** — múltiplas estações. (`bridge` existe para o PtP dedicado, onde “só um” é justamente o contrato.)
2. Coerência: `signal-strength` é o que **a torre ouve do cliente...** cuidado — na

tabela *do cliente*, é o que o cliente ouve da torre (−58 dBm, forte → download bom); `tx-signal-strength` é o eco reportado do outro lado: a torre ouve o cliente a −79 dBm (fraco → upload ruim). Assimetria de 21 dB. Causas prováveis: (a) potência de transmissão do cliente muito menor que a da torre (comum: torre “no talo”, cliente comedido — e o enlace é mão dupla); (b) desalinhamento/polarização — a antena do cliente aponta bem para receber o feixe largo do setor, mas o feixe estreito dela não volta certo (ou um conector/pigtail ruim no transmissor do cliente).

3. O pseudobridge faz **NAT de MAC**: apresenta ao AP o MAC do próprio rádio para todos os quadros e mantém uma tabela interna IP MAC para devolver as respostas. Quebra onde a camada 2 importa de verdade: tráfego **não-IP** ou baseado em broadcast/MAC — exemplo concreto: um DHCP **server** atrás da estação (os OFFERs unicast ao MAC do cliente real nunca chegam), descoberta de vizinhos, ARP gratuito de failover (VRRP). Sintoma clássico: “uns aparelhos funcionam, outros não”.

4. Problemas: (a) **revogação impossível** — cancelou um cliente? A chave que ele conhece continua abrindo o setor; trocar a chave = reconfigurar 80 rádios; (b) **zero identidade** — todos são “a chave”: não há como medir, limitar ou cortar **um** assinante na camada wireless. Alternativas vindouras: **access-list** por MAC com chaves individuais (11) e — a resposta definitiva do ISP — autenticação **por sessão PPPoE** em cima do rádio (Volume 6), onde identidade, plano e corte são por assinante.

28.6 Resumo

- v7 tem **duas pilhas**: `/interface wireless` (rádios de provedor — este capítulo) e `/interface wifi` (wave2/ax, casa do cliente — Capítulo 32).
- Modos pelo papel (Figura 28.1): **ap-bridge** torre PtMP, **bridge** PtP, **station-bridge** cliente com ponte (proprietário MikroTik), **station** só quando o rádio roteia — e pseudobridge é gambiarra.
- Enlace mínimo = banda + largura de canal + frequência + SSID + **WPA2 em security profile** — largura dobrada, capacidade e vulnerabilidade dobradas.
- **Registration table** é a ficha médica: sinal **nos dois sentidos**, CCQ (> 90% saudável), taxa negociada (MCS) e uptime — a tríade sinal→taxa→CCQ se move junta.

No próximo capítulo, o ar deixa de ser padrão: **Nstreme e NV2** — os protocolos proprietários que fazem enlaces PtP de provedor renderem o que o 802.11 puro não entrega.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/interface wireless` (modos, security profiles, registration table) e a pilha nova em *WiFi*. Discher (2016) traz capítulos extensos de wireless para WISPs. As referências completas, em ABNT, estão no apêndice [Referências](#).

29 Enlaces PtP: Nstreme e NV2

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar por que o CSMA/CA do 802.11 padrão desperdiça capacidade em enlaces ponto a ponto longos (o problema do *ACK timeout*);
- Descrever o que Nstreme e NV2 mudam no acesso ao meio — *polling* e TDMA — e por que isso importa a partir de alguns quilômetros;
- Escolher entre 802.11, Nstreme e NV2 para um enlace PtP concreto;
- Configurar um enlace PtP completo com NV2 nos dois lados, incluindo os parâmetros de TDMA (`tdma-period-size`, `nv2-cell-radius`);
- Dimensionar a capacidade real esperada de um enlace (taxa aérea `throughput` útil) e validá-la com `bandwidth-test`.

No 10 montamos um enlace sem fio e aprendemos a lê-lo pela *registration table*. Ele funcionava — mas usava o protocolo 802.11 de fábrica, o mesmo do WiFi doméstico. Para dois rádios a 30 cm um do outro, tudo bem. Para o backhaul que liga sua torre rural à sede, a 15 km, o 802.11 padrão joga fora boa parte da capacidade que você pagou. Este capítulo explica por quê — e apresenta as duas respostas da MikroTik: **Nstreme** e **NV2**.

29.1 O problema: 802.11 foi feito para dentro de casa

O 802.11 usa **CSMA/CA** (*Carrier Sense Multiple Access with Collision Avoidance*): antes de transmitir, o rádio escuta o canal; se está ocupado, espera um tempo aleatório e tenta de novo. Cada quadro entregue exige um **ACK** de volta — se o ACK não chega dentro de um prazo curto, o transmissor assume perda e retransmite.

Esse desenho embute duas suposições de ambiente doméstico:

1. **Todo mundo se escuta.** O *carrier sense* só evita colisões se as estações detectam as transmissões umas das outras. Em enlaces longos, com antenas diretivas apontadas para lados diferentes, isso raramente é verdade.
2. **A distância é desprezível.** O prazo do ACK foi calibrado para metros. A luz percorre ~300 m por microssegundo; em 15 km, só a viagem de ida e volta do quadro + ACK consome ~100 µs. O rádio precisa **esperar parado** esse tempo a cada quadro — e se o prazo não for esticado (o famoso ajuste de *distance*), ele retransmite quadros que teriam chegado.

O resultado prático: quanto mais longo o enlace, maior a fração do tempo de ar gasta em espera e retransmissão, e menor o throughput útil. Um enlace 802.11 de 15 km pode entregar **menos da metade** do que a taxa aérea sugere — e degrada ainda mais com qualquer interferência.

💡 Analogia para guardar

O CSMA/CA é uma reunião educada: cada um espera o silêncio para falar, e quem fala espera o “entendi” do outro lado antes de continuar. Numa sala pequena, funciona. Agora ponha os participantes a 15 km de distância gritando por megafone: o “entendi” demora tanto a voltar que todo mundo passa a reunião esperando — ou repetindo o que já tinha dito. Nstreme e NV2 trocam a reunião educada por um **moderador com cronômetro**: cada um recebe sua janela de fala e ninguém espera confirmação a cada frase.

29.2 Nstreme: o polling da primeira geração

O **Nstreme** é o protocolo proprietário original da MikroTik (era do RouterOS v3/v4). Ele substitui a disputa do CSMA/CA por **polling**: o AP pergunta a cada estação, uma por vez, se tem algo a transmitir. Além disso:

- **Elimina o ACK por quadro** — os quadros são agregados em blocos (*framer*) e confirmados em lote;
- **Sem limite de distância por protocolo** — como não há prazo de ACK por quadro, o enlace não “expira” com a distância;
- Só funciona **entre equipamentos MikroTik** (os dois lados precisam falar Nstreme).

Parâmetros que você verá em configurações legadas:

Tabela 29.1: Parâmetros do Nstreme

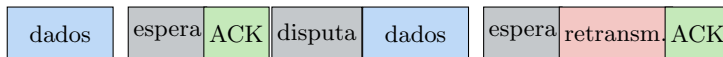
Parâmetro	O que faz	Valor típico PtP
wireless-protocol=nstreme	ativa o protocolo	nos dois lados
framer-policy	como agregar quadros	best-fit
framer-limit	tamanho máx. do bloco agregado (bytes)	3200
disable-csma	desliga o <i>carrier sense</i> (PtP puro)	yes

O Nstreme resolve o ACK e a distância, mas o polling tem um custo: o AP gasta tempo de ar *perguntando*, mesmo a estações sem tráfego. Num PtP (uma estação só) isso é pequeno; num PtMP com dezenas de clientes, o overhead cresce. Foi para isso que veio o sucessor.

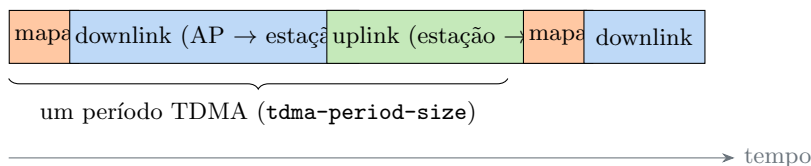
29.3 NV2: TDMA de verdade

O **NV2** (*Nstreme version 2*) abandona o polling e implementa **TDMA** (*Time Division Multiple Access*): o tempo de ar é dividido em **períodos fixos**, e o AP publica, a cada período, um **mapa de alocação** dizendo quem transmite em qual fatia — downlink e uplink. Ninguém escuta o canal, ninguém disputa, ninguém espera ACK individual: cada rádio já sabe **quando** é sua vez.

802.11 (CSMA/CA) — disputa + ACK por quadro



NV2 (TDMA) — períodos fixos, sem disputa



No 802.11, cada quadro paga disputa, espera de ACK e eventuais retransmissões — e o custo cresce com a distância. No NV2, o AP publica um mapa e o tempo de ar vira fatias contíguas de downlink e uplink, sem disputa nem ACK individual.

Figura 29.1

As consequências práticas do TDMA:

- **Latência previsível:** cada estação transmite a intervalos regulares. O *jitter* despenca — crucial para VoIP e jogos;
- **Throughput estável sob carga:** sem disputa, adicionar tráfego não multiplica colisões;
- **Escala em PtMP:** o mapa de alocação distribui o tempo entre dezenas de estações sem overhead de polling (por isso o NV2 é a base do 11);
- **Proprietário:** como o Nstreme, exige MikroTik dos dois lados.

29.3.1 Os dois parâmetros que você precisa entender

tdma-period-size (1–10 ms, padrão dinâmico): o tamanho do período. Período **maior** = mais tempo útil por mapa publicado = **mais throughput**, porém **mais latência** (cada estação espera mais pela sua fatia). Período **menor** = o inverso. Para PtP de backhaul, 2–4 ms costuma equilibrar; deixe **dynamic** na dúvida.

nv2-cell-radius (km, padrão 30): informa ao AP a distância da estação **mais distante**. O AP usa isso para dimensionar os tempos de guarda entre fatias (o sinal precisa ter tempo de viajar!). Valor menor que o real → fatias se atropelam e o enlace perde quadros; valor

muito maior que o real → tempo de guarda desperdiçado. Meça no mapa e arredonde para cima.

! O conceito mais importante do capítulo

Protocolo de acesso ao meio **não muda a física** — sinal, SNR e taxa aérea continuam os do Capítulo 27. O que Nstreme e NV2 mudam é **quanto do tempo de ar vira dado útil**. Num enlace curto e limpo, a diferença para o 802.11 é pequena; num enlace longo, ou num PtMP carregado, é a diferença entre entregar 40% e entregar 80% da capacidade aérea. Primeiro garanta o enlace saudável (Volume 5 até aqui); depois escolha o protocolo que não desperdiça o que o enlace oferece.

29.4 Qual protocolo usar?

Tabela 29.2: Escolha de protocolo por cenário

Cenário	Protocolo	Por quê
PtP curto (< 3 km), limpo	802.11 ou NV2	diferença pequena; 802.11 mantém interoperabilidade
PtP longo (> 3 km)	NV2	TDMA elimina o custo de ACK/distância
PtP com hardware antigo (um lado sem suporte a NV2)	Nstreme	ainda melhor que 802.11 puro
PtMP (torre com vários clientes)	NV2	mapa TDMA escala; polling e CSMA não
Enlace com equipamento de outro fabricante	802.11	Nstreme/NV2 são proprietários

Regra de bolso do provedor: **NV2 em tudo que for MikroTik MikroTik e esteja fora da sua sala**. Use `wireless-protocol=any` na estação durante a migração (ela segue o AP), e trave em `nv2` quando o setor inteiro tiver migrado.

29.4.1 Taxa aérea não é throughput

A registration table do capítulo anterior mostrava, por exemplo, `rate 866Mbps-80MHz/2S`. Essa é a **taxa aérea** (PHY): o ritmo com que os bits voam *enquanto* o rádio transmite. O throughput útil desconta preâmbulos, mapas TDMA, tempos de guarda, retransmissões e o fato de o canal ser **half-duplex** (download e upload dividem o mesmo ar).

Estimativa honesta para dimensionar backhaul:

$\text{throughput útil} \approx \text{taxa aérea} \times 0,55$ (NV2, enlace saudável, soma down+up)

Um enlace que negocia 866 Mbps entrega na prática ~450–480 Mbps agregados. Se o seu plano de capacidade precisa de 400 Mbps de download, esse enlace **não sobra** — dimensione o rádio pela conta, não pelo número da caixa.

29.5 Laboratório

i Ambiente

O CHR não tem rádio, então este é um **laboratório de configuração e leitura**: os comandos abaixo são exatamente os que você aplicaria em um par de rádios reais (ex.: dois LHG 5) formando o backhaul sede torre do nosso provedor fictício. Se você tem dois equipamentos MikroTik com wireless na bancada, tudo funciona neles; senão, valide a sintaxe num CHR (os menus `/interface wireless` existem, sem interfaces) e concentre-se em **prever** cada saída antes de lê-la. Topologia de referência do livro: Capítulo 5.

29.5.1 Comandos passo a passo

Passo 1 — lado A (sede, AP do enlace). PtP com NV2, banda de 5 GHz, canal de 40 MHz fora da faixa DFS mais disputada, e SSID exclusivo do enlace:

```
/interface wireless security-profiles
add name=ptp-sede-torre authentication-types=wpa2-psk mode=dynamic-keys \
    wpa2-pre-shared-key=Backhaul!Forte2026

/interface wireless
set wlan1 disabled=no mode=ap-bridge band=5ghz-a/n/ac \
    channel-width=20/40mhz-Ce frequency=5500 ssid=ptp-sede-torre-01 \
    wireless-protocol=nv2 security-profile=ptp-sede-torre \
    nv2-cell-radius=15 tdma-period-size=3
```

O `nv2-cell-radius=15` cobre nossos 15 km de enlace com folga mínima; `tdma-period-size=3` favorece throughput sem arruinar a latência.

Passo 2 — lado B (torre, estação). `station-bridge` (queremos L2 transparente para o backhaul, como visto no 10) e `wireless-protocol=nv2`:


```

/interface wireless security-profiles
add name=ptp-sede-torre authentication-types=wpa2-psk mode=dynamic-keys \
    wpa2-pre-shared-key=Backhaul!Forte2026

/interface wireless
set wlan1 disabled=no mode=station-bridge band=5ghz-a/n/ac \
    channel-width=20/40mhz-Ce ssid=ptp-sede-torre-01 \
    wireless-protocol=nv2 security-profile=ptp-sede-torre

```

Na estação não se define **frequency** nem parâmetros de TDMA: ela **segue o AP** — varre a banda, encontra o SSID e obedece ao mapa de alocação que ele publica.

Passo 3 — conferir que o NV2 está de fato ativo. Em rádios reais:

```

/interface wireless registration-table print detail

```

```

0 interface=wlan1 mac-address=CC:2D:E0:11:22:33 ap=no wds=no
  rx-rate="390Mbps-40MHz/2S" tx-rate="390Mbps-40MHz/2S"
  uptime=12m4s signal-strength=-61dBm tx-signal-strength=-62dBm
  signal-to-noise=39dB nv2=yes tx-ccq=96% rx-ccq=95%

```

A linha que interessa: **nv2=yes**. Se aparecer **nv2=no**, um dos lados caiu para 802.11 (protocolo any + AP mal configurado é a causa clássica) e todo o ganho do capítulo evaporou.

Passo 4 — medir o throughput real. O RouterOS traz um gerador de tráfego embutido, o **bandwidth-test**. Do lado B contra o lado A (IP 10.99.0.1 na gerência do enlace):

```

/tool bandwidth-test address=10.99.0.1 protocol=udp direction=both \
    user=vitor password=Lab!Segura2026

```

```

    status: running
    duration: 18s
tx-current: 231.4Mbps
rx-current: 224.9Mbps

```

Aviso

O **bandwidth-test** **satura o enlace e o CPU** dos dois rádios. Em produção, rode fora do horário de pico, por poucos segundos, e nunca a partir do rádio que está roteando clientes — os resultados saem distorcidos e os clientes sentem.

Passo 5 — a prova da conta. Taxa aérea negociada: 390 Mbps (40 MHz/2S). Nossa estimativa: $390 \times 0,55 \approx 214$ Mbps agregados. O teste mostrou $\sim 230 + 225 = 455$? Não — **direction=both** mede os dois sentidos **simultaneamente** dividindo o ar: o agregado real

é tx+rx 456 Mbps *aparentes* apenas se o enlace fosse full-duplex. Compare cada sentido isolado (`direction=transmit`, depois `receive`) com a taxa aérea para ver o half-duplex em ação.

29.5.2 Verificação

1. `registration-table print detail` mostra `nv2=yes` nos dois lados;
2. O CCQ ficou 90% **durante** o `bandwidth-test` (rode os dois ao mesmo tempo em janelas separadas) — TDMA sob carga não colapsa;
3. `direction=transmit` e `direction=receive` isolados somam mais que o `both` de cada sentido — você viu o half-duplex;
4. Você sabe dizer, sem olhar o texto, o que acontece se `nv2-cell-radius` for menor que a distância real do enlace.

29.5.3 Desafios

1. Um enlace PtP de 25 km com NV2 funciona, mas perde ~3% dos pacotes de forma constante, com bom sinal (-58 dBm) e SNR de 35 dB. O `nv2-cell-radius` está no padrão de fábrica de um firmware antigo:
10. Explique a relação.
2. O gerente quer “dobrar a banda do backhaul” trocando `tdma-period-size` de 3 para 10. Estime o efeito real no throughput e o efeito colateral na latência — e proponha a mudança que de fato dobraria a capacidade (dica: física, não protocolo).
3. Você precisa fechar um PtP entre um SXT MikroTik e um rádio de outro fabricante que “fala 802.11ac”. Quais linhas dos Passos 1 e 2 mudam, e o que você perde?
4. Num PtP saudável, `bandwidth-test protocol=tcp` mostra bem menos que `protocol=udp`. O enlace está ruim? Justifique pensando em quem gera e quem confirma o tráfego em cada protocolo.

Solução dos desafios

1. O `nv2-cell-radius=10` dimensiona os tempos de guarda para 10 km, mas o sinal leva o tempo de 25 km para chegar. Transmissões da estação chegam ao AP **depois** do fim da fatia reservada, invadindo a fatia seguinte: quadros corrompidos de forma constante e independente do sinal — exatamente o sintoma. Ajustar `nv2-cell-radius=25` resolve.
2. Período maior reduz a fração de ar gasta em mapas e guardas — ganho real, porém modesto (tipicamente < 10–15%, longe de dobrar), ao custo de a estação esperar até 10 ms pela sua fatia: a latência (e o jitter percebido em VoIP) sobe. Para dobrar capacidade de verdade: mais espectro ou mais fluxos — canal de 40→80 MHz (se o espectro local permitir) ou hardware com mais cadeias/frequência mais limpa. Protocolo não cria hertz.
3. Muda uma linha em cada lado: `wireless-protocol=nv2` → `wireless-`

`protocol=802.11` (ou `any` no lado MikroTik). Perde-se o TDMA: volta o ACK por quadro com prazo sensível à distância (ajuste `distance` no AP!), a latência fica irregular e o throughput cai — e `nv2-cell-radius/tdma-period-size` passam a ser ignorados.

4. Não necessariamente. O teste UDP mede o ar bruto: o gerador empurra datagramas sem esperar confirmação. O TCP embute controle de congestionamento e confirmações — que atravessam o mesmo enlace half-duplex no sentido contrário e disputam CPU no rádio. Menos TCP que UDP é esperado; preocupante é TCP *muito* abaixo (aí investigue perda de pacotes, que derruba TCP desproporcionalmente).

29.6 Resumo

- O 802.11 paga **disputa + ACK por quadro**, e o prazo do ACK foi calibrado para ambientes internos: em enlaces longos, a espera e as retransmissões comem o throughput;
- **Nstreme** (polling, sem ACK por quadro) foi a primeira resposta; **NV2** (TDMA com mapa de alocação) é a atual — latência previsível, throughput estável e escala em PtMP; ambos exigem MikroTik dos dois lados;
- `tdma-period-size` troca latência por throughput; **`nv2-cell-radius` deve cobrir a estação mais distante**, senão as fatias TDMA se atropelam;
- Regra de bolso: **NV2 em todo enlace MikroTik MikroTik externo**; 802.11 apenas para interoperar com outros fabricantes;
- Taxa aérea throughput: estime ~55% da PHY (agregado, half-duplex) e **valide com `bandwidth-test`** — fora do pico e com cautela.

No próximo capítulo (11) o NV2 mostra sua força de verdade: uma setorial atendendo dezenas de assinantes rurais em PtMP.

Bibliografia do capítulo

- Documentação oficial: seções *Wireless — Nstreme/NV2 protocols* e *Bandwidth Test* (MikroTik, 2026a);
- Fundamentos de acesso ao meio (CSMA/CA, TDMA): (Tanenbaum; Feamster; Wetherall, 2021);
- Configuração wireless no RouterOS v7: (Discher, 2016);
- Prática de enlaces MikroTik: (Burgess, 2011).

30 PtMP rural: a torre que atende dezenas de clientes

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

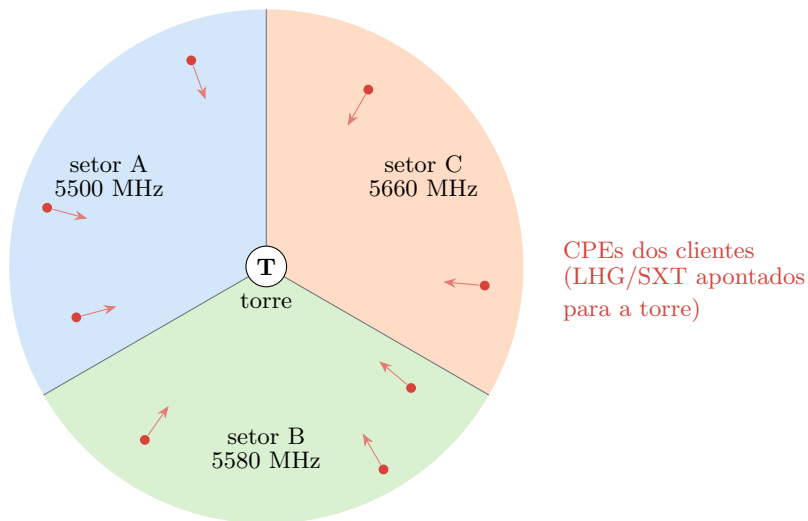
- Projetar uma célula PtMP: setoriais no AP, LHG/SXT nos clientes, reuso de frequência entre setores;
- Configurar o AP setorial com NV2 e os CPEs dos assinantes;
- Usar `access-list` para controlar **quem** conecta e **com que qualidade mínima** (signal range) — e por que isso salva o setor;
- Aplicar limites por cliente no rádio (`ap-tx-limit/client-tx-limit`) e entender onde eles terminam e o QoS do Volume 7 começa;
- Reconhecer o problema do *hidden node* e o do “cliente lento que arrasta o setor” — e as defesas contra ambos.

O PtP do 9 levou a internet da sede até a torre. Agora vem a parte que paga as contas do provedor rural: **distribuir** essa banda para as fazendas, sítios e vilas ao redor — dezenas de assinantes pendurados na mesma torre. Isso é **PtMP** (*ponto-multiponto*): um AP com antena **setorial** servindo muitas estações com antenas diretivas (LHG, SXT) apontadas de volta.

A física é a mesma do Capítulo 27 e os modos são os do 10. O que muda é a **escala** — e com ela aparecem problemas que não existiam no PtP: clientes que não se escutam, vizinhos que conectam sem contrato, e o assinante de sinal ruim que degrada o setor inteiro.

30.1 Anatomia de uma célula PtMP

No lado da torre, **antenas setoriais**: cobrem um leque horizontal (60°, 90° ou 120°) com ganho moderado. Três setoriais de 120° fecham o círculo. No lado do cliente, o **CPE** (*Customer Premises Equipment*): um rádio integrado a uma antena diretiva — LHG (grade parabólica, mais ganho, enlaces longos) ou SXT (painel compacto, instalações mais simples).



Célula PtMP vista de cima: três setoriais de 120° na torre, cada uma em uma frequência diferente para não interferirem entre si, e os CPEs dos assinantes apontados de volta. Cada setor é um domínio TDMA independente.

Figura 30.1

Regras de projeto que evitam dor de cabeça:

- **Uma frequência por setor**, com separação suficiente para não haver sobreposição (com canais de 20 MHz: ex.: 5500/5580/5660);
- **NV2 obrigatório**: com dezenas de estações que não se escutam (fazendas em lados opostos da torre — o *hidden node* clássico), o CSMA/CA colapsa em colisões; o mapa TDMA do 9 simplesmente ignora o problema, pois ninguém disputa o ar;
- **Canais de 20 MHz** no rural: metade do throughput de 40 MHz, mas o dobro de opções de frequência e ~3 dB a mais de sensibilidade — num setor, robustez vale mais que pico;
- **nv2-cell-radius = distância do cliente mais distante** do setor (arredondada para cima), como no capítulo anterior;
- CPE sempre em **mode=station** (roteando ou com NAT) ou **station-bridge** se o projeto exigir L2 — decisão que o Volume 6 revisita com PPPoE.

30.2 O AP setorial

Configuração de um setor (ex.: setor A da figura), consolidando o que os capítulos anteriores ensinaram:

```
/interface wireless security-profiles
add name=setor-clientes authentication-types=wpa2-psk mode=dynamic-keys \
```

```
wpa2-pre-shared-key=SetorRural!2026

/interface wireless
set wlan1 disabled=no mode=ap-bridge band=5ghz-a/n \
    channel-width=20mhz frequency=5500 ssid=isp-rural-setor-a \
    wireless-protocol=nv2 security-profile=setor-clientes \
    nv2-cell-radius=12 tdma-period-size=dynamic \
    default-authentication=no
```

Duas escolhas pedem explicação:

- **band=5ghz-a/n** (e não **a/n/ac**): CPEs rurais de entrada (SXT Lite, LHG 5) são 802.11n; travar a banda evita negociações instáveis;
- **default-authentication=no**: eis a novidade do capítulo. Com **no**, o AP **recusa qualquer estação que não esteja na access-list**. O setor deixa de ser “quem tem a senha entra” e vira “entra quem está no contrato”.

30.3 access-list: o porteiro do setor

A **access-list** é avaliada **antes** da associação: quando uma estação tenta conectar, o AP procura a primeira entrada que case com o MAC (e opcionalmente com a interface e a faixa de sinal) e obedece ao **authentication=yes/no** dela. Com **default-authentication=no** no AP, quem não casa com nenhuma entrada fica de fora.

```
/interface wireless access-list
add interface=wlan1 mac-address=CC:2D:E0:AA:11:01 comment="cli-fazenda-boavista" \
    signal-range=-80..120
add interface=wlan1 mac-address=CC:2D:E0:AA:11:02 comment="cli-sitio-ipe" \
    signal-range=-80..120
```

O parâmetro discreto que muda o jogo é **signal-range=-80..120**: a entrada só autoriza a associação se o sinal do cliente estiver **acima de -80 dBm**. Cliente com antena desalinhada (ou árvore que cresceu na zona de Fresnel, Capítulo 27) não conecta a -87 dBm com taxa de 6 Mbps — ele **fica fora** até a instalação ser corrigida.

Por que isso importa: no TDMA, tempo de ar é o recurso compartilhado. Um cliente a 6 Mbps ocupa **~24× mais tempo de ar** para transferir os mesmos bytes que um a 144 Mbps. Um único CPE ruim consome as fatias do setor inteiro — o “cliente lento que arrasta o setor”. O **signal-range** transforma um problema técnico invisível em um chamado de instalação explícito.

! O conceito mais importante do capítulo

Num setor PtMP, **o recurso escasso é o tempo de ar, e ele é de todos**. Cada decisão do capítulo — NV2, 20 MHz, `access-list` com `signal-range`, limites por cliente — existe para proteger o tempo de ar coletivo de um indivíduo mal instalado ou mal-intencionado. Setor bom não é o que aceita todo mundo; é o que **só aceita enlaces saudáveis** e reparte o ar de forma previsível.

💡 Analogia para guardar

O setor é um ônibus rural com horário certo (TDMA): cada passageiro tem seu assento e a viagem rende. A `access-list` é o motorista conferindo a passagem na porta (`default-authentication=no`: sem lista, ninguém sobe) — e o `signal-range` é a regra de não deixar embarcar quem vai viajar pendurado na porta com o ônibus andando: atrasa a viagem de **todos**, não só a dele.

30.3.1 Limites de banda no rádio

A `access-list` também aceita `ap-tx-limit` (download para o cliente) e `client-tx-limit` (upload dele, bits/s):

```
/interface wireless access-list
set [find comment="cli-fazenda-boavista"] ap-tx-limit=20M client-tx-limit=5M
```

Isso é um limitador **bruto, por rádio** — útil como cinto de segurança no setor. O controle de planos de verdade (fila por assinante, burst, prioridade) é assunto do Volume 7 (QoS), e a entrega comercial por cliente será via PPPoE no Volume 6. Regra prática: limite no rádio um pouco **acima** do plano vendido, e deixe a fila fazer o trabalho fino.

30.4 O CPE do cliente

No telhado da fazenda, um LHG 5 em `mode=station`:

```
/interface wireless security-profiles
add name=setor-clientes authentication-types=wpa2-psk mode=dynamic-keys \
    wpa2-pre-shared-key=SetorRural!2026

/interface wireless
set wlan1 disabled=no mode=station band=5ghz-a/n ssid=isp-rural-setor-a \
    wireless-protocol=nv2 security-profile=setor-clientes
```

E o teste de aceitação da instalação — a leitura que o instalador deve mandar no grupo antes de descer do telhado:

```
/interface wireless registration-table print detail
```

CrITÉRIOS de aceite (nossa política de instalação): **signal-strength** e **tx-signal-strength** **melhores que -65 dBm** e equilibrados ($|\text{diferença}| \leq 5$ dB), **signal-to-noise** 25 dB, **CCQ** 90%. Instalação que não bate isso volta para o alinhamento (Capítulo 31) — antes de o **signal-range** da torre fazer isso virar um chamado noturno.

30.5 Laboratório

i Ambiente

Laboratório de configuração e leitura (o CHR não tem rádio), como no 9: os blocos são os que você aplicaria numa torre real com uma setorial e dois CPEs. Com hardware físico (um AP wireless e dois clientes — até roteadores domésticos MikroTik servem), você reproduz tudo de verdade. Topologia de referência: Capítulo 5.

30.5.1 Comandos passo a passo

Passo 1 — o setor. Aplique no AP o bloco do setor A (security profile + `set wlan1 ... default-authentication=no`) mostrado acima.

Passo 2 — tente conectar um CPE *sem* access-list. Configure o CPE (bloco do LHG acima) e observe no AP:

```
/log print where topics~"wireless"
```

```
12:40:18 wireless,info CC:2D:E0:AA:11:01@wlan1: connected
12:40:18 wireless,info CC:2D:E0:AA:11:01@wlan1: disconnected, banned by access-
list
```

O CPE conecta e cai imediatamente: sem entrada na lista e com `default-authentication=no`, o porteiro barra — mesmo com a senha certa.

Passo 3 — autorize o cliente. No AP:

```
/interface wireless access-list
add interface=wlan1 mac-address=CC:2D:E0:AA:11:01 \
    comment="cli-fazenda-boavista" signal-range=-80..120 \
    ap-tx-limit=20M client-tx-limit=5M
```


Agora o CPE associa e **permanece**:

```
/interface wireless registration-table print
```

#	INTERFACE	MAC-ADDRESS	AP	SIGNAL-STRENGTH	TX-RATE	UPTIME
0	wlan1	CC:2D:E0:AA:11:01	no	-61dBm	130Mbps	4m12s

Passo 4 — repita para o segundo CPE (...:02, comment “cli-sitio-ipe”) e confirme os dois na registration table.

Passo 5 — simule o cliente degradado. Edite a entrada do cliente 1 apertando o piso de sinal para um valor que ele não alcança:

```
/interface wireless access-list
set [find comment="cli-fazenda-boavista"] signal-range=-55..120
/log print where topics~"wireless"
```

Com sinal real de -61 dBm (< -55), o CPE cai e o log registra o **banned by access-list**. Foi exatamente o que aconteceria com um cliente desalinhado num setor com política de sinal. Restaure `signal-range=-80..120` ao final.

30.5.2 Verificação

1. Sem entrada na access-list, o CPE **não** fica associado (**banned by access-list** no log) apesar da senha correta;
2. Com a entrada, os dois CPEs aparecem na registration table com uptime crescendo;
3. O Passo 5 derrubou só o cliente com `signal-range` apertado — o outro nem piscou (uptime preservado);
4. Você sabe apontar, na configuração do setor, as **quatro** defesas do tempo de ar coletivo (protocolo, largura de canal, porteiro, limites).

30.5.3 Desafios

1. Um setor com 40 clientes NV2 funciona bem até ~19h, quando a latência de **todos** sobe para 300 ms. A registration table mostra um único cliente com `tx-rate=13Mbps` e tráfego alto constante. Explique o mecanismo e proponha duas correções (uma imediata, uma definitiva).
2. O vizinho do cliente descobriu a senha WPA2 do setor (ela é única, 10 alertou). Com a configuração deste capítulo, o que ele consegue? E se o AP estivesse com `default-authentication=yes`?
3. Projete as frequências de uma torre com **quatro** setores de 90° em 5 GHz, canais de 20 MHz, sabendo que setores opostos (costas um para o outro) podem reusar frequência. Quantas frequências você precisa no mínimo? Desenhe a atribuição.

4. Um cliente reclama de velocidade à noite, mas sua leitura é impecável (-58 dBm, SNR 32 dB, CCQ 97%). O `ap-tx-limit` dele é 20M e o plano vendido é 20M. Onde mais o gargalo pode estar? Liste três suspeitos na ordem em que você investigaria.

💡 Solução dos desafios

1. É o cliente lento arrastando o setor: a 13 Mbps, cada byte dele custa $\sim 10\times$ mais tempo de ar que o de um cliente saudável; no pico de tráfego, as fatias TDMA que sobram para os demais encolhem e a fila de espera (latência) explode para todos. Imediata: **signal-range** na entrada dele (ou **disabled=yes**) para tirá-lo do ar e proteger o coletivo. Definitiva: visita técnica — realinhar/trocar o CPE (ou elevar, se a Fresnel foi obstruída) até ele voltar a taxas altas.
2. Quase nada: ele associa por ter a senha, mas o MAC dele não está na `access-list` e **default-authentication=no** o derruba (**banned by access-list**). Com **default-authentication=yes**, ele entraria como cliente pleno — consumindo tempo de ar e alcançando a rede do setor. (Clonar um MAC autorizado ainda seria possível — por isso a senha por assinante do PPPoE, no Volume 6, fecha o ciclo.)
3. Duas frequências bastam. Setores opostos irradiam para lados contrários com antenas diretivas: o isolamento costas-com-costas permite reuso. Atribuição clássica em X: N=5500, L=5580, S=5500, O=5580 (padrão A-B-A-B). Na prática, confirme o isolamento real medindo o sinal do setor oposto num CPE de borda — reuso é projeto, não fé.
4. (a) O **backhaul** da torre no pico — se o PtP do 9 satura às 19h, todos os setores sofrem juntos (verifique o gráfico da interface do PtP); (b) o **tempo de ar do setor** consumido pelos vizinhos (um cliente lento como no desafio 1 prejudica até quem tem leitura perfeita); (c) o **lado LAN do cliente** — WiFi doméstico ruim atrás do CPE (teste cabeado no CPE isola). A leitura de rádio impecável só isenta o *enlace dele*, nada mais.

30.6 Resumo

- PtMP rural: **setoriais** na torre (uma frequência por setor, TDMA independente) e **CPEs diretivos** (LHG/SXT) nos clientes;
- **NV2 é obrigatório** em PtMP: estações que não se escutam inviabilizam o CSMA/CA (*hidden node*); o mapa TDMA elimina a disputa;
- **Canais de 20 MHz** privilegiam robustez e reuso de espectro; `nv2-cell-radius` cobre o cliente mais distante;
- **default-authentication=no + access-list** = só entra quem está no contrato; **signal-range** barra instalações ruins antes que arrastem o setor; `ap-tx-limit/client-tx-limit` são o cinto de segurança bruto (QoS fino no Volume 7, entrega por assinante no Volume 6);
- Política de aceite de instalação (sinal, simetria, SNR, CCQ) vale mais que qualquer troubleshooting posterior — e o Capítulo 31 ensina a alcançá-la.

Bibliografia do capítulo

- Documentação oficial: seções *Wireless — Access List* e *NV2* (MikroTik, 2026a);
- Hidden node e acesso ao meio: (Tanenbaum; Feamster; Wetherall, 2021);
- Wireless no RouterOS v7: (Discher, 2016);
- Operação de provedor com MikroTik: (Burgess, 2011);
- Linhas de produto (setoriais, LHG, SXT): (MikroTik, 2026c).

31 Alinhamento e troubleshooting de enlaces

i Objetivos de aprendizagem

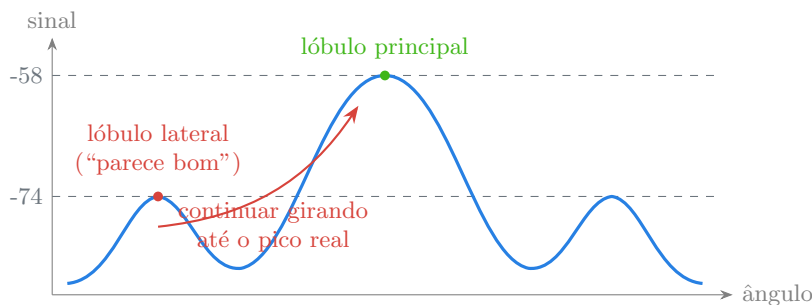
Ao final deste capítulo, você será capaz de:

- Alinhar uma antena diretiva com método — usando **align**, o *beep* de sinal e a registration table como instrumentos;
- Reconhecer (e não cair em) um **lóculo lateral** durante o alinhamento;
- Usar as ferramentas de diagnóstico do RouterOS: **scan**, **frequency-usage**, **snooper** e o histórico da registration table;
- Diagnosticar os quatro vilões clássicos de enlace degradado — desalinhamento, interferência, obstrução de Fresnel e hardware/cabo — distinguindo-os pela **assinatura** de cada um nos indicadores;
- Conduzir um troubleshooting ordenado, da leitura remota à visita técnica, sem trocar peças por adivinhação.

O 11 terminou com uma política de aceite: sinal melhor que -65 dBm, simétrico, SNR 25 dB, CCQ 90%. Este capítulo ensina a **alcançar** esses números no telhado — e a **recuperá-los** quando um enlace que funcionava começa a degradar. É o capítulo mais “de campo” do volume: métodos, instrumentos e assinaturas de defeito.

31.1 Alinhar é maximizar, não “achar”

Uma antena diretiva concentra energia num **lóculo principal** estreito — mas o diagrama de radiação (Capítulo 27) também tem **lóculos laterais**: direções secundárias onde a antena ainda ganha alguns dB. Eis a armadilha do instalador apressado: girando a antena, o sinal sobe de -85 para -74 dBm e ele para — só que -74 dBm era um lóculo lateral, e o principal daria -58 dBm alguns graus adiante.



O sinal em função do ângulo da antena: os lóbulos laterais formam picos falsos ~15 dB abaixo do principal. O método correto varre **todo** o arco, anota o melhor valor e só então retorna a ele — nunca para no primeiro pico.

Figura 31.1

31.1.1 O método (vale para PtP e CPE)

1. **Aponte no grosso** pela geografia: bússola/aplicativo de mapa, ou mire um marco visível (a torre, morro conhecido). Isso coloca você perto do lóbulo principal;
2. **Trave um eixo e varra o outro, devagar** (azimute primeiro, depois elevação). Velocidade: a leitura de sinal atualiza $\sim 1 \times /s$ — gire $2-3^\circ$ e espere a leitura assentar;
3. **Varra o arco inteiro** anotando o pico real — não pare no primeiro máximo (Figura 31.1);
4. Volte ao pico, **aperte o eixo, e repita no outro eixo**. Aperto de parafuso desloca a antena: confira a leitura *depois* de apertar;
5. Valide contra a política de aceite: sinal, **simetria** (rx e tx), SNR, CCQ e taxa — os cinco, não só o primeiro.

O RouterOS ajuda com o modo de alinhamento, que emite *beeps* no compartimento do rádio (frequência do beep sobe com o sinal — mãos livres no mastro):

```
/interface wireless align
set active-mode=yes audio-monitor=CC:2D:E0:00:00:01 filter-mac=CC:2D:E0:00:00:01
monitor wlan1
```

E na estação, o monitor contínuo (que você já conhece do 10) durante a varredura:

```
/interface wireless registration-table print interval=1
```

💡 Analogia para guardar

Alinhar antena é sintonizar rádio AM antigo no escuro: girando o botão, você passa por estações fracas (lóbulos laterais) antes de achar a forte (principal). Quem para na

primeira estação audível ouve chiado para sempre. O método é o mesmo: varre a faixa **toda**, decora onde tocou mais alto e volta lá.

31.2 A caixa de ferramentas de diagnóstico

Quatro instrumentos do RouterOS, cada um respondendo a uma pergunta:

Tabela 31.1: Ferramentas de diagnóstico wireless

Ferramenta	Pergunta que responde	Cuidado
<code>scan</code>	“que redes existem neste canal/banda?”	derruba o enlace durante a varredura
<code>frequency-usage</code>	“quais frequências estão ocupadas, e quanto?”	também interrompe o serviço
<code>snooper</code>	“quem está ocupando o ar, por SSID/estação?”	idem — use em janela de manutenção
<code>registration-table</code>	“como está <i>o meu</i> enlace agora?”	passivo, não interrompe nada

Aviso

`scan`, `frequency-usage` e `snooper` colocam o rádio em modo de varredura: **todos os clientes do setor caem** enquanto rodam. Em produção, use dentro de janela de manutenção ou num rádio de teste dedicado — nunca “só um scan rapidinho” no setor cheio às 20h.

Exemplos de leitura:

```
/interface wireless frequency-usage wlan1 duration=30s
```

FREQ	USAGE
5500	12.5%
5520	3.1%
5580	67.4%
5660	8.9%

A frequência 5580 está 67% ocupada — se o seu setor está nela, achou o vilão; migre para 5520 (após conferir com `scan` quem é o ocupante).

```
/interface wireless scan wlan1 duration=20s
```

FLAGS	ADDRESS	SSID	CHANNEL	SIG	NF	SNR
AP	D4:CA:6D:99:88:77	outro-provedor	5580/20/an	-66	-110	44
AP	CC:2D:E0:BB:22:33	isp-rural-setor-b	5660/20/an	-71	-110	39

31.3 As quatro assinaturas de enlace degradado

O ouro do troubleshooting é saber que **cada causa deixa uma impressão digital diferente** nos indicadores:

Tabela 31.2: Assinaturas de degradação e seus vilões

Assinatura	Sinal	SNR	CCQ	Padrão temporal	Vilão provável
Sinal caiu nos dois lados, piso de ruído normal	↓↓	↓	↓	após vento/tempestade; degrada de vez	desalinhamento (antena girou)
Sinal normal, ruído subiu, CCQ despenca	=	↓↓	↓↓	horários certos (ex.: 18–23h)	interferência (novo AP vizinho)
Sinal caiu aos poucos ao longo de meses	↓ (lento)	↓	↓	sazonal/progressivo	Fresnel (árvore cresceu, Capítulo 27)
Sinal assimétrico ou instável, quedas secas	errático			aleatório, piora com chuva	hardware/cabo (conector, RF, fonte)

! O conceito mais importante do capítulo

Não existe “o enlace está ruim” — existe **desalinhamento**, **interferência**, **obstrução** ou **hardware**, e cada um assina os indicadores de um jeito (Tabela 31.2). Antes de subir no telhado, leia a assinatura: *o que* mudou (sinal? ruído? simetria?), *quando* mudou (de vez? por horário? aos poucos?) e *dos dois lados ou de um só*. Três perguntas separam a visita técnica certa da troca de peças por adivinhação.

O interrogatório remoto, em ordem:

1. **Sinal mudou?** Compare com a leitura de aceite da instalação (você guardou, certo? O Volume 13 automatiza esse histórico);

2. **Simétrico?** `signal-strength tx-signal-strength`? Assimetria forte = problema de **um** lado (potência, cabo, LNA queimado);
3. **SNR acompanhou o sinal?** Sinal igual + SNR pior = ruído novo = interferência; sinal e SNR caindo juntos = enlace (alinhamento ou obstrução);
4. **Padrão no tempo?** Por horário = interferência (ou saturação de tráfego — confira antes!); pós-tempestade = mecânico; progressivo = vegetação;
5. Só então: `frequency-usage/scan` em janela de manutenção, e visita com a Figura 31.1 na cabeça.

31.4 Laboratório

i Ambiente

Este é um **laboratório de diagnóstico**: sem rádio no CHR, o exercício é ler cenários reais e apontar o vilão — exatamente a habilidade que o plantão exige. Com hardware físico (qualquer par wireless dos capítulos anteriores), reproduza os passos 1–2 de verdade; os passos 3–5 são casos clínicos em texto. Topologia de referência: Capítulo 5.

31.4.1 Comandos passo a passo

Passo 1 — (bancada) monte o “alinhamento de mesa”. Com o enlace do 10 ativo, rode na estação:

```
/interface wireless registration-table print interval=1
```

Gire lentamente a antena (ou o roteador) e observe o sinal responder a cada segundo. Procure de propósito um “falso pico”: posições diferentes com sinal parecido — a versão de mesa dos lóbulos laterais.

Passo 2 — (bancada) meça o ambiente.

```
/interface wireless frequency-usage wlan1 duration=30s  
/interface wireless scan wlan1 duration=20s
```

Anote as frequências mais ocupadas da sua vizinhança e confira com o `scan` quem são os ocupantes. Note que o enlace **caiu** durante as varreduras — a lição do callout-warning em carne própria.

Passo 3 — **caso clínico A**. Enlace PtP de 8 km, funcionava com -59/-60 dBm. Após um temporal: -78 dBm/-79 dBm, SNR de 20 dB, piso de ruído inalterado (-110 dBm), CCQ 61%. Aponte o vilão e o plano.

Passo 4 — caso clínico B. CPE rural: sinal -62 dBm estável o dia todo, mas das 19h às 23h o SNR cai de 34 dB para 14 dB e o CCQ para 55%. `frequency-usage` (madrugada, em manutenção) mostra 5500 com 8%; às 20h, 71%. Vilão e plano.

Passo 5 — caso clínico C. PtP: `signal-strength=-58dBm`, `tx-signal-strength=-81dBm`. Chove, os dois pioram 10 dB e o lado fraco derruba o enlace. Vilão e plano.

31.4.2 Verificação

1. No Passo 1 você encontrou pelo menos um falso pico e o pico real — e o pico real permaneceu após apertar/soltar o suporte;
2. No Passo 2 o enlace caiu durante o `scan` (confirme pelo `uptime` zerado na `registration table`) — você viu o custo da ferramenta;
3. Nos casos A–C, você apontou o vilão citando a linha correspondente da Tabela 31.2 — antes de ler as soluções.

31.4.3 Desafios

1. Resolva os casos clínicos A, B e C (Passos 3–5) por escrito: vilão, evidência que o denuncia e plano de ação em ordem.
2. Um instalador alinhou um LHG e reportou -67 dBm “aprovado”. Dez minutos de varredura teriam achado -55 dBm. Usando a Figura 31.1, explique o que provavelmente aconteceu e escreva a instrução de UMA frase que você colocaria no procedimento de instalação para impedir isso.
3. Por que a política de aceite exige **simetria** entre `signal-strength` e `tx-signal-strength`, e não apenas um bom valor de recepção? Dê um defeito concreto que só a assimetria revela.
4. Seu setor está na frequência 5580 — a mesma que o `scan` mostrou ocupada por **outro-provedor** com sinal -66 dBm. Antes de migrar de frequência, que conversa (humana) pode resolver melhor que qualquer comando? O que propor?

Solução dos desafios

1. Caso A: queda simétrica e abrupta pós-temporal, ruído inalterado — assinatura de **desalinhamento mecânico** (vento girou uma antena). Plano: visita ao lado com suporte mais suspeito, realinhar pelo método completo, apertar e re-medir. **Caso B:** sinal estável, SNR desabando por horário, frequência 67%+ ocupada no pico — **interferência** de um transmissor ativo à noite. Plano: `scan` para identificar, migrar o setor para frequência limpa (5520) em janela de manutenção, re-testar às 20h. **Caso C:** assimetria de 23 dB é **hardware do lado fraco** — o transmissor de um lado (ou a recepção do outro) está comprometido: conector mal crimpado, pigtail, umidade; chuva piorando aponta infiltração. Plano: inspecionar conectores/vedação do lado que transmite fraco, trocar cabo/rádio em bancada, e só realinhar depois de sanar o hardware.

2. Ele parou no primeiro pico — um lóbulo lateral ~12 dB abaixo do principal (exatamente o ponto vermelho da figura). Instrução de uma frase: “Varra o arco completo de azimuth e elevação anotando o melhor valor e retorne a ele; só aceite a instalação se nenhuma outra posição do arco superar a leitura final.”
3. Recepção boa prova só metade do enlace: **signal-strength** é o que **eu ouço**; **tx-signal-strength** é o que **o outro lado me ouve**. Um amplificador de transmissão degradado, um conector ruim no caminho de TX ou potências mal configuradas produzem exatamente rx ótimo + tx péssimo — download bom, upload péssimo (o cenário do desafio 2 do 10). Só a comparação dos dois lados pega defeitos unidirecionais.
4. Coordenação de espectro entre provedores: vocês dois perdem com a sobreposição (o TDMA de cada um enxerga o outro como ruído). Propor divisão do espectro local — cada um com suas frequências fixas documentadas, potência mínima necessária e, idealmente, um canal de contato para mudanças futuras. Uma planilha compartilhada de frequências por torre resolve o que meses de “guerra de potência” só pioram — subir potência contra interferência eleva o piso de ruído de todos (Capítulo 27).

31.5 Resumo

- Alinhamento é **método**: apontar no grosso, varrer o arco inteiro eixo a eixo, desconfiar de picos precoces (lóbulos laterais, Figura 31.1), apertar e re-conferir, validar pelos cinco indicadores;
- Instrumentos: `align/beep` e `registration-table print interval=1` para alinhar; `scan`, `frequency-usage` e `snooper` para investigar o espectro — **os três interrompem o serviço**;
- Cada vilão tem assinatura (Tabela 31.2): desalinhamento (queda abrupta simétrica, ruído normal), interferência (SNR cai com sinal estável, padrão por horário), Fresnel (degradação lenta e progressiva), hardware (assimetria e comportamento errático);
- Interrogatório remoto antes da visita: o que mudou, quando mudou, de que lado — e leitura de aceite arquivada vale ouro (o Volume 13 automatiza esse histórico).

Com enlaces montados, protegidos e alinhados, falta o último capítulo do volume: o WiFi **dentro** da casa do cliente, com CAPsMAN e o pacote `wifi` moderno (Capítulo 32).

Bibliografia do capítulo

- Documentação oficial: seções *Wireless* — *Scan*, *Frequency Usage*, *Snooper*, *Align* (MikroTik, 2026a);
- Propagação e antenas (lóbulos, diagrama de radiação): (Tanenbaum; Feamster; Wetherall, 2021);
- Wireless no RouterOS v7: (Discher, 2016);
- Operação de campo com MikroTik: (Burgess, 2011).

32 CAPsMAN e o WiFi moderno

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Distinguir os **dois stacks** de wireless do RouterOS v7 — o legado `/interface wireless` e o moderno `/interface wifi` (wave2/ax) — e saber qual hardware usa qual;
- Explicar o modelo controlador/AP do **CAPsMAN**: o que fica no controlador (configuração), o que fica no CAP (o rádio) e como o tráfego pode fluir (local forwarding vs. datapath centralizado);
- Configurar um CAPsMAN v2 (pacote `wifi`) completo: provisionamento automático, configuration/datapath/security por perfil;
- Provisionar um AP da casa do cliente de fábrica ao ar sem tocar nele (*zero-touch*);
- Decidir quando CAPsMAN vale a pena para um provedor — e quando um AP avulso bem configurado basta.

O volume até aqui viveu do lado de fora: torres, setoriais, CPEs. Este último capítulo entra na **casa do assinante** — o WiFi que ele de fato usa e pelo qual julga o provedor inteiro. E traz um problema novo, de **gestão**: 500 clientes = 500 APs. Mudar a senha de um SSID corporativo em 30 pontos de um cliente empresarial, ou padronizar o canal de 200 comodatos, um a um, não escala. A resposta da MikroTik é o **CAPsMAN** (*Controlled Access Point system Manager*): APs que buscam sua configuração num controlador central.

32.1 Os dois stacks de wireless do v7

Antes do CAPsMAN, um esclarecimento que evita horas de confusão — no RouterOS v7 convivem **dois subsistemas wireless**:

Tabela 32.1: Os dois stacks wireless do RouterOS v7

	<code>/interface wireless</code> (legado)	<code>/interface wifi</code> (moderno)
Padrões	até 802.11n/ac (wave1)	802.11ac wave2, 802.11ax (WiFi 6)
Hardware	rádios Atheros antigos (SXT, LHG, setoriais clássicas)	hAP ax ² , hAP ax ³ , cAP ax, Audience...

	<code>/interface wireless</code> (legado)	<code>/interface wifi</code> (moderno)
Protocolos MikroTik	Nstreme, NV2	— (só 802.11)
Segurança	WPA/WPA2	WPA2, WPA3
CAPsMAN	CAPsMAN “v1” (<code>/caps-man</code>)	CAPsMAN “v2” (<code>/interface wifi capsman</code>)
Papel no provedor	enlaces externos (Vol. 5 até aqui)	WiFi indoor do cliente

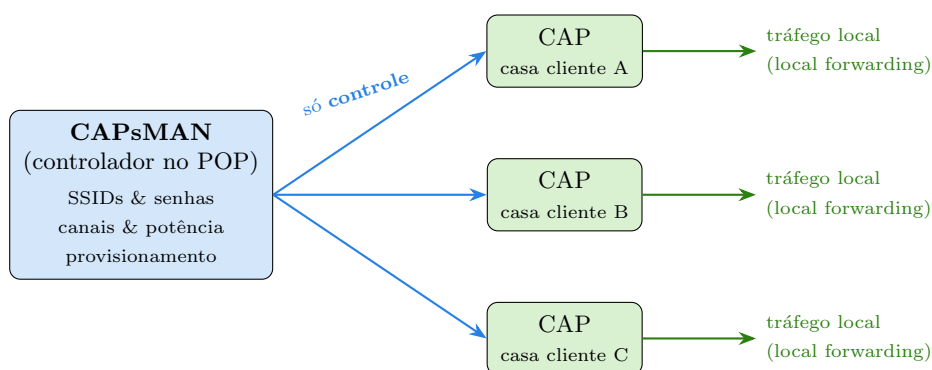
Os capítulos 10 a Capítulo 31 usaram o stack legado — é ele que fala NV2 e vive nas torres. Dentro de casa a história inverte: ninguém precisa de TDMA na sala, e sim de WiFi 6, WPA3 e roaming decente. **Os dois CAPsMANs são incompatíveis entre si:** controlador `wifi` gerencia CAPs `wifi`; o `/caps-man` legado, apenas rádios legados. Este capítulo usa o moderno — é o presente dos APs indoor e o futuro da linha.

32.2 O modelo: controlador e CAPs

No CAPsMAN há dois papéis:

- **CAPsMAN (controlador):** qualquer RouterOS com o pacote `wifi` (nem precisa ter rádio — pode ser o roteador de borda do POP ou até um CHR). Guarda **toda** a configuração: SSIDs, senhas, canais, datapaths;
- **CAP** (*Controlled Access Point*): o AP na casa do cliente. De fábrica, só precisa saber **encontrar o controlador**; recebe dele a configuração e passa a ser gerenciado remotamente.

A descoberta é automática na L2 (broadcast na mesma rede/VLAN) ou por IP na L3 (`caps-man-addresses` apontando para o controlador — o caso típico do provedor, com o controlador no POP).



A decisão de arquitetura mais importante é **por onde fluem os dados**:

CAPsMAN com *local forwarding*: entre controlador e CAPs viaja apenas **controle** (configuração, estatísticas); os dados dos usuários saem direto na rede local de cada casa. O controlador pode gerenciar centenas de CAPs sem tocar num único pacote de dados deles.

Figura 32.1

- **Local forwarding** (padrão no stack `wifi`): o tráfego WiFi entra direto na bridge do CAP, na casa do cliente. O controlador só configura. Escala sem esforço — o caso do provedor;
- **Datapath centralizado**: o tráfego é tunelado até o controlador, que o injeta na rede dele. Útil quando as políticas (VLANs por SSID, firewall central) precisam se aplicar num ponto único — comum em ambiente corporativo, raro em casa de assinante.

💡 Analogia para guardar

CAPsMAN é a **franquia de fast-food**: a matriz (controlador) define o cardápio, os preços e o padrão da loja (SSID, senha, canal); cada loja (CAP) atende os clientes localmente — a comida não passa pela matriz (*local forwarding*). Abrir loja nova é assinar o contrato e pendurar a placa: o padrão desce sozinho da matriz (*provisionamento*). E quando a matriz muda o cardápio, muda em todas as lojas no mesmo instante.

32.3 Configurando o controlador

Cenário: o roteador de borda do POP (nosso `cli-paoquente-r1` de tantos capítulos) vira controlador dos APs em comodato. Três blocos: perfil de segurança, configuração e provisionamento.

```
/interface wifi security
add name=sec-clientes authentication-types=wpa2-psk,wpa3-psk \
    passphrase=WFiDoCliente!2026

/interface wifi configuration
add name=cfg-clientes-2g ssid=PaoQuente-WiFi security=sec-clientes \
    country=brazil channel.band=2ghz-ax channel.width=20mhz
add name=cfg-clientes-5g ssid=PaoQuente-WiFi security=sec-clientes \
    country=brazil channel.band=5ghz-ax channel.width=20/40/80mhz

/interface wifi capsman
set enabled=yes interfaces=all upgrade-policy=suggest-same-version

/interface wifi provisioning
add action=create-dynamic-enabled supported-bands=2ghz-ax,2ghz-n \
    master-configuration=cfg-clientes-2g
```

```
add action=create-dynamic-enabled supported-bands=5ghz-ax,5ghz-ac \
    master-configuration=config-clientes-5g
```

Lendo de cima para baixo:

- **security:** WPA2+WPA3 misto (`wpa2-psk,wpa3-psk`) — dispositivos novos sobem em WPA3, antigos caem para WPA2. `country=brazil` na configuração aplica os limites da Anatel automaticamente (Capítulo 27);
- **configuration:** um perfil por banda. Mesmo SSID nas duas — os dispositivos escolhem (e o *band steering* dos clientes modernos prefere 5 GHz);
- **capsman set enabled=yes:** liga o controlador; `upgrade-policy=suggest-same-version` mantém os CAPs na mesma versão do RouterOS que o controlador, sem forçar;
- **provisioning:** a regra de ouro — **quando um CAP aparecer**, case suas bandas com um perfil e crie as interfaces automaticamente (`create-dynamic-enabled`). É isso que torna o sistema *zero-touch*.

32.4 Ligando um CAP

No AP da casa do cliente (ex.: um cAP ax em comodato), uma linha:

```
/interface wifi cap
set enabled=yes caps-man-addresses=10.99.0.1
```

`caps-man-addresses` aponta para o controlador (nosso plano de gerência da VLAN 99, definido no Capítulo 26). Na mesma L2, o CAP acharia o controlador sozinho, sem endereço. Segundos depois, no controlador:

```
/interface wifi remote-cap print
```

#	ADDRESS	NAME	BOARD	VERSION	STATE
0	10.99.0.57	cAP-casa-A	cAPGi-5HaxD2HaxD	7.15	provisioned

E as interfaces dinâmicas criadas pelo provisionamento:

```
/interface wifi print where dynamic
```

#	NAME	MASTER	SSID	STATE
0	cap-wifi1	yes	PaoQuente-WiFi	running (2ghz-ax)
1	cap-wifi2	yes	PaoQuente-WiFi	running (5ghz-ax)

O AP saiu da caixa, recebeu cabo e energia, e está no ar com o padrão da empresa — **ninguém configurou WiFi nele**. Trocar a senha dos 200 comodatos amanhã é editar `sec-clientes` no controlador: os CAPs recebem a mudança sozinhos.

! O conceito mais importante do capítulo

CAPsMAN separa **configuração** de **equipamento**. A configuração mora no controlador, versionada e única; o AP é descartável — quebrou, troca-se a caixa e o provisionamento reveste o hardware novo com a mesma configuração em segundos. Depois de operar assim, configurar AP por AP parece o que é: artesanato que não escala. (É o mesmo princípio que você verá em provisionamento de clientes PPPoE no Volume 6 e em automação no Volume 12.)

32.4.1 Quando *não* usar CAPsMAN

CAPsMAN compensa quando há **frota**: comodatos padronizados, clientes corporativos multi-AP, WiFi público em praças. Não compensa para o AP único da casa de um cliente avulso com configuração própria — ali, o hAP configurado à mão (ou pelo app) resolve sem criar dependência do controlador. E lembre: com *local forwarding*, se o controlador cair os CAPs **continuam operando** com a última configuração recebida — só o provisionamento de novos APs para; com datapath centralizado, o tráfego morre junto com o controlador. Mais um ponto para o local forwarding no provedor.

32.5 Laboratório

i Ambiente

Finalmente um laboratório de wireless que o CHR roda de verdade! O **controlador** CAPsMAN não precisa de rádio: suba-o no **lab-r1**. O CAP, sim, precisa — use um AP físico com pacote **wifi** (hAP ax, cAP ax) ligado na rede do laboratório, se tiver. Sem hardware: os Passos 1–2 e 5 rodam integralmente no CHR; os Passos 3–4 ficam como leitura guiada. Topologia: Capítulo 5, snapshot **pop-segmentado**.

32.5.1 Comandos passo a passo

Passo 1 — controlador no lab-r1. Aplique os quatro blocos da seção “Configurando o controlador” (security, duas configurations, capsman enabled, duas regras de provisioning).

Passo 2 — confira o que existe antes de qualquer CAP:

```
/interface wifi configuration print
/interface wifi provisioning print
/interface wifi capsman print
```

A saída deve mostrar os dois perfis, as duas regras e **enabled: yes** — o controlador está pronto, ainda sem nenhum CAP.

Passo 3 — (hardware) plugue o CAP. No AP físico, de fábrica: reset sem configuração padrão (3), IP via DHCP na rede do lab, e:

```
/interface wifi cap
set enabled=yes caps-man-addresses=192.168.56.11
```

Passo 4 — (hardware) acompanhe o provisionamento no lab-r1:

```
/interface wifi remote-cap print
/interface wifi print where dynamic
/interface wifi registration-table print
```

Conecte um celular no SSID PaoQuente-WiFi e veja-o na registration table **do controlador** — a visão central de todos os clientes WiFi da frota.

Passo 5 — a mágica da gestão central. Troque a senha da frota inteira com um comando:

```
/interface wifi security
set sec-clientes passphrase=NovaSenha!2026
```

Com hardware: o celular cai e só volta com a senha nova — sem ninguém tocar no CAP. Sem hardware: `/interface wifi security print` confirma a mudança que os CAPs receberiam.

32.5.2 Verificação

1. `capsman print` mostra `enabled: yes` e as regras de provisioning listam `create-dynamic-enabled` com as bandas corretas;
2. (Hardware) o CAP aparece em `remote-cap print` como `provisioned` sem nenhuma configuração de WiFi feita nele;
3. (Hardware) as interfaces em `print where dynamic` são **dinâmicas** — some o CAP, somem elas; volte o CAP, voltam provisionadas;
4. Você sabe explicar o que acontece com o WiFi das casas se o lab-r1 (controlador) for desligado — e por quê.

32.5.3 Desafios

1. Sua frota tem 150 cAP ax (stack `wifi`) e 40 wAP ac antigos (stack legado). Um único CAPsMAN resolve? Desenhe a solução de gestão para a frota mista.
2. Um cliente empresarial quer dois SSIDs em cada um dos seus 12 APs: **Escritório** (rede interna) e **Visitantes** (isolada, só internet). Esboce os objetos CAPsMAN necessários (configurations, securities, e o papel do datapath/VLAN) — sem escrever os comandos completos.

3. O controlador CAPsMAN do POP caiu numa sexta à noite e só será restaurado na segunda. Liste o que **para** e o que **continua** na frota (local forwarding), e o que muda nessa lista se você tivesse escolhido datapath centralizado.
4. Por que `authentication-types=wpa2-psk,wpa3-psk` em vez de só `wpa3-psk`, se WPA3 é mais seguro? Que tipo de chamado de suporte a escolha “só WPA3” geraria?

💡 Solução dos desafios

1. Não — os CAPsMANs são incompatíveis entre stacks. Solução: dois controladores (podem morar no **mesmo** roteador do POP): `/interface wifi capsman` para os 150 cAP ax e `/caps-man` legado para os 40 wAP ac, cada frota apontando para o seu. Padronize os perfis (mesmos SSIDs/senhas nos dois) e planeje a aposentadoria do stack legado junto com a troca do hardware.
2. Duas **security** (uma por rede — senhas distintas), duas **configuration** por banda (**Escritorio**, **Visitantes**), e o isolamento via **datapath**: cada configuration com um datapath que etiqueta uma VLAN diferente (`vlan-id=10` interna, `vlan-id=20` visitantes) na bridge do CAP — as VLANs do Volume 4 fazendo o isolamento, e o firewall do escritório cuidando de “visitantes só internet”. As regras de provisioning criam os SSIDs extras como *slave configurations* nos mesmos rádios.
3. Com local forwarding **continua**: o WiFi de todas as casas no ar, com a última configuração válida — clientes autenticam, tráfego flui. **Para**: provisionar CAPs novos/trocados, mudanças de configuração da frota, e a visão central (registration table agregada, estatísticas). Com datapath centralizado, a lista muda dramaticamente: o **tráfego** dos usuários morre com o controlador — a frota inteira fica sem internet até segunda. É o argumento decisivo pró-local forwarding no provedor.
4. Porque a base instalada manda: dispositivos anteriores a ~2019/2020 (smart TVs, câmeras, celulares antigos) não falam WPA3. Com “só WPA3”, eles simplesmente não encontram como conectar — e o suporte recebe uma enxurrada de “a TV não pega o WiFi novo” sem mensagem de erro útil. O modo misto sobe cada dispositivo no melhor que ele suporta e deixa a migração acontecer pela renovação natural dos aparelhos.

32.6 Resumo

- O v7 tem **dois stacks**: `/interface wireless` (legado — NV2, torres, Vol. 5 até aqui) e `/interface wifi` (wave2/ax — WPA3, WiFi 6, indoor); cada um com **seu** CAPsMAN, incompatíveis entre si;
- CAPsMAN = **configuração no controlador, rádio no CAP**: security + configuration + provisioning no controlador; no CAP, só `cap set enabled=yes caps-man-addresses=...` — *zero-touch*;
- **Local forwarding** (dados saem na casa do cliente) escala e sobrevive à queda do controlador; datapath centralizado só quando políticas centrais exigem;
- Frota padronizada (comodatos, corporativo multi-AP) → CAPsMAN; AP avulso personalizado → configuração local;
- Fecha o Volume 5: física (Capítulo 27) → enlace e leitura

(10) → PtP (9) → PtMP

(11) → campo (Capítulo 31) → a casa do cliente. O Volume 6 pega daqui: **PPPoE**, o contrato técnico entre o provedor e cada assinante.

Bibliografia do capítulo

- Documentação oficial: seções *WiFi* e *WiFi CAPsMAN* do RouterOS v7 (MikroTik, 2026a);
- WPA2/WPA3 e 802.11ax: (Tanenbaum; Feamster; Wetherall, 2021);
- RouterOS v7 na prática: (Discher, 2016);
- Gestão de frota em provedores: (Burgess, 2011);
- Linha de APs (hAP ax, cAP ax, Audience): (MikroTik, 2026c).

Parte VI

Volume 6 — PPPoE e gestão de assinantes

33 Como funciona o PPPoE

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar por que provedores entregam internet via **PPPoE** em vez de DHCP simples — autenticação por assinante, IP controlado, contabilização;
- Descrever as duas fases do protocolo: **descoberta** (PADI → PADO → PADR → PADS) e **sessão** (LCP → autenticação → IPCP);
- Explicar de onde vem o MTU de **1492** e diagnosticar o sintoma clássico de MTU errado (“ping funciona, site não abre”);
- Aplicar a correção universal de MSS (*clamping*) e saber quando ela é necessária;
- Montar um enlace PPPoE mínimo entre dois CHRs e observar cada fase acontecendo no log.

O Volume 5 terminou com o sinal chegando à casa do assinante. Falta o **contrato técnico**: como o provedor sabe *quem* é aquele cliente, entrega o IP *daquele plano*, mede o consumo *daquele contrato* e corta *só ele* quando a fatura atrasa? A resposta que a indústria consagrou é o **PPPoE** (*Point-to-Point Protocol over Ethernet*) — e este volume inteiro é sobre ele. Este capítulo explica o protocolo; os próximos constroem o concentrador de produção.

33.1 Por que não DHCP e pronto?

No Capítulo 11 você viu que o DHCP entrega IP a qualquer um que peça na rede. Para uma casa, perfeito. Para um provedor, três problemas:

1. **Identidade**: o DHCP identifica por MAC — clonável, trocável (o cliente troca de roteador e “some”). Não há senha, não há contrato;
2. **Sessão**: o DHCP não tem noção de “conectado/desconectado”. Não dá para derrubar um inadimplente *agora*, nem saber quanto tempo e quantos bytes cada assinante consumiu;
3. **Individualização**: na rede compartilhada (a VLAN de clientes do Capítulo 26, o setor PtMP do 11), vizinhos se enxergam em L2 — e qualquer política por cliente vira malabarismo.

O PPPoE resolve os três de uma vez: cria, sobre a Ethernet compartilhada, um **túnel ponto a ponto individual** entre o roteador do cliente e o **concentrador** do provedor (também chamado **BNG** — *Broadband Network Gateway*). Dentro do túnel roda o velho **PPP** —

o protocolo das conexões discadas — que traz de fábrica autenticação por usuário/senha, negociação de IP e contabilização de sessão. Cada assinante vira uma sessão: com login próprio, IP atribuído pelo provedor, contadores próprios e um botão de desligar só dele.

💡 Analogia para guardar

A rede de acesso é o **saguão de um prédio comercial**: todo mundo circula pelo mesmo espaço (a Ethernet/VLAN compartilhada). O PPPoE é a **catraca com crachá**: para entrar de fato, cada visitante se identifica (usuário/senha), recebe um crachá numerado (IP da sessão) e a portaria registra entrada, saída e por onde andou (accounting). O DHCP, em comparação, é um saguão sem catraca com um bedel distribuindo crachás para quem estiver na fila — sem perguntar o nome.

33.2 Fase 1: descoberta (PPPoED)

O cliente não sabe onde está o concentrador — pode haver vários na mesma rede. A fase de **descoberta** resolve isso com quatro quadros Ethernet (EtherType 0x8863):

Tabela 33.1: Os quatro quadros da descoberta PPPoE

Quadro	Direção	Papel
PADI	cliente → broadcast	“Algum concentrador aí?” (<i>Initiation</i>)
PADO	concentrador → cliente	“Eu! Meu nome é X, ofereço o serviço Y” (<i>Offer</i>)
PADR	cliente → concentrador	“Fechado, quero conectar com você” (<i>Request</i>)
PADS	concentrador → cliente	“Confirmado — sua sessão é a n° N” (<i>Session-confirmation</i>)

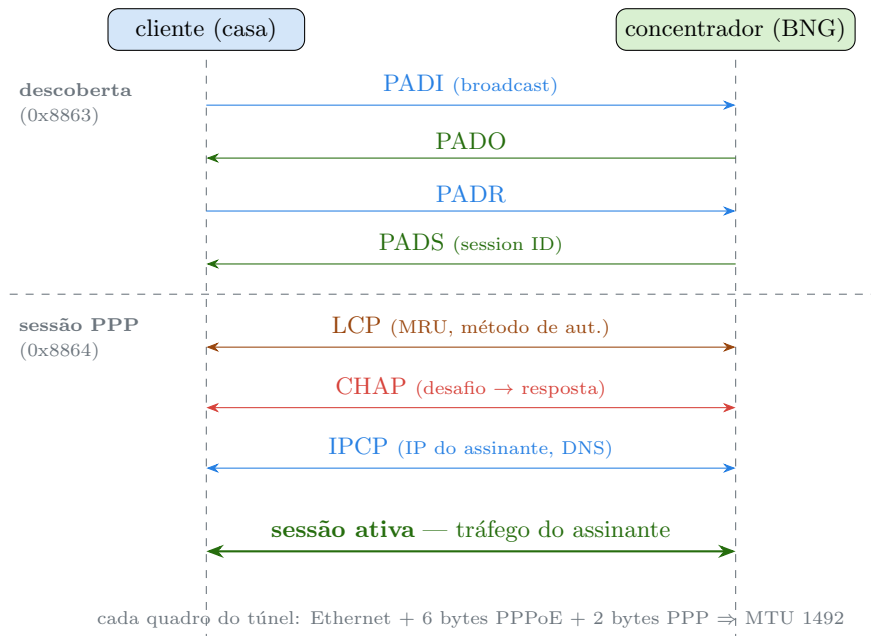
Ao final, cliente e concentrador compartilham um **session ID**: o identificador do túnel. Tudo o que vier depois viaja em quadros EtherType 0x8864 carregando esse ID — o túnel está de pé, mas ainda vazio.

33.3 Fase 2: sessão (PPP)

Dentro do túnel, o PPP negocia em três atos:

1. **LCP** (*Link Control Protocol*): os dois lados acertam os parâmetros do enlace — principalmente **MRU** (o tamanho máximo de pacote que cada um aceita receber) e qual método de autenticação será usado;

2. **Autenticação:** o cliente prova quem é. **PAP** manda a senha em claro; **CHAP** responde a um desafio com um hash — a senha nunca viaja. Use CHAP;
3. **IPCP** (*IP Control Protocol*): o concentrador entrega ao cliente o **IP da sessão**, e opcionalmente DNS. É aqui que o IP do plano contratado chega ao roteador da casa.



As duas fases do PPPoE: a descoberta (PADI/PADO/PADR/PADS, EtherType 0x8863) estabelece o session ID; dentro do túnel (0x8864), o PPP negocia enlace (LCP), autentica (CHAP) e entrega o IP (IPCP). Só então o tráfego do assinante flui.

Figura 33.1

Quando a sessão fecha — logout, queda de enlace, corte por inadimplência — o lado que encerra manda um **PADT** (*Terminate*), e o túnel morre. O ciclo de vida completo da sessão é visível e controlável: exatamente o que faltava ao DHCP.

33.4 O pedágio: MTU 1492 e o site que não abre

Nada é de graça. Cada quadro do túnel carrega **8 bytes extras** de cabeçalho (6 do PPPoE + 2 do PPP). Como o quadro Ethernet continua limitado a 1500 bytes de payload, sobram **1492** para o pacote IP do assinante — o famoso MTU do PPPoE.

O sintoma clássico de MTU mal tratado é dos mais traiçoeiros:

“O ping funciona, o WhatsApp funciona, mas vários sites não abrem — fica carregando para sempre.”

O mecanismo: pacotes pequenos (ping, mensagens) passam; pacotes **grandes** (a resposta HTTPS de 1500 bytes) não cabem no túnel. O roteador deveria avisar o remetente para fragmentar (ICMP *Fragmentation Needed*), mas firewalls mal configurados pelo caminho bloqueiam justamente esse ICMP — e o pacote grande morre em silêncio, para sempre (*PMTUD black hole*).

A defesa do provedor é o **MSS clamping**: reescrever, no handshake de cada conexão TCP que atravessa o túnel, o tamanho máximo de segmento anunciado, forçando-o a caber em 1492:

```
/ip firewall mangle
add chain=forward protocol=tcp tcp-flags=syn action=change-mss new-mss=1452 \
    passthrough=yes tcp-mss=1453-65535 out-interface-list=WAN \
    comment="MSS clamping p/ tuneis PPPoE"
```

(1452 = 1492 de MTU — 40 bytes de cabeçalhos IP+TCP. A regra usa o mangle que você conheceu no 4.) No RouterOS, os *profiles* PPP fazem isso automaticamente com `change-tcp-mss=yes` — o Capítulo 34 mostra onde.

! O conceito mais importante do capítulo

PPPoE = **sessão individual sobre rede compartilhada**. A descoberta acha o concentrador, o PPP autentica e entrega o IP — e a partir daí o assinante não é mais “um MAC numa VLAN”: é uma **sessão** com dono, plano, contadores e ciclo de vida. Todo o modelo comercial do provedor (planos, corte, histórico de consumo) se apoia nessa individualização. O preço é fixo e conhecido: 8 bytes por quadro — e quem esquece o MSS clamping paga em chamados de “internet que abre metade dos sites”.

33.5 Laboratório

i Ambiente

De volta aos CHRs de verdade! Dois bastam: `lab-r1` (concentrador) e `lab-r2` (cliente), ligados pela rede interna `lab-a` (`ether2 ether2`). Parta do snapshot **pop-segmentado** (Capítulo 5) — ou de qualquer estado com a `lab-a` funcional. O objetivo aqui é **ver o protocolo**; o concentrador caprichado é assunto do próximo capítulo.

33.5.1 Comandos passo a passo

Passo 1 — servidor PPPoE mínimo no lab-r1. Um usuário de teste, um pool e o servidor escutando na `ether2`:

```
/ip pool add name=pool-pppoe-teste ranges=100.64.50.10-100.64.50.250
/ppp profile add name=perfil-teste local-address=100.64.50.1 \
    remote-address=pool-pppoe-teste
/ppp secret add name=cliente-teste password=Pppoe!2026 profile=perfil-teste \
    service=pppoe
/interface pppoe-server server add interface=ether2 service-name=isp-lab \
    default-profile=perfil-teste authentication=chap disabled=no
```

Passo 2 — cliente PPPoE no lab-r2. Antes, remova qualquer IP estático da ether2 dele (o IP virá pelo IPCP!):

```
/ip address remove [find interface=ether2]
/interface pppoe-client add interface=ether2 name=pppoe-wan user=cliente-teste \
    password=Pppoe!2026 add-default-route=yes disabled=no
```

Passo 3 — assista às duas fases. No lab-r2:

```
/log print where topics~"pppoe|ppp"
```

```
12:02:11 pppoe,ppp,info pppoe-wan: initializing...
12:02:11 pppoe,ppp,info pppoe-wan: dialing...
12:02:12 pppoe,ppp,info pppoe-wan: authenticated
12:02:12 pppoe,ppp,info pppoe-wan: connected
```

O dialing é a descoberta (PADI→PADS); authenticated é o CHAP; connected fecha o IPCP. Todo o diagrama da Figura 33.1 em quatro linhas de log.

Passo 4 — inspecione o resultado. Ainda no lab-r2:

```
/interface pppoe-client monitor pppoe-wan once
/ip address print where interface=pppoe-wan
```

```
status: connected
service: isp-lab
ac-name: lab-r1
mtu: 1480
mru: 1480
```

Repare: interface nova (pppoe-wan) com IP do pool do servidor — o túnel é uma interface de rede como outra qualquer, e a rota padrão aponta para dentro dele.

Passo 5 — o lado do concentrador. No lab-r1:


```
/ppp active print
```

#	NAME	SERVICE	CALLER-ID	ADDRESS	UPTIME
0	cliente-teste	pppoe	0C:11:22:33:44:02	100.64.50.10	3m21s

Eis a sessão: nome, MAC de origem, IP entregue, uptime. Derrube-a do lado do provedor — o “botão de corte” em ação:

```
/ppp active remove [find name=cliente-teste]
```

No lab-r2, o log mostra **disconnected** (o PADT chegou)... e segundos depois **dialing** de novo: o cliente rediscou sozinho e a sessão voltou.

33.5.2 Verificação

1. `/ppp active print` no lab-r1 mostra a sessão com o IP do pool;
2. No lab-r2, `/ip route print` mostra a rota padrão via **pppoe-wan**;
3. Ping do lab-r2 para 100.64.50.1 (o **local-address** do túnel) responde;
4. Após o **remove** do Passo 5, a sessão caiu e **voltou** sozinha — você viu PADT + rediscagem no log.

33.5.3 Desafios

1. Troque **authentication=chap** por **authentication=pap** no servidor e reconecte. Funciona? Agora rode `/tool sniffer quick interface=ether2` no lab-r1 durante a conexão. O que aparece que justifica a regra “use CHAP”?
2. No lab-r2, o **monitor** mostrou MTU 1480 em vez de 1492. A conta dos 8 bytes está errada? Investigue `/interface pppoe-server server print` (parâmetro **max-mtu**) e explique o valor conservador do RouterOS.
3. Um assinante liga: “ping funciona, mas vários sites HTTPS não carregam”. Descreva (a) o mecanismo provável, (b) o teste com `ping ... size=1500 do-not-fragment` que confirma, e (c) as duas correções possíveis — no cliente e no concentrador.
4. Dois concentradores respondem PADO na mesma VLAN (o antigo e o novo da migração). Como o cliente escolhe? Que parâmetro do **pppoe-client** (dica: **ac-name**) torna a escolha determinística — e por que isso importa numa migração?

Solução dos desafios

1. Com PAP a conexão funciona igual — mas no sniffer a senha **Pppoe!2026** aparece **em texto claro** dentro do quadro PAP Request. Qualquer um na rede de acesso (um vizinho no setor PtMP!) capturando quadros teria a credencial do assinante. Com CHAP, o sniffer mostra apenas desafio e hash — a senha nunca viaja. Regra: **chap** (ou

mschap2) sempre; pap só para depurar em bancada.

2. A conta está certa ($1500 - 8 = 1492$); o RouterOS é que assume o pior caso por padrão: `max-mtu/max-mru` de fábrica valem 1480, uma folga extra de 12 bytes para caminhos com encapsulamentos adicionais (VLAN, túneis). Defina `max-mtu=1492` `max-mru=1492` no servidor e `max-mtu=1492` no cliente para operar no valor pleno — como faremos no Capítulo 34.

3. (a) *PMTUD black hole*: a resposta HTTPS vem em pacotes de 1500 bytes com DF ligado; eles não cabem no túnel (1492), o ICMP *Fragmentation Needed* é bloqueado em algum firewall do caminho e o pacote morre em silêncio — pacotes pequenos (ping) passam. (b) `ping 8.8.8.8 size=1500 do-not-fragment` de dentro da casa: se retornar “packet too large / sem resposta” enquanto `size=1400` funciona, confirmado. (c) No concentrador: `change-tcp-mss=yes` no profile PPP (ou a regra mangle `change-mss` deste capítulo); no cliente: reduzir o MTU da conexão/dispositivo. A do concentrador conserta todos os assinantes de uma vez — é a correta para o provedor.

4. O cliente padrão conecta ao **primeiro PADO que chegar** — numa migração, metade dos clientes cairia no concentrador errado ao acaso. Configurando `ac-name=<nome-do-concentrador>` no `pppoe-client` (o nome vem do PADO — o `ac-name` que o monitor mostrou), o cliente ignora ofertas de outros concentradores e a migração vira uma decisão sua, não da corrida de quadros. No lado servidor, o `service-name` cumpre papel análogo de seleção.

33.6 Resumo

- PPPoE cria uma **sessão ponto a ponto autenticada** por assinante sobre a Ethernet compartilhada — identidade (usuário/senha), IP controlado (IPCP), contabilização e corte individual, tudo que o DHCP não dá;
- **Descoberta** (PADI→PADO→PADR→PADS, EtherType 0x8863) estabelece o session ID; **sessão PPP** (0x8864) negocia LCP → autentica (**CHAP**, nunca PAP em produção) → entrega IP via IPCP; PADT encerra;
- O túnel custa **8 bytes por quadro** → MTU 1492; o sintoma “ping funciona, site não abre” é *PMTUD black hole* e a defesa é **MSS clamping** (`change-tcp-mss=yes` no profile ou `change-mss` no mangle);
- No RouterOS: `/interface pppoe-server server` + `/ppp secret` de um lado, `/interface pppoe-client` do outro; `/ppp active` é a lista viva de sessões — e o botão de corte.

No próximo capítulo (Capítulo 34), o servidor de teste vira um concentrador de verdade: profiles por plano, pools organizados e a entrega do serviço que o cliente contratou.

Bibliografia do capítulo

- Documentação oficial: seções *PPPoE* e *PPP* (MikroTik, 2026a);

- PPP, LCP e autenticação (RFC 2516 e família PPP): (Tanenbaum; Feamster; Wetherall, 2021);
- RouterOS v7 na prática: (Discher, 2016);
- Operação de provedor: (Burgess, 2011).

34 PPPoE server: profiles, pools e secrets

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Organizar o concentrador em três camadas: **pool** (estoque de IPs), **profile** (o plano) e **secret** (o assinante);
- Configurar profiles que carregam tudo do plano: endereços, DNS, limite de banda (**rate-limit**), MSS clamping e isolamento entre clientes;
- Publicar o serviço com `/interface pppoe-server server` na interface certa (a VLAN de clientes do POP) com MTU pleno e CHAP;
- Aplicar as operações do dia a dia: mudar cliente de plano, suspender por inadimplência (sem apagar o cadastro!), enxergar e derrubar sessões;
- Explicar como o **rate-limit** do profile vira fila dinâmica — a ponte para o QoS do Volume 7.

O Capítulo 33 montou um servidor de teste com um usuário. Um provedor real tem centenas — distribuídos em planos diferentes, com inadimplentes para suspender, upgrades para aplicar e IPs para não deixar acabar. A diferença entre o teste e a produção não é quantidade de comandos: é **arquitetura**. Este capítulo organiza o concentrador do jeito que escala.

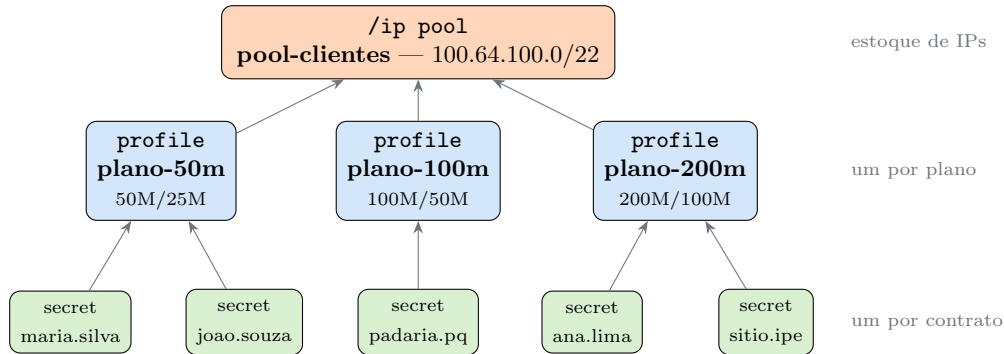
34.1 As três camadas do concentrador

O RouterOS separa a configuração PPP em três objetos, e a disciplina de usar cada um para o que serve é o que mantém 500 assinantes administráveis:

Tabela 34.1: As três camadas da configuração PPP

Camada	Objeto	Responde por	Quantos existem
Estoque	<code>/ip pool</code>	de onde saem os IPs	1 por classe de serviço
Plano	<code>/ppp profile</code>	banda, DNS, endereços, políticas	1 por plano comercial
Assinante	<code>/ppp secret</code>	login, senha, qual plano	1 por contrato

A regra de ouro: **tudo que é do plano fica no profile; no secret, só o que é daquele cliente** (login, senha, profile e — exceções — um IP fixo contratado). Quando o plano de 50 Mbps virar 60 Mbps na promoção, você muda **uma linha** no profile e os 300 assinantes dele herdam a mudança na próxima conexão. Se a banda estivesse espalhada pelos secrets, seriam 300 edições.



As três camadas: secrets (contratos) apontam para profiles (planos), que consomem IPs de um pool (estoque). Mudança de plano comercial = editar um profile; mudança de um cliente = editar um secret.

Figura 34.1

34.2 Camada 1 — o pool

Para os IPs dos assinantes usaremos 100.64.0.0/10 — a faixa reservada para redes de provedor (*shared address space*, RFC 6598), que o Volume 9 destrinchará com o CGNAT. Um /22 dá ~1000 endereços:

```
/ip pool add name=pool-clientes ranges=100.64.100.2-100.64.103.254
```

Dimensione pelo **pico de sessões simultâneas** + 20% de folga: pool esgotado significa cliente novo que não conecta — e o alerta de pool cheio é um dos primeiros que você vai automatizar no Volume 13.

34.3 Camada 2 — profiles (os planos)

Um profile por plano comercial, com tudo que o plano implica:

```
/ppp profile
add name=plano-50m local-address=100.64.100.1 remote-address=pool-clientes \
  dns-server=100.64.100.1 rate-limit=50M/25M change-tcp-mss=yes \
  only-one=yes use-encryption=no
```

```
add name=plano-100m local-address=100.64.100.1 remote-address=pool-clientes \
    dns-server=100.64.100.1 rate-limit=100M/50M change-tcp-mss=yes \
    only-one=yes use-encryption=no
add name=plano-200m local-address=100.64.100.1 remote-address=pool-clientes \
    dns-server=100.64.100.1 rate-limit=200M/100M change-tcp-mss=yes \
    only-one=yes use-encryption=no
```

Linha a linha do que importa:

- **local-address**: o “lado de cá” de todos os túneis — pode ser o mesmo IP para todos os perfis (cada sessão é ponto a ponto, não há conflito);
- **remote-address=pool-clientes**: de onde sai o IP do assinante. Vários perfis compartilhando um pool é normal;
- **dns-server**: o DNS que o IPCP entrega — o resolver do provedor (Capítulo 12), não um público, para manter o cache perto do cliente;
- **rate-limit=50M/25M**: a entrega do plano. Formato **download/upload do ponto de vista do cliente** (o RouterOS lê como rx/tx do concentrador: o que ele recebe do cliente/o que envia a ele — atenção à ordem: **tx/rx** na documentação é enviado-ao-cliente/recebido-do-cliente, ou seja, download/upload). Ao conectar, o concentrador cria uma **fila dinâmica** com esses tetos para a sessão;
- **change-tcp-mss=yes**: o MSS clamping do Capítulo 33, automático por sessão — sem regra de mangle manual;
- **only-one=yes**: um login = uma sessão simultânea. Sem isso, a senha compartilhada com o vizinho vira duas casas no preço de uma;
- **use-encryption=no**: MPPE não protege nada que TLS já não proteja e custa CPU no concentrador — desnecessário em acesso de provedor.

Analogia para guardar

Pense numa **academia**: o pool é o estoque de armários, o profile é o tipo de plano (musculação, completo, premium — cada um define a que horas pode entrar e quais áreas usa), e o secret é a matrícula de cada aluno com sua digital. Mudar o horário do plano completo se faz **no contrato do plano**, não aluno por aluno; suspender um aluno inadimplente é bloquear **a matrícula dele**, não demolir o armário nem mudar o plano dos outros.

34.4 Camada 3 — secrets (os assinantes)

Um secret por contrato, apontando para o profile do plano:

```
/ppp secret
add name=maria.silva password=Ms!7201xk service=pppoe profile=plano-50m \
    comment="contrato 1042 - Rua das Flores 12"
```

```
add name=padaria.paoquente password=Pq!9034mz service=pppoe profile=plano-100m \
    comment="contrato 0007 - cliente ancora"
add name=sitio.ipe password=Si!5518ab service=pppoe profile=plano-200m \
    remote-address=100.64.100.250 comment="contrato 0831 - IP fixo contratado"
```

Repare no `sitio.ipe`: `remote-address` **no secret** sobrepõe o pool do profile — é assim que se entrega IP fixo a quem contratou, sem criar um profile só para isso. E o `comment` amarra o login ao contrato no sistema de gestão: seu futuro eu às 3h da manhã agradece.

Aviso

Senhas de assinante são **credenciais em produção**: gere-as aleatórias (nunca `nome123`), guarde-as no sistema de gestão e lembre que `/ppp secret print` as exhibe em claro para quem tem acesso `full` ao roteador — mais um motivo para as contas restritas do 5.

34.5 Publicando o serviço

Falta o servidor escutar onde os clientes estão. No POP segmentado do Capítulo 26, os assinantes chegam pela VLAN 101 (`vlan101-clientes`):

```
/interface pppoe-server server
add interface=vlan101-clientes service-name=isp-paoquente \
    default-profile=plano-50m authentication=chap \
    max-mtu=1492 max-mru=1492 keepalive-timeout=10 disabled=no
```

- `interface=vlan101-clientes`: o serviço existe **só** onde deve — um PPPoE server em `ether1` (WAN) seria um convite à internet;
- `max-mtu/max-mru=1492`: MTU pleno (o desafio 2 do capítulo anterior explicou o 1480 conservador de fábrica);
- `authentication=chap`: PAP proibido fora da bancada;
- `keepalive-timeout=10`: sessões de clientes desligados somem em ~10 s (o Capítulo 36 ajusta fino);
- `default-profile`: usado se o `secret` não indicar profile — deixe apontando para o **menor** plano, nunca para o maior (falha conservadora).

Um detalhe de arquitetura que economiza dores: com PPPoE, o firewall de forward do concentrador (Capítulo 18) enxerga cada assinante como uma interface própria — e como o tráfego cliente cliente passa **pelo concentrador** (cada sessão é um túnel), o isolamento entre vizinhos deixa de depender de truques de L2.

! O conceito mais importante do capítulo

O **profile** é o plano comercial em forma de configuração. Toda decisão de negócio — quanto custa, quanto entrega, DNS de quem, uma ou duas sessões — mora num objeto só, versionável com `/export` e auditável. O `secret` carrega apenas a identidade. Provedor que mistura as camadas (banda no `secret`, IP fixo num `profile` clonado às pressas) acumula uma dívida técnica que explode exatamente no pior dia: o da migração de concentrador. Disciplina nas três camadas é o sistema de gestão embrionário — e é o que o RADIUS do próximo capítulo vai substituir sem dor.

34.6 Operações do dia a dia

Mudar de plano (upgrade da maria.silva):

```
/ppp secret set [find name=maria.silva] profile=plano-100m
/ppp active remove [find name=maria.silva]
```

O `profile` novo vale para a **próxima** sessão — o `active remove` derruba a atual e o cliente reconecta já no plano novo (segundos de queda; combine com o cliente ou deixe para a madrugada).

Suspender por inadimplência — a operação mais frequente do provedor. Nunca apague o `secret` (perde-se o cadastro, o histórico e o IP fixo); **desabilite**:

```
/ppp secret set [find name=maria.silva] disabled=yes
/ppp active remove [find name=maria.silva]
```

Reativar é `disabled=no`. (Provedores maduros preferem mover o cliente para um `profile plano-suspenso` que redireciona para a página de cobrança — dá para fazer com `address-list` + NAT do Capítulo 19; o RADIUS do próximo capítulo automatiza isso.)

Enxergar a operação:

```
/ppp active print stats
/ppp secret print count-only where disabled=yes
/ip pool used print
```


34.7 Laboratório

i Ambiente

lab-r1 (concentrador) e lab-r2 (cliente), rede lab-a, a partir do estado do laboratório anterior (Capítulo 33). Vamos **substituir** o servidor de teste pela estrutura de produção — comece removendo os objetos ***-teste** se ainda existirem: `/interface pppoe-server server remove [find]`, `/ppp secret remove [find name=cliente-teste]`, `/ppp profile remove [find name=perfil-teste]`, `/ip pool remove [find name=pool-pppoe-teste]`.

34.7.1 Comandos passo a passo

Passo 1 — as três camadas no lab-r1. Aplique, na ordem: o pool, os três profiles e os três secrets das seções acima. Depois o servidor — no laboratório, em **ether2** (não temos a VLAN 101 do POP real aqui; se você guardou o snapshot **pop-segmentado** com as VLANs, use **vlan101-clientes** e ganhe fidelidade):

```
/interface pppoe-server server
add interface=ether2 service-name=isp-paoquente default-profile=plano-50m \
    authentication=chap max-mtu=1492 max-mru=1492 keepalive-timeout=10 \
    disabled=no
```

Passo 2 — conecte como maria.silva no lab-r2:

```
/interface pppoe-client set pppoe-wan user=maria.silva password=Ms!7201xk
```

Passo 3 — confira o que o profile entregou. No lab-r1:

```
/ppp active print detail
/queue simple print
```

```
0 name="<pppoe-maria.silva>" target=100.64.100.2/32
  max-limit=25M/50M dynamic=yes
```

Eis o **rate-limit** materializado: uma **fila simples dinâmica** criada com a sessão (e destruída com ela). O Volume 7 começa exatamente desta linha.

Passo 4 — teste o only-one. Configure um segundo cliente PPPoE no lab-r3 (ou uma segunda interface **pppoe-client** no próprio lab-r2) com as MESMAS credenciais de **maria.silva** e observe o log do lab-r1: a segunda tentativa é rejeitada enquanto a primeira sessão vive.

Passo 5 — o ciclo comercial completo. Upgrade, suspensão e reativação da maria.silva (blocos da seção “Operações do dia a dia”), observando após cada passo o `/ppp active print` e, no lab-r2, o log reconectando (ou falhando com `user disabled` durante a suspensão).

Passo 6 — IP fixo. Conecte o lab-r3 como `sitio.ipe` e confirme que o IP é **sempre** 100.64.100.250, mesmo reconectando — enquanto maria.silva pega o próximo livre do pool.

34.7.2 Verificação

1. `/queue simple print` mostra a fila dinâmica com o `max-limit` correspondente ao profile do cliente conectado — e ela **some** quando a sessão cai;
2. A segunda sessão com o mesmo login foi rejeitada (`only-one`);
3. Durante a suspensão, o lab-r2 não conecta e o log do concentrador registra a recusa; após `disabled=no`, reconecta sozinho;
4. `sitio.ipe` recebe sempre o IP fixo; `/ip pool used print` mostra os dinâmicos saindo de `pool-clientes`.

34.7.3 Desafios

1. O comercial lançou o “plano 100M turbo de fim de semana”. Sem tocar em nenhum secret, o que você edita para que TODOS os clientes do plano-100m passem o sábado a 150M — e qual é a pegadinha sobre **quando** a mudança vale para cada cliente?
2. Um cliente com IP fixo (`sitio.ipe`) pediu downgrade para o plano-50m mantendo o IP. Escreva o comando único que resolve, e explique por que o IP sobrevive à mudança de profile.
3. Você encontrou num concentrador herdado 40 secrets com `rate-limit` configurado **no secret** (via RADIUS antigo), cada um diferente do profile. Qual valor vence quando os dois existem? Monte um experimento de bancada para provar.
4. Projete o “profile de inadimplente” citado no texto: que parâmetros ele teria (`rate-limit`? `address-list`?) e que regras dos volumes 3 (Capítulo 19) fariam o redirecionamento para a página de cobrança? Esboce, sem precisar testar.

Solução dos desafios

1. `/ppp profile set plano-100m rate-limit=150M/75M` na sexta e o inverso no domingo. A pegadinha: o profile é lido **na conexão** — quem já está conectado continua na fila antiga até reconectar. Para valer imediatamente: derrubar as sessões do plano (`/ppp active remove [find where ...]`) e deixar a rediscagem aplicar o novo — ou, mais elegante, editar as filas dinâmicas ativas via script (Volume 12) / esperar o ciclo natural de reconexões.
2. `/ppp secret set [find name=sitio.ipe] profile=plano-50m` (e um `active remove` para aplicar já). O IP sobrevive porque `remote-address` está **no secret**, e atributos do secret **sobrepõem** os do profile — a mudança de profile troca só o que o

secret não define (a banda, o DNS...).

3. O secret vence — a regra geral do PPP no RouterOS é que o atributo mais específico (secret > profile) prevalece. Experimento: profile com `rate-limit=50M/25M`, secret com `rate-limit=10M/5M`, conectar e ler o `max-limit` da fila dinâmica em `/queue simple print`: virá 5M/10M (upload/download do ponto de vista da fila). Depois inverte (remove do secret) e confirme 25M/50M.

4. Profile plano-suspenso: `rate-limit=512k/512k` (suficiente para a página de cobrança carregar), `address-list=inadimplentes` (todo IP de sessão desse profile entra na lista automaticamente — o PPP faz isso via profile!), mesmo pool. Regras: no NAT, `chain=dstnat src-address-list=inadimplentes protocol=tcp dst-port=80 action=dst-nat to-addresses=<servidor-da-pagina>` (Capítulo 19); no filter, `chain=forward src-address-list=inadimplentes` liberando só o servidor de cobrança/DNS e dropando o resto (Capítulo 18). HTTPS não se redireciona (certificado não bate — Capítulo 12 explicou por quê); o drop + página em HTTP é o padrão da indústria.

34.8 Resumo

- Concentrador em **três camadas**: pool (estoque de IPs, dimensione pelo pico + folga), profile (o **plano**: `rate-limit`, `change-tcp-mss=yes`, `only-one=yes`, DNS) e secret (o **contrato**: login, senha, profile; IP fixo como exceção via `remote-address` no secret, que sobrepõe o profile);
- Servidor na **interface certa** (VLAN de clientes, jamais na WAN), `authentication=chap`, `max-mtu/max-mru=1492`;
- `rate-limit` vira **fila simples dinâmica** por sessão — a entrega técnica do plano comercial (e o gancho para o Volume 7);
- Dia a dia: upgrade = editar secret + derrubar sessão; inadimplência = `disabled=yes` (nunca `remove`) ou profile de suspensão com redirecionamento;
- Tudo isso ainda é **local** do concentrador. Quando houver dois concentradores — ou um sistema de gestão — o cadastro precisa sair do roteador: **RADIUS**, no próximo capítulo (12).

Bibliografia do capítulo

- Documentação oficial: seções *PPP*, *PPPoE* e *IP Pools* (MikroTik, 2026a);
- Shared address space RFC 6598 e endereçamento: (Rekhter et al., 1996);
- RouterOS v7 na prática: (Discher, 2016);
- Operação comercial de provedor: (Burgess, 2011).

35 RADIUS, User Manager e gestão

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o modelo **AAA** (autenticação, autorização, contabilização) e o papel do RADIUS entre o concentrador e o sistema de gestão;
- Descrever o diálogo RADIUS: Access-Request → Access-Accept (com **atributos** que carregam plano, IP e banda) e os pacotes de Accounting;
- Configurar o concentrador como **cliente RADIUS** e o **User Manager** do RouterOS v7 como servidor, movendo os assinantes do `/ppp secret` local para a base central;
- Usar **CoA/Disconnect** (RFC 3576) para mudar plano e cortar cliente **sem** derrubar/reconectar manualmente;
- Situar os sistemas de gestão de provedor (ERP/billing): o que eles fazem por baixo é exatamente isto.

O Capítulo 34 deixou um concentrador exemplar — com um defeito invisível: o cadastro mora **dentro do roteador**. Funciona com um concentrador e um operador disciplinado. Mas e quando houver dois POPs? Sincronizar secrets na mão? E quando o financeiro precisar suspender o contrato 1042 sem saber o que é SSH? A resposta da indústria tem nome desde os anos 1990: **RADIUS** — o protocolo que separa quem *autentica* de quem *decide*.

35.1 AAA: o modelo por trás de tudo

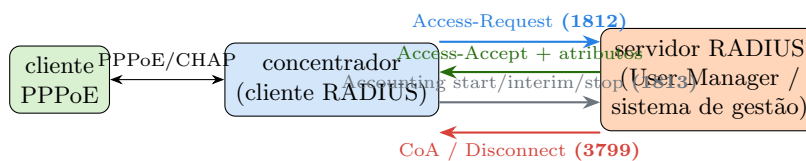
AAA é a sigla que organiza o problema:

- **Autenticação** — *quem é você?* (a senha confere?);
- **Autorização** — *o que você pode?* (qual plano, qual IP, quanta banda?);
- **Accounting** (contabilização) — *o que você usou?* (bytes, tempo, quando conectou e desconectou).

No modelo local do capítulo anterior, o concentrador fazia os três sozinho, consultando seus próprios secrets. Com RADIUS, ele **terceiriza as decisões**: vira um *cliente* que pergunta a um servidor central — e o servidor central é (ou conversa com) o sistema de gestão do provedor.

O diálogo, na conexão de um assinante:

1. O cliente PPPoE manda usuário/senha (CHAP, como sempre);
2. O concentrador embala tudo num **Access-Request** UDP (porta 1812) para o servidor RADIUS, assinado com um **segredo compartilhado**;
3. O servidor consulta a base (o cadastro do provedor!) e responde:
 - **Access-Accept** + **atributos**: “aceita; entregue o IP X, DNS Y, banda Z” — a autorização viaja como pares atributo=valor (**Framed-IP-Address**, **Mikrotik-Rate-Limit**, ...);
 - ou **Access-Reject**: “recusa” (senha errada, contrato suspenso);
4. Conectado, o concentrador manda **Accounting-Start** (porta 1813), atualizações periódicas (*interim*) com contadores de bytes, e **Accounting-Stop** na desconexão — o extrato da sessão.



O concentrador terceiriza o AAA: autenticação e autorização via Access-Request/Accept (com os atributos do plano), extrato via Accounting — e, no sentido inverso, o servidor comanda a sessão viva com CoA/Disconnect.

Figura 35.1

💡 Analogia para guardar

O concentrador com secrets locais é o **porteiro com caderninho**: ele mesmo decide quem entra, e cada portaria tem seu caderno — atualizar todos é um pesadelo. RADIUS é a portaria ligada à **central de segurança**: o porteiro (concentrador) coleta o crachá e liga para a central (Access-Request); a central consulta o cadastro único e responde “deixa entrar, sala 204, elevador B” (Accept + atributos). E a central pode ligar de volta — “aquele visitante: troque-o de sala” (CoA) ou “retire-o do prédio” (Disconnect) — sem trocar o porteiro.

35.2 Os atributos: onde mora a autorização

O Access-Accept não diz só “sim” — carrega a configuração da sessão. Os atributos que importam num provedor MikroTik:

Tabela 35.1: Atributos RADIUS úteis no acesso PPPoE

Atributo	O que define	Equivalente local
Framed-IP-Address	IP fixo do assinante	remote-address no secret

Atributo	O que define	Equivalente local
Framed-Pool	pool de onde tirar o IP	<code>remote-address</code> no profile
Mikrotik-Rate-Limit	banda (ex.: "100M/50M")	<code>rate-limit</code> no profile
Mikrotik-Group / profile	perfil PPP a aplicar	<code>profile</code> no secret
Mikrotik-Address-List	address-list da sessão	<code>address-list</code> no profile
Session-Timeout	duração máxima da sessão	—

Os Mikrotik-* são **VSAs** (*vendor-specific attributes*): extensões da MikroTik ao dicionário RADIUS padrão. É por eles que o sistema de gestão controla banda e políticas por assinante — o profile local vira um **molde genérico**, e o RADIUS preenche as diferenças por cliente.

Note a inversão arquitetural: no capítulo anterior, plano = profile e mudar o cliente de plano = editar secret. Com RADIUS, o cadastro (e o plano de cada um) mora na base central; o concentrador só precisa de um ou dois profiles genéricos. Os secrets locais podem — devem — sumir.

35.3 O lado concentrador: cliente RADIUS

Dois comandos transformam o concentrador do capítulo anterior:

```
/radius add service=ppp address=10.99.0.20 secret=SegredoRadius!2026 \
    timeout=800ms comment="user manager no servidor de gestao"

/ppp aaa set use-radius=yes accounting=yes interim-update=5m
```

- **service=ppp**: vale para PPPoE (e L2TP, PPTP...); o mesmo servidor RADIUS pode atender login, hotspot, dhcp;
- **address**: o servidor RADIUS — na VLAN de serviços do POP (Capítulo 26), nunca exposto à internet;
- **secret**: o segredo compartilhado que assina os pacotes — trate como senha de infraestrutura;
- **use-radius=yes**: a chave-geral. A ordem de consulta do RouterOS é: **secret local primeiro; se não existir, pergunta ao RADIUS**. Ótimo para migrar aos poucos — e uma pegadinha clássica: um secret local esquecido “rouba” o login antes do RADIUS ver;
- **interim-update=5m**: contadores de consumo atualizados a cada 5 min (é o que alimenta gráficos de consumo por assinante na gestão).

E para aceitar comandos remotos sobre sessões vivas (CoA/Disconnect, porta UDP 3799):

```
/radius incoming set accept=yes port=3799
```

Aviso

`/radius incoming accept=yes` abre uma porta de **controle** no concentrador: quem falar RADIUS com o segredo certo derruba qualquer assinante. Restrinja no firewall de input (Capítulo 17) à VLAN de serviços/IP do servidor de gestão — jamais alcançável da internet ou da rede de clientes.

35.4 O lado servidor: User Manager

O **User Manager** é o servidor RADIUS da própria MikroTik — no RouterOS v7, um pacote opcional (`user-manager.npk`) que roda em qualquer roteador ou CHR. Para um provedor pequeno (ou para aprender), ele fecha o ciclo sem software externo:

```
/user-manager set enabled=yes

/user-manager router add name=bng-pop01 address=10.99.0.1 \
    shared-secret=SegredoRadius!2026

/user-manager profile add name=um-plano-50m name-for-users="Plano 50 Mega"
/user-manager profile-limitation add profile=um-plano-50m \
    limitation=lim-50m
/user-manager limitation add name=lim-50m rate-limit-rx=25M rate-limit-tx=50M

/user-manager user add name=maria.silva password=Ms!7201xk group=default
/user-manager user-profile add user=maria.silva profile=um-plano-50m
```

A anatomia espelha o que você já sabe: **router** cadastra **quem pode perguntar** (cada concentrador, com seu segredo); **limitation** + **profile** são o plano (repare: **rate-limit-tx** é o download do cliente — tx do ponto de vista do concentrador); **user** é o assinante. O User Manager responde na porta 1812/1813 e envia CoA na 3799.

E os sistemas de gestão comerciais (os ERPs de provedor do mercado brasileiro, integrados a boleto/PIX e estoque)? **Todos falam RADIUS** — por baixo, fazem exatamente o que o User Manager faz, com o financeiro apertando os botões. Aprendido o mecanismo aqui, qualquer um deles é interface nova sobre protocolo conhecido.

35.5 CoA: mudar a sessão sem derrubá-la

No capítulo anterior, upgrade de plano exigia **active remove** + reconexão. O **CoA** (*Change of Authorization*, RFC 3576) elimina até isso: o servidor manda novos atributos para a sessão **viva** e o concentrador os aplica no ato — a fila dinâmica muda de teto sem o cliente perceber. No User Manager, editar o profile do usuário e aplicar dispara o CoA; o irmão dele, **Disconnect-Request**, derruba a sessão por comando remoto (é o “cortar

inadimplente” apertado pelo financeiro no sistema de gestão — que nunca precisou de SSH no concentrador).

! O conceito mais importante do capítulo

RADIUS inverte o fluxo de autoridade: **o concentrador deixa de ser dono do cadastro e vira executor de decisões centrais**. Autenticação, plano, IP, corte — tudo passa a nascer na base de gestão e viajar como atributos. É isso que permite N concentradores com um cadastro só, financeiro operando sem tocar em roteador, e o extrato (accounting) alimentando cobrança e gráficos. O `/ppp secret` local não morre à toa: ele vira o **fallback de emergência** — e a pegadinha de migração, porque é consultado *antes* do RADIUS.

35.6 Laboratório

i Ambiente

Três CHRs (Capítulo 5): `lab-r1` segue como concentrador, `lab-r2` como cliente PPPoE — e o `lab-r3` estreia como **servidor de gestão**, rodando o User Manager, alcançável pelo `lab-r1` (rede `lab-b` via `lab-r2` em bridge, ou ligue `lab-r3` direto na `lab-a` no VirtualBox — qualquer caminho IP serve). Instale o pacote `user-manager.npk` no `lab-r3` (mesmo procedimento de pacotes do Capítulo 8: upload do `.npk` da versão exata e reboot).

35.6.1 Comandos passo a passo

Passo 1 — User Manager no lab-r3 (IP dele na rede: `172.16.0.3`, ajuste ao seu lab): aplique os blocos da seção “O lado servidor” — `enabled=yes`, o `router` apontando para o IP do `lab-r1` com o segredo, o trio `limitation/profile/profile-limitation` do plano 50M e a usuária `maria.silva`.

Passo 2 — concentrador vira cliente RADIUS no lab-r1:

```
/radius add service=ppp address=172.16.0.3 secret=SegredoRadius!2026 \  
    timeout=800ms  
/ppp aaa set use-radius=yes accounting=yes interim-update=1m  
/radius incoming set accept=yes port=3799
```

Passo 3 — a prova da inversão. Apague (ou desabilite) o `secret` local da `maria.silva` no `lab-r1`:


```
/ppp secret set [find name=maria.silva] disabled=yes
```

Reconecte o lab-r2. A sessão **sobe mesmo assim** — autenticada pelo lab-r3. Confira os dois lados:

```
/ppp active print detail  
/radius monitor 0
```

```
requests: 4  
accepts: 3  
rejects: 1  
timeouts: 0
```

No lab-r3, `/user-manager session print` mostra a sessão com contadores — o accounting chegando.

Passo 4 — Reject. Mude a senha no lab-r2 para uma errada e reconecte: o log do lab-r1 mostra a recusa vinda do RADIUS (`radius reject`), e o contador `rejects` do `monitor` incrementa. Restaure a senha.

Passo 5 — corte remoto (Disconnect). No lab-r3:

```
/user-manager session print  
/user-manager session remove [find where user=maria.silva active=yes]
```

No lab-r1, a sessão morre **sem nenhum comando local** — o Disconnect-Request chegou pela porta 3799. O lab-r2 redisca e volta (o cadastro continua válido). Você acabou de apertar o botão que o sistema de gestão aperta.

Passo 6 — timeout do RADIUS. Desligue o lab-r3 e reconecte o lab-r2: com o secret local desabilitado e o RADIUS mudo, a conexão falha após o `timeout`. Religue o lab-r3 (ou reative o secret local e observe-o atender como fallback).

35.6.2 Verificação

1. Com o secret local desabilitado, a autenticação acontece via RADIUS (`accepts` incrementando no `/radius monitor`);
2. A fila dinâmica da sessão reflete o `rate-limit` vindo do User Manager (50M/25M) — a banda viajou como atributo;
3. O accounting aparece em `/user-manager session print` com bytes subindo a cada `interim-update`;
4. O `session remove` no lab-r3 derrubou a sessão no lab-r1 — CoA/ Disconnect funcionando na porta 3799.

35.6.3 Desafios

1. Durante a migração de secrets locais para RADIUS, o assinante `padaria.paoquente` “não pega o plano novo configurado no sistema” — continua conectando com a banda antiga. Ninguém mexeu no concentrador. Qual é a causa mais provável e o comando que a confirma?
2. O RADIUS caiu numa segunda-feira 9h. O que acontece com (a) as sessões já conectadas e (b) as novas tentativas? Proponha duas mitigações — uma no protocolo (dica: segundo servidor) e uma de emergência local.
3. Seu firewall de input do Capítulo 21 bloqueia tudo que não é explicitamente permitido. Liste as regras exatas (portas/origens) a adicionar no **lab-r1** para o RADIUS + CoA funcionarem — e nada além disso.
4. O financeiro pede: “quando o cliente atrasar, não corta — reduz para 1 Mbps até pagar”. Descreva a implementação com CoA e atributos (sem User Manager na frente, só o mecanismo): que atributo muda, quem dispara, e por que isso é melhor que derrubar a sessão para um profile de suspensão.

Solução dos desafios

1. Sobrou um **secret local ativo** com esse login: a ordem de consulta é local → RADIUS, então o secret antigo (com o profile/banda velha) atende primeiro e o RADIUS nem é consultado. `/ppp secret print where name=padaria.paoquente` confirma; desabilitar/remover o secret resolve e a próxima conexão vem do RADIUS com o plano novo.
2. (a) Sessões conectadas **continuam** — a autorização já foi dada; no máximo o accounting interim para de ser registrado. (b) Novas tentativas falham por timeout (nada de novo conecta — clientes que caírem não voltam!). Mitigações: um **segundo servidor RADIUS** (`/radius add` adicional; o RouterOS tenta na ordem) em outro hardware/POP; e o **fallback local de emergência** — manter secrets locais desabilitados dos clientes críticos + um script/procedimento que os reabilita em pane prolongada (lembrando de desabilitá-los depois, pela pegadinha do desafio 1).
3. No input do lab-r1, além do estabelecido/relacionado que o Capítulo 21 já aceita: liberar **UDP porta 3799** (CoA entrando) com `src-address=172.16.0.3` (só o servidor de gestão). E é só — os Access-Request/Accounting são **originados pelo lab-r1** (portas de destino 1812/1813 no lab-r3), então as respostas voltam casando com `established,related`. Quem precisa liberar 1812/1813 no input é o **lab-r3**, restrito ao IP do lab-r1.
4. O sistema de gestão (na função do nosso lab-r3) envia um **CoA-Request** para a sessão viva com `Mikrotik-Rate-Limit="1M/1M"` (e tipicamente `Mikrotik-Address-List=inadimplentes` para o redirecionamento de página do Capítulo 34); o concentrador reescreve a fila dinâmica no ato. Melhor que derrubar porque: não gera evento de reconexão em massa no vencimento (pense em 500 inadimplentes rediscando juntos), o cliente percebe a degradação imediatamente (e a causa, na página de cobrança), e o restabelecimento pós-pagamento é outro CoA — instantâneo, sem queda.

35.7 Resumo

- **AAA**: autenticação (quem é), autorização (o que pode — viaja como **atributos**: `Framed-IP-Address`, `Mikrotik-Rate-Limit...`), accounting (o que usou — `start/interim/stop`);
- Concentrador = **cliente RADIUS**: `/radius add service=ppp + /ppp aaa set use-radius=yes accounting=yes`; ordem de consulta: **secret local antes do RADIUS** (a pegadinha de migração);
- **User Manager** (pacote do v7) é um servidor RADIUS completo: routers (quem pergunta), limitations/profiles (planos), users (assinantes) — e os sistemas de gestão comerciais fazem o mesmo por baixo;
- **CoA/Disconnect** (UDP 3799, proteja no firewall!): mudar banda e cortar/restabelecer sem derrubar sessão nem tocar no concentrador;
- Com o cadastro central, o concentrador vira **executor substituível** — o que abre caminho para múltiplos concentradores e dimensionamento, assunto do próximo capítulo (Capítulo 36).

Bibliografia do capítulo

- Documentação oficial: seções *RADIUS*, *User Manager* e *PPP AAA* (MikroTik, 2026a);
- AAA e protocolos de autenticação: (Tanenbaum; Feamster; Wetherall, 2021);
- RouterOS v7 na prática: (Discher, 2016);
- Gestão de assinantes em provedores: (Burgess, 2011).

36 Boas práticas de concentrador (BNG)

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Dimensionar um concentrador: o que limita primeiro (CPU de um núcleo, sessões, throughput) e como medir antes de estourar;
- Ajustar **keepalive** e entender o compromisso entre detecção rápida de queda e tempestade de rediscagem;
- Blindar o concentrador: firewall específico do papel, PPPoE só nas interfaces de acesso, proteção contra PADI floods e logins de força bruta;
- Planejar **múltiplos concentradores** (por POP, por serviço) com RADIUS central — e sobreviver à queda de um deles;
- Reconhecer os sintomas de concentrador saturado e o plano de fuga (dividir, não engordar).

Os três capítulos anteriores construíram o serviço: protocolo, estrutura, gestão central. Este fecha o volume com a pergunta de quem já tem centenas de sessões no ar: **como esse concentrador se comporta sob estresse — e como evitar descobrir da pior forma?** O BNG é o equipamento mais crítico do provedor: quando o roteador de borda cai, os clientes ficam sem internet; quando o concentrador cai, eles ficam sem internet e **rediscam todos juntos** quando volta.

36.1 O que limita primeiro

A intuição diz “throughput”. A prática diz outra coisa:

Tabela 36.1: Recursos do concentrador e sintomas de esgotamento

Recurso	O que consome	Sintoma de esgotamento
CPU (um núcleo!)	criptografia, filas, PPPoE em si	latência sobe p/ todos no pico
Sessões	memória (~alguns KB/sessão) + tabelas	conexões novas recusadas
Throughput agregado	soma dos planos × uso simultâneo	filas dinâmicas sempre cheias

Recurso	O que consome	Sintoma de esgotamento
Pool de IPs	1 IP por sessão	“acabou o pool”, cliente não conecta

O detalhe cruel do RouterOS: **o tráfego de uma interface PPPoE é processado majoritariamente em um núcleo de CPU**. Um CCR de 16 núcleos com um único servidor PPPoE gigante pode saturar o núcleo 0 com os outros 15 ociosos. Por isso concentradores se dimensionam olhando `/tool profile` (uso de CPU **por processo**) e `/system resource cpu print` (por núcleo) — nunca só a média total, que esconde o gargalo:

```
/tool profile duration=10
/system resource cpu print
```

Regras de bolso testadas em campo: um núcleo passando de **60–70% no pico** = comece a planejar divisão; filas dinâmicas + criptografia desnecessária (`use-encryption=yes` sem motivo) são os dois maiores ladrões de CPU; e CHR bem provisionado (Capítulo 5) é concentrador perfeitamente viável — muitos provedores rodam BNGs virtualizados.

36.2 Keepalive: detectar a queda sem criar avalanche

O `keepalive-timeout` do servidor define em quantos segundos sem resposta uma sessão é declarada morta. O compromisso:

- **Curto (10 s)**: sessões fantasma somem rápido — mas qualquer microcorte de 10 s no rádio PtMP (11) derruba **todas** as sessões daquele setor, que rediscam em massa;
- **Longo (60 s)**: tolera instabilidades curtas — mas um cliente que desligou o roteador segura IP e sessão por um minuto (irrelevante) e uma queda real demora a aparecer no monitoramento (relevante).

Para acesso via rádio, 30–60 s é mais sensato que os 10 s do capítulo anterior; em fibra, 10–30 s. E lembre-se da **tempestade de rediscagem**: após uma queda de energia no POP, centenas de clientes tentam PADI simultaneamente — o pico de CPU da reconexão em massa é o momento mais perigoso do dia. É para ele que se dimensiona, não para a tarde tranquila. O RouterOS ajuda se o cadastro for RADIUS: a autenticação distribui a carga entre concentrador e servidor.

Analogia para guardar

O concentrador é o **caixa do supermercado**: no fluxo normal, dá conta sorrindo. O que o derruba não é o cliente a mais — é a **reabertura depois do blecaute**, quando a fila inteira chega junto. Keepalive curto demais é o gerente que fecha o caixa ao primeiro cliente que demora a achar o cartão: esvazia rápido, mas obriga todo mundo a refazer a fila. Dimensione o caixa para a reabertura, não para a terça-feira à tarde.

36.3 Blindando o papel

O hardening geral do Capítulo 21 vale todo; o concentrador adiciona riscos próprios:

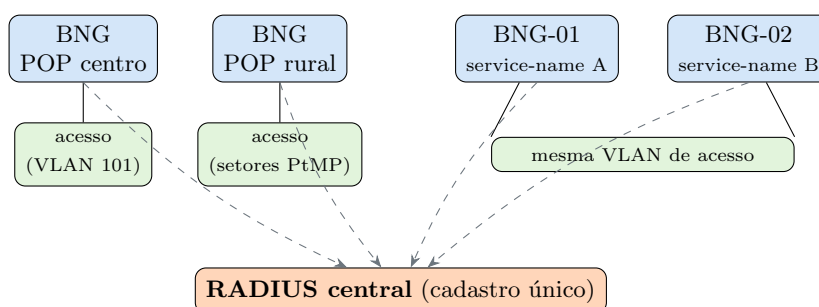
- 1. PPPoE só onde deve existir.** O servidor escuta na(s) interface(s) de acesso — e em mais nenhuma. Confira com `/interface pppoe-server server print`: cada entrada é uma porta aberta em L2 (PPPoE nem precisa de IP para ser atacado — PADI é quadro Ethernet).
- 2. Contra PADI flood.** Um atacante na rede de acesso pode inundar o concentrador de PADIs (descoberta custa CPU). Mitigações em camadas: o próprio TDMA/access-list do rádio (11) já barra desconhecidos; nos switches do POP (7), *storm-control*/limite de broadcast na VLAN de clientes; e no concentrador, monitorar a taxa de tentativas (`/log print where message~"PADI"` em picos anormais).
- 3. Contra força bruta de logins.** Com RADIUS, cada tentativa custa uma consulta — um atacante com dicionário vira carga no AAA inteiro. O accounting entrega o padrão (rajadas de Access-Reject do mesmo CALLER-ID); a resposta é bloquear o MAC na access-list do rádio ou na bridge do POP. O Volume 13 automatiza a detecção.
- 4. O concentrador não é bastião.** Gerência só pela VLAN 99 (Capítulo 26), RADIUS/CoA restritos por firewall ao servidor de gestão (12), e **nenhum** serviço de gerência alcançável pelos IPs das sessões — o assinante fala *através* do BNG, nunca *com* o BNG (exceto DNS/página de cobrança, se você os hospeda ali — melhor não).

36.4 Múltiplos concentradores

Chega o dia em que um só não basta — por capacidade ou por prudência. As duas topologias que escalam:

a) um BNG por POP

b) dois BNGs, mesma rede



- **Por POP** (a divisão natural): cada concentrador atende sua rede de acesso, com seu pool próprio (documente! O roteamento do Volume 8 precisa saber qual /22 sai de onde). A queda de um afeta só seu POP;

Escalando concentradores: (a) um BNG por POP, cada um dono da sua rede de acesso — a divisão natural; (b) dois BNGs na mesma rede, diferenciados por `service-name/ac-name`, para migração ou balanceamento. Em ambas, o RADIUS central mantém o cadastro único — sem ele, nada disso é administrável.

Figura 36.1

- **Mesmo segmento** (migração/balanceamento): dois servidores PPPoE na mesma VLAN respondem PADO; sem controle, os clientes caem em qualquer um (a corrida do desafio 4 do Capítulo 33). O controle é `service-name` no servidor + `service-name/ac-name` no cliente — ou aceitar a loteria como balanceamento rústico (funciona, com pools distintos!);
- **Sobrevivência**: com cadastro no RADIUS (12) e configuração dos BNGs exportada (3), perder um concentrador é evento de **reprovisionar hardware**, não de reconstruir cadastro. Clientes do BNG morto rediscam; se houver outro servidor alcançável com `service-name` compatível, conectam nele — failover de pobre, mas failover.

! O conceito mais importante do capítulo

Concentrador se escala **dividindo, não engordando**. O limite real é o núcleo de CPU que processa o PPPoE — e nenhum upgrade de caixa muda essa aritmética por muito tempo. A arquitetura sustentável é: BNGs médios e substituíveis (um por POP), cadastro central via RADIUS, pools documentados por concentrador e a certeza de que qualquer um deles pode morrer numa sexta à noite sem levar o cadastro junto. Quem segue isso troca “milagre na madrugada” por “manutenção na agenda”.

36.5 Laboratório

i Ambiente

Os três CHRs na configuração final do volume (12): lab-r1 = BNG-01, lab-r3 = User Manager — e agora o **lab-r2 acumula dois papéis**: cliente PPPoE e **BNG-02** (um CHR aceita ser servidor numa interface e cliente noutra; na falta de uma quarta VM, é o nosso segundo concentrador). Snapshot recomendado antes de começar: `pppoe-radius-ok`.

36.5.1 Comandos passo a passo

Passo 1 — medição de base no BNG-01. Com a sessão da maria.silva ativa:

```
/tool profile duration=10
/system resource cpu print
/ppp active print count-only
```

Anote o uso por processo (**ppp**, **queuing**, **networking**) — a fotografia do “normal” contra a qual você comparará picos.

Passo 2 — keepalive na prática. No lab-r1, ajuste para acesso via rádio e provoque um “microcorte” desligando o cabo da lab-a no VirtualBox por ~15 s:

```
/interface pppoe-server server set [find service-name=isp-paoquente] \
    keepalive-timeout=30
```

Com 30 s, a sessão **sobrevive** ao corte de 15 s (confira o **uptime** intacto em **/ppp active print**). Repita com **keepalive-timeout=10**: a sessão morre e redisca. Você acabou de ver o compromisso da seção inteira num parâmetro.

Passo 3 — suba o BNG-02 no lab-r2. Servindo na lab-b (ether3), com service-name próprio e o MESMO RADIUS:

```
/ip pool add name=pool-bng02 ranges=100.64.104.2-100.64.107.254
/ppp profile add name=plano-generico local-address=100.64.104.1 \
    remote-address=pool-bng02 change-tcp-mss=yes only-one=yes
/interface pppoe-server server add interface=ether3 \
    service-name=isp-paoquente-b default-profile=plano-generico \
    authentication=chap max-mtu=1492 max-mru=1492 disabled=no
/radius add service=ppp address=172.16.0.3 secret=SegredoRadius!2026
/ppp aaa set use-radius=yes accounting=yes
```

No lab-r3 (User Manager), autorize o novo cliente RADIUS:

```
/user-manager router add name=bng-pop02 address=<IP-do-lab-r2> \
    shared-secret=SegredoRadius!2026
```

Passo 4 — conecte o lab-r3 como assinante do BNG-02 (o lab-r3 também acumula papéis — servidor RADIUS e cliente de teste na lab-b):

```
/interface pppoe-client add interface=ether2 name=pppoe-teste \
    user=maria.silva password=Ms!7201xk disabled=no
```

A mesma **maria.silva** autentica nos **dois** BNGs — cadastro único, concentradores independentes. (**only-one** aqui é por concentrador; RADIUS pode impor unicidade global — pergunta do desafio 3.)

Passo 5 — mate um BNG. Desligue o servidor PPPoE do lab-r1 (**/interface pppoe-server server disable [find]**) e observe: a sessão no lab-r2-cliente cai e fica rediscando (PADI sem PADO na lab-a — não há segundo BNG **naquele segmento**); enquanto isso, a sessão do BNG-02 na lab-b segue intacta. Falha isolada por segmento = topologia (a) da Figura 36.1. Reative ao final.

36.5.2 Verificação

1. Você tem a fotografia de CPU/processos do BNG-01 sob carga normal;
2. Sessão sobreviveu ao microcorte com `keepalive 30 s` e caiu com 10 s;
3. `maria.silva` conectada simultaneamente nos dois BNGs, ambos aparecendo como `router` distintos no accounting do User Manager;
4. A queda do BNG-01 não afetou a sessão do BNG-02 — e a rediscagem contínua do cliente órfão ficou visível no log dele.

36.5.3 Desafios

1. Seu BNG mostra CPU total em 45%, mas os clientes reclamam de lentidão às 20h30. `/system resource cpu print` mostra: `cpu1: 96%`, demais núcleos < 20%. Explique o que está acontecendo, por que a média engana, e as duas saídas possíveis (curto e longo prazo).
2. Após um blecaute de 40 min no bairro, o POP volta e o concentrador trava por 3 minutos com CPU 100%, derrubando até o WinBox. Nomeie o fenômeno, explique por que o RADIUS **amortece** (em vez de piorar) e cite um parâmetro deste volume que, mal configurado, agrava o quadro.
3. Com dois BNGs e RADIUS central, `only-one=yes` no profile local ainda deixa a `maria.silva` conectar duas vezes (uma em cada BNG), como o Passo 4 provou. De quem é a responsabilidade de impor “uma sessão por contrato” nesse cenário — e com que informação o RADIUS decide?
4. Projete o failover do POP rural: hoje, um BNG único (RB numa torre) atende 3 setores PtMP. Proponha a topologia com um segundo BNG que assuma em caso de falha, listando: onde ele fica, o que compartilham (`service-name?` `pools?`), o que o cliente percebe na falha e o tempo estimado de recuperação.

Solução dos desafios

1. O processamento PPPoE/filas está preso a um núcleo — `cpu1` saturado é o teto real da caixa, e a média de 45% dilui o gargalo nos núcleos ociosos. Curto prazo: aliviar o núcleo — desligar criptografia PPP se estiver ativa, revisar filas/firewall (regras com contadores altos no topo, Capítulo 21), mover serviços acessórios (DNS, gráficos) para fora do BNG. Longo prazo: **dividir** — segundo concentrador por POP/segmento (Figura 36.1), não caixa maior.
2. Tempestade de rediscagem (*reconnect storm*): centenas de PADI/CHAP/IPCP simultâneos + criação de filas dinâmicas = pico de CPU muito acima do regime. O RADIUS amortece porque a autenticação vira requisição-resposta com o servidor externo: o BNG processa em série o que o timeout do RADIUS naturalmente espaça (fila, não colapso) — e rejeições em massa não custam busca em base local. Agrava o quadro: `keepalive-timeout curto` (sessões declaradas mortas rápido demais → mais rediscagem em cascata durante a própria recuperação).
3. Do **RADIUS** (o único que enxerga os dois BNGs). Ele decide com o accounting: se

existe sessão ativa (Accounting-Start sem Stop) para `maria.silva`, o Access-Request seguinte é rejeitado — ou a sessão antiga é derrubada via Disconnect (política “última vence”). No User Manager, o atributo de sessões simultâneas do grupo/perfil aplica isso; a armadilha operacional são sessões fantasma (Stop perdido) travando o cliente — por isso o par `interim-update` + `timeout` de sessão órfã no servidor.

4. Segundo BNG no **mesmo POP/torre** (outro RB, idealmente em fonte/nobreak separados), ligado à mesma VLAN de acesso dos 3 setores. Compartilham: o RADIUS central e o **mesmo service-name** (para o cliente rediscar sem reconfiguração); **pools distintos e roteados** (cada BNG anuncia o seu — Volume 8). Operação normal: BNG-02 com o servidor PPPoE **desabilitado** (standby frio) ou ativo com PADO levemente atrasado. Na falha: clientes caem, PADI acha só o BNG-02, rediscam e voltam — com IP de outro pool (quem tinha IP fixo via `Framed-IP-Address` no RADIUS mantém!). Recuperação = `keepalive` da detecção + rediscagem **1–3 minutos**, sem intervenção humana. Custo: um RB — contra uma madrugada de subida de serra.

36.6 Resumo

- O gargalo real do BNG é **um núcleo de CPU** (PPPoE/filas não paralelizam bem): monitore por núcleo e por processo (`/tool profile`), planeje divisão a partir de 60–70% no pico — e dimensione para a **tempestade de rediscagem**, não para o regime;
- `keepalive-timeout`: 10–30 s em fibra, 30–60 s em rádio — curto demais transforma microcorte em queda de setor + rediscagem em massa;
- Blindagem do papel: PPPoE **só** nas interfaces de acesso, storm control contra PADI flood, RADIUS/CoA cercados por firewall, gerência fora do alcance das sessões;
- Escala = **dividir**: um BNG por POP com pools próprios documentados, **service-name** disciplinando quem conecta onde, RADIUS central mantendo o cadastro único e a unicidade de sessão;
- Fecha o Volume 6: o assinante agora conecta (Capítulo 33), recebe seu plano (Capítulo 34), é gerido centralmente

(12) e o serviço aguenta o mundo real. O Volume 7 pega a fila dinâmica que o profile criou e a transforma em **QoS de verdade**.

Bibliografia do capítulo

- Documentação oficial: seções *PPPoE Server*, *PPP AAA* e *Tool Profile* (MikroTik, 2026a);
- Dimensionamento e arquitetura de acesso: (Tanenbaum; Feamster; Wetherall, 2021);
- RouterOS v7 na prática: (Discher, 2016);
- Operação de provedor com MikroTik: (Burgess, 2011);
- Linhas CCR/RB para BNG: (MikroTik, 2026c).

Parte VII

Volume 7 — QoS e controle de banda

37 Fundamentos de QoS e simple queues

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que uma **fila** faz com pacotes — atrasar, reordenar, descartar — e por que limitar banda é *administrar descarte*;
- Entender por que QoS só funciona **antes** do gargalo (e no sentido de saída);
- Criar **simple queues** com **target** e **max-limit** corretos, e prever a que tráfego cada fila se aplica;
- Explicar por que a **ordem** das simple queues importa e como uma fila “pega-tudo” mal posicionada anula as demais;
- Ler as estatísticas de fila (**rate**, **queued-bytes**, **dropped**) e o gráfico do WinBox como instrumentos de diagnóstico.

O Volume 6 terminou com cada assinante dentro de uma fila dinâmica criada pelo profile PPPoE — e prometeu explicar o que essa fila realmente faz. Este volume paga a promessa. QoS (*Quality of Service*) é o conjunto de técnicas para decidir, quando não há banda para todos, **quem espera e quem passa** — e no provedor isso aparece em dois papéis: entregar o plano vendido (limitar) e proteger o que é sensível (priorizar). Começamos pelo instrumento fundamental: a fila.

37.1 O que uma fila realmente faz

Todo enlace tem uma capacidade. Quando chegam mais pacotes do que o enlace escoa, só existem três destinos possíveis para o excedente:

1. **Esperar** numa fila (ganha latência);
2. Ser **descartado** (o TCP do remetente percebe e desacelera — é assim que a internet se autorregula);
3. Não existe terceira opção. “Guardar para sempre” é só a espera com outro nome — e memória finita.

Limitar banda a 50 Mbps é, portanto, **criar um gargalo artificial administrado**: uma fila que escoa a 50 Mbps, segura rajadas curtas no buffer e descarta o excesso persistente para o TCP do cliente entender o recado. QoS não cria banda — organiza a escassez.

Duas consequências práticas que valem o capítulo:

QoS só funciona no sentido de saída. Uma interface controla o que **transmite**; o que chega já atravessou o enlace — descartar depois de receber não desafoga nada. Por isso a fila do plano atua no tráfego *saindo* em direção ao cliente (download dele) e *saindo* pela WAN (upload dele).

QoS só funciona antes do gargalo real. Se o seu limite é 100 Mbps mas o enlace de rádio à frente entrega 40, quem decide quem espera é a fila **do rádio** (que você não controla), não a sua. A fila administrada precisa ser o gargalo mais estreito do caminho — nem que seja por 1 Mbps.

 Analogia para guardar

A fila de QoS é a **eclusa de um canal**: o rio inteiro não passa de uma vez, e o eclusceiro decide quanto passa por hora (max-limit), quantos barcos aguardam no reservatório (buffer) e o que fazer quando o reservatório enche — recusar barcos (descarte), sinalizando rio acima que parem de mandar (o TCP desacelerando). Construir a eclusa **depois** da corredeira não organiza nada: quem ordenou a passagem foi a corredeira. A eclusa tem que ser o ponto mais estreito.

37.2 Simple queue: o canivete suíço

A **simple queue** é a forma mais direta de QoS no RouterOS: uma regra que casa tráfego por endereço e impõe limites nos dois sentidos:

```
/queue simple
add name=cliente-padaria target=192.168.10.0/24 max-limit=100M/100M \
    comment="LAN da padaria: 100M down / 100M up"
```

Os campos que definem tudo:

- **target** — *quem*: o IP, sub-rede ou **interface** cujo tráfego a fila governa. É o referencial da fila: com **target**, o RouterOS sabe qual sentido é “download” (indo *para* o target) e qual é “upload” (vindo *do* target);
- **max-limit=down/up** — o teto de cada sentido, **na perspectiva do target** (primeiro valor = o que o target baixa). Compare com o **rate-limit** do profile PPPoE (Capítulo 34) — é o mesmo padrão;
- **queue=default/default** — o **tipo** de fila de cada sentido (o algoritmo interno: fifo, sfq, pcq... — os tipos são assunto do Capítulo 40).

A fila dinâmica que o PPPoE cria é exatamente isto: uma simple queue com **target** = IP da sessão e **max-limit** = **rate-limit** do profile. Você já operava simple queues sem saber.

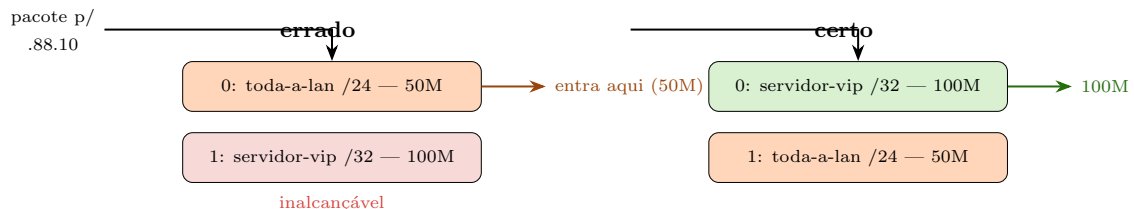
37.3 A ordem importa (muito)

Simple queues são avaliadas **em ordem, de cima para baixo, e o pacote entra na primeira fila cujo target o abranja** — as demais nem o veem. O erro clássico:

```
/queue simple
add name=toda-a-lan target=192.168.88.0/24 max-limit=50M/50M
add name=servidor-vip target=192.168.88.10/32 max-limit=100M/100M
```

O servidor-vip **nunca funciona**: 192.168.88.10 está contido em 192.168.88.0/24, a fila de cima captura o pacote primeiro e o servidor navega a 50M. A regra de projeto é a mesma do firewall (Capítulo 16): **específico em cima, genérico embaixo** — e `print + move` para auditar:

```
/queue simple print
/queue simple move [find name=servidor-vip] destination=0
```



O pacote entra na **primeira** simple queue cujo target o abrange. Fila genérica acima da específica torna a específica letra morta — a mesma lógica de ordem do firewall.

Figura 37.1

37.4 Lendo uma fila como um instrumento

Uma fila conta a história do enlace em três números (`/queue simple print stats`):

Tabela 37.1: Os três instrumentos de uma fila

Indicador	O que significa	Leitura operacional
rate	vazão instantânea (down/up)	encostado no <code>max-limit</code> = fila saturada
queued-bytes/packets	quanto espera no buffer agora	cronicamente alto = latência alta para o cliente

Indicador	O que significa	Leitura operacional
dropped	descartes acumulados	crescendo sob saturação = TCP sendo domado (esperado)

A tríade diagnóstica: **rate** colado no teto + **queued-bytes** alto + **dropped** subindo = **o cliente usa tudo que comprou** (e talvez seja hora de lhe oferecer upgrade — dado comercial saído de `/queue`). Já **rate** baixo com reclamação de lentidão = o problema **não é a fila**: volte ao troubleshooting de enlace (Capítulo 31) ou ao concentrador (Capítulo 36).

! O conceito mais importante do capítulo

Limitar banda é administrar filas e descartes — e só funciona no gargalo, no sentido de saída. Toda técnica deste volume (burst, priorização, PCQ) é variação sobre esse tema: decidir quais pacotes esperam, quais passam e quais morrem, num ponto onde essa decisão ainda importa. Quem entende a fila como instrumento — **rate**, **queued-bytes**, **dropped** — para de “achar” que o cliente usa o plano e passa a **saber**.

37.5 Laboratório

i Ambiente

Dois CHRs bastam: **lab-r1** e **lab-r2** pela rede **lab-a** (Capítulo 5), com conectividade IP simples (ex.: 172.16.1.1 e 172.16.1.2/24 — sem PPPoE desta vez: queremos as filas em primeiro plano). O gerador de tráfego é o **bandwidth-test** do 9 — no **lab-r1**, habilite o servidor: `/tool bandwidth-server set enabled=yes`.

37.5.1 Comandos passo a passo

Passo 1 — meça o “sem QoS”. Do **lab-r2** contra o **lab-r1**:

```
/tool bandwidth-test address=172.16.1.1 protocol=udp direction=receive \
user=vitor password=Lab!Segura2026 duration=15s
```

Anote a taxa — é a capacidade bruta da rede virtual (centenas de Mbps num host razoável). Sem gargalo administrado não há QoS: só a abundância.

Passo 2 — crie o gargalo. No **lab-r1**, uma simple queue governando o **lab-r2**:

```
/queue simple add name=cliente-r2 target=172.16.1.2/32 max-limit=20M/10M
```

Repita o teste do Passo 1: a taxa despenca para ~20M (`direction=receive` = download do ponto de vista do lab-r2). Rode com `direction=transmit` e confirme os 10M do upload. A ordem down/up do `max-limit`, sempre na perspectiva do **target**.

Passo 3 — assista à fila trabalhar. Durante um teste de 60 s, no lab-r1:

```
/queue simple print stats interval=1
```

```
0 name="cliente-r2" target=172.16.1.2/32 rate=19.8Mbps/56kbps
  queued-bytes=48.1KiB/0 dropped=1284/0
```

A tríade completa: `rate` no teto, bytes esperando, descartes crescendo — a fila **é** o gargalo, exatamente como projetado.

Passo 4 — prove que a ordem importa. Adicione uma fila genérica **acima** da específica:

```
/queue simple add name=rede-toda target=172.16.1.0/24 max-limit=5M/5M \
  place-before=0
```

Teste de novo: 5M — a fila `cliente-r2` de 20M virou letra morta (confira: `rate=0` nela, tudo contando na `rede-toda`). Agora corrija sem apagar nada:

```
/queue simple move [find name=cliente-r2] destination=0
```

Teste: 20M de volta. `print stats` mostra a específica trabalhando e a genérica pegando só o que sobrar (outros IPs da sub-rede).

Passo 5 — limpeza consciente. Remova a `rede-toda` e deixe só a `cliente-r2` — o próximo capítulo constrói sobre ela:

```
/queue simple remove [find name=rede-toda]
```

37.5.2 Verificação

1. Sem fila: taxa livre; com fila: `rate` cravado no `max-limit` — nos **dois** sentidos, cada um com seu teto;
2. Durante saturação, `queued-bytes` > 0 e `dropped` crescendo — e ambos esvaziam quando o teste para;
3. Com a genérica acima, a específica marcou `rate=0` (inalcançável); após o `move`, voltou a trabalhar;
4. Você sabe dizer, sem testar, em qual fila cairia um segundo IP (172.16.1.3) em cada um dos dois arranjos do Passo 4.

37.5.3 Desafios

1. Um colega criou `target=172.16.1.0/24 max-limit=100M/100M` “para a rede toda” e reclama que “o QoS não funciona: um cliente sozinho ocupa 95M e os outros ficam sem”. A fila faz exatamente o que foi pedido. Explique a diferença entre **limitar o agregado** e **repartir por cliente** — e qual capítulo deste volume resolve.
2. O download de um assinante obedece ao plano, mas o upload fica **acima** do contratado. A fila dele foi criada num roteador **atrás** do concentrador, por onde só o tráfego de download passa. Por que o upload não obedece, e onde a fila deveria morar?
3. Um cliente de 300 Mbps reclama de “internet travando” em jogos, mas os speedtests dão 300 cravados. `print stats` na fila: `rate=298M`, `queued-bytes=2.1MiB` constante, `dropped` alto. Converta os 2.1 MiB parados em **milissegundos** de espera a 300 Mbps e explique por que o speedtest não vê o problema. (Palavra-chave: *bufferbloat*.)
4. No lab, com a `cliente-r2` em 20M e um teste partindo do próprio lab-r2, a fila funcionou. Se o teste partisse de um **terceiro** roteador (172.16.1.3) contra o lab-r1, o que a `cliente-r2` registraria — e por quê?

Solução dos desafios

1. `max-limit` num target de sub-rede cria **um** balde de 100M que todos dividem por disputa — o TCP mais agressivo (mais conexões paralelas) leva mais. Limitar agregado protege o enlace, não os vizinhos entre si. Repartir por cliente exigiria uma fila por IP — inviável à mão para centenas — ou o **PCQ** (Capítulo 40), que cria sub-filas dinâmicas por endereço dentro de uma única fila.
2. Um roteador só enfileira o que **passa por ele**. No caminho do download ele é o gargalo administrado; o upload do cliente sobe pelo concentrador direto à WAN sem atravessá-lo — a fila nunca vê esses pacotes. Filas de plano moram onde os **dois** sentidos passam: no concentrador (por isso a fila dinâmica do PPPoE vive lá, Capítulo 34).
3. 2,1 MiB = 17,6 Mbit; a 300 Mbps, **~59 ms** de espera permanente — cada pacote do jogo atravessa essa fila cheia: *bufferbloat*. O speedtest mede **vazão** (perfeita: a fila escoar 300M), não **latência sob carga**. O buffer generoso segura bytes em vez de descartar cedo; o TCP não desacelera e a latência mora alta. Caminhos: buffer menor ou tipos de fila que gerenciam atraso — a conversa de tipos do Capítulo 40.
4. `rate=0`: o target 172.16.1.2/32 não abrange 172.16.1.3 — o tráfego do terceiro roteador não casa com a fila e passa **sem limite algum** (não existe mais a genérica). A lição operacional: fila específica sem uma genérica embaixo = todo IP fora da lista navega livre. No concentrador PPPoE isso não ocorre porque **cada sessão** ganha sua fila dinâmica — mais um ponto para o modelo do Volume 6.

37.6 Resumo

- Fila = espera + descarte administrados; limitar banda é criar um **gargalo proposital** onde o TCP sente o teto. QoS não cria banda — organiza a escassez;

- Só funciona **no sentido de saída e antes do gargalo real** — e precisa morar onde os dois sentidos do tráfego passam;
- **Simple queue**: **target** (quem, e o referencial down/up), **max-limit=down/up**, avaliação **em ordem** (primeira-que-casa) — específico acima do genérico, como no firewall;
- A fila dinâmica do PPPoE (Capítulo 34) é uma simple queue criada pelo profile — você já as opera em produção;
- Instrumentação: **rate** (uso real), **queued-bytes** (latência embutida), **dropped** (TCP sendo regulado) — leitura antes de opinião.

O plano de velocidade de verdade tem mais que um teto: tem rajada inicial, garantia mínima e desenho comercial. É o próximo capítulo (Capítulo 38).

Bibliografia do capítulo

- Documentação oficial: seção *Queues* (MikroTik, 2026a);
- Filas, TCP e controle de congestionamento: (Tanenbaum; Feamster; Wetherall, 2021);
- RouterOS v7 na prática: (Discher, 2016);
- QoS em provedores MikroTik: (Burgess, 2011).

38 Burst e planos de velocidade

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar os quatro parâmetros do burst — **burst-limit**, **burst-threshold**, **burst-time** e a **média móvel** que os liga — e calcular quanto tempo uma rajada realmente dura;
- Distinguir **limit-at** (garantia/CIR) de **max-limit** (teto/MIR) e saber quando a garantia é honrada;
- Usar filas **pai e filho** (parent) para vender mais banda do que o enlace tem (*overbooking*) sem quebrar as garantias;
- Desenhar um plano comercial completo (teto + rajada + garantia) e escrevê-lo tanto numa simple queue quanto no **rate-limit** do profile PPPoE;
- Prever o comportamento do burst num speedtest — e explicar ao cliente por que “os primeiros segundos voam”.

O Capítulo 37 entregou o teto: **max-limit** e pronto. Mas o plano que o comercial vende raramente é só um teto — é “**até 80 Mega com turbo de início**”, “garantido 20 no horário de pico”. Esses adereços têm nome técnico e ficam na própria fila: **burst** (a rajada) e **limit-at** (a garantia). Este capítulo transforma a fila do capítulo anterior num produto comercial.

38.1 Burst: velocidade emprestada por tempo limitado

A ideia: navegação típica é feita de **rajadas curtas** — abrir uma página, baixar um e-mail — separadas por silêncio. Se o cliente de 50M pudesse ir a 80M *só nos primeiros segundos*, as páginas abririam visivelmente mais rápido sem custar capacidade no agregado (ninguém sustenta 80M por rajada de 3 s). O burst formaliza o empréstimo:

```
/queue simple
add name=cliente-r2 target=172.16.1.2/32 max-limit=50M/25M \
    burst-limit=80M/40M burst-threshold=40M/20M burst-time=16s/16s
```

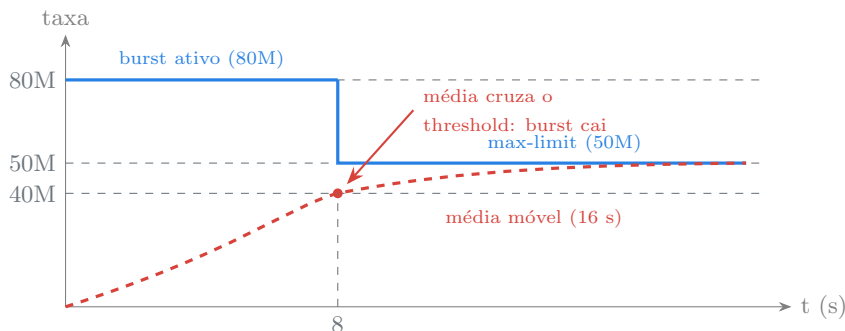
- **burst-limit** — o teto emprestado (80M): vale **enquanto o burst está ativo**;
- **burst-threshold** — o gatilho (40M): o burst fica disponível enquanto o **uso médio recente** estiver *abaixo* dele;

- **burst-time** — a janela da média (16 s): o RouterOS calcula a taxa média dos últimos **burst-time** segundos; é essa média que se compara ao **threshold**.

O mecanismo, passo a passo: cliente ocioso → média recente $0 < \text{threshold}$ → burst **armado**. Começa um download → o cliente voa a 80M → a média da janela sobe... ao cruzar 40M, o burst **desarma** e a fila cai para o **max-limit** de 50M. Parou de baixar → a média escorre para baixo do **threshold** → burst re-armado para a próxima página.

Quanto dura a rajada? Não é **burst-time**! A 80M com janela de 16 s e gatilho de 40M, a média cruza o gatilho quando:

$$t = \frac{\text{burst-threshold}}{\text{burst-limit}} \times \text{burst-time} = \frac{40}{80} \times 16 = 8 \text{ segundos}$$



Burst em ação: a taxa real (azul) voa a 80M enquanto a **média móvel** dos últimos 16 s (vermelha tracejada) fica abaixo do gatilho de 40M — o cruzamento acontece em $t = (40/80) \times 16 = 8$ s, e a fila assenta no **max-limit**.

Figura 38.1

Receita de projeto que funciona: **burst-limit** $1,5\text{--}2 \times$ o **max-limit**; **burst-threshold** 50–75% do **max-limit** (sempre abaixo dele, senão o burst re-arma no meio do uso contínuo!); **burst-time** de 8–32 s lembrando que a duração real é a fração do gatilho. E honestidade comercial: burst é “turbo de abertura de página”, não banda vendável — anuncie o **max-limit**.

38.2 limit-at: a garantia (CIR)

O **max-limit** é o **MIR** (*Maximum Information Rate*) — teto. O **limit-at** é o **CIR** (*Committed Information Rate*) — o piso **garantido**: a banda que a fila recebe *mesmo com o enlace disputado*. A pergunta imediata: garantido **contra quem**? Contra as outras filas — e isso só faz sentido quando todas disputam um teto comum. Entra o **parent**:

```

/queue simple
add name=agregado-pop target=172.16.1.0/24 max-limit=90M/90M \
    comment="teto fisico do enlace do POP"
add name=cliente-a parent=agregado-pop target=172.16.1.2/32 \
    limit-at=20M/10M max-limit=50M/25M
add name=cliente-b parent=agregado-pop target=172.16.1.3/32 \
    limit-at=20M/10M max-limit=50M/25M

```

A fila-pai representa o **enlace real** (os 90M do rádio, do transporte); as filhas, os clientes. O algoritmo de partilha (HTB, o motor por trás de todas as filas do RouterOS) então promete:

1. Cada filha recebe **até o limit-at** antes de qualquer filha ganhar excedente — a garantia;
2. O que sobrar do pai é distribuído entre as filhas que querem mais, até seus **max-limit** (e nunca além do pai);
3. Soma dos **limit-at** **max-limit do pai**: garantia que não cabe no enlace é promessa de político.

É assim que se faz *overbooking* honesto: num enlace de 90M, dez clientes de **max-limit=50M cabem** — desde que a soma dos **limit-at** ($10 \times 5M = 50M$, por exemplo) caiba no pai. Todos têm teto alto para rajadas e piso garantido para o pico.

💡 Analogia para guardar

O plano de velocidade é um **buffet de churrascaria**: o **max-limit** é “coma à vontade” (teto físico do prato), o **limit-at** é a reserva — seu lugar à mesa com um mínimo de picanha guardado mesmo no salão cheio; e o **burst** é o garçom liberando a primeira rodada em dobro para quem acabou de sentar — sabendo que ninguém janta no ritmo da primeira rodada. O salão (fila-pai) tem capacidade fixa: o dono pode vender mais “à vontade” do que picanha existe, mas as **reservas** somadas nunca podem passar do estoque.

❗ O conceito mais importante do capítulo

Um plano de velocidade completo é um contrato em três números: **max-limit** (o que anuncio), **limit-at** (o que garanto no pico) e **burst** (a cortesia de abertura). E o contrato só é honesto dentro de uma **hierarquia**: a fila-pai encarna o enlace físico, e a regra **limit-at** pai é a diferença matemática entre *overbooking* sustentável e venda de banda fantasma. Quem projeta planos sem fila-pai está prometendo garantias que nenhum algoritmo consegue cumprir.

38.3 Levando para o PPPoE

No concentrador (Capítulo 34), tudo isso viaja **no rate-limit do profile**, cuja sintaxe completa é:

```
rate-limit="tx/rx burst-tx/burst-rx threshold-tx/threshold-rx time-tx/time-rx
prioridade limit-at-tx/limit-at-rx"
```

(perspectiva do concentrador: tx = download do cliente). O plano 50M com turbo e garantia de 20M:

```
/ppp profile set plano-50m \
    rate-limit="50M/25M 80M/40M 40M/20M 16s/16s 8 20M/10M"
```

Cada sessão nasce com a fila dinâmica completa — burst, garantia e tudo. Via RADIUS (12), a mesma string viaja no atributo **Mikrotik-Rate-Limit**: o sistema de gestão descreve o plano inteiro numa linha.

38.4 Laboratório

i Ambiente

O mesmo do capítulo anterior (Capítulo 37): lab-r1 + lab-r2 na lab-a, **bandwidth-server** habilitado no lab-r1 e a fila **cliente-r2** (20M/10M) ainda de pé. Um terceiro CHR (lab-r3) na mesma rede (172.16.1.3) entra no Passo 4 para a disputa de garantias.

38.4.1 Comandos passo a passo

Passo 1 — adicione o burst à fila existente. No lab-r1:

```
/queue simple set cliente-r2 burst-limit=40M/20M \
    burst-threshold=15M/7M burst-time=16s/16s
```

Conta antes de medir: gatilho 15M, rajada 40M, janela 16 s → duração esperada $\approx (15/40) \times 16 = 6$ s.

Passo 2 — veja a rajada. No lab-r2, um teste de 30 s (**direction=receive**), acompanhando no lab-r1:

```
/queue simple print stats interval=1
```

Os primeiros ~6 s marcam **rate** 40M; depois, queda seca para 20M — a Figura 38.1 ao vivo. Espere ~20 s de silêncio (a média precisa escorrer) e repita: o burst re-armou.

Passo 3 — burst mal projetado. Suba o gatilho para cima do max-limit (**burst-threshold=25M/12M**) e rode um teste **longo**: a taxa oscila — vai a 40M, cai, re-armar no meio do uso contínuo (a média em 20M fica abaixo do gatilho de 25M!), sobe de novo. Serrote. Por isso o gatilho fica **abaixo** do max-limit. Restaure 15M/7M.

Passo 4 — hierarquia e garantia. Monte o pai e duas filhas com garantias distintas (lab-r3 já com IP 172.16.1.3 e testes prontos):

```
/queue simple remove [find name=cliente-r2]
/queue simple
add name=enlace-pop target=172.16.1.0/24 max-limit=30M/30M
add name=cliente-a parent=enlace-pop target=172.16.1.2/32 \
    limit-at=20M/5M max-limit=30M/10M
add name=cliente-b parent=enlace-pop target=172.16.1.3/32 \
    limit-at=5M/5M max-limit=30M/10M
```

Passo 5 — a disputa. Rode **direction=receive** **simultaneamente** no lab-r2 e no lab-r3 (duas janelas). Com 30M no pai e demandas de 30M cada: **cliente-a** estabiliza 22–24M, **cliente-b** 6–8M — cada um recebeu seu **limit-at** primeiro (20M vs 5M) e a sobra (~5M) foi repartida. Pare o teste do lab-r2: o lab-r3 sobe na hora para os 30M do teto — a garantia limita a **disputa**, não a fatura.

38.4.2 Verificação

1. A duração medida da rajada bate com a conta **threshold/limit × time** (± 1 s);
2. Com gatilho acima do max-limit, você viu o “serrote” de re-arme em uso contínuo;
3. Na disputa, cada filha recebeu seu **limit-at**, a soma respeitou o pai, e a folga foi realocada quando um parou;
4. Você sabe escrever o plano completo do Passo 4 como string **rate-limit** de profile PPPoE.

38.4.3 Desafios

1. Plano “100M com turbo de 200M”: o comercial quer que o turbo dure **15 segundos**. O max-limit é 100M e você escolheu **threshold = 75M**. Calcule o **burst-time** necessário — e aponte o efeito colateral de janelas tão longas no re-arme (quanto tempo de ocioso até o próximo turbo?).
2. Num pai de 100M, o júnior configurou 8 filhas com **limit-at=20M** cada (soma: 160M). Nada “deu erro”. O que acontece de fato no pico quando todas demandam 20M+ — e por que essa falha é pior do que se o RouterOS recusasse a configuração?

3. Speedtests do cliente do plano 50M (com o burst deste capítulo) dão sempre ~77M nos servidores próximos e 50M nos distantes. O suporte acha que “tem servidor mentindo”. Explique com a mecânica do burst (duração da rajada $\times$ duração do teste) qual medição reflete o plano.
4. O **rate-limit** de profile aceita um campo de **prioridade** (o 8 do exemplo). Investigue na documentação/no lab: prioridade de HTB compara **o quê** com **quem**, e por que ela só tem efeito entre filas que disputam o mesmo pai?

Solução dos desafios

1. $t = (\text{thr}/\text{limit}) \times \text{time} \Rightarrow 15 = (75/200) \times \text{time} \Rightarrow \text{time} = 40 \text{ s}$. Efeito colateral: a média móvel de 40 s também demora ~40 s de ociosidade (ou proporcionalmente mais de uso leve) para escorrer de volta abaixo de 75M — o cliente que encadeia downloads curtos verá o turbo “sumir” nas repetições. Janela longa = turbo raro e demorado para re-armar; é o preço do turbo longo.
2. O HTB entrega os **limit-at na ordem/proporção possível**: com 160M prometidos num pai de 100M, as garantias viram rateio (~12,5M cada em vez de 20M) — nenhuma filha recebe o prometido no pico, exatamente quando a garantia importa. É pior que um erro de configuração porque **falha silenciosamente**: passa em todo teste fora do pico (onde sobra banda) e quebra o SLA no único momento em que ele vale. A validação **limit-at** pai é responsabilidade do projetista.
3. Rajada deste plano: $(40/80) \times 16 = 8 \text{ s}$ a 80M. Speedtest curto (~10–15 s, típico com servidor próximo/rápido): a maior parte da medição acontece **dentro** da rajada $\rightarrow$ média 75–80M. Teste mais longo/lento (servidor distante): a rajada pesa pouco na média $\rightarrow$ ~50M. O plano vendido é o max-limit: a medição “distante” é a fiel. (E os speedtests conhecidos usam ~15 s — clientes com burst curto veem números acima do plano; explique antes que virem chamado.)
4. A prioridade (1 = mais alta, 8 = padrão/mais baixa) desempata a distribuição do **excedente acima dos limit-at** entre filhas do **mesmo pai**: quem tem prioridade melhor enche seu max-limit primeiro. Entre filas sem pai comum não há recurso disputado a desempatar — cada uma corre contra seu próprio teto — e a prioridade é inócua. (É também por isso que priorizar *tipos de tráfego* exigirá a queue tree do Capítulo 39, onde a hierarquia é desenhada para isso.)

38.5 Resumo

- **Burst** = velocidade emprestada controlada por média móvel: **burst-limit** (teto emprestado), **burst-threshold** (gatilho, **sempre abaixo do max-limit**), **burst-time** (janela da média); duração real $\text{threshold}/\text{limit} \times \text{time}$;
- **limit-at** (CIR) é o piso garantido; **max-limit** (MIR) o teto — e a garantia só existe dentro de uma hierarquia com **fila-pai** encarnando o enlace físico;
- Regra de ouro do overbooking: **limit-at** **max-limit do pai** — tetos podem somar mais que o enlace; garantias, jamais;

- No PPPoE, o plano inteiro vira a string `rate-limit` do profile ("`tx/rx burst threshold time prioridade limit-at`") ou o atributo `Mikrotik-Rate-Limit` via RADIUS;
- Speedtest curto mede o burst; teste longo mede o plano — saiba qual número o cliente vai ver antes dele ligar.

Falta a terceira dimensão do QoS: **priorizar por tipo de tráfego** (VoIP antes de torrent) — mangle marcando e queue tree executando, no próximo capítulo (Capítulo [39](#)).

Bibliografia do capítulo

- Documentação oficial: seções *Queues* — *Burst* e *HTB* (MikroTik, 2026a);
- Traffic shaping, CIR/MIR e HTB: (Tanenbaum; Feamster; Wetherall, 2021);
- RouterOS v7 na prática: (Discher, 2016);
- Desenho de planos em provedores: (Burgess, 2011).

39 Mangle e queue tree: priorizando por tipo de tráfego

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Marcar tráfego com **mangle** usando o padrão de dois passos (connection-mark → packet-mark) sem desperdiçar CPU;
- Explicar o que a **queue tree** faz que a simple queue não faz — e o que ela *não* faz (limitar por cliente nos dois sentidos numa regra só);
- Construir uma árvore HTB com classes por tipo de tráfego (VoIP, jogos/interativo, navegação, resto) usando **priority** e **limit-at**;
- Escolher o ponto de pendura certo (**parent=interface** vs **global**) e entender onde cada um enxerga o tráfego;
- Combinar as duas camadas do provedor: plano por assinante (simple queue/PP-PoE) + prioridade por tipo (queue tree no agregado).

Os capítulos anteriores trataram todos os pacotes de um cliente como iguais: 50M é 50M, seja chamada de voz ou torrent. Mas quando a fila enche, **a ordem de saída importa**: o pacote de VoIP que espera 60 ms atrás de um torrent vira chiado; o torrent que espera 60 ms não vira nada. Priorizar exige duas ferramentas trabalhando juntas: o **mangle** (4) para *classificar* — “este pacote é VoIP” — e a **queue tree** para *executar* — “VoIP sai primeiro”.

39.1 Marcar barato: o padrão connection-mark → packet-mark

O mangle marca pacotes, mas inspecionar **cada pacote** contra listas de portas/endereços custa CPU — e o concentrador já vive de CPU contada (Capítulo 36). O padrão eficiente usa o connection tracking (16) a favor: classifique **a conexão uma vez**, e herde a marca em todos os pacotes dela:

```
/ip firewall mangle
# 1) classificar a CONEXAO (só pacotes novos chegam aqui)
add chain=prerouting connection-state=new protocol=udp dst-port=5060,10000-20000 \
    action=mark-connection new-connection-mark=conn-voip passthrough=yes \
    comment="SIP + RTP tipico"
```

```
add chain=prerouting connection-state=new protocol=udp dst-port=53 \
    action=mark-connection new-connection-mark=conn-interativo passthrough=yes \
    comment="DNS"
add chain=prerouting connection-state=new protocol=tcp dst-port=80,443 \
    connection-bytes=0-2000000 action=mark-connection \
    new-connection-mark=conn-navegacao passthrough=yes \
    comment="web ate 2 MB (paginas)"

# 2) converter marca de conexao em marca de PACOTE (barato: so le a marca)
add chain=prerouting connection-mark=conn-voip action=mark-packet \
    new-packet-mark=pkt-voip passthrough=no
add chain=prerouting connection-mark=conn-interativo action=mark-packet \
    new-packet-mark=pkt-interativo passthrough=no
add chain=prerouting connection-mark=conn-navegacao action=mark-packet \
    new-packet-mark=pkt-navegacao passthrough=no
add chain=prerouting action=mark-packet new-packet-mark=pkt-resto \
    passthrough=no comment="tudo que nao casou acima"
```

Três detalhes que separam isso de uma configuração ingênua:

- **connection-state=new** nas regras de classificação: só o primeiro pacote da conexão paga a inspeção de portas; os milhões seguintes casam direto pela **connection-mark** (uma leitura de memória);
- **passthrough=yes** na marcação de conexão (o pacote continua para as regras seguintes) e **passthrough=no** na de pacote (marcou, saiu — não desce o resto da lista à toa);
- **connection-bytes=0-2000000** na navegação: os primeiros 2 MB de uma conexão web são “página abrindo” (prioridade); além disso é download grande — a conexão *deixa* de casar e cai para **pkt-resto**. Classificação por **comportamento**, não só por porta.

Analogia para guardar

O mangle com dois passos é a **triagem do pronto-socorro**: o enfermeiro avalia o paciente **uma vez** na chegada e prende a pulseira colorida (**connection-mark**); dali em diante, ninguém reavalia — todo atendente decide pela cor da pulseira (**packet-mark**). A queue tree é a sala de espera que **chama pela cor**: vermelho passa na frente, verde espera. Sem triagem, cada balcão teria que refazer o diagnóstico; sem a chamada por cor, a triagem seria enfeite.

39.2 Queue tree: o HTB exposto

A **queue tree** é o motor HTB sem o açúcar da simple queue: você pendura classes numa interface (ou num ponto global) e cada classe casa pacotes **pela packet-mark**. O que muda em relação à simple queue:

Tabela 39.1: Simple queue × queue tree

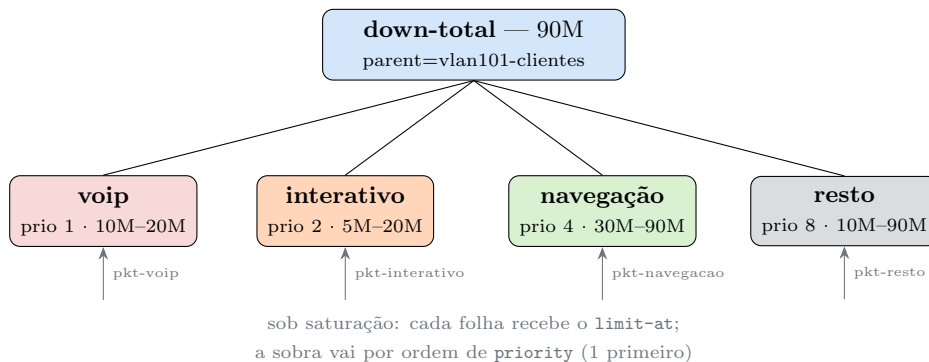
	Simple queue	Queue tree
Casa tráfego por Sentidos	target (IP/interface) os dois numa regra	packet-mark (mangle) um por árvore (a da interface de saída)
Hierarquia Prioridade	parent opcional entre filhas do mesmo pai	é a essência (HTB puro) idem — e é o motivo de usá-la
Ordem de avaliação	primeira-que-casa	todas as classes-folha que casarem a marca

A árvore para o **sentido download** (saindo pela interface dos clientes) do nosso POP:

```
/queue tree
add name=down-total parent=vlan101-clientes max-limit=90M \
    comment="teto fisico do enlace de download"
add name=down-voip parent=down-total packet-mark=pkt-voip \
    priority=1 limit-at=10M max-limit=20M
add name=down-interativo parent=down-total packet-mark=pkt-interativo \
    priority=2 limit-at=5M max-limit=20M
add name=down-navegacao parent=down-total packet-mark=pkt-navegacao \
    priority=4 limit-at=30M max-limit=90M
add name=down-resto parent=down-total packet-mark=pkt-resto \
    priority=8 limit-at=10M max-limit=90M
```

E a espelho para upload (**parent=ether1**, a WAN — cada sentido tem sua árvore, pendurada na interface por onde **sai**). A leitura do HTB:

- Com folga no **down-total**, todo mundo flui — prioridade é invisível na abundância;
- Sob saturação, cada classe recebe seu **limit-at** primeiro (garantias, como no Capítulo 38); a sobra é distribuída **por ordem de priority** (1 = primeiro): VoIP enche até 20M antes de a navegação ver um bit da sobra;
- **pkt-resto** com **priority=8** não morre de fome: o **limit-at=10M** é o seu piso — prioridade decide a *sobra*, garantia decide o *mínimo*.



A árvore de download: a raiz encarna o enlace (90M) e as folhas casam as packet-marks do mangle. Garantia (`limit-at`) define o mínimo de cada classe; `priority` define quem fica com a sobra — VoIP primeiro, resto por último.

Figura 39.1

39.2.1 Onde pendurar: interface ou global

`parent=<interface>` enfileira o que **sai** por ela — é o normal e o que o hardware agradece. `parent=global` captura em um ponto único todo tráfego (antes da decisão de interface), útil quando o mesmo tráfego pode sair por várias interfaces — ao custo de enfileirar também o que não precisaria. Regra prática: comece pela interface; vá a global só com motivo (e o Capítulo 40 traz um).

! O conceito mais importante do capítulo

Prioridade é **quem sofre menos quando falta** — não “quem ganha mais banda”. A arquitetura completa tem papéis separados: o **mangle classifica** (uma vez por conexão, barato), a **queue tree executa** (garantia + prioridade sob o teto físico), e o **plano por assinante continua nas filas do PPPoE** — as duas camadas convivem: a simple queue do plano decide *quanto* cada cliente pode usar; a queue tree do agregado decide *o que sai primeiro* quando o enlace do POP satura. Priorizar sem classificar é chute; classificar sem hierarquia é enfeite.

39.3 Laboratório

i Ambiente

`lab-r1` (nosso roteador de QoS) entre `lab-r2` e `lab-r3` (Capítulo 5): `lab-r2` na `lab-a` (172.16.1.2) e `lab-r3` na `lab-b` (ether3 do `lab-r1`, ex.: 172.16.2.3/24) — assim o tráfego `r2 r3` **atravessa** o `lab-r1`, que enfileira na interface de saída. `bandwidth-server`

habilitado em r2 e r3. Vamos gerar duas classes de tráfego: UDP “VoIP” (porta destino no range RTP) e TCP “resto”, e ver a prioridade decidir sob saturação.

39.3.1 Comandos passo a passo

Passo 1 — mangle no lab-r1. Versão reduzida do padrão de dois passos (só voip e resto):

```
/ip firewall mangle
add chain=prerouting connection-state=new protocol=udp \
    dst-port=10000-20000 action=mark-connection \
    new-connection-mark=conn-voip passthrough=yes
add chain=prerouting connection-mark=conn-voip action=mark-packet \
    new-packet-mark=pkt-voip passthrough=no
add chain=prerouting action=mark-packet new-packet-mark=pkt-resto \
    passthrough=no
```

Passo 2 — a árvore, com gargalo apertado de propósito (10M para a saturação ser fácil), na interface para a lab-b:

```
/queue tree
add name=down-total parent=ether3 max-limit=10M
add name=down-voip parent=down-total packet-mark=pkt-voip \
    priority=1 limit-at=2M max-limit=10M
add name=down-resto parent=down-total packet-mark=pkt-resto \
    priority=8 limit-at=1M max-limit=10M
```

Passo 3 — sature com “resto”. Do lab-r3, um teste TCP longo contra o lab-r2 (direction=receive duration=120s) — o download atravessa o lab-r1 e satura os 10M. Confirme no lab-r1:

```
/queue tree print stats
```

down-resto com rate 9.9M, queued-packets alto; down-voip zerada.

Passo 4 — o VoIP fura a fila. Com o teste TCP ainda rodando, dispare do lab-r3 um segundo teste, agora UDP na faixa marcada (simula o fluxo RTP):

```
/tool bandwidth-test address=172.16.1.2 protocol=udp direction=receive \
    local-udp-tx-size=200 remote-udp-tx-size=200 \
    user=vitor password=Lab!Segura2026
```

No `print stats: down-voip` sobe na hora (o UDP flui), e `down-resto` **cede** exatamente o que o VoIP pediu — a soma cravada nos 10M do pai. Pare o UDP: o resto reocupa em segundos.

Passo 5 — prove que era a prioridade. Inverta (`set down-voip priority=8` e `set down-resto priority=1`) e repita o Passo 4: agora é o “VoIP” que espera na fila (rate baixo, drops) — o tráfego é o mesmo; a política mudou. Restaure ao final.

39.3.2 Verificação

1. `/queue tree print stats` mostra os contadores subindo nas classes certas (marcas do mangle funcionando — se tudo cai em `pkt-resto`, revise portas/ordem das regras);
2. Sob saturação, a chegada do tráfego prioritário **desloca** o resto sem estourar o teto do pai;
3. Com prioridades invertidas, o comportamento inverte — mesma carga, política oposta;
4. `queued-packets` alto no resto e baixo no voip durante a disputa — a latência foi para quem aguenta.

39.3.3 Desafios

1. As regras de classificação do Passo 1 usam `connection-state=new`, mas a regra de `mark-packet` roda para **todo** pacote. Um colega propõe “economizar” marcando pacote também só em `new`. O que quebraria?
2. Um provedor marcou VoIP por `dst-port=5060` apenas (SIP) e reclama que “a priorização não funciona: as chamadas continuam picotando sob carga”. As chamadas *sinalizam* mas o áudio pica. O que ele esqueceu de classificar, e por que a chamada “completa” mesmo assim?
3. Na árvore do POP (`down-total=90M`), a classe `down-resto` tem `limit-at=10M`. Durante um ataque de download em massa (resto saturado), usuários reclamam que “até o DNS demora”. O `down-interativo` tem `limit-at=5M priority=2`. Use a mecânica HTB para explicar por que o DNS **não deveria** estar sofrendo — e o que investigar se sofre (dica: a marca está certa?).
4. Queue tree numa interface **bridge** (o cenário do CPE do cliente,
 - 13) tem uma pegadinha famosa: o tráfego encaminhado em L2 com hardware offload nem passa pelo CPU. O que é preciso aceitar (ou perder) para enfileirar ali — e por que no concentrador PPPoE esse problema não existe?

Solução dos desafios

1. A marca de **pacote** não é persistente — cada pacote nasce sem ela e precisa ser marcado individualmente (é a marca de **conexão** que persiste no conntrack). Marcando pacote só em `new`, apenas o primeiro pacote de cada conexão entraria na classe certa; todos os demais cairiam em `pkt-resto` — a árvore veria 99,9% do tráfego como resto. A economia real já está feita: a regra de pacote casa por `connection-mark` (leitura

barata), não por portas.

2. Esqueceu o **RTP** — o áudio. SIP (5060) é só a sinalização: convite, toque, desligar; a mídia viaja em UDP de portas altas negociadas na própria sinalização (tipicamente 10000–20000, ou a faixa que o PABX/operadora usar). A chamada completa porque a sinalização está priorizada e é levíssima; o áudio pica porque compete como **pkt-resto**. Classificar VoIP = SIP e a faixa RTP (como no texto).

3. Pela mecânica, impossível o interativo passar fome: seu **limit-at=5M** é servido **antes** de qualquer sobra ir ao resto, e DNS mal usa kilobits. Se o DNS sofre, o mais provável é que os pacotes **não estão na classe down-interativo**: consultas indo a um resolver externo por TCP/853 (DoT) ou 443 (DoH) não casam com **udp/53** — e caem no resto saturado. **print stats** (classe interativa zerada?) + **/ip firewall mangle print stats** confirmam. Moral: priorização erra silenciosamente quando a **classificação** envelhece.

4. Para a queue tree ver o tráfego da bridge, ele precisa subir ao CPU: **use-ip-firewall=yes** nas configurações da bridge (e aceitar a perda do **hardware offload** — todo o encaminhamento vira software, 13). O custo pode ser proibitivo num switch. No concentrador PPPoE o problema não existe porque o tráfego de cada assinante **já** termina túneis no CPU (é roteado, não switchado) — mais uma razão estrutural para o QoS de provedor morar no BNG.

39.4 Resumo

- **Mangle classifica, queue tree executa**: o padrão de dois passos (mark-connection em **new** → mark-packet por connection-mark) marca com custo mínimo de CPU; **passthrough** calibrado evita passeios inúteis pela lista;
- **Queue tree** = HTB puro na interface de **saída** (uma árvore por sentido), casando pacotes por packet-mark; sob o teto do pai, cada classe recebe seu **limit-at** e a sobra segue a **priority** (1 primeiro);
- Prioridade decide **quem espera quando falta** — garantias decidem o mínimo de cada classe; resto com garantia não morre de fome;
- Classificação envelhece (portas mudam, DoH existe, RTP não é SIP): audite **print stats** — classe zerada é sintoma de marca errada;
- As duas camadas do provedor convivem: fila por assinante no concentrador (plano) + árvore por tipo no agregado (prioridade).

Falta escalar: uma fila **por cliente**, para milhares de clientes, sem criar milhares de filas à mão — o **PCQ**, no próximo e último capítulo do volume (Capítulo 40).

Bibliografia do capítulo

- Documentação oficial: seções *Mangle*, *Queue Trees* e *HTB* (MikroTik, 2026a);
- Disciplinas de fila e priorização: (Tanenbaum; Feamster; Wetherall, 2021);

- RouterOS v7 na prática: (Discher, 2016);
- QoS em provedores MikroTik: (Burgess, 2011).

40 PCQ: controle de banda por assinante em escala

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Situar os **tipos de fila** (/queue type): fifo, sfq, red — e onde o PCQ se encaixa;
- Explicar o mecanismo do **PCQ**: sub-filas dinâmicas por classificador (pcq-classifier) com o mesmo pcq-rate para cada uma;
- Distinguir os dois usos do PCQ: **repartir igualmente** um enlace (pcq-rate=0) e **plano único em massa** (pcq-rate=NM);
- Montar o par clássico download/upload (classificador dst-address / src-address) em queue tree ou simple queue;
- Decidir com critério: **PPPoE + filas dinâmicas ou PCQ?** — e onde cada modelo vence no provedor real.

O Capítulo 39 fechou com uma promessa: uma fila por cliente, para milhares de clientes, sem criar milhares de filas. O problema é real — no hotspot da praça, na rede do condomínio atendido por IP fixo, no setor PtMP legado sem PPPoE, você tem *uma sub-rede inteira* de usuários e nenhuma sessão individual para pendurar fila dinâmica. A resposta do RouterOS é um **tipo de fila** especial: o **PCQ** (*Per Connection Queue* — nome historicamente infeliz: ele enfileira por *classificador*, quase sempre por endereço, não por conexão).

40.1 Tipos de fila: o algoritmo dentro da fila

Toda fila deste volume tem um campo que ignoramos até agora: o **tipo** (queue=default nas simple queues, queue=default nas classes da tree). O tipo é o algoritmo que decide a **ordem interna** dos pacotes que esperam:

Tabela 40.1: Tipos de fila no RouterOS

Tipo	Mecanismo	Uso típico
pfifo/bfifo	primeiro a chegar, primeiro a sair (por pacotes/bytes)	o default; simples e previsível
sfq	embaralha fluxos para nenhum monopolizar	mitigar um “porco” dentro da fila

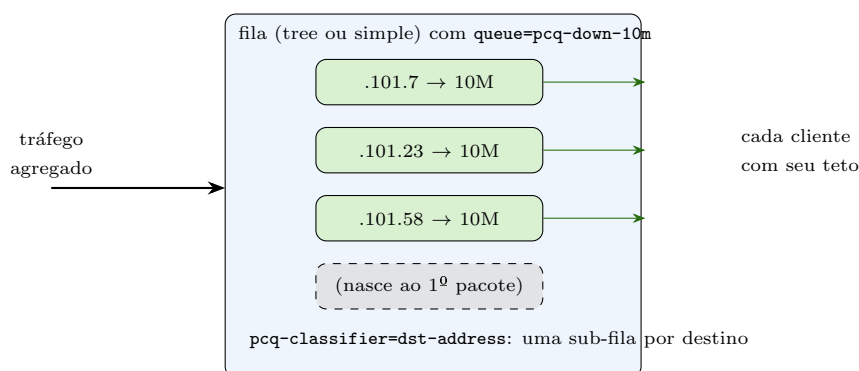
Tipo	Mecanismo	Uso típico
red	descarta probabilisticamente antes de encher	domar TCP sem estourar buffer (anti-bufferbloat)
pcq	uma sub-fila por valor do classificador	banda por assinante em escala

O PCQ é o único que multiplica: enquanto os demais organizam *uma* fila, ele cria **uma sub-fila FIFO por valor distinto do classificador** — e aplica a *cada* sub-fila o mesmo teto.

40.2 O mecanismo

```
/queue type
add name=pcq-down-10m kind=pcq pcq-classifier=dst-address pcq-rate=10M \
    pcq-limit=50KiB pcq-total-limit=2000KiB
add name=pcq-up-5m kind=pcq pcq-classifier=src-address pcq-rate=5M \
    pcq-limit=50KiB pcq-total-limit=2000KiB
```

- **pcq-classifier** — o que define “um usuário”. No sentido download, cada **destino** é um cliente (**dst-address**); no upload, cada **origem** (**src-address**). Errar esse par é o bug nº 1 do PCQ;
- **pcq-rate** — o teto **de cada sub-fila**: 10M *por cliente*, não no total;
- **pcq-limit** — o buffer de cada sub-fila (50 KiB latência controlada; lembre do *bufferbloat* do Capítulo 37);
- **pcq-total-limit** — a memória de todas as sub-filas somadas.



Uma fila PCQ por dentro: o classificador reparte o agregado em sub-filas dinâmicas — uma por endereço — e aplica o **pcq-rate** a cada uma. Milhares de “filas por cliente” pelo preço de configurar uma.

Figura 40.1

O tipo sozinho não faz nada: ele precisa ser **o algoritmo de uma fila** que capture o agregado. Em queue tree (com marcas do Capítulo 39 ou marcando tudo):

```
/queue tree
add name=down-clientes parent=vlan101-clientes packet-mark=pkt-clientes \
    queue=pcq-down-10m max-limit=90M
add name=up-clientes parent=ether1 packet-mark=pkt-clientes \
    queue=pcq-up-5m max-limit=90M
```

Ou, sem mangle, numa simple queue da sub-rede inteira:

```
/queue simple
add name=bairro-fibra target=100.64.101.0/24 max-limit=90M/90M \
    queue=pcq-down-10m/pcq-up-5m
```

Uma regra. Cada morador do /24 com 10M de download e 5M de upload, sub-filas nascendo e morrendo sozinhas, e o teto do enlace respeitado pelo **max-limit** externo.

40.2.1 Os dois modos do PCQ

pcq-rate=10M — *plano único em massa*: todo cliente do agregado tem o mesmo teto. Perfeito para hotspot, condomínio, “plano social” de uma faixa;

pcq-rate=0 — *repartição justa*: sem teto individual; as sub-filas ativas **dividem igualmente** o que o **max-limit** externo der. 90M com 3 ativos = 30M cada; chegou o 4º, 22,5M cada. É o “ninguém monopoliza” instantâneo para enlaces compartilhados — o backhaul de rádio do Volume 5 agradece.

Analogia para guardar

O PCQ é o **rodízio da churrascaria** (de novo ela — mas agora o salão): em vez de contratar um garçom por mesa (uma fila por cliente, o modelo PPPoE), um garçom só percorre todas as mesas servindo **a mesma cota** a cada uma (**pcq-rate**) — e mesas novas entram no rodízio automaticamente, sem contratar ninguém. Com **pcq-rate=0**, o rodízio vira “divide o que tiver”: a picanha que há é repartida por igual entre as mesas ocupadas, sejam 3 ou 30.

40.3 PCQ ou PPPoE? O critério

Os dois resolvem “banda por assinante”. A escolha é arquitetural:

Tabela 40.2: PPPoE × PCQ para banda por assinante

Critério	PPPoE + fila dinâmica	PCQ
Planos diferentes por cliente	natural (profile/RADIUS)	1 fila PCQ por plano + marcação por address-list
Identidade/corte/accounting	nativo (Volume 6)	não é papel dele (só banda)
Burst/garantia por cliente	string <code>rate-limit</code> completa	<code>pcq-burst-*</code> (global à fila)
Custo por assinante	sessão + fila (CPU/RAM)	sub-fila (leve)
Sem sessão possível (hotspot, IP fixo, trânsito)	inviável	é o caso dele

No provedor da nossa narrativa: assinantes de fibra e rádio conectam por PPPoE (identidade + plano + corte — o pacote completo do Volume 6); o PCQ entra nas beiradas — o WiFi da praça, o condomínio B2B com faixa fixa, a repartição justa de um backhaul — e como **cinto de segurança agregado** por cima das filas individuais.

! O conceito mais importante do capítulo

PCQ transforma *uma* fila em *milhares*, pelo preço de um classificador — mas repare no que ele **não** faz: não autentica, não contabiliza, não corta, não diferencia planos dentro da mesma fila. PCQ é **fila em escala**; PPPoE é **assinante em escala**. O provedor maduro usa os dois no lugar certo — e desconfia de qualquer projeto que use um só para tudo.

40.4 Laboratório

i Ambiente

O trio completo (Capítulo 5): lab-r1 no meio (172.16.1.1 na lab-a, 172.16.2.1 na lab-b), lab-r2 (172.16.1.2) e lab-r3 (172.16.2.3) como “clientes” em redes opostas — e o próprio host (VirtualBox host-only, 192.168.56.1) pode ser o terceiro cliente se você quiser mais disputa. `bandwidth-server` habilitado em r2 e r3. Limpe filas e mangles dos capítulos anteriores no lab-r1 (`/queue tree remove [find]`, `/queue simple remove [find]`, `/ip firewall mangle remove [find]`).

40.4.1 Comandos passo a passo

Passo 1 — o tipo e a fila. No lab-r1, plano único de 8M/4M por cliente, agregado de 20M (apertado de propósito), tudo numa simple queue da “rede de clientes” (as duas sub-redes do lab):

```

/queue type
add name=pcq-down-8m kind=pcq pcq-classifier=dst-address pcq-rate=8M
add name=pcq-up-4m kind=pcq pcq-classifier=src-address pcq-rate=4M
/queue simple
add name=clientes-pcq target=172.16.1.0/24,172.16.2.0/24 \
    max-limit=20M/20M queue=pcq-down-8m/pcq-up-4m

```

Passo 2 — um cliente sozinho. Teste receive do lab-r2: ~8M — o `pcq-rate` limitou, não o `max-limit` (sobrava agregado). O `print stats` da simple queue mostra `rate=8M`: a fila externa enxerga a soma.

Passo 3 — dois clientes. Testes simultâneos em lab-r2 e lab-r3: ~8M **cada** (16M no agregado — ainda coube). Cada um na sua sub-fila, tetos independentes. Um terceiro cliente (o host) levaria o agregado a 20M — e aí o `max-limit` externo passa a repartir: ~6,6M cada.

Passo 4 — repartição justa. Troque para o modo `pcq-rate=0`:

```

/queue type set pcq-down-8m pcq-rate=0

```

Um cliente sozinho: **20M** (o agregado inteiro!). Dois: ~10M cada. O PCQ virou divisor igualitário do `max-limit` — nenhum número de plano envolvido.

Passo 5 — o clássico bug do classificador. Inverta de propósito o classificador do download (`set pcq-down-8m pcq-classifier=src-address pcq-rate=8M`) e rode os dois clientes: agora o download de **todos** divide UMA sub-fila de 8M (a “origem” é o mesmo servidor de teste!) — 4M cada, caindo com cada cliente novo. É exatamente o que acontece em produção quando se copia o tipo do upload para o download. Restaure `dst-address`.

40.4.2 Verificação

1. Um cliente: teto = `pcq-rate`; agregado com folga;
2. Vários clientes: cada um com seu teto até o agregado apertar — e então divisão do `max-limit`;
3. Com `pcq-rate=0`, a banda de cada um depende só de **quantos** estão ativos;
4. Com o classificador errado, todos os downloads dividiram uma única sub-fila — e você sabe explicar por quê.

40.4.3 Desafios

1. O provedor tem três planos (10M, 50M, 100M) numa rede de IP fixo sem PPPoE. Projete a solução PCQ completa: quantos queue types, como separar os clientes de cada plano (dica: `address-list` + `mangle` do Capítulo 39), e onde ficam os `max-limit`.

2. Num hotspot com PCQ `pcq-rate=0` sobre `max-limit=50M`, um usuário abre 30 conexões de download e outro navega com 2. O PCQ garante divisão justa entre os **dois**? Justifique pelo classificador — e diga o que aconteceria se o classificador fosse `dst-port`.
3. Calcule a latência máxima que o `pcq-limit=50KiB` impõe a um cliente cuja sub-fila está cheia com `pcq-rate=10M` — e compare com o mesmo buffer a `pcq-rate=1M`. O que isso diz sobre PCQ para planos muito lentos?
4. Arquiteto A propõe: “migre tudo para PCQ e desligue o PPPoE — menos CPU, menos complexidade”. Escreva o contra-argumento em três pontos concretos (o que se perde), e o cenário em que A estaria certo.

Solução dos desafios

1. Três pares de queue types (down/up por plano: `pcq-down-10m/50m/100m` com os rates respectivos, idem up). Separação: uma **address-list por plano** (`clientes-10m`, ...) mantida pelo cadastro; mangle marcando pacote por `dst-address-list/src-address-list` → três packet-marks por sentido; queue tree com três classes por sentido, cada uma com seu `queue=pcq-*` e `max-limit` = teto do agregado (ou sub-tetos por plano, se o negócio pedir), todas sob um pai com o teto físico do enlace. É o padrão de mercado para redes IP-fixa sem BNG.
2. Garante: o classificador é `src-address/dst-address` — as 30 conexões do usuário A caem **na mesma sub-fila** dele (o endereço é um só), que disputa de igual para igual com a sub-fila do usuário B: ~25M cada, independentemente do número de conexões. Com `dst-port` como classificador, cada porta viraria “um usuário”: as 30 conexões de A virariam dezenas de sub-filas contra as 2 de B — A levaria ~94% do enlace. Classificador define o que é “justo”.
3. Fila cheia = 50 KiB = 409,6 kbit esperando. A 10M: $409,6/10\,000 \approx 41$ ms — aceitável. A 1M: **410 ms** — inutilizável para qualquer coisa interativa. Moral: `pcq-limit` deve escalar com o `pcq-rate` (planos lentos pedem buffers menores), senão o plano social vira fábrica de bufferbloat; é a mesma aritmética do desafio 3 do Capítulo 37.
4. Perde-se: (1) **identidade e corte** — sem sessão, suspender inadimplente volta a ser caça ao MAC/IP na mão (o Volume 6 inteiro); (2) **accounting** por assinante (consumo, uptime — dados de cobrança e suporte); (3) **planos e exceções por cliente** viram malabarismo de address-lists (IP fixo contratado, burst individual, CoA). A ganha em: rede **sem contrato individual** — hotspot público, evento, cortesia de condomínio — onde identidade não existe mesmo e PCQ entrega tudo que se precisa com uma fração da complexidade. Contexto decide.

40.5 Resumo

- O **tipo de fila** é o algoritmo interno (`fifo`, `sfq`, `red`, `pcq`); o **PCQ** multiplica: uma sub-fila FIFO **por valor do classificador**, cada uma com o mesmo `pcq-rate`;
- Par clássico: download classifica por `dst-address`, upload por `src-address` — inverter é o bug nº 1 (todos numa sub-fila só);

- **pcq-rate=N** = plano único em massa; **pcq-rate=0** = repartição igualitária do **max-limit** externo entre os ativos;
- **pcq-limit** controla a latência por sub-fila ($\text{buffer} \times \text{rate} = \text{milissegundos}$ — planos lentos pedem buffers menores);
- **PCQ é fila em escala; PPPoE é assinante em escala**: identidade, plano individual e corte continuam sendo papel do Volume 6 — PCQ brilha onde não há contrato individual (hotspot, faixas fixas, repartição de backhaul).

Fecha o Volume 7 — o tráfego do assinante agora tem plano, turbo, garantia, prioridade e escala. O Volume 8 sai da fila e entra no mapa: **roteamento**, estático e dinâmico, para a rede que cresceu além de um roteador.

Bibliografia do capítulo

- Documentação oficial: seções *Queue Types* — *PCQ* e *Queues* (MikroTik, 2026a);
- Disciplinas de fila e fairness: (Tanenbaum; Feamster; Wetherall, 2021);
- RouterOS v7 na prática: (Discher, 2016);
- QoS em escala de provedor: (Burgess, 2011).

Parte VIII

Volume 8 — Roteamento

41 Rotas estáticas, distância e failover

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Ler a tabela de rotas do RouterOS v7 (flags, `dst-address`, `gateway`, `distance`) e prever qual rota vence para cada destino;
- Aplicar as duas regras da decisão de roteamento: **prefixo mais longo primeiro; menor distância como desempate**;
- Construir failover de dois links com rotas flutuantes + **check-gateway** — e conhecer os limites dessa detecção;
- Usar rotas **blackhole** para descartar tráfego com propósito (agregados, proteção anti-loop);
- Diagnosticar roteamento com `/ip route check` e `traceroute` antes de culpar o resto da rede.

Até aqui, nossa rede tinha um roteador de cada lado e uma rota padrão resolvia tudo. Mas o provedor cresceu: dois POPs, um enlace de rádio entre eles (Volume 5), concentradores com pools distintos (Capítulo 36)... e de repente “para onde mando este pacote?” tem mais de uma resposta certa — e várias erradas. Este volume é sobre **roteamento**: primeiro o controle manual (rotas estáticas), depois a automação (OSPF, BGP). Como sempre, o fundamento manda: quem não domina a tabela de rotas só transfere a confusão para o protocolo dinâmico.

41.1 Como o roteador decide

Para cada pacote, o RouterOS consulta a tabela (`/ip route print`) e aplica **duas regras, nesta ordem**:

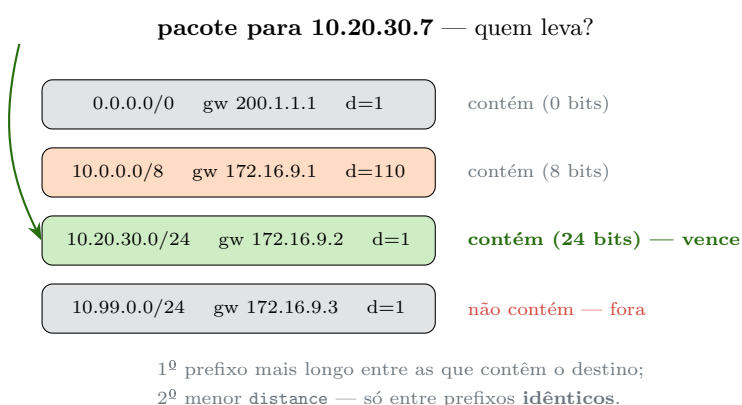
1. **Prefixo mais longo vence** (*longest prefix match*): entre as rotas cujo `dst-address` contém o destino, ganha a mais específica. `10.20.30.0/24` vence `10.0.0.0/8`, que vence `0.0.0.0/0` — a rota padrão é só “o prefixo mais curto possível”, o plano Z de todo pacote;
2. **Menor distância desempata prefixos iguais**: se duas rotas têm o MESMO prefixo, vence a de menor `distance`.

A **distância administrativa** é a nota de confiança da *origem* da rota — quanto menor, mais confiável:

Tabela 41.1: Distâncias administrativas no RouterOS

Origem	Distância padrão
Conectada (IP na interface)	0
Estática	1
eBGP	20
OSPF	110
RIP	120
iBGP	200

É ela que responde “quem manda quando o OSPF e uma estática discordam” (a estática, 1 < 110) — e é ela que vamos explorar de propósito no failover.



A decisão para 10.20.30.7: das rotas que contêm o destino, vence a de prefixo mais longo (/24) — a distância nem é consultada. Ela só desempata rotas com o **mesmo** prefixo.

Figura 41.1

! O conceito mais importante do capítulo

Prefixo primeiro, distância depois — e nunca o contrário. Uma estática /32 vence qualquer rota padrão, por melhor que seja a distância desta; e duas rotas padrão disputam **apenas** pela distância. Metade dos mistérios de roteamento (“por que isso está saindo pelo link errado?!”) morre quando se pergunta, na ordem: *qual é o prefixo mais longo que contém o destino? entre iguais, qual a menor distância? e essa rota está ativa?*

41.2 Anatomia de uma rota no v7

```
/ip route add dst-address=10.20.30.0/24 gateway=172.16.9.2 \
    distance=1 comment="rede do POP rural via radio"
/ip route print
```

Flags: D - DYNAMIC; A - ACTIVE; c - CONNECT, s - STATIC

Columns: DST-ADDRESS, GATEWAY, DISTANCE

	DST-ADDRESS	GATEWAY	DISTANCE
DAc	172.16.9.0/24	ether2	0
As	10.20.30.0/24	172.16.9.2	1
As	0.0.0.0/0	200.1.1.1	1

As flags do v7 que interessam: **A** (active — está valendo), **D** (dynamic — criada por protocolo/sistema, não por você), **c/s/o/b** (connect/static/ospf/bgp — a origem). Uma rota **sem A** existe mas não decide nada — os motivos clássicos: gateway inalcançável (não há rota conectada que o cubra) ou perdeu a disputa para outra de mesma dst e menor distância.

O **gateway** pode ser um IP (resolvido para uma interface via tabela) ou uma **interface** diretamente — o normal em túneis e PPPoE (**gateway=pppoe-wan**, como o **add-default-route** do Volume 6 faz por baixo).

41.3 Failover: rotas flutuantes + check-gateway

O caso real: o POP tem dois caminhos para a internet — fibra (principal) e um rádio de contingência. Queremos: tudo pela fibra; se ela cair, migrar sozinho; quando voltar, voltar.

```
/ip route
add dst-address=0.0.0.0/0 gateway=200.1.1.1 distance=1 \
    check-gateway=ping comment="fibra (principal)"
add dst-address=0.0.0.0/0 gateway=189.50.7.1 distance=10 \
    comment="radio (contingencia) - rota flutuante"
```

Mesmo prefixo, distâncias diferentes: a de **distance=10** fica **inativa** (perdeu o desempate) — é a *rota flutuante*, flutuando à espera. O **check-gateway=ping** pinga o gateway a cada 10 s; **duas falhas seguidas** e a rota principal é marcada inativa → a flutuante assume no mesmo instante. Fibra voltou a responder → hierarquia restaurada.

O limite honesto do **check-gateway**: ele testa o **vizinho**, não o mundo. Se a operadora aceita o ping do gateway mas está sem trânsito (cai o upstream dela), sua rota continua “ativa” apontando para um buraco. O antídoto é vigiar um alvo remoto: rotas /32 para IPs de teste + **netwatch** trocando rotas — receita que fica para os scripts do Volume 12 — ou roteamento dinâmico de verdade (BGP com a operadora, Capítulo 44), onde a queda de trânsito derruba a sessão e as rotas juntas.

💡 Analogia para guardar

Rotas flutuantes são a **geladeira com gerador**: a casa liga na rede elétrica (distância 1); o gerador está instalado e pronto (distância 10), mas só entra quando o sensor detecta a queda (check-gateway) — e sai de cena quando a rede volta. E o limite é o mesmo: o sensor mede a tomada da SUA casa. Se a concessionária está de pé mas a subestação do bairro caiu... a tomada tem energia, a rua não — e o sensor não vê.

41.4 Blackhole: descartar com propósito

Uma rota **blackhole** descarta silenciosamente o que casar com ela:

```
/ip route add dst-address=100.64.100.0/22 blackhole \  
comment="agregado dos pools PPPoE - anti-loop"
```

Por que um provedor descarta o próprio agregado? Anti-loop: o concentrador anuncia 100.64.100.0/22 inteiro para a rede (Capítulo 44 fará isso), mas cada sessão PPPoE cria rota /32 individual. Um pacote para um IP do bloco **sem sessão ativa** (cliente desconectado) casaria com... a rota padrão, sairia para a operadora, voltaria (o bloco é seu), sairia de novo — loop até o TTL morrer, banda queimada em atacante escaneando seu bloco. Com a blackhole: IP com sessão casa a /32 (mais específica!); IP sem sessão casa a blackhole e morre em paz. O prefixo mais longo trabalhando a seu favor.

41.5 Diagnóstico em dois comandos

```
/ip route check 8.8.8.8  
/tool traceroute 8.8.8.8
```

O **route check** responde “qual rota este destino casaria AGORA” — a decisão da Figura 41.1 sem adivinhação. O **traceroute** mostra o caminho de fato, salto a salto: divergência entre os dois = rota assimétrica ou NAT no caminho. Diagnóstico de roteamento começa nesses dois, não no reboot.

41.6 Laboratório

i Ambiente

O trio em linha (Capítulo 5): lab-r2 (172.16.1.2, lab-a) — lab-r1 (172.16.1.1 / 172.16.2.1) — lab-r3 (172.16.2.3, lab-b), e o lab-r1 também com a NAT WAN (ether3/ether1 conforme seu setup). Vamos fazer o lab-r2 alcançar a lab-b sem rota padrão, montar failover artificial e ver a blackhole proteger.

41.6.1 Comandos passo a passo

Passo 1 — a dor da falta de rota. No lab-r2, remova a rota padrão (se houver) e pingue o lab-r3:

```
/ip route remove [find dst-address=0.0.0.0/0]
/ping 172.16.2.3 count=3
```

no route to host — o pacote nem sai. Roteador sem rota não “tenta”: descarta e avisa.

Passo 2 — a estática específica. Ainda no lab-r2:

```
/ip route add dst-address=172.16.2.0/24 gateway=172.16.1.1 \
    comment="lab-b via lab-r1"
/ping 172.16.2.3 count=3
```

O ping sai... e pode voltar? Só se o lab-r3 souber **voltar** — confira nele a rota para 172.16.1.0/24 (a lição eterna: rota é ida E volta, cada uma decidida na sua tabela).

Passo 3 — prefixo vence distância. No lab-r2, adicione uma rota padrão e uma /32 provocadora:

```
/ip route add dst-address=0.0.0.0/0 gateway=172.16.1.1 distance=1
/ip route add dst-address=172.16.2.3/32 gateway=172.16.1.99 distance=200
/ip route check 172.16.2.3
```

O check aponta a /32 de distância **200** (gateway até inexistente!) — prefixo mais longo primeiro, sempre. Como o gateway não resolve, a rota fica inativa e o tráfego cai para a /24... remova a provocação e siga.

Passo 4 — failover flutuante. No lab-r2, duas rotas padrão:

```
/ip route remove [find dst-address=0.0.0.0/0]
/ip route add dst-address=0.0.0.0/0 gateway=172.16.1.1 distance=1 \
    check-gateway=ping comment="principal"
/ip route add dst-address=0.0.0.0/0 gateway=172.16.1.254 distance=10 \
    comment="contingencia (gateway ficticio p/ demonstrar)"
/ip route print where dst-address=0.0.0.0/0
```

Só a `distance=1` está Ativa. Agora derrube o “principal”: no VirtualBox, desconecte o cabo da lab-a do **lab-r1** (não do r2!) e acompanhe no lab-r2:

```
/ip route print interval=2 where dst-address=0.0.0.0/0
```

Em ~20–30 s (2 pings falhos), a principal perde o A e a flutuante assume. Reconecte o cabo: a hierarquia volta sozinha. (A contingência aponta para um gateway fictício — o objetivo aqui é ver a **troca de flags**, não navegar por ela.)

Passo 5 — blackhole anti-varredura. No lab-r1, simule o agregado de clientes:

```
/ip route add dst-address=100.64.100.0/22 blackhole
/ping 100.64.100.77 count=2
```

Resposta imediata: inalcançável — sem viajar até a rota padrão, sem loop. Uma rota /32 de “sessão” (`add dst-address=100.64.100.77/32 gateway=172.16.1.2`) e o mesmo ping muda de destino: a específica fura a blackhole, exatamente como as sessões PPPoE farão.

41.6.2 Verificação

1. Sem rota: no `route to host`; com estática + rota de **volta** no r3: ping completo;
2. No Passo 3, o `route check` escolheu a /32 (prefixo > distância);
3. No failover, você viu a flag A migrar entre as rotas e voltar;
4. A blackhole responde na hora (sem timeout) e a /32 a sobrepõe.

41.6.3 Desafios

1. Um júnior configurou o failover com as duas rotas padrão em `distance=1`. O que o RouterOS faz com rotas idênticas em tudo — e por que o resultado (dica: ECMP) pode “funcionar” no teste e quebrar streaming/VoIP na prática?
2. `check-gateway=ping` na rota principal, e a operadora da fibra **bloqueia ICMP** no gateway. Descreva o desastre em produção e as duas saídas (uma no próprio `check-gateway`, uma na arquitetura).
3. O suporte reclama: “criei a rota para a nova sub-rede do cliente, está na tabela, mas não passa tráfego”. `print` mostra a rota **sem** a flag A, gateway 10.7.7.1. Liste as duas causas mais prováveis e o comando que distingue uma da outra.

4. Projete as rotas do lab-r1 para o cenário final do provedor: pools PPPoE (100.64.100.0/22) no BNG-01 (172.16.1.2) e (100.64.104.0/22) no BNG-02 (172.16.2.3), com blackhole anti-loop **nos BNGs** e agregação no lab-r1. Escreva os comandos dos três roteadores.

💡 Solução dos desafios

1. Duas rotas com mesma dst e mesma distância ativas viram **ECMP** (*Equal Cost Multi-Path*): o roteador balanceia fluxos entre os gateways. No teste (um ping, um site) parece failover funcionando; na prática, metade das conexões sai por cada link — com IPs de origem diferentes pós-NAT, sessões de streaming/bancos/VoIP quebram ao alternar caminho, e a queda de um link ainda leva ~metade do tráfego junto até o gateway morto ser detectado. Failover pede distâncias **diferentes**; ECMP é ferramenta de balanceamento consciente, não failover acidental.

2. O check marca a rota principal como morta (pings nunca respondem) e TODO o tráfego migra para a contingência — você paga rádio lento com a fibra perfeita de pé; pior: é estável, ninguém percebe por dias. Saídas: **check-gateway=arp** (testa resolução ARP do gateway — funciona se ele está na mesma L2 e responde ARP) ou tirar a detecção do vizinho: monitorar alvo remoto via netwatch/script (Volume 12) ou BGP com a operadora (Capítulo 44).

3. Sem A com gateway IP: (a) **gateway irresolúvel** — nenhuma rota conectada/ativa cobre 10.7.7.1 (interface caiu? IP errado?); (b) **perdeu o desempate** — existe outra rota com a MESMA dst e distância menor. Distinguir: `/ip route print detail where dst-address=<dst>` mostra todas as candidatas e seus estados; e `/ip route check 10.7.7.1` (o gateway como destino!) revela se ele é alcançável. (a) aparece como **unreachable**; (b) mostra a rota rival ativa.

4. BNGs (anti-loop local): lab-r2: `/ip route add dst-address=100.64.100.0/22 blackhole` e lab-r3: `/ip route add dst-address=100.64.104.0/22 blackhole` (as /32 das sessões furam a blackhole). lab-r1 (agregação): `/ip route add dst-address=100.64.100.0/22 gateway=172.16.1.2` e `/ip route add dst-address=100.64.104.0/22 gateway=172.16.2.3` — e, quando houver anúncio externo, anunciar o agregado 100.64.100.0/21... que o Capítulo 42 fará circular sozinho.

41.7 Resumo

- Decisão de roteamento: **prefixo mais longo primeiro**; **distance** desempata só prefixos **idênticos**; rota sem flag A não decide nada (gateway irresolúvel ou desempate perdido);
- Distâncias padrão: conectada 0, estática 1, eBGP 20, OSPF 110, iBGP 200 — a nota de confiança da origem da rota;
- **Failover** = rotas flutuantes (mesma dst, distâncias diferentes) + **check-gateway=ping/arp**; limite: testa o **vizinho**, não o trânsito — para mais, netwatch/BGP;

- **Blackhole** descarta com propósito: agregados de pools nos BNGs contra loops de varredura (as /32 de sessão furam por prefixo);
- Diagnóstico: `/ip route check` (a decisão) + `traceroute` (o caminho real) — sempre antes de mexer.

Rotas estáticas não escalam com a rede: cada enlace novo são N tabelas para editar à mão. O próximo capítulo (Capítulo 42) coloca os roteadores para **conversarem**: OSPF.

Bibliografia do capítulo

- Documentação oficial: seção *IP Routing* do RouterOS v7 (MikroTik, 2026a);
- Algoritmos de roteamento e longest prefix match: (Tanenbaum; Feamster; Wetherall, 2021);
- RouterOS v7 na prática: (Discher, 2016);
- Redes de provedor: (Burgess, 2011).

42 OSPF: conceitos e configuração

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

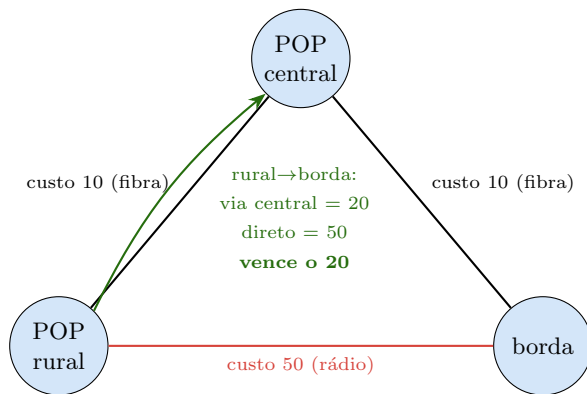
- Explicar por que um protocolo de estado de enlace (**link-state**) substitui com vantagem as rotas estáticas quando a rede tem malha;
- Descrever o ciclo do OSPF: descoberta de vizinhos (Hello), sincronização do banco de dados (LSAs) e cálculo de menor caminho (Dijkstra) sobre o **custo**;
- Entender **áreas**, o papel da área 0 (backbone) e por que existem DR/BDR em redes multiacesso;
- Configurar OSPFv2 no RouterOS v7 (a tríade **instance** → **area** → **interface-template**) e ler **neighbor/lsa**;
- Redistribuir rotas (conectadas, estáticas) e ajustar **custo** para moldar o caminho — sem criar caminhos assimétricos.

O Capítulo 41 terminou com um incômodo: cada enlace novo são várias tabelas para editar à mão, e um link que cai não avisa ninguém. Numa rede com **malha** — dois POPs, um anel de fibra, o rádio de contingência — isso não escala nem sobrevive a falhas. O **OSPF** (*Open Shortest Path First*) resolve os dois: os roteadores trocam a topologia entre si, cada um calcula sozinho o melhor caminho, e quando algo muda **a rede reconverge sem intervenção**. É o protocolo de roteamento interno padrão de qualquer provedor.

42.1 Link-state: cada roteador tem o mapa inteiro

Há duas famílias de protocolos internos. Os **distance-vector** (RIP) dizem ao vizinho “sei chegar em X a N saltos” — cada um confia na palavra do outro, e mudanças se propagam de boca em boca (lento, sujeito a loops). Os **link-state** (OSPF) fazem o oposto: cada roteador anuncia **apenas seus próprios enlaces** (a quem está diretamente conectado e com que custo), e esses anúncios — **LSAs** (*Link-State Advertisements*) — são inundados intactos por toda a área. Resultado: **todo roteador monta o mesmo mapa completo** da rede e roda o algoritmo de Dijkstra sobre ele para achar o menor caminho até cada destino.

A diferença prática: no link-state ninguém “acredita” em rota de terceiro — todos veem a topologia e calculam. Convergência rápida, sem loops de contagem ao infinito, ao custo de mais CPU/memória (o mapa) e de disciplina de projeto (áreas).



OSPF escolhe por **custo somado**, não por número de saltos: o caminho POP rural → borda via POP central (10+10=20) vence o enlace direto de rádio (50), embora tenha um salto a mais. Ajustar custos é como você molda o tráfego.

Figura 42.1

42.2 Vizinhança, custo e áreas

Adjacências. OSPF só troca LSAs com vizinhos com quem formou **adjacência** — e para isso os pacotes Hello dos dois lados precisam concordar em: mesma área, mesmos temporizadores (hello/dead), mesma sub-rede, autenticação compatível. Hello divergente = vizinho que “aparece e some” — o bug de campo nº 1 do OSPF.

Custo. O peso de cada enlace. No RouterOS o custo padrão deriva da referência de banda; o que importa é a regra: **caminho vence pela menor soma de custos**. Você ajusta custo para preferir a fibra e deixar o rádio como reserva (como na figura) — mas com cuidado: mexer no custo de **um lado** cria caminho assimétrico (ida pela fibra, volta pelo rádio), fonte de dor de cabeça diagnóstica.

DR/BDR. Numa rede multiacesso (vários roteadores num mesmo segmento L2, ex.: um switch no POP), se todos formassem adjacência com todos seriam N^2 relações. OSPF elege um **DR** (*Designated Router*) e um **BDR** (backup): todos falam com o DR, que redistribui — reduz a N . Em enlaces ponto a ponto (a maioria no provedor: um PtP de rádio, um túnel) **não há DR** — são só dois, adjacência direta.

Áreas. OSPF divide a rede em **áreas** para não inundar LSAs pelo mundo todo. A **área 0** é o *backbone* — obrigatória, e toda outra área precisa tocá-la. Numa rede pequena (nossos poucos POPs), **tudo na área 0** é o certo; áreas adicionais são assunto do próximo capítulo (14), quando a rede cresce a ponto de o mapa único pesar.

💡 Analogia para guardar

Rotas estáticas são cada motorista decorando um trajeto fixo: mudou uma rua, todos se perdem até você reensinar um a um. OSPF é o **app de trânsito** que todos usam: cada carro reporta as ruas que **ele** vê (os LSAs), o app monta o mapa completo da cidade e cada um calcula sua própria melhor rota (Dijkstra). Fechou uma via? Os relatos se espalham em segundos e todos recalculam sozinhos. O custo do enlace é o “tempo estimado” de cada rua — e é isso, não a distância em quarteirões, que o app minimiza.

42.3 Configurando OSPFv2 no RouterOS v7

O v7 reorganizou o OSPF (quem vem do v6 estranha). São três objetos:

```
# 1) a instância: o "processo" OSPF e seu router-id (identidade única!)
/routing ospf instance
add name=isp-ospf router-id=10.255.0.1

# 2) a área: aqui, só o backbone
/routing ospf area
add name=backbone area-id=0.0.0.0 instance=isp-ospf

# 3) em quais interfaces falar OSPF (e como) - via template por rede
/routing ospf interface-template
add area=backbone networks=172.16.9.0/30 interfaces=ether2 \
    comment="enlace PtP para o POP rural"
add area=backbone networks=172.16.9.4/30 interfaces=ether3 \
    comment="enlace PtP para a borda"
```

- **router-id**: a identidade do roteador no OSPF — único no domínio, convenção usar um IP de *loopback* (uma /32 numa bridge sem portas, sempre no ar mesmo se uma interface física cair). Router-id duplicado = adjacências instáveis e LSAs em guerra;
- **interface-template**: casa interfaces por **networks** (as sub-redes que participam) e/ou **interfaces**; é onde se define custo, tipo (**ptp** desliga a eleição de DR num enlace ponto a ponto), temporizadores e autenticação.

Anunciar as redes conectadas dos clientes (LANs, pools) sem colocar OSPF **nas interfaces dos clientes** (não queremos vizinhança com assinante!): use um template passivo ou redistribuição:

```
/routing ospf instance
set isp-ospf redistribute=connected out-filter-chain=ospf-out

/routing filter rule
```

```
add chain=ospf-out rule="if (dst==100.64.100.0/22) { accept }"
add chain=ospf-out rule="reject"
```

Assim o backbone aprende o agregado dos pools (Capítulo 41) sem expor o OSPF ao lado do cliente. Filtros de redistribuição são o tema do Capítulo 45 — aqui, o mínimo para não vaziar tudo.

42.4 Lendo o OSPF

```
/routing ospf neighbor print
/routing ospf lsa print
/ip route print where ospf
```

```
# neighbor print
0 instance=isp-ospf router-id=10.255.0.2 address=172.16.9.2
  state="Full" ...
```

O estado que você quer é **Full**: adjacência completa, bancos sincronizados. Estados travados contam a história: **Init/2-Way** (vê o Hello mas não fecha — cheque área/timers/autenticação); **ExStart/Exchange** preso (quase sempre **MTU divergente** entre os lados — OSPF exige MTU igual). Rota OSPF na tabela aparece com flag **Do** (dynamic/ospf) e **distância 110** (Tabela 41.1) — daí a estática de distância 1 vencê-la, como você já sabia.

! O conceito mais importante do capítulo

OSPF entrega **um mapa idêntico a todos os roteadores** e deixa cada um calcular o menor caminho por **custo somado** — é isso que dá convergência rápida e sem loops que a estática nunca terá. Mas o preço é disciplina: **router-id** único, temporizadores e MTU iguais entre vizinhos, custos ajustados nos **dois** sentidos e OSPF **jamais** na interface do cliente. A tríade v7 (instance → area → template) só funciona bem quando esses fundamentos estão de pé; o resto do volume constrói sobre eles.

42.5 Laboratório

i Ambiente

O trio em triângulo (Capítulo 5) — a topologia da Figura 42.1: lab-r1 = POP central, lab-r2 = POP rural, lab-r3 = borda. Enlaces PtP /30: r1-r2 em 172.16.9.0/30, r1-r3 em 172.16.9.4/30, r2-r3 em 172.16.9.8/30 (crie a terceira interface/ rede interna no VirtualBox fechando o triângulo). Cada roteador com um loopback /32 para router-id:

```
r1=10.255.0.1, r2=.2, r3=.3.
```

42.5.1 Comandos passo a passo

Passo 1 — loopback e router-id. Em cada roteador (exemplo r1):

```
/interface bridge add name=lo  
/ip address add address=10.255.0.1 interface=lo
```

Passo 2 — a tríade OSPF em cada um. No lab-r1 (POP central, tocando os dois enlaces):

```
/routing ospf instance add name=isp router-id=10.255.0.1  
/routing ospf area add name=backbone area-id=0.0.0.0 instance=isp  
/routing ospf interface-template \  
    add area=backbone networks=172.16.9.0/30,172.16.9.4/30,10.255.0.1/32 \  
    type=ptp
```

Réplica análoga em r2 (redes 172.16.9.0/30, 172.16.9.8/30, 10.255.0.2/32) e r3 (172.16.9.4/30, 172.16.9.8/30, 10.255.0.3/32), cada um com seu router-id.

Passo 3 — veja a vizinhança nascer. No lab-r1:

```
/routing ospf neighbor print
```

Espere `state=Full` com r2 e r3. Se travar em `ExStart`, é MTU — confira `/interface ethernet print` (iguale os MTUs). Se ficar em `Init`, revise área/timers.

Passo 4 — o mapa se formou. Em qualquer roteador:

```
/ip route print where ospf
```

O lab-r2 (rural) aprendeu sozinho a chegar na rede de r3 (borda) — **sem uma linha de rota estática**. Confirme o caminho:

```
/tool traceroute 10.255.0.3
```

Passo 5 — molde o caminho pelo custo. Hoje r2→r3 pode ir direto (um enlace) ou via r1. Encareça o enlace direto r2-r3 nos **dois** lados (r2 e r3), simulando “rádio caro”:

```
/routing ospf interface-template set [find networks~"172.16.9.8"] cost=50
```

Ref faça o traceroute de r2 para r3: agora passa por **r1** (custo $10+10=20 < 50$) — a Figura 42.1 ao vivo. Mude o custo em **um só lado** e observe a assimetria (ida por um caminho, volta por outro) no traceroute dos dois sentidos.

Passo 6 — reconvergência. Com o tráfego indo $r2 \rightarrow r1 \rightarrow r3$, derrube o enlace r1–r3 (desconecte o cabo no VirtualBox). Em segundos, o traceroute de r2 para r3 passa a usar o enlace direto (custo 50, agora o único vivo) — a rede se curou sozinha. Religue e ela volta ao ótimo.

42.5.2 Verificação

1. `neighbor print` mostra Full em todas as adjacências esperadas;
2. Cada roteador tem, sem rota estática, caminho para os loopbacks dos outros dois;
3. Encarecer o enlace direto (dois lados) desviou o tráfego pelo POP central; um lado só criou assimetria visível no traceroute;
4. A queda de um enlace reconvergeu automaticamente em segundos.

42.5.3 Desafios

1. Dois roteadores veem o Hello um do outro (`neighbor` aparece) mas nunca passam de 2-Way num enlace ponto a ponto. Liste as três causas mais prováveis e o comando que confirma cada uma.
2. Você configurou tudo certo, mas as adjacências ficam **presas em Exchange** e caem. `ping` entre os roteadores funciona com pacotes pequenos e falha com `size=1500 do-not-fragment`. Nomeie a causa e a correção — e por que o OSPF é tão sensível a isso quando o ping “normal” funcionava.
3. Por que colocar OSPF na interface **do cliente** (LAN/PPPoE) é um erro de segurança e de estabilidade? Descreva o que um cliente mal-intencionado (ou um roteador doméstico “espancado”) poderia fazer, e a diretiva de template que anuncia a rede sem formar adjacência ali.
4. Sua rede tem 40 roteadores numa única área 0. Tudo funciona, até que um enlace instável em um POP começa a “piscar” (flapping) e a CPU de **todos** os 40 sobe em ondas. Explique o mecanismo (LSA $\rightarrow$ SPF) e antecipe qual recurso do próximo capítulo

(14) contém o estrago.

Solução dos desafios

1. Preso em 2-Way num PtP indica que não avançou para adjacência plena. Causas: (a) **áreas diferentes** nos dois lados (`/routing ospf interface-template print` — o `area`); (b) **temporizadores** hello/dead divergentes (mesmo comando, campos de timer); (c) **autenticação** só de um lado ou com chave diferente. Em rede multiacesso, 2-Way também é o estado *normal* entre dois não-DR — mas num PtP (`type=ptp`) deve ir a Full.

2. MTU divergente entre as interfaces. O OSPF, na fase **Exchange**, troca o Database Description e exige que o MTU anunciado bata; além disso os LSAs grandes precisam trafegar inteiros. O ping pequeno funciona porque cabe em qualquer MTU; o `size=1500 df` falha no enlace de MTU menor — o mesmo diagnóstico do PPPoE (Capítulo 33). Correção: igualar o MTU dos dois lados (ou, em último caso, `mtu-ignore` no template — trata o sintoma, não a causa).

3. Segurança: OSPF sem autenticação aceita LSAs de qualquer vizinho — um cliente injeta uma rota 0.0.0.0/0 mais atraente e **sequestra o tráfego** do POP (ataque clássico). Estabilidade: cada roteador doméstico ruidoso vira uma adjacência que sobe e cai, disparando recálculos. A diretiva certa: template **passivo** (`passive` / sem formar adjacência) na interface do cliente, ou redistribuição **connected** filtrada — anuncia-se a rede, nunca se fala OSPF *com* o cliente. E autenticação obrigatória nos enlaces internos.

4. Cada flap gera novos LSAs, inundados por toda a área 0; cada LSA novo obriga **todos** os 40 roteadores a rodar Dijkstra (SPF) sobre o mapa inteiro — ondas de CPU sincronizadas. Numa área única, não há barreira: o ruído local é problema global. O 14 contém o estrago com **áreas** (o flap fica confinado à sua área; para o resto da rede, um ABR resume a área como um prefixo estável) e com temporizadores de SPF/LSA que amortecem o flapping.

42.6 Resumo

- **Link-state (OSPF):** cada roteador anuncia só seus enlaces (LSAs), todos montam o **mesmo mapa** e calculam o menor caminho por **custo somado** (Dijkstra) — convergência rápida, sem loops, ao custo de CPU/memória e disciplina;
- Fundamentos que fazem ou quebram: **router-id único** (use loopback), Hello/timers/MTU iguais entre vizinhos, `type=ptp` em enlaces ponto a ponto (sem DR), **tudo na área 0** enquanto a rede é pequena;
- No **v7**: `instance` → `area` → `interface-template`; redes de cliente entram por **redistribuição filtrada** ou template passivo — **nunca** OSPF na interface do assinante;
- Diagnóstico: `neighbor print` (quer Full; 2-Way = área/timers, **Exchange** preso = MTU), `lsa print`, `route where ospf` (distância 110);
- Custo molda o caminho — sempre nos **dois** sentidos, sob risco de assimetria.

Uma área só não escala para dezenas de roteadores nem confina falhas. O próximo capítulo (14) parte a rede em áreas, autentica os enlaces e refina custos e filtros.

Bibliografia do capítulo

- Documentação oficial: seção *OSPF* do RouterOS v7 (MikroTik, 2026a);
- Roteamento link-state, Dijkstra e OSPF: (Tanenbaum; Feamster; Wetherall, 2021);
- RouterOS v7 na prática: (Discher, 2016);
- OSPF em provedores: (Burgess, 2011).

43 OSPF avançado: áreas, autenticação e filtros

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar por que uma **única área** deixa de escalar e como partir a rede em **múltiplas áreas** com um ABR (*Area Border Router*);
- Escolher o **tipo de área** certo — normal, **stub**, **NSSA** — para encolher a tabela dos roteadores de ponta;
- Proteger os enlaces internos com **autenticação** OSPF e conter *flapping* com temporizadores de SPF/LSA;
- Modelar o tráfego com **custo** e **cost referência**, e filtrar o que entra/sai do OSPF com *routing filters* do v7;
- Reconhecer e evitar os erros clássicos de projeto multiárea (área desconexa do backbone, redistribuição em excesso, sumário mal-feito).

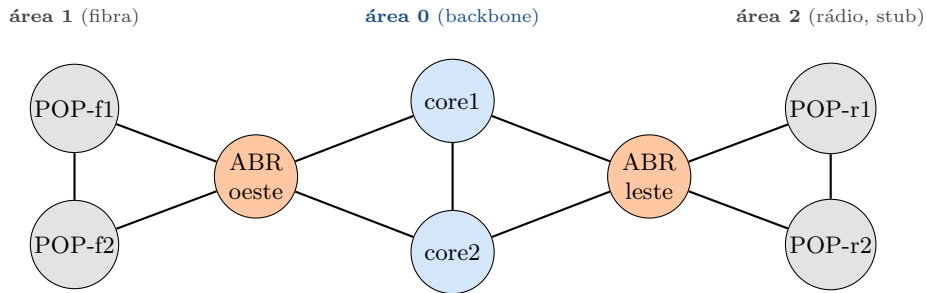
O Capítulo 42 deixou a rede rodando OSPF com **tudo na área 0** — o certo enquanto são poucos POPs. Mas o desafio 4 daquele capítulo já avisou: 40 roteadores numa área única viram um mapa gigante que todos recalculam a cada flap, e o ruído de um POP vira problema de todos. Este capítulo resolve isso com o instrumento que o OSPF foi feito para usar: **áreas**. De quebra, fecha as brechas que uma rede de produção não pode ter — autenticação e filtros.

43.1 Por que múltiplas áreas

Numa área, cada roteador guarda o **grafo completo** e roda Dijkstra sobre ele. O custo cresce com o tamanho: mais LSAs para inundar, mais memória para o mapa, mais CPU a cada mudança. A solução do OSPF é **hierarquia**: dividir a rede em áreas que só trocam entre si um **resumo**, não a topologia inteira.

O roteador que fica na fronteira de duas áreas é o **ABR** (*Area Border Router*). Ele participa das duas, guarda o mapa detalhado de cada uma e **anuncia para uma área apenas os prefixos da outra** (LSAs tipo 3, *summary*) — não a topologia crua. Assim, um flap dentro da área 2 gera recálculo **só na área 2**; para a área 0, o ABR continua anunciando o mesmo prefixo resumido, estável. A falha fica **confinada**.

A regra inviolável: **toda área não-backbone precisa tocar a área 0**. O backbone é o centro da estrela; áreas não conversam entre si direto, sempre via área 0. Uma área que perde contato com a 0 fica isolada (resolve-se com *virtual-link*, um remendo que o bom projeto evita).



Cada área é um domínio de inundação próprio. O ABR fica na fronteira, participa da área 0 e da sua área lateral, e resume os prefixos de um lado para o outro. Um flap na área 2 não faz a área 1 recalcular nada — o ABR leste continua anunciando o mesmo resumo.

Figura 43.1

43.2 Tipos de área: encolhendo a tabela da ponta

Um POP de rádio na ponta da rede não precisa conhecer **cada** prefixo externo do provedor; ele só precisa saber “para sair, vá pelo ABR”. O OSPF oferece tipos de área que trocam detalhe por simplicidade:

- **Normal:** recebe tudo — prefixos internos (tipo 1/2), sumários de outras áreas (tipo 3) e rotas externas redistribuídas (tipo 5);
- **Stub:** o ABR **bloqueia as externas (tipo 5)** e injeta no lugar uma **rota padrão**. O POP para de ver as centenas de rotas de BGP redistribuídas e usa 0.0.0.0/0 → ABR. Tabela menor, CPU menor;
- **NSSA (Not-So-Stubby Area):** como a stub, mas **permite originar** externas localmente (tipo 7, convertidas em tipo 5 pelo ABR). É a escolha quando a área de ponta **também** tem redistribuição própria (ex.: rotas estáticas de um cliente ali).

Regra prática de provedor: **áreas de acesso/ponta como stub** (ou NSSA se precisarem redistribuir algo local); **backbone e áreas de trânsito como normais**. Menos LSA na ponta = convergência e memória melhores nos equipamentos mais fracos.

```
# no ABR e em todos os roteadores da área: o tipo precisa BATER nos dois lados
/routing ospf area
add name=acesso-radio area-id=0.0.0.2 instance=isp type=stub
```

Aviso

O `type` da área tem de ser **idêntico em todos os roteadores da área** — se um acha que é **stub** e o vizinho acha que é **normal**, a adjacência **não forma** (o bit E do Hello diverge). É o mesmo tipo de armadilha do MTU no capítulo anterior: divergência silenciosa que só aparece no `neighbor print`.

43.3 Autenticação: OSPF de produção não roda “no escuro”

O Capítulo 42 mostrou o ataque: sem autenticação, qualquer um no segmento injeta LSAs e sequestra tráfego. Em produção, **todo enlace interno autentica**. O v7 configura no `interface-template`:

```
/routing ospf interface-template
set [find area=backbone] auth=md5 auth-id=1 auth-key="Fibr@-POP-2026"
```

- `auth=md5` (HMAC-MD5) é o mínimo aceitável; prefira quando o hardware suportar chaves mais fortes. **Nunca** `auth=none` num enlace que sai do rack;
- a chave e o `auth-id` precisam **bater nos dois lados** — de novo, a regra de ouro do OSPF é *simetria*. Chave diferente = adjacência que não fecha.

43.4 Contendo o flapping: temporizadores de SPF e LSA

Um enlace que “pisca” gera uma enxurrada de LSAs; cada um dispara um SPF. Os temporizadores **amortecem** essa rajada, espaçando os recálculos em vez de fazer um por evento:

```
/routing ospf instance
set isp in-filter-chain=ospf-in \
    metric-connected=20 \
    routing-table=main
# temporizadores de SPF/LSA moram no instance (throttle exponencial):
# valor inicial pequeno (reage rápido); teto maior (não recalcula em looping)
```

A ideia: o primeiro evento reage quase de imediato (bom para convergência); eventos em sequência aumentam o intervalo exponencialmente até um teto (bom para conter flapping). Combinado com **áreas** (que já confinam o flap), a rede fica estável mesmo com um enlace ruim num canto.

43.5 Filtrando o que entra e sai do OSPF

O v7 unificou a filtragem em **routing filters** — cadeias de regras (iguais em espírito ao firewall) referenciadas por `in-filter-chain/out-filter-chain`. Dois usos típicos:

```
# NÃO redistribua rotas de gestão/loopbacks de terceiros para dentro do OSPF
/routing filter rule
add chain=ospf-out rule="if (dst in 100.64.100.0/22) { accept }"
add chain=ospf-out rule="if (dst in 10.255.0.0/24) { accept }"
add chain=ospf-out rule="reject"

# NÃO aceite uma rota-padrão vinda de onde não deveria (anti-sequestro)
/routing filter rule
add chain=ospf-in rule="if (dst==0.0.0.0/0 && ospf-type==external) { reject }"
add chain=ospf-in rule="accept"
```

Filtrar na **saída** (redistribuição) evita vaziar o que não é para rotear; filtrar na **entrada** protege contra rota indevida (defesa em profundidade, mesmo com autenticação). O tema volta aprofundado no BGP (Capítulo 45), onde os filtros são a peça central.

! O conceito mais importante do capítulo

Áreas trocam escala por hierarquia: o ABR resume uma área para a outra, e com isso um flap deixa de ser global — vira local. Some a isso **stub/NSSA** (encolhe a tabela da ponta), **autenticação** (fecha o sequestro de rota) e **temporizadores + filtros** (contêm ruído e vazamento), e você tem um OSPF de provedor de verdade. O erro capital é o oposto de tudo isso: **uma área só, sem auth, redistribuindo tudo** — funciona no lab, colapsa em campo. E lembre da regra que não se dobra: **toda área toca a área 0**.

43.6 Laboratório

i Ambiente

Estenda o trio do Capítulo 42 para **quatro** CHRs (Capítulo 5): lab-r1 e lab-r2 formam o **backbone (área 0)**; lab-r1 é **ABR** para a **área 1** (0.0.0.1), onde vive lab-r3; lab-r2 é **ABR** para a **área 2** (0.0.0.2), onde vive lab-r4. Reaproveite os loopbacks 10.255.0.x/32 e enlaces PtP /30 do capítulo anterior, criando as duas novas redes internas (r1-r3 e r2-r4) no VirtualBox.

43.6.1 Comandos passo a passo

Passo 1 — declare as áreas nos ABRs. No lab-r1 (backbone + área 1):

```
/routing ospf area
add name=backbone area-id=0.0.0.0 instance=isp
add name=area1 area-id=0.0.0.1 instance=isp
/routing ospf interface-template
add area=backbone networks=172.16.9.0/30 type=ptp auth=md5 auth-id=1 auth-key="lab-backbone"
add area=area1 networks=172.16.9.12/30 type=ptp auth=md5 auth-id=1 auth-key="lab-area1"
```

No lab-r2, análogo: backbone + area2 (0.0.0.2). Nos roteadores de ponta (r3, r4), só a sua área lateral.

Passo 2 — torne a área 2 uma stub. No lab-r2 e no lab-r4:

```
/routing ospf area set [find name=area2] type=stub
```

Passo 3 — veja as adjacências e o tipo das rotas. Em qualquer nó:

```
/routing ospf neighbor print
/ip route print where ospf
```

Nos roteadores da área 1 (normal), as rotas da área 2 aparecem como **inter-area** (tipo 3). No lab-r4 (área stub), confirme que **não há** externas e que existe uma **rota padrão** aprendida via OSPF apontando para o ABR.

Passo 4 — prove o confinamento do flap. No lab-r3 (área 1), suba e derrube o loopback repetidamente:

```
/interface disable lo ; :delay 1s ; /interface enable lo
```

Observe (com `/routing ospf lsa print` e o log) que os roteadores da **área 2 não recalculam** — o ABR lab-r2 continua anunciando o mesmo resumo. O ruído da área 1 ficou na área 1.

Passo 5 — teste a autenticação. Troque a `auth-key` de **um** lado do enlace de backbone e veja a adjacência cair:

```
/routing ospf interface-template set [find area=backbone] auth-key="errada"
/routing ospf neighbor print
```

A adjacência some. Restaure a chave e ela volta — a prova de que o enlace só fala com quem tem a chave.

43.6.2 Verificação

1. `neighbor print` mostra Full no backbone e em cada área lateral;
2. Roteador da área normal enxerga prefixos da outra área como inter-area; roteador da área stub vê **rota padrão** e nenhuma externa;
3. Flap na área 1 **não** dispara SPF na área 2;
4. Chave de autenticação divergente derruba a adjacência.

43.6.3 Desafios

1. Você criou a **area3** (0.0.0.3) num POP novo, mas ela só tem enlace para outra área lateral (a **area1**), **não** para a área 0. As rotas não propagam. Explique por quê e liste as duas saídas — a “certa” (reprojeto físico/lógico) e o remendo (**virtual-link**), com o custo de cada uma.
2. Um cliente corporativo na sua área stub precisa **originar** algumas rotas estáticas para dentro do OSPF. A área stub não deixa. Que tipo de área resolve isso sem abrir mão do resto dos benefícios, e por quê?
3. Depois de tornar uma área **stub**, **metade** dos roteadores dela perdeu a adjacência. Qual é a causa mais provável (pense no bit E do Hello) e como você confirma?
4. Sua rede tem 200 prefixos de cliente redistribuídos no OSPF e a CPU dos POPs de ponta vive alta. Descreva **duas** mudanças de projeto — uma no tipo de área, outra em filtros/sumarização — que derrubam a carga sem perder alcançabilidade.

Solução dos desafios

1. Toda área não-backbone precisa de **contiguidade com a área 0**; a **area3** tocando só a **area1** fica órfã — o OSPF não roteia área área sem passar pela 0. Saída certa: dar à **area3** um enlace (físico ou lógico) até um ABR do backbone, ou reclassificá-la como parte de uma área existente. Remendo: um **virtual-link** através da **area1** (que vira *transit area*), tunelando a **area3** até a 0 — funciona, mas é frágil, depende da **area1** estar sempre de pé e complica o diagnóstico. Prefira sempre o reprojeto.
2. **NSSA**. Ela mantém o bloqueio das externas *vindas de fora* (tabela enxuta como a stub), mas permite **originar** externas locais como LSA tipo 7, que o ABR converte em tipo 5 para o resto da rede. É exatamente o caso “área de ponta que também redistribui algo próprio”.
3. O **type da área não bate** em todos os roteadores: os que você marcou como **stub** têm o bit E=0 no Hello; os que ficaram **normal** têm E=1 — e Hello com E divergente **não forma adjacência**. Confirme com `/routing ospf area print` em cada nó (todos têm de dizer **type=stub**) e com o `neighbor print` mostrando quem caiu.
4. (a) Transformar as áreas de ponta em **stub/NSSA**: o ABR troca os 200 prefixos por uma **rota padrão**, e a tabela do POP despenca. (b) **Sumarizar** no ABR: em vez de vaziar 200 rotas específicas, anunciar o **agregado** (ex.: um /16 que cobre os pools) com um *routing filter* na redistribuição — menos LSA, menos SPF, mesma

alcançabilidade. As duas se somam.

43.7 Resumo

- **Múltiplas áreas** trocam escala por hierarquia: o **ABR** resume uma área para a outra e **confina** flaps — mudança dentro de uma área não faz as outras recalcular;
- **Regra inviolável**: toda área não-backbone toca a **área 0**;
- **Tipos de área** encolhem a tabela da ponta: **stub** (sem externas, injeta rota padrão), **NSSA** (stub que *pode* originar externas locais); o **type** tem de ser **idêntico** em toda a área;
- **Autenticação** (`auth=md5`, chave simétrica) fecha o sequestro de rota; **temporizadores de SPF/LSA** amortecem flapping;
- **Routing filters** (`in/out-filter-chain`) evitam vazar e evitam aceitar rota indevida — e **sumarizar** no ABR derruba a carga.

Com OSPF interno sólido, falta o protocolo que conversa com o **mundo de fora**: o BGP. O próximo capítulo (Capítulo 44) abre a conexão com a operadora e o IX.

Bibliografia do capítulo

- Documentação oficial: seção *OSPF* (áreas, NSSA, autenticação) do RouterOS v7 (MikroTik, 2026a);
- Hierarquia de áreas, LSAs e SPF: (Tanenbaum; Feamster; Wetherall, 2021);
- Configuração v7 na prática: (Discher, 2016);
- Projeto OSPF em provedores: (Burgess, 2011).

44 BGP: conceitos e peering

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

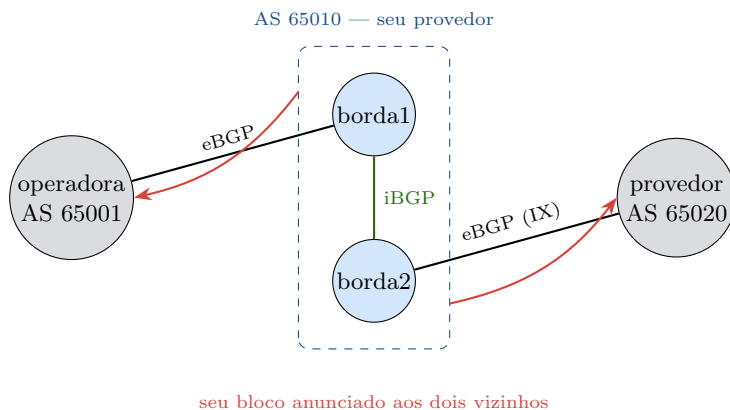
- Explicar o que é um **AS** (Sistema Autônomo), o que é ASN público e por que o BGP é o protocolo que **cola a internet**;
- Distinguir **eBGP** (com a operadora/IX) de **iBGP** (dentro do seu AS) e saber por que os dois existem;
- Entender o **processo de decisão** do BGP (o caminho mais curto em ASNs não é o único critério) e os atributos que você mais usa: **local-pref**, **AS-path**, **MED**;
- Configurar uma sessão BGP no RouterOS v7 (**connection**, **template**) e anunciar seu bloco com segurança;
- Ler o estado de uma sessão (**/routing bgp session print**) e diagnosticar por que ela **não sobe** ou **não troca prefixos**.

Até agora a rede olhava só para dentro: OSPF distribui o que **é seu**. Mas para o provedor existir na internet ele precisa **anunciar seus prefixos ao mundo** e **aprender o caminho para todos os outros** — isso é trabalho do **BGP** (*Border Gateway Protocol*), o protocolo de roteamento **entre** organizações. Enquanto o OSPF pergunta “qual o menor caminho *dentro* da minha rede?”, o BGP pergunta “por qual **vizinho** eu alcanço aquele destino lá fora, e qual política eu aplico a essa escolha?”.

44.1 AS, ASN e a lógica do BGP

A internet é um conjunto de **Sistemas Autônomos** (AS): cada operadora, provedor ou grande empresa é um AS, identificado por um **ASN** (número de 32 bits, atribuído pelo registro regional — no Brasil, o **NIC.br/LACNIC**). Seu provedor, para rodar BGP de verdade, precisa de **ASN próprio e bloco IP próprio** (PI — *provider independent*).

O BGP não pensa em “custo de enlace” como o OSPF. Ele pensa em **caminhos de AS** (*AS-path*): “para chegar no prefixo X, o caminho passa pelos AS 65010 → 65001 → destino”. A partir daí, aplica **política** — e é aí que mora a arte. O BGP é um protocolo de *policy*, não de métrica.



O provedor (AS 65010) fala **eBGP** com dois vizinhos externos — uma operadora de trânsito e um par no IX — e **iBGP** entre seus próprios roteadores de borda, para que os dois concordem sobre o que aprenderam de fora. Anunciar o próprio bloco aos vizinhos é o que coloca o provedor “no mapa” da internet.

Figura 44.1

44.2 eBGP e iBGP: dois sabores, um protocolo

- **eBGP** (*external*): sessão entre **ASNs diferentes** — você e a operadora, você e um par no IX. É por aqui que entra a *default* (ou a tabela cheia) e por onde sai o seu anúncio. TTL curto (vizinho diretamente conectado, em geral);
- **iBGP** (*internal*): sessão entre roteadores do **mesmo AS**. Serve para os seus roteadores de borda **combinarem** o que cada um aprendeu do mundo externo. Regra crucial: o que um roteador aprende por iBGP ele **não repassa** a outro peer iBGP (prevenção de loop) — por isso iBGP clássico exige **malha completa** (*full mesh*) ou um **route-reflector** quando a rede cresce.

Uma sutileza que derruba iniciantes: o iBGP **não instala** rota se o *next-hop* não for alcançável pela IGP. É por isso que **OSPF e BGP trabalham juntos**: o OSPF (capítulo anterior) leva os loopbacks e enlaces internos, e o iBGP roda **sobre** esses loopbacks. OSPF é o esqueleto; BGP é a pele que toca o mundo.

44.3 Como o BGP decide

Quando o BGP conhece **vários** caminhos para o mesmo prefixo, ele escolhe **um** por uma lista ordenada de critérios. Os que você realmente ajusta no dia a dia:

1. **local-preference** (maior vence): política **de saída** — “prefiro sair pela operadora A”. Vale só dentro do seu AS;
2. **AS-path mais curto** (menos ASNs): o desempate “natural” do BGP;

3. **origem / MED**: o **MED** sugere ao vizinho por qual dos **seus** enlaces ele deveria **entrar** — uma dica, que o outro lado pode ignorar;
4. **eBGP sobre iBGP**, depois menor custo IGP até o next-hop, e por fim critérios de desempate (menor router-id).

Para influenciar o tráfego **que sai**, você mexe em **local-pref** (dentro de casa). Para influenciar o que **entra**, você trabalha o **AS-path prepend** e **communities** (Capítulo 45) — porque a decisão final de entrada é do **outro** AS, não sua.

44.4 Configurando BGP no RouterOS v7

O v7 reescreveu o BGP: some o **instance**, entram **connection** (a sessão) e **template** (parâmetros reutilizáveis). Uma sessão de trânsito eBGP:

```
# template com o essencial do NOSSO lado
/routing bgp template
add name=t-transito as=65010 router-id=10.255.0.1 \
    output.network=203.0.113.0/24 \
    address-families=ip

# a rede que vamos anunciar precisa existir como rota (agregado/estática)
/ip route add dst-address=203.0.113.0/24 type=blackhole comment="agregado anunciado ao BGP"

# a sessão eBGP com a operadora
/routing bgp connection
add name=operadora template=t-transito \
    remote.address=198.51.100.1 remote.as=65001 \
    local.role=ebgp
```

- **as=65010**: seu ASN. **remote.as=65001**: o da operadora — diferente, logo **eBGP**;
- **output.network=203.0.113.0/24**: o bloco que você anuncia. A rota **blackhole** correspondente garante que o prefixo exista na tabela (o BGP só anuncia o que ele tem) — o truque padrão do Capítulo 41;
- **jamais** anuncie sem filtro de saída em produção: o Capítulo 45 trata disso. Aqui, o mínimo para a sessão subir.

Para **iBGP** entre suas bordas, **remote.as** é o seu **próprio** ASN e o **remote.address** é o **loopback** do par (alcançado via OSPF):

```
/routing bgp connection
add name=ibgp-borda2 template=t-transito \
    remote.address=10.255.0.2 remote.as=65010 \
    local.role=ibgp local.address=10.255.0.1
```

44.5 Lendo e diagnosticando a sessão

```
/routing bgp session print
/ip route print where bgp
```

O estado que você quer é **established**. Se travar, o diagnóstico segue um roteiro curto:

- **connect/active sem subir**: problema de **transporte** — não há rota/alcance TCP/179 até o peer (firewall bloqueando, next-hop iBGP sem OSPF), ou o `remote.address` está errado;
- **sobe e cai (idle established)**: `remote.as` divergente, ou temporizadores/multihop mal-configurados;
- **established mas sem prefixos**: **filtro** barrando (in/out), ou você não declarou `output.network`, ou o next-hop dos prefixos recebidos é inalcançável (o clássico iBGP sem IGP).

! O conceito mais importante do capítulo

BGP não é “OSPF do mundo externo” — é **política sobre caminhos de AS**. Ele cola ASNs, e você controla o tráfego com atributos, não com custo: `local-pref` para o que **sai**, `prepend/communities` para influenciar o que **entra**. Duas dependências que, se ignoradas, geram horas de diagnóstico: **iBGP precisa da IGP** (OSPF) para resolver next-hops, e **anúncio sem filtro é acidente esperando acontecer** (vazar prefixo alheio já tirou provedores do ar). O BGP dá poder; o próximo capítulo dá a disciplina.

44.6 Laboratório

i Ambiente

Três CHRs (Capítulo 5) simulando ASNs distintos: **lab-r1** = seu provedor (**AS 65010**), com bloco 203.0.113.0/24; **lab-r2** = operadora de trânsito (**AS 65001**), com um bloco 198.18.0.0/24 para fingir “a internet”; **lab-r3** = seu **segundo roteador de borda** (também AS 65010), para o iBGP. r1–r2 por eBGP; r1–r3 por iBGP sobre loopbacks (rodando OSPF, como no Capítulo 42).

44.6.1 Comandos passo a passo

Passo 1 — eBGP com a “operadora”. No lab-r1:

```
/ip route add dst-address=203.0.113.0/24 type=blackhole
/routing bgp connection
add name=operadora as=65010 router-id=10.255.0.1 \
    remote.address=<ip-r2> remote.as=65001 local.role=ebgp \
    output.network=203.0.113.0/24 address-families=ip
```

No lab-r2, o espelho: as=65001, remote.as=65010, anunciando 198.18.0.0/24.

Passo 2 — a sessão sobe? No lab-r1:

```
/routing bgp session print
```

Espere established. Se ficar em active, cheque alcance TCP/179 (firewall) e o remote.address.

Passo 3 — trocaram prefixos? Em cada lado:

```
/ip route print where bgp
```

O lab-r1 deve ter aprendido 198.18.0.0/24 (via BGP, distância 20) e o lab-r2 deve ter aprendido 203.0.113.0/24. Confirme com um ping entre os blocos.

Passo 4 — iBGP entre suas bordas. Garanta OSPF de pé entre r1 e r3 (loopbacks 10.255.0.1 e .3 alcançáveis). No lab-r1:

```
/routing bgp connection
add name=ibgp-r3 as=65010 remote.address=10.255.0.3 remote.as=65010 \
    local.role=ibgp local.address=10.255.0.1 address-families=ip
```

Espelhe em r3. Confirme que o lab-r3 **aprendeu** o 198.18.0.0/24 que veio da operadora **através** do r1 — a prova de que o iBGP propagou o externo por dentro do AS.

Passo 5 — quebre o next-hop de propósito. No lab-r3, desligue o OSPF por um instante e veja a rota iBGP virar **inativa** (next-hop inalcançável). Religue o OSPF e ela reativa — a dependência OSPF iBGP ao vivo.

44.6.2 Verificação

1. session print mostra established na sessão eBGP e na iBGP;
2. lab-r1 aprende o bloco da operadora e vice-versa; ping cruzado passa;
3. lab-r3 aprende o prefixo externo **via iBGP** (sem falar com a operadora diretamente);
4. Derrubar a IGP torna a rota iBGP inativa; restaurá-la reativa.

44.6.3 Desafios

1. Sua sessão eBGP fica em **active** e nunca sobe. **ping** para o peer funciona. Liste, em ordem de probabilidade, as causas e o comando que confirma cada uma (pense na porta 179 e no **remote.as**).
2. A sessão iBGP está **established**, mas o lab-r3 mostra o prefixo externo como rota **inativa**. Ele não aprendeu errado — está tudo “certo” no BGP. Qual é a peça que falta e por quê?
3. Você tem **dois** links de trânsito (duas operadoras) e quer que **todo o tráfego de saída** prefira a operadora A, deixando a B como reserva. Qual atributo você ajusta, em que sentido, e por que ele (e não o AS-path) é a ferramenta certa?
4. Um estagiário anunciou, sem filtro de saída, **toda a tabela** que o roteador tinha — incluindo prefixos da própria operadora aprendidos por BGP. Descreva o estrago (você virou “trânsito” involuntário) e a regra de ouro de filtro de saída que o teria evitado.

Solução dos desafios

1. (a) **TCP/179 bloqueado** — o **ping** (ICMP) passa, mas o firewall pode barrar a 179; confirme com o contador da regra de firewall. (b) **remote.as divergente** — o que você configurou não bate com o ASN real do peer; confira com o outro lado. (c) **router-id/local.address** ausente ou conflitante. O **session print** costuma trazer a pista no campo de estado/último erro.
2. O **next-hop** é **inalcançável pela IGP**. O iBGP passou o prefixo, mas manteve o next-hop original (o IP da operadora, do outro AS), que o lab-r3 não sabe alcançar por OSPF. Solução: **next-hop-self** no r1 (reescreve o next-hop para o próprio loopback, que a IGP conhece) — a prática padrão em bordas iBGP.
3. **local-preference**, aumentando-o nas rotas aprendidas da operadora A (maior local-pref vence, e ele é avaliado **antes** do AS-path na decisão). O AS-path não serve aqui porque é um atributo que influencia sobretudo o tráfego de **entrada** e depende do que os outros ASNs fazem; **local-pref** é **interno** e decide o que **sai** — exatamente o que o enunciado pede.
4. Ao reanunciar prefixos aprendidos de A para B (e vice-versa), você se ofereceu como **trânsito grátis** entre as duas operadoras: o tráfego delas passaria a atravessar sua rede, estourando seus links e, pior, podendo desviar tráfego de terceiros (vazamento de rota, um incidente sério e público). Regra de ouro: **filtro de saída explícito que só anuncia os SEUS prefixos** (e, no IX, os de seus clientes) — **reject** para todo o resto. É o tema do Capítulo 45.

44.7 Resumo

- **BGP** cola ASNs: cada provedor é um **AS** (ASN + bloco próprios) e o BGP escolhe **caminhos de AS** aplicando **política**, não custo de enlace;
- **eBGP** fala com outro AS (operadora, IX); **iBGP** alinha seus próprios roteadores — e **depende da IGP** (OSPF) para resolver next-hops (**next-hop-self** na borda);

- **Decisão:** `local-pref` (o que **sai**), `AS-path`, `MED` (dica de entrada) — para o que **entra**, trabalha-se `prepend/communities`;
- No **v7**: `template + connection (local.role=ebgp|ibgp)`; anuncie via `output.network` com a rota (blackhole) correspondente;
- Diagnóstico: `session print (established)`, estados `active` = transporte/ASN, sem prefixos = filtro ou next-hop inalcançável.

Uma sessão que sobe é só o começo: sem **filtros**, BGP é perigoso. O próximo capítulo (Capítulo 45) transforma o peering em política de verdade — o que anunciar, o que aceitar, e como não virar trânsito de ninguém.

Bibliografia do capítulo

- Documentação oficial: seção *BGP* do RouterOS v7 (MikroTik, 2026a);
- Roteamento interdomínio, `AS-path` e política: (Tanenbaum; Feamster; Wetherall, 2021);
- BGP no v7 na prática: (Discher, 2016);
- BGP em provedores: (Burgess, 2011).

45 BGP: filtros, communities e segurança

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Escrever **routing filters** do v7 para BGP: cadeias in/out, ordem das regras, **accept/reject** e ações que **modificam** atributos;
- Construir o **filtro de saída mínimo seguro** (anuncie só o que é seu) e o **filtro de entrada** (não aceite bogons nem prefixo seu de volta);
- Usar **communities** para marcar rotas e disparar políticas (blackhole na operadora, no-export, seleção de saída);
- Fazer **AS-path prepend** para influenciar o tráfego de **entrada** e balancear dois trânsitos;
- Entender, em nível conceitual, **RPKI/ROA** e por que a validação de origem virou obrigatória.

O Capítulo 44 subiu as sessões e já cravou o aviso: **BGP sem filtro é perigoso**. Anunciar demais transforma você em trânsito de graça; aceitar demais deixa qualquer vizinho sequestrar destinos. Este capítulo é sobre **política** — a parte do BGP que realmente importa em produção. No v7, tudo passa pelos **routing filters**, e é neles que se constrói um provedor que não sai no noticiário por vazamento de rota.

45.1 Routing filters: firewall de rotas

Um *routing filter* é uma **cadeia** de regras avaliadas em ordem, do topo para baixo, com a mesma lógica *first-match* do firewall (Capítulo 16): a primeira regra que casa e decide (**accept/reject**) encerra. Cada *connection* referencia uma cadeia de entrada (**input.filter**) e/ou de saída (**output.filter**).

```
/routing filter rule
add chain=bgp-out-transito rule="if (dst==203.0.113.0/24) { accept }"
add chain=bgp-out-transito rule="reject"
```

A gramática das regras é uma minilinguagem: condições (**if (...)**), ações entre chaves e **accept/reject/return**. Você casa por prefixo (**dst in 100.64.0.0/10**), por AS-path, por community, e **modifica** atributos antes de aceitar:

```
add chain=bgp-in-transito \  
    rule="if (dst in 0.0.0.0/0) { set bgp-local-pref 200; accept }"
```

Aviso

Toda cadeia de filtro **precisa terminar** com uma política explícita. Uma cadeia sem **reject** final tem comportamento de *default* que varia e já causou vazamentos. Regra de ouro: **última linha sempre reject (ou accept) consciente** — nunca deixe a decisão ao acaso.

45.2 O filtro de saída mínimo seguro

Este é o filtro que todo provedor precisa ter antes de ligar um eBGP em produção. A regra é simples de dizer e crítica de cumprir: **anuncie apenas os seus próprios prefixos** (e, se você é trânsito de clientes, os deles) — **reject** para todo o resto.

```
/routing filter rule  
# só os prefixos que são MEUS (o bloco PI do provedor)  
add chain=bgp-out rule="if (dst==203.0.113.0/24) { accept }"  
add chain=bgp-out rule="if (dst==2001:db8:aaaa::/32) { accept }"  
# tudo mais: NÃO anuncie (nada de reanunciar o que aprendi de trânsito!)  
add chain=bgp-out rule="reject"
```

Sem esse **reject**, um prefixo aprendido da operadora A poderia ser reanunciado para a operadora B — e você vira **trânsito involuntário**, o incidente descrito no capítulo anterior. O filtro de saída explícito é a **cinta de segurança** do BGP.

45.3 O filtro de entrada: bogons e o próprio bloco

Na entrada, três higieneas valem para qualquer sessão eBGP:

1. **Rejeitar bogons** — prefixos que **nunca** deveriam vir de fora: RFC1918 (10/8, 172.16/12, 192.168/16), 100.64/10 (CGNAT), default de quem não deveria mandá-la, prefixos muito específicos (mais longos que /24 em IPv4);
2. **Rejeitar o seu próprio bloco de volta** — se alguém te anuncia o 203.0.113.0/24, é erro ou ataque;
3. **Marcar o que entra** com `local-pref` conforme a política (trânsito com pref menor que peering no IX, por exemplo).


```
/routing filter rule
add chain=bgp-in rule="if (dst in 10.0.0.0/8)      { reject }"
add chain=bgp-in rule="if (dst in 172.16.0.0/12)   { reject }"
add chain=bgp-in rule="if (dst in 192.168.0.0/16)  { reject }"
add chain=bgp-in rule="if (dst in 100.64.0.0/10)   { reject }"
add chain=bgp-in rule="if (dst==203.0.113.0/24)    { reject }"
add chain=bgp-in rule="if (dst-len > 24)          { reject }"
add chain=bgp-in rule="accept"
```

45.4 Communities: etiquetas que disparam política

Uma **community** é uma etiqueta (ASN:valor) que você prende numa rota para **sinalizar intenção**. Dois papéis:

- **Você marca** para organizar sua própria política (ex.: 65010:100 = “aprendi no IX”, 65010:200 = “aprendi de trânsito”) e depois decide por community nos filtros;
- **A operadora publica** communities que **você usa** para pedir ações a ela: a mais famosa é a **blackhole community** (RFC 7999, 65535:666), que diz “descarte tudo para este /32” — a defesa padrão contra DDoS: você anuncia o IP atacado com a community de blackhole e a operadora **dropa o ataque na borda dela**, antes de chegar no seu link.

```
# marcar na entrada
add chain=bgp-in-ix rule="set bgp-communities 65010:100; accept"

# pedir blackhole de um /32 sob ataque para a operadora
/ip route add dst-address=203.0.113.66/32 type=blackhole
add chain=bgp-out rule="if (dst==203.0.113.66/32) { set bgp-communities 65535:666; accept"
```

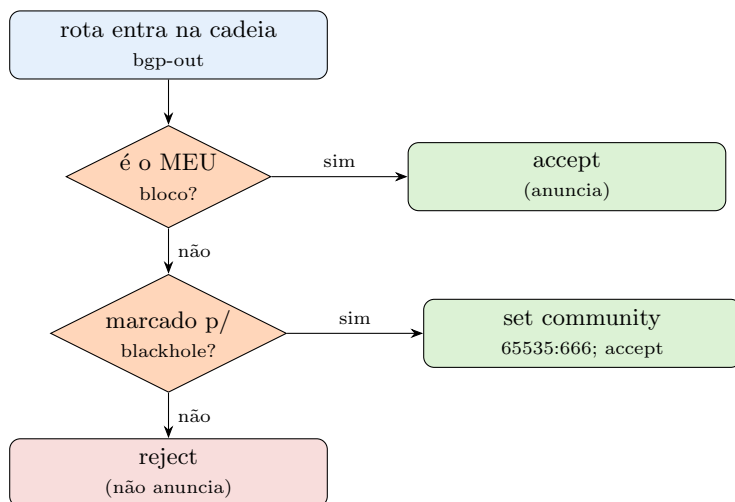
no-export é outra community bem útil: marca uma rota para **não sair** do AS vizinho (fica no peering, não vaza adiante).

45.5 AS-path prepend: influenciando o tráfego de entrada

Você controla o que **sai** com **local-pref**. Mas o tráfego que **entra** é decidido pelos **outros** ASNs — e o AS-path é a alavanca. Ao **repetir seu próprio ASN** no anúncio de um dos links (prepend), você o faz parecer “mais distante”, e os outros passam a preferir o outro link para chegar até você:

```
# anunciar pela operadora B com o AS-path alongado -> entrada prefere A
add chain=bgp-out-opB \
    rule="if (dst==203.0.113.0/24) { set bgp-path-prepend 2; accept }"
```

bgp-path-prepend 2 acrescenta seu ASN duas vezes. É a ferramenta de engenharia de tráfego de **entrada** mais simples — grosseira, mas funciona, e é o que se usa para tirar carga de um link cheio.



Um filtro de saída lido de cima para baixo: aceita o próprio bloco, trata o caso especial (blackhole com community) e **rejeita todo o resto**. É a estrutura que impede o vazamento de rota — a decisão nunca “escapa” para um default implícito.

Figura 45.1

45.6 RPKI e ROA: validação de origem (conceitual)

Filtro de prefixo protege você; **RPKI** protege a internet de você (e de erros alheios). Um **ROA** (*Route Origin Authorization*) é um registro assinado que diz “o prefixo P só pode ser originado pelo AS N”. Roteadores com **validação RPKI** classificam anúncios recebidos como **valid**, **invalid** ou **unknown**, e a boa prática é **rejeitar os invalid** — barrando sequestros de rota por origem incorreta (o famoso caso em que um AS anuncia prefixo alheio, por erro ou ataque, e desvia tráfego global).

No RouterOS v7 há suporte a RPKI (sessão com um *validator* e uso do resultado nos filtros). O detalhe operacional foge do escopo aqui; o que você **precisa** levar: **crie o ROA do seu bloco** no sistema do LACNIC/NIC.br e **rejeite invalids** nas suas sessões. É barato e evita que seu prefixo seja sequestrado — e que você aceite o sequestro de outro.

! O conceito mais importante do capítulo

O poder do BGP é político, e a política mora nos **filtros**. Três peças salvam o provedor: **saída = só o que é meu** (**reject** no fim, a cinta de segurança contra vazamento); **entrada = higiene** (bogons, seu bloco de volta, **dst-len** grande — fora); e **RPKI** (crie o ROA, rejeite invalids). Communities e prepend são o ajuste fino — blackhole

contra DDoS, prepend para balancear entrada. Quem liga eBGP sem o filtro de saída mínimo não configurou BGP: armou uma bomba de vazamento.

45.7 Laboratório

i Ambiente

Reaproveite a topologia do Capítulo 44: **lab-r1** (AS 65010, seu provedor, bloco 203.0.113.0/24) em eBGP com **lab-r2** (AS 65001, operadora). Adicione um **terceiro** vizinho eBGP em lab-r1 simulando uma **segunda operadora** — **lab-r3** (AS 65002) — para exercitar seleção de saída e prepend.

45.7.1 Comandos passo a passo

Passo 1 — o filtro de saída mínimo. No lab-r1, crie a cadeia e prenda-a nas duas sessões de trânsito:

```
/routing filter rule
add chain=bgp-out rule="if (dst==203.0.113.0/24) { accept }"
add chain=bgp-out rule="reject"
/routing bgp connection set [find name=operadora] output.filter=bgp-out
```

Passo 2 — prove que o vazamento parou. Antes do filtro, anuncie de propósito uma rota extra (add dst-address=198.18.0.0/24 type=blackhole) e veja que ela **não** aparece mais no advertisements para o vizinho depois do reject:

```
/routing bgp advertisements print where peer=operadora
```

Só o 203.0.113.0/24 deve sair. O 198.18.0.0/24 foi barrado.

Passo 3 — filtro de entrada com bogons. Aplique a cadeia bgp-in (bogons + seu bloco de volta) na sessão. No lab-r2, tente anunciar 192.168.50.0/24 e 203.0.113.0/24 de volta:

```
/routing bgp connection set [find name=operadora] input.filter=bgp-in
/ip route print where bgp
```

Nenhum dos dois deve entrar na tabela do lab-r1.

Passo 4 — community de blackhole. Coloque um /32 sob “ataque” em blackhole e marque-o para a operadora:

```
/ip route add dst-address=203.0.113.66/32 type=blackhole
/routing filter rule
add chain=bgp-out place-before=0 \
    rule="if (dst==203.0.113.66/32) { set bgp-communities 65535:666; accept }"
```

Confirme no `advertisements` que o /32 sai **com** a community.

Passo 5 — prepend na segunda operadora. Na sessão com lab-r3 (AS 65002), alongue o AS-path do seu anúncio e compare o AS-path visto pelos vizinhos:

```
/routing filter rule
add chain=bgp-out-r3 rule="if (dst==203.0.113.0/24) { set bgp-path-prepend 2; accept }"
```

No lab-r3, `/ip route print detail where bgp` deve mostrar o 203.0.113.0/24 com o AS 65010 **repetido** — logo, ele preferirá chegar até você pela operadora lab-r2.

45.7.2 Verificação

1. Após o filtro de saída, só o seu bloco é anunciado; a rota extra some do `advertisements`;
2. Bogons e o próprio bloco de volta **não** entram pela sessão;
3. O /32 de blackhole sai com a community **65535:666**;
4. O anúncio via segunda operadora mostra AS-path com prepend.

45.7.3 Desafios

1. Você aplicou o filtro de saída, mas esqueceu o **reject** final da cadeia. O que acontece com um prefixo que **não casa** nenhuma regra, e por que isso pode virar um vazamento? Escreva a linha que faltou.
2. Um cliente seu (que você faz trânsito) precisa que **você** anuncie o bloco /24 dele para suas operadoras. Como o filtro de saída muda, e que cuidado de **segurança** (validação) você toma antes de anunciar prefixo de terceiro?
3. Sob DDoS volumétrico de 20 Gbps num único IP, seu link de 10 Gbps satura e **derruba todos** os clientes. Explique passo a passo como a **blackhole community** resolve, onde o tráfego é descartado, e qual é o “preço” dessa mitigação (o que acontece com o IP atacado).
4. Você quer que o tráfego de **entrada** prefira a operadora A, mas **local-pref** não muda nada no vizinho. Explique por que **local-pref** é a ferramenta errada aqui e qual atributo faz o serviço — e por que ele é, ainda assim, apenas uma “sugestão forte”.



Solução dos desafios

1. Sem **reject** final, o prefixo que não casa cai no comportamento *default* da cadeia — que não é garantidamente “negar”. Na prática, uma cadeia incompleta pode **aceitar** o que deveria barrar, reanunciando rota aprendida de trânsito e causando vazamento. A

linha que faltou: `add chain=bgp-out rule="reject"` como **última** regra.

2. O filtro de saída ganha uma regra `accept` para o /24 do cliente (antes do `reject`). **Antes** de anunciar, valide que o bloco é **mesmo** do cliente e que ele autorizou você a originá-lo: cheque o **ROA/RPKI** (o ROA deve permitir seu AS ou o dele conforme o desenho) e os registros de rota (IRR). Anunciar prefixo de terceiro sem essa validação é como assinar em nome dos outros — e é assim que sequestros acidentais acontecem.

3. Você cria uma rota /32 para o IP atacado e a anuncia à operadora **com a community 65535:666** (blackhole). A operadora, ao receber, instala um descarte para esse /32 **na borda dela** — o tráfego de ataque é jogado fora **antes** de entrar no seu link, que volta a respirar. O preço: aquele IP específico fica **inacessível** (você sacrifica um endereço para salvar todos os outros clientes). É mitigação de contenção, não de continuidade do serviço atacado.

4. `local-pref` só existe **dentro** do seu AS — ele nunca sai nos anúncios, então o vizinho jamais o vê. Para influenciar **entrada**, a alavanca é o **AS-path prepend** (alongar o caminho pelo link que você quer despriorizar) e/ou communities de *traffic engineering* que a operadora publique. Ainda assim é só uma “sugestão forte”: a decisão final é do outro AS, que pode ter política própria (`local-pref` dele) que ignore seu `prepend`.

45.8 Resumo

- **Routing filters** (v7) são o firewall de rotas: cadeias *first-match* com `accept/reject` e ações que **modificam** atributos; **sempre** termine com política explícita;
- **Filtro de saída mínimo**: anuncie **só o que é seu**, `reject` no resto — a cinta de segurança contra vazamento/trânsito involuntário;
- **Filtro de entrada**: barre **bogons**, seu bloco de volta e prefixos longos demais; marque `local-pref` por política;
- **Communities** sinalizam intenção: 65535:666 (blackhole anti-DDoS), `no-export`, tags próprias; **prepend** influencia o tráfego de **entrada**;
- **RPKI/ROA**: crie o ROA do seu bloco e **rejeite invalids** — origem validada barra sequestro de rota.

Roteamento dinâmico dominado — do enlace interno ao peering global. O próximo capítulo (Capítulo 46) fecha o volume com **VRP** e as boas práticas de tabelas de roteamento no v7, o alicerce para separar serviços (gerência, clientes, trânsito) numa mesma caixa.

Bibliografia do capítulo

- Documentação oficial: *Routing filters*, *BGP* e *RPKI* do RouterOS v7 (MikroTik, 2026a);
- Política de roteamento e communities: (Tanenbaum; Feamster; Wetherall, 2021);
- Filtros e segurança BGP no v7: (Discher, 2016);

- Boas práticas de peering em provedores: (Burgess, 2011).

46 VRF e boas práticas de roteamento no v7

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que é uma **VRF** (roteador virtual dentro do roteador) e quando ela resolve um problema que sub-redes e VLANs não resolvem;
- Distinguir **VRF** de **routing tables** (/routing table) e de **routing rules** (/routing rule) no RouterOS v7;
- Criar uma VRF, colocar interfaces nela e vaziar rotas entre VRFs (*route leaking*) com controle;
- Aplicar **policy routing** com **routing-mark**/regras para direcionar tráfego por origem/serviço;
- Fechar o volume com um **checklist** de boas práticas de roteamento no v7 (loopbacks, IGP+BGP, filtros, tabelas).

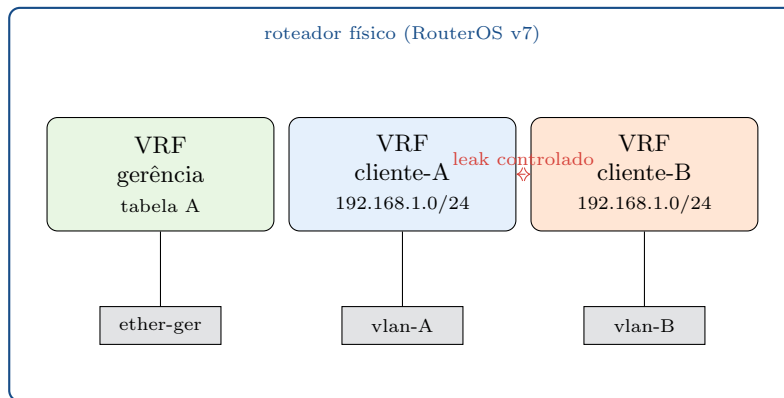
O volume construiu o roteamento do provedor camada por camada: estáticas, OSPF, BGP e filtros. Falta a peça que **separa mundos** dentro de um mesmo equipamento: a **VRF** (*Virtual Routing and Forwarding*). Ela deixa uma caixa se comportar como **vários roteadores independentes**, cada um com sua tabela — indispensável quando gerência, clientes e trânsito precisam conviver sem se enxergar.

46.1 O problema que a VRF resolve

VLANs separam **camada 2**; firewall separa **quem fala com quem**. Mas os dois compartilham a **mesma tabela de rotas**. Há casos em que isso não basta:

- **Sobreposição de endereços**: dois clientes usam 192.168.1.0/24 no mesmo BNG. Numa tabela única, é impossível — o roteador não sabe qual 192.168.1.10 é qual;
- **Isolamento forte de gerência**: você quer uma rede de administração que **fisicamente não alcança** a internet do cliente, nem por engano de firewall;
- **Serviços L3 multi-tenant**: entregar a clientes corporativos redes privadas isoladas sobre o mesmo backbone (a base de uma L3VPN/MPLS, Volume 11).

A **VRF** dá a cada contexto sua **própria tabela de rotas e seu próprio espaço de encaminhamento**. 192.168.1.10 na VRF-clienteA e o mesmo IP na VRF-clienteB são destinos **diferentes** — porque vivem em roteadores virtuais diferentes.



Três VRFs numa mesma caixa: cliente-A e cliente-B reusam 192.168.1.0/24 **sem conflito**, porque cada VRF tem tabela própria. O tráfego só cruza de uma para outra por **route leaking** explícito — o padrão é o isolamento total.

Figura 46.1

46.2 VRF, routing tables e routing rules no v7

O v7 tem três conceitos parecidos que confundem — vale separá-los:

- **/ip vrf**: cria o roteador virtual e **associa interfaces** a ele. É isolamento de *forwarding* — cada VRF tem sua FIB;
- **/routing table**: uma tabela nomeada com **fib** (para instalar rotas). Toda VRF tem uma tabela associada, mas você também usa tabelas **sem VRF** para *policy routing* (mandar tráfego marcado por uma tabela alternativa);
- **/routing rule**: decide **qual tabela** consultar para um pacote, por critérios (origem, interface, **routing-mark**). É o equivalente v7 das antigas *routing rules* do v6.

Regra mental: **VRF isola**; **routing table + rule direciona**. VRF é para “estes mundos não se enxergam”; policy routing é para “este tráfego sai por outro caminho”.

46.3 Criando e usando uma VRF

```
# 1) a VRF e suas interfaces
/ip vrf
add name=gerencia interfaces=ether5

# 2) endereço e rota DENTRO da VRF (note routing-table=gerencia)
/ip address add address=10.99.0.1/24 interface=ether5
/ip route add dst-address=0.0.0.0/0 gateway=10.99.0.254%gerencia routing-table=gerencia
```


A partir daí, **ether5** vive num roteador virtual próprio: seu 0.0.0.0/0 é **outro**, isolado da tabela **main**. Um ping originado na gerência **não** usa a rota da internet do cliente — o isolamento é de encaminhamento, não de firewall.

Para serviços do próprio roteador (DNS, ping, SSH) **saírem por uma VRF**, muitos deles aceitam `vrf=` ou `routing-table=`; caso contrário, usa-se *route leaking*.

46.4 Route leaking: vazar com controle

Isolamento total é o padrão, mas às vezes duas VRFs **precisam** de um caminho pontual (a gerência precisa alcançar um servidor central na VRF de trânsito). O vazamento é feito com rotas que apontam para outra tabela/VRF, sempre **explícito e mínimo**:

```
# levar só a rota do servidor central para dentro da VRF gerencia
/ip route add dst-address=203.0.113.10/32 \
    gateway=<nexthop-na-vrf-transito>@main routing-table=gerencia
```

A sintaxe `@tabela/%interface` diz ao v7 onde resolver o next-hop. A disciplina aqui é a mesma dos filtros de BGP: **vaze o mínimo**, nomeie o propósito no `comment`, e nunca “abra tudo” para resolver um caso pontual — isso mata o motivo de existir da VRF.

Aviso

Colocar uma interface numa VRF **muda a tabela** que o tráfego dela usa. Configurações que assumiam a tabela **main** (rota padrão, NAT, serviços) param de funcionar até serem replicadas na VRF. Migre uma interface para VRF com **console/acesso alternativo** garantido — é fácil se trancar para fora.

46.5 Policy routing: tráfego por caminho, sem VRF

Nem todo direcionamento precisa de VRF. Para “clientes do plano X saem pela operadora B”, basta **marcar** o tráfego (mangle, 4) e mandá-lo por uma **tabela alternativa** via *routing rule*:

```
/routing table add name=via-opB fib
/ip route add dst-address=0.0.0.0/0 gateway=<opB> routing-table=via-opB
/routing rule add src-address=100.64.8.0/22 action=lookup table=via-opB
```

Aqui não há isolamento de VRF — só uma **segunda decisão de rota** para um tráfego selecionado. É o que sustenta ECMP/PCC e failover multi-link do Volume 14, e o balanceamento de trânsito do Capítulo 45.

46.6 Checklist de boas práticas de roteamento no v7

Fechando o volume, o que separa um roteamento de provedor sólido de um frágil:

- **Loopbacks para tudo:** router-id de OSPF/BGP e sessões iBGP sobre /32 de loopback — sobrevivem à queda de uma interface física;
- **Camadas na ordem certa:** IGP (OSPF) carrega a **infraestrutura** (loopbacks, enlaces); BGP carrega **prefixos de cliente/internet**. Não misture — não jogue rota de cliente no OSPF em escala;
- **Filtros sempre:** OSPF com filtros de redistribuição; BGP com filtro de **saída mínimo** e de **entrada** (bogons/RPKI). Nenhuma sessão eBGP sem filtro;
- **Tabelas e VRFs com propósito nomeado:** cada `routing table/vrf` com `comment` do porquê; route leaking mínimo;
- **check-gateway/recursivo** onde a detecção de falha importa (Capítulo 41), e temporizadores de SPF/BGP ajustados contra flapping;
- **Documentar o desenho:** ASN, blocos, áreas OSPF, comunidades — o Volume 15 retoma isso no desenho de rede fim a fim.

! O conceito mais importante do capítulo

VRF isola; policy routing direciona — são respostas a perguntas diferentes. Use VRF quando dois mundos **não podem se enxergar** (gerência, multi-tenant, endereços sobrepostos); use `routing table + rule` quando o mesmo mundo precisa **sair por caminhos diferentes**. Os dois compartilham a mesma disciplina do volume inteiro: **isolamento é o padrão, vazamento é exceção explícita e mínima**. Roteamento de provedor não é sobre fazer o pacote chegar — é sobre controlar, com política, por onde ele chega.

46.7 Laboratório

i Ambiente

Um CHR (Capítulo 5) com três interfaces internas: `ether5` (gerência), `ether6` (cliente-A) e `ether7` (cliente-B). Um segundo CHR faz de “servidor central” na tabela `main`. O objetivo é isolar gerência e provar sobreposição de endereços entre A e B — tudo numa caixa só.

46.7.1 Comandos passo a passo

Passo 1 — VRF de gerência isolada.

```
/ip vrf add name=gerencia interfaces=ether5
/ip address add address=10.99.0.1/24 interface=ether5
/ip route add dst-address=0.0.0.0/0 gateway=10.99.0.254%gerencia routing-table=gerencia
```

Confirme que a tabela **main** **não** tem essa rota: `/ip route print where routing-table=gerencia` mostra a rota; `/ip route print (main)`, não.

Passo 2 — sobreposição de endereços. Ponha A e B em VRFs distintas e dê a **ambas** o mesmo 192.168.1.0/24:

```
/ip vrf add name=cli-a interfaces=ether6
/ip vrf add name=cli-b interfaces=ether7
/ip address add address=192.168.1.1/24 interface=ether6
/ip address add address=192.168.1.1/24 interface=ether7
```

Sem VRF isso seria rejeitado (endereço duplicado); **com** VRF, o v7 aceita — cada 192.168.1.1 vive na sua tabela.

Passo 3 — prove o isolamento. De um host na cli-a, tente pingar um host na cli-b (mesmo IP de rede). **Não** deve alcançar — são roteadores virtuais diferentes.

Passo 4 — route leaking controlado. Leve **só** o servidor central (203.0.113.10/32) para dentro da VRF de gerência:

```
/ip route add dst-address=203.0.113.10/32 gateway=<nexthop>@main routing-table=gerencia
```

De um host na gerência, agora o ping ao servidor central passa — mas **nada mais** da **main** está acessível. Vazamento mínimo, como manda a regra.

Passo 5 — policy routing sem VRF. Na tabela **main**, mande um bloco de clientes sair por um gateway alternativo:

```
/routing table add name=via-opB fib
/ip route add dst-address=0.0.0.0/0 gateway=<opB> routing-table=via-opB
/routing rule add src-address=100.64.8.0/22 action=lookup table=via-opB
```

Com `/tool traceroute` de um IP dentro e fora do 100.64.8.0/22, observe caminhos de saída diferentes — sem nenhuma VRF envolvida.

46.7.2 Verificação

1. A rota padrão da gerência existe **só** na tabela **gerencia**;
2. O mesmo 192.168.1.0/24 coexiste em duas VRFs sem erro de duplicado;
3. Hosts de cli-a e cli-b **não** se alcançam;
4. Após o leak, a gerência alcança **só** o servidor central;
5. O bloco marcado sai por outro gateway (policy routing) sem VRF.

46.7.3 Desafios

1. Depois de mover **ether5** para a VRF de gerência, você perdeu o acesso ao roteador por essa interface. Explique por que (pense em qual tabela o tráfego de resposta passou a usar) e como você teria evitado o “auto-trancamento”.
2. A gerência precisa resolver DNS por um servidor na **main**. O **ping** ao IP do DNS (após leak) funciona, mas a resolução de nomes falha. Qual é a peça que ainda falta e por quê?
3. Quando escolher **VRF** e quando escolher **routing table + rule** para o requisito “clientes do plano premium saem por outro trânsito”? Justifique pela pergunta que cada ferramenta responde.
4. Um cliente corporativo quer uma rede privada isolada entre duas filiais, sobre o seu backbone, sem enxergar nem ser enxergado pela internet. Descreva por que **VRF** é a base e qual tecnologia do Volume 11 a estende de um roteador para **toda a rede**.

Solução dos desafios

1. Ao entrar na VRF, **ether5** passou a usar a tabela **gerencia**, que (antes de você configurar) não tinha rota de volta para o seu host de administração — o pacote de resposta não sabia sair. Evita-se **configurando a rota/serviço na VRF antes** (ou junto) da migração, e sempre com um **acesso alternativo** garantido (outra interface na **main**, console) para não depender da interface que está migrando.
2. O **serviço de DNS do roteador** precisa saber usar a rota vazada / a VRF certa para alcançar o servidor. O **ping** funciona porque é tráfego de trânsito comum sujeito às rotas; a resolução do **próprio roteador** parte de um serviço que, por padrão, usa a tabela **main**. Falta apontar o resolvedor para sair pela rota correta (leak do /32 do DNS e o serviço configurado para usá-la), ou rodar a resolução dentro do contexto da VRF.
3. Pergunte “os mundos precisam se **enxergar**?”. Se o premium é o mesmo cliente/rede que só precisa **sair por outro caminho** — sem isolamento, sem endereços sobrepostos — a resposta é **routing table + rule** (policy routing): mais simples, menos coisa para manter. **VRF** só se houver requisito de **isolamento** (não devem alcançar os demais) ou **sobreposição de endereços**. Ferramenta pesada para problema leve vira dívida operacional.
4. A VRF dá o isolamento **local**: a rede do cliente vive numa tabela própria, invisível para o resto. Mas uma VRF sozinha morre no roteador onde foi criada. Para levar esse isolamento **por todo o backbone** — filial A e filial B em roteadores distintos, no mesmo contexto privado — usa-se **L3VPN sobre MPLS** (Volume 11), que carrega as VRFs entre PEs com labels. VRF é o tijolo; MPLS é a argamassa que a espalha pela rede.

46.8 Resumo

- **VRF** = roteador virtual dentro do roteador: tabela e encaminhamento próprios, resolve **isolamento forte** e **endereços sobrepostos** que VLAN/firewall não resolvem;
- No v7: **/ip vrf** isola (associa interfaces); **/routing table + /routing rule** direcionam (policy routing) — perguntas diferentes, ferramentas diferentes;
- **Route leaking** conecta VRFs de forma **explícita e mínima** (@tabela/%interface); isolamento é o padrão;
- Cuidado operacional: mover interface para VRF muda a tabela dela — garanta acesso alternativo para não se trancar;
- **Checklist do volume**: loopbacks, IGP para infra + BGP para prefixos, filtros sempre, tabelas/VRFs com propósito nomeado.

Fecha-se o Volume 8: a rede do provedor agora roteia com método, do enlace interno ao peering global, com isolamento onde importa. O **Volume 9** ataca o problema que todo provedor enfrenta hoje — o esgotamento do IPv4 (CGNAT) e a transição para **IPv6**.

Bibliografia do capítulo

- Documentação oficial: *VRF*, *Routing tables* e *Routing rules* do RouterOS v7 (MikroTik, 2026a);
- Roteamento virtual e L3VPN (fundamentos): (Tanenbaum; Feamster; Wetherall, 2021);
- VRF e policy routing no v7: (Discher, 2016);
- Aplicação em provedores: (Burgess, 2011).

Parte IX

Volume 9 — CGNAT e IPv6

47 CGNAT e netmap

i Objetivos de aprendizagem

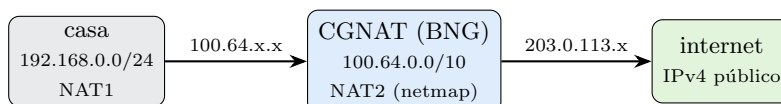
Ao final deste capítulo, você será capaz de:

- Explicar por que o **esgotamento do IPv4** forçou o **CGNAT** (*Carrier-Grade NAT*) e onde ele entra na rede do provedor;
- Usar corretamente o bloco **100.64.0.0/10** (RFC 6598) e entender por que ele existe (e por que **não** é RFC1918);
- Diferenciar **masquerade/srcnat** de **netmap** e implementar CGNAT **determinístico** (assinante IP público + faixa de portas fixa);
- Dimensionar **portas por assinante** e a razão de compartilhamento (*oversubscription*), pesando esgotamento de portas vs. economia de IP;
- Reconhecer o que o CGNAT **quebra** (port-forward do cliente, alguns jogos/P2P) e como mitigar.

O provedor cresceu e a conta não fecha: são **milhares de assinantes** e um punhado de **IPv4 públicos**. Comprar mais IPv4 é caro e escasso. A saída da indústria inteira foi **compartilhar** cada IP público entre dezenas de clientes com um NAT em escala de operadora — o **CGNAT**. Ele é a diferença entre “acabou o IP, não dá para vender” e “seguimos crescendo”. Este capítulo monta um CGNAT que funciona e que atende à lei (o próximo, 15, cuida do registro legal).

47.1 Por que CGNAT e onde ele fica

No Capítulo 13, a LAN privada do cliente saía para a internet via NAT no roteador de borda: **um IP público servia uma casa**. O CGNAT sobe esse conceito uma camada: agora **um IP público serve muitas casas**. Cada assinante recebe um **endereço 100.64.x.x** (nem privado “de casa”, nem público), e o **BNG/roteador de CGNAT** traduz o tráfego de todos eles para um punhado de IPv4 públicos na saída.



NAT duplo: casa → 100.64 → público

O tráfego do assinante passa por **dois** NATs: o da própria casa (privado → 100.64.x.x) e o do provedor (100.64.x.x → IPv4 público compartilhado). O 100.64.0.0/10 é o “espaço de manobra” entre os dois.

Figura 47.1

47.2 O espaço 100.64.0.0/10 (RFC 6598)

Por que não usar 10.0.0.0/8 para endereçar os assinantes? Porque a **casa do cliente já usa** RFC1918 (192.168.x.x, às vezes 10.x). Se o provedor também usasse 10.x entre o CPE/ONU e o BNG, haveria **colisão** com a LAN do cliente — o NAT ficaria ambíguo. A IANA reservou o 100.64.0.0/10 (*Shared Address Space*, RFC 6598) **exatamente** para CGNAT: é um espaço que **não aparece** na LAN doméstica nem na internet pública, dedicado ao trecho assinante operadora.

Regras práticas: **nunca roteie 100.64/10 para a internet; nunca aceite 100.64/10 num peering BGP** (é bogon, Capítulo 45); e **não** o confunda com RFC1918 nas regras de firewall.

47.3 masquerade, srcnat e netmap

O RouterOS oferece três ações de srcnat relevantes aqui:

- **masquerade**: reescreve o *source* para o IP **da interface de saída**, escolhido dinamicamente. Simples, mas o mapeamento assinante→IP público **muda** e é difícil de rastrear;
- **src-nat (to-addresses)**: fixa o IP público de saída, mas ainda distribui portas dinamicamente num pool;
- **netmap**: mapeamento **1:1 de faixas** — pega um bloco de origem e o mapeia, na mesma ordem, para um bloco de destino. É a base do CGNAT **determinístico**.

Para a operação (e para a lei, 15), o que interessa é **previsibilidade**: dado um IP público, uma porta e um instante, saber **qual assinante** era. **masquerade** puro dificulta isso; o CGNAT **determinístico** resolve.

47.4 CGNAT determinístico com netmap

A ideia: dividir cada IP público em **N faixas de portas** e atribuir **cada faixa a um assinante fixo**. Assim, o mapeamento (IP público, faixa de portas) → assinante é **calculável**, sem depender de consultar logs de conexão a cada evento. Menos log, resposta legal mais simples, diagnóstico direto.


```
# mapear a faixa de assinantes 100.64.0.0/28 para 1 IP público,
# fatiando as portas - netmap preserva a ordem do bloco
/ip firewall nat
add chain=srcnat action=netmap \
    src-address=100.64.0.0/28 to-addresses=203.0.113.10 \
    protocol=tcp to-ports=1024-4095 comment="assinantes 0-15 -> portas 1024-4095"
```

Na prática, provedores usam um **script** ou a funcionalidade de CGNAT determinístico para gerar as regras: cada bloco de assinantes recebe um IP público e uma janela de portas contígua. O 15 mostra como isso reduz o volume de log de “toda conexão” para “só a tabela de alocação”.

Aviso

CGNAT é **destrutivo de simplicidade**: uma vez que o assinante está atrás de CGNAT, o **port-forward que ele fazia em casa deixa de funcionar** (o IP público não é dele). Câmeras, DVRs, servidores de jogo caseiros, alguns P2P — tudo que dependia de porta aberta de fora para dentro quebra. Deixe isso claro no contrato e ofereça **IP público dedicado** (ou IPv6, o resto deste volume) para quem precisar.

47.5 Portas por assinante e oversubscription

Cada IPv4 público tem ~**64 mil portas** por protocolo. Se você dá **2.048 portas** a cada assinante, cabem ~**32 assinantes por IP público** (65536 / 2048). Menos portas por cliente = mais clientes por IP (mais economia), porém maior chance de **esgotamento de portas** — um cliente que abre muitas conexões (abas, apps, um torrent) fica sem porta e sente lentidão/falhas intermitentes.

O ajuste é um trade-off:

- **Poucas portas/assinante** (ex.: 512–1024): máxima economia de IP, risco de esgotamento com usuários pesados;
- **Muitas portas/assinante** (ex.: 4096+): folga por cliente, menos clientes por IP;
- **Regra de bolso**: comece com ~**1.000–2.000 portas/assinante** e monitore o esgotamento (contadores/log) antes de apertar.

O conceito mais importante do capítulo

CGNAT é como você **vende internet sem IPv4 suficiente**: um IP público compartilhado por dezenas de assinantes endereçados em 100.64/10. Faça **determinístico** (netmap + faixas de portas fixas) — não por elegância, mas porque isso torna o mapeamento **IP:porta:tempo → assinante calculável**, o que simplifica a operação e a obrigação legal do próximo capítulo. E lembre do preço: CGNAT **quebra** o acesso de fora para dentro do cliente — planeje IP dedicado/IPv6 para quem precisa. A

economia de IPv4 é real; a dívida de complexidade também.

47.6 Laboratório

i Ambiente

Três CHRs (Capítulo 5): **lab-bng** = roteador CGNAT do provedor; **lab-cli** = “casa do assinante” (com sua própria LAN 192.168.88.0/24 e NAT interno), recebendo um 100.64.0.10/24 do lado do provedor; **lab-net** = simulando a internet, com um IP “público” 203.0.113.1 e um serviço para pingar. O lab-bng faz o CGNAT entre 100.64.0.0/24 e o público 203.0.113.10.

47.6.1 Comandos passo a passo

Passo 1 — endereçar o trecho CGNAT. No lab-bng:

```
/ip address add address=100.64.0.1/24 interface=ether2 comment="lado assinantes"
/ip address add address=203.0.113.10/32 interface=ether3 comment="IP publico CGNAT"
```

Passo 2 — o NAT determinístico (netmap). Mapeie um bloco pequeno de assinantes para o IP público, fatiando portas:

```
/ip firewall nat
add chain=srcnat action=netmap src-address=100.64.0.0/28 \
    to-addresses=203.0.113.10 protocol=tcp to-ports=1024-8191 \
    comment="assinantes .0-.15"
add chain=srcnat action=netmap src-address=100.64.0.0/28 \
    to-addresses=203.0.113.10 protocol=udp to-ports=1024-8191
```

Passo 3 — prove a tradução. Do lab-cli (origem 100.64.0.10), gere tráfego para o lab-net e observe no lab-bng:

```
/ip firewall connection print where src-address~"100.64"
```

Você verá a conexão com **reply-src** no 203.0.113.10 e a porta traduzida dentro da faixa definida — o assinante escondido atrás do IP público.

Passo 4 — bogon no lugar certo. Garanta que o 100.64/10 **não** escapa: no firewall filter do lab-bng, confirme que nada roteia esse bloco para fora sem NAT (regra de higiene do Capítulo 45).

Passo 5 — sinta o esgotamento. Reduza a faixa para pouquíssimas portas (`to-ports=1024-1034`) e gere muitas conexões simultâneas do lab-cli. Observe conexões falhando por falta de porta — a demonstração viva do trade-off de oversubscription. Restaure a faixa depois.

47.6.2 Verificação

1. Conexões do assinante saem com *source* = IP público e porta na faixa determinística;
2. O `connection print` casa `100.64.x.x 203.0.113.10:porta`;
3. Estreitar a faixa de portas provoca falhas de conexão sob carga;
4. `100.64/10` não é roteado/anunciado para fora.

47.6.3 Desafios

1. Um cliente reclama que a câmera dele “sumiu” da internet depois que você o migrou para CGNAT — o app não conecta de fora. Explique **exatamente** por que, e liste duas soluções (uma que envolve IPv4, outra que aponta para o resto deste volume).
 2. A autoridade pede: “quem usava o IP `203.0.113.10`, porta de origem 5000, às 21h34?”. Com CGNAT **determinístico**, por que você responde **sem** vasculhar um log gigante de conexões? Descreva o raciocínio de cálculo faixa→assinante.
 3. Você tem 4.000 assinantes e quer no máximo esgotamento raro. Com IPs públicos de 64k portas e a meta de ~2.000 portas/assinante, quantos IPv4 públicos você precisa? Mostre a conta e o que muda se apertar para 1.000 portas.
 4. Por que usar **masquerade** puro no CGNAT é uma má ideia do ponto de vista **legal e operacional**, mesmo que “funcione”? Relacione com o
- 15.

Solução dos desafios

1. A câmera dependia de **port-forward** no roteador de casa, que só funciona quando o IP público **é do cliente**. Atrás de CGNAT, o IP público é **compartilhado** e o provedor não redireciona portas de entrada para cada assinante — logo, conexões **de fora para dentro** não têm como chegar. Soluções: (a) vender/atribuir um **IPv4 público dedicado** ao cliente (sai do CGNAT); (b) usar **IPv6** (resto do volume), em que cada casa tem endereços roteáveis próprios e o acesso de fora volta a ser possível (com firewall adequado, Capítulo 51).
2. No CGNAT determinístico, cada IP público é fatiado em **faixas de portas fixas por assinante**. A porta de origem 5000 cai numa faixa específica, e essa faixa está **pré-atribuída** a um assinante conhecido (a tabela de alocação). Basta calcular a qual bloco a porta pertence — não é preciso ter registrado **aquela conexão**, só a **regra de alocação** (que é estática). É por isso que o determinístico reduz o log de “toda conexão” para “a tabela”.
3. 65536 portas / 2000 portas por assinante 32 assinantes por IP. Para

4.000 assinantes: 4000 / 32 125 IPv4 públicos. Apertando para 1.000 portas/assinante: 65536 / 1000 65 assinantes por IP → 4000 / 65 62 IPv4 — **metade** dos IPs, ao custo de dobrar o risco de esgotamento de portas para usuários pesados.

4. **masquerade** escolhe IP/porta de saída **dinamicamente**, então o mesmo assinante aparece com mapeamentos que **mudam** o tempo todo. Para atender a lei (15) você precisaria **logar cada conexão** (volume enorme) para reconstruir “quem era quem”. Operacional: diagnosticar/atribuir tráfego a um cliente vira garimpo. O determinístico troca esse custo por uma **tabela estática** — mesma economia de IP, sem a dívida de log e rastreo.

47.7 Resumo

- **CGNAT** compartilha um IPv4 público entre dezenas de assinantes, endereçados em **100.64.0.0/10** (RFC 6598) — nem RFC1918, nem público, dedicado ao trecho operadora assinante;
- Prefira **netmap determinístico**: faixas de portas fixas por assinante tornam o mapeamento IP:porta:tempo → cliente **calculável** (essencial para 15);
- **Portas por assinante** é um trade-off: menos portas = mais clientes por IP, mais risco de esgotamento; comece em ~1.000–2.000 e monitore;
- CGNAT **quebra** acesso de fora para dentro (port-forward do cliente); ofereça IPv4 dedicado ou **IPv6** a quem precisa;
- Higiene: 100.64/10 **nunca** roteado/anunciado para fora.

A economia de IPv4 tem prazo de validade — a solução definitiva é o **IPv6**, com endereço de sobra para cada casa. Mas antes, uma obrigação que não é opcional no Brasil: o **registro legal** do CGNAT. O 15 trata do que a lei exige e como cumpri-la sem afogar o provedor em logs.

Bibliografia do capítulo

- RFC 6598 — *IANA-Reserved IPv4 Prefix for Shared Address Space*;
- Documentação oficial: *NAT e CGNAT/netmap* do RouterOS (MikroTik, 2026a);
- NAT em escala e esgotamento de portas: (Tanenbaum; Feamster; Wetherall, 2021);
- CGNAT em provedores: (Burgess, 2011).

48 Logging de CGNAT e requisitos legais

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar **o que a lei brasileira exige** de um provedor quanto a registros de conexão (Marco Civil da Internet, Lei 12.965/2014) — em linguagem operacional, não jurídica;
- Entender por que o CGNAT **cria** uma obrigação de registro que o NAT 1:1 não tinha, e o que precisa estar num registro **útil**;
- Escolher entre **logar toda conexão** e o **CGNAT determinístico** — e por que o segundo reduz o problema a uma **tabela de alocação**;
- Configurar o registro no RouterOS e **exportá-lo** para um coletor externo (a caixa não guarda meses de log);
- Responder a uma **requisição judicial** (“quem era este IP:porta neste instante?”) de forma correta e auditável.

O capítulo anterior (Capítulo 47) montou o CGNAT que faz o provedor crescer. Mas ele criou um problema novo: quando **dezenas de assinantes compartilham um IP público**, o endereço deixou de identificar alguém. Se uma autoridade pede “quem usou 203.0.113.10 às 21h34?”, a resposta “qualquer um dos 32 clientes daquele IP” não serve — nem para você, nem para a lei. **Registrar a tradução** deixou de ser opcional. Este capítulo trata dessa obrigação com a cabeça de engenheiro: cumprir a lei **sem** afogar a operação em terabytes de log.

48.1 O que a lei pede (em português de engenheiro)

O **Marco Civil da Internet** (Lei 12.965/2014) e sua regulamentação obrigam o provedor de conexão a **guardar os registros de conexão** por um prazo mínimo (**1 ano**), sob sigilo, disponibilizando-os **mediante ordem judicial**. Na prática do CGNAT, o registro precisa permitir reconstruir, para um dado momento:

IP público + porta de origem (+ protocolo) + **timestamp** → **assinante** (o IP 100.64.x.x, e por ele o contrato/login).

Sem isso, o IP público é anônimo. **Com** isso, você atende à requisição e se protege: o registro correto é o que diz “não fui eu, foi o cliente X” de forma auditável. Três propriedades importam:

- **Precisão temporal:** hora sincronizada (NTP) — um log com relógio errado é inútil ou pior;
- **Completo:** cobrir **todas** as traduções que a lei alcança;
- **Retenção e sigilo:** guardar pelo prazo, protegido, e entregar **só** sob ordem.

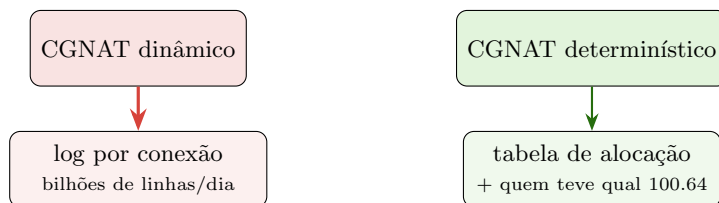
⚠ Aviso

Nada aqui é aconselhamento jurídico. Prazos, escopo e forma de guarda mudam com regulamentação e com o tipo de serviço. **Consulte um advogado** para o desenho definitivo — este capítulo cobre a parte **técnica** de tornar o registro possível e barato.

48.2 Dois caminhos: logar tudo vs. determinístico

Há duas formas de tornar a tradução rastreável, com custos muito diferentes:

1. **Logar cada conexão:** o roteador emite um registro por fluxo (quem→quem, IP:porta origem/traduzida, início/fim). É **completo**, mas o **volume é brutal** — milhares de assinantes geram **bilhões** de linhas/dia. Exige coletor robusto, muito disco e processamento. É o preço do CGNAT **dinâmico** (*masquerade/pool*);
2. **CGNAT determinístico** (Capítulo 47): como a faixa de portas é **fixa por assinante**, o mapeamento IP:porta → assinante é uma **função estática**. Você guarda apenas a **tabela de alocação** (qual bloco de portas de qual IP público pertence a qual 100.64.x.x) e a **associação 100.64.x.x assinante** ao longo do tempo. Some 99% do volume de log.



mesmo poder de resposta legal, **fração** do volume

O CGNAT determinístico transforma “guardar bilhões de conexões” em “guardar uma tabela estática + quem usou cada 100.64.x.x”. A resposta à Justiça é a mesma; o custo de armazenamento e o risco operacional são uma fração.

Figura 48.1

48.3 O registro do lado do assinante: quem teve qual 100.64

Mesmo determinístico, falta o elo final: o 100.64.x.x sozinho não é uma pessoa. Quem amarra o endereço ao **contrato** é a autenticação — tipicamente o **PPPoE/RADIUS** (12): a cada login, o RADIUS registra **qual usuário** recebeu **qual IP** e **quando** (*accounting* — Start/Stop). Esse par de registros:

- **Tabela de alocação CGNAT** (estática): IP público:faixa-portas → 100.64.x.x;
- **Accounting RADIUS** (por sessão): 100.64.x.x → usuário, com timestamps.

...fecha a cadeia IP público:porta:tempo → 100.64 → assinante. Guardar esses dois é bem mais leve do que logar todo fluxo — e é o desenho que a maioria dos provedores adota.

48.4 Configurando o registro no RouterOS

O ponto de partida é **hora certa** e **envio para fora** (a caixa não é o cofre):

```
# 1) NTP: sem relógio sincronizado, nenhum log vale
/system ntp client set enabled=yes servers=a.ntp.br,b.ntp.br

# 2) syslog remoto: log NÃO mora no roteador (memória volátil, espaço)
/system logging action
add name=coletor target=remote remote=10.99.0.20 remote-port=514
/system logging
add topics=info,account action=coletor
```

Se optar por **logar conexões** (CGNAT dinâmico), habilite o log da regra de NAT com parcimônia — e saiba que o destino **tem** de ser um coletor externo dimensionado (o Volume 13, ?@sec-logs, trata de syslog centralizado):

```
/ip firewall nat set [find action=netmap] log=yes log-prefix="cgnat"
```

Para o desenho **determinístico**, você não liga log por conexão: guarda a **tabela de alocação** (um export das regras/NAT) e o **accounting do RADIUS** — versionados e retidos pelo prazo legal.

! O conceito mais importante do capítulo

CGNAT sem registro é **passivo legal**: você não consegue dizer quem fez o quê, e a lei cobra que consiga. A escolha inteligente não é “logar mais” — é **projetar para logar menos**: CGNAT **determinístico** (tabela estática) + **accounting RADIUS** (100.64 → assinante) reconstroem qualquer requisição com uma fração do volume de um log de conexões. Sobre isso, três não-negociáveis: **NTP** (hora certa), **coletor externo** (o

log não mora na caixa) e **retenção com sigilo** pelo prazo. E o lembrete honesto: a parte técnica é esta; a jurídica, confirme com advogado.

48.5 Laboratório

i Ambiente

Reaproveite o **lab-bng** do Capítulo 47 (CGNAT determinístico ativo) e adicione **lab-log** = um coletor syslog (pode ser outro CHR com o serviço de log, ou um host Linux com rsyslog) em 10.99.0.20. Um **lab-cli** gera tráfego como assinante 100.64.0.10.

48.5.1 Comandos passo a passo

Passo 1 — hora certa em tudo. No lab-bng e no lab-log:

```
/system ntp client set enabled=yes servers=a.ntp.br
/system clock print
```

Confirme que o relógio está sincronizado antes de qualquer log.

Passo 2 — syslog remoto. No lab-bng:

```
/system logging action add name=coletor target=remote remote=10.99.0.20
/system logging add topics=info action=coletor
```

No lab-log, verifique que as mensagens chegam (/log print se for CHR, ou o arquivo do rsyslog).

Passo 3 — a tabela de alocação como registro. Exporte as regras de CGNAT — é o “documento” que amarra IP público:portas → 100.64:

```
/ip firewall nat export file=cgnat-alocacao
```

Guarde esse export (versionado, com data). Ele é a metade estática da resposta legal.

Passo 4 — accounting do assinante. Se o lab tem PPPoE/RADIUS (12), confirme que o login do assinante gera um registro **usuário** → 100.64.0.10 com timestamp. Sem RADIUS no lab, simule anotando manualmente a associação e a hora.

Passo 5 — simule uma requisição. Escolha um IP público:porta de uma conexão real do lab-cli (visto no **connection print**) e **percorra a cadeia**: porta → faixa (tabela do Passo 3) → 100.64.0.10 → assinante (Passo 4). Cronometre: com o determinístico, são minutos, não horas.

48.5.2 Verificação

1. Relógio sincronizado (NTP) em BNG e coletor;
2. Mensagens de log chegam ao coletor externo;
3. O export da tabela de alocação amarra IP público:portas → 100.64;
4. A cadeia IP:porta:tempo → 100.64 → assinante fecha a partir dos dois registros (tabela + accounting).

48.5.3 Desafios

1. Seu coletor de log encheu o disco em três dias com CGNAT **dinâmico** logando toda conexão. Sem comprar mais disco, que **mudança de projeto** derruba o volume mantendo a capacidade de resposta legal? Explique por que funciona.
2. Uma ordem judicial pede o responsável por 203.0.113.10:5000 às 2026-03-01 21:34. Liste, na ordem, **quais registros** você consulta e **como** eles se encadeiam até o nome do assinante.
3. Você descobre que o NTP do BNG estava errado em +7 minutos durante uma semana. Qual é o impacto nos registros daquele período e o que você faz? Por que hora errada é tão grave num registro legal?
4. Um técnico sugere guardar os logs **apenas no próprio roteador** “para simplificar”. Aponte **dois** motivos técnicos pelos quais isso falha o requisito de retenção — e o desenho correto.

Solução dos desafios

1. Migrar para **CGNAT determinístico**: a tradução vira uma **função estática** (faixa de portas fixa por assinante), então você guarda só a **tabela de alocação** + o **accounting** (100.64 → usuário), não cada conexão. O volume cai ordens de grandeza porque você deixou de registrar **eventos** e passou a registrar **regras** — e a resposta legal continua completa porque a regra é suficiente para recalcular quem era quem.
2. (a) **Tabela de alocação CGNAT** vigente naquela data: a porta 5000 no IP 203.0.113.10 cai numa **faixa** atribuída a um 100.64.x.x. (b) **Accounting do RADIUS/PPPoE** no instante 21:34: qual **usuário** estava com aquele 100.64.x.x (sessão Start/Stop cobrindo o horário). (c) **Cadastro**: usuário → contrato/pessoa. Os três encadeados dão o responsável, com hora sincronizada validando o instante.
3. Todos os registros daquele período têm o carimbo de tempo **deslocado em +7 min**, então uma requisição que aponta para um horário pode cair no assinante **errado** (se a alocação mudou nesse intervalo). Você **documenta o desvio** (o offset conhecido) para poder corrigir as consultas daquele período e **conserta o NTP**. Hora errada é grave porque o registro legal **é** uma afirmação temporal: sem hora confiável, ele não identifica ninguém com segurança.
4. (a) **Volatilidade/espço**: a memória do roteador é pequena e alguns logs se perdem no reboot; não há como reter **1 ano**. (b) **Risco e integridade**: se o roteador é comprometido ou substituído, o registro vai junto — e um log que o próprio operador

pode alterar tem valor probatório frágil. Desenho correto: **syslog remoto** para um coletor dedicado, com retenção pelo prazo, backup e acesso controlado (?@sec-logs).

48.6 Resumo

- O **Marco Civil** obriga guardar **registros de conexão** (~1 ano, sob sigilo, entregues por ordem judicial); com CGNAT, o registro tem de reconstruir IP **público:porta:tempo** → assinante;
- **Logar toda conexão** (CGNAT dinâmico) é completo mas de volume brutal; **CGNAT determinístico** reduz tudo a uma **tabela de alocação** estática + **accounting RADIUS** (100.64 → usuário);
- Não-negociáveis: **NTP** (hora certa), **coletor externo** (o log não mora na caixa) e **retenção com sigilo**;
- A resposta a uma requisição encadeia **tabela de alocação + accounting + cadastro** — minutos, se o projeto for determinístico;
- Isto é a camada técnica: **confirme o jurídico com advogado**.

Resolvido o presente (CGNAT + registro), resta o **futuro**, que já é presente: o **IPv6**, onde cada casa tem endereço de sobra e o acesso fim a fim volta a existir. O Capítulo 49 abre essa parte.

Bibliografia do capítulo

- Lei 12.965/2014 (Marco Civil da Internet) e sua regulamentação;
- Documentação oficial: *Log e NAT* do RouterOS (MikroTik, 2026a);
- Registro e accounting em provedores: (Burgess, 2011);
- RADIUS e accounting: (Discher, 2016).

49 IPv6 no RouterOS: fundamentos

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Ler e escrever **endereços IPv6** (notação, abreviação, prefixos) e distinguir os tipos que importam: **GUA**, **link-local** (**fe80::**), **ULA** e multicast;
- Entender o que **substitui** o ARP e o DHCP “obrigatório” do IPv4: **ND** (*Neighbor Discovery*), **RA** (*Router Advertisement*) e **SLAAC**;
- Habilitar IPv6 no RouterOS, endereçar interfaces e anunciar prefixo para a LAN com **RA**;
- Planejar o **endereçamento do provedor** com os blocos certos (/32 do provedor → /48 por POP → /56 ou /64 por cliente);
- Diagnosticar IPv6 com `/ipv6 neighbor`, `/ipv6 route` e `ping` link-local — e evitar as pegadinhas de quem “pensa em IPv4”.

O CGNAT (Capítulo 47) é um remendo genial para um problema real: **não há IPv4 suficiente**. O **IPv6** é a solução que torna o remendo desnecessário — são **340 undecilhões** de endereços, o bastante para dar a **cada casa** um bloco maior que a internet IPv4 inteira. Não é “IPv4 com mais dígitos”: muda o jeito de endereçar, de descobrir vizinhos e de configurar hosts. Este capítulo estabelece o fundamento; os próximos entregam IPv6 ao assinante (Capítulo 50) e o protegem (Capítulo 51).

49.1 Lendo um endereço IPv6

Um endereço IPv6 tem **128 bits**, escritos em **8 grupos** de 4 dígitos hexadecimais separados por `::`:

```
2001:0db8:0000:0000:0000:0000:0000:0001
```

Duas regras de abreviação tornam isso administrável:

- **Zeros à esquerda** de cada grupo somem: `0db8` → `db8`, `0000` → `0`;
- **Uma** sequência de grupos zero vira `::` (só uma vez por endereço):

```
2001:db8::1
```

O **prefixo** é o análogo da máscara: 2001:db8:abcd::/48 significa “os primeiros 48 bits identificam a rede”. Um fato que organiza toda a cabeça em IPv6: **a rede de um segmento é quase sempre um /64** — os 64 bits de rede + 64 bits de host são o padrão, e o SLAAC depende disso.

49.2 Os tipos de endereço que importam

- **GUA** (*Global Unicast*, 2000::/3): o endereço **público e roteável**, o equivalente ao IP público do IPv4 — mas aqui **cada host** tem um. É o que o provedor delega ao cliente;
- **Link-local** (fe80::/10): existe **em toda** interface IPv6, automaticamente, e só vale **naquele enlace** (não roteia). É o que ND, RA e os protocolos de roteamento usam para conversar entre vizinhos — você vai vê-lo o tempo todo;
- **ULA** (*Unique Local*, fc00::/7, na prática fd00::/8): o “privado” do IPv6, para comunicação interna que não deve sair. Útil, mas **não** substitui GUA (o objetivo do IPv6 é dar público a todos);
- **Multicast** (ff00::/8): substitui o broadcast do IPv4 (que **não existe** em IPv6). ND e RA são multicast.

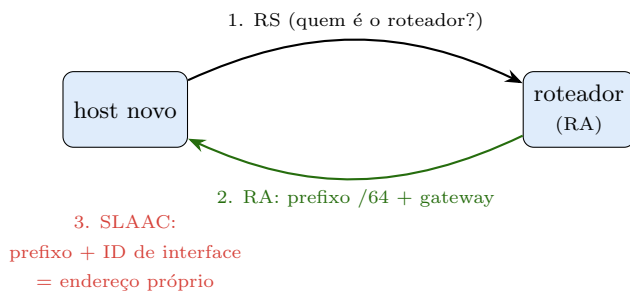
💡 Pegadinha de quem pensa em IPv4

Não existe **broadcast** nem **NAT** “de casa” no IPv6 bem-feito. O instinto de “esconder tudo atrás de um IP” não se aplica: cada dispositivo é **roteável de ponta a ponta**. Isso devolve funcionalidade (o que o CGNAT quebrava volta a funcionar), mas **transfere a segurança para o firewall** — sem o NAT servindo de “muro accidental”, o Capítulo 51 vira obrigatório, não opcional.

49.3 ND, RA e SLAAC: como o host se configura sozinho

No IPv4, um host precisava de **ARP** (achar o MAC do vizinho) e, em geral, de **DHCP** (pegar IP/gateway/DNS). O IPv6 reorganiza isso com o **ND** (*Neighbor Discovery*, sobre ICMPv6):

- **NS/NA** (*Neighbor Solicitation/Advertisement*): substitui o ARP — descobre o endereço de enlace do vizinho;
- **RS/RA** (*Router Solicitation/Advertisement*): o host pergunta “quem é o roteador?”; o roteador responde com um **RA** anunciando o **prefixo** do segmento, o gateway (ele mesmo) e flags de como o host deve se endereçar;
- **SLAAC** (*Stateless Address Autoconfiguration*): com o prefixo /64 do RA, o host **monta seu próprio endereço** (prefixo + identificador de interface) **sem DHCP**. É por isso que um /64 por segmento não é luxo — é requisito do SLAAC.



Sem DHCP: o host pede (RS), o roteador anuncia o prefixo /64 (RA) e o host **forma o próprio endereço** (SLAAC). O provedor controla o endereçamento da LAN do cliente **pelo RA** — o que ele anuncia é o que a casa usa.

Figura 49.1

49.4 Habilitando IPv6 no RouterOS

O pacote **ipv6** precisa estar **ativo** (no v7 costuma vir junto, mas confirme):

```
/ipv6 settings set disable-ipv6=no forward=yes
```

Endereçar uma interface e anunciar o prefixo para a LAN:

```
# endereço na interface de upstream (recebido do trânsito/PPPoE)
/ipv6 address add address=2001:db8:0:1::1/64 interface=ether1-wan advertise=no

# LAN do cliente: endereço + RA com SLAAC
/ipv6 address add address=2001:db8:abcd:10::1/64 interface=bridge-lan advertise=yes
```

O **advertise=yes** faz o roteador **emitir RA** naquele segmento — é o que dispara o SLAAC nos hosts. Detalhes do RA (DNS via RA, lifetime, flags managed/other) moram em `/ipv6 nd`:

```
/ipv6 nd set [find interface=bridge-lan] ra-interval=3s-10m
```

49.5 Planejando o endereçamento do provedor

IPv6 é abundante — mas abundância sem plano vira bagunça. A hierarquia consagrada:

- O provedor recebe um /32 (ou /29, /36...) do LACNIC/NIC.br;
- Cada **POP/região** recebe um /48 (65 mil /64 — folga enorme);

- Cada **cliente** recebe um **/56** (256 redes **/64** — sobra para a casa inteira) ou, no mínimo, um **/64**;
- Cada **segmento/LAN** é um **/64** (nunca menor — SLAAC exige).

Repare: você **delega prefixo**, não um endereço. É a mudança mental do IPv6 — o assinante ganha uma **rede**, não um IP. O Capítulo 50 mostra como entregar esse **/56** automaticamente pelo PPPoE.

! O conceito mais importante do capítulo

IPv6 não é “IPv4 grande” — é outro modelo: **cada host é roteável**, o **link-local** (**fe80::**) faz a plumbing entre vizinhos, e o host se autoconfigura por **RA + SLAAC** (o roteador anuncia o **/64**, o host forma o endereço). O provedor pensa em **prefixos delegados** (**/32→/48→/56→/64**), não em IPs avulsos. E a consequência de segurança é imediata: sem NAT fazendo de muro, o **firewall** (Capítulo 51) **passa a ser a única proteção** da LAN do cliente. Fim a fim de volta — com responsabilidade.

49.6 Laboratório

i Ambiente

Dois CHRs (Capítulo 5): **lab-rt** = roteador do provedor, com um **/48** fictício **2001:db8:abcd::/48**; **lab-host** = “cliente” no mesmo segmento, sem endereço configurado (vai pegar por SLAAC). Uma rede interna liga os dois.

49.6.1 Comandos passo a passo

Passo 1 — ligar o IPv6 e ver o link-local. No lab-rt:

```
/ipv6 settings set disable-ipv6=no forward=yes
/ipv6 address print
```

Note que já existe um endereço **fe80::/10 automático** na interface — o link-local, sem você configurar nada.

Passo 2 — endereçar a LAN e anunciar RA. No lab-rt:

```
/ipv6 address add address=2001:db8:abcd:10::1/64 interface=ether2 advertise=yes
```

Passo 3 — SLAAC em ação. No lab-host, ligue o IPv6 e veja o endereço **global** aparecer sozinho:

```
/ipv6 settings set disable-ipv6=no
/ipv6 address print
```

Deve surgir um 2001:db8:abcd:10:... formado por SLAAC — **sem** DHCP, só a partir do RA do lab-rt.

Passo 4 — vizinhança e rota. No lab-rt:

```
/ipv6 neighbor print
/ipv6 route print
```

O **neighbor** mostra o host (o “ARP do IPv6”); a rota do /64 da LAN está conectada.

Passo 5 — ping fim a fim. Do lab-host, pingue o GUA do lab-rt e o **link-local** (note a sintaxe com a interface):

```
/ping 2001:db8:abcd:10::1
/ping fe80::1%ether2
```

O ping ao link-local **exige** dizer a interface (%ether2) — porque fe80:: é ambíguo entre enlaces. Essa é a pegadinha nº 1 do IPv6 na prática.

49.6.2 Verificação

1. Link-local fe80:: aparece automaticamente ao ligar o IPv6;
2. O host recebe um GUA por **SLAAC** a partir do RA (sem DHCP);
3. /ipv6 neighbor lista o vizinho; a rota do /64 está conectada;
4. Ping fim a fim funciona; ping link-local exige %interface.

49.6.3 Desafios

1. Você anunciou um /48 como prefixo de RA de um segmento e o SLAAC **não** configurou os hosts. Qual é o erro conceitual, e qual é o único tamanho de prefixo que faz SLAAC funcionar? Por quê?
2. Um colega quer “esconder” a LAN IPv6 do cliente atrás de NAT, como fazia no IPv4. Explique por que isso **desperdiça** o IPv6 e cria problema em vez de resolver — e o que substitui o “muro” do NAT.
3. Do roteador, o ping para 2001:db8:... de um vizinho funciona, mas o ping para o fe80:... dele dá “no route to host”. O que falta no comando, e por que o link-local é diferente?
4. Planeje o endereçamento de um provedor com 3 POPs, cada um com até 500 clientes residenciais. Partindo de um /32, que prefixo cada POP e cada cliente recebem, e por que sobra tanto espaço?

Solução dos desafios

1. **SLAAC exige um /64** por segmento: o host forma o endereço combinando os **64 bits de prefixo** anunciados com um **identificador de interface de 64 bits**. Se você anuncia um /48 (ou qualquer coisa diferente de /64), a conta não fecha e o host não se autoconfigura. Um segmento IPv6 é, por convenção e por requisito do SLAAC, **sempre um /64** — o /48//56 é para **delegar** (dividir em muitos /64), não para um único enlace.
2. No IPv6 há endereço público de sobra para **cada** dispositivo; usar NAT joga fora justamente o benefício (fim a fim) e reintroduz a complexidade que o IPv6 elimina. O NAT do IPv4 servia de “muro accidental” só por efeito colateral — no IPv6, essa função é **explícita e correta** no **firewall** (Capítulo 51): você bloqueia o que entra e libera o que sai, com regras, em vez de depender de tradução de endereço.
3. Falta dizer **a interface de saída**: ping fe80::...%etherX. O link-local só é válido **naquele enlace** e o mesmo fe80:: pode existir em várias interfaces — sem o %interface, o roteador não sabe por onde mandar. GUA não precisa disso porque é globalmente único e roteável.
4. Do /32, cada **POP** recebe um /48 (o /32 comporta 65.536 /48, então 3 é trivial). Dentro de um POP, cada **cliente** recebe um /56 (um /48 tem 256 /56... apertado para 500 clientes — na prática dá-se /56 de um /44//40 por POP, ou /60//64 por cliente residencial). Mesmo com /64 por cliente, um /48 acomoda 65.536 clientes. Sobra tanto porque o espaço é **projetado para hierarquia folgada**: agregação limpa (uma rota por POP) vale mais que economia de bits.

49.7 Resumo

- IPv6 tem **128 bits**: abrevia zeros e usa :: uma vez; o segmento é quase sempre um /64 (requisito do SLAAC);
- Tipos-chave: **GUA** (público roteável, um por host), **link-local fe80::** (só no enlace, usado por ND/RA/roteamento), **ULA** (privado), **multicast** (substitui o broadcast);
- Host se configura por **ND + RA + SLAAC**: o roteador anuncia o /64 (advertise=yes), o host **forma** seu endereço sem DHCP;
- Provedor **delega prefixos**: /32 → /48 (POP) → /56 (cliente) → /64 (segmento) — rede, não IP avulso;
- Sem NAT de muro, o **firewall** (Capítulo 51) é a única proteção — e o ping link-local **exige %interface**.

O fundamento está de pé. Falta o que o cliente realmente recebe: um **prefixo delegado automaticamente** pelo PPPoE, em **dual stack** com o IPv4. É o Capítulo 50.

Bibliografia do capítulo

- Documentação oficial: seção *IPv6* do RouterOS (MikroTik, 2026a);

- Endereçamento IPv6, ND e SLAAC: (Tanenbaum; Feamster; Wetherall, 2021);
- IPv6 no v7 na prática: (Discher, 2016);
- Planejamento IPv6 em provedores: (Burgess, 2011).

50 DHCPv6-PD e dual stack no PPPoE

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar **prefix delegation** (DHCPv6-PD): por que o cliente recebe um **prefixo** (/56) e não um endereço, e o que ele faz com isso;
- Montar um **servidor DHCPv6-PD** no concentrador que delega prefixos de um pool a cada assinante que autentica;
- Entregar IPv6 **sobre PPPoE** em **dual stack** (IPv4 + IPv6 na mesma sessão), amarrando a delegação à autenticação;
- Fazer o **CPE do cliente** pedir o prefixo (client-PD) e distribuí-lo na LAN da casa por RA/SLAAC;
- Diagnosticar a cadeia PD ponta a ponta e reconhecer as falhas comuns (pool errado, RA ausente, rota do prefixo não instalada).

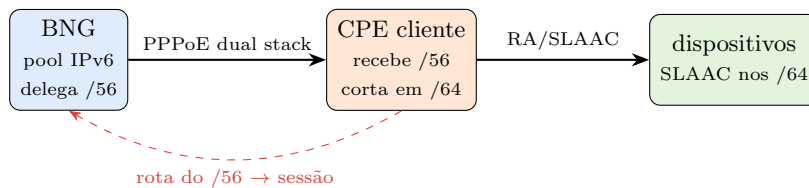
O Capítulo 49 estabeleceu a mudança mental: no IPv6 o provedor **delega uma rede**, não um IP. Falta o **como**: entregar, de forma **automática e por assinante**, o /56 que a casa vai usar — e fazer isso **junto** com o IPv4 na mesma sessão PPPoE (Capítulo 34), o chamado **dual stack**. É assim que um provedor moderno liga IPv6 sem trocar nada da autenticação que já tem.

50.1 Prefix Delegation: delegar uma rede

No IPv4, o PPPoE dava ao cliente **um** endereço /32. No IPv6 isso seria desperdício e limitação: a casa tem vários dispositivos e pode ter várias redes (LAN, WiFi de visitante, IoT). A solução é o **DHCPv6-PD** (*Prefix Delegation*): o concentrador **delega um prefixo** (tipicamente /56) ao roteador do cliente, que passa a ser **dono** dessa rede — subdivide em /64, anuncia cada um por RA na sua LAN, e roteia.

Repare a divisão de papéis:

- O **provedor** delega o /56 e **roteia** para ele (a rota do prefixo aponta para a sessão PPPoE do cliente);
- O **CPE do cliente** recebe o /56, corta em /64, e faz **RA/SLAAC** para os dispositivos da casa (Capítulo 49).



A delegação amarra tudo: o BNG delega o /56 na autenticação, instala a **rota** para ele pela sessão do cliente, e o CPE distribui /64 na casa. O IPv6 chega ao dispositivo final **sem NAT** e sem configuração manual.

Figura 50.1

50.2 O servidor DHCPv6-PD no concentrador

Primeiro, o **pool** de prefixos a delegar (o /48 do POP, cortado em /56 por cliente) e o servidor DHCPv6 em modo PD:

```
# pool: um /48 do POP entregue em pedaços /56 (256 clientes por /48... ajuste)
/ipv6 pool
add name=pd-clientes prefix=2001:db8:abcd::/48 prefix-length=56

# servidor DHCPv6 apenas para PD (não distribui endereço, distribui prefixo)
/ipv6 dhcp-server
add name=pd interface=all pool=pd-clientes address-pool=none
```

O ponto central: o `prefix-length=56` diz “corte o /48 em /56”. Cada assinante que pedir PD recebe o **próximo** /56 livre do pool.

50.3 Dual stack sobre PPPoE

O truque para ligar IPv6 sem refazer a operação: amarrar a delegação ao **profile PPPoE** (Capítulo 34). Quando a sessão sobe, além do IPv4 de sempre, o cliente recebe também o IPv6:

```
# no profile PPPoE, habilitar o DHCPv6-PD para as sessões
/ppp profile
set default-pppoe dhcpv6-pd-pool=pd-clientes \
    # endereço IPv6 do lado do concentrador na sessão (link):
    remote-ipv6-prefix-pool=pd-clientes
```

Assim, a **mesma autenticação** (secret local ou RADIUS, 12) que entrega o IPv4 passa a entregar o /56 IPv6. O RADIUS pode inclusive **atribuir o prefixo por atributo**,

mantendo a gestão centralizada — e o accounting registra a delegação (útil para o 15, já que IPv6 **não** tem CGNAT, mas ainda precisa de rastreabilidade).

 Aviso

IPv6 dual stack **não** passa por CGNAT — cada casa é roteável. Isso é o objetivo, mas significa que a **LAN do cliente fica exposta** assim que o IPv6 sobe, se você não tiver firewall IPv6. **Não ligue dual stack em produção sem** o Capítulo 51 pronto: no IPv4 o CGNAT/NAT mascarava a ausência de regras; no IPv6 não há rede de segurança.

50.4 O lado do cliente: pedindo o prefixo (client-PD)

No CPE (que também pode ser um MikroTik), o cliente **pede** o prefixo e o usa. O cliente DHCPv6 em modo PD:

```
# no CPE: pedir prefixo pela interface PPPoE
/ipv6 dhcp-client
add interface=pppoe-out1 pool-name=casa pool-prefix-length=64 request=prefix

# usar um /64 do pool recebido na LAN da casa, com RA/SLAAC
/ipv6 address add from-pool=casa interface=bridge-lan advertise=yes
```

- request=prefix (não address): é **PD**, o cliente quer a rede;
- from-pool=casa corta um /64 do /56 recebido e o coloca na LAN, com advertise=yes para o SLAAC dos dispositivos.

A casa inteira ganha IPv6 público automaticamente — o que o CGNAT tornava impossível volta a funcionar (câmeras, jogos, servidores), agora com **firewall** no lugar do NAT como controle.

50.5 Diagnóstico ponta a ponta

```
# no BNG: quem recebeu qual /56
/ipv6 dhcp-server binding print
# a rota do /56 aponta para a sessão do cliente?
/ipv6 route print where dst-address~"2001:db8:abcd"
# no CPE: recebeu o prefixo?
/ipv6 dhcp-client print
/ipv6 address print
```

A cadeia sadia: **binding** no servidor → **rota** do /56 pela sessão → **dhcp-client** no CPE com prefixo → **endereço /64** na LAN → **SLAAC** nos dispositivos. Quebra em qualquer elo, o IPv6 “não anda”.

! O conceito mais importante do capítulo

Dual stack é **evolução, não revolução**: você mantém toda a autenticação PPPoE e **acrescenta** a delegação de um /56 por sessão (DHCPv6-PD amarrado ao profile). O cliente vira **dono de uma rede**, a distribui em casa por RA/SLAAC, e o IPv6 chega ao dispositivo **sem NAT**. Duas verdades andam juntas: isso **conserta** o que o CGNAT quebrou (fim a fim de volta) e exige o firewall IPv6 pronto **antes** de ligar — porque sem o muro do NAT, a única defesa da casa é a regra que você escreve. É o assunto que fecha o volume.

50.6 Laboratório

i Ambiente

Reaproveite o cenário PPPoE do Capítulo 34: **lab-bng** = concentrador PPPoE (com IPv4 já funcionando) e **lab-cpe** = roteador do cliente (PPPoE client). Adicione o IPv6: o BNG tem o /48 2001:db8:abcd::/48; o CPE tem uma LAN interna (**ether3**) com um **lab-host** para provar o SLAAC. Objetivo: /56 delegado e um dispositivo da casa com IPv6 público.

50.6.1 Comandos passo a passo

Passo 1 — pool e delegação no BNG.

```
/ipv6 pool add name=pd-clientes prefix=2001:db8:abcd::/48 prefix-length=56
/ppp profile set default-pppoe dhcpv6-pd-pool=pd-clientes
```

Passo 2 — o CPE pede o prefixo. No lab-cpe, sobre a interface PPPoE:

```
/ipv6 dhcp-client add interface=pppoe-out1 pool-name=casa \
    pool-prefix-length=64 request=prefix
```

Confirme a recepção:

```
/ipv6 dhcp-client print
```

Deve mostrar **status=bound** e o prefixo /56 recebido.

Passo 3 — **distribuir na casa**. Ainda no lab-cpe, corte um /64 para a LAN com RA:

```
/ipv6 address add from-pool=casa interface=ether3 advertise=yes
```

Passo 4 — o dispositivo ganha IPv6 público. No lab-host (na LAN da casa), ligue o IPv6 e veja o endereço global do /56 do cliente aparecer por SLAAC:

```
/ipv6 address print
```

Passo 5 — a rota fecha o ciclo. No lab-bng, confirme que existe rota para o /56 apontando para a sessão PPPoE do cliente:

```
/ipv6 dhcp-server binding print  
/ipv6 route print where dst-address~"2001:db8:abcd"
```

Pingue, do lab-host, um endereço IPv6 do lab-bng — fim a fim, sem NAT.

50.6.2 Verificação

1. dhcp-client no CPE com status=bound e /56 recebido;
2. binding no BNG e rota do /56 pela sessão do cliente;
3. Dispositivo na LAN da casa com **GUA** por SLAAC;
4. Ping IPv6 fim a fim host BNG funciona (sem NAT).

50.6.3 Desafios

1. O CPE recebeu o /56 (dhcp-client bound), mas os dispositivos da casa **não** pegam IPv6. A delegação está certa. Que passo do lado do CPE falta, e qual comando o revela?
2. O dispositivo da casa tem endereço IPv6 global, mas **não navega**: ping para IPv6 externo falha, embora o ping ao CPE funcione. No BNG, o que você checa primeiro (pense na **rota** do /56 e no upstream)?
3. Por que entregar IPv6 **dual stack** é mais seguro para o provedor do que continuar só com CGNAT — e por que, ao mesmo tempo, ligá-lo sem firewall IPv6 é mais **perigoso** para o cliente? Concilie as duas afirmações.
4. Você quer que o **mesmo** cliente receba **sempre** o mesmo /56 (estável entre reconexões), para não confundir o cliente nem os registros. Descreva duas formas de conseguir isso (uma no pool/lease, outra via RADIUS) e por que a estabilidade importa para o

15.

Solução dos desafios

1. Falta **usar o prefixo na LAN**: o dhcp-client só **recebe** o /56; é preciso `/ipv6 address add from-pool=casa interface=<lan> advertise=yes` para

cortar um /64 e **anunciá-lo por RA**. Sem RA, não há SLAAC e os dispositivos não se endereçam. `/ipv6 address print` (sem endereço da LAN) e `/ipv6 nd print` (sem advertise na interface) revelam a falta.

2. No BNG, cheque: (a) a **rota do /56** aponta mesmo para a sessão PPPoE do cliente (`/ipv6 route print`) — se não, o retorno não acha a casa; (b) o **upstream IPv6** do BNG funciona (o provedor tem rota default IPv6 e alcança a internet). O host tem endereço, mas endereço sem rota de ida e de volta não navega — é o análogo IPv6 do next-hop do BGP.

3. Mais **seguro para o provedor**: dual stack tira carga do CGNAT (menos tradução, menos log, menos ponto único de falha) e reduz a superfície de problemas de porta esgotada. Mais **perigoso para o cliente** se ligado sem firewall: no IPv4 o CGNAT/NAT bloqueava, por efeito colateral, conexões de entrada; no IPv6 cada dispositivo é **roteável**, então sem regras o mundo alcança a IoT da casa. Conciliem-se assim: o IPv6 **transfere** a proteção do NAT para o **firewall** (Capítulo 51) — que passa a ser obrigatório, não opcional.

4. (a) **Binding estático/lease fixo**: amarrar o /56 ao identificador do cliente (DUID) no servidor DHCPv6, para reentregar o mesmo prefixo. (b) **RADIUS**: atribuir o prefixo por **atributo** no perfil do assinante, de modo que a autenticação sempre devolva o mesmo /56. Estabilidade importa porque um prefixo que muda a cada reconexão espalha registros (o accounting vira um emaranhado) e dificulta atender requisições legais — o mesmo princípio de previsibilidade do 15, agora no IPv6.

50.7 Resumo

- **DHCPv6-PD** delega um **prefixo** (/56) por assinante, não um endereço: o cliente vira **dono de uma rede** e a distribui em casa;
- No BNG: **pool** (/48 cortado em /56) + **DHCPv6 em modo PD** amarrado ao **profile PPPoE** → **dual stack** na mesma autenticação (secret/RADIUS);
- No CPE: `dhcp-client ... request=prefix` recebe o /56; `address from-pool ... advertise=yes` corta /64 e faz RA/SLAAC na LAN;
- Cadeia sadia: **binding** → **rota do /56 pela sessão** → `dhcp-client bound` → **/64 na LAN** → **SLAAC** nos dispositivos;
- IPv6 **não** passa por CGNAT (fim a fim de volta) — por isso o **firewall IPv6** tem de estar pronto **antes** de ligar dual stack.

Falta a peça que o volume inteiro vem prometendo: sem o NAT fazendo de muro, quem protege a casa IPv6 é o **firewall**. O Capítulo 51 fecha o Volume 9 mostrando as diferenças em relação ao IPv4 — ICMPv6 que **não** se bloqueia, e a proteção da LAN do cliente.

Bibliografia do capítulo

- Documentação oficial: *DHCPv6 (server/client, PD)* e *PPP* do RouterOS (MikroTik, 2026a);
- Prefix delegation e dual stack: (Tanenbaum; Feamster; Wetherall, 2021);
- IPv6 sobre PPPoE no v7: (Discher, 2016);
- Implantação de IPv6 em provedores: (Burgess, 2011).

51 Firewall IPv6

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar por que o IPv6 **exige** firewall (sem o “muro acidental” do NAT) e como isso muda a postura de segurança;
- Escrever regras em **/ipv6 firewall filter** aproveitando o que já sabe de chains e connection tracking (Capítulo 16, 16) — e o que **muda** em relação ao IPv4;
- Tratar o **ICMPv6** corretamente: por que bloqueá-lo como no IPv4 **quebra** o IPv6 (ND, PMTUD) e quais tipos liberar;
- Proteger o **roteador** (chain input) e a **LAN do cliente** (chain forward) em IPv6, incluindo o CPE dual stack;
- Montar um **conjunto mínimo** de regras IPv6 seguro para provedor e reconhecer os erros que “funcionam mas deixam tudo aberto”.

O Volume 9 devolveu ao cliente o que o CGNAT tirou: com IPv6 dual stack (Capítulo 50), **cada dispositivo da casa é roteável de ponta a ponta**. Isso é o objetivo — e é também o perigo. No IPv4, o NAT/CGNAT funcionava como um muro involuntário: o que não tinha port-forward **não** era alcançável de fora. No IPv6 esse muro **não existe**. A única coisa entre a IoT barata da casa do cliente e a internet inteira é o **firewall**. Este capítulo o constrói — e fecha o volume.

51.1 Por que IPv6 sem firewall é um erro

Repita o conceito até virar reflexo: **IPv6 não tem NAT protegendo por acidente**. Um endereço GUA numa câmera, numa TV, num roteador doméstico mal atualizado está **exposto à internet** no instante em que o dual stack sobe. Botnets varrem o espaço IPv6 procurando exatamente isso.

A boa notícia: você **já sabe** firewall. Chains (input/forward/output), connection tracking (new/established/related/invalid), address-lists — a lógica do Volume 3 vale **igual** em IPv6. Só muda o menu (**/ipv6 firewall** em vez de **/ip firewall**) e **um detalhe crítico**: o **ICMPv6**.

51.2 O que muda: ICMPv6 não é o ICMP do IPv4

No IPv4, era comum (e quase inofensivo) bloquear ICMP “por segurança”. Em IPv6 isso é **fatal**: o ICMPv6 **não é** só “ping” — ele carrega o **Neighbor Discovery** (o ARP do IPv6), o **Router Advertisement** (o SLAAC, Capítulo 49) e o **Packet Too Big** (essencial para o *Path MTU Discovery*). Bloquear ICMPv6 indiscriminadamente **quebra** o IPv6: vizinhos não se descobrem, hosts não se autoconfiguram, e conexões “grandes” travam de forma misteriosa (o mesmo sintoma de MTU do OSPF e do PPPoE, agora por PMTUD).

Regra de ouro: **libere ICMPv6** (com sensatez de taxa), nunca o bloqueie por inteiro. Os tipos essenciais: NS/NA (135/136), RS/RA (133/134), Packet Too Big (2), Time Exceeded (3), Echo (128/129) e Parameter Problem (4).



A postura é a mesma do IPv4 sem NAT: aceite o que a **casa iniciou** (established/related), **descarte** o que vem de fora sem solicitação — mas **deixe o ICMPv6 passar**, porque ele é a plumbing do protocolo, não uma ameaça a bloquear.

Figura 51.1

51.3 Protegendo o roteador (chain input)

Igual ao Capítulo 17, mas no menu IPv6. A espinha dorsal: aceitar established/related, aceitar o ICMPv6 necessário, aceitar gerência só de fontes confiáveis, **dropar o resto**:

```
/ipv6 firewall filter
add chain=input connection-state=established,related action=accept
add chain=input connection-state=invalid action=drop
# ICMPv6 é OBRIGATÓRIO - libere (com limite de taxa saudável)
add chain=input protocol=icmpv6 action=accept
# link-local: o ND/RA vive aqui, não bloqueie o fe80::/10 no input
add chain=input src-address=fe80::/10 action=accept
# gerência só da rede de gestão
add chain=input src-address-list=gerencia action=accept
add chain=input action=drop
```

Aviso

Não copie regras IPv4 trocando só o menu. Duas armadilhas: (1) **não existe** 100.64/10 nem RFC1918 “de casa” para tratar aqui, mas **existe** o fe80::/10 (link-local) que **precisa** passar no input, senão o ND morre; (2) o drop final do input **tem** de vir **depois** de aceitar o ICMPv6 e o link-local — invertido, você derruba o próprio IPv6 do roteador.

51.4 Protegendo a LAN do cliente (chain forward)

Aqui está o coração do capítulo: sem NAT, a **chain forward** é o **único muro** entre a internet e os dispositivos da casa. A política é “stateful”: deixe sair o que a casa inicia, deixe **voltar** as respostas, e **bloqueie conexões novas vindas de fora**:

```
/ipv6 firewall filter
add chain=forward connection-state=established,related action=accept
add chain=forward connection-state=invalid action=drop
# ICMPv6 fim a fim (PMTUD!) - libere
add chain=forward protocol=icmpv6 action=accept
# tráfego iniciado pela LAN do cliente para a internet: ok
add chain=forward in-interface-list=lan-clientes action=accept
# conexões NOVAS vindas da internet para a LAN: DROP (o "muro")
add chain=forward connection-state=new in-interface-list=wan action=drop
```

Esse conjunto reproduz, **explicitamente**, a proteção que o NAT dava de graça no IPv4 — com a vantagem de que agora o cliente **pode** abrir uma porta específica (uma exceção **accept** para um serviço), coisa que o CGNAT impedia. Poder com controle.

51.5 O CPE dual stack: firewall dos dois lados

O CPE MikroTik do cliente (Capítulo 50) precisa do **mesmo** cuidado: por padrão, o RouterOS traz um firewall IPv6 razoável, mas em provisão em massa você **empurra** um conjunto mínimo (input protege o CPE, forward protege a LAN da casa). É o mesmo template acima, aplicado no CPE, e é o que separa “IPv6 ligado” de “IPv6 ligado com segurança”. Provisão de firewall em escala é assunto do Volume 12 (Capítulo 62).

O conceito mais importante do capítulo

No IPv6, o **firewall não é opcional** — ele **É a segurança**. O NAT do IPv4 protegia por acidente; o IPv6 tira esse muro e o substitui por regras **explícitas** na chain forward: aceite o que a casa iniciou, drop o **new** que vem de fora. E o erro que arruína tudo: **bloquear ICMPv6 como se fosse o ICMP do IPv4** — ele carrega ND, RA e

PMTUD; sem ele, o IPv6 não anda. Libere ICMPv6, libere link-local no input, drope conexões novas de fora no forward. Faça isso **antes** de ligar dual stack em produção — depois é tarde.

51.6 Laboratório

i Ambiente

Reaproveite o dual stack do Capítulo 50: **lab-cpe** (roteador do cliente, com /56 delegado e LAN IPv6 no ether3) e **lab-host** na casa. Adicione **lab-attacker** = um host no lado “internet” que tentará alcançar o **lab-host** diretamente pelo GUA — para provar que o firewall barra o acesso não solicitado.

51.6.1 Comandos passo a passo

Passo 1 — o estado inicial perigoso. Antes de qualquer regra, do lab-attacker, pingue o GUA do lab-host:

```
/ping <gua-do-lab-host>
```

Sem firewall, **responde** — a casa está exposta. Esse é o problema.

Passo 2 — proteger o roteador (input). No lab-cpe:

```
/ipv6 firewall filter
add chain=input connection-state=established,related action=accept
add chain=input connection-state=invalid action=drop
add chain=input protocol=icmpv6 action=accept
add chain=input src-address=fe80::/10 action=accept
add chain=input action=drop
```

Confirme que o IPv6 **continua funcionando** (SLAAC, ND) — se o ND quebrasse, você teria bloqueado o ICMPv6/link-local por engano.

Passo 3 — o muro da LAN (forward). No lab-cpe:

```
/interface list add name=wan ; /interface list add name=lan-clientes
/interface list member add list=wan interface=pppoe-out1
/interface list member add list=lan-clientes interface=ether3
/ipv6 firewall filter
add chain=forward connection-state=established,related action=accept
add chain=forward protocol=icmpv6 action=accept
```

```
add chain=forward in-interface-list=lan-clientes action=accept
add chain=forward connection-state=new in-interface-list=wan action=drop
```

Passo 4 — prove o muro. Do lab-attacker, pingue de novo o lab-host: agora **não** responde (conexão nova de fora, dropada). Do lab-host, pingue a internet: **funciona** (established volta). O muro está de pé.

Passo 5 — abrir uma exceção controlada. Suponha que o cliente queira expor um serviço (porta 443) num dispositivo. Adicione **antes** do drop:

```
/ipv6 firewall filter
add chain=forward protocol=tcp dst-port=443 \
    dst-address=<gua-do-servidor> connection-state=new action=accept \
    place-before=[find chain=forward connection-state=new action=drop]
```

Do lab-attacker, a 443 daquele host agora conecta — e **só** ela. O que o CGNAT tornava impossível, o firewall IPv6 faz com precisão cirúrgica.

51.6.2 Verificação

1. Antes das regras, o “atacante” alcança a casa (exposição);
2. Após o input, o roteador segue com IPv6 sadio (ND/SLAAC intactos — prova de que o ICMPv6 foi liberado);
3. Após o forward, conexões **novas** de fora são dropadas; as iniciadas pela casa funcionam;
4. A exceção **accept** abre **exatamente** um serviço, sem abrir o resto.

51.6.3 Desafios

1. Depois de aplicar o firewall IPv6, os dispositivos da casa **pararam** de receber endereço (SLAAC morreu) e vizinhos não se enxergam. Qual regra você errou, e por que o ICMPv6/link-local é diferente de “só ping”?
2. O IPv6 “funciona”, mas downloads grandes e alguns sites **travam** no meio, enquanto páginas leves abrem normal. O ping simples passa. Qual tipo de ICMPv6 você bloqueou sem querer, e que mecanismo ele sustenta?
3. Um colega diz: “no IPv6 não preciso de firewall porque ninguém sabe o endereço aleatório da minha câmera”. Refute com dois argumentos concretos (varredura e vazamento de endereço).
4. Escreva a regra mínima que permite ao cliente **um** servidor web IPv6 acessível de fora, mantendo o resto da casa protegido. Onde ela entra na ordem das regras e por quê?

💡 Solução dos desafios

1. Você bloqueou o **ICMPv6** e/ou o **link-local fe80::/10**. O ICMPv6 não é “só ping”: ele carrega o **Neighbor Discovery** (NS/NA — o ARP do IPv6) e o **Router Advertisement** (RS/RA — o SLAAC). Bloqueado, os vizinhos não se descobrem e os hosts não se autoconfiguram. Precisa haver **accept** para **protocol=icmpv6** e para **src-address=fe80::/10** antes do drop.
2. Você bloqueou o **Packet Too Big** (ICMPv6 tipo 2), que sustenta o **Path MTU Discovery**. Sem ele, quando um caminho tem MTU menor, o emissor não é avisado para reduzir o pacote — conexões que mandam pacotes grandes (downloads) **travam**, enquanto tráfego pequeno (ping, páginas leves) passa. É o mesmo sintoma de MTU do OSPF/PPPoE, agora por PMTUD. Libere o ICMPv6 (que inclui o tipo 2).
3. (a) **Varredura**: botnets varrem o espaço IPv6 ativo (usando DNS reverso, prefixos delegados conhecidos, padrões de EUI-64) — “endereço aleatório” não é segredo confiável. (b) **Vazamento**: o endereço aparece em cada conexão que o dispositivo faz (headers, logs de servidores, telemetria) e em anúncios/mDNS na rede — basta o dispositivo falar com o mundo uma vez. Segurança por obscuridade de endereço **não é** segurança; a proteção é o firewall stateful.
4. `add chain=forward protocol=tcp dst-port=80,443 dst-address=<gua-do-servidor> connection-state=new action=accept`, posicionada **antes** da regra que dropa **new** vindo da WAN (é *first-match*: se o drop viesse primeiro, a exceção nunca seria avaliada). Assim, só aquele host e aquelas portas são alcançáveis de fora; todo o resto da casa continua barrado.

51.7 Resumo

- No IPv6 **não há NAT de muro**: cada dispositivo é roteável, então o **firewall é a segurança** — a chain forward é o único muro da casa;
- A lógica é a do Volume 3 (chains, conntrack, address-lists) no menu **/ipv6 firewall** — aceite established/related, drope **new** de fora;
- **ICMPv6 não é o ICMP do IPv4**: carrega ND, RA e Packet Too Big (PMTUD) — **libere-o**; bloqueá-lo quebra descoberta, SLAAC e downloads grandes;
- No input, libere **link-local fe80::/10** (o ND vive lá) antes do drop; no CPE dual stack, empurre o mesmo conjunto mínimo;
- IPv6 devolve o fim a fim que o CGNAT tirou — o cliente **pode** abrir um serviço com uma exceção **accept** cirúrgica.

Fecha-se o Volume 9: o provedor sobrevive ao esgotamento do IPv4 (CGNAT com registro legal) e implanta IPv6 de verdade (dual stack com firewall). Rede endereçada e protegida ponta a ponta. O **Volume 10** interliga os POPs e dá acesso remoto seguro — o mundo dos túneis e VPN.

Bibliografia do capítulo

- Documentação oficial: *IPv6 Firewall* e *ICMPv6* do RouterOS (MikroTik, 2026a);
- ICMPv6, ND e PMTUD: (Tanenbaum; Feamster; Wetherall, 2021);
- Firewall IPv6 no v7: (Discher, 2016);
- Segurança IPv6 em provedores: (Burgess, 2011).

Parte X

Volume 10 — Túneis e VPN

52 Panorama de túneis no RouterOS

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que é um **túnel** (encapsular um pacote dentro de outro) e as duas perguntas que classificam qualquer um deles: **que camada** ele transporta e **se cifra**;
- Mapear os túneis do RouterOS — **WireGuard, IPsec, GRE, EoIP, L2TP/SSTP, ZeroTier** — em uma tabela de decisão por caso de uso;
- Entender **overhead** e o problema de **MTU/MSS** que **todo** túnel cria, e como o **mss-clamp** evita a dor de cabeça nº 1 de VPN;
- Escolher o túnel certo para três cenários de provedor: **gerência remota, site-to-site entre POPs e camada 2 estendida**;
- Preparar o terreno para os capítulos seguintes, que detalham cada tecnologia.

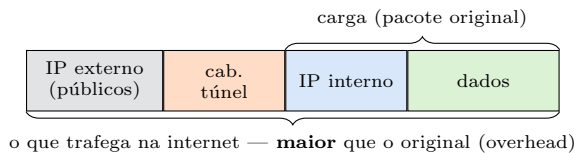
O provedor tem POPs espalhados, técnicos em campo e a necessidade de alcançar equipamentos que estão atrás de CGNAT (Capítulo 47) ou sem IP público. A resposta é o **túnel**: uma forma de fazer dois pontos distantes se comportarem como se estivessem no mesmo cabo, atravessando a internet no meio. O RouterOS oferece **muitos** tipos de túnel — e a primeira habilidade é **escolher**, porque usar o errado custa segurança, desempenho ou noites de diagnóstico de MTU. Este capítulo é o mapa; os próximos são o território.

52.1 O que é um túnel, em uma frase

Um túnel **encapsula** um pacote dentro de outro: o pacote “de dentro” (o que você quer transportar) vira **carga** de um pacote “de fora” que viaja pela internet até o outro extremo, onde é desencapsulado. É por isso que dois roteadores em cidades diferentes podem trocar tráfego como se um cabo os ligasse — o cabo é **virtual**, feito de encapsulamento.

Duas perguntas classificam **qualquer** túnel:

1. **Que camada ele transporta?** Camada 3 (IP dentro de IP — a maioria) ou camada 2 (Ethernet dentro de IP — EoIP/VXLAN, para “esticar” uma LAN)?
2. **Ele cifra?** Alguns protegem o conteúdo (WireGuard, IPsec, SSTP); outros **só encapsulam, sem segredo** (GRE, EoIP, L2TP puro) e precisam de IPsec por baixo se o sigilo importa.



Encapsular **umenta** o pacote: os cabeçalhos externo e de túnel são overhead que “come” MTU. Guardar essa imagem evita o erro clássico — esquecer que o pacote encapsulado pode não caber no caminho.

Figura 52.1

52.2 O menu de túneis do RouterOS

Túnel	Camada	Cifra?	Caso de uso típico
WireGuard	L3	Sim	Gerência remota, site-to-site moderno — simples e rápido
IPsec	L3	Sim	Site-to-site padrão de mercado, interoperar com outros fabricantes
GRE	L3	Não*	Transportar roteamento (OSPF/BGP) entre POPs; *cifra com IPsec
EoIP	L2	Não*	Esticar uma LAN/bridge entre sites (camada 2); *cifra com IPsec
L2TP	L3/L2	Não*	Acesso remoto legado; *quase sempre com IPsec (L2TP/IPsec)
SSTP	L3	Sim	Acesso remoto atravessando firewalls (roda em TCP/443)
ZeroTier	L2	Sim	Malha gerenciada na nuvem, atravessa NAT sem IP público

* GRE, EoIP e L2TP **puros não cifram** — combine com IPsec quando o tráfego for sensível.

A leitura da tabela: **WireGuard** é a escolha padrão moderna (simples, rápido, cifrado); **IPsec** quando precisa interoperar com equipamento de terceiros; **GRE/EoIP** quando o que importa é **transportar roteamento ou camada 2** (com IPsec por baixo para sigilo); **SSTP/L2TP** para acesso remoto (SSTP fura firewall em 443); **ZeroTier** quando os dois lados estão atrás de CGNAT/NAT sem IP público.

52.3 O overhead e o problema de MTU (o inimigo comum)

Todo túnel **adiciona cabeçalhos**, então o pacote encapsulado é **maior** que o original. Se o original já tinha 1500 bytes (o MTU típico da Ethernet), o encapsulado passa de 1500 e **não cabe** no caminho — ele precisa fragmentar (lento) ou, se marcado *don't fragment*, é **descartado**. O sintoma é conhecido do OSPF, do PPPoE e do PMTUD (Capítulo 51): **ping pequeno funciona, tráfego real trava**.

A defesa universal é o **MSS clamp**: o roteador reescreve o *Maximum Segment Size* anunciado nas conexões TCP que entram no túnel, forçando os dois lados a mandar segmentos que **cabem** mesmo com o overhead:

```
# aplicar MSS clamp ao tráfego que entra num túnel (evita travamento TCP)
/ip firewall mangle
add chain=forward tcp-flags=syn protocol=tcp \
    out-interface=<interface-do-tunel> action=change-mss \
    new-mss=clamp-to-pmtu comment="MSS clamp no tunel"
```

Aviso

80% dos “problemas de VPN” são MTU/MSS. O túnel sobe, o ping passa, e-mails pequenos vão — mas páginas grandes, transferências e HTTPS travam no meio. Antes de culpar o túnel, **aplique o MSS clamp**. É a primeira coisa a verificar, não a última.

52.4 Escolhendo por cenário de provedor

- **Gerência remota** de um roteador atrás de CGNAT: **WireGuard** (leve, sempre ativo, cifrado) ou **ZeroTier** (se nenhum lado tem IP público). Detalhe no Capítulo 53;
- **Site-to-site entre dois POPs** com roteamento dinâmico: **GRE + IPsec** (o GRE carrega o OSPF/BGP, o IPsec cifra) ou **WireGuard** (moderno, roteia direto). Fecha o volume no Capítulo 56;
- **Estender uma VLAN/bridge** entre sites (mesmo domínio L2): **EoIP + IPsec** ou **VXLAN**. Detalhe no Capítulo 55;

- **Interoperar** com um firewall de outro fabricante: **IPsec IKEv2**, o padrão universal (Capítulo 54).

! O conceito mais importante do capítulo

Escolher túnel é responder **duas** perguntas — **que camada** (L3 ou L2) e **cifra ou não** — e casá-las com o caso de uso: **WireGuard** como padrão moderno cifrado, **IPsec** para interoperar, **GRE/EoIP** para transportar roteamento/camada 2 (com IPsec por baixo), **SSTP/ZeroTier** para atravessar firewall/NAT. E acima de qualquer escolha, uma verdade que vale para **todos**: o túnel adiciona overhead, o pacote cresce, e sem **MSS clamp** o TCP trava. Domine a tabela de decisão e o clamp, e os próximos capítulos viram só sintaxe.

52.5 Laboratório

i Ambiente

Dois CHRs (Capítulo 5) simulando dois POPs em cidades diferentes, ligados por uma “internet” (uma rede de trânsito no meio, ou ligação direta representando o público): **lab-pop-a** e **lab-pop-b**, cada um com um IP “público” e uma LAN interna. Este laboratório é **conceitual**: o objetivo é enxergar overhead e MSS antes de montar túneis reais nos próximos capítulos.

52.5.1 Comandos passo a passo

Passo 1 — meça o MTU do caminho. Do lab-pop-a para o “público” do lab-pop-b, ache o maior ping que passa sem fragmentar:

```
/ping <publico-pop-b> size=1500 do-not-fragment
/ping <publico-pop-b> size=1472 do-not-fragment
```

Anote onde começa a passar — é o MTU útil do caminho, o teto que qualquer túnel terá de respeitar **menos** o overhead.

Passo 2 — simule o overhead. Estime: um túnel WireGuard adiciona ~60 bytes; GRE, ~24; IPsec, ~50–70. Se o caminho aceita 1500, um TCP de 1500 dentro de WireGuard vira ~1560 — **não cabe**. Essa é a origem do travamento.

Passo 3 — a regra de MSS clamp (padrão). Prepare a regra que os próximos capítulos vão aplicar em cada túnel:

```
/ip firewall mangle
add chain=forward protocol=tcp tcp-flags=syn action=change-mss \
    new-mss=clamp-to-pmtu passthrough=yes comment="MSS clamp universal"
```

Passo 4 — planeje a escolha. Para cada cenário, escreva qual túnel usaria e por quê (confira depois com a tabela): (a) acessar um CPE atrás de CGNAT; (b) rodar OSPF entre os dois POPs de forma cifrada; (c) esticar a VLAN de gerência entre os POPs.

52.5.2 Verificação

1. Você identificou o MTU útil do caminho com **do-not-fragment**;
2. Sabe estimar o overhead de cada túnel e por que ele estoura o MTU;
3. A regra de MSS clamp está entendida e pronta para reuso;
4. Você consegue justificar a escolha de túnel para os três cenários.

52.5.3 Desafios

1. Um túnel site-to-site sobe, o ping entre as LANs passa, mas os sistemas internos (ERP, compartilhamento de arquivos) travam ao transferir arquivos grandes. Qual é a causa quase certa e a regra que resolve? Por que o ping “enganou” o diagnóstico?
2. Você precisa que o **OSPF** (Capítulo 42) rode entre dois POPs por um túnel, com o tráfego **cifrado**. Por que WireGuard sozinho ou GRE+IPsec servem, mas IPsec puro (policy-based) tradicional **complica** rodar um IGP? (dica: interface vs. política)
3. Um técnico quer “esticar” a bridge de gerência (camada 2) de um POP para outro para que os equipamentos apareçam no mesmo domínio de broadcast. Que **família** de túnel ele precisa, e qual seria a escolha no RouterOS?
4. Dois roteadores estão **ambos** atrás de CGNAT, sem IP público em nenhum lado. Qual(is) túnel(is) da tabela ainda funciona(m), e por que os demais falham?

Solução dos desafios

1. **MTU/overhead do túnel sem MSS clamp.** O ping usa pacotes pequenos que cabem; a transferência real manda segmentos TCP de 1500 que, somados ao cabeçalho do túnel, estouram o MTU do caminho e são descartados (ou fragmentados com perda). A regra: **change-mss new-mss=clamp-to-pmtu** no **forward** para o tráfego do túnel. O ping enganou porque testou tamanho pequeno, não o tamanho real do tráfego.
2. OSPF precisa de uma **interface** sobre a qual formar adjacência e trocar Hellos multicast. WireGuard e **GRE** criam uma **interface de túnel** (ponto a ponto) — o IGP roda sobre ela naturalmente, e o IPsec cifra por baixo. O IPsec **policy-based** clássico não cria interface: ele casa tráfego por **política** (seletores), sem um objeto de interface para o OSPF usar — daí a preferência por GRE+IPsec ou por IPsec **em modo interface/VTI** quando se quer roteamento dinâmico.
3. Ele precisa de um túnel de **camada 2** (Ethernet dentro de IP). No RouterOS, a escolha é **EoIP** (com IPsec por baixo para cifrar) ou **VXLAN**. Túneis L3 (WireGuard, GRE, IPsec puro) transportam **IP**, não quadros Ethernet, então não colocam os dois lados no mesmo domínio de broadcast.
4. Túneis que dependem de **um lado ter porta/IP público alcançável** (WireGuard clássico com peer fixo, IPsec, GRE, EoIP, SSTP) falham quando **ambos** estão atrás

de CGNAT — ninguém consegue iniciar para o outro. **ZeroTier** funciona porque usa **servidores intermediários** (na nuvem) para orquestrar a conexão e furar o NAT dos dois lados (hole punching). A alternativa é dar IP público (ou porta redirecionada) a **um** dos lados, aí os demais túneis voltam a ser possíveis.

52.6 Resumo

- Um **túnel** encapsula um pacote dentro de outro; classifique qualquer um por **camada transportada** (L3/L2) e **se cifra**;
- Menu: **WireGuard** (padrão moderno cifrado), **IPsec** (interoperar), **GRE/EoIP** (transportar roteamento/L2, cifrar com IPsec), **SSTP** (fura firewall em 443), **ZeroTier** (malha atravessa NAT);
- **Todo** túnel adiciona **overhead** → o pacote cresce → sem **MSS clamp** o TCP trava (o ping engana): é a causa nº 1 de “problema de VPN”;
- Escolha por cenário: gerência atrás de CGNAT → WireGuard/ZeroTier; site-to-site com IGP → GRE+IPsec/WireGuard; esticar L2 → EoIP/VXLAN;
- Os próximos capítulos detalham cada um; a tabela de decisão é o mapa.

Começamos pela escolha padrão de hoje — simples, rápida e cifrada. O Capítulo 53 monta o WireGuard do zero, do acesso de gerência ao site-to-site.

Bibliografia do capítulo

- Documentação oficial: seções de *Interfaces/Tunnels* e *VPN* do RouterOS (MikroTik, 2026a);
- Encapsulamento, overhead e PMTUD: (Tanenbaum; Feamster; Wetherall, 2021);
- Túneis no v7 na prática: (Discher, 2016);
- VPNs em provedores: (Burgess, 2011).

53 WireGuard

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o modelo do **WireGuard**: interface + **par de chaves** por ponta, **peers** com **allowed-address**, e por que ele é tão simples e rápido;
- Montar um túnel WireGuard **de gerência** (seu notebook → roteador atrás de CGNAT) e um **site-to-site** entre dois POPs;
- Entender **allowed-address** — o conceito central que decide “que IPs vão por este peer” e é a fonte nº 1 de erro;
- Usar **persistent-keepalive** para manter o túnel vivo através de CGNAT/NAT e ler o *handshake* para diagnosticar;
- Rotear e cifrar tráfego real pelo túnel, aplicando o **MSS clamp** do 17.

O 17 elegeu o WireGuard como a escolha padrão moderna: cifrado por concepção, rápido (roda no kernel), e com uma configuração tão enxuta que cabe num cartão. Para o provedor, ele resolve dois problemas do dia a dia: **alcançar equipamentos atrás de CGNAT** (Capítulo 47) para gerência, e **interligar POPs** com criptografia sem o peso do IPsec. Este capítulo monta os dois, do zero.

53.1 O modelo mental do WireGuard

WireGuard é radicalmente simples comparado a IPsec. Só três ideias:

1. **Interface**: você cria uma interface **wg** no roteador; ela tem um **par de chaves** (privada + pública). A privada nunca sai; a pública você entrega aos outros;
2. **Peer**: para cada ponta remota, você adiciona um **peer** com a **chave pública dela**, o **endereço/porta** onde alcançá-la (o **endpoint**) e — o campo que tudo decide — o **allowed-address**;
3. **Sem “fases”, sem propostas, sem negociação de algoritmos**: as chaves definem tudo. Ou os dois lados têm a chave pública um do outro e conversam, ou não há túnel. Isso elimina 90% do que dá errado no IPsec.



Cada lado guarda a **chave pública** do outro e um **allowed-address** que diz “as redes que alcanço por este peer”. A simetria é a chave: A conhece B e a rede de B; B conhece A e a rede de A. Sem terceiros, sem autoridade — só chaves.

Figura 53.1

53.2 allowed-address: o conceito que tudo decide

allowed-address faz **dois** trabalhos, e confundi-los é o erro nº 1:

- **Na saída** (roteamento): “quais destinos devem ser **enviados** por este peer”. É como uma rota — WireGuard manda para o peer os pacotes cujo destino casa o **allowed-address**;
- **Na entrada** (filtro): “quais *sources* eu **aceito** vindos deste peer”. Um pacote que chega pelo túnel com origem fora do **allowed-address** é **descartado**.

Regras práticas: para **site-to-site**, o **allowed-address** de cada peer é a **rede do outro lado** (10.0.2.0/24 no peer que aponta para o POP-B). Para **acesso de gerência de um cliente roaming** (seu notebook), o peer no roteador tem **allowed-address** = o /32 do notebook no túnel. Colocar 0.0.0.0/0 manda **todo** o tráfego pelo túnel (VPN “full tunnel”) — poderoso, mas raramente o que você quer num site-to-site.

53.3 Montando: túnel de gerência

Cenário: alcançar um roteador de POP atrás de CGNAT a partir do seu notebook. O roteador do POP **inicia** para um concentrador WireGuard com IP público (porque ele, atrás de CGNAT, não pode receber conexão) — ou, mais simples aqui, o notebook alcança um roteador com IP público e por ele chega aos demais.

```
# no roteador (lado servidor): interface + porta
/interface wireguard
add name=wg-mgmt listen-port=13231
# veja a chave pública gerada:
/interface wireguard print

# endereço do túnel
/ip address add address=10.255.9.1/24 interface=wg-mgmt

# o peer (seu notebook): chave pública dele + o /32 dele no túnel
/interface wireguard peers
add interface=wg-mgmt public-key="<pub-notebook>" \
    allowed-address=10.255.9.10/32
```


No notebook (app WireGuard), o espelho: chave pública do roteador, **endpoint** = IP público:13231, **allowed-address** = 10.255.9.1/32 (ou a rede de gerência que você quer alcançar por ele).

53.4 Montando: site-to-site entre POPs

```
# POP-A
/interface wireguard add name=wg-ab listen-port=13231
/ip address add address=10.255.0.1/30 interface=wg-ab
/interface wireguard peers
add interface=wg-ab public-key="<pub-B>" endpoint-address=<publico-B> \
    endpoint-port=13231 allowed-address=10.0.2.0/24,10.255.0.2/32 \
    persistent-keepalive=25s

# POP-B (espelho)
/interface wireguard add name=wg-ab listen-port=13231
/ip address add address=10.255.0.2/30 interface=wg-ab
/interface wireguard peers
add interface=wg-ab public-key="<pub-A>" endpoint-address=<publico-A> \
    endpoint-port=13231 allowed-address=10.0.1.0/24,10.255.0.1/32 \
    persistent-keepalive=25s
```

Com o túnel de pé, uma **rota** manda o tráfego da rede remota pela interface **wg-ab** (ou o próprio **allowed-address** já cuida do encaminhamento). E, como todo túnel, **aplique o MSS clamp** (17) para não travar o TCP.

53.5 persistent-keepalive: mantendo vivo através do NAT

WireGuard é **silencioso** — se ninguém fala, ele não manda nada, e o **mapeamento NAT/CGNAT** do caminho expira, “fechando” o túnel de fora para dentro. O **persistent-keepalive=25s** faz o peer mandar um pequeno pacote a cada 25 s, mantendo o furo do NAT aberto. **Sempre** use keepalive no lado que está atrás de CGNAT/NAT — sem ele, o túnel “funciona e depois some”.

Diagnóstico: o campo de **último handshake** diz se o túnel está vivo:

```
/interface wireguard peers print
# procure "last-handshake" recente (segundos). "never" = chaves/endpoint/porta errados.
```

! O conceito mais importante do capítulo

WireGuard troca a complexidade do IPsec por **chaves e allowed-address**. Guarde três coisas: (1) cada lado tem a **chave pública** do outro — sem terceiros; (2) **allowed-address é rota E filtro** — é ele que decide “o que vai por este peer” e “o que aceito dele”, e é onde 90% dos erros moram; (3) atrás de CGNAT, use **persistent-keepalive** ou o túnel expira. Acerte esses três, aplique o **MSS clamp**, e o WireGuard é o túnel mais fácil e rápido do RouterOS.

53.6 Laboratório

i Ambiente

Dois CHRs (Capítulo 5) como dois POPs: **lab-a** (LAN 10.0.1.0/24, “público” alcançável) e **lab-b** (LAN 10.0.2.0/24). Um host em cada LAN para testar tráfego fim a fim. Opcional: um terceiro CHR “notebook” para o cenário de gerência.

53.6.1 Comandos passo a passo

Passo 1 — interfaces e chaves. Em cada POP:

```
/interface wireguard add name=wg-ab listen-port=13231
/interface wireguard print
```

Anote a **chave pública** de cada lado (você vai trocá-las).

Passo 2 — endereços do túnel.

```
# lab-a
/ip address add address=10.255.0.1/30 interface=wg-ab
# lab-b
/ip address add address=10.255.0.2/30 interface=wg-ab
```

Passo 3 — peers cruzados. No lab-a, adicione o peer com a **pública do lab-b**; no lab-b, com a **pública do lab-a** (veja os blocos de site-to-site acima), cada um com **allowed-address = rede do outro + /32 do túnel remoto**, e **persistent-keepalive=25s**.

Passo 4 — handshake. Em qualquer lado:

```
/interface wireguard peers print
```

`last-handshake` deve mostrar poucos segundos. Se **never**, revise: chave pública trocada certo? endpoint/porta corretos? firewall liberando o UDP 13231?

Passo 5 — tráfego fim a fim + MSS clamp. Do host em 10.0.1.x, pingue o host em 10.0.2.x. Depois aplique o clamp e teste uma transferência maior:

```
/ip firewall mangle
add chain=forward protocol=tcp tcp-flags=syn out-interface=wg-ab \
    action=change-mss new-mss=clamp-to-pmtu
```

Passo 6 — prove o keepalive. Remova o `persistent-keepalive` do lado “atrás de NAT”, gere silêncio e observe o handshake **envelhecer** até o túnel parar de responder de fora; recoloque e veja reviver.

53.6.2 Verificação

1. `last-handshake` recente nos dois lados;
2. Ping fim a fim entre as LANs 10.0.1.0/24 10.0.2.0/24;
3. `allowed-address` correto (a rede do outro lado) em cada peer;
4. Sem keepalive, o túnel atrás de NAT expira; com ele, permanece.

53.6.3 Desafios

1. O handshake acontece (`last-handshake` recente), mas os hosts das LANs **não** se pingam. As chaves estão certas. Qual campo do peer está errado, e por que ele afeta o **encaminhamento** e não o handshake?
2. Você quer que **todo** o tráfego de internet de um cliente roaming saia cifrado pelo túnel (full tunnel). Que `allowed-address` isso exige no lado do cliente, e que cuidado extra de NAT/rota no lado do servidor?
3. O túnel de gerência para um roteador atrás de CGNAT sobe, funciona por uns minutos e “some” quando ninguém usa. Qual parâmetro faltou e por que o CGNAT causa exatamente esse sintoma?
4. Por que o WireGuard dispensa a negociação de “fases” e propostas de algoritmo do IPsec (Capítulo 54), e o que isso implica para interoperar com um firewall de outro fabricante que só fala IPsec?

Solução dos desafios

1. O `allowed-address` está incompleto ou errado. O handshake só depende das **chaves** e do endpoint; o **tráfego** só é encaminhado para o peer se o destino casar o `allowed-address` — e só é aceito na volta se a origem casar. Se o `allowed-address` não inclui a rede da LAN remota (10.0.2.0/24), os pings são descartados apesar do túnel vivo. Corrija incluindo a rede do outro lado em cada peer.
2. No cliente, `allowed-address=0.0.0.0/0` (manda tudo pelo túnel). No **servidor**,

é preciso: (a) uma **rota/allowed-address** que aceite o /32 do cliente e (b) **NAT (masquerade)** na saída para a internet, senão o tráfego do cliente sai com IP do túnel e não retorna. É essencialmente transformar o servidor WireGuard num gateway NAT para o cliente.

3. Faltou **persistent-keepalive**. WireGuard não envia nada quando está ocioso; o CGNAT, sem tráfego, **expira o mapeamento** de porta e fecha o caminho de fora para dentro. O keepalive (25 s) mantém o mapeamento vivo. É o sintoma clássico “funciona e some” de túnel atrás de NAT.

4. WireGuard usa um conjunto **fixo** e moderno de algoritmos e autentica só por **chaves públicas** — não há o que negociar, o que o torna simples e rápido. A contrapartida: ele **não interopera** com IPsec. Para conversar com um firewall de terceiro que só fala IPsec, você usa **IPsec** (Capítulo 54), justamente pela padronização/negociação que o WireGuard abre mão.

53.7 Resumo

- WireGuard = **interface** + **par de chaves** por ponta e **peers** com a **chave pública** do outro lado; sem fases nem negociação — as chaves definem tudo;
- **allowed-address** é **rota e filtro**: define o que vai por cada peer e o que se aceita dele — a fonte nº 1 de erro (inclua a **rede do outro lado**);
- **persistent-keepalive=25s** mantém o túnel vivo através de **CGNAT/NAT** — sem ele, “funciona e some”;
- Diagnóstico pelo **last-handshake** (recente = vivo; **never** = chave/endpoint/porta/firewall);
- Como todo túnel, aplique **MSS clamp** (17) para o TCP não travar.

WireGuard cobre a maioria dos casos modernos — mas quando você precisa **interoperar** com equipamento de outro fabricante, o padrão universal é outro. O Capítulo 54 monta o IPsec IKEv2.

Bibliografia do capítulo

- Documentação oficial: seção *WireGuard* do RouterOS (MikroTik, 2026a);
- Criptografia de túnel e troca de chaves: (Tanenbaum; Feamster; Wetherall, 2021);
- WireGuard no v7 na prática: (Discher, 2016);
- VPN de gerência em provedores: (Burgess, 2011).

54 IPsec

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

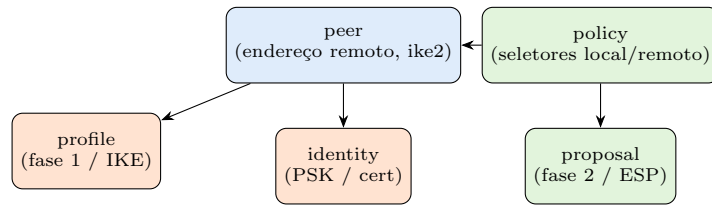
- Explicar as peças do **IPsec** no RouterOS — **profile**, **peer**, **proposal**, **policy**, **identity** — e como elas se encaixam;
- Distinguir as duas fases (**IKE** para negociar a chave, **ESP** para cifrar os dados) e por que **IKEv2** é a escolha atual;
- Diferenciar IPsec **policy-based** (por seletores) de **tunnel/VTI** (por interface) e quando cada um serve;
- Montar um **site-to-site IKEv2** entre dois POPs e interoperar com firewall de outro fabricante;
- Diagnosticar o “clássico do IPsec”: fase que não fecha, seletores que não batem, e o NAT-Traversal.

O Capítulo 53 entregou o túnel cifrado mais simples. Então por que aprender IPsec, mais complexo? **Interoperabilidade**: o IPsec é o **padrão universal** de VPN — todo firewall sério (Fortinet, Cisco, pfSense, Palo Alto) fala IPsec, e frequentemente o provedor precisa fechar túnel com um equipamento que **não** é Mikrotik. Onde o WireGuard exige os dois lados falando WireGuard, o IPsec conversa com qualquer um. O preço é a configuração em peças — que este capítulo desmonta.

54.1 As peças do IPsec no RouterOS

IPsec no RouterOS é um quebra-cabeça de objetos que se referenciam. Entender **o papel de cada peça** é metade da batalha:

- **profile**: parâmetros da **fase 1 (IKE)** — algoritmos de hash/criptografia, grupo DH, tempo de vida. É “como negociar a chave”;
- **peer**: **com quem** falar (endereço do outro lado), qual **profile** usar e a versão do IKE (**exchange-mode=ike2**);
- **proposal**: parâmetros da **fase 2 (ESP)** — como **cifrar os dados** depois que a chave foi negociada;
- **identity**: **como se autenticar** com aquele peer — chave pré-compartilhada (**secret/PSK**), certificado, etc.;
- **policy**: **qual tráfego** proteger — os **seletores** (rede local rede remota) e o modo (tunnel).



os **algoritmos** do profile e da proposal precisam BATER nos dois lados

O **peer** (com quem, fase 1) usa um **profile**; a **identity** diz como provar quem é; a **policy** (que tráfego) usa uma **proposal** (fase 2). Se os **algoritmos** de qualquer fase não coincidirem entre os lados, o túnel não fecha — a causa nº 1 de dor de cabeça no IPsec.

Figura 54.1

54.2 Duas fases: IKE negocia, ESP cifra

O IPsec trabalha em duas etapas:

1. **Fase 1 (IKE)**: os dois lados se **autenticam** (PSK ou certificado) e negociam um canal seguro para conversar — a “chave da chave”. O **IKEv2** é a versão atual: mais rápido, mais robusto, com melhor suporte a NAT e roaming. Use **ike2** salvo quando o outro lado só suportar o antigo IKEv1;
2. **Fase 2 (ESP)**: usando o canal da fase 1, negociam como **cifrar o tráfego real** (a **proposal**) e passam a proteger os pacotes que casam a **policy**.

A divisão explica os sintomas: se a **fase 1 não fecha**, é peer/profile/autenticação; se a fase 1 fecha mas a **fase 2 não**, é proposal/seletores.

54.3 policy-based vs. tunnel/VTI

Há dois estilos de IPsec, e a escolha muda o que você consegue fazer:

- **Policy-based** (clássico): a **policy** casa tráfego por **seletores** (ex.: **src=10.0.1.0/24 dst=10.0.2.0/24**) e o cifra. **Não cria interface** — logo, **não** dá para rodar OSPF/BGP por cima
(17) nem fazer roteamento dinâmico. Simples para site-to-site fixo;
- **Tunnel/VTI ou IPsec sobre outra interface**: quando você precisa de **roteamento dinâmico** ou de uma interface roteável, o padrão é **GRE/IP-in-IP + IPsec** (Capítulo 55): o GRE dá a interface, o IPsec cifra. É o desenho recomendado para interligar POPs com IGP (Capítulo 56).

Regra: **site-to-site fixo e simples** → policy-based direto; **roteamento dinâmico entre POPs** → GRE+IPsec.

54.4 Montando um site-to-site IKEv2 (PSK)

```
# POP-A
/ip ipsec profile
add name=ike2-prof hash-algorithm=sha256 enc-algorithm=aes-256 dh-group=modp2048
/ip ipsec peer
add name=para-B address=<publico-B> profile=ike2-prof exchange-mode=ike2
/ip ipsec identity
add peer=para-B auth-method=pre-shared-key secret="Ch@ve-Forte-2026"
/ip ipsec proposal
add name=esp-prop auth-algorithms=sha256 enc-algorithms=aes-256-cbc pfs-group=modp2048
/ip ipsec policy
add peer=para-B tunnel=yes proposal=esp-prop \
    src-address=10.0.1.0/24 dst-address=10.0.2.0/24
```

No **POP-B**, o espelho: mesmo **profile/proposal** (algoritmos batendo), **peer** apontando para o público de A, **mesmo secret**, e a **policy** com **src/dst invertidos** (**src=10.0.2.0/24 dst=10.0.1.0/24**). Essa inversão simétrica dos seletores é obrigatória — se não baterem em espelho, a fase 2 não fecha.

Aviso

Quando há **NAT no caminho** (comum: o POP atrás de CGNAT), habilite o **NAT-Traversal** (o IPsec encapsula o ESP em UDP/4500). Sem ele, o ESP “puro” (protocolo 50) é frequentemente bloqueado/quebrado pelo NAT e o túnel fecha a fase 1 mas **não passa tráfego**. No RouterOS o NAT-T é automático no IKEv2, mas confirme que o **UDP 500 e 4500** estão liberados no firewall.

54.5 Diagnóstico do IPsec

```
/ip ipsec active-peers print
/ip ipsec policy print    # veja a coluna de flags/ph2-state
/ip ipsec installed-sa print
```

- **active-peers** vazio / fase 1 não estabelece → peer, profile (algoritmos), autenticação (PSK diferente), ou UDP 500 bloqueado;
- fase 1 ok mas **policy sem SA (ph2) instalada** → proposal não bate, ou **seletores** (src/dst) não estão em espelho;
- fase 2 ok mas **sem tráfego** → NAT-T ausente (libere 4500), rota, ou o tráfego não casa os seletores.

! O conceito mais importante do capítulo

IPsec é a moeda de **interoperabilidade**: quando o outro lado não é MikroTik, é ele que fecha o túnel. O custo é montar peças que se referenciam — **profile/peer** (fase 1, IKE) e **proposal/policy** (fase 2, ESP) — e a regra que resolve a maioria dos problemas é uma só: **os algoritmos precisam bater e os seletores precisam estar em espelho** nos dois lados. Para **roteamento dinâmico**, não force o policy-based: use **GRE+IPsec**. E onde houver NAT, **NAT-Traversal (UDP 4500)** ou o túnel fecha e não anda.

54.6 Laboratório

i Ambiente

Dois CHRs (Capítulo 5) como POPs: **lab-a** (LAN 10.0.1.0/24, público alcançável) e **lab-b** (LAN 10.0.2.0/24). Opcional: um NAT no meio para exercitar NAT-T. Objetivo: site-to-site IKEv2 com PSK protegendo o tráfego entre as duas LANs.

54.6.1 Comandos passo a passo

Passo 1 — profile e proposal (algoritmos idênticos). Em ambos:

```
/ip ipsec profile add name=p1 hash-algorithm=sha256 enc-algorithm=aes-256 dh-group=modp2048  
/ip ipsec proposal add name=p2 auth-algorithms=sha256 enc-algorithms=aes-256-cbc pfs-group=modp2048
```

Passo 2 — peer + identity. No lab-a (apontando para o público de B), e o espelho no lab-b, com o **mesmo secret**:

```
/ip ipsec peer add name=peer address=<público-outro> profile=p1 exchange-mode=ike2  
/ip ipsec identity add peer=peer auth-method=pre-shared-key secret="lab-psk-forte"
```

Passo 3 — policy com seletores em espelho. No lab-a:

```
/ip ipsec policy add peer=peer tunnel=yes proposal=p2 \  
src-address=10.0.1.0/24 dst-address=10.0.2.0/24
```

No lab-b, **invertido**: src=10.0.2.0/24 dst=10.0.1.0/24.

Passo 4 — evite NAT no tráfego do túnel. Garanta que o srcnat/ masquerade da WAN **não** capture o tráfego 10.0.1.0/24 → 10.0.2.0/24 (uma regra **accept** antes do masquerade, casando dst da rede remota) — senão o IPsec não reconhece os seletores.

Passo 5 — suba e verifique.


```
/ip ipsec active-peers print
/ip ipsec policy print
```

`active-peers` com a fase 1, e a `policy` mostrando a SA da fase 2 instalada. Pingue `host host` entre as LANs e confirme, no `installed-sa`, os contadores subindo (tráfego cifrado).

54.6.2 Verificação

1. `active-peers` mostra a fase 1 estabelecida;
2. A `policy` tem SA de fase 2 instalada (seletores em espelho);
3. Ping `host host` passa e os contadores de SA sobem;
4. O tráfego do túnel **não** é capturado pelo `masquerade` da WAN.

54.6.3 Desafios

1. A fase 1 fecha (`active-peers` ok), mas nenhum tráfego passa e a `policy` não instala SA de fase 2. Cite as **duas** causas mais comuns e como confirmar cada uma.
2. Tudo “certo”, mas os hosts não se pingam — e você descobre que o `masquerade` da WAN está reescrevendo o tráfego antes do IPsec. Explique por que isso quebra os seletores e qual regra corrige.
3. O provedor precisa rodar **OSPF** entre os dois POPs sobre um enlace **cifrado**. Por que o IPsec **policy-based** deste capítulo não serve, e qual é o desenho recomendado (aponte o capítulo)?
4. Um dos POPs está atrás de **CGNAT**. A fase 1 fecha às vezes, mas o tráfego é errático. Qual recurso do IPsec está em jogo, e que portas você confere no firewall?

Solução dos desafios

1. (a) **proposal (fase 2) não bate**: os algoritmos de ESP (`enc/auth/PFS`) diferem entre os lados — confira que **proposal** é idêntica nos dois. (b) **Seletores não estão em espelho**: o `src/dst` da `policy` de um lado tem de ser o inverso exato do outro; qualquer divergência (uma máscara diferente) impede a fase 2. Confirme com `/ip ipsec policy print` comparando os dois lados.
2. O `masquerade` reescreve o *source* do pacote **antes** de o IPsec avaliar os seletores; o pacote deixa de casar `src=10.0.1.0/24` e o IPsec o ignora (sai em claro pela WAN). Corrige-se com uma regra de NAT **accept antes** do `masquerade`: `chain=srcnat dst-address=10.0.2.0/24 action=accept` (não traduzir o tráfego destinado à rede remota do túnel).
3. **Policy-based não cria interface**, e o OSPF precisa de uma interface para formar adjacência e trocar Hellos. O desenho recomendado é **GRE + IPsec** (Capítulo 55): o GRE fornece a interface roteável (onde o OSPF roda) e o IPsec cifra o transporte. Também interligação de POPs no Capítulo 56.
4. **NAT-Traversal**. Atrás de CGNAT, o ESP puro (protocolo 50) não sobrevive à

tradução; o IKEv2 encapsula o ESP em **UDP/4500** (NAT-T) para atravessar. Se o tráfego é errático, confira que **UDP 500 e 4500** estão liberados no firewall (input) e que o NAT-T está ativo — sem ele, a fase 1 até fecha, mas o ESP não passa de forma confiável.

54.7 Resumo

- IPsec é o **padrão de interoperabilidade** — use quando o outro lado **não** é MikroTik (ou exige IPsec);
- Peças: **profile/peer** (fase 1, IKE — negocia a chave, use **ike2**), **proposal/policy** (fase 2, ESP — cifra os dados), **identity** (PSK/cert);
- Regra de ouro: **algoritmos batendo** (profile e proposal) e **seletores em espelho** (src/dst invertidos) nos dois lados;
- **Policy-based** para site-to-site fixo; **GRE+IPsec** quando precisa de **inter-face**/roteamento dinâmico;
- Com NAT no caminho, **NAT-Traversal (UDP 4500)** é obrigatório; e não deixe o **masquerade** capturar o tráfego do túnel.

IPsec e WireGuard cifram **camada 3**. Falta o que transporta **roteamento** e **camada 2** entre POPs — e os túneis de acesso legado. O Capítulo 55 cobre GRE, EoIP, L2TP/SSTP e ZeroTier.

Bibliografia do capítulo

- Documentação oficial: seção *IPsec* do RouterOS (MikroTik, 2026a);
- IKE, ESP e arquitetura IPsec: (Tanenbaum; Feamster; Wetherall, 2021);
- IPsec no v7 e interoperabilidade: (Discher, 2016);
- Túneis site-to-site em provedores: (Burgess, 2011).

55 GRE, EoIP, L2TP/SSTP e ZeroTier

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Montar **GRE** para transportar roteamento (OSPF/BGP) entre POPs e **cifrá-lo** com IPsec — o desenho recomendado do Capítulo 54;
- Usar **EoIP** para estender **camada 2** (uma bridge/VLAN) entre sites, e saber quando isso é bom (e quando é perigoso);
- Configurar **L2TP/IPsec** e **SSTP** para **acesso remoto** — incluindo o SSTP rodando em TCP/443 para furar firewalls;
- Entender **ZeroTier** como malha gerenciada que atravessa CGNAT/NAT sem IP público em nenhum lado;
- Escolher, entre todos, a ferramenta certa para “transportar roteamento”, “esticar L2” e “acesso remoto” — e respeitar o MTU.

WireGuard (Capítulo 53) e IPsec (Capítulo 54) cifram **camada 3**. Mas o provedor tem necessidades que eles não cobrem sozinhos: **transportar um protocolo de roteamento** por uma interface, **estender uma LAN** de um POP a outro (camada 2), dar **acesso remoto** a quem só tem um cliente VPN comum, ou conectar caixas que **não têm IP público**. Este capítulo reúne os túneis que resolvem esses casos — cada um com seu nicho.

55.1 GRE: a interface para transportar roteamento

O **GRE** (*Generic Routing Encapsulation*) encapsula IP dentro de IP e, crucialmente, **cria uma interface** de túnel ponto a ponto. Isso é o que faltava no IPsec policy-based: com uma interface, você roda **OSPF/BGP por cima** (Capítulo 42). O GRE **não cifra** — então o par de produção é **GRE + IPsec**.

```
# POP-A: interface GRE ponto a ponto
/interface gre
add name=gre-ab remote-address=<publico-B> local-address=<publico-A>
/ip address add address=10.255.1.1/30 interface=gre-ab

# cifrar o GRE com IPsec (o campo ipsec-secret liga os dois)
/interface gre set gre-ab ipsec-secret="Ch@ve-GRE-2026"
```

O `ipsec-secret` faz o RouterOS montar **automaticamente** a policy IPsec que protege esse GRE — a forma mais limpa de ter “GRE cifrado”. Com a interface `gre-ab` de pé, o OSPF a trata como qualquer enlace PtP (`type=ptp`). É o alicerce do Capítulo 56.

55.2 EoIP: esticar a camada 2 entre sites

O **EoIP** (*Ethernet over IP*, proprietário MikroTik) transporta **quadros Ethernet** — camada 2 — dentro de IP. O resultado: dois sites distantes ficam no **mesmo domínio de broadcast**, como se um cabo ligasse as bridges. Útil para levar uma VLAN de gerência a um POP remoto, ou unir dois pontos que precisam de L2 comum.

```
# EoIP e adição à bridge (os dois lados na mesma bridge = mesma L2)
/interface eoip
add name=eoip-ab remote-address=<publico-B> tunnel-id=100 ipsec-secret="Ch@ve-EoIP"
/interface bridge port add bridge=br-gerencia interface=eoip-ab
```

Aviso

Estender camada 2 é **poderoso e perigoso**. Você une domínios de broadcast: uma tempestade de broadcast, um loop de STP ou um DHCP rogue em um POP **contamina o outro**. Use EoIP com parcimônia, sempre com **STP** ativo, e prefira **rotear** (L3) a **esticar** (L2) sempre que possível. L2 estendido é a última opção, não a primeira.

55.3 L2TP e SSTP: acesso remoto

Para dar **acesso remoto** a um técnico ou a um cliente com cliente VPN padrão (Windows, celular), dois protocolos servem:

- **L2TP/IPsec**: L2TP puro **não cifra**; na prática usa-se sempre **L2TP sobre IPsec**. Amplamente suportado por sistemas operacionais, bom para “VPN de acesso” clássica;
- **SSTP**: roda sobre **TLS em TCP/443** — logo, **atravessa quase qualquer firewall/proxy** (parece HTTPS). É a escolha quando o técnico está numa rede hostil que bloqueia UDP e portas de VPN. Precisa de **certificado** no servidor.

```
# servidor L2TP com IPsec (use-ipsec) e pool para os clientes
/interface l2tp-server server set enabled=yes use-ipsec=yes ipsec-secret="Ch@ve-L2TP"
/ppp secret add name=tecnico1 service=l2tp password="senha-forte" profile=default

# servidor SSTP (TCP/443) - exige certificado
/interface sstp-server server set enabled=yes certificate=cert-sstp port=443
```

Note o reuso do que você já sabe: L2TP e SSTP são **serviços PPP** (Capítulo 34) — usam `ppp secret/profiles/pools` exatamente como o PPPoE, e podem autenticar por **RADIUS** (12). Acesso remoto é PPPoE “pela internet”.

55.4 ZeroTier: malha que atravessa NAT

E quando **nenhum** dos lados tem IP público — os dois atrás de CGNAT? Os túneis anteriores precisam de alguém “alcangável” para iniciar. O **ZeroTier** resolve com uma **malha gerenciada na nuvem**: os nós se registram num controlador (ZeroTier Central), que os ajuda a **furar o NAT** dos dois lados (hole punching) e a formar uma rede virtual L2/L3. Ideal para gerência de CPEs e roteadores sem IP público.

```
# habilitar ZeroTier e juntar-se a uma rede (ID vem do painel ZeroTier)
/zerotier set zt1 comment="malha de gerencia"
/zerotier interface add network=<network-id> instance=zt1
```

O trade-off: você depende de um **serviço externo** (o controlador na nuvem) e de uma conta ZeroTier. Para gerência, costuma valer; para tráfego crítico de produção, prefira túneis que você controla ponta a ponta.

55.5 Escolhendo entre eles

Necessidade	Escolha	Por quê
Rodar OSPF/BGP cifrado entre POPs	GRE + IPsec	Interface + criptografia
Esticar VLAN/L2 entre sites	EoIP (+IPsec)	Transporta Ethernet
Acesso remoto de técnico/cliente	L2TP/IPsec ou SSTP	Cliente VPN padrão; SSTP fura firewall
Ambos os lados atrás de CGNAT	ZeroTier	Malha na nuvem fura o NAT

! O conceito mais importante do capítulo

Estes túneis preenchem os nichos que WireGuard/IPsec sozinhos não cobrem: **GRE** dá a **interface** para roteamento dinâmico (cifre com IPsec); **EoIP** estende **camada 2** (com o risco de unir domínios de broadcast — use com cuidado); **L2TP/SSTP** dão **acesso remoto** com cliente padrão (SSTP fura firewall em 443); **ZeroTier** conecta quem **não tem IP público**. A regra de escolha é sempre a mesma do 17: **que camada e cifra ou não** — e, em todos, respeite o **MTU/MSS**.

55.6 Laboratório

i Ambiente

Dois CHRs (Capítulo 5) como POPs (**lab-a**, **lab-b**) com IPs “públicos” alcançáveis e uma LAN interna cada. Um terceiro CHR **lab-tec** simula o “técnico remoto” para o acesso L2TP/SSTP. O foco é GRE+IPsec (transportar roteamento) e um serviço de acesso remoto.

55.6.1 Comandos passo a passo

Passo 1 — GRE cifrado entre os POPs. No lab-a (e espelho no lab-b):

```
/interface gre add name=gre-ab remote-address=<publico-b> ipsec-secret="lab-gre"  
/ip address add address=10.255.1.1/30 interface=gre-ab
```

No lab-b: 10.255.1.2/30 e remote-address=<publico-a>.

Passo 2 — OSPF sobre o GRE. Rode OSPF (Capítulo 42) na interface **gre-ab** dos dois lados (**type=ptp**) e confirme a adjacência Full — o roteamento dinâmico viajando cifrado.

```
/routing ospf interface-template add interfaces=gre-ab area=backbone type=ptp  
/routing ospf neighbor print
```

Passo 3 — prove que está cifrado. Verifique que o IPsec subiu automaticamente com o GRE:

```
/ip ipsec active-peers print
```

Passo 4 — acesso remoto SSTP. No lab-a, habilite o SSTP-server (com um certificado autoassinado para o lab) e crie um usuário; no lab-tec, conecte como cliente SSTP e alcance a LAN do POP.

```
/interface sstp-server server set enabled=yes certificate=cert-lab  
/ppp secret add name=tecnico service=sstp password="lab-forte"
```

Passo 5 — MSS clamp. Aplique o clamp (17) nas interfaces de túnel e teste uma transferência maior entre as LANs.

55.6.2 Verificação

1. GRE de pé, com **IPsec ativo** (cifrado) — **active-peers** mostra a SA;
2. OSPF forma adjacência **Full sobre** o GRE (roteamento dinâmico cifrado);
3. O técnico remoto conecta por SSTP e alcança a LAN do POP;
4. Com MSS clamp, transferências grandes não travam.

55.6.3 Desafios

1. Você montou o GRE e o OSPF formou adjacência, mas quer garantir que ninguém no caminho leia o tráfego. Como confirma que o GRE está **realmente** cifrado, e o que aconteceria sem o **ipsec-secret**?
2. Um técnico esticou por **EoIP** a VLAN de gerência de dois POPs e, dias depois, um loop em um POP derrubou a gerência **dos dois**. Explique o mecanismo e duas medidas que teriam contido o estrago.
3. Um cliente corporativo em viagem precisa de VPN, mas a rede do hotel bloqueia UDP e portas de VPN — só deixa passar navegação web. Qual protocolo deste capítulo funciona, e por quê?
4. Você precisa gerenciar 200 CPEs, **todos** atrás de CGNAT, sem IP público em nenhum. Qual solução do capítulo se aplica, qual é o trade-off que você aceita, e por que os túneis “clássicos” não servem aqui?

Solução dos desafios

1. Confirme com `/ip ipsec active-peers print` e `installed-sa` — se há SA ativa para os endpoints do GRE e os contadores sobem, o tráfego vai cifrado. **Sem o ipsec-secret**, o GRE transporta tudo **em claro**: qualquer um no caminho (a operadora de trânsito, um atacante no meio) lê o conteúdo, incluindo os pacotes OSPF. GRE puro **não** protege nada.
2. EoIP uniu os **domínios de broadcast** dos dois POPs; um loop de camada 2 num POP gera tempestade de broadcast que atravessa o túnel e satura a gerência **do outro** também. Contenção: (a) **STP** ativo na bridge (detecta e bloqueia o loop); (b) limitar/segmentar o que trafega no L2 estendido (ou, melhor, **rotear** em vez de esticar). L2 estendido propaga falhas — por isso é a última opção.
3. **SSTP**: roda sobre **TLS em TCP/443**, indistinguível de HTTPS. Como o hotel libera navegação web (443), o SSTP passa onde L2TP/IPsec (UDP 500/4500) e WireGuard (UDP) são bloqueados. É a razão de existir do SSTP: atravessar firewalls restritivos.
4. **ZeroTier**: forma uma malha gerenciada na nuvem que **fura o NAT** dos dois lados via hole punching, então funciona mesmo com todos os CPEs atrás de CGNAT sem IP público. Trade-off aceito: **dependência de um serviço externo** (o controlador ZeroTier) e de uma conta. Os túneis clássicos (WireGuard peer fixo, IPsec, GRE, SSTP) precisam que **um** lado seja alcançável de fora — o que não existe quando ambos estão atrás de CGNAT.

55.7 Resumo

- **GRE** cria **interface** para roteamento dinâmico entre POPs; **não cifra** — use **GRE** + **IPsec** (`ipsec-secret`), o desenho recomendado;
- **EoIP** estende **camada 2** (mesmo domínio de broadcast) — poderoso e **perigoso**: STP obrigatório, prefira rotear a esticar;
- **L2TP/IPsec** e **SSTP** dão **acesso remoto** com cliente VPN padrão (reusam PPP/RADIUS); **SSTP** em TCP/443 **fura firewalls**;
- **ZeroTier** conecta caixas **sem IP público** (ambos atrás de CGNAT), ao custo de depender de um controlador na nuvem;
- Escolha por camada + criptografia, e **respeite o MTU/MSS** em todos.

Com o arsenal completo de túneis, o volume fecha aplicando tudo ao problema real do provedor: **interligar POPs** de forma resiliente e cifrada, com roteamento dinâmico por cima. É o Capítulo [56](#).

Bibliografia do capítulo

- Documentação oficial: *GRE*, *EoIP*, *L2TP*, *SSTP*, *ZeroTier* do RouterOS (MikroTik, 2026a);
- Encapsulamento L2/L3 e VPN de acesso: (Tanenbaum; Feamster; Wetherall, 2021);
- Túneis no v7 na prática: (Discher, 2016);
- Interligação e acesso remoto em provedores: (Burgess, 2011).

56 Interligação de POPs por túnel

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

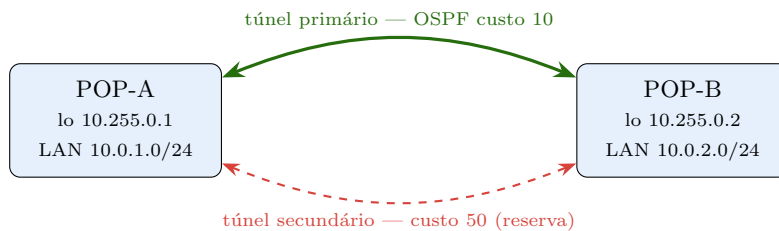
- Projetar a **interligação de dois POPs** por túnel, com roteamento dinâmico por cima e criptografia por baixo;
- Montar o caso **com IP público dos dois lados** (GRE+IPsec ou WireGuard) e o caso **sem IP público** (um lado atrás de CGNAT);
- Adicionar **redundância**: dois túneis por caminhos diferentes e failover automático via **OSPF/custo** (Capítulo 42);
- Integrar o túnel ao desenho de roteamento do provedor (loopbacks, áreas, MSS clamp) sem criar caminhos assimétricos ou loops;
- Reconhecer os erros que derrubam a interligação: MTU, next-hop, recursão de rota (o túnel que “se engole”).

O Volume 10 construiu o arsenal; este capítulo **aplica**. O cenário é o mais comum do provedor em crescimento: **dois POPs em cidades diferentes**, cada um com seus clientes, que precisam se comportar como **uma rede só** — trocar rotas, compartilhar trânsito, servir de contingência um ao outro. A cola é o túnel; a inteligência é o roteamento dinâmico por cima. Fechamos o volume transformando peças soltas em uma **rede interligada e resiliente**.

56.1 O desenho: túnel + roteamento + criptografia

A interligação de POPs de verdade tem **três camadas**, e confundi-las é a origem dos problemas:

1. **Transporte cifrado**: o túnel que atravessa a internet — **GRE + IPsec** (interface para roteamento, cifrado) ou **WireGuard** (moderno, cifra e roteia). É o “cabo virtual”;
2. **Roteamento dinâmico**: **OSPF** (Capítulo 42) rodando **sobre** a interface do túnel, para que cada POP aprenda as redes do outro **automaticamente** e reconverja se algo cair;
3. **Endereçamento de controle**: **loopbacks** (Capítulo 46) como router-id e âncora das sessões — sobrevivem à troca de caminho.



Dois túneis por caminhos diferentes; o OSPF escolhe o de **menor custo** e cai para o outro se ele falhar — a mesma lógica de custo do Capítulo 42, agora sobre túneis. Loopbacks dão identidade estável a cada POP.

Figura 56.1

56.2 Caso 1: os dois POPs têm IP público

O cenário limpo. Monte o túnel (GRE+IPsec do Capítulo 55, ou WireGuard do Capítulo 53) e rode OSPF por cima:

```
# POP-A: GRE cifrado + endereço de controle
/interface gre add name=gre-ab remote-address=<publico-b> ipsec-secret="Ch@ve-POP"
/ip address add address=10.255.1.1/30 interface=gre-ab
/interface bridge add name=lo ; /ip address add address=10.255.0.1/32 interface=lo

# OSPF sobre o túnel (ptp) - anuncia loopback e LAN
/routing ospf instance add name=isp router-id=10.255.0.1
/routing ospf area add name=backbone area-id=0.0.0.0 instance=isp
/routing ospf interface-template \
    add area=backbone interfaces=gre-ab type=ptp \
    networks=10.255.1.0/30
/routing ospf interface-template \
    add area=backbone networks=10.0.1.0/24,10.255.0.1/32 passive=yes
```

O espelho no POP-B. Com isso, cada POP **aprende sozinho** a rede do outro, e uma queda do túnel some das rotas em segundos (reconvergência OSPF). Não esqueça o **MSS clamp** (17) na gre-ab.

56.3 Caso 2: um POP atrás de CGNAT

Quando o POP-B não tem IP público (atrás de CGNAT, Capítulo 47), ele **não pode receber** conexão — precisa **iniciar** para o A. O **WireGuard** resolve elegantemente: o B tem o **endpoint** do A e um **persistent-keepalive** (Capítulo 53) para manter o furo do CGNAT aberto; o A **não** precisa do endpoint de B (aprende quando B fala).

```
# POP-A (IP público): peer B sem endpoint fixo (B inicia)
/interface wireguard add name=wg-ab listen-port=13231
/interface wireguard peers add interface=wg-ab public-key="<pub-B>" \
    allowed-address=10.0.2.0/24,10.255.0.2/32

# POP-B (atrás de CGNAT): peer A COM endpoint e keepalive
/interface wireguard peers add interface=wg-ab public-key="<pub-A>" \
    endpoint-address=<publico-a> endpoint-port=13231 \
    allowed-address=10.0.1.0/24,10.255.0.1/32 persistent-keepalive=25s
```

OSPF roda por cima igual ao Caso 1. A regra de ouro: **quem está atrás de CGNAT inicia e mantém keepalive**; quem tem IP público espera.

56.4 Redundância: dois túneis, failover por OSPF

Um único túnel é um ponto único de falha. A resiliência vem de **dois túneis por caminhos diferentes** (ex.: um pela operadora A, outro pela B) e deixar o **OSPF escolher** por custo: primário com custo baixo, secundário com custo alto. Se o primário cai, o OSPF reconverge para o secundário **sem intervenção** — a mesma mecânica do Capítulo 42, agora aplicada a túneis.

```
# custo baixo no túnel primário, alto no secundário (nos DOIS lados!)
/routing ospf interface-template set [find interfaces=gre-ab] cost=10
/routing ospf interface-template set [find interfaces=wg-ab] cost=50
```

Lembre da lição do OSPF: ajuste o custo nos **dois** sentidos, ou você cria caminho **assimétrico** (ida por um túnel, volta por outro) — chato de diagnosticar e ruim para o desempenho.

Aviso

Recursão de rota é a armadilha traiçoeira da interligação: se a rota para o **endpoint do túnel** (o IP público do outro POP) for aprendida **pelo próprio túnel**, ele “se engole” — o túnel precisa da rota que só existe se o túnel estiver de pé. Garanta que o endpoint público seja alcançado pela **rota da internet** (default/trânsito), **nunca** pela rota interna que o OSPF do túnel anuncia. Na prática: não redistribua a default de forma que ela concorra com o caminho para o endpoint.

56.5 Integração com o roteamento do provedor

Fechando o ciclo com o Volume 8:

- **Loopbacks** como router-id e para sessões estáveis (Capítulo 46);

- **Áreas OSPF** se a interligação crescer (o backbone entre POPs na área 0; cada POP pode ter sua área, 14);
- **BGP** por cima quando os POPs trocam **rotas de internet** (não só internas) ou têm trânsito próprio — iBGP entre loopbacks sobre o OSPF do túnel (Capítulo 44);
- **Filtros** para não vazarem rota de gerência/loopbacks indevidamente.

! O conceito mais importante do capítulo

Interligar POPs são **três camadas** que você não deve misturar: **transporte cifrado** (GRE+IPsec ou WireGuard), **roteamento dinâmico** (OSPF sobre a interface do túnel, para reconvergir sozinho) e **endereçamento de controle** (loopbacks). Redundância = **dois túneis + custo OSPF**, sempre simétrico. E a armadilha que derruba tudo: a **recursão de rota** — o endpoint do túnel tem de ser alcançado pela rota da internet, nunca pela rota que o próprio túnel carrega. Acerte isso e dois POPs viram uma rede só, que se cura sozinha.

56.6 Laboratório

i Ambiente

Três CHRs (Capítulo 5): **lab-a** e **lab-b** como POPs (cada um com LAN e loopback), e **lab-net** no meio simulando a internet/trânsito (dá o “público” a cada POP e roteia entre eles). Monte **dois** túneis A B por caminhos lógicos distintos para exercitar o failover.

56.6.1 Comandos passo a passo

Passo 1 — túnel primário GRE+IPsec. Monte o **gre-ab** cifrado (Capítulo 55) e endereços de controle 10.255.1.0/30 + loopbacks (10.255.0.1/32 em A, .2/32 em B).

Passo 2 — OSPF sobre o túnel. Rode OSPF nos dois lados sobre **gre-ab** (**type=ptp**), anunciando LANs e loopbacks (**passive** nas LANs). Confirme **neighbor print = Full** e que cada POP aprendeu a LAN do outro em **/ip route where ospf**.

Passo 3 — tráfego fim a fim + MSS clamp. Ping host host entre as LANs; aplique o clamp na **gre-ab** e teste transferência maior.

Passo 4 — segundo túnel (WireGuard) como reserva. Monte um **wg-ab** (Capítulo 53) por um caminho lógico diferente, rode OSPF nele também, e ajuste custos: **gre-ab=10**, **wg-ab=50** (nos dois lados).

Passo 5 — prove o failover. Com o tráfego indo pelo primário (custo 10), **derrube** o **gre-ab** (desabilite a interface ou bloqueie o endpoint). Em segundos, o traceroute A→B passa pelo **wg-ab**. Religue e volta ao primário.

Passo 6 — provoque a recursão de rota. Anuncie, de propósito, uma default pelo OSPF do túnel e observe o túnel instabilizar (o endpoint passa a ser “aprendido” pelo próprio túnel). Corrija garantindo rota específica/estática para o endpoint pela internet. Entenda o sintoma.

56.6.2 Verificação

1. OSPF Full sobre o túnel; cada POP aprende a LAN do outro sem rota estática;
2. Ping/transferência fim a fim funcionam (com MSS clamp);
3. Queda do túnel primário reconverge para o secundário em segundos;
4. Custos simétricos (sem assimetria no traceroute dos dois sentidos);
5. Você reproduziu e corrigiu a recursão de rota.

56.6.3 Desafios

1. Os dois túneis estão de pé, mas o tráfego A→B vai pelo primário e o B→A volta pelo secundário. Nada “quebrou”. Qual é a causa e como corrigir? Por que a assimetria é indesejável mesmo “funcionando”?
2. Após um reboot do POP-B (atrás de CGNAT), o túnel WireGuard demora para voltar e às vezes não volta sozinho. Qual parâmetro garante o retorno automático, e por que o lado com IP público não precisa dele?
3. O túnel primário cai e **não** há failover: o OSPF do secundário nunca formou adjacência porque o endpoint do secundário ficou inalcançável junto. Que erro de projeto (dica: caminhos independentes) causou isso?
4. Você quer que os POPs troquem, além das redes internas, as **rotas de internet** que cada um recebe de sua operadora. OSPF basta? Se não, o que você adiciona e sobre o quê ele roda?

Solução dos desafios

1. O custo OSPF foi ajustado em um só lado. A ida (A→B) vê o primário mais barato; a volta (B→A), por o custo não ter sido espelhado, prefere o secundário. Corrija igualando os custos **nos dois lados**. A assimetria é ruim porque dificulta diagnóstico (traceroute divergente), pode passar por caminhos de qualidade/latência diferentes e complica firewall/conntrack (tráfego de volta por interface inesperada).
2. **persistent-keepalive** no lado atrás de CGNAT (o POP-B): ele reinicia o envio periódico que refuta o NAT e restabelece o handshake sem depender de tráfego. O lado com **IP público** (POP-A) não precisa porque é **alcançável** — o B sempre pode iniciar para ele; o problema é manter o caminho **de volta** ao B vivo, e é isso que o keepalive de B garante.
3. Os dois túneis usaram, na prática, o **mesmo caminho físico** (ou o mesmo endpoint/uplink): quando o primário caiu, o secundário caiu junto porque dependia do mesmo recurso. Redundância exige **caminhos independentes** — endpoints/uplinks distintos (operadora A vs. B). Túnel “redundante” sobre um único link é redundância

de mentira.

4. OSPF é para a **infraestrutura interna** (loopbacks, LANs, enlaces) — não para carregar a tabela de internet. Para trocar **rotas de internet**, adicione **iBGP** entre os loopbacks dos POPs (Capítulo 44), rodando **sobre** o OSPF do túnel (que resolve os next-hops). OSPF é o esqueleto; o BGP transporta os prefixos externos.

56.7 Resumo

- Interligar POPs = **três camadas**: transporte cifrado (GRE+IPsec / WireGuard), **roteamento dinâmico** (OSPF sobre a interface do túnel) e **loopbacks** de controle;
- **Com IP público** dos dois lados: GRE+IPsec/WireGuard + OSPF direto; **um lado atrás de CGNAT**: WireGuard, com o lado NATeado **iniciando** e mantendo **keepalive**;
- **Redundância** = dois túneis por **caminhos independentes** + failover por **custo OSPF** (simétrico, sob pena de assimetria);
- Armadilha: **recursão de rota** — o endpoint do túnel deve ser alcançado pela **rota da internet**, nunca pelo próprio túnel;
- Cresceu? Integre com **áreas OSPF** e **iBGP** (Capítulo 44) para trocar rotas de internet; sempre com **MSS clamp**.

Fecha-se o Volume 10: os POPs do provedor agora se interligam de forma cifrada, resiliente e com roteamento que se cura sozinho. O **Volume 11** leva a rede a outro patamar de backbone — **MPLS**, os rótulos que transportam serviços de camada 2 e 3 em escala de operadora.

Bibliografia do capítulo

- Documentação oficial: *Tunnels, OSPF, IPsec* do RouterOS (MikroTik, 2026a);
- Projeto de backbone e roteamento sobre túnel: (Tanenbaum; Feamster; Wetherall, 2021);
- Interligação de POPs no v7: (Discher, 2016);
- Redes de provedor multi-POP: (Burgess, 2011).

Parte XI

Volume 11 — MPLS

57 MPLS: conceitos e LDP

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que é **MPLS** (comutar por **rótulos**, não por IP) e por que provedores o adotam: **VPNs escaláveis** e transporte uniforme;
- Entender o **rótulo** (label), a **pilha de rótulos**, o **LSP** (caminho comutado) e os papéis **LER/LSR** (borda vs. núcleo);
- Descrever como o **LDP** distribui rótulos automaticamente **apoiado na IGP** (OSPF, Capítulo 42) — MPLS **precisa** de roteamento interno de pé;
- Habilitar MPLS + LDP no RouterOS v7 e ler a tabela de rótulos (`/mpls forwarding-table`, `/mpls ldp neighbor`);
- Reconhecer o pré-requisito de **MTU** do MPLS (o rótulo adiciona bytes) e os erros que impedem o LSP de subir.

Os volumes anteriores rotearam a rede por **IP**: cada roteador olha o destino e decide o próximo salto. O **MPLS** (*Multiprotocol Label Switching*) muda o jogo no **núcleo** do provedor: em vez de olhar o IP a cada salto, os roteadores comutam por um **rótulo** curto, colado na entrada e trocado salto a salto até a saída. Por que isso importa? Porque o rótulo permite **carregar serviços** (VPNs L2 e L3, Capítulo 58) por cima de um núcleo que **não precisa conhecer** as rotas dos clientes — a chave da escalabilidade de operadora. Este capítulo estabelece o fundamento e liga o LDP.

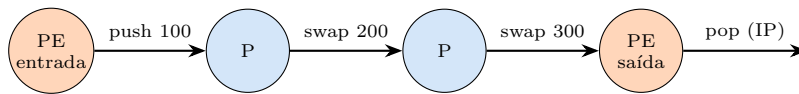
57.1 Comutar por rótulo, não por IP

No roteamento IP tradicional, **todo** roteador no caminho faz a mesma consulta cara: “qual o próximo salto para este destino?”. No MPLS, essa decisão é tomada **uma vez, na borda**: o roteador de entrada classifica o pacote e **empurra um rótulo**; do núcleo em diante, cada roteador só olha o **rótulo** (um número), **troca** por outro e encaminha — rápido e uniforme. Na saída, o rótulo é **removido** e o pacote volta a ser IP (ou Ethernet).

Os papéis:

- **LER** (*Label Edge Router*) / **PE** (*Provider Edge*): a **borda** — empurra o rótulo na entrada e o remove na saída. É quem “entende” o serviço do cliente;

- **LSR** (*Label Switch Router*) / **P** (*Provider*): o **núcleo** — só **troca rótulos** (swap), sem olhar IP. Não precisa conhecer as rotas dos clientes, só como alcançar os outros roteadores MPLS.



núcleo comuta por **rótulo**; só a borda olha o IP

O caminho que o pacote rotulado segue é um **LSP** (*Label Switched Path*). A borda **empurra** (push) o rótulo; o núcleo **troca** (swap) a cada salto; a borda de saída **remove** (pop). O núcleo nunca consulta o IP do cliente — daí a escala.

Figura 57.1

57.2 A pilha de rótulos e o LSP

O MPLS permite **empilhar** rótulos (um “pacote de pacotes”): o rótulo de baixo identifica o **serviço** (uma VPN, um circuito L2) e o de cima leva o pacote pelo **túnel de transporte** (o LSP) até o PE de saída. É essa pilha que faz VPLS e L3VPN (Capítulo 58) funcionarem: o núcleo só olha o rótulo de **transporte**; o de **serviço** só é lido na borda de saída.

Um detalhe de implementação comum é o **PHP** (*Penultimate Hop Popping*): o penúltimo roteador remove o rótulo de transporte antes de entregar ao PE final, poupando-o de uma consulta dupla. Você verá isso como um “rótulo implícito” nas tabelas.

57.3 LDP: distribuindo rótulos automaticamente

Configurar rótulo à mão em cada roteador seria insustentável. O **LDP** (*Label Distribution Protocol*) automatiza: cada roteador MPLS **anuncia aos vizinhos** qual rótulo usar para alcançar cada destino da IGP. O ponto crucial — e a fonte de confusão de quem começa:

MPLS/LDP não roteia. Ele **apenas distribui rótulos** para os destinos que a **IGP** (OSPF) já conhece. Sem OSPF de pé, o LDP não tem o que rotular.

Ou seja, a base é o Volume 8: **OSPF** carrega os loopbacks e enlaces do núcleo; o **LDP** cola um rótulo em cada um. É por isso que os loopbacks /32 (Capítulo 46) são tão importantes — eles são os “destinos” que os LSPs conectam, e as VPNs se ancoram neles.

57.4 Habilitando MPLS + LDP no RouterOS v7

Assumindo OSPF já rodando entre os roteadores do núcleo (loopbacks alcançáveis):

```
# 1) habilitar o LDP e dar a ele o transport-address (o loopback!)
/mpls ldp
add transport-addresses=10.255.0.1 lsr-id=10.255.0.1 vrf=main

# 2) em quais interfaces falar LDP (as do núcleo, entre roteadores MPLS)
/mpls ldp interface
add interface=ether2
add interface=ether3
```

- `lsr-id/transport-addresses` = o **loopback** — a identidade do roteador no MPLS, estável (não use IP de interface física);
- **LDP nas interfaces do núcleo** (entre PEs e Ps), **nunca** nas interfaces de cliente.

57.5 Lendo e diagnosticando

```
/mpls ldp neighbor print      # vizinhos LDP (devem casar os vizinhos OSPF do núcleo)
/mpls forwarding-table print  # rótulos: in-label -> out-label + next-hop
/mpls local-bindings print    # que rótulo ESTE roteador atribuiu a cada prefixo
```

A cadeia sadia: **OSPF Full** entre os roteadores → **LDP neighbor** estabelecido → **forwarding-table** com rótulos para os loopbacks. Se o LDP não sobe, quase sempre é: OSPF não alcança o loopback usado como `transport-address`, ou **MTU**.

Aviso

MTU é pré-requisito do MPLS. Cada rótulo adiciona 4 bytes; uma pilha de 2 rótulos, 8 bytes. Se o MTU do núcleo não comportar o pacote + rótulos, o LSP sobe mas o **tráfego grande é descartado** (o fantasma de sempre). Ajuste o **MPLS MTU** das interfaces do núcleo para acomodar os rótulos (tipicamente MTU físico 1508–1520). Sem isso, VPLS/L3VPN “funcionam no ping e travam no uso”.

O conceito mais importante do capítulo

MPLS comuta por **rótulo**, não por IP: a borda (**PE/LER**) empurra e remove; o núcleo (**P/LSR**) só troca rótulos, sem conhecer as rotas dos clientes — é isso que dá **escala** e permite carregar VPNs (Capítulo 58). Mas a base é o Volume 8: **MPLS/LDP não roteia — ele rotula o que a IGP (OSPF) já conhece**. Loopbacks /32 como `lsr-id`, LDP só no núcleo, e **MTU** ajustado para os rótulos. Sem OSPF sólido e MTU certo, não há MPLS.

57.6 Laboratório

i Ambiente

Três CHRs (Capítulo 5) em linha, simulando o núcleo: **lab-pe1** — **lab-p** — **lab-pe2**. Cada um com loopback /32 (10.255.0.1/.2/.3) e enlaces /30 entre vizinhos. **OSPF** rodando entre os três (Volume 8), com todos os loopbacks alcançáveis — é o pré-requisito.

57.6.1 Comandos passo a passo

Passo 1 — confirme a IGP. Antes do MPLS, garanta OSPF Full e loopbacks alcançáveis:

```
/routing ospf neighbor print  
/ip route print where ospf
```

Cada roteador tem de alcançar os loopbacks dos outros dois. **Sem isso, não siga** — o MPLS não tem base.

Passo 2 — ajuste o MTU para os rótulos. Nas interfaces do núcleo, garanta MTU suficiente:

```
/interface ethernet set [find name=ether2] mtu=1520 l2mtu=1580
```

Passo 3 — ligue LDP nos três. Exemplo lab-pe1:

```
/mpls ldp add transport-addresses=10.255.0.1 lsr-id=10.255.0.1  
/mpls ldp interface add interface=ether2
```

Análogo em lab-p (interfaces para os dois lados) e lab-pe2.

Passo 4 — veja os vizinhos e os rótulos.

```
/mpls ldp neighbor print  
/mpls forwarding-table print
```

Os vizinhos LDP devem casar os vizinhos OSPF do núcleo; a forwarding-table mostra rótulos (push/swap) para os loopbacks.

Passo 5 — prove a comutação por rótulo. Gere tráfego entre os loopbacks de lab-pe1 e lab-pe2 e observe no lab-p que ele **comuta por rótulo** (a forwarding-table incrementa), sem consultar o IP do cliente.

Passo 6 — quebre o MTU de propósito. Reduza o MTU de um enlace do núcleo abaixo do necessário e observe LSP de pé, mas tráfego grande falhando — o sintoma de MTU. Restaure.

57.6.2 Verificação

1. OSPF Full e loopbacks alcançáveis (pré-requisito);
2. `ldp neighbor` estabelecido nos enlaces do núcleo;
3. `forwarding-table` com rótulos para os loopbacks (push/swap/pop);
4. MTU insuficiente derruba o tráfego grande, mesmo com LSP de pé.

57.6.3 Desafios

1. O OSPF está Full, mas o LDP **não** forma vizinhança. `ping` entre os loopbacks funciona. Qual é a causa mais provável relacionada ao `transport-address`, e como confirma?
2. LDP subiu, os rótulos aparecem, mas ao subir uma VPLS (próximo capítulo) o tráfego trava em pacotes grandes. Qual é o ajuste que faltou e por que o ping enganou?
3. Um colega quer ligar LDP também na **interface do cliente** “para simplificar”. Explique por que isso é errado (segurança e arquitetura) — o que o núcleo MPLS deve e não deve conhecer.
4. Por que MPLS **depende** da IGP e não a substitui? Descreva o que aconteceria com os LSPs se o OSPF do núcleo caísse, e o que isso diz sobre a ordem de projeto (o que montar primeiro).



Solução dos desafios

1. O `transport-address/lsr-id` aponta para um endereço que a IGP não distribui, ou o LDP não está ligado nas interfaces certas. O `ping` entre loopbacks funciona (OSPF os conhece), mas o LDP usa o `transport-address` para estabelecer a sessão TCP — se ele não for um loopback alcançável e anunciado, a sessão não sobe. Confirme `/mpls ldp print` (transport correto = loopback) e `/mpls ldp interface print` (interfaces do núcleo listadas).
2. Faltou **ajustar o MTU** do núcleo para acomodar os **rótulos** (cada um +4 bytes; VPLS/L3VPN empilham). O LSP sobe e o ping (pequeno) passa, mas o tráfego real com pilha de rótulos estoura o MTU e é descartado. Aumente o MTU/l2mtu das interfaces do núcleo (ex.: 1520/1580).
3. LDP na interface do cliente exporia o núcleo a rótulos vindos de fora (risco de injeção/instabilidade) e quebraria a arquitetura: o **núcleo (P/LSR) não deve conhecer nem o IP nem os rótulos do cliente**, só como alcançar os outros roteadores MPLS. O cliente entra pela **borda (PE)**, que empurra/remove rótulos — o LDP vive **entre roteadores do provedor**, nunca voltado ao cliente.
4. LDP **rotula o que a IGP conhece** — ele não calcula caminhos, só cola rótulos nas rotas do OSPF. Se o OSPF cair, os destinos somem da tabela e os **LSPs colapsam** (sem rota, sem rótulo, sem caminho). Ordem de projeto: **IGP primeiro** (OSPF sólido, loopbacks alcançáveis), **depois** MPLS/LDP, **depois** os serviços (VPLS/L3VPN). MPLS é uma camada **sobre** o roteamento, não um substituto dele.

57.7 Resumo

- **MPLS** comuta por **rótulo**, não por IP: a **borda (PE/LER)** empurra/remove; o **núcleo (P/LSR)** só **troca rótulos**, sem conhecer rotas de cliente — a chave da **escala** e das VPNs;
- **Pilha de rótulos**: transporte (leva pelo LSP) + serviço (VPN/L2) — a base de VPLS/L3VPN (Capítulo 58);
- **LDP** distribui rótulos **automaticamente**, mas **não roteia**: rotula o que a **IGP (OSPF)** conhece — loopbacks /32 como `lsr-id`, LDP **só no núcleo**;
- Diagnóstico: OSPF Full → `ldp neighbor` → `forwarding-table` com rótulos; falhas comuns = transport-address não alcançável ou **MTU**;
- **MTU é pré-requisito** — rótulos adicionam bytes; sem folga, o LSP sobe e o tráfego grande trava.

Com o transporte MPLS de pé, o provedor pode oferecer o que vende caro: **circuitos de camada 2 fim a fim** sobre o backbone. O Capítulo 58 monta o VPLS.

Bibliografia do capítulo

- Documentação oficial: seção *MPLS* e *LDP* do RouterOS (MikroTik, 2026a);
- Comutação por rótulos e arquitetura MPLS: (Tanenbaum; Feamster; Wetherall, 2021);
- MPLS no v7 na prática: (Discher, 2016);
- MPLS em backbones de provedor: (Burgess, 2011).

58 VPLS: camada 2 sobre o backbone

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que é **VPLS** (uma “LAN virtual” fim a fim sobre o núcleo MPLS) e o produto que ela vende: **transporte L2 dedicado** entre sites do cliente;
- Diferenciar **pseudowire** (ponto a ponto, VLL/EoMPLS) de **VPLS** (multiponto, uma bridge distribuída);
- Montar uma VPLS no RouterOS v7 sobre o LSP do Capítulo 57, amarrando-a a uma **bridge** e às portas do cliente;
- Entender **MTU**, **aprendizado de MAC** e os riscos de camada 2 distribuída (broadcast, loop) — e como contê-los;
- Reconhecer quando **VPLS** é a resposta certa vs. quando **L3VPN/ roteamento** seria melhor.

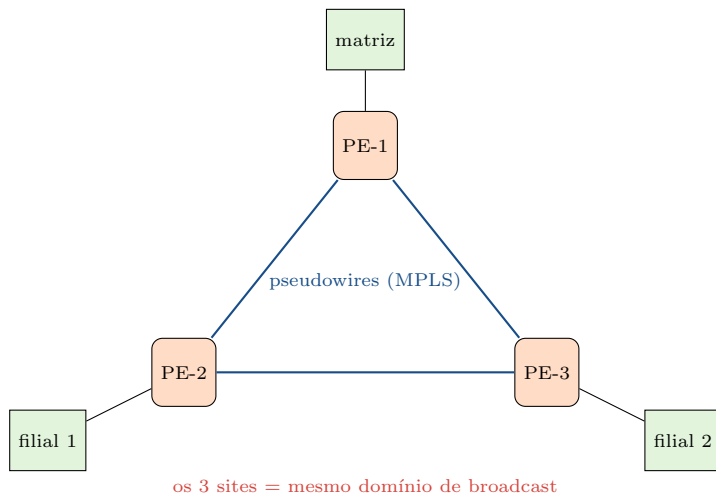
O Capítulo 57 montou o transporte por rótulos. Agora o provedor colhe o que o MPLS foi feito para dar: **serviços**. O primeiro e mais vendável é o **VPLS** (*Virtual Private LAN Service*) — entregar a um cliente corporativo dois (ou mais) sites que se comportam como se estivessem no **mesmo switch**, mesmo domínio de broadcast, através do backbone. É o “link L2 dedicado” que se cobra caro: a filial e a matriz na mesma LAN, sem o cliente saber que há uma operadora inteira no meio.

58.1 O que a VPLS entrega

Do ponto de vista do cliente, a VPLS é **um switch**: ele pluga a matriz numa ponta e a filial na outra, e elas ficam no **mesmo segmento Ethernet** — mesmo broadcast, mesmo ARP, endereços L2 visíveis entre si. Do ponto de vista do provedor, isso é feito **encapsulando quadros Ethernet em MPLS** (uma pilha de rótulos, Capítulo 57) e transportando- os pelo LSP entre os PEs.

Dois sabores:

- **Pseudowire / VLL / EoMPLS: ponto a ponto** — um “fio virtual” entre **dois** sites. Simples, sem aprendizado de MAC (é um túnel L2 direto);
- **VPLS: multiponto** — uma **bridge distribuída** entre **vários** PEs; cada PE aprende MACs e replica broadcast para os outros, como um switch. Mais poderoso, mais cuidado.



Três sites do cliente ligados por uma VPLS: os PEs formam uma **bridge distribuída** por pseudowires MPLS, e para o cliente os três sites estão no **mesmo switch**. O núcleo transporta quadros Ethernet rotulados sem conhecer os MACs do cliente.

Figura 58.1

58.2 Montando VPLS no RouterOS v7

Sobre o núcleo MPLS/LDP já pronto (Capítulo 57), a VPLS é uma **interface** que você adiciona a uma **bridge** junto com a porta do cliente. Entre dois PEs (pseudowire), em cada lado:

```
# PE-1: interface VPLS apontando para o loopback do PE remoto
/interface vpls
add name=vpls-clienteX remote-peer=10.255.0.3 vpls-id=100:100

# juntar a VPLS e a porta do cliente na MESMA bridge (mesmo L2)
/interface bridge add name=br-clienteX
/interface bridge port add bridge=br-clienteX interface=vpls-clienteX
/interface bridge port add bridge=br-clienteX interface=ether5
```

- **remote-peer** = o **loopback** do PE do outro site (alcançado pela IGP, rotulado pelo LDP);
- **vpls-id** **casa** os dois lados — o identificador do circuito;
- a **bridge** é o que une o quadro do cliente (em **ether5**) ao pseudowire — é assim que o L2 “atravessa” o backbone.

Para VPLS **multiponto**, repete-se o pseudowire para cada PE do cliente (malha) e todos entram na mesma bridge; o aprendizado de MAC e a replicação de broadcast fazem o resto.

58.3 MTU, MACs e os perigos da L2 distribuída

VPLS herda as duas grandes preocupações de camada 2 estendida — e adiciona a de MPLS:

- **MTU:** o quadro do cliente + rótulos MPLS precisa **caber** no núcleo. Como o cliente espera enviar 1500 bytes “limpos”, o núcleo tem de ter MTU **maior** (jumbo: 1520–1600), senão o tráfego grande trava (Capítulo 57). **Configure o MTU do núcleo antes** de prometer VPLS;
- **Aprendizado de MAC:** cada PE aprende os MACs do cliente, como um switch. Uma explosão de MACs (cliente com rede grande/rogue) pode pressionar a tabela — limite quando possível;
- **Broadcast/loop:** é uma bridge distribuída — uma tempestade de broadcast ou um loop no site do cliente **atravessa** para os outros sites. **STP** e controle de broadcast são obrigatórios; o ideal é entregar a VPLS com **isolamento** por VLAN e limites.

Aviso

VPLS é **camada 2 fim a fim** — os riscos do EoIP (Capítulo 55) valem amplificados por ser multiponto. Um loop no cliente vira problema de **todos** os sites dele (e, se mal isolado, um risco ao seu backbone). Entregue VPLS com **STP ativo**, controle de broadcast e, sempre que a necessidade real for “os sites se falarem” (e não “estarem no mesmo broadcast”), prefira **roteamento/L3VPN** — mais seguro e escalável.

58.4 VPLS ou roteamento? A pergunta certa

Nem todo cliente que pede “meus sites conectados” precisa de **L2**. Pergunte o que ele **realmente** precisa:

- **Mesmo domínio de broadcast** obrigatório (aplicação legada que depende de L2, cluster, protocolo não roteável): **VPLS**;
- Só precisa que os sites **se alcancem** (IP): **roteamento** — uma **L3VPN** (VRF por MPLS, Capítulo 46) ou até um túnel roteado (Capítulo 56). Mais seguro (falha não propaga broadcast), mais escalável, tabela de MAC não explode.

A regra de ouro dos volumes anteriores se repete: **roteie quando puder, estenda L2 só quando precisar**. VPLS é uma ferramenta poderosa e lucrativa — e uma responsabilidade.

O conceito mais importante do capítulo

VPLS entrega ao cliente **um switch virtual** sobre o backbone: quadros Ethernet encapsulados em MPLS (Capítulo 57), transportados entre PEs, unidos por **bridge** às portas do cliente. É o produto L2 que se vende caro — e a responsabilidade que vem junto: **MTU** do núcleo para os rótulos, **aprendizado de MAC** sob controle e

os perigos de **broadcast/loop** de uma bridge distribuída (STP obrigatório). Antes de montar, faça a pergunta certa: o cliente precisa mesmo de **L2**, ou **roteamento (L3VPN)** resolveria com menos risco? Poder com julgamento.

58.5 Laboratório

i Ambiente

Reaproveite o núcleo MPLS do Capítulo 57: **lab-pe1** — **lab-p** — **lab-pe2** com OSPF + LDP de pé e MTU ajustado. Adicione um “site do cliente” em cada PE: **lab-cliA** (ligado ao ether5 do lab-pe1) e **lab-cliB** (ligado ao ether5 do lab-pe2), ambos na mesma sub-rede 192.168.50.0/24 — para provar que ficam no **mesmo L2**.

58.5.1 Comandos passo a passo

Passo 1 — confirme o transporte. OSPF Full, LDP com rótulos para os loopbacks, MTU do núcleo folgado (Capítulo 57). Sem isso, a VPLS não sobe.

Passo 2 — pseudowire entre os PEs. No lab-pe1:

```
/interface vpls add name=vpls-cli remote-peer=10.255.0.3 vpls-id=100:100
```

No lab-pe2, o espelho (remote-peer=10.255.0.1, mesmo vpls-id).

Passo 3 — bridge unindo cliente + VPLS. Nos dois PEs:

```
/interface bridge add name=br-cli protocol-mode=rstp
/interface bridge port add bridge=br-cli interface=vpls-cli
/interface bridge port add bridge=br-cli interface=ether5
```

Passo 4 — prove o mesmo L2. Configure lab-cliA=192.168.50.10/24 e lab-cliB=192.168.50.20/24 (sem gateway, mesma sub-rede). Pingue um do outro:

```
/ping 192.168.50.20
```

Funciona **sem roteamento** — eles estão no mesmo broadcast, através do backbone. Confirme o aprendizado de MAC:

```
/interface bridge host print where bridge=br-cli
```

O MAC do lab-cliB aparece **atrás da interface vpls** no lab-pe1.

Passo 5 — MTU ao vivo. Reduza o MTU do núcleo abaixo do necessário e tente transferir algo grande entre os sites: trava. Restaure e volta.

Passo 6 — o perigo do loop. Com RSTP ativo, crie um loop proposital no site do cliente e observe o STP bloquear a porta — a prova de por que STP é obrigatório em VPLS.

58.5.2 Verificação

1. Núcleo MPLS de pé (OSPF+LDP+MTU) — pré-requisito;
2. Pseudowire VPLS estabelecido entre os PEs (mesmo **vpls-id**);
3. Sites do cliente se pingam **no mesmo L2**, sem roteamento;
4. MAC do site remoto aprendido atrás da interface VPLS;
5. STP contém um loop no site do cliente.

58.5.3 Desafios

1. A VPLS “subiu” (interface up), mas os sites não se pingam e nenhum MAC remoto é aprendido. O núcleo está OK. Qual detalhe do **vpls-id** ou da **bridge** você provavelmente errou?
2. Os sites se pingam, mas transferências de arquivos grandes entre eles travam. Você já viu esse sintoma antes (duas vezes neste manual). Qual é a causa no contexto MPLS e como resolve?
3. Um cliente pede “conectar 5 filiais”. Ele descreve que só precisa que os **sistemas se acessem por IP**, nada de aplicação L2 legada. VPLS ou L3VPN? Justifique pelos riscos e pela escala.
4. Após ligar a VPLS, um loop na filial do cliente começou a degradar **todos** os sites dele. Explique o mecanismo (bridge distribuída) e as duas proteções que o teriam contido.

Solução dos desafios

1. Ou o **vpls-id não casa** entre os dois PEs (o circuito não “emparelha”), ou você **esqueceu de colocar a porta do cliente (ether5) e a vpls na mesma bridge** — sem a bridge unindo os dois, o quadro do cliente não entra no pseudowire. Confirme **vpls-id** idêntico e **/interface bridge port print** mostrando ambas as interfaces na mesma bridge.
2. **MTU**: o quadro de 1500 bytes do cliente + rótulos MPLS estoura o MTU do núcleo. É o mesmo sintoma do PPPoE, do OSPF e do próprio Capítulo 57. Solução: **aumentar o MTU/l2mtu** das interfaces do núcleo para acomodar a pilha de rótulos (jumbo). O ping pequeno passa; a transferência grande, não — até corrigir o MTU.
3. **L3VPN** (VRF sobre MPLS, Capítulo 46). Se o requisito é só **alcançabilidade IP**, roteamento é mais **seguro** (uma falha L2 num site não vira tempestade de broadcast nos outros), mais **escalável** (a tabela de MAC não cresce com o cliente) e mais simples de operar com 5 sites. VPLS só se justifica por necessidade **real** de mesmo domínio de

broadcast.

4. É uma **bridge distribuída**: um loop na filial gera tempestade de broadcast que os PEs **replicam para todos os outros sites** do cliente, saturando a VPLS inteira. Proteções: (a) **STP/RSTP** ativo (detecta e bloqueia o loop); (b) **controle/limite de broadcast** e isolamento por VLAN na entrega — e, no fundo, avaliar se **L3VPN** não seria mais adequado para aquele cliente.

58.6 Resumo

- **VPLS** entrega ao cliente um **switch virtual** sobre o backbone: quadros Ethernet encapsulados em **MPLS** e unidos às portas do cliente por **bridge**;
- **Pseudowire/VLL** é ponto a ponto; **VPLS** é multiponto (bridge distribuída, com aprendizado de MAC e replicação de broadcast);
- No v7: interface `vpls` (`remote-peer=loopback`, `vpls-id` casando os lados) + **bridge** com a porta do cliente;
- Cuidados: **MTU** do núcleo para os rótulos, **MACs** sob controle, **STP**/broadcast obrigatórios — os riscos de L2 distribuída, amplificados;
- **Pergunta certa**: precisa mesmo de **L2**? Se é só alcançabilidade IP, **L3VPN/roteamento** (Capítulo 46) é mais seguro e escalável.

VPLS transporta L2; falta a camada que decide **por onde** o tráfego corre no backbone com garantias — reservar banda, escolher caminhos explícitos. O Capítulo 59 fecha o volume com **Traffic Engineering**.

Bibliografia do capítulo

- Documentação oficial: seção *VPLS* e *MPLS* do RouterOS (MikroTik, 2026a);
- Pseudowires, VPLS e L2VPN: (Tanenbaum; Feamster; Wetherall, 2021);
- VPLS no v7 na prática: (Discher, 2016);
- Serviços L2 em backbones de provedor: (Burgess, 2011).

59 Traffic Engineering

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

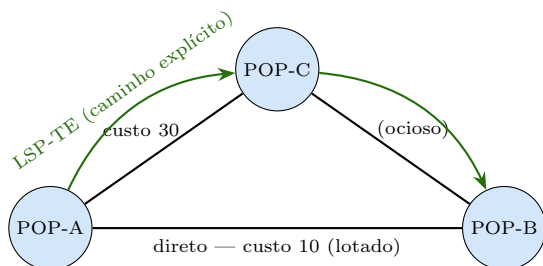
- Explicar o problema que o **Traffic Engineering (TE)** resolve: a IGP manda tudo pelo “menor custo”, deixando caminhos ociosos e enlaces congestionados;
- Entender **túneis TE** (LSPs com caminho **explícito**), **reserva de banda** e como o **CSPF** calcula um caminho que respeita restrições;
- Reconhecer os papéis do **RSVP-TE** e o estado do suporte no RouterOS (o que dá para fazer hoje, e as alternativas práticas);
- Aplicar TE a casos de provedor: **desviar de um enlace caro**, **separar tráfego** por caminho e **balancear** o backbone;
- Fechar o Volume 11 sabendo **quando** TE compensa e quando ajustar **custo/métrica** já basta.

O Capítulo 57 e o Capítulo 58 deram transporte e serviços. Mas ambos seguem o caminho que a **IGP** escolheu — o de **menor custo**. E aí mora um problema clássico de backbone: se a rota mais barata concentra **todo** o tráfego, ela congestiona enquanto um caminho alternativo, um pouco “mais caro”, fica **ocioso**. A IGP otimiza distância, não **utilização**. O **Traffic Engineering** resolve isso: escolher o caminho pelo que **importa ao operador** — banda disponível, latência, evitar um enlace — e não só pelo custo. É a engenharia fina do backbone.

59.1 O problema: a IGP não pensa em capacidade

OSPF (Capítulo 42) escolhe por **custo somado** e manda **tudo** por ali. Imagine dois caminhos entre os POPs A e B: um direto de custo 10 (10 Gbps) e um alternativo de custo 30 (10 Gbps, via C). A IGP usa **só** o de custo 10 — mesmo quando ele satura e o de custo 30 está vazio. Você tem 20 Gbps de capacidade e usa 10.

Ajustar custo ajuda até certo ponto, mas é grosseiro: mexer no custo **move todo** o tráfego de uma vez, podendo só transferir o congestionamento. O que falta é **colocar tráfego específico num caminho específico**, com **reserva** — e é isso que o TE faz.



A IGP mandaria tudo pelo caminho direto (custo 10), congestionando-o. Um **túnel TE** roteia parte do tráfego **explicitamente** por POP-C, aproveitando a capacidade ociosa. TE otimiza **utilização**, não só distância.

Figura 59.1

59.2 Túneis TE, caminho explícito e reserva

O TE trabalha com **túneis** — LSPs (Capítulo 57) que **não** seguem a IGP, mas um **caminho definido**:

- **Caminho explícito** (*explicit path*): você lista os saltos por onde o LSP deve passar (ex.: A → C → B), forçando o desvio;
- **Reserva de banda**: o LSP **reserva** X Gbps ao longo do caminho; os roteadores contabilizam a banda comprometida e **rejeitam** novos LSPs que não caibam — evitando sobrecarga por projeto;
- **CSPF** (*Constrained SPF*): em vez de você listar todos os saltos, o roteador **calcula** o melhor caminho que **satisfaz as restrições** (banda mínima, evitar tal enlace, latência) — Dijkstra com regras.

Para o CSPF funcionar, a IGP precisa **anunciar os atributos TE** dos enlaces (banda disponível, cores/afinidades) — extensões TE do OSPF.

59.3 RSVP-TE e o estado no RouterOS

O protocolo que **sinaliza** e **reserva** os túneis TE é o **RSVP-TE** (*Resource Reservation Protocol — Traffic Engineering*): ele estabelece o LSP ao longo do caminho e mantém a reserva. No mundo MPLS-TE clássico, RSVP-TE é a peça central; LDP (Capítulo 57) faz o transporte “sem engenharia”, RSVP-TE faz o “com caminho e banda”.

⚠ Aviso

O suporte a **RSVP-TE/MPLS-TE completo** no RouterOS é **limitado e varia por versão** — não assuma paridade com roteadores de core dedicados (Cisco/Juniper).

Confirme, na documentação da sua versão v7 (MikroTik, 2026a), o que está disponível **antes** de projetar um backbone dependente de TE. Muitas redes MikroTik obtêm 90% do benefício com **engenharia de custo/métrica e múltiplos LSPs/ECMP**, sem RSVP-TE formal. Este capítulo cobre o **conceito** e as alternativas práticas.

59.4 TE na prática de um provedor MikroTik

Mesmo sem RSVP-TE pleno, você faz “engenharia de tráfego” com as ferramentas dos volumes anteriores:

- **Métrica/custo por serviço:** ajustar custo OSPF (Capítulo 42) para mover **classes** de tráfego a caminhos diferentes — grosseiro, mas eficaz para separar, por exemplo, tráfego de trânsito de tráfego interno;
- **Policy routing** (Capítulo 46): marcar tráfego (mangle) e mandá-lo por uma **tabela**/next-hop específico — coloca tráfego selecionado num caminho, sem depender de RSVP;
- **Múltiplos caminhos / ECMP:** quando dois caminhos têm o **mesmo custo**, a IGP balanceia entre eles (o Volume 14 aprofunda, 18) — capacidade somada sem TE;
- **VPLS/pseudowire por caminho:** ancorar um serviço L2 (Capítulo 58) a um LSP específico para isolá-lo do resto.

A regra: **comece pelo simples** (custo, ECMP, policy routing) e só vá para TE formal quando o backbone for grande e complexo o bastante para justificar a operação de RSVP-TE — e depois de confirmar o suporte.

! O conceito mais importante do capítulo

TE existe porque a **IGP otimiza distância, não utilização**: ela lota o caminho barato e ignora o ocioso. A solução conceitual é o **túnel TE** — um LSP com **caminho explícito e reserva de banda**, calculado por **CSPF** e sinalizado por **RSVP-TE**. Mas o conselho honesto para um provedor MikroTik: **confirme o suporte da sua versão** e saiba que a maior parte do ganho vem de ferramentas que você já domina — **custo, ECMP e policy routing**. TE formal é para quando o backbone realmente exige; antes disso, engenharia de métrica resolve.

59.5 Laboratório

i Ambiente

Quatro CHRs (Capítulo 5) formando um losango: **lab-a** e **lab-b** nas pontas, com **dois** caminhos entre eles — um direto (**lab-a-lab-b**) e um via **lab-c** (**lab-a-lab-c-lab-b**). OSPF + MPLS/LDP de pé (Capítulo 57). O objetivo é **desviar** tráfego selecionado do caminho direto para o via-C, demonstrando o conceito de TE com as ferramentas disponíveis.

59.5.1 Comandos passo a passo

Passo 1 — o estado padrão da IGP. Com custos iguais ou o direto mais barato, confirme que **todo** o tráfego A→B vai pelo caminho direto:

```
/tool traceroute 10.255.0.2
/ip route print where dst-address~"10.255.0.2"
```

Passo 2 — engenharia por custo. Suba o custo do enlace direto para que **uma classe** de tráfego prefira o via-C — ou, melhor, deixe os dois com **mesmo custo** e observe o **ECMP** balancear:

```
/routing ospf interface-template set [find interfaces=ether-direto] cost=20
```

Ref faça o traceroute: o caminho mudou/balanceou.

Passo 3 — policy routing (TE seletivo). Marque um tráfego específico (ex.: de uma origem) e force-o pelo via-C, deixando o resto no direto:

```
/ip firewall mangle add chain=prerouting src-address=192.168.70.0/24 \
    action=mark-routing new-routing-mark=via-c
/routing table add name=via-c fib
/ip route add dst-address=10.255.0.2/32 gateway=<nexthop-c> routing-table=via-c
```

Prove com traceroute de dentro e fora do 192.168.70.0/24: caminhos diferentes — tráfego colocado **explicitamente** num caminho.

Passo 4 — capacidade somada (ECMP). Com os dois caminhos em custo igual, gere tráfego e observe a distribuição pelos dois — capacidade agregada sem RSVP-TE.

Passo 5 — confira o suporte TE da sua versão. Consulte `/mpls` e a documentação (MikroTik, 2026a) para ver o que sua versão v7 oferece de RSVP-TE, antes de qualquer projeto dependente disso.

59.5.2 Verificação

1. Por padrão, o tráfego A→B usa o caminho de menor custo;
2. Ajuste de custo/ECMP redistribui ou balanceia o tráfego;
3. Policy routing coloca uma classe **explicitamente** no via-C;
4. Você verificou o suporte real de TE da sua versão.

59.5.3 Desafios

1. O enlace direto A–B vive congestionado enquanto o via-C está ocioso. Você sobe o custo do direto e... o congestionamento só **migrou** para o via-C. Por que o ajuste de custo “puro” falhou, e que abordagem coloca **parte** do tráfego em cada caminho?
2. Explique, com suas palavras, a diferença entre o LDP (Capítulo 57) e o RSVP-TE quanto ao **caminho** que o LSP segue. Quando o LDP basta e quando o RSVP-TE agrega valor?
3. Um projeto assume RSVP-TE com reserva de banda estrita num backbone MikroTik. Que verificação **crítica** você faz antes de prometer isso ao cliente/gestão, e qual é o plano B se o suporte for limitado?
4. Você quer somar a capacidade de **dois** caminhos de mesma banda entre A e B sem TE formal. Qual recurso usa, que condição os caminhos precisam satisfazer, e qual capítulo aprofunda o tema?

Solução dos desafios

1. Ajustar custo **move todo** o tráfego de um caminho para o outro — você não dividiu a carga, só a transferiu, e o via-C agora congestionava. Para colocar **parte** em cada caminho: **policy routing** (marcar uma classe e mandá-la por um caminho, o resto por outro) ou **ECMP** (custos iguais, a IGP balanceia). O verdadeiro TE reservaria banda por LSP; as alternativas práticas dividem por classe ou por hash de fluxo.
2. O **LDP** distribui rótulos seguindo o caminho da **IGP** (menor custo) — o LSP vai por onde o OSPF mandaria. O **RSVP-TE** estabelece o LSP por um **caminho explícito/CSPF** que pode **divergir** da IGP, respeitando restrições (banda, evitar enlace). LDP basta para transporte simples e VPNs sobre o caminho ótimo da IGP; RSVP-TE agrega quando você precisa **forçar caminho** e **reservar banda**.
3. Verificação crítica: **confirmar, na documentação da versão v7 específica**, o nível de suporte a RSVP-TE/MPLS-TE do RouterOS — ele é limitado e varia. Plano B: obter o resultado com **engenharia de custo/métrica, ECMP e policy routing** (que cobrem a maior parte dos casos), reservando o RSVP-TE formal para quando o hardware/versão o suportarem plenamente — ou usar equipamento de core dedicado onde TE estrito for mandatório.
4. **ECMP** (*Equal-Cost Multi-Path*): os dois caminhos precisam ter **custo IGP idêntico** para a IGP instalá-los juntos e balancear os fluxos entre eles. Aprofundado no Volume 14 (18), junto com PCC e failover multi-link.

59.6 Resumo

- **TE** resolve o que a IGP não vê: ela otimiza **distância**, lotando o caminho barato e deixando o alternativo **ocioso**;
- Conceitos: **túnel TE** (LSP com **caminho explícito**), **reserva de banda**, **CSPF** (SPF com restrições) e **RSVP-TE** (sinaliza/reserva);
- No RouterOS, o suporte a RSVP-TE é **limitado/variável** — **confirme a versão** (MikroTik, 2026a) antes de projetar;
- Na prática MikroTik, a “engenharia de tráfego” vem de **custo/métrica**, **policy routing** (Capítulo 46) e **ECMP** (18) — comece pelo simples;
- Regra: TE formal só quando o backbone justificar; antes disso, engenharia de métrica entrega a maior parte do ganho.

Fecha-se o Volume 11: o backbone do provedor agora transporta por rótulos (MPLS/LDP), entrega serviços L2 (VPLS) e escolhe caminhos com intenção (TE). Da tecnologia de núcleo, o **Volume 12** volta ao operador: **scripts e automação** para provisionar e manter tudo isso em escala.

Bibliografia do capítulo

- Documentação oficial: *MPLS-TE / RSVP* do RouterOS (verifique a versão) (MikroTik, 2026a);
- Traffic engineering, CSPF e reserva de recursos: (Tanenbaum; Feamster; Wetherall, 2021);
- MPLS no v7 na prática: (Discher, 2016);
- Engenharia de backbone em provedores: (Burgess, 2011).

Parte XII

Volume 12 — Scripts e automação

60 Linguagem de script do RouterOS

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Escrever scripts no RouterOS: **variáveis** (:local/:global), **tipos**, **condicionais** (:if) e **loops** (:foreach/:for);
- Entender a peça central do RouterOS script: percorrer e **agir sobre itens** de menus (/interface, /ip address) com **find**, [] e **do**;
- Capturar saída de comandos, **parsear** valores e montar mensagens (log, e-mail) com interpolação;
- Criar **funções** reutilizáveis (via :global) e tratar erros com :do { } on-error={ };
- Escrever scripts **idempotentes e seguros** — que podem rodar de novo sem duplicar nem quebrar a configuração.

Até aqui, cada configuração foi **manual**. Funciona para um roteador; não escala para um parque de centenas. A **linguagem de script** do RouterOS transforma tarefas repetitivas em código: verificar links, aplicar mudanças em massa, reagir a eventos, gerar relatórios. Não é uma linguagem “de programador” — é enxuta e cheia de peculiaridades — mas domina-se o essencial rápido, e o retorno é enorme: o que você faria em 200 roteadores à mão, um script faz em segundos. Este capítulo é a base; os próximos automatizam (Capítulo 61) e provisionam em massa (Capítulo 62).

60.1 Variáveis e tipos

Toda variável é declarada com **escopo**:

- **:local** — vive só dentro do script/bloco atual (o padrão que você deve usar);
- **:global** — persiste entre scripts e sessões (use com parcimônia; é como uma variável global de verdade — poderosa e perigosa).

```
:local nome "borda-pop1"
:local contador 0
:local ativo true
:local rede 192.168.1.0/24
```

O RouterOS **infere o tipo** pelo valor: `num`, `str`, `bool`, `ip`, `ip-prefix`, `time`, `array`, e outros. Tipos importam — comparar um `str` “10” com um `num` 10 dá resultado inesperado. Converta quando necessário (`[:tonum $x]`, `[:tostr $x]`).

Aviso

A sintaxe do RouterOS script é **exigente e sem perdão**: `:set x 5` (não `x = 5`), o **\$** só para ler a variável (`:put $x`), espaços ao redor de operadores importam, e `{ }` de blocos têm regras próprias. Erros de sintaxe costumam falhar em silêncio ou com mensagens crípticas. **Teste incrementalmente** — linha a linha no terminal antes de montar o script inteiro.

60.2 Condicionais e a estrutura do controle

```
:local sinal -65
:if ($sinal < -70) do={
    :log warning "Sinal fraco: $sinal dBm"
} else={
    :log info "Sinal ok: $sinal dBm"
}
```

Note a interpolação: dentro de "...", `$sinal` é **substituído** pelo valor. É assim que se montam mensagens de log e e-mail.

60.3 Loops e a operação sobre itens de menu

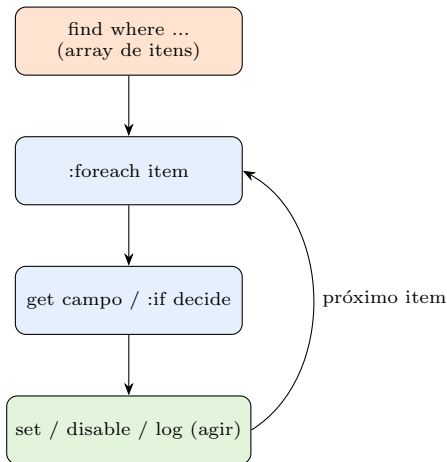
O coração do RouterOS script é **agir sobre os itens** de um menu. Duas peças fazem isso: **find** (retorna os itens que casam um critério) e **foreach** (percorre e age). Exemplo: desabilitar todas as interfaces cujo comentário contém “teste”:

```
:foreach i in=[/interface find where comment~"teste"] do={
    /interface disable $i
    :log info "desabilitada: $[/interface get $i name]"
}
```

- `[/interface find where ...]` retorna um **array** de referências internas;
- `/interface get $i name` lê um campo daquele item;
- `/interface disable $i` age sobre ele.

Esse padrão — `find` → `foreach` → `get/set/ação` — é 80% dos scripts úteis de RouterOS. O `:for` numérico também existe para contagens:

```
:for n from=1 to=5 do={ :log info "iteracao $n" }
```



O padrão que resolve a maioria dos scripts: **find** seleciona os itens, **:foreach** percorre, **get/:if** decidem, e **set/ação** agem. Domine este ciclo e você automatiza quase tudo no RouterOS.

Figura 60.1

60.4 Capturando saída e parseando

Comandos entre [] retornam valor. Combinado com **:tonum**, **:pick** (fatiar string) e os arrays, você extrai o que precisa:

```
# quantos clientes PPPoE ativos?
:local n [:len [/interface pppoe-server active find]]
:log info "Sessoes PPPoE ativas: $n"

# ler um campo e comparar
:local up [/interface get ether1 running]
:if ($up = false) do={ :log error "ether1 caiu!" }
```

60.5 Funções reutilizáveis

Funções são criadas como **:global** contendo um bloco, e aceitam parâmetros nomeados:

```
# define uma função de notificação
:global notificar do={
    :log warning "[ALERTA] $mensagem"
    # (aqui poderia disparar e-mail/telegram)
}

# usa em qualquer script
$notificar mensagem="link de trânsito degradado"
```

Isso permite centralizar lógica (notificação, verificação) e reusá-la — a base de scripts de operação organizados.

60.6 Tratamento de erro e idempotência

Um script de produção **não pode** derrubar o roteador ao encontrar um erro. Envolve o que pode falhar:

```
:do {
    /ip address add address=10.0.0.1/24 interface=ether2
} on-error={
    :log warning "endereço já existe ou interface inválida"
}
```

E, crucialmente, torne os scripts **idempotentes** — seguros para rodar de novo. Em vez de **add** cego (que duplica), **verifique antes**:

```
:if ([ :len [/ip address find where address="10.0.0.1/24"] ] = 0 ) do={
    /ip address add address=10.0.0.1/24 interface=ether2
}
```

! O conceito mais importante do capítulo

O RouterOS script gira em torno de **agir sobre itens de menu**: o ciclo **find** → **:foreach** → **get**/**:if** → **set**/**ação** resolve a maioria das tarefas reais. Sobre isso, duas disciplinas que separam script de brinquedo de script de produção: **idempotência** (verifique antes de **add**, para poder rodar de novo sem duplicar) e **tratamento de erro** (**:do {} on-error={}**, para não derrubar o roteador). A sintaxe é exigente — teste incrementalmente. Dominado o ciclo, automação, agenda (Capítulo 61) e provisionamento em massa (Capítulo 62) viram aplicação direta.

60.7 Laboratório

i Ambiente

Um CHR (Capítulo 5) com algumas interfaces e endereços configurados — o suficiente para percorrer, ler e agir. Todo o laboratório roda no terminal (`/system script` para salvar, ou colando direto). **Não** é preciso topologia complexa: o foco é a linguagem.

60.7.1 Comandos passo a passo

Passo 1 — variáveis e tipos. No terminal:

```
:local x 10
:local y "10"
:put ($x = 10)
:put ($x = $y)
```

Observe: comparar num com **str** **não** dá igual — a lição de tipos.

Passo 2 — o ciclo find/foreach. Liste e aja sobre interfaces:

```
:foreach i in=[/interface find] do={
    :put "$[/interface get $i name] running=$[/interface get $i running]"
}
```

Passo 3 — condicional útil. Alerte sobre interface caída:

```
:foreach i in=[/interface find where !running] do={
    :log warning "interface caída: $[/interface get $i name]"
}
```

Passo 4 — idempotência. Rode este bloco **duas vezes** e confirme que o endereço **não** duplica:

```
:if ([ :len [/ip address find where address="10.9.9.1/24"] ] = 0) do={
    /ip address add address=10.9.9.1/24 interface=ether1
    :log info "endereço adicionado"
} else={ :log info "endereço já existe, nada a fazer" }
```

Passo 5 — função reutilizável. Defina `$notificar` (bloco acima) e chame-a de dois scripts diferentes; veja as mensagens no log.

Passo 6 — salve como script nomeado.

```

/system script add name=checa-links source={
    :foreach i in=[/interface find where !running] do={
        :log warning "caiu: $[/interface get $i name]"
    }
}
/system script run checa-links

```

60.7.2 Verificação

1. Comparação **num** vs **str** mostra a diferença de tipos;
2. O ciclo **find/foreach** percorre e lê campos das interfaces;
3. O bloco idempotente **não** duplica o endereço ao rodar de novo;
4. A função **\$notificar** funciona chamada de scripts distintos;
5. O script nomeado roda por **/system script run**.

60.7.3 Desafios

1. Um colega escreveu um script com **add** cego que rodou no scheduler a cada 5 min por um dia — e agora há centenas de regras/endereços duplicados. Explique o erro e reescreva o trecho de forma **idempotente**.
2. Um script trava o processamento e o roteador fica estranho quando uma interface esperada não existe. Qual construção evita isso, e como você a aplicaria ao **add** de um endereço numa interface que **pode** não estar lá?
3. Você precisa desabilitar **todas** as regras de firewall com o comentário “temporario” e logar quantas foram. Escreva o script usando o ciclo **find/foreach** + contador.
4. Por que usar **:global** para tudo é perigoso, e quando ele é **realmente** a escolha certa? Dê um exemplo de cada.

Solução dos desafios

1. O erro é **não verificar antes de adicionar**: **add** sempre cria, então rodar de novo duplica. Idempotente:

```

:if ([ :len [/ip firewall address-list find where list="bloqueados" address="1.2.3.4"] ] = 0 ) {
    /ip firewall address-list add list=bloqueados address=1.2.3.4
}

```

A regra: **find primeiro**, **add só se não existir** (ou **set** se existir). Limpeza dos duplicados: um **foreach** removendo os excedentes.

2. **:do { } on-error={ }**: envolve a operação que pode falhar, de modo que o erro seja tratado (logado) em vez de abortar o script.

```

:do { /ip address add address=10.0.0.1/24 interface=ether9 } \
    on-error={ :log warning "ether9 inexistente ou endereco duplicado" }

```


Melhor ainda, combinar com verificação de existência da interface antes.

3.

```
:local c 0
:foreach r in=[/ip firewall filter find where comment="temporario"] do={
    /ip firewall filter disable $r
    :set c ($c + 1)
}
:log info "regras temporarias desabilitadas: $c"
```

4. **:global** **persiste** entre scripts e sessões: usado à toa, vira estado escondido que outro script sobrescreve, causando bugs difíceis de achar (ação à distância). Perigoso: **:global** **i** num loop que colide com outro script. **Certo:** quando você **quer** compartilhar deliberadamente — uma **função** reutilizável (**:global** **notificar**) ou uma configuração lida por vários scripts. Regra: **:local** por padrão, **:global** só com intenção explícita.

60.8 Resumo

- Variáveis com **escopo** (**:local** padrão, **:global** com parcimônia); o RouterOS **inere tipos** — e comparar tipos diferentes engana;
- O ciclo central: **find** → **:foreach** → **get/****:if** → **set/ação** — resolve a maioria dos scripts (agir sobre itens de menu);
- Capture saída com **[]**, parseie com **:len/****:tonum/****:pick**, interpole em **"...\$var..."** para log/mensagens;
- **Funções** via **:global do={}** centralizam e reusam lógica;
- Produção exige **idempotência** (verifique antes de **add**) e **tratamento de erro** (**:do {} on-error={}**) — e teste incrementalmente, a sintaxe é exigente.

Sabendo escrever scripts, o próximo passo é fazê-los rodar **sozinhos** — no horário certo, em reação a um evento, guardando backups. O Capítulo 61 cobre scheduler, netwatch e backups automáticos.

Bibliografia do capítulo

- Documentação oficial: seção *Scripting* do RouterOS (MikroTik, 2026a);
- Lógica de programação e automação: (Tanenbaum; Feamster; Wetherall, 2021);
- Scripts no v7 na prática: (Discher, 2016);
- Automação de parque em provedores: (Burgess, 2011).

61 Scheduler, netwatch e backups automáticos

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Agendar scripts com o **scheduler** (por horário, intervalo ou no boot) e entender **start-time/interval**;
- Reagir a **eventos de rede** com o **netwatch** (host up/down, disparando scripts de **on-up/on-down**);
- Automatizar **backups** — binário e **export** — e enviá-los para fora por **e-mail** e **fetch** (FTP/HTTP);
- Combinar as três peças em rotinas reais de provedor: failover por netwatch, backup diário externo, verificação periódica;
- Escrever automações **seguras** (idempotentes, com log e limites) que não viram um tiro no pé.

O 19 deu a linguagem; falta fazer os scripts rodarem **sozinhos**. Três ferramentas transformam código em **operação autônoma**: o **scheduler** (rodar no tempo certo), o **netwatch** (rodar quando algo cai ou volta) e os **backups automáticos** (guardar a configuração fora da caixa, sem depender de você lembrar). Juntas, elas são a diferença entre um provedor que **reage** a problemas de madrugada e um que **se defende** sozinho — e sempre tem backup quando precisa.

61.1 Scheduler: rodar no tempo certo

O **scheduler** dispara scripts por horário ou intervalo:

```
# todo dia às 03:00 - backup noturno
/system scheduler
add name=backup-diario start-time=03:00:00 interval=1d \
    on-event="/system script run backup-externo"

# a cada 5 minutos - verificação de links
/system scheduler
add name=checa-links interval=5m on-event="/system script run checa-links"
```

```
# no boot - reaplica algo após reinício
/system scheduler
add name=pos-boot start-time=startup on-event="/system script run ajustes-boot"
```

- `interval=1d/5m/1h` — período de repetição;
- `start-time=03:00:00` — horário fixo; `start-time=startup` — roda no boot;
- `on-event` — o que executar (chame um script nomeado, não cole lógica gigante inline).

⚠ Aviso

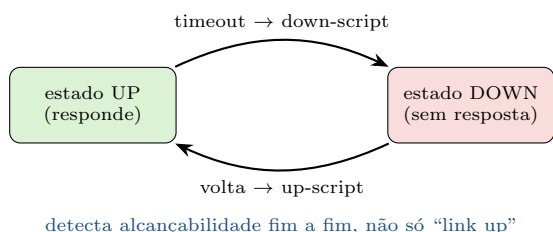
Scheduler + script **não idempotente** = desastre em escala de tempo: um `add` cego rodando a cada 5 min gera milhares de duplicatas por dia (19). **Antes** de agendar, garanta que o script pode rodar **infinitas vezes** sem estragar nada. E **sempre logue** o que a automação faz — automação silenciosa que falha é a pior de diagnosticar.

61.2 Netwatch: reagir a eventos de rede

O **netwatch** monitora um host (ping/TCP/HTTP) e dispara scripts quando ele **cai** (`on-down`) ou **volta** (`on-up`). É a base de failover caseiro e de alertas:

```
# monitorar o gateway de trânsito; agir na queda e na volta
/tool netwatch
add host=198.51.100.1 interval=10s timeout=1s \
    down-script="/system script run transito-caiu" \
    up-script="/system script run transito-voltou"
```

Um uso clássico é **failover de rota** (complementar ao `check-gateway` do Capítulo 41): quando o trânsito primário cai, o `down-script` ativa a rota reserva; quando volta, o `up-script` a restaura. O netwatch detecta **alcançabilidade fim a fim** (o host lá longe), não só “o link está up” — mais confiável que olhar a interface.



61.3 Backups automáticos: binário, export e envio externo

Há **dois** tipos de backup (você viu no Volume 1):

O netwatch acompanha a **alcançabilidade** de um host e dispara scripts nas transições. É a peça reativa da automação: failover, alerta, autorrecuperação — tudo pendurado no `up-script/down-script`.

Figura 61.1

- **Backup binário** (`/system backup save`): imagem completa, restaura idêntico **na mesma versão/hardware** — bom para recuperação rápida;
- **Export** (`/export`): texto legível dos comandos, **portável** e versionável (ideal para git/auditoria) — mas não guarda senhas por padrão.

O ponto crítico: o backup **não pode morar só na caixa** (se ela pifa, o backup vai junto). Automatize o **envio para fora**:

```
# script backup-externo: gera e envia por e-mail
/system script add name=backup-externo source={
    :local nome ("bkp-" . [/system identity get name] . "-" . \
        [:pick [/system clock get date] 0 11])
    /system backup save name=$nome
    /export file=$nome
    # enviar por e-mail (configurar /tool e-mail antes)
    /tool e-mail send to="noc@provedor.net" subject="Backup $nome" \
        file="$nome.backup"
}
```

Alternativas de envio: `/tool fetch` para um servidor FTP/HTTP central (`upload=yes`), ou copiar via script para um repositório. O Volume 15 (`?@sec-hardening-operacional`) integra isso à rotina de parque. Combine com o scheduler (bloco `backup-diario` acima) e você tem backup diário externo **sem depender de memória humana**.

61.4 Juntando tudo: uma rotina de provedor

As três peças se combinam em automações reais:

- **Backup diário externo**: scheduler 03:00 → `backup-externo` → e-mail/FTP;
- **Failover de trânsito**: netwatch no gateway → `down-script` ativa reserva, `up-script` restaura (com log e, idealmente, alerta);
- **Sentinela de saúde**: scheduler 5m → script que checa links, CPU, temperatura e **notifica** (a função `$notificar` do
19) só quando há problema;
- **Autolimpeza**: scheduler diário removendo address-lists expiradas, logs antigos, etc.

! O conceito mais importante do capítulo

Automação no RouterOS são **três gatilhos**: **tempo** (scheduler), **evento de rede** (netwatch) e a garantia que a operação exige — o **backup externo automático**. O netwatch reage a **alcançabilidade fim a fim** (mais confiável que “link up”) e o backup só serve se sair da caixa. Duas regras acima de tudo: automação agendada **exige scripts idempotentes** (senão o tempo multiplica o erro) e **tudo loga** (uma automação silenciosa que falha é pior que não ter automação). Faça a rede se defender sozinha — e ter backup quando não conseguir.

61.5 Laboratório

i Ambiente

Dois CHRs (Capítulo 5): **lab-rt** (onde rodam scheduler/netwatch/ backup) e **lab-alvo** (o host que o netwatch monitora, para você poder derrubá-lo e ver a reação). Opcional: um servidor FTP/HTTP simples para o envio de backup, ou apenas o log/e-mail simulado.

61.5.1 Comandos passo a passo

Passo 1 — script idempotente de verificação. Reuse o `checa-links` do 19 (logando interfaces caídas).

Passo 2 — agende-o.

```
/system scheduler add name=checa interval=1m on-event="/system script run checa-links"
```

Observe o log a cada minuto. Confirme que rodar repetido **não** causa efeito colateral (idempotência).

Passo 3 — netwatch reagindo. Monitore o lab-alvo:

```
/system script add name=alvo-caiu source={ :log warning "ALVO CAIU" }  
/system script add name=alvo-voltou source={ :log info "ALVO VOLTOU" }  
/tool netwatch add host=<ip-lab-alvo> interval=5s \  
    down-script="/system script run alvo-caiu" \  
    up-script="/system script run alvo-voltou"
```

Passo 4 — provoque a transição. Desligue a interface do lab-alvo (ou o CHR) e veja o `down-script` disparar; religue e veja o `up-script`. A automação reativa ao vivo.

Passo 5 — backup automático. Crie o script `backup-externo` (gere `backup + export`) e agende-o. Se tiver FTP, use `/tool fetch upload=yes` para enviá-lo; senão, confirme que os arquivos foram gerados (`/file print`) e simule o envio.

Passo 6 — failover por netwatch (opcional). Faça o `down-script` ativar uma rota reserva e o `up-script` restaurá-la; derrube o alvo e confirme a troca de caminho.

61.5.2 Verificação

1. O scheduler dispara o script no intervalo, sem efeito colateral;
2. O netwatch dispara `down/up` nas transições do alvo;
3. O backup é gerado (binário + export) e enviado/salvo fora da caixa;
4. (Opcional) o failover por netwatch troca a rota na queda e restaura.

61.5.3 Desafios

1. Um script de backup agendado enche o disco do roteador em duas semanas. O envio externo funciona. Qual passo faltou na automação, e como você o adiciona sem apagar o backup do dia?
2. Um netwatch com `interval=1s timeout=1s` num link de rádio com jitter fica **oscilando** (up/down/up), disparando os scripts o tempo todo. Explique o problema e como estabilizar sem perder a detecção real de queda.
3. O failover por netwatch ativou a rota reserva, mas quando o primário voltou o tráfego **não** retornou a ele. O que faltou no `up-script`, e por que confiar só no `down-script` é meio failover?
4. Por que o netwatch (host remoto) costuma ser **mais confiável** para failover do que olhar só o estado da interface local? Dê um cenário em que a interface está “up” mas o caminho está morto.

Solução dos desafios

1. Faltou a **rotação/limpeza** dos arquivos antigos. O backup é gerado e enviado, mas as cópias locais se acumulam. Adicione ao script uma limpeza dos backups locais com mais de N dias:

```
:foreach f in=[/file find where name~"bkp-" && creation-time<([/system clock get date] -
```

(ajuste a comparação de data à sua versão). Assim o disco não enche e o backup do dia permanece.

2. `timeout` curto demais num link com jitter marca “down” a cada atraso momentâneo — **flapping**. Estabilize aumentando o `timeout` e o `interval` (ex.: `interval=10s timeout=3s`) e/ou exigindo várias falhas seguidas antes de agir (lógica de contador no script). Você troca um pouco de tempo de detecção por **estabilidade** — um flap não pode disparar failover.

3. Faltou o `up-script` **restaurar** o estado primário (reativar a rota primária / desativar

a reserva). Só o **down-script** faz **metade**: detecta a queda e desvia, mas sem o retorno automático você fica preso na reserva (pior caminho) até intervir à mão. Failover completo = **desviar na queda E voltar na recuperação**, ambos automáticos e com log.

4. A interface pode estar **up** (portadora presente) enquanto o **caminho** além dela está morto — o equipamento do outro lado travou, o upstream da operadora caiu, uma rota sumiu. Olhar só a interface não vê isso. O netwatch pinga um **host remoto** (ex.: o gateway da operadora ou um IP na internet): se ele não responde, o caminho está realmente quebrado, ainda que sua interface diga “up”. Detecção fim a fim > estado de link.

61.6 Resumo

- **Scheduler** roda scripts por **horário/intervalo/boot** (`start-time`, `interval`, `on-event`);
- **Netwatch** reage a **eventos de rede** (`down-script/up-script`), detectando **alcançabilidade fim a fim** — base de failover e alerta;
- **Backups automáticos: binário** (recuperação) + **export** (portável/versionável), **enviados para fora** (e-mail/`fetch`) — nunca só na caixa;
- Combine as três em rotinas: backup diário externo, failover por netwatch (desviar e voltar), sentinela de saúde, autolimpeza;
- Regras de ouro: **idempotência** (o tempo multiplica erros) e **log em tudo** — automação silenciosa que falha é a pior.

Scheduler e netwatch automatizam **um** roteador. Falta o salto de escala: configurar e provisionar **centenas** de uma vez. O Capítulo 62 cobre a API (binária e REST) e o provisionamento em massa.

Bibliografia do capítulo

- Documentação oficial: *Scheduler*, *Netwatch*, *Backup*, *E-mail*, *Fetch* do RouterOS (MikroTik, 2026a);
- Automação reativa e monitoramento: (Tanenbaum; Feamster; Wetherall, 2021);
- Rotinas de operação no v7: (Discher, 2016);
- Automação de parque em provedores: (Burgess, 2011).

62 API e provisionamento em massa

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Distinguir as formas de automação **externa** do RouterOS: **API binária**, **REST API** (v7) e **SSH scripting**, e quando usar cada uma;
- Habilitar a **REST API** com segurança e fazer chamadas (GET/PUT/POST/PATCH/DELETE) para ler e alterar configuração;
- Estruturar **provisionamento em massa**: um sistema central que configura centenas de equipamentos a partir de um modelo (*template*);
- Entender a **segurança** da automação em escala: usuário dedicado, **allowed-address**, TLS, e o risco de credenciais em scripts;
- Escolher entre configurar **na caixa** (script local, Capítulo 61) e **de fora** (API), e integrar com o sistema de gestão do provedor.

Os capítulos anteriores automatizaram **dentro** de um roteador. Mas o provedor tem **centenas** deles, e configurá-los um a um — mesmo com scripts locais — não escala nem se integra ao sistema de gestão (o ERP que cadastra clientes, emite boletos, provisiona planos). A resposta é a **automação externa**: um sistema central que **fala** com os roteadores por uma interface programável (**API**) e os configura em massa, a partir de modelos. Este capítulo fecha o Volume 12 conectando o RouterOS ao mundo dos sistemas — e à operação de verdade.

62.1 As três formas de falar com o RouterOS de fora

Forma	Transporte	Quando usar
API binária	porta 8728 (8729 TLS)	Alto volume, bibliotecas maduras (Python librouteros , PHP); protocolo próprio
REST API (v7)	HTTP(S), porta 80/443	Integração moderna, JSON, fácil de qualquer linguagem/ferramenta

Forma	Transporte	Quando usar
SSH scripting	SSH (22)	Rodar comandos/scripts pontuais, sem serviço extra; bom para automação simples

A **REST API** do v7 é a novidade que muda o jogo: fala **JSON** sobre **HTTPS**, então qualquer linguagem (Python, Go, até `curl`) integra sem biblioteca especial. A **API binária** ainda vale para alto volume e onde já há bibliotecas consolidadas. **SSH** é o canivete para o pontual.

62.2 Habilitando a REST API com segurança

A REST API roda sobre o serviço **www-ssl** (nunca sobre HTTP puro em produção — credenciais em claro!):

```
# certificado + www-ssl (a REST API vive aqui)
/ip service set www-ssl certificate=cert-api disabled=no address=10.99.0.0/24
/ip service set www disabled=yes    # desligue o HTTP puro

# usuário DEDICADO para automação, com acesso restrito
/user add name=api-prov group=write \
    address=10.99.0.10 comment="sistema de provisionamento"
```

Aviso

Automação em escala **multiplica** o custo de um erro de segurança: uma credencial de API vazada abre **todos** os roteadores de uma vez. Regras inegociáveis: **TLS sempre** (`www-ssl`, `api-ssl`), **usuário dedicado** com o **mínimo** de privilégio, **allowed-address** limitando de onde a API aceita conexão (só o IP do sistema de gestão), e **nunca** hardcode de senha em scripts versionados. Trate a credencial de provisionamento como a chave do reino — porque é.

62.3 Fazendo chamadas REST

A REST API espelha a estrutura de menus do RouterOS. O caminho da URL é o menu; o método HTTP é a ação:

```
# LER endereços IP (GET)
curl -sk -u api-prov:senha \
    https://10.99.0.1/rest/ip/address
```

```
# ADICIONAR um endereço (PUT com JSON)
curl -sk -u api-prov:senha -X PUT \
  https://10.99.0.1/rest/ip/address \
  -H "Content-Type: application/json" \
  -d '{"address":"192.168.50.1/24","interface":"ether2"}'

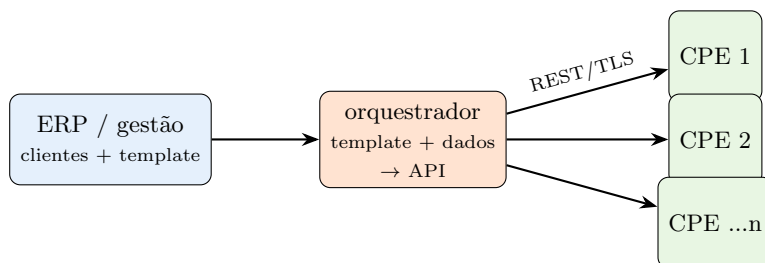
# ALTERAR um item existente (PATCH pelo .id)
curl -sk -u api-prov:senha -X PATCH \
  https://10.99.0.1/rest/ip/address/*A \
  -d '{"disabled":"true"}'

# executar comando (POST em /rest/<caminho>/<comando>)
curl -sk -u api-prov:senha -X POST \
  https://10.99.0.1/rest/system/reboot
```

O mapeamento é direto: `/ip address` vira `/rest/ip/address`, `/interface print` vira `GET /rest/interface`, e assim por diante. Isso torna a REST API previsível — se você sabe o CLI, sabe a URL.

62.4 Provisionamento em massa: o padrão template

Configurar 300 CPEs à mão é inviável; o padrão é **template** + **parâmetros**. O sistema central guarda um **modelo** de configuração e, para cada equipamento, **preenche** os valores específicos (IP, VLAN, identidade, secret) e aplica via API:



O provisionamento em massa combina um **template** único com os **dados** de cada cliente (do ERP) e aplica via **API sobre TLS** a centenas de equipamentos. Mudou o padrão?

Ajusta-se o template e reaplica — a configuração vira **código**, não trabalho manual repetido.

Figura 62.1

Na prática, muitos provedores usam sistemas prontos (o próprio **RADIUS/User Manager** do 12 já provisiona parâmetros de sessão; sistemas de gestão como SGP, IXC, etc. falam com o MikroTik), mas o **conceito** é o mesmo: **modelo** + **dados** → **API** → **equipamento**. E

idempotência (19) segue essencial — reaplicar o template não pode duplicar nem quebrar o que já existe.

62.5 Na caixa ou de fora?

Uma decisão recorrente: automatizar **dentro** do roteador (scripts + scheduler/netwatch, Capítulo 61) ou **de fora** (API)?

- **Na caixa:** bom para lógica **local** e autônoma — failover, autolimpeza, verificação de saúde. Não depende de conectividade com um central; roda mesmo isolado;
- **De fora (API):** bom para o que precisa de **visão de conjunto** e **integração** — provisionar cliente novo, aplicar mudança em massa, puxar dados para o monitoramento (Volume 13). O sistema central é a fonte da verdade.

O provedor maduro usa **os dois**: scripts locais para autonomia, API/central para orquestração e integração com a gestão.

! O conceito mais importante do capítulo

Escala se resolve **de fora**: um sistema central fala com os roteadores por **API** (REST no v7 — JSON sobre HTTPS, a mais fácil de integrar) e aplica **template + dados** a centenas de equipamentos. Duas coisas não se negociam. **Segurança**: TLS, usuário dedicado de privilégio mínimo, **allowed-address**, zero senha hardcoded — porque uma credencial vazada abre o parque inteiro. **Idempotência**: reaplicar o modelo não pode duplicar nem quebrar. Configuração vira **código**; a operação, um sistema — não um humano repetindo cliques.

62.6 Laboratório

i Ambiente

Um CHR (Capítulo 5) como o roteador a provisionar (**lab-rt**) e uma máquina de “gestão” (seu host, com curl/Python) na mesma rede de gerência 10.99.0.0/24. Gere um certificado no lab-rt para o www-ssl. O foco é exercitar a REST API com segurança — ler, criar e alterar configuração de fora.

62.6.1 Comandos passo a passo

Passo 1 — certificado e www-ssl. No lab-rt:

```
/certificate add name=cert-api common-name=lab-rt
/certificate sign cert-api
/ip service set www-ssl certificate=cert-api disabled=no address=10.99.0.0/24
/ip service set www disabled=yes
```

Passo 2 — usuário dedicado restrito.

```
/user add name=api-prov password="Api-Forte-2026" group=write address=10.99.0.10
```

Passo 3 — leia pela REST API. Da máquina de gestão:

```
curl -sk -u api-prov:Api-Forte-2026 https://10.99.0.1/rest/system/resource
curl -sk -u api-prov:Api-Forte-2026 https://10.99.0.1/rest/ip/address
```

Você recebe **JSON** com os dados — configuração legível por qualquer linguagem.

Passo 4 — crie e altere. Adicione um endereço e depois desabilite-o:

```
curl -sk -u api-prov:Api-Forte-2026 -X PUT https://10.99.0.1/rest/ip/address \
-H "Content-Type: application/json" \
-d '{"address":"192.168.77.1/24","interface":"ether2"}'
```

Confirme no lab-rt (`/ip address print`) que apareceu — configuração aplicada **de fora**.

Passo 5 — idempotência via API. Antes de criar, **consulte** se já existe (GET com filtro) e só faça o PUT se não houver — o mesmo princípio do 19, agora do lado do orquestrador.

Passo 6 — trave a segurança. Tente acessar a API de um IP **fora** do `allowed-address/address` configurado e confirme que é **recusado**. Verifique que o HTTP puro (porta 80) está desligado.

62.6.2 Verificação

1. A REST API responde JSON sobre **HTTPS** (www-ssl), com HTTP puro desligado;
2. GET lê configuração; PUT/PATCH criam e alteram itens de fora;
3. O acesso é recusado de IP fora do permitido (usuário/serviço restrito);
4. A lógica idempotente evita duplicar ao reaplicar.

62.6.3 Desafios

1. Um provedor expôs a API binária (8728) na internet, sem TLS e sem **allowed-address**, e teve o parque comprometido. Aponte **três** erros e a configuração correta para cada um.
2. Você precisa provisionar 300 CPEs idênticos, mudando só IP de gerência, VLAN e identidade por equipamento. Descreva a arquitetura (template + dados + API) e onde a **idempotência** entra ao reaplicar.
3. Quando você escolheria **script local + scheduler** (Capítulo 61) em vez de **API central**, e vice-versa? Dê um exemplo de tarefa para cada e justifique pela dependência de conectividade.
4. Um script de provisionamento guarda a senha de admin em texto no repositório git da empresa. Explique o risco concreto e duas formas de eliminá-lo sem perder a automação.

Solução dos desafios

1. (a) **API na internet** → exponha só na rede de gerência com **address=/allowed-address** (e, idealmente, atrás de VPN, Volume 10). (b) **Sem TLS** (8728) → use **api-ssl** (8729) ou a REST sobre **www-ssl**; credenciais nunca em claro. (c) **Sem usuário dedicado/ privilégio mínimo** → crie um usuário só para API, grupo restrito, com **address** do sistema de gestão. Some a isso senha forte e monitoramento de acesso.
2. **Template + dados + API**: o ERP guarda, por CPE, os três valores variáveis (IP de gerência, VLAN, identidade); um orquestrador combina o **template** com esses dados e aplica via **REST/TLS** a cada equipamento. **Idempotência** entra ao reaplicar: cada passo consulta (GET) se o item já existe e só cria/ajusta o que difere — assim rodar o provisionamento de novo (correção, nova versão do template) **não** duplica endereços/VLANs nem derruba o que já estava certo.
3. **Local + scheduler**: para lógica **autônoma** que deve funcionar mesmo sem contato com o central — failover por netwatch, autolimpeza, verificação de saúde. Roda no equipamento isolado. **API central**: para o que precisa de **visão de conjunto/integração** — provisionar cliente novo, mudança em massa, coleta para monitoramento. Depende de conectividade com o central. Exemplo: failover é local (não pode depender da nuvem); cadastrar um cliente novo é central (nasce no ERP).
4. Senha de admin em git = qualquer um com acesso ao repositório (ou a um vazamento dele) tem **acesso total** ao parque, e o histórico do git **guarda** a senha mesmo depois de removida. Soluções: (a) **cofre de segredos** (variável de ambiente/secret manager) — o script lê a credencial em runtime, nunca a versiona; (b) **usuário de API dedicado** com privilégio mínimo e **allowed-address**, rotacionado periodicamente, de modo que mesmo um vazamento tenha alcance limitado e revogável. Nunca credencial em texto no código.

62.7 Resumo

- Automação **externa** tem três formas: **API binária** (alto volume, bibliotecas), **REST API v7** (JSON/HTTPS, fácil de integrar) e **SSH** (pontual);
- A **REST API** espelha os menus (`/ip address` → `/rest/ip/address`); método HTTP = ação (GET/PUT/PATCH/DELETE/POST);
- **Provisionamento em massa** = **template + dados (ERP)** → **API** → **equipamento**; configuração vira **código**, com **idempotência** essencial;
- **Segurança inegociável**: TLS, usuário dedicado de privilégio mínimo, `allowed-address`, zero senha hardcoded — credencial vazada abre o parque;
- **Na caixa vs. de fora**: scripts locais para autonomia (failover), API central para orquestração/integração — o provedor maduro usa os dois.

Fecha-se o Volume 12: da linguagem de script à automação reativa e ao provisionamento em escala. A rede se configura e se defende com código. Falta **enxergá-la** — logs, gráficos, tráfego em tempo real, diagnóstico com método. É o **Volume 13**, monitoramento e diagnóstico.

Bibliografia do capítulo

- Documentação oficial: *REST API*, *API* e *Services* do RouterOS (MikroTik, 2026a);
- Integração de sistemas e automação em escala: (Tanenbaum; Feamster; Wetherall, 2021);
- API e provisionamento no v7: (Discher, 2016);
- Integração com sistemas de gestão de provedor: (Burgess, 2011).

Referências

Obras e documentos citados ao longo do manual, em formato ABNT.

BURGESS, Dennis. **Learn RouterOS**. 2. ed. Raleigh: Lulu Press, 2011.

DISCHER, Stephen R. W. **RouterOS by Example**. 2. ed. College Station: ISP Services, Inc., 2016.

MIKROTIK. **RouterOS Documentation**. <https://help.mikrotik.com/docs/>, a2026.

MIKROTIK. **Cloud Hosted Router (CHR) Documentation**. <https://help.mikrotik.com/docs/display/ROS/Cloud+Hosted+Router,+CHR>, b2026.

MIKROTIK. **MikroTik Product Catalog**. <https://mikrotik.com/products>, c2026.

REKHTER, Yakov *et al.* **Address Allocation for Private Internets**. RFC 1918, Internet Engineering Task Force, 1996.

TANENBAUM, Andrew S.; FEAMSTER, Nick; WETHERALL, David J. **Redes de Computadores**. 6. ed. Porto Alegre: Bookman, 2021.